

Statistics for Data Science

229711

Parichart Pattarapanitchai

2026-10-06

Table of contents

Preface	3
Course Overview	4
About This Course	4
Learning Objectives	4
Software Requirements	4
Course Structure	6
Part I — Foundations (Chapters 1–4)	6
Part II — Data Preparation (Chapters 5–6)	6
Part III — Modeling and Machine Learning (Chapters 7–10)	7
Prerequisites	8
Chapter Previews	9
Chapter 1: Descriptive Statistics	9
Chapter 2: Data Distribution and Probability	9
Chapter 3: Hypothesis Testing	9
Chapter 4: Test of Independence	10
Chapter 5: Data Sampling	10
Chapter 6: Data Preprocessing	10
Chapter 7: Dimension Reduction	10
Chapter 8: Data Clustering	10
Chapter 9: Data Classification	11
Chapter 10: Regression Analysis	11
A Note on Learning Statistics	12
Chapter 1: Descriptive Statistics	13
Chapter Overview	14
Introduction	14
Measures of Central Tendency	15
Introduction	15
Theory	15
The Arithmetic Mean	15
The Median	16

The Mode	16
When to Use Which Measure?	16
Weighted Mean	16
Example: Measures of Central Tendency	17
R Example: Measures of Central Tendency	17
Exercises	19
Measures of Dispersion	21
Introduction	21
Theory	21
Range	21
Interquartile Range (IQR)	21
Variance	22
Standard Deviation	22
Coefficient of Variation (CV)	22
Example: Measures of Dispersion	23
R Example: Measures of Dispersion	23
Exercises	25
Measures of Shape: Skewness and Kurtosis	27
Introduction	27
Theory	27
Skewness	27
Kurtosis	28
Example: Measures of Shape	28
R Example: Measures of Shape	29
Exercises	31
Data Visualization for Descriptive Statistics	32
Introduction	32
Theory	32
Histograms and Density Plots	32
Boxplots	32
Bar Charts and Frequency Tables	32
QQ Plots (Quantile-Quantile Plots)	33
R Example: Data Visualization	33
Exercises	35
Multivariate Descriptive Statistics	36
Introduction	36
Theory	36
Covariance	36
Pearson Correlation Coefficient	36
Spearman Rank Correlation	37
The Covariance and Correlation Matrix	38

Example: Multivariate Descriptive Statistics	38
R Example: Multivariate Descriptive Statistics	39
Exercises	41
Chapter Lab Activity: Exploring the mtcars Dataset	42
Objectives	42
The Dataset	42
Lab Task 1: Full Descriptive Summary	43
Lab Task 2: Central Tendency and Spread by Group	44
Lab Task 3: Visualizing the Full Distribution	44
Lab Task 4: Correlation Analysis	47
Lab Discussion Questions	47
Chapter Summary	49
Summary	49
References	49
Chapter 2: Data Distribution and Probability	51
Chapter Overview	52
Introduction	52
Types of Data and Measurement Scales	53
Introduction	53
Theory	53
Nominal Scale	53
Ordinal Scale	53
Interval Scale	54
Ratio Scale	54
Summary Table	54
Continuous vs. Discrete Variables	55
Example: Identifying Measurement Scales	55
R Example: Identifying and Converting Variable Types	55
Exercises	58
Probability Fundamentals	60
Introduction	60
Theory	60
Sample Space and Events	60
Kolmogorov's Axioms	60
Derived Rules	61
Frequentist vs. Bayesian Interpretation	61
Example: Probability Rules	61
R Example: Probability Rules and Simulation	62
Exercises	64

Conditional Probability and Bayes' Theorem	66
Introduction	66
Theory	66
Conditional Probability	66
Bayes' Theorem	66
Sensitivity, Specificity, and Predictive Value	67
Example: Medical Diagnosis with Bayes' Theorem	67
R Example: Bayes' Theorem and the Base Rate Effect	68
Exercises	70
Discrete Probability Distributions	72
Introduction	72
Theory	72
The Binomial Distribution	72
The Poisson Distribution	73
Example: Binomial and Poisson Distributions	73
R Example: Binomial and Poisson Distributions	74
Exercises	77
Continuous Probability Distributions	78
Introduction	78
Theory	78
The Normal (Gaussian) Distribution	78
The Uniform Distribution	79
The Exponential Distribution	79
Example: Normal, Uniform, and Exponential	79
R Example: Continuous Distributions	80
Exercises	82
Sampling Distributions and the Central Limit Theorem	84
Introduction	84
Theory	84
Sampling Distribution of the Mean	84
The Central Limit Theorem (CLT)	85
The t -Distribution	85
Example: Demonstrating the CLT	85
R Example: Demonstrating the Central Limit Theorem	86
Exercises	88
Assessing Normality	90
Introduction	90
Theory	90
Graphical Methods	90
Formal Normality Tests	91
Transformations for Non-Normal Data	91

Example: Assessing Normality of Exam Scores	91
R Example: Testing Normality	92
Exercises	95
Chapter Lab Activity: Exploring Distributions with the airquality Dataset	97
Objectives	97
The Dataset	97
Lab Task 1: Scale Identification and Missing Data	98
Lab Task 2: Distribution Characterization	99
Lab Task 3: Monthly Distributions and Probability Questions	102
Lab Task 4: Normality Assessment and Transformations	103
Lab Discussion Questions	104
Chapter Summary	105
Summary	105
References	106
Chapter 3: Hypothesis Testing	108
Chapter Overview	109
Introduction	109
The Logic of Hypothesis Testing	110
Introduction	110
Theory	110
The Null and Alternative Hypotheses	110
The p-value	111
The Significance Level α	111
Type I and Type II Errors	111
The General Procedure	112
Quantiles and Critical Values	112
Confidence Intervals and Hypothesis Tests	114
Example: The Logic of Hypothesis Testing	114
R Example: Visualizing the p-value	115
Exercises	116
One-Sample Tests	118
Introduction	118
Theory	118
One-Sample z-Test	118
One-Sample t-Test	118
Confidence Interval Approach	119
Example: One-Sample t-Test	119
R Example: One-Sample Tests	120
Exercises	122

Two-Sample Tests	124
Introduction	124
Theory	124
Independent Samples t-Test (Equal Variances)	124
Welch's t-Test (Unequal Variances)	124
Testing for Equal Variances: Levene's Test	125
Example: Two-Sample t-Test	125
R Example: Two-Sample Tests	126
Exercises	128
 Paired Sample Test	 130
Introduction	130
Theory	130
The Paired t-Test	130
Example: Paired t-Test	131
R Example: Paired t-Test	132
Exercises	134
 One-Way ANOVA	 135
Introduction	135
Theory	135
The ANOVA Model	135
Partitioning Variance	135
Assumptions	136
Post-Hoc Tests	136
Example: One-Way ANOVA	137
R Example: One-Way ANOVA	137
Exercises	140
 Non-Parametric Alternatives	 142
Introduction	142
Theory	142
Wilcoxon Signed-Rank Test (One-Sample / Paired)	142
Mann-Whitney U Test (Two Independent Samples)	142
Kruskal-Wallis Test (Three or More Groups)	143
When to Choose Non-Parametric Tests	143
Example: Mann-Whitney U Test	144
R Example: Non-Parametric Tests	144
Exercises	146
 Effect Size and Statistical Power	 148
Introduction	148
Theory	148
Effect Size Measures	148
Statistical Power	149

Power Analysis for Sample Size Planning	149
Example: Effect Size and Power	150
R Example: Effect Size and Power Analysis	150
Exercises	153
Chapter Lab Activity: Exploring Hypothesis Testing with the ToothGrowth Dataset	155
Objectives	155
The Dataset	155
Lab Task 1: Descriptive Overview and Assumption Checks	156
Lab Task 2: Two-Sample Test — OJ vs. VC	157
Lab Task 3: One-Way ANOVA — Effect of Dose	158
Lab Task 4: Comprehensive Visualization	159
Lab Discussion Questions	163
Chapter Summary	164
Summary	164
References	165
Chapter 4: Test of Independence of Variables	167
Chapter Overview	168
Introduction	168
The Concept of Independence	169
Introduction	169
Theory	169
Independence for Categorical Variables	169
Independence for Continuous Variables	170
A Taxonomy of Association Measures	170
Example: Identifying the Correct Test	170
R Example: Exploring Variable Relationships	171
Exercises	173
Chi-Square Test of Independence	174
Introduction	174
Theory	174
The Test Statistic	174
Critical Value for the Chi-Square Test	174
Assumptions	175
Yates' Continuity Correction	175
Standardized Residuals	176
Example: Chi-Square Test	176
R Example: Chi-Square Test of Independence	177
Exercises	180

Fisher's Exact Test	181
Introduction	181
Theory	181
The Hypergeometric Distribution	181
Two-Sided Fisher's Test	181
When to Use Fisher's Exact Test	181
Example: Fisher's Exact Test	182
R Example: Fisher's Exact Test	182
Exercises	186
Cramér's V and Effect Size for Categorical Association	187
Introduction	187
Theory	187
Cramér's V	187
The Phi Coefficient	188
Goodman and Kruskal's Lambda	188
Example: Cramér's V	188
R Example: Cramér's V and Association Measures	188
Exercises	191
Correlation Tests	192
Introduction	192
Theory	192
Pearson Correlation Test	192
Spearman Rank Correlation Test	193
Interpreting Correlation	193
Example: Pearson and Spearman Correlation Tests	193
R Example: Pearson and Spearman Correlation Tests	194
Exercises	196
Point-Biserial and Phi Coefficients	198
Introduction	198
Theory	198
Point-Biserial Correlation (r_{pb})	198
Phi Coefficient (ϕ)	199
Biserial vs. Point-Biserial	199
Example: Point-Biserial Correlation	199
R Example: Point-Biserial and Phi Coefficients	199
Exercises	203
Partial Correlation	204
Introduction	204
Theory	204
First-Order Partial Correlation	204
Significance Testing	204

Partial vs. Semi-Partial Correlation	205
Spurious Correlation	205
Example: Partial Correlation	205
R Example: Partial Correlation	205
Exercises	209
Chapter Lab Activity: Exploring Independence with the titanic and mtcars	
Datasets	210
Objectives	210
The Datasets	210
Lab Task 1: Survival $\times$ Class — Chi-Square Test	211
Lab Task 2: Survival $\times$ Sex — Fisher’s Exact Test	212
Lab Task 3: Correlation Analysis — mtcars	213
Lab Task 4: Partial Correlation — Controlling for Weight	214
Lab Task 5: Comprehensive Visualization	215
Lab Discussion Questions	219
Chapter Summary	220
Summary	220
References	221
Chapter 5: Data Sampling Techniques	223
Chapter Overview	224
Introduction	224
Why Sampling Matters	225
Introduction	225
Theory	225
Key Terminology	225
Two Sources of Error	225
The Mean Squared Error Framework	226
When Is a Census Preferable?	226
Example: Sampling Error vs. Bias	227
R Example: Sampling Error vs. Bias	227
Exercises	230
Probability Sampling Methods	231
Introduction	231
Theory	231
Simple Random Sampling (SRS)	231
Systematic Sampling	232
Stratified Sampling	232
Cluster Sampling	232
Summary Comparison	233
Example: Stratified vs. Simple Random Sampling	233

R Example: Probability Sampling Methods	233
Exercises	238
Non-Probability Sampling Methods	239
Introduction	239
Theory	239
Convenience Sampling	239
Purposive (Judgmental) Sampling	239
Snowball Sampling	240
Quota Sampling	240
When Non-Probability Sampling Is Acceptable	240
Example: Comparing Sampling Methods in Practice	240
R Example: Simulating Non-Probability Sampling Bias	241
Exercises	243
Sample Size Determination	244
Introduction	244
Theory	244
Sample Size for Estimating a Mean	244
Sample Size for Estimating a Proportion	245
Sample Size for Hypothesis Testing	245
Common z-Values	245
Example: Sample Size Calculation	245
R Example: Sample Size Calculations	246
Exercises	249
Sampling Bias and Common Pitfalls	251
Introduction	251
Theory	251
Selection Bias	251
Non-Response Bias	251
Undercoverage	252
Measurement Bias	252
Publication Bias	252
Example: Survivorship Bias	252
R Example: Detecting Non-Response Bias	253
Exercises	255
Bootstrap Resampling	256
Introduction	256
Theory	256
The Bootstrap Principle	256
Bootstrap Confidence Intervals	257
When to Use the Bootstrap	257
Example: Bootstrapping the Median	257

R Example: Bootstrap Resampling	257
Exercises	260
Evaluating Sample Quality	262
Introduction	262
Theory	262
Comparing Sample and Population Distributions	262
Weighting for Representativeness	263
Key Diagnostics	263
Example: Goodness-of-Fit for Sample Representativeness	263
R Example: Evaluating Sample Quality	263
Exercises	267
Chapter Lab Activity: Exploring Sampling with rhanes-Style Data	268
Objectives	268
Simulated Population	268
Lab Task 1: Implement Four Sampling Methods	269
Lab Task 2: Sample Size Planning	271
Lab Task 3: Bootstrap Inference	272
Lab Task 4: Representativeness Check	273
Lab Discussion Questions	275
Chapter Summary	276
Summary	276
References	277
Chapter 6: Data Preprocessing	278
Chapter Overview	279
Introduction	279
Why Preprocessing Matters	280
Introduction	280
Theory	280
The Data Preprocessing Pipeline	280
Why Each Step Matters	281
The Leakage Problem	281
Example: The Cost of Skipping Preprocessing	281
R Example: Initial Data Inspection	282
Exercises	284
Handling Missing Data	285
Introduction	285
Theory	285
Types of Missingness	285
Strategies for Handling Missing Data	286

Diagnosing the Missingness Mechanism	287
Example: Missing Data in a Clinical Study	287
R Example: Missing Data Analysis and Imputation	287
Exercises	291
Outlier Detection and Treatment	293
Introduction	293
Theory	293
Univariate Methods	293
Multivariate Methods	294
Treatment Options	294
Example: Outlier Detection	295
R Example: Outlier Detection and Treatment	295
Exercises	299
Data Transformation	300
Introduction	300
Theory	300
Log Transformation	300
Square Root Transformation	300
Box-Cox Transformation	301
Yeo-Johnson Transformation	301
Normalization vs. Standardization	301
Example: Choosing a Transformation	301
R Example: Data Transformations	302
Exercises	305
Encoding Categorical Variables	306
Introduction	306
Theory	306
Dummy Coding (Treatment Coding)	306
One-Hot Encoding	306
Ordinal Encoding	307
Target Encoding (Mean Encoding)	307
Binary Encoding	307
Encoding Selection Guide	307
Example: Encoding Faculty Variable	308
R Example: Categorical Encoding	308
Exercises	311
Feature Scaling	313
Introduction	313
Theory	313
Min-Max Normalization	313
Z-Score Standardization	313

Robust Scaling	314
MaxAbs Scaling	314
When to Scale	314
Example: Effect of Scaling on KNN	314
R Example: Feature Scaling	315
Exercises	317
Data Integration and Reshaping	318
Introduction	318
Theory	318
Tidy Data Principles	318
Pivoting: Wide Long	318
Joining Tables	319
String and Date Handling	319
Example: Reshaping for Analysis	319
R Example: Data Integration and Reshaping	320
Exercises	323
Chapter Lab Activity: Full Preprocessing Pipeline with msleep	325
Objectives	325
The Dataset	325
Lab Task 1: Missing Data Handling	326
Lab Task 2: Outlier Detection	327
Lab Task 3: Transformation	328
Lab Task 4: Encoding and Scaling	329
Lab Task 5: Final Clean Dataset and Visualization	330
Lab Discussion Questions	333
Chapter Summary	334
Summary	334
References	335
Chapter 7: Data Dimension Reduction	337
Chapter Overview	338
Introduction	338
The Curse of Dimensionality	339
Introduction	339
Theory	339
Sparsity in High Dimensions	339
Distance Concentration	339
The Sample Size Requirement	340
Practical Consequences	340
Example: Distance Concentration	341
R Example: Visualizing the Curse of Dimensionality	341

Exercises	343
Principal Component Analysis (PCA)	345
Introduction	345
Theory	345
Mathematical Foundation	345
Proportion of Variance Explained	346
Key Assumptions and Properties	346
Loadings vs. Scores	346
Example: PCA on Student Exam Data	347
R Example: PCA	347
Exercises	352
Factor Analysis	353
Introduction	353
Theory	353
The Factor Model	353
PCA vs. Factor Analysis	353
Factor Rotation	354
Determining the Number of Factors	354
Example: Factor Analysis of a Psychological Scale	355
R Example: Factor Analysis	355
Exercises	359
Linear Discriminant Analysis (LDA)	360
Introduction	360
Theory	360
Fisher's Discriminant Criterion	360
Number of Discriminants	360
LDA Assumptions	361
LDA vs. PCA	361
Example: LDA for Species Separation	361
R Example: Linear Discriminant Analysis	361
Exercises	365
t-SNE	366
Introduction	366
Theory	366
The t-SNE Algorithm	366
The Perplexity Parameter	367
Critical Limitations of t-SNE	367
Example: t-SNE on MNIST-style Data	367
R Example: t-SNE	368
Exercises	371

Feature Selection Methods	372
Introduction	372
Theory	372
Filter Methods	372
Wrapper Methods	372
Embedded Methods	373
Comparison	373
Example: Feature Selection for Predicting MPG	373
R Example: Feature Selection	373
Exercises	377
Evaluating Dimension Reduction	378
Introduction	378
Theory	378
Scree Plot and the Elbow Rule	378
Kaiser's Criterion	378
Cumulative Variance Threshold	378
Reconstruction Error	379
Parallel Analysis	379
Cross-Validation for Dimension Selection	379
Example: Choosing k for PCA	380
R Example: Evaluating Dimension Reduction	380
Exercises	384
Chapter Lab Activity: Dimension Reduction Pipeline with decathlon2	385
Objectives	385
The Dataset	385
Lab Task 1: PCA — Structure of Athletic Performance	386
Lab Task 2: Factor Analysis — Latent Athletic Dimensions	387
Lab Task 3: t-SNE Visualization	389
Lab Task 4: Feature Selection for Predicting Total Score	390
Lab Task 5: Comprehensive Evaluation	391
Lab Discussion Questions	393
Chapter Summary	395
Summary	395
References	396
Chapter 8: Data Clustering	398
Chapter Overview	399
Introduction	399
Introduction to Clustering	400
Introduction	400

Theory	400
Types of Clustering Algorithms	400
The Similarity/Distance Problem	401
Key Challenges	401
Example: When Do Clusters Exist?	402
R Example: Exploring Cluster Structure	402
Exercises	404
K-Means Clustering	405
Introduction	405
Theory	405
The K-Means Objective	405
K-Means++ Initialization	406
Choosing K : The Elbow Method	406
Choosing K : The Silhouette Method	406
Limitations of K-Means	407
Example: K-Means on Customer Data	407
R Example: K-Means Clustering	407
Exercises	411
Hierarchical Clustering	412
Introduction	412
Theory	412
Agglomerative Clustering (Bottom-Up)	412
Linkage Methods	412
The Dendrogram	413
Divisive Clustering (Top-Down)	413
Example: Hierarchical Clustering of Countries	413
R Example: Hierarchical Clustering	414
Exercises	418
DBSCAN	419
Introduction	419
Theory	419
Core Concepts	419
The DBSCAN Algorithm	420
Advantages and Limitations	420
Choosing ε : The k -Distance Plot	420
Example: DBSCAN on Spatial Data	420
R Example: DBSCAN	421
Exercises	424
Cluster Validation	426
Introduction	426

Theory	426
Internal Validation Indices	426
External Validation Indices	427
Stability-Based Validation	427
Example: Comparing Validation Indices	427
R Example: Cluster Validation	428
Exercises	430
Gaussian Mixture Models	431
Introduction	431
Theory	431
The Mixture Model	431
The EM Algorithm	431
Covariance Structure	432
Model Selection: BIC	432
Example: GMM vs. K-Means	433
R Example: Gaussian Mixture Models	433
Exercises	436
Practical Clustering Workflow	438
Introduction	438
Theory	438
The End-to-End Clustering Workflow	438
Algorithm Selection Guide	439
Cluster Profiling	439
R Example: Complete Clustering Workflow	439
Exercises	444
Chapter Lab Activity: Customer Segmentation with wholesales Data	445
Objectives	445
The Dataset	445
Lab Task 1: Preprocessing	446
Lab Task 2: K-Means Segmentation	447
Lab Task 3: Hierarchical and GMM Comparison	448
Lab Task 4: Visualization	449
Lab Discussion Questions	452
Chapter Summary	453
Summary	453
References	454
Chapter 9: Data Classification	456
Chapter Overview	457
Introduction	457

Introduction to Classification	458
Introduction	458
Theory	458
Types of Classification Problems	458
The Decision Boundary	458
The Training-Test Split	459
The Bayes Error Rate	459
Example: Classification Problem Setup	459
R Example: Setting Up a Classification Problem	460
Exercises	462
Logistic Regression	463
Introduction	463
Theory	463
The Logistic Model	463
The Log-Odds (Logit) Interpretation	463
Parameter Estimation: Maximum Likelihood	464
Hypothesis Testing for Coefficients	464
Multinomial Logistic Regression	464
Regularized Logistic Regression	465
Example: Logistic Regression for Diabetes Prediction	465
R Example: Logistic Regression	465
Exercises	469
K-Nearest Neighbors	470
Introduction	470
Theory	470
The KNN Algorithm	470
Choosing K	470
The Bias-Variance Tradeoff in KNN	471
KNN Requirements	471
Weighted KNN	471
Example: KNN Decision Boundary	471
R Example: KNN Classification	472
Exercises	474
Decision Trees	475
Introduction	475
Theory	475
Tree Construction: Recursive Binary Splitting	475
Impurity Measures	475
The Information Gain (and Gini Reduction)	476
Worked Example: Computing a Split's Gini Reduction	476
Variable Importance in a Single Tree	477
Pruning	477

Advantages and Limitations	477
Example: Decision Tree for Diabetes Classification	478
R Example: Decision Trees	478
Exercises	482
Random Forest and Ensemble Methods	483
Introduction	483
Theory	483
Bagging (Bootstrap Aggregating)	483
Random Forest	484
Boosting	484
Bagging vs. Boosting	484
Example: Random Forest Feature Importance	485
R Example: Random Forest and Ensemble Methods	485
Exercises	488
Model Evaluation	490
Introduction	490
Theory	490
The Confusion Matrix	490
Derived Metrics	491
The Precision-Recall Tradeoff	491
ROC Curve and AUC	491
Metrics for Imbalanced Classes	492
Handling Imbalanced Classes	492
Example: Why Accuracy Alone is Insufficient	492
R Example: Comprehensive Model Evaluation	493
Exercises	496
Model Comparison and Selection	498
Introduction	498
Theory	498
K-Fold Cross-Validation	498
The Bias-Variance of CV Estimates	499
Statistical Comparison of Models	499
The Test Set Discipline	499
Model Selection Criteria	499
Example: Cross-Validated Model Comparison	500
R Example: Cross-Validation and Model Comparison	501
Exercises	505
Chapter Lab Activity: Medical Diagnosis Classification with Pima Data	507
Objectives	507
The Dataset	507
Lab Task 1: Exploratory Analysis	508

Lab Task 2: Train All Classifiers	509
Lab Task 3: Comprehensive Evaluation	510
Lab Task 4: ROC and Threshold Analysis	510
Lab Discussion Questions	513
Chapter Summary	515
Summary	515
References	516
Chapter 10: Regression Analysis	518
Chapter Overview	519
Introduction	519
Simple Linear Regression	520
Introduction	520
Theory	520
The Simple Linear Regression Model	520
Ordinary Least Squares (OLS) Estimation	520
Inference for Regression Coefficients	521
The Coefficient of Determination R^2	521
Prediction vs. Confidence Intervals	521
Example: Simple Linear Regression	522
R Example: Simple Linear Regression	522
Exercises	525
Multiple Linear Regression	526
Introduction	526
Theory	526
The Multiple Regression Model	526
OLS in Matrix Form	526
Interpretation of Coefficients	526
Adjusted R^2	527
The F-Test for Overall Significance	527
Multicollinearity	527
Classical Regression Assumptions (LINE)	528
Example: Multiple Regression	528
R Example: Multiple Linear Regression	529
Exercises	531
Regression Diagnostics	532
Introduction	532
Theory	532
Residual Analysis	532
Four Diagnostic Plots	532
Leverage and Influence	533

Formal Tests for Assumption Violations	533
Remedies for Assumption Violations	533
Example: Diagnosing a Regression Model	534
R Example: Regression Diagnostics	534
Exercises	537
Variable Selection	538
Introduction	538
Theory	538
Information Criteria	538
Stepwise Selection	538
Cross-Validation for Variable Selection	539
Example: Variable Selection for Housing Prices	539
R Example: Variable Selection	539
Exercises	542
Regularized Regression	543
Introduction	543
Theory	543
Ridge Regression (L2 Regularization)	543
Lasso Regression (L1 Regularization)	543
Choosing λ : Cross-Validation	544
Example: Lasso for Variable Selection	544
R Example: Regularized Regression	544
Exercises	548
Non-Linear Regression	549
Introduction	549
Theory	549
Polynomial Regression	549
Regression Splines	549
Generalized Additive Models (GAM)	550
Example: Non-Linear Effects	550
R Example: Non-Linear Regression	550
Exercises	554
Regression for Count and Categorical Outcomes	555
Introduction	555
Theory	555
The Generalized Linear Model Framework	555
Poisson Regression	556
Model Assessment for GLMs	556
Example: Poisson Regression	556
R Example: GLMs — Poisson and Extensions	557
Exercises	561

Chapter Lab Activity: Housing Price Regression with Boston Dataset	562
Objectives	562
The Dataset	562
Lab Task 1: Exploratory Regression Analysis	564
Lab Task 2: OLS and Diagnostics	565
Lab Task 3: Model Comparison	567
Lab Task 4: GAM Visualization	569
Lab Discussion Questions	570
Chapter Summary	572
Summary	572
Textbook Epilogue	574
References	574

Preface

This book is a hands-on introduction to statistics for students and practitioners in data science. It is designed to bridge the gap between mathematical theory and practical application in R — every concept is accompanied by worked examples, derivations, and reproducible code.

The course progresses from foundational descriptive tools (Chapter 1) through probability and inference (Chapters 2–4), to the full data-science pipeline: sampling, preprocessing, dimensionality reduction, clustering, classification, and regression (Chapters 5–10). By the final chapter, you will be equipped to build, evaluate, and interpret predictive models from raw data.

How to Use This Book

Each chapter follows a consistent four-part structure:

1. **Theory** — mathematical foundations with derivations and formulas
2. **Worked Examples** — step-by-step calculations by hand
3. **R Examples** — fully reproducible code with line-by-line explanations
4. **Exercises** — practice problems ranging from computational to conceptual

Read the theory, work through the examples by hand, then reproduce and extend the R code. The exercises at the end of each section are designed to reinforce understanding — do not skip them.

Course Overview

About This Course

Statistics for Data Science introduces the probabilistic and statistical foundations that underpin modern data analysis and machine learning. The course emphasizes both mathematical rigor and practical fluency in R, the leading language for statistical computing.

At its core, data science is applied statistics. Understanding *why* methods work — not just *how* to call a function — makes the difference between a practitioner who applies tools blindly and one who chooses the right tool, diagnoses failures, and communicates results with confidence. This course builds that understanding from the ground up.

Learning Objectives

i By the End of This Course, You Will Be Able To:

1. Describe, visualize, and interpret any dataset using appropriate descriptive statistics.
2. Apply probability theory and common distributions to model real-world phenomena.
3. Design and conduct hypothesis tests and correctly interpret p -values, confidence intervals, and effect sizes.
4. Test for independence and association in categorical and continuous data.
5. Design and evaluate sampling strategies to collect valid data.
6. Clean, transform, and encode messy data for downstream analysis.
7. Reduce high-dimensional data using PCA and related methods.
8. Apply and evaluate clustering algorithms to discover structure in unlabeled data.
9. Build and assess classification models to predict categorical outcomes.
10. Fit, diagnose, and interpret simple and multiple linear regression models.

Software Requirements

All examples in this book use **R** (version 4.3) and are written within the **Quarto** publishing framework. The following packages are used throughout:

Table 1: Core R packages used in this course. Install all at once with `install.packages(c("tidyverse", "moments", "psych", "knitr", "kableExtra", "GGally", "corrplot", "MASS", "survey", "recipes", "caret", "factoextra", "cluster", "e1071", "rpart", "car", "lmtest"))`.

Package	Purpose	Chapters
<code>tidyverse</code>	Data manipulation and visualization (<code>ggplot2</code> , <code>dplyr</code>)	All
<code>moments</code>	Skewness and kurtosis	1
<code>psych</code>	Comprehensive descriptive statistics	1
<code>knitr</code> / <code>kableExtra</code>	Table formatting	All
<code>GGally</code> / <code>corrplot</code>	Correlation visualization	1, 10
<code>MASS</code>	Statistical distributions and datasets	2, 3
<code>stats</code>	Hypothesis tests (base R)	3, 4
<code>survey</code>	Complex survey design	5
<code>recipes</code> / <code>caret</code>	Preprocessing and modeling pipelines	6, 9
<code>factoextra</code>	PCA and clustering visualization	7, 8
<code>cluster</code>	Clustering algorithms	8
<code>class</code> / <code>e1071</code> / <code>rpart</code>	Classification models	9
<code>car</code> / <code>lmtest</code>	Regression diagnostics	10

Course Structure

The ten chapters are organized into three parts that reflect the natural progression of a data science project.

Part I — Foundations (Chapters 1–4)

Before modeling data, you must understand it. Part I builds the statistical vocabulary and inferential framework that all subsequent chapters rely on.

Table 1: Part I — Statistical Foundations

Chapter	Title	Key Topics
1	Descriptive Statistics	Mean, median, variance, SD, IQR, skewness, kurtosis, correlation, visualization
2	Data Distribution & Probability	Probability rules, random variables, Binomial, Poisson, Normal, Central Limit Theorem
3	Hypothesis Testing	Null/alternative hypotheses, z -test, t -test, Type I/II errors, p -values, power, confidence intervals
4	Test of Independence	Chi-square test, Fisher's exact test, Cramér's V, ANOVA, post-hoc tests

Part II — Data Preparation (Chapters 5–6)

Real data is never clean. Part II covers how to collect it properly and prepare it for analysis.

Table 2: Part II — Data Collection and Preparation

Chapter	Title	Key Topics
5	Data Sampling	Simple random, stratified, cluster, systematic sampling; sample size determination; sampling bias
6	Data Preprocessing	Missing values, outlier detection, normalization, encoding, feature engineering, train/test split

Part III — Modeling and Machine Learning (Chapters 7–10)

With clean data in hand, Part III introduces the core modeling techniques used in applied data science.

Table 3: Part III — Modeling and Machine Learning

Chapter	Title	Key Topics
7	Dimension Reduction	PCA, scree plots, biplots, variance explained, factor rotation
8	Data Clustering	k-Means, hierarchical clustering, DBSCAN, silhouette scores, elbow method
9	Data Classification	k-NN, Naive Bayes, decision trees, confusion matrix, precision, recall, ROC/AUC
10	Regression Analysis	Simple and multiple linear regression, OLS, model diagnostics, regularization, interpretation

Prerequisites

This course assumes the following background knowledge. Students lacking prerequisites in any area should review the suggested resources before beginning.

! Required Background

Mathematics

- High school algebra (solving equations, working with fractions and exponents)
- Basic calculus (derivatives and integrals at the level of a first-year university course) — used in Chapters 2–3 for probability density functions
- Linear algebra fundamentals (vectors, matrices, matrix multiplication) — used in Chapters 7, 10

Computing

- R installed (download from <https://www.r-project.org>)
- RStudio or Positron IDE recommended
- Quarto installed for rendering this book (<https://quarto.org>)
- Familiarity with basic R syntax (`<-`, vectors, data frames, `library()`)

Statistics

- No prior statistics knowledge is assumed — this course starts from scratch.

Chapter Previews

Chapter 1: Descriptive Statistics

Every data analysis begins here. Before fitting any model, a data scientist must develop an intimate understanding of the data — its center, spread, shape, and relationships. This chapter covers all major descriptive tools: measures of central tendency (mean, median, mode), measures of dispersion (range, IQR, variance, standard deviation, CV), measures of shape (skewness and kurtosis), and a complete toolkit of descriptive visualizations (histograms, boxplots, QQ plots, scatterplot matrices).

A unifying theme: numerical summaries compress information and can mislead. Anscombe's Quartet demonstrates that four datasets with identical means, variances, and correlations can look completely different. **Always visualize.**

Chapter 2: Data Distribution and Probability

Statistical inference is built on probability. This chapter introduces the mathematical language of uncertainty: sample spaces, events, conditional probability, and Bayes' theorem. The core of the chapter is a tour of the most important probability distributions — Binomial, Poisson, Normal, t , χ^2 , and F — with attention to when each arises in practice. The chapter closes with the Central Limit Theorem, arguably the most important result in statistics: the key that unlocks inference from sample data to population parameters.

Chapter 3: Hypothesis Testing

The workhorse of inferential statistics. We formalize the logic of hypothesis testing — null and alternative hypotheses, test statistics, p -values, and critical regions — then apply it to one-sample and two-sample z -tests and t -tests. Special attention is paid to the concept of **statistical power**, Type I and Type II errors, and how to determine appropriate sample sizes. The chapter ends with a critical discussion of p -value misinterpretation, effect sizes, and confidence intervals as a more informative alternative.

Chapter 4: Test of Independence

When variables are categorical, standard tests of means do not apply. This chapter covers the χ^2 test of independence, Fisher’s exact test for small samples, and Cramér’s V for measuring association strength. We then extend hypothesis testing to compare means across more than two groups via **ANOVA**, including post-hoc multiple comparison procedures (Tukey’s HSD) and the non-parametric Kruskal-Wallis alternative.

Chapter 5: Data Sampling

How data is collected determines what conclusions can be drawn. This chapter covers the major probability sampling designs — simple random, stratified, cluster, and systematic — and the sources of bias that arise when sampling is not done carefully. We examine how to calculate required sample sizes for a desired margin of error and power, and introduce complex survey design using the `survey` package in R.

Chapter 6: Data Preprocessing

Raw data is rarely analysis-ready. This chapter is a practical guide to the full preprocessing pipeline: diagnosing and imputing missing values, detecting and handling outliers, scaling and normalizing features, encoding categorical variables, engineering new features, and constructing reproducible train/test splits. We use the `recipes` package from the `tidymodels` ecosystem, which provides a principled, auditable workflow for all preprocessing steps.

Chapter 7: Dimension Reduction

High-dimensional data is computationally expensive, statistically challenging, and difficult to visualize. **Principal Component Analysis (PCA)** addresses all three problems by finding a lower-dimensional representation that retains as much variance as possible. This chapter derives PCA from the eigendecomposition of the covariance matrix, then demonstrates interpretation via scree plots, biplots, and loadings. We discuss how many components to retain, when PCA is appropriate, and introduce factor analysis as a related approach.

Chapter 8: Data Clustering

Clustering is **unsupervised learning**: finding structure in unlabeled data. We cover the three most important paradigms — **k-Means** (partition-based), **hierarchical clustering** (agglomerative and divisive), and **DBSCAN** (density-based) — along with methods for

evaluating cluster quality (silhouette scores, elbow method, dendrogram inspection). A key theme: clustering algorithms always find clusters, even in random data. Rigorous evaluation is essential.

Chapter 9: Data Classification

Supervised learning for categorical outcomes. This chapter introduces the classification pipeline — feature engineering, training, cross-validation, and evaluation — and applies it to three foundational classifiers: k -Nearest Neighbors (k-NN), Naive Bayes, and decision trees. Model performance is evaluated using the full toolkit: confusion matrix, accuracy, precision, recall, F1 score, and the ROC/AUC curve. The chapter emphasizes the **bias-variance tradeoff** and the danger of evaluating a model on the data it was trained on.

Chapter 10: Regression Analysis

The capstone of the course. We develop **linear regression** in full — from simple regression with one predictor to multiple regression with many. We derive the Ordinary Least Squares (OLS) estimator, prove its optimality under the Gauss-Markov assumptions, and build a complete diagnostic toolkit: residual plots, leverage and influence measures, tests for heteroskedasticity and autocorrelation. The chapter closes with regularization (Ridge and Lasso), connecting regression to modern machine learning and the challenge of prediction in high dimensions.

A Note on Learning Statistics

Statistics has a reputation for being abstract, dry, and disconnected from practice. This book takes the opposite view. Every formula in this course exists to solve a real problem — and understanding the problem makes the formula memorable.

When you encounter a formula, ask: *What is this measuring? Why is it defined this way? What goes wrong if I use something simpler?* When you encounter a concept, ask: *Where would I see this in real data? How would I explain it to someone who has never seen a statistics textbook?*

The R code in each chapter is not decoration. It is the primary vehicle for building intuition. Run every block, change the parameters, break things, and observe what happens. The most productive way to learn statistics in 2026 is to spend half your time computing and the other half thinking about what the numbers mean.

Getting Help

- **Course material:** All .qmd source files for this book are available on the accompanying GitHub repository.
- **R help:** Use `?function_name` in the R console for documentation, or search <https://www.rdocumentation.org>.
- **Quarto:** Official documentation at <https://quarto.org/docs>.
- **Community:** [Stack Overflow](#) (tag: `r`), [Posit Community](#), and [Cross Validated](#) are excellent resources for R and statistics questions.

Begin with Chapter 1: Descriptive Statistics.

Chapter 1: Descriptive Statistics

```
# Load all libraries needed for this chapter
library(tidyverse)
library(moments)      # skewness and kurtosis
library(psych)        # describe()
library(knitr)         # tables
library(kableExtra)   # styled tables
library(GGally)        # ggpairs for multivariate plots
library(corrplot)     # correlation matrix visualization
```

Chapter Overview

Introduction

Descriptive statistics is the branch of statistics concerned with **summarizing and communicating the key features of a dataset** without making inferences beyond the data at hand. Before fitting models, testing hypotheses, or making predictions, a data scientist must first develop an intimate understanding of their data — its center, its spread, its shape, and its relationships.

This chapter covers:

- **Measures of Central Tendency** — where the data is centered
- **Measures of Dispersion** — how spread out the data is
- **Measures of Shape** — skewness and kurtosis
- **Data Visualization** — graphical summaries
- **Multivariate Descriptive Statistics** — describing relationships between variables

Learning Objectives

By the end of this chapter, you will be able to:

1. Compute and interpret all major descriptive statistics by hand and in R.
2. Choose the appropriate measure of center and spread for different data types.
3. Diagnose distributional shape using skewness and kurtosis.
4. Produce and interpret publication-quality descriptive plots in R.
5. Describe bivariate and multivariate relationships using correlation and summary tables.

Measures of Central Tendency

Introduction

The first question we ask about any variable is: *Where is the data centered?* Measures of central tendency answer this by identifying a single representative value around which the observations cluster. The three most widely used measures are the **mean**, the **median**, and the **mode** — and the choice among them is never arbitrary. It depends on the scale of measurement, the shape of the distribution, and the presence of outliers.

Theory

The Arithmetic Mean

The **arithmetic mean** (commonly called “the average”) is the sum of all observed values divided by the number of observations. For a sample of n values $x_1, x_2, \dots, x_n$:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

For a population of size N , we use the symbol μ :

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

The mean minimizes the sum of squared deviations: $\sum (x_i - \bar{x})^2$ is smaller for $\bar{x}$ than for any other value. This makes it the **least squares** center of the data. However, the mean is sensitive to extreme values (outliers), because every observation contributes equally to the sum.

The Median

The **median** is the middle value when data are sorted in ascending order. For n observations:

$$\text{Median} = \begin{cases} x_{(n+1)/2} & \text{if } n \text{ is odd} \\ \frac{x_{(n/2)} + x_{(n/2+1)}}{2} & \text{if } n \text{ is even} \end{cases}$$

The median is **robust to outliers** — it is insensitive to the exact values of extreme observations. This makes it the preferred measure of center for skewed distributions such as income, house prices, and reaction times.

The Mode

The **mode** is the value (or values) that appear most frequently in the data. It is the only measure of central tendency applicable to **nominal (categorical) data**. A distribution may be unimodal (one mode), bimodal (two modes), or multimodal.

When to Use Which Measure?

Table 1: Choosing the appropriate measure of central tendency

Situation	Recommended Measure
Symmetric distribution, no outliers	Mean
Skewed distribution or outliers present	Median
Categorical or nominal data	Mode
Reporting income, house prices, reaction times	Median
Reporting height, temperature, test scores (approx. symmetric)	Mean

Weighted Mean

When observations carry different importance (weights w_i), the **weighted mean** accounts for this:

$$\bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

This is commonly used in survey research (sampling weights), grade calculation, and portfolio analysis.

Example: Measures of Central Tendency

Example 1.1. A researcher collects the following annual salaries (in thousands of USD) from 9 employees in a small firm:

32, 35, 38, 40, 42, 45, 48, 52, 180

Step 1 — Mean:

$$\bar{x} = \frac{32 + 35 + 38 + 40 + 42 + 45 + 48 + 52 + 180}{9} = \frac{512}{9} \approx 56.9 \text{ thousand}$$

Step 2 — Median ($n = 9$, odd $\rightarrow$ middle value is the 5th):

Median = 42 thousand

Step 3 — Mode: No value repeats, so this dataset has **no mode**.

Interpretation: The mean (\$56.9K) is substantially higher than the median (\$42K) because of the outlier salary of \$180K. The median better represents the “typical” employee salary in this firm. This illustrates why the median is preferred when data is right-skewed or contains outliers.

R Example: Measures of Central Tendency

```
# --- Define the salary data ---
salary <- c(32, 35, 38, 40, 42, 45, 48, 52, 180)

# --- Compute the three measures ---
mean_val <- mean(salary)
median_val <- median(salary)

# R has no built-in mode() for continuous data; we write a helper:
get_mode <- function(x) {
  ux <- unique(x)
  ux[which.max(tabulate(match(x, ux)))]
}
mode_val <- get_mode(salary)
```

```
# --- Display results ---
cat("Mean:  ", round(mean_val, 2), "thousand USD\n")
```

Mean: 56.89 thousand USD

```
cat("Median:", median_val, "thousand USD\n")
```

Median: 42 thousand USD

```
cat("Mode:  ", mode_val, "(appears once - effectively no mode)\n")
```

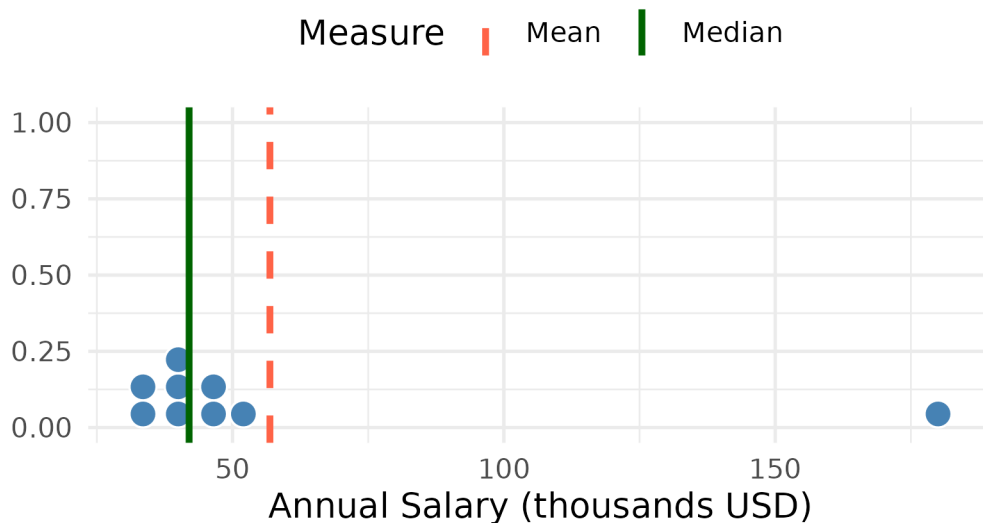
Mode: 32 (appears once - effectively no mode)

```
# --- Visualize: dotplot showing mean vs median ---
salary_df <- data.frame(salary = salary)
```

```
ggplot(salary_df, aes(x = salary)) +
  geom_dotplot(fill = "steelblue", color = "white", binwidth = 5) +
  geom_vline(aes(xintercept = mean_val, color = "Mean"), linewidth = 1.2, linetype = "solid") +
  geom_vline(aes(xintercept = median_val, color = "Median"), linewidth = 1.2, linetype = "solid") +
  scale_color_manual(values = c("Mean" = "tomato", "Median" = "darkgreen")) +
  labs(
    title = "Employee Salaries: Mean vs. Median",
    subtitle = "The outlier ($180K) pulls the mean far above the median",
    x = "Annual Salary (thousands USD)",
    y = "",
    color = "Measure"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")
```

Employee Salaries: Mean vs. Median

The outlier (\$180K) pulls the mean far above the median



Code explanation:

- `mean()` and `median()` are base R functions.
- We define `get_mode()` manually because R's built-in `mode()` returns the storage type, not the statistical mode.
- `geom_vline()` adds vertical reference lines, making the gap between mean and median visually apparent.
- The `scale_color_manual()` call controls line colors via a named vector.

Try modifying: Replace 180 with 55 (removing the outlier) and observe how the mean and median converge.

Exercises

💡 Exercise 1.1

The following values represent the number of publications per faculty member in a department:

3, 5, 7, 2, 8, 5, 12, 5, 6, 4

- Compute the mean, median, and mode by hand.
- Which measure best represents the “typical” faculty member? Justify your answer.

(c) Verify your answers in R.

 Exercise 1.2

A student's course grades are: Midterm (weight = 30%) = 78, Final (weight = 50%) = 85, Project (weight = 20%) = 91.

Compute the weighted mean grade. How does it compare to the simple arithmetic mean?

Measures of Dispersion

Introduction

Knowing where data is centered is necessary but not sufficient. Two datasets can have identical means yet look completely different — one tightly clustered, the other wildly spread. **Measures of dispersion** quantify the variability or spread of data around the center. This is critical in data science: high variability in model predictions indicates instability; high variability in features may require normalization.

Theory

Range

The simplest measure of spread: the difference between the maximum and minimum values.

$$\text{Range} = x_{\max} - x_{\min}$$

The range uses only two data points and is highly sensitive to outliers. It is most useful as a quick preliminary check.

Interquartile Range (IQR)

The **IQR** is the range of the middle 50% of the data — the distance between the 75th percentile (Q_3) and the 25th percentile (Q_1):

$$\text{IQR} = Q_3 - Q_1$$

The IQR is **robust to outliers** because it ignores the tails. It is the basis for the boxplot and for the standard outlier detection rule: an observation is a potential outlier if it falls below $Q_1 - 1.5 \times \text{IQR}$ or above $Q_3 + 1.5 \times \text{IQR}$.

Variance

The **variance** measures the average squared deviation from the mean. For a sample:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

We divide by $n - 1$ (not n) to obtain an **unbiased estimator** of the population variance σ^2 . This correction is known as Bessel's correction. Squaring the deviations ensures all contributions are positive and penalizes large deviations more heavily.

Standard Deviation

The **standard deviation** is the square root of the variance, returning units to the original scale of the data:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

It is the most widely used measure of dispersion and is interpretable as the “typical” distance of observations from the mean.

Coefficient of Variation (CV)

The **CV** expresses the standard deviation as a percentage of the mean, enabling comparisons of variability across variables with different units or scales:

$$CV = \frac{s}{\bar{x}} \times 100\%$$

For example, comparing the variability of blood pressure (measured in mmHg) to heart rate (beats per minute) using CV makes the comparison unit-free.

Example: Measures of Dispersion

Example 1.2. Two machine production lines produce bolts with the following lengths (mm):

- **Line A:** 50.1, 49.8, 50.3, 50.0, 49.9, 50.2, 50.1, 49.7
- **Line B:** 48.5, 51.2, 49.0, 52.1, 48.8, 51.5, 49.3, 51.6

Both lines have $\bar{x}_A \approx \bar{x}_B \approx 50$ mm. But the quality implications differ dramatically.

Line A:

$$s_A^2 = \frac{(0.1^2 + 0.2^2 + 0.3^2 + 0^2 + 0.1^2 + 0.2^2 + 0.1^2 + 0.3^2)}{7} \approx 0.041, \quad s_A \approx 0.203 \text{ mm}$$

Line B:

$$s_B \approx 1.48 \text{ mm}$$

Despite identical means, Line B's standard deviation is about 7 times larger. Line A produces consistent, high-quality bolts; Line B has unacceptable variability that would cause assembly failures.

R Example: Measures of Dispersion

```
# --- Production line data ---
line_A <- c(50.1, 49.8, 50.3, 50.0, 49.9, 50.2, 50.1, 49.7)
line_B <- c(48.5, 51.2, 49.0, 52.1, 48.8, 51.5, 49.3, 51.6)

# --- Compute dispersion measures ---
dispersion_table <- data.frame(
  Measure = c("Mean", "Range", "IQR", "Variance", "Std. Deviation", "CV (%)"),
  Line_A = c(
    mean(line_A),
    diff(range(line_A)),
    IQR(line_A),
    var(line_A),
    sd(line_A),
    sd(line_A) / mean(line_A) * 100
  ),
  Line_B = c(
    mean(line_B),
    diff(range(line_B)),
    IQR(line_B),
```



```

    var(line_B),
    sd(line_B),
    sd(line_B) / mean(line_B) * 100
  )
)

dispersion_table[, 2:3] <- round(dispersion_table[, 2:3], 4)

kable(dispersion_table,
      caption = "Dispersion Measures: Line A vs. Line B",
      col.names = c("Measure", "Line A", "Line B")) |>
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Table 1: Dispersion Measures: Line A vs. Line B

Measure	Line A	Line B
Mean	50.0125	50.2500
Range	0.6000	3.6000
IQR	0.2500	2.5750
Variance	0.0412	2.1914
Std. Deviation	0.2031	1.4803
CV (%)	0.4061	2.9460

```

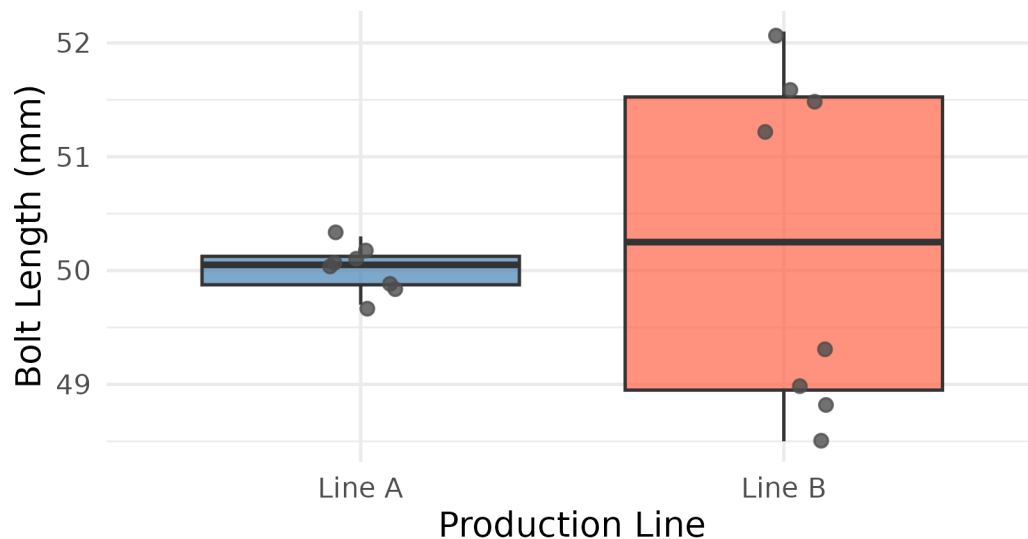
# --- Side-by-side boxplots ---
bolt_data <- data.frame(
  length = c(line_A, line_B),
  line    = rep(c("Line A", "Line B"), each = 8)
)

ggplot(bolt_data, aes(x = line, y = length, fill = line)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21, outlier.size = 3) +
  geom_jitter(width = 0.1, size = 2, alpha = 0.8, color = "gray30") +
  scale_fill_manual(values = c("Line A" = "steelblue", "Line B" = "tomato")) +
  labs(
    title     = "Bolt Lengths by Production Line",
    subtitle  = "Both lines centered at ~50 mm, but Line B has far greater spread",
    x         = "Production Line",
    y         = "Bolt Length (mm)"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")

```

Bolt Lengths by Production Line

Both lines centered at ~50 mm, but Line B has far greater



Code explanation:

- `var()`, `sd()`, `IQR()`, and `range()` are all base R functions.
- `diff(range(x))` computes the range efficiently: `range()` returns min and max, `diff()` subtracts them.
- `kable()` + `kable_styling()` produce a clean, styled comparison table suitable for reports.
- `geom_jitter()` overlays raw data points on the boxplot, revealing the actual observations behind the summary.

Exercises

💡 Exercise 1.3

Using the salary data from Exercise 1.1 (3, 5, 7, 2, 8, 5, 12, 5, 6, 4 publications):

- Compute the variance and standard deviation by hand (show all steps).
- Identify any outliers using the IQR rule ($Q_1 - 1.5 \times \text{IQR}$, $Q_3 + 1.5 \times \text{IQR}$).
- How does removing any outlier change the standard deviation?

Exercise 1.4

Two university courses each have a class mean of 75% on the midterm. Course A has $s = 5$, Course B has $s = 18$.

- (a) Compute the CV for each course.
- (b) What does this tell the instructor about each class? Which class is more homogeneous?
- (c) Simulate this scenario in R and produce a density plot comparing both distributions.

Measures of Shape: Skewness and Kurtosis

Introduction

The mean and standard deviation describe where data is centered and how spread out it is, but they say nothing about the **shape** of the distribution. Two datasets can share the same mean and variance yet differ in symmetry and tail heaviness. Measures of shape — **skewness** and **kurtosis** — capture these features and are essential for deciding which statistical methods are appropriate (many assume normality) and for understanding the nature of extreme observations.

Theory

Skewness

Skewness measures the degree and direction of asymmetry in a distribution. The sample skewness is:

$$\text{Skewness} = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3$$

Table 1: Interpreting skewness

Skewness Value	Shape	Relationship
≈ 0	Symmetric	Mean $\approx$ Median $\approx$ Mode
> 0	Right-skewed (positive)	Mean $>$ Median $>$ Mode
< 0	Left-skewed (negative)	Mean $<$ Median $<$ Mode

Right skew (positive skew) is common in income, wealth, reaction times, and biological measurements where values cannot go below zero but can take large positive values. **Left skew** occurs in data like scores on an easy exam (most cluster near the maximum).

As a rule of thumb: $|\text{skewness}| < 0.5$ is approximately symmetric; 0.5–1.0 is moderately skewed; > 1.0 is highly skewed.

Kurtosis

Kurtosis measures the “tailedness” of a distribution — the concentration of probability in the tails relative to a normal distribution. The **excess kurtosis** (relative to the normal distribution, which has kurtosis = 3) is:

$$\text{Excess Kurtosis} = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}$$

Table 2: Interpreting excess kurtosis

Excess Kurtosis	Distribution Type	Description
= 0	Mesokurtic	Normal-like tails
> 0	Leptokurtic	Heavy tails, peaked center (e.g., Student’s t)
< 0	Platykurtic	Light tails, flatter shape (e.g., Uniform)

High (positive) kurtosis warns of **tail risk** — a higher probability of extreme values than the normal distribution predicts. This is critical in financial risk management and anomaly detection.

Example: Measures of Shape

Example 1.3. A researcher measures response times (milliseconds) for a cognitive task across 200 participants. Summary statistics:

- $\bar{x} = 485$ ms, Median = 440 ms, $s = 210$ ms
- Skewness = 1.82, Excess Kurtosis = 4.15

Interpretation:

- The mean (485 ms) exceeds the median (440 ms), consistent with positive skewness.
- Skewness = 1.82 indicates strong right skew — a small number of participants with very long response times pulls the mean upward.
- Excess kurtosis = 4.15 (leptokurtic) suggests heavier tails than a normal distribution — extreme response times are more common than normality would predict.

- **Implication:** Standard parametric tests assuming normality may be inappropriate. A log transformation or non-parametric alternatives should be considered.

R Example: Measures of Shape

```
# --- Simulate three distributions with same mean and variance ---
set.seed(42)
n <- 500
sym <- rnorm(n, mean = 50, sd = 10) # symmetric
right <- c(rnorm(400, 40, 5), rnorm(100, 85, 10)) # right-skewed
left <- c(rnorm(400, 60, 5), rnorm(100, 15, 10)) # left-skewed

# --- Compute skewness and kurtosis using moments package ---
shape_summary <- data.frame(
  Distribution = c("Symmetric", "Right-Skewed", "Left-Skewed"),
  Mean = round(c(mean(sym), mean(right), mean(left)), 2),
  SD = round(c(sd(sym), sd(right), sd(left)), 2),
  Skewness = round(c(skewness(sym), skewness(right), skewness(left)), 3),
  Excess_Kurtosis = round(c(kurtosis(sym)-3, kurtosis(right)-3, kurtosis(left)-3), 3)
)

kable(shape_summary,
  caption = "Shape Measures Across Three Distributions",
  col.names = c("Distribution", "Mean", "SD", "Skewness", "Excess Kurtosis")) |>
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Table 3: Shape Measures Across Three Distributions

Distribution	Mean	SD	Skewness	Excess Kurtosis
Symmetric	49.70	9.72	0.021	0.016
Right-Skewed	48.98	19.69	1.551	1.063
Left-Skewed	50.74	19.13	-1.487	0.803

```
# --- Density plots for all three ---
shape_df <- data.frame(
  value = c(sym, right, left),
  dist = rep(c("Symmetric", "Right-Skewed", "Left-Skewed"), each = n)
)
shape_df$dist <- factor(shape_df$dist,
  levels = c("Left-Skewed", "Symmetric", "Right-Skewed"))
```

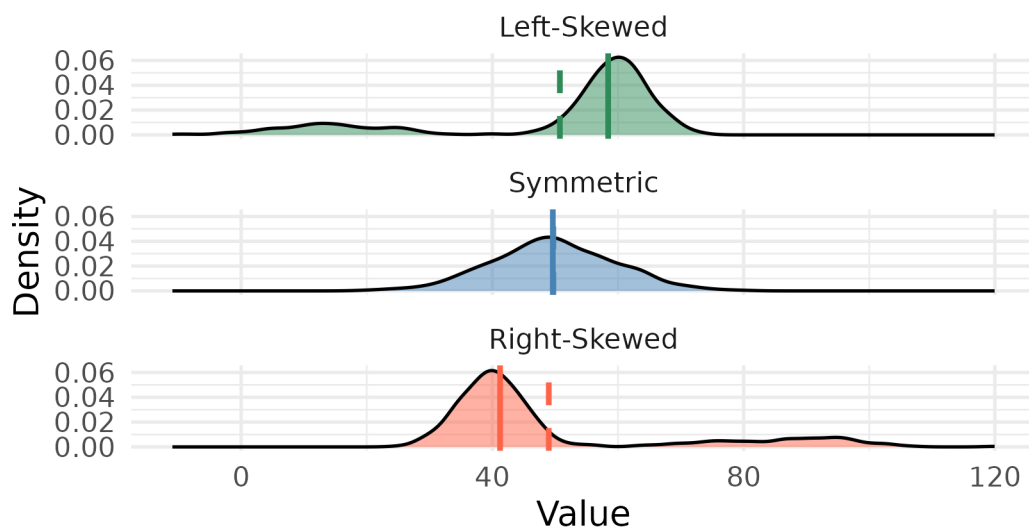
```

ggplot(shape_df, aes(x = value, fill = dist)) +
  geom_density(alpha = 0.5) +
  geom_vline(data = shape_df |>
    group_by(dist) |>
    summarise(m = mean(value), med = median(value)),
    aes(xintercept = m, color = dist), linetype = "dashed", linewidth = 1) +
  geom_vline(data = shape_df |>
    group_by(dist) |>
    summarise(m = mean(value), med = median(value)),
    aes(xintercept = med, color = dist), linetype = "solid", linewidth = 1) +
  scale_fill_manual(values = c("Symmetric" = "steelblue",
    "Right-Skewed" = "tomato",
    "Left-Skewed" = "seagreen")) +
  scale_color_manual(values = c("Symmetric" = "steelblue",
    "Right-Skewed" = "tomato",
    "Left-Skewed" = "seagreen")) +
  labs(
    title = "Distributional Shape: Skewness Comparison",
    subtitle = "Dashed = Mean, Solid = Median",
    x = "Value", y = "Density", fill = "Distribution"
  ) +
  facet_wrap(~dist, ncol = 1) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")

```

Distributional Shape: Skewness Comparison

Dashed = Mean, Solid = Median



Code explanation:

- `skewness()` and `kurtosis()` come from the `moments` package. Note that `kurtosis()` returns the **raw** kurtosis (normal = 3), so we subtract 3 for excess kurtosis.
- `facet_wrap(~dist, ncol = 1)` creates stacked panels — useful for comparing shapes visually.
- The `factor()` call controls the panel order for logical display (left → symmetric → right).

Exercises

Exercise 1.5

Given the following frequency distribution of exam scores, determine:

Score Range	Frequency
40–50	2
50–60	8
60–70	25
70–80	35
80–90	20
90–100	10

- Based on the frequency table alone, predict the direction of skewness. Explain your reasoning.
- Generate similar data in R and compute skewness and kurtosis to confirm.
- Plot a histogram with a superimposed density curve.

Data Visualization for Descriptive Statistics

Introduction

Numerical summaries are powerful, but they inevitably compress information. A single summary statistic can mask patterns that are immediately obvious in a plot. The famous **Anscombe's Quartet** (1973) demonstrates that four datasets with nearly identical means, variances, correlations, and regression lines can look completely different when plotted. Visualization is not optional — it is an indispensable component of responsible data analysis.

Theory

Histograms and Density Plots

A **histogram** divides the range of values into bins and plots the count (or proportion) of observations in each bin. The choice of bin width significantly affects interpretation: too few bins hides structure; too many bins shows noise as pattern. **Kernel density estimates (KDE)** provide a smooth continuous estimate of the distribution without imposing arbitrary bins.

Boxplots

A **boxplot** (box-and-whisker plot) summarizes a distribution using five numbers: minimum, Q1, median, Q3, and maximum (with outliers plotted separately). Boxplots excel at **comparing distributions across groups** and at identifying outliers. Their weakness is that they can hide bimodality or other complex shapes.

Bar Charts and Frequency Tables

For categorical data, **bar charts** display the frequency or proportion of each category. Unlike histograms, the bars do not touch (no ordering implied for nominal data). **Frequency tables** provide the raw counts and relative frequencies underlying any bar chart.

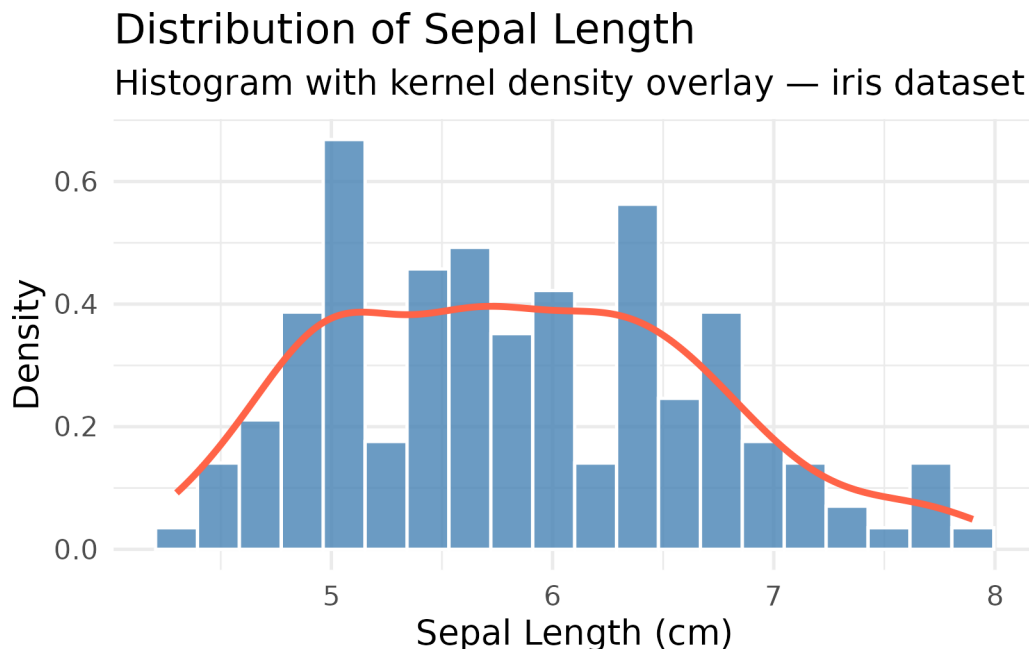
QQ Plots (Quantile-Quantile Plots)

A **QQ plot** compares the quantiles of observed data against the quantiles of a theoretical distribution (usually normal). If the data follow that distribution, points fall approximately on a straight diagonal line. Deviations from the line reveal departures from normality — S-curves indicate skewness, heavy tails appear as points curving away at the ends.

R Example: Data Visualization

```
# --- Use the built-in 'iris' dataset ---
data(iris)

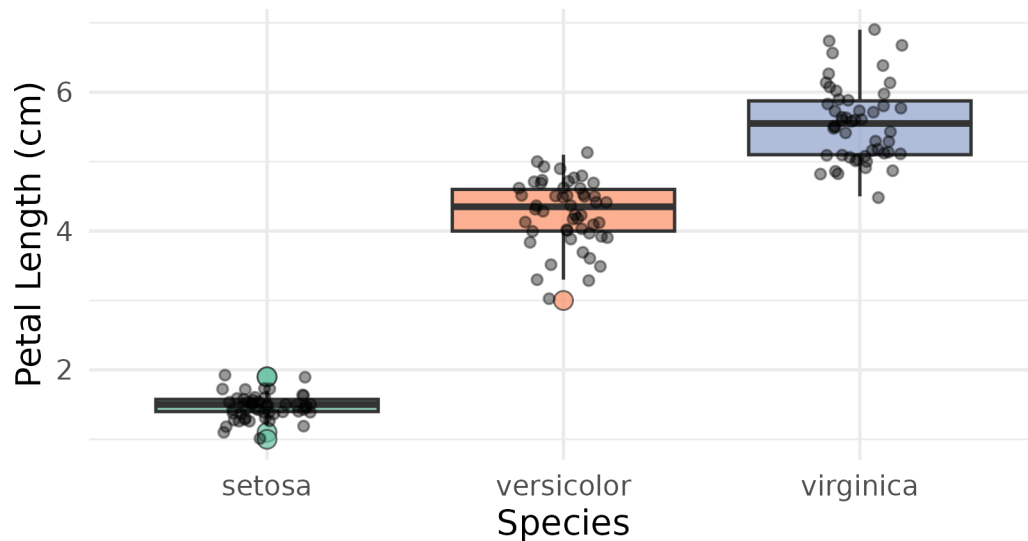
# 1. Histogram with density overlay for Sepal.Length
ggplot(iris, aes(x = Sepal.Length)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 20, fill = "steelblue", color = "white", alpha = 0.8) +
  geom_density(color = "tomato", linewidth = 1.2) +
  labs(title = "Distribution of Sepal Length",
       subtitle = "Histogram with kernel density overlay - iris dataset",
       x = "Sepal Length (cm)", y = "Density") +
  theme_minimal(base_size = 13)
```



```
# 2. Grouped boxplot by species
ggplot(iris, aes(x = Species, y = Petal.Length, fill = Species)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21, outlier.size = 3) +
  geom_jitter(width = 0.15, alpha = 0.4, size = 1.5) +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Petal Length by Species",
       subtitle = "Boxplot with individual observations - iris dataset",
       x = "Species", y = "Petal Length (cm)") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```

Petal Length by Species

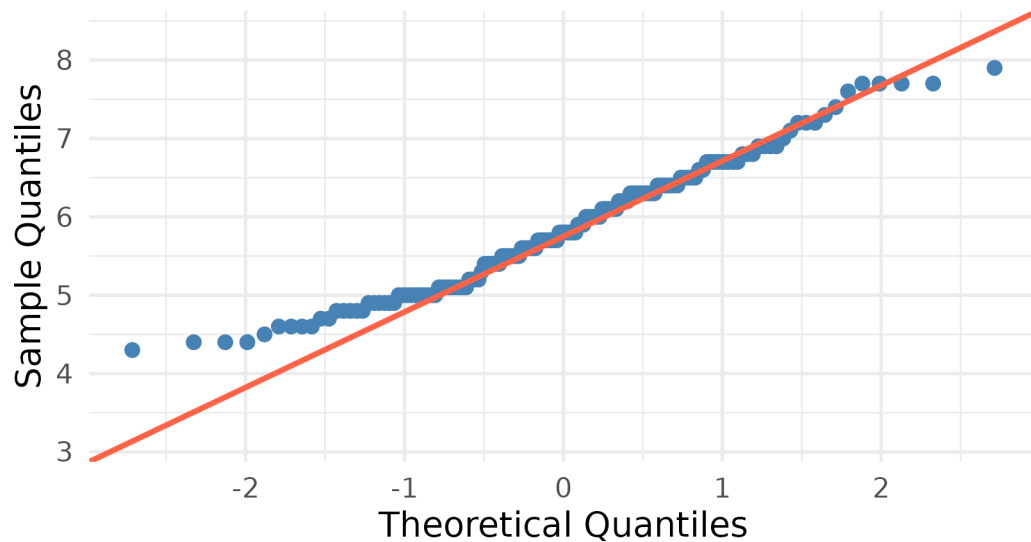
Boxplot with individual observations — iris dataset



```
# 3. QQ plot to assess normality
ggplot(iris, aes(sample = Sepal.Length)) +
  stat_qq(color = "steelblue", size = 2) +
  stat_qq_line(color = "tomato", linewidth = 1) +
  labs(title = "Normal QQ Plot - Sepal Length",
       subtitle = "Points near the line suggest approximate normality",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal(base_size = 13)
```

Normal QQ Plot — Sepal Length

Points near the line suggest approximate normality



Code explanation:

- `after_stat(density)` in the histogram aesthetic maps bar heights to density (not count), so the density curve overlays correctly.
- `stat_qq()` + `stat_qq_line()` produce a QQ plot entirely within `ggplot2` — no additional packages needed.
- `scale_fill_brewer(palette = "Set2")` uses a colorblind-friendly palette from ColorBrewer.

Exercises

💡 Exercise 1.6

Using the `mtcars` dataset in R:

- (a) Create a histogram of `mpg` (miles per gallon). Experiment with 5, 15, and 30 bins. Which bin count best reveals the distribution's shape?
- (b) Create side-by-side boxplots of `mpg` grouped by number of cylinders (`cyl`). What pattern do you observe?
- (c) Produce a QQ plot of `mpg` and assess whether it follows a normal distribution.

Multivariate Descriptive Statistics

Introduction

Real-world datasets rarely consist of a single variable. Data scientists routinely work with dozens or hundreds of features simultaneously. **Multivariate descriptive statistics** extend single-variable summaries to describe **relationships between variables** — whether two variables tend to move together (covariance, correlation), and how to summarize many variables at once.

Theory

Covariance

Covariance measures the direction of the linear relationship between two variables X and Y :

$$s_{XY} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

- $s_{XY} > 0$: X and Y tend to increase together (positive relationship).
- $s_{XY} < 0$: When X increases, Y tends to decrease (negative relationship).
- $s_{XY} = 0$: No linear relationship.

Covariance has a critical limitation: its magnitude depends on the scales of X and Y , making direct comparisons across variable pairs impossible.

Pearson Correlation Coefficient

The **Pearson correlation coefficient** r standardizes covariance to a dimensionless scale from -1 to $+1$:

$$r_{XY} = \frac{S_{xy}}{\sqrt{S_{xx}S_{yy}}} = \frac{s_{XY}}{s_X \cdot s_Y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

where the **sums of squares and cross-products** are

$$S_{xx} = \sum (x_i - \bar{x})^2, \quad S_{yy} = \sum (y_i - \bar{y})^2, \quad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}).$$

These connect to the covariance and standard deviations by $s_{XY} = S_{xy}/(n-1)$, $s_X = \sqrt{S_{xx}/(n-1)}$, and $s_Y = \sqrt{S_{yy}/(n-1)}$. The $1/(n-1)$ factors cancel, so all three forms give the same r — the square root in the last form is simply *hidden* inside s_X and s_Y .

Computational form. For hand calculation, the three sums can be obtained in a single pass through the data without first computing the means:

$$S_{xx} = \sum x_i^2 - \frac{(\sum x_i)^2}{n}, \quad S_{yy} = \sum y_i^2 - \frac{(\sum y_i)^2}{n}, \quad S_{xy} = \sum x_i y_i - \frac{(\sum x_i)(\sum y_i)}{n}.$$

Note $\sum x_i^2$ (square first, then sum) is **not** $(\sum x_i)^2$ (sum first, then square). This algebraic shortcut is convenient by hand but numerically unstable for large or highly correlated data — in software, always use `cor()` rather than coding this directly.

Table 1: Interpreting Pearson correlation

Magnitude of r	Strength of Linear Relationship
0.00 – 0.19	Very weak or none
0.20 – 0.39	Weak
0.40 – 0.59	Moderate
0.60 – 0.79	Strong
0.80 – 1.00	Very strong

Critical caveat: Pearson’s r measures only **linear** relationships. Two variables can be strongly related (e.g., quadratically) yet have $r \approx 0$. Always visualize with a scatterplot alongside computing r .

Spearman Rank Correlation

When data violates the assumptions of Pearson’s r (non-normality, outliers, or ordinal scale), **Spearman’s** ρ provides a robust alternative based on the ranks of the data rather than raw values:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

where d_i is the difference in ranks for observation i .

The Covariance and Correlation Matrix

For p variables, the **covariance matrix** is a $p \times p$ matrix where the diagonal contains variances and off-diagonal entries contain pairwise covariances. The **correlation matrix** $\mathbf{R}$ standardizes this to show all correlations. These matrices are fundamental to multivariate methods like PCA (Chapter 7) and linear regression (Chapter 10).

Example: Multivariate Descriptive Statistics

Example 1.4. For four students, compute the correlation between study hours (X) and exam score (Y):

Student	Hours (X)	Score (Y)
1	2	55
2	4	70
3	6	80
4	8	90

$$\bar{x} = 5, \bar{y} = 73.75$$

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = (-3)(-18.75) + (-1)(-3.75) + (1)(6.25) + (3)(16.25) = 56.25 + 3.75 + 6.25 + 48.75 = 115$$

Method 1 — deviation / covariance form:

$$s_X = \sqrt{\frac{20}{3}} \approx 2.582, \quad s_Y = \sqrt{\frac{668.75}{3}} \approx 14.93$$

$$r = \frac{s_{XY}}{s_X \cdot s_Y} = \frac{115/3}{2.582 \times 14.93} \approx \frac{38.33}{38.55} \approx 0.994$$

Method 2 — computational form (five running sums, $n = 4$):

$$\sum x_i = 20, \quad \sum y_i = 295, \quad \sum x_i^2 = 120, \quad \sum y_i^2 = 22425, \quad \sum x_i y_i = 1590$$

$$S_{xx} = 120 - \frac{20^2}{4} = 20, \quad S_{yy} = 22425 - \frac{295^2}{4} = 668.75, \quad S_{xy} = 1590 - \frac{20 \times 295}{4} = 115$$

$$r = \frac{S_{xy}}{\sqrt{S_{xx} S_{yy}}} = \frac{115}{\sqrt{20 \times 668.75}} = \frac{115}{115.65} \approx 0.994$$

Both routes give the same $r \approx 0.994$.

This near-perfect positive correlation confirms a strong linear relationship between study time and exam performance.

R Example: Multivariate Descriptive Statistics

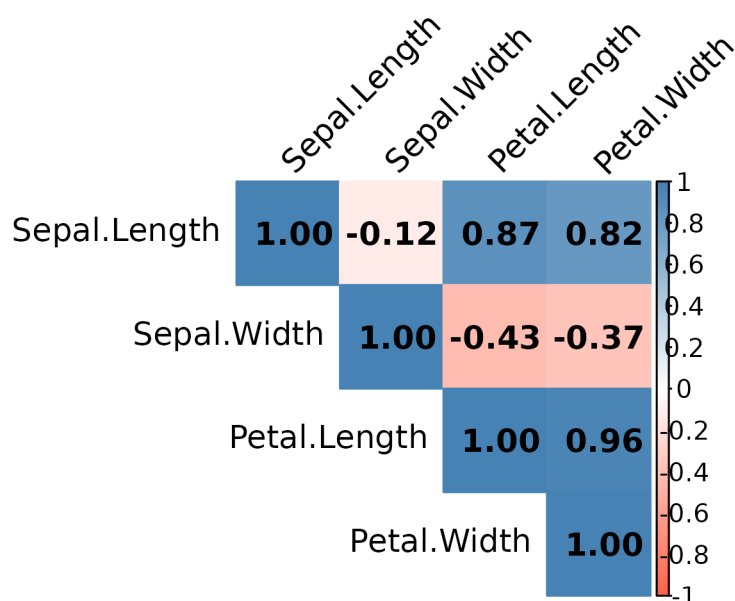
```
# --- Correlation matrix for iris numeric variables ---  
iris_num <- iris[, 1:4]
```

```
# Pearson correlation matrix  
cor_matrix <- cor(iris_num, method = "pearson")  
round(cor_matrix, 3)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Sepal.Length	1.000	-0.118	0.872	0.818
Sepal.Width	-0.118	1.000	-0.428	-0.366
Petal.Length	0.872	-0.428	1.000	0.963
Petal.Width	0.818	-0.366	0.963	1.000

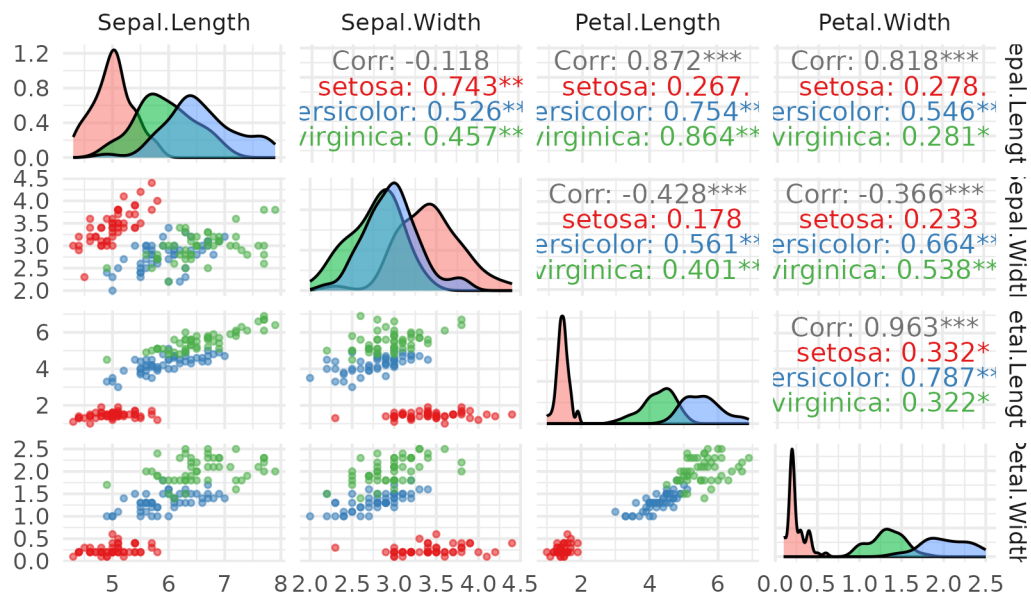
```
# --- Visualize the correlation matrix ---  
corrplot(cor_matrix,  
  method = "color",  
  type = "upper",  
  addCoef.col = "black",  
  tl.col = "black",  
  tl.srt = 45,  
  col = colorRampPalette(c("tomato", "white", "steelblue"))(200),  
  title = "Pearson Correlation Matrix - iris Dataset",  
  mar = c(0, 0, 2, 0))
```

Pearson Correlation Matrix — iris Dataset




```
# --- Scatterplot matrix (pairs plot) ---
ggpairs(iris,
  columns = 1:4,
  aes(color = Species, alpha = 0.6),
  upper = list(continuous = wrap("cor", size = 3.5)),
  lower = list(continuous = wrap("points", size = 0.8)),
  diag = list(continuous = wrap("densityDiag")) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "Scatterplot Matrix - iris Dataset") +
  theme_minimal(base_size = 11)
```

Scatterplot Matrix — iris Dataset



Code explanation:

- `cor(iris_num, method = "pearson")` computes the full correlation matrix. Change to "spearman" for rank-based correlation.
- `corrplot()` visualizes the matrix with colors encoding strength and direction; `addCoef.col` overlays the numeric values.
- `ggpairs()` from `GGally` creates a comprehensive pairs plot: scatterplots (lower), correlations by species (upper), and density distributions (diagonal) — all in one call.

Exercises

💡 Exercise 1.7

Using the `mtcars` dataset:

- (a) Compute the Pearson and Spearman correlation between `hp` (horsepower) and `mpg` (fuel efficiency). Do they differ substantially? Why?
- (b) Produce a correlation matrix for `mpg`, `hp`, `wt`, and `qsec`. Which pair has the strongest relationship?
- (c) Create a `ggpairs` plot for these four variables, colored by number of cylinders (`cyl`). Describe any multivariate patterns you observe.

💡 Exercise 1.8 (Challenge)

The Anscombe Quartet is built into R as `anscombe`. It contains four pairs of variables (`x1/y1`, `x2/y2`, `x3/y3`, `x4/y4`).

- (a) Compute the mean, variance, and correlation for each pair. What do you notice?
 - (b) Create a 2×2 scatterplot grid for all four pairs.
 - (c) What lesson does this exercise teach about relying on summary statistics alone?
-

Chapter Lab Activity: Exploring the mtcars Dataset

Objectives

In this lab, you will apply all descriptive statistics concepts from this chapter to a real-world dataset — the `mtcars` dataset, containing specifications and performance data for 32 automobiles from the 1974 *Motor Trend* magazine. The goal is to build a complete descriptive profile of the data, practicing the full workflow from initial exploration to visual communication of findings.

The Dataset

```
# Load and preview the dataset
data(mtcars)

# Display first 6 rows
kable(head(mtcars),
      caption = "First 6 Rows of the mtcars Dataset") |>
  kable_styling(bootstrap_options = c("striped", "hover"), font_size = 11)
```

Table 1: First 6 Rows of the mtcars Dataset

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# Dataset dimensions
cat("Dimensions:", nrow(mtcars), "rows x", ncol(mtcars), "columns\n")
```

Dimensions: 32 rows x 11 columns

Variable descriptions:

Variable	Description	Type
mpg	Miles per gallon (fuel efficiency)	Continuous
cyl	Number of cylinders	Categorical (4, 6, 8)
disp	Displacement (cu.in.)	Continuous
hp	Gross horsepower	Continuous
wt	Weight (1000 lbs)	Continuous
am	Transmission (0 = automatic, 1 = manual)	Binary
gear	Number of forward gears	Categorical

Lab Task 1: Full Descriptive Summary

```
# Comprehensive descriptive statistics using psych::describe()
mtcars_desc <- describe(mtcars[, c("mpg", "hp", "wt", "disp")])

kable(round(mtcars_desc[, c("n", "mean", "sd", "median", "min", "max", "skew", "kurtosis")], 3),
       caption = "Descriptive Statistics for Key mtcars Variables") |>
  kable_styling(bootstrap_options = c("striped", "hover")) |>
  column_spec(7, bold = TRUE, color = "tomato") # highlight skewness column
```

Table 2: Descriptive Statistics for Key mtcars Variables

	n	mean	sd	median	min	max	skew	kurtosis
mpg	32	20.091	6.027	19.200	10.400	33.900	0.611	-0.373
hp	32	146.688	68.563	123.000	52.000	335.000	0.726	-0.136
wt	32	3.217	0.978	3.325	1.513	5.424	0.423	-0.023
disp	32	230.722	123.939	196.300	71.100	472.000	0.382	-1.207

Interpretation guide: Examine the skewness column carefully. Which variables are approximately symmetric? Which are right-skewed? How does this inform your choice of summary statistics (mean vs. median) for each variable?

Lab Task 2: Central Tendency and Spread by Group

```
# Compare mpg across cylinder groups (4, 6, 8 cylinders)
mtcars |>
  mutate(cyl = factor(cyl, labels = c("4-cyl", "6-cyl", "8-cyl"))) |>
  group_by(cyl) |>
  summarise(
    n      = n(),
    Mean   = round(mean(mpg), 2),
    Median = round(median(mpg), 2),
    SD     = round(sd(mpg), 2),
    IQR    = round(IQR(mpg), 2),
    CV     = round(sd(mpg)/mean(mpg)*100, 1)
  ) |>
  kable(caption = "Fuel Efficiency (mpg) by Number of Cylinders") |>
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Table 3: Fuel Efficiency (mpg) by Number of Cylinders

cyl	n	Mean	Median	SD	IQR	CV
4-cyl	11	26.66	26.0	4.51	7.60	16.9
6-cyl	7	19.74	19.7	1.45	2.35	7.4
8-cyl	14	15.10	15.2	2.56	1.85	17.0

Lab Task 3: Visualizing the Full Distribution

```
# Multi-panel visualization: histogram, boxplot, QQ plot for mpg
p1 <- ggplot(mtcars, aes(x = mpg)) +
  geom_histogram(aes(y = after_stat(density)), bins = 12,
    fill = "steelblue", color = "white", alpha = 0.8) +
  geom_density(color = "tomato", linewidth = 1.2) +
  labs(title = "A. Distribution of mpg", x = "Miles per Gallon", y = "Density") +
  theme_minimal(base_size = 12)

p2 <- mtcars |>
  mutate(cyl = factor(cyl)) |>
  ggplot(aes(x = cyl, y = mpg, fill = cyl)) +
  geom_boxplot(alpha = 0.7) +
  geom_jitter(width = 0.1, alpha = 0.6, size = 2) +
```

```

scale_fill_brewer(palette = "Set2") +
labs(title = "B. mpg by Cylinders", x = "Cylinders", y = "Miles per Gallon") +
theme_minimal(base_size = 12) +
theme(legend.position = "none")

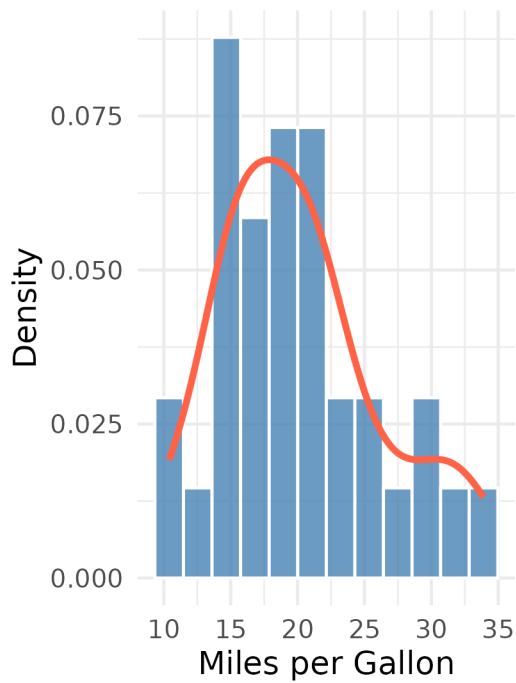
p3 <- ggplot(mtcars, aes(sample = mpg)) +
  stat_qq(color = "steelblue", size = 2.5) +
  stat_qq_line(color = "tomato", linewidth = 1) +
  labs(title = "C. Normal QQ Plot of mpg",
       x = "Theoretical Quantiles", y = "Sample Quantiles") +
  theme_minimal(base_size = 12)

p4 <- mtcars |>
  mutate(am = factor(am, labels = c("Automatic", "Manual"))) |>
  ggplot(aes(x = am, y = mpg, fill = am)) +
  geom_violin(alpha = 0.6, trim = FALSE) +
  geom_boxplot(width = 0.1, fill = "white", alpha = 0.8) +
  scale_fill_manual(values = c("Automatic" = "steelblue", "Manual" = "tomato")) +
  labs(title = "D. mpg by Transmission Type",
       x = "Transmission", y = "Miles per Gallon") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

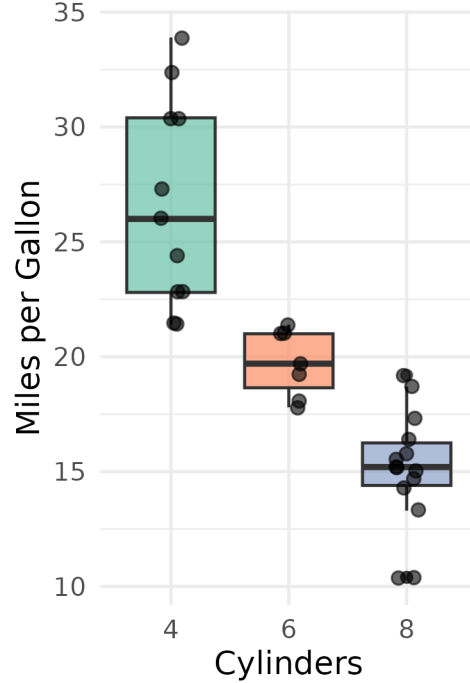
# Arrange in 2x2 grid using patchwork
library(patchwork)
(p1 + p2) / (p3 + p4)

```

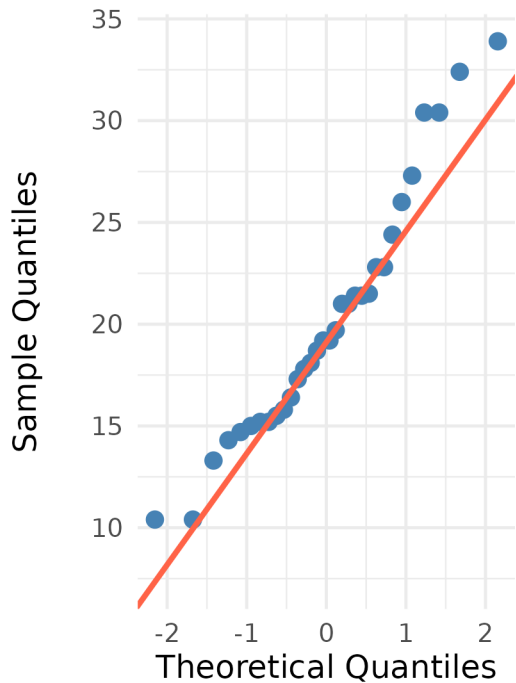
A. Distribution of mpg



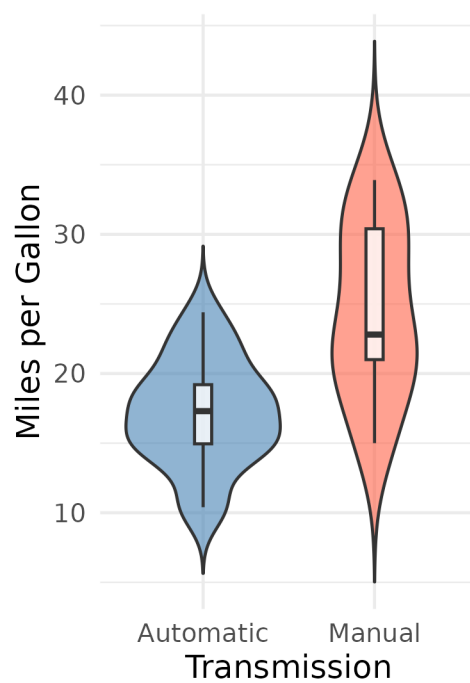
B. mpg by Cylinders



C. Normal QQ Plot of mpg



D. mpg by Transmission

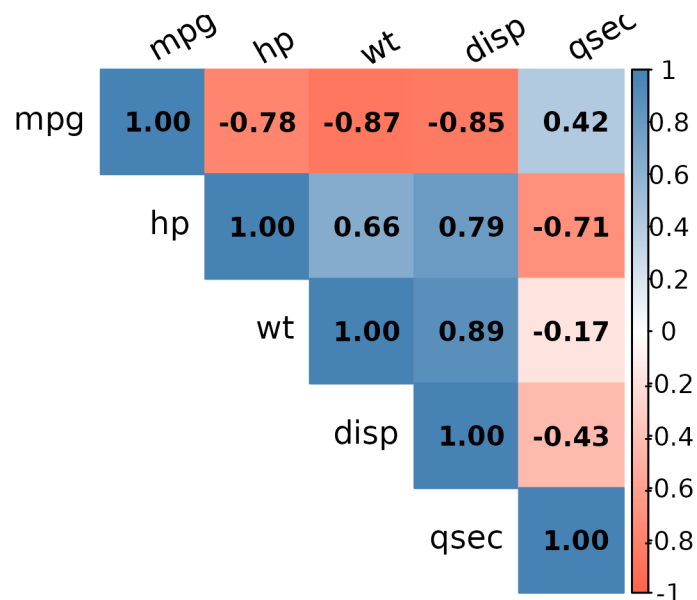


Lab Task 4: Correlation Analysis

```
# Compute and visualize correlations among continuous variables
cont_vars <- mtcars[, c("mpg", "hp", "wt", "disp", "qsec")]
cor_mat    <- cor(cont_vars)

corrplot(cor_mat,
  method      = "color",
  type        = "upper",
  addCoef.col = "black",
  number.cex  = 0.85,
  tl.col      = "black",
  tl.srt      = 30,
  col         = colorRampPalette(c("tomato", "white", "steelblue"))(200),
  title       = "Correlation Matrix: mtcars Continuous Variables",
  mar         = c(0, 0, 2, 0))
```

Correlation Matrix: mtcars Continuous Variables



Lab Discussion Questions

Answer the following in writing (100–150 words each):

1. **Central Tendency:** For the variable mpg, which measure — mean or median — is more appropriate as a summary statistic? Justify your answer using skewness and the presence/absence of outliers.

2. **Group Comparison:** Based on Task 2, describe the relationship between engine size (cylinders) and fuel efficiency. Is the variability (CV) similar across groups or does it differ? What might explain this?
 3. **Normality:** Based on the QQ plot in Task 3, does `mpg` appear normally distributed? What features of the plot support your conclusion?
 4. **Correlation vs. Causation:** The correlation between `hp` and `mpg` is strongly negative. Does this mean that reducing horsepower *causes* better fuel efficiency? What other variables might explain this relationship?
 5. **Visualization Choice:** In Panel D, a violin plot is used alongside a boxplot. What additional information does the violin plot reveal that the boxplot alone does not?
-

Chapter Summary

Summary

This chapter established the descriptive foundation for all subsequent statistical analysis. The key takeaways are:

- **Central tendency** (mean, median, mode) describes where data is centered. The mean is efficient but sensitive to outliers; the median is robust.
- **Dispersion** (range, IQR, variance, SD, CV) describes how spread out the data is. The IQR and SD are complementary — one robust, one efficient.
- **Shape** (skewness, kurtosis) reveals asymmetry and tail behavior, which guide the choice of statistical methods and transformations.
- **Visualization** is never optional. QQ plots, boxplots, histograms, and scatterplot matrices each reveal different features of the data that numbers alone cannot capture.
- **Correlation** quantifies linear relationships between pairs of variables but does not imply causation and cannot detect non-linear relationships.

! Key Formulas to Know

$$\bar{x} = \frac{1}{n} \sum x_i \quad s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2 \quad CV = \frac{s}{\bar{x}} \times 100\% \quad IQR = Q_3 - Q_1$$
$$r = \frac{S_{xy}}{\sqrt{S_{xx} S_{yy}}} = \frac{s_{XY}}{s_X s_Y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

References

- Anderson, E. (1935). The irises of the Gaspé Peninsula. *Bulletin of the American Iris Society*, 59, 2–5.
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Komsta, L., & Novomestky, F. (2022). *moments: Moments, cumulants, skewness, kurtosis and related tests* (R package version 0.14.1). <https://CRAN.R-project.org/package=moments>

- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- Posit Team. (2024). *Quarto: An open-source scientific and technical publishing system*. Posit, PBC. <https://quarto.org>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Spearman, C. (1904). “General intelligence,” objectively determined and measured. *American Journal of Psychology*, 15(2), 201–292.
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- Wei, T., & Simko, V. (2021). *corrplot: Visualization of a correlation matrix* (R package version 0.92). <https://CRAN.R-project.org/package=corrplot>
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.
- Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

End of Chapter 1. Proceed to Chapter 2: Data Distribution and Probability.

Chapter 2: Data Distribution and Probability

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(nortest)      # normality tests (ad.test, lillie.test)
```

Chapter Overview

Introduction

Understanding how data is distributed — and the probability rules that govern those distributions — is the theoretical backbone of all statistical inference. Before we can test hypotheses, build models, or make predictions, we must know what kind of data we have, what probability laws apply to it, and how random variation behaves.

This chapter covers:

- **Types of Data and Measurement Scales** — what kind of variable are we dealing with?
- **Probability Fundamentals** — the rules of reasoning under uncertainty
- **Conditional Probability and Bayes' Theorem** — updating beliefs with evidence
- **Discrete Probability Distributions** — Binomial and Poisson
- **Continuous Probability Distributions** — Normal, Uniform, and Exponential
- **Sampling Distributions and the Central Limit Theorem** — why the normal distribution is everywhere
- **Assessing Normality** — how to test whether data is normally distributed

Learning Objectives

By the end of this chapter, you will be able to:

1. Classify variables by measurement scale and identify the appropriate analysis for each.
2. Apply the fundamental rules of probability to solve real problems.
3. Use Bayes' Theorem to update probabilities based on new evidence.
4. Compute probabilities from discrete and continuous distributions by hand and in R.
5. Explain the Central Limit Theorem and its implications for statistical inference.
6. Assess normality using graphical and formal test methods in R.

Types of Data and Measurement Scales

Introduction

Not all data is the same. A zip code, a temperature reading, a Likert-scale rating, and a weight measurement may all look like numbers — but they carry fundamentally different amounts of information and support different mathematical operations. Misidentifying the scale of measurement is one of the most common errors in data analysis, leading to inappropriate statistical methods and misleading conclusions.

The **Stevens classification system** (1946) organizes variables into four hierarchical levels of measurement, each building on the properties of the one before it.

Theory

Nominal Scale

Nominal variables classify observations into **mutually exclusive, unordered categories**. Numbers assigned to nominal categories are labels only — arithmetic operations are meaningless.

- **Properties:** Identity only (categories are distinct)
- **Examples:** Blood type (A, B, AB, O), country of origin, species, gender
- **Permissible statistics:** Frequency counts, mode, chi-square tests
- **Not permissible:** Mean, median, standard deviation

Ordinal Scale

Ordinal variables have categories with a **meaningful order or ranking**, but the intervals between ranks are not necessarily equal.

- **Properties:** Identity + order
- **Examples:** Likert scales (Strongly Disagree to Strongly Agree), education level, race finishing position, satisfaction ratings
- **Permissible statistics:** Median, percentiles, rank-based (non-parametric) tests
- **Not permissible:** Mean (technically), standard deviation

The Likert Scale Debate

Whether Likert-scale data can be treated as interval (and thus analyzed with means and parametric tests) is a long-standing debate in statistics. Many researchers treat 5- or 7-point Likert scales as approximately interval, particularly when summing multiple items into a composite score. Be prepared to justify your choice.

Interval Scale

Interval variables have **equal intervals** between consecutive values, but there is **no true zero point** — zero does not mean “none of the quantity.”

- **Properties:** Identity + order + equal intervals
- **Examples:** Temperature in Celsius or Fahrenheit, calendar year, IQ scores
- **Permissible statistics:** Mean, standard deviation, correlation, t-tests
- **Not permissible:** Ratios (20°C is not “twice as hot” as 10°C)

Ratio Scale

Ratio variables possess all properties of interval scales **plus a true zero point**, meaning zero indicates the complete absence of the quantity. Ratios between values are meaningful.

- **Properties:** Identity + order + equal intervals + true zero
- **Examples:** Height, weight, income, reaction time, temperature in Kelvin
- **Permissible statistics:** All statistical operations, including geometric mean and coefficient of variation
- **Note:** This is the most informative scale and the most common in quantitative research

Summary Table

Table 1: Stevens’ Four Levels of Measurement

Scale	Order	Equal Intervals	True Zero	Examples	Key Statistics
Nominal				Blood type, species	Mode, frequency
Ordinal				Likert, rank	Median, percentile
Interval				Temperature (°C), IQ	Mean, SD, correlation
Ratio				Height, weight, income	All operations

Continuous vs. Discrete Variables

Independently of scale, variables are also classified as:

- **Discrete:** Take countable values, often integers (number of children, number of visits, count of defects). There are gaps between possible values.
- **Continuous:** Can take any value within a range (height, temperature, time). Between any two values, infinitely many values are possible.

This distinction determines whether we use a **probability mass function** (discrete) or a **probability density function** (continuous) — covered in Sections 4 and 5.

Example: Identifying Measurement Scales

Example 2.1. A health survey collects the following variables. Identify the measurement scale for each.

Table 2: Example 2.1 — Identifying Measurement Scales

Variable	Values Recorded	Scale	Justification
Patient ID	001, 002, 003	Nominal	Labels only; no order or arithmetic
Pain level	1 (none) to 10 (severe)	Ordinal	Ordered, but intervals not guaranteed equal
Body temperature	36.5°C, 37.2°C	Interval	Equal intervals; 0°C absence of temperature
Blood pressure	120 mmHg, 80 mmHg	Ratio	True zero exists; 120 is twice 60 mmHg
Hospital ward	Ward A, Ward B	Nominal	Categories with no meaningful order
Recovery days	3, 7, 14	Ratio	True zero (0 days = no recovery needed)

Key insight: Patient ID looks like a number but is nominal. Recovery days looks similar to pain level, but has a true zero. Always ask: *Does zero mean “none of this”?* and *Are the intervals equal?*

R Example: Identifying and Converting Variable Types


```
# --- Load a dataset and inspect variable types ---
data(mtcars)
```

```
# Check R's automatic type assignment
str(mtcars)
```

```
'data.frame':  32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num   6  6  4  6  8  6  8  4  4  6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num   3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num   2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num   0  0  1  1  0  1  0  1  1  1 ...
 $ am  : num   1  1  1  0  0  0  0  0  0  0 ...
 $ gear: num   4  4  4  3  3  3  3  4  4  4 ...
 $ carb: num   4  4  1  1  2  1  4  2  2  4 ...
```

```
# --- Several variables in mtcars are stored as numeric
#      but should be treated as categorical (nominal/ordinal)
#      cyl: number of cylinders (4, 6, 8) - ordinal/nominal
#      am:  transmission type (0=auto, 1=manual) - nominal
#      gear: number of gears - ordinal
```

```
# Convert to appropriate types
mtcars_clean <- mtcars |>
  mutate(
    cyl = factor(cyl, levels = c(4, 6, 8),
                 labels = c("4-cyl", "6-cyl", "8-cyl"),
                 ordered = TRUE),          # ordinal
    am  = factor(am, levels = c(0, 1),
                 labels = c("Automatic", "Manual")), # nominal
    gear = factor(gear, levels = c(3, 4, 5),
                  ordered = TRUE)          # ordinal
  )
```

```
# Inspect the corrected structure
str(mtcars_clean)
```

```
'data.frame':  32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : Ord.factor w/ 3 levels "4-cyl"<"6-cyl"<...: 2 2 1 2 3 2 3 1 1 2 ...
```

```

$ disp: num 160 160 108 258 360 ...
$ hp : num 110 110 93 110 175 105 245 62 95 123 ...
$ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
$ wt : num 2.62 2.88 2.32 3.21 3.44 ...
$ qsec: num 16.5 17 18.6 19.4 17 ...
$ vs : num 0 0 1 1 0 1 0 1 1 1 ...
$ am : Factor w/ 2 levels "Automatic","Manual": 2 2 2 1 1 1 1 1 1 1 ...
$ gear: Ord.factor w/ 3 levels "3"<"4"<"5": 2 2 2 1 1 1 1 2 2 2 ...
$ carb: num 4 4 1 1 2 1 4 2 2 4 ...

```

```

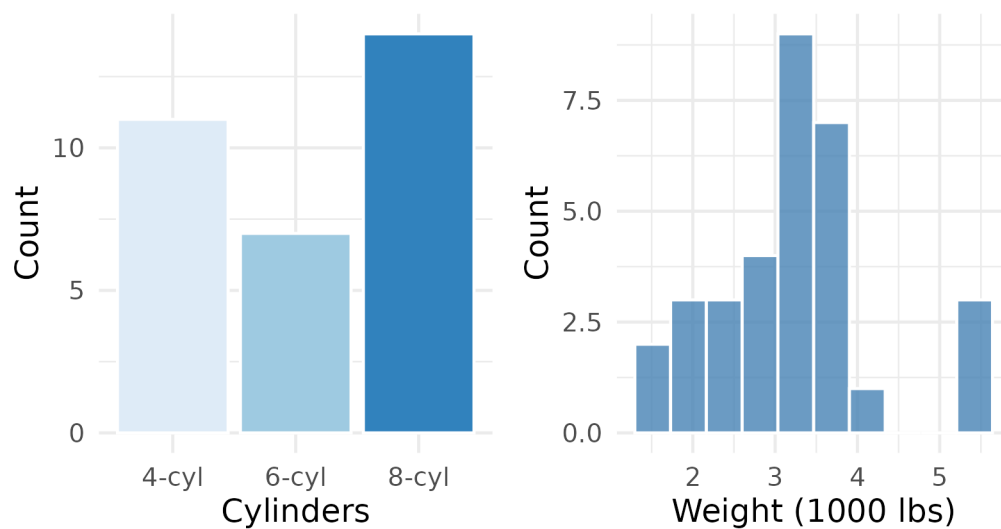
# --- Visualize appropriate to scale ---
# Nominal/Ordinal: bar chart (NOT histogram - no meaningful continuity)
p1 <- ggplot(mtcars_clean, aes(x = cyl, fill = cyl)) +
  geom_bar(color = "white") +
  scale_fill_brewer(palette = "Blues") +
  labs(title = "Ordinal Variable: Cylinders",
       subtitle = "Bar chart - appropriate for categorical data",
       x = "Cylinders", y = "Count") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

# Ratio: histogram (continuous, true zero)
p2 <- ggplot(mtcars_clean, aes(x = wt)) +
  geom_histogram(bins = 10, fill = "steelblue",
                color = "white", alpha = 0.8) +
  labs(title = "Ratio Variable: Car Weight",
       subtitle = "Histogram - appropriate for continuous data",
       x = "Weight (1000 lbs)", y = "Count") +
  theme_minimal(base_size = 12)

p1 + p2

```

Ordinal Variable: Cylinders Ratio Variable: Car Weight
 Bar chart — appropriate for categorical data Histogram — appropriate for continuous data



Code explanation:

- `factor(..., ordered = TRUE)` creates an ordered factor in R, correctly representing ordinal data.
- `str()` reveals how R internally stores each variable — a critical first step in any analysis.
- The choice of plot type (bar chart vs. histogram) follows directly from the measurement scale: bar charts for categorical, histograms for continuous.

Try modifying: Use `class()` and `levels()` on the converted variables to confirm their types and order.

Exercises

💡 Exercise 2.1

For each variable below, identify: (a) the measurement scale, (b) whether it is discrete or continuous, and (c) the most appropriate graph type.

1. Number of customer complaints per day
2. Country of birth of survey respondents
3. Annual household income in USD
4. Student satisfaction (1 = Very Dissatisfied, 5 = Very Satisfied)
5. Time taken to complete an online task (in seconds)
6. Zip code of a business location

 Exercise 2.2

Load the `iris` dataset in R. For each of the five variables, identify the measurement scale. Convert any variables that are incorrectly stored in R to their appropriate type using `factor()`. Justify each decision.

Probability Fundamentals

Introduction

Probability is the mathematical framework for **reasoning under uncertainty**. Every time a data scientist makes a prediction, estimates a parameter, or runs a hypothesis test, they are implicitly using probability theory. Understanding the foundational rules — not just the formulas — is essential for building correct intuition about statistical methods.

Theory

Sample Space and Events

A **random experiment** is any process whose outcome is uncertain. The **sample space** S is the set of all possible outcomes. An **event** A is any subset of the sample space.

$$S = \{\text{all possible outcomes}\}, \quad A \subseteq S$$

Example: Rolling a six-sided die: $S = \{1, 2, 3, 4, 5, 6\}$. The event “rolling an even number” is $A = \{2, 4, 6\}$.

Kolmogorov’s Axioms

All of probability theory rests on three axioms proposed by Andrey Kolmogorov in 1933:

1. **Non-negativity:** $P(A) \geq 0$ for any event A
2. **Normalization:** $P(S) = 1$
3. **Countable additivity:** For mutually exclusive events $A_1, A_2, \dots$:

$$P(A_1 \cup A_2 \cup \dots) = P(A_1) + P(A_2) + \dots$$

From these three axioms, all other probability rules are derived.

Derived Rules

Complement Rule:

$$P(A^c) = 1 - P(A)$$

Addition Rule (for any two events):

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Multiplication Rule (for independent events $A \perp B$):

$$P(A \cap B) = P(A) \cdot P(B)$$

Law of Total Probability: If $B_1, B_2, \dots, B_k$ partition the sample space:

$$P(A) = \sum_{i=1}^k P(A | B_i) \cdot P(B_i)$$

Frequentist vs. Bayesian Interpretation

Two schools of thought define what a probability *means*:

Table 1: Two interpretations of probability

Interpretation	Definition	Use Case
Frequentist	Long-run relative frequency in repeated trials	Classical hypothesis testing
Bayesian	Degree of belief, updated as evidence arrives	Bayesian inference, machine learning

Both interpretations are mathematically consistent with Kolmogorov's axioms — they differ in *meaning*, not in the rules.

Example: Probability Rules

Example 2.2. A data science program has 60 students. From records: 35 have programming experience (P), 28 have statistics experience (S), and 15 have both.

Find: (a) $P(P \cup S)$, (b) $P(\text{neither})$, (c) $P(P \text{ only})$

Step 1 — Convert to probabilities:

$$P(P) = \frac{35}{60} \approx 0.583, \quad P(S) = \frac{28}{60} \approx 0.467, \quad P(P \cap S) = \frac{15}{60} = 0.25$$

(a) **Addition Rule:**

$$P(P \cup S) = 0.583 + 0.467 - 0.25 = 0.80$$

So 80% of students have at least one type of experience.

(b) **Complement Rule:**

$$P(\text{neither}) = 1 - P(P \cup S) = 1 - 0.80 = 0.20$$

20% of students have neither programming nor statistics experience.

(c) **P only (programming but not statistics):**

$$P(P \cap S^c) = P(P) - P(P \cap S) = 0.583 - 0.25 = 0.333$$

R Example: Probability Rules and Simulation

```
# --- Simulate probability with repeated experiments ---
# Verify the Addition Rule via simulation

set.seed(123)
n_trials <- 100000

# Simulate 60-student cohorts: each student gets P and S independently
# P(P) = 35/60, P(S) = 28/60, P(P and S) = 15/60
# We'll simulate individual student memberships

p_prog <- 35 / 60
p_stats <- 28 / 60
p_both <- 15 / 60

# Simulate events for each trial
has_prog <- runif(n_trials) < p_prog
has_stats <- runif(n_trials) < p_stats

# Correct for the joint probability (to match P(P S) = 15/60)
# Use the conditional: P(S|P) = 15/35
has_stats_given_prog <- runif(n_trials) < (p_both / p_prog)
has_both <- has_prog & has_stats_given_prog
has_stats_not_prog <- (!has_prog) & (runif(n_trials) < ((p_stats - p_both) / (1 - p_prog)))
has_stats_final <- has_both | has_stats_not_prog
```

```
# Compute simulated probabilities
sim_results <- data.frame(
  Event      = c("P(Programming)", "P(Statistics)",
                 "P(Both)", "P(At least one)", "P(Neither)"),
  Theoretical = c(35/60, 28/60, 15/60, 0.80, 0.20),
  Simulated   = c(
    mean(has_prog),
    mean(has_stats_final),
    mean(has_both),
    mean(has_prog | has_stats_final),
    mean(!has_prog & !has_stats_final)
  )
)

sim_results[, 2:3] <- round(sim_results[, 2:3], 4)

kable(sim_results,
      caption = "Theoretical vs. Simulated Probabilities (100,000 trials)",
      col.names = c("Event", "Theoretical", "Simulated")) |>
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Table 2: Theoretical vs. Simulated Probabilities (100,000 trials)

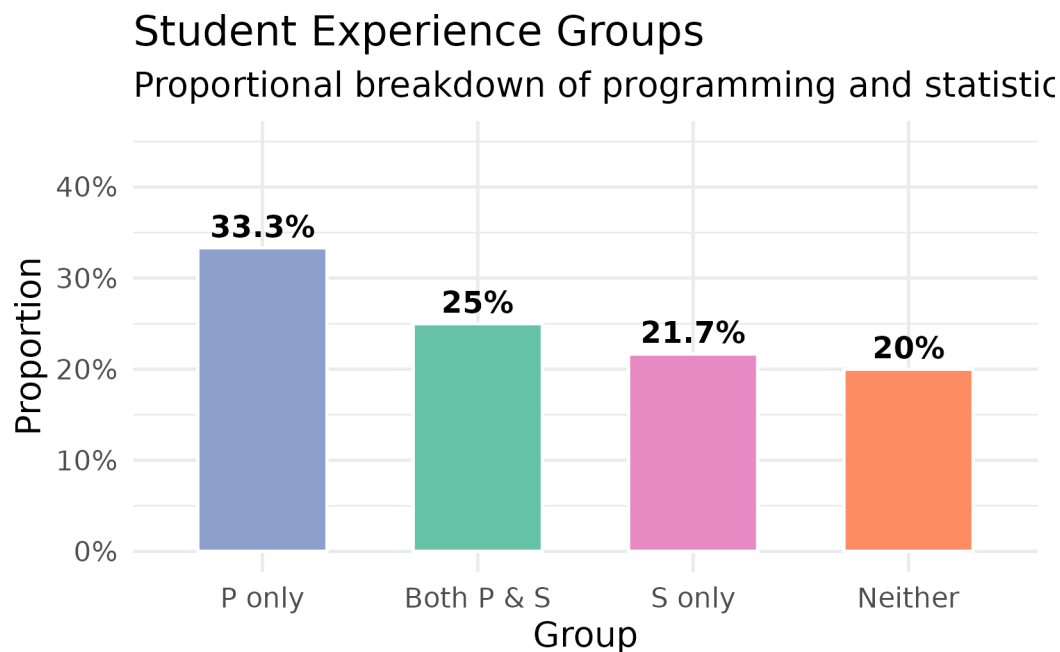
Event	Theoretical	Simulated
P(Programming)	0.5833	0.5847
P(Statistics)	0.4667	0.4648
P(Both)	0.2500	0.2505
P(At least one)	0.8000	0.7990
P(Neither)	0.2000	0.2010

```
# --- Visualize with a proportional bar chart (Venn alternative) ---
event_df <- data.frame(
  Group      = c("P only", "S only", "Both P & S", "Neither"),
  Count      = c(20, 13, 15, 12),
  Proportion = c(20/60, 13/60, 15/60, 12/60)
)

ggplot(event_df, aes(x = reorder(Group, -Proportion),
                     y = Proportion, fill = Group)) +
  geom_col(color = "white", width = 0.6) +
  geom_text(aes(label = paste0(round(Proportion * 100, 1), "%")),
            vjust = -0.5, size = 4, fontface = "bold") +
  scale_fill_brewer(palette = "Set2") +
```



```
scale_y_continuous(labels = scales::percent, limits = c(0, 0.45)) +
labs(title = "Student Experience Groups",
      subtitle = "Proportional breakdown of programming and statistics experience",
      x = "Group", y = "Proportion") +
theme_minimal(base_size = 13) +
theme(legend.position = "none")
```



Code explanation:

- `runif(n_trials)` generates uniform random numbers between 0 and 1; comparing to a probability threshold simulates Bernoulli trials.
- `set.seed(123)` ensures reproducibility — always set a seed when using random number generators.
- Simulation confirms theoretical results: with 100,000 trials, simulated and theoretical probabilities converge (Law of Large Numbers).

Exercises

💡 Exercise 2.3

A quality control inspector examines products from two machines. Machine A produces 60% of output with a 3% defect rate. Machine B produces 40% with a 5% defect rate.

- (a) What is the probability that a randomly selected product is defective?

- (b) If a product is not defective, what is the probability it came from Machine A?
- (c) Verify your answers using simulation in R (at least 50,000 trials).

Exercise 2.4

Using the `sample()` function in R, simulate rolling two fair dice 10,000 times.

- (a) Estimate $P(\text{sum} = 7)$ and compare to the theoretical value.
 - (b) Estimate $P(\text{sum} > 9)$.
 - (c) Plot the simulated distribution of sums as a bar chart.
-

Conditional Probability and Bayes' Theorem

Introduction

In practice, we rarely ask about probabilities in a vacuum. We ask: *Given that a patient tested positive, what is the probability they actually have the disease?* Or: *Given that a customer clicked an ad, what is the probability they will purchase?* **Conditional probability** formalizes how information about one event changes our assessment of another. **Bayes' Theorem** takes this further, providing a rigorous method for updating beliefs in light of new evidence — the foundation of Bayesian statistics and many machine learning algorithms.

Theory

Conditional Probability

The **conditional probability** of event A given that event B has occurred is:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad \text{provided } P(B) > 0$$

Intuitively, we restrict our universe to the world where B has already happened, and ask what fraction of that restricted world includes A .

Independence is a special case: A and B are independent if and only if:

$$P(A | B) = P(A) \quad \Longleftrightarrow \quad P(A \cap B) = P(A) \cdot P(B)$$

Bayes' Theorem

Bayes' Theorem inverts conditional probabilities — it tells us $P(A | B)$ when we know $P(B | A)$:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

Expanding the denominator using the Law of Total Probability:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B | A) \cdot P(A) + P(B | A^c) \cdot P(A^c)}$$

In Bayesian language:

Table 1: Components of Bayes' Theorem

Term	Name	Meaning
$P(A)$	Prior	Belief about A before seeing evidence B
$P(B A)$	Likelihood	Probability of the evidence if A is true
$P(B)$	Marginal likelihood	Total probability of the evidence
$P(A B)$	Posterior	Updated belief after seeing evidence B

Sensitivity, Specificity, and Predictive Value

In medical testing and classification, Bayes' Theorem connects several important metrics:

- **Sensitivity** (True Positive Rate): $P(\text{Test}^+ | \text{Disease}^+)$
- **Specificity** (True Negative Rate): $P(\text{Test}^- | \text{Disease}^-)$
- **Positive Predictive Value (PPV)**: $P(\text{Disease}^+ | \text{Test}^+)$ — computed via Bayes

$$\text{PPV} = \frac{\text{Sensitivity} \times \text{Prevalence}}{\text{Sensitivity} \times \text{Prevalence} + (1 - \text{Specificity}) \times (1 - \text{Prevalence})}$$

Example: Medical Diagnosis with Bayes' Theorem

Example 2.3. A disease affects 1% of the population (prevalence = 0.01). A diagnostic test has:

- Sensitivity = 0.95: $P(\text{Test}^+ | \text{Disease}) = 0.95$
- Specificity = 0.90: $P(\text{Test}^- | \text{No Disease}) = 0.90$, so $P(\text{Test}^+ | \text{No Disease}) = 0.10$

Question: If a patient tests positive, what is the probability they actually have the disease?

Step 1 — Prior: $P(D) = 0.01$, $P(D^c) = 0.99$

Step 2 — Marginal probability of a positive test:

$$\begin{aligned} P(\text{Test}^+) &= P(\text{Test}^+ \mid D) \cdot P(D) + P(\text{Test}^+ \mid D^c) \cdot P(D^c) \\ &= (0.95)(0.01) + (0.10)(0.99) = 0.0095 + 0.099 = 0.1085 \end{aligned}$$

Step 3 — Posterior (PPV):

$$P(D \mid \text{Test}^+) = \frac{(0.95)(0.01)}{0.1085} = \frac{0.0095}{0.1085} \approx 0.0876$$

Interpretation: Despite a positive test, there is only an **8.76% chance** the patient actually has the disease. This counterintuitive result — known as the **base rate fallacy** — occurs because the disease is rare (1%) and the false positive rate (10%) is not negligible. Prevalence (the prior) matters enormously in diagnostic settings.

R Example: Bayes' Theorem and the Base Rate Effect

```
# --- Bayes' Theorem function ---
bayes_ppv <- function(sensitivity, specificity, prevalence) {
  p_pos_given_d <- sensitivity
  p_pos_given_nd <- 1 - specificity
  p_d <- prevalence
  p_nd <- 1 - prevalence

  p_pos <- p_pos_given_d * p_d + p_pos_given_nd * p_nd
  ppv <- (p_pos_given_d * p_d) / p_pos
  npv <- (specificity * p_nd) / (1 - p_pos)

  cat("=== Bayes' Theorem: Diagnostic Test ===\n")
  cat("Sensitivity:      ", sensitivity, "\n")
  cat("Specificity:       ", specificity, "\n")
  cat("Prevalence:         ", prevalence, "\n")
  cat("P(Test+):           ", round(p_pos, 4), "\n")
  cat("PPV (P(D|Test+)):   ", round(ppv, 4), "\n")
  cat("NPV (P(D^c|Test-)): ", round(npv, 4), "\n")
}

# Run for the example
bayes_ppv(sensitivity = 0.95, specificity = 0.90, prevalence = 0.01)
```

=== Bayes' Theorem: Diagnostic Test ===

Sensitivity: 0.95
Specificity: 0.9
Prevalence: 0.01
P(Test+): 0.1085
PPV (P(D|Test+)): 0.0876
NPV (P(D^c|Test-)): 0.9994

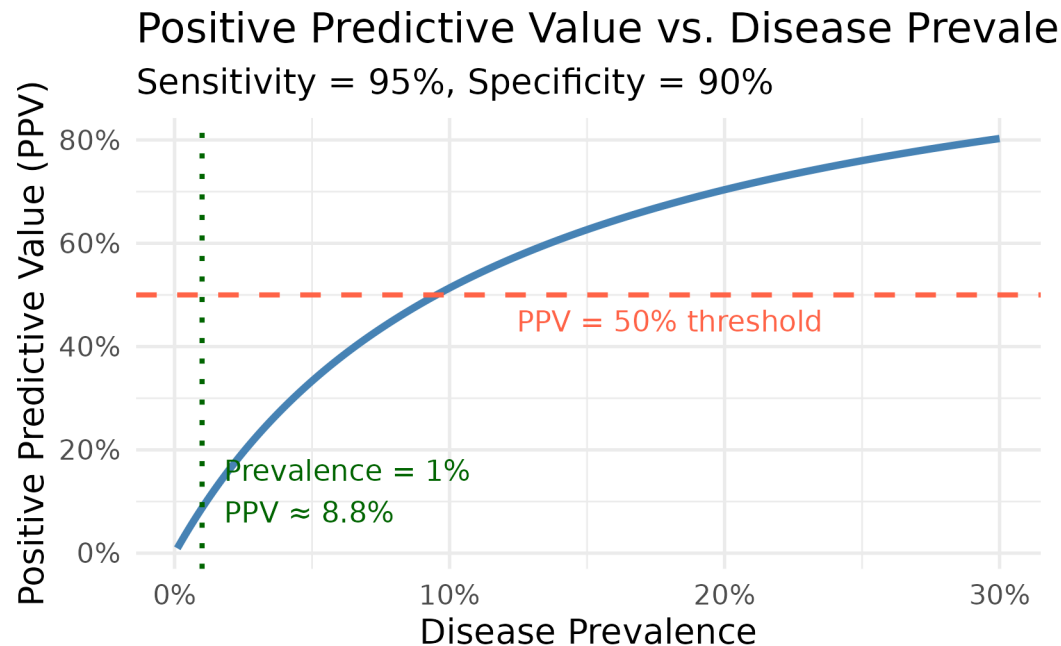
--- How does PPV change as prevalence changes? ---

```
prevalence_seq <- seq(0.001, 0.30, by = 0.001)
```

```
ppv_values <- sapply(prevalence_seq, function(prev) {  
  p_pos <- 0.95 * prev + 0.10 * (1 - prev)  
  (0.95 * prev) / p_pos  
})
```

```
ppv_df <- data.frame(prevalence = prevalence_seq, ppv = ppv_values)
```

```
ggplot(ppv_df, aes(x = prevalence, y = ppv)) +  
  geom_line(color = "steelblue", linewidth = 1.3) +  
  geom_hline(yintercept = 0.50, linetype = "dashed",  
            color = "tomato", linewidth = 0.9) +  
  geom_vline(xintercept = 0.01, linetype = "dotted",  
            color = "darkgreen", linewidth = 0.9) +  
  annotate("text", x = 0.018, y = 0.12,  
          label = "Prevalence = 1%\nPPV 8.8%",  
          color = "darkgreen", size = 3.8, hjust = 0) +  
  annotate("text", x = 0.18, y = 0.45,  
          label = "PPV = 50% threshold",  
          color = "tomato", size = 3.8) +  
  scale_x_continuous(labels = scales::percent) +  
  scale_y_continuous(labels = scales::percent) +  
  labs(title = "Positive Predictive Value vs. Disease Prevalence",  
       subtitle = "Sensitivity = 95%, Specificity = 90%",  
       x = "Disease Prevalence",  
       y = "Positive Predictive Value (PPV)") +  
  theme_minimal(base_size = 13)
```



Code explanation:

- `bayes_ppv()` is a reusable function that takes sensitivity, specificity, and prevalence as arguments — you can apply it to any diagnostic scenario.
- `sapply()` applies the PPV computation across a vector of prevalence values, enabling the sensitivity curve.
- The plot shows a critical insight: PPV improves dramatically as prevalence increases, even with fixed test characteristics.

Try modifying: Change specificity to 0.99 and observe how PPV improves at low prevalence. This illustrates why high specificity matters so much for screening rare conditions.

Exercises

💡 Exercise 2.5

A spam filter has sensitivity = 0.98 (correctly identifies spam) and specificity = 0.95 (correctly identifies legitimate email). In a typical inbox, 20% of emails are spam.

- (a) What is the probability that an email flagged as spam is actually spam (PPV)?
- (b) What is the probability that an email passed as legitimate is actually spam (1 - NPV)?
- (c) How do these values change if spam increases to 40% of all emails? Use your R function.

💡 Exercise 2.6 (Challenge)

A factory uses two independent tests to detect a defect (prevalence = 5%). Test 1: sensitivity = 0.90, specificity = 0.85. Test 2: sensitivity = 0.85, specificity = 0.92. A product is flagged only if **both** tests are positive.

- (a) Derive the combined sensitivity and specificity of the two-test system.
- (b) Compute the PPV of the combined system.
- (c) Compare to using Test 1 alone. Which system is more reliable?

Discrete Probability Distributions

Introduction

A **probability distribution** is a mathematical function that assigns probabilities to all possible values of a random variable. For **discrete** random variables — those that take countable values — the distribution is described by a **probability mass function (PMF)**. In data science, discrete distributions model count data, binary outcomes, and rare events. Two distributions are fundamental: the **Binomial** (fixed trials with two outcomes) and the **Poisson** (counts of rare events over an interval).

Theory

The Binomial Distribution

The **Binomial distribution** $X \sim \text{Binomial}(n, p)$ models the number of successes in n **independent** trials, each with success probability p .

PMF:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n$$

where $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ is the binomial coefficient (number of ways to choose k from n).

Key properties:

Table 1: Binomial distribution properties

Property	Formula
Mean	$\mu = np$
Variance	$\sigma^2 = np(1 - p)$
Standard Deviation	$\sigma = \sqrt{np(1 - p)}$

Assumptions: Fixed n , constant p , independent trials, binary outcome (success/failure).

The Poisson Distribution

The **Poisson distribution** $X \sim \text{Poisson}(\lambda)$ models the number of events occurring in a fixed interval of time or space, when events occur independently at a constant average rate λ .

PMF:

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

Key properties:

Table 2: Poisson distribution properties

Property	Formula
Mean	$\mu = \lambda$
Variance	$\sigma^2 = \lambda$
Standard Deviation	$\sigma = \sqrt{\lambda}$

Note that in the Poisson distribution, the **mean equals the variance** — a useful diagnostic. If observed count data shows variance much larger than the mean, **overdispersion** is present and a negative binomial model may be more appropriate.

Poisson as approximation to Binomial: When n is large and p is small (rare events), $\text{Binomial}(n, p) \approx \text{Poisson}(\lambda = np)$.

Example: Binomial and Poisson Distributions

Example 2.4 — Binomial. A multiple-choice exam has 20 questions, each with 4 options. A student guesses randomly on every question. Let X = number of correct answers.

$$X \sim \text{Binomial}(n = 20, p = 0.25)$$

- $\mu = 20 \times 0.25 = 5$ correct answers expected
- $\sigma^2 = 20 \times 0.25 \times 0.75 = 3.75$, $\sigma \approx 1.94$

Probability of passing (at least 10 correct):

$$P(X \geq 10) = 1 - P(X \leq 9) = 1 - \sum_{k=0}^9 \binom{20}{k} (0.25)^k (0.75)^{20-k} \approx 0.014$$

There is only a 1.4% chance of passing by pure guessing.

Example 2.5 — Poisson. A hospital emergency room receives an average of 8 patients per hour. Let X = number of patients in the next hour.

$$X \sim \text{Poisson}(\lambda = 8)$$

- $P(X = 10) = \frac{e^{-8.8^{10}}}{10!} \approx 0.099$ (about 9.9% chance of exactly 10 arrivals)
- $P(X > 12) = 1 - P(X \leq 12) \approx 0.064$ (6.4% chance of more than 12 arrivals)

R Example: Binomial and Poisson Distributions

```
# === BINOMIAL DISTRIBUTION ===
# X ~ Binomial(n=20, p=0.25)
n <- 20; p <- 0.25
x_binom <- 0:n

# PMF and CDF
binom_df <- data.frame(
  x      = x_binom,
  pmf    = dbinom(x_binom, size = n, prob = p),
  cdf    = pbinom(x_binom, size = n, prob = p)
)

# Probability of passing (X >= 10)
p_pass <- 1 - pbinom(9, size = n, prob = p)
cat("Binomial(20, 0.25):\n")
```

Binomial(20, 0.25):

```
cat("  Mean:                ", n * p, "\n")
```

Mean: 5

```
cat("  Variance:             ", n * p * (1-p), "\n")
```

Variance: 3.75

```
cat("  P(X >= 10) [pass]:   ", round(p_pass, 4), "\n\n")
```

P(X >= 10) [pass]: 0.0139

```
# === POISSON DISTRIBUTION ===
# X ~ Poisson(lambda = 8)
lambda <- 8
x_pois <- 0:25

pois_df <- data.frame(
  x = x_pois,
  pmf = dpois(x_pois, lambda = lambda),
  cdf = ppois(x_pois, lambda = lambda)
)

p_gt12 <- 1 - ppois(12, lambda = lambda)
cat("Poisson(lambda = 8):\n")
```

Poisson(lambda = 8):

```
cat("  Mean = Variance:      ", lambda, "\n")
```

Mean = Variance: 8

```
cat("  P(X = 10):           ", round(dpois(10, lambda), 4), "\n")
```

P(X = 10): 0.0993

```
cat("  P(X > 12):           ", round(p_gt12, 4), "\n")
```

P(X > 12): 0.0638

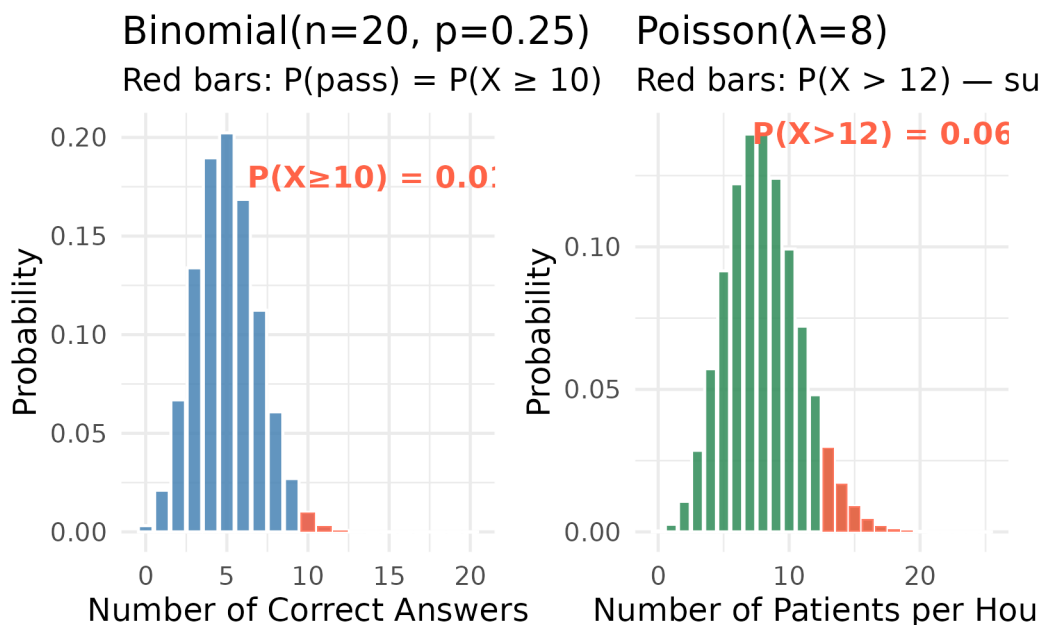
```
# --- Plot both PMFs side by side ---
p1 <- ggplot(binom_df, aes(x = x, y = pmf)) +
  geom_col(fill = "steelblue", color = "white", alpha = 0.85) +
  geom_col(data = subset(binom_df, x >= 10),
    aes(x = x, y = pmf), fill = "tomato", alpha = 0.85) +
  annotate("text", x = 15, y = 0.18,
    label = paste0("P(X 10) = ", round(p_pass, 3)),
    color = "tomato", fontface = "bold", size = 4) +
  labs(title = "Binomial(n=20, p=0.25)",
    subtitle = "Red bars: P(pass) = P(X 10)",
    x = "Number of Correct Answers",
    y = "Probability") +
  theme_minimal(base_size = 12)
```

```

p2 <- ggplot(pois_df, aes(x = x, y = pmf)) +
  geom_col(fill = "seagreen", color = "white", alpha = 0.85) +
  geom_col(data = subset(pois_df, x > 12),
    aes(x = x, y = pmf), fill = "tomato", alpha = 0.85) +
  annotate("text", x = 18, y = 0.14,
    label = paste0("P(X>12) = ", round(p_gt12, 3)),
    color = "tomato", fontface = "bold", size = 4) +
  labs(title = "Poisson(=8)",
    subtitle = "Red bars: P(X > 12) - surge risk",
    x = "Number of Patients per Hour",
    y = "Probability") +
  theme_minimal(base_size = 12)

p1 + p2

```



Code explanation:

- `dbinom(x, size, prob)` computes the PMF (exact probability at each value); `pbinom()` computes the CDF (cumulative probability up to and including x).
- `dpois(x, lambda)` and `ppois(x, lambda)` are the Poisson equivalents.
- Highlighting specific bars (using `subset()` inside a second `geom_col()`) draws attention to the probability region of interest — a technique useful in reports and teaching.

Exercises

Exercise 2.7

A new medical treatment is effective in 70% of patients. In a clinical trial of 15 patients:

- (a) What is the expected number of patients who respond?
- (b) Find $P(X \geq 12)$ — the probability that at least 12 patients respond.
- (c) Plot the full PMF and shade the region corresponding to $P(X \geq 12)$.
- (d) What sample size would be needed so that $P(X \geq 0.8n) \geq 0.90$? Explore this in R.

Exercise 2.8

Website traffic follows a Poisson process with an average of 5 visits per minute.

- (a) Find the probability of exactly 3 visits in one minute.
 - (b) Find the probability of more than 8 visits in one minute.
 - (c) In a 3-minute window, what distribution does the total visits follow, and what is the probability of more than 20 visits?
 - (d) Check whether the mean and variance of a $\text{Poisson}(5)$ sample of 1,000 observations are approximately equal. Simulate in R.
-

Continuous Probability Distributions

Introduction

When a random variable can take any value within a continuous range, we describe its distribution using a **probability density function (PDF)** $f(x)$. Unlike a PMF, the PDF does not give the probability at a single point (which is always zero for continuous variables) — instead, probabilities are areas under the curve:

$$P(a \leq X \leq b) = \int_a^b f(x) dx$$

Three continuous distributions are essential foundations in data science: the **Normal**, the **Uniform**, and the **Exponential**.

Theory

The Normal (Gaussian) Distribution

$X \sim N(\mu, \sigma^2)$ is the most important distribution in statistics. Its symmetric, bell-shaped PDF is:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad -\infty < x < \infty$$

The **standard normal** $Z \sim N(0, 1)$ is obtained by standardizing: $Z = (X - \mu)/\sigma$.

Empirical Rule (68-95-99.7 rule):

$$P(\mu - \sigma \leq X \leq \mu + \sigma) \approx 0.6827$$

$$P(\mu - 2\sigma \leq X \leq \mu + 2\sigma) \approx 0.9545$$

$$P(\mu - 3\sigma \leq X \leq \mu + 3\sigma) \approx 0.9973$$

The Uniform Distribution

$X \sim \text{Uniform}(a, b)$ assigns equal probability density to every value between a and b :

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

Mean: $\mu = \frac{a+b}{2}$, **Variance:** $\sigma^2 = \frac{(b-a)^2}{12}$

The Uniform distribution is used in random number generation, simulation, and as a non-informative (flat) prior in Bayesian analysis.

The Exponential Distribution

$X \sim \text{Exponential}(\lambda)$ models the **waiting time** between consecutive Poisson events:

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

Mean: $\mu = 1/\lambda$, **Variance:** $\sigma^2 = 1/\lambda^2$

The exponential distribution has the **memoryless property**: $P(X > s + t \mid X > s) = P(X > t)$. The future waiting time does not depend on how long you have already waited — relevant for modelling system failures, service times, and inter-arrival times.

Example: Normal, Uniform, and Exponential

Example 2.6 — Normal. IQ scores are distributed as $N(\mu = 100, \sigma^2 = 15^2)$.

(a) $P(85 \leq X \leq 115)$ — within one standard deviation: By the empirical rule $\approx 68.27\%$.
Exact: $P(-1 \leq Z \leq 1) = 0.6827$.

(b) $P(X > 130)$ — probability of “gifted” range:

$$Z = \frac{130 - 100}{15} = 2.0 \quad P(X > 130) = P(Z > 2) = 1 - 0.9772 = 0.0228$$

About 2.3% of the population has IQ above 130.

Example 2.7 — Exponential. A server processes requests at a rate of $\lambda = 4$ per minute (mean waiting time = 15 seconds = 0.25 minutes).

$P(X > 0.5) = e^{-4 \times 0.5} = e^{-2} \approx 0.135$ — 13.5% chance of waiting more than 30 seconds.

R Example: Continuous Distributions

```
# === NORMAL DISTRIBUTION ===  
mu <- 100; sigma <- 15  
  
# Empirical rule verification  
p_1sd <- pnorm(mu + sigma, mu, sigma) - pnorm(mu - sigma, mu, sigma)  
p_2sd <- pnorm(mu + 2*sigma, mu, sigma) - pnorm(mu - 2*sigma, mu, sigma)  
p_gt130 <- 1 - pnorm(130, mu, sigma)  
  
cat("Normal(mu=100, sigma=15):\n")
```

Normal(mu=100, sigma=15):

```
cat(" P(within 1 SD): ", round(p_1sd, 4), "\n")
```

P(within 1 SD): 0.6827

```
cat(" P(within 2 SD): ", round(p_2sd, 4), "\n")
```

P(within 2 SD): 0.9545

```
cat(" P(IQ > 130): ", round(p_gt130, 4), "\n\n")
```

P(IQ > 130): 0.0228

```
# === EXPONENTIAL DISTRIBUTION ===  
lambda_exp <- 4 # rate = 4 per minute  
p_wait_gt30sec <- 1 - pexp(0.5, rate = lambda_exp)  
cat("Exponential(lambda=4 per min):\n")
```

Exponential(lambda=4 per min):

```
cat(" Mean wait time (min): ", 1/lambda_exp, "\n")
```

Mean wait time (min): 0.25

```
cat(" P(wait > 30 sec):      ", round(p_wait_gt30sec, 4), "\n")
```

```
P(wait > 30 sec):      0.1353
```

```
# --- Plot all three distributions ---
x_norm <- seq(55, 145, length.out = 300)
x_unif <- seq(-0.2, 1.2, length.out = 300)
x_exp <- seq(0, 2, length.out = 300)

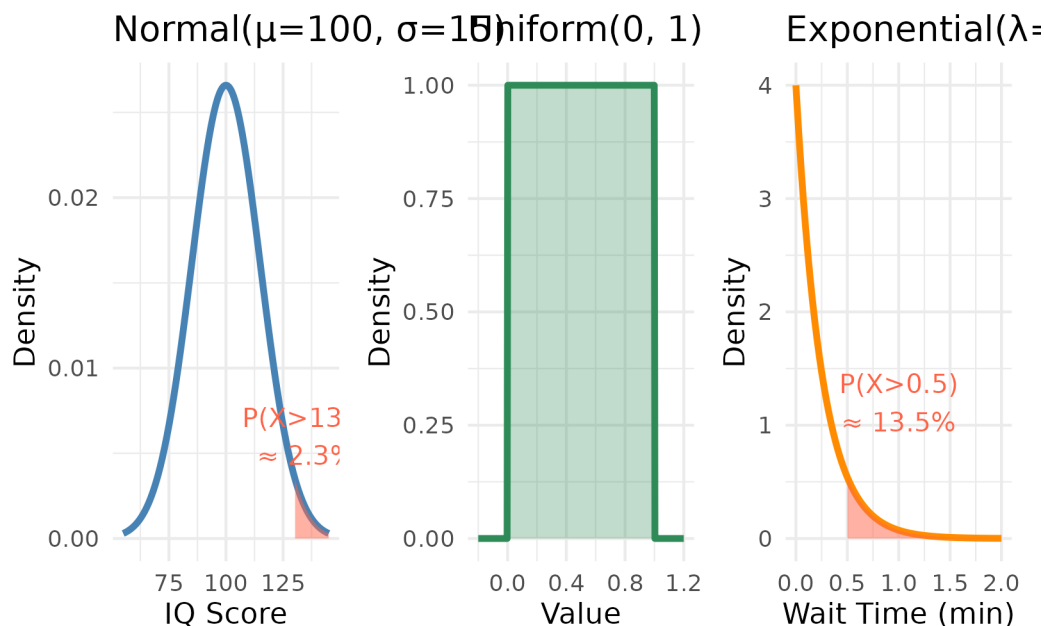
# Normal
norm_df <- data.frame(x = x_norm, y = dnorm(x_norm, 100, 15),
                      shade = x_norm >= 130)
p1 <- ggplot(norm_df, aes(x, y)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_area(data = subset(norm_df, shade),
            fill = "tomato", alpha = 0.5) +
  annotate("text", x = 135, y = 0.006,
           label = "P(X>130)\n 2.3%", color = "tomato", size = 3.5) +
  labs(title = "Normal(=100, =15)",
       x = "IQ Score", y = "Density") +
  theme_minimal(base_size = 11)

# Uniform
unif_df <- data.frame(x = x_unif,
                      y = dunif(x_unif, min = 0, max = 1))
p2 <- ggplot(unif_df, aes(x, y)) +
  geom_line(color = "seagreen", linewidth = 1.2) +
  geom_area(data = subset(unif_df, x >= 0 & x <= 1),
            fill = "seagreen", alpha = 0.3) +
  labs(title = "Uniform(0, 1)",
       x = "Value", y = "Density") +
  theme_minimal(base_size = 11)

# Exponential
exp_df <- data.frame(x = x_exp, y = dexp(x_exp, rate = 4),
                      shade = x_exp > 0.5)
p3 <- ggplot(exp_df, aes(x, y)) +
  geom_line(color = "darkorange", linewidth = 1.2) +
  geom_area(data = subset(exp_df, shade),
            fill = "tomato", alpha = 0.5) +
  annotate("text", x = 1, y = 1.2,
           label = "P(X>0.5)\n 13.5%", color = "tomato", size = 3.5) +
  labs(title = "Exponential(=4)",
```

```
x = "Wait Time (min)", y = "Density") +
theme_minimal(base_size = 11)
```

```
p1 + p2 + p3
```



Code explanation:

- `dnorm()`, `dunif()`, `dexp()` compute the PDF (density) — used for plotting the curve.
- `pnorm()`, `punif()`, `pexp()` compute the CDF (cumulative probability) — used for probability calculations.
- `qnorm()`, `qunif()`, `qexp()` compute quantiles (inverse CDF) — used to find critical values.
- Shaded areas under the PDF represent probabilities — a key visual concept for students learning integration-based probability.

Exercises

💡 Exercise 2.9

Heights of adult males in a population follow $N(\mu = 175 \text{ cm}, \sigma^2 = 6.25^2)$.

- What proportion of males are taller than 185 cm?
- What height marks the 90th percentile?

- (c) Between what two heights (symmetric around the mean) do the middle 95% of males fall?
- (d) Plot the full PDF and shade the region corresponding to (a).

Exercise 2.10

A call center receives calls at a rate of 12 per hour (Poisson process).

- (a) What is the distribution of waiting time between calls? State the parameter(s).
 - (b) What is the probability of waiting more than 10 minutes between calls?
 - (c) Simulate 10,000 inter-arrival times in R and compare the sample mean and variance to theoretical values.
 - (d) Plot a histogram of the simulated times with the theoretical PDF overlaid.
-

Sampling Distributions and the Central Limit Theorem

Introduction

In practice, we observe a **sample** and wish to draw conclusions about the **population**. The key bridge between sample and population is the concept of a **sampling distribution** — the probability distribution of a statistic (like the sample mean) computed from all possible samples of size n . Understanding sampling distributions, and especially the **Central Limit Theorem (CLT)**, is what makes inference possible and explains why the normal distribution appears so ubiquitously in statistics.

Theory

Sampling Distribution of the Mean

Suppose we draw a random sample $X_1, X_2, \dots, X_n$ from a population with mean μ and variance σ^2 . The sample mean is:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

The sampling distribution of $\bar{X}$ has:

$$\begin{aligned} E[\bar{X}] &= \mu && \text{(unbiased)} \\ \text{Var}(\bar{X}) &= \frac{\sigma^2}{n} && \text{(variance decreases with sample size)} \\ \text{SE}(\bar{X}) &= \frac{\sigma}{\sqrt{n}} && \text{(standard error)} \end{aligned}$$

The Central Limit Theorem (CLT)

! Central Limit Theorem

Let $X_1, X_2, \dots, X_n$ be independent and identically distributed random variables from **any distribution** with mean μ and finite variance σ^2 . Then as $n \rightarrow \infty$:

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0, 1)$$

The standardized sample mean converges in distribution to the standard normal, **regardless of the shape of the original population distribution**.

Practical rule of thumb: For most distributions, $n \geq 30$ is sufficient for the normal approximation to be reliable. For highly skewed distributions, larger n may be needed.

Why this matters: The CLT justifies the use of z -tests and t -tests even when the population is not normal, provided the sample is large enough. It explains why the normal distribution appears so frequently in real data — many measurements are averages or sums of numerous independent contributions.

The t -Distribution

When σ is unknown (always in practice) and we substitute the sample standard deviation s , the standardized mean follows a **Student's t -distribution** with $n - 1$ degrees of freedom:

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}} \sim t(n - 1)$$

The t -distribution is symmetric and bell-shaped like the normal, but has **heavier tails** — reflecting the additional uncertainty from estimating σ . As $n \rightarrow \infty$, $t(n - 1) \rightarrow N(0, 1)$.

A note on notation you'll need soon: every distribution has a CDF (`pnorm()`, `pt()`, ...) that goes from a value to a cumulative probability. It also has an inverse — the **quantile function** (`qnorm()`, `qt()`, ...) — that goes the other way: from a cumulative probability back to a value. We write the quantile using a subscript equal to that cumulative probability, e.g. $z_{0.975}$ or $t_{0.95, df}$. You'll use this constantly in Chapter 3 to find critical values — we'll build it up properly there.

Example: Demonstrating the CLT

Example 2.8. Consider sampling from a highly skewed **Exponential(1)** distribution ($\mu = 1$, $\sigma = 1$). The CLT predicts that for large n , the sampling distribution of $\bar{X}$ will be approximately $N(1, 1/n)$.

We verify this empirically by taking 10,000 samples of size $n = 2, 5, 30, 100$ and examining the distribution of sample means. As n grows, the histogram of $\bar{X}$ becomes increasingly bell-shaped, regardless of the skewed parent distribution.

R Example: Demonstrating the Central Limit Theorem

```
# --- CLT Demonstration ---
# Parent distribution: Exponential(rate=1) - highly right-skewed
set.seed(42)
n_samples <- 10000
sample_sizes <- c(2, 5, 30, 100)

# Function: draw n_samples means of size n
get_means <- function(n, n_samp = 10000) {
  replicate(n_samp, mean(rexp(n, rate = 1)))
}

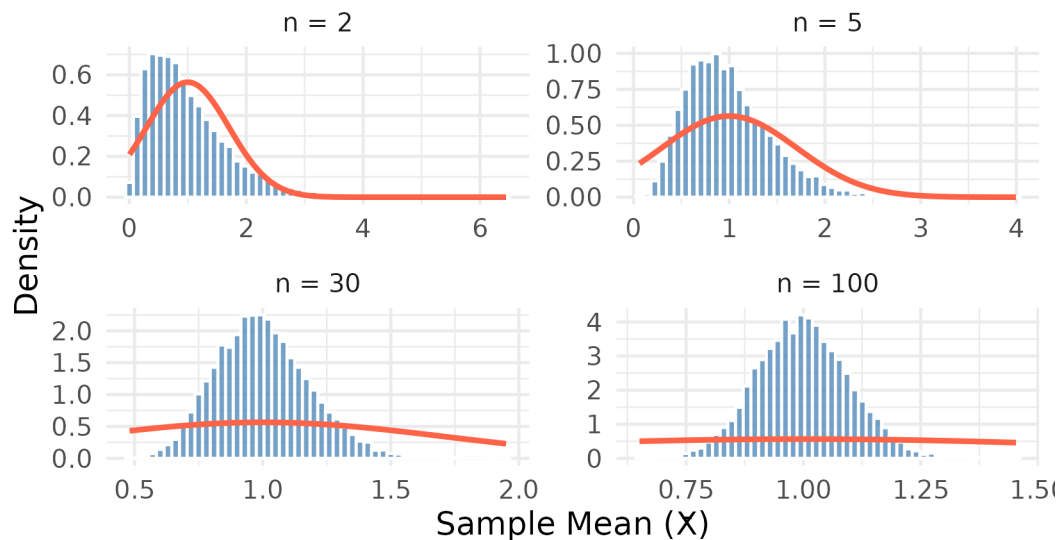
# Collect sample means for each sample size
clt_df <- map_dfr(sample_sizes, function(n) {
  data.frame(
    sample_size = paste0("n = ", n),
    xbar        = get_means(n)
  )
})

clt_df$sample_size <- factor(clt_df$sample_size,
                             levels = paste0("n = ", sample_sizes))

# --- Plot ---
ggplot(clt_df, aes(x = xbar)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 50, fill = "steelblue",
                 color = "white", alpha = 0.75) +
  stat_function(fun = dnorm,
               args = list(mean = 1, sd = 1 / sqrt(as.numeric(
                 gsub("n = ", "", clt_df$sample_size[1])))),
               color = "tomato", linewidth = 1) +
  facet_wrap(~sample_size, scales = "free", ncol = 2) +
  labs(title = "Central Limit Theorem: Sampling from Exponential(1)",
       subtitle = "As n increases, the distribution of X approaches Normal",
       x = "Sample Mean (X)", y = "Density") +
  theme_minimal(base_size = 12)
```

Central Limit Theorem: Sampling from Exponential

As n increases, the distribution of $\bar{X}$ approaches Normal



```
# --- Overlay theoretical normal for each panel ---
clt_summary <- clt_df |>
  mutate(n_val = as.numeric(gsub("n = ", "", sample_size))) |>
  group_by(sample_size, n_val) |>
  summarise(.groups = "drop")

# Better approach: compute theoretical curves per panel
norm_curves <- map_dfr(sample_sizes, function(n) {
  x_seq <- seq(0, 3, length.out = 300)
  data.frame(
    sample_size = paste0("n = ", n),
    x           = x_seq,
    y           = dnorm(x_seq, mean = 1, sd = 1 / sqrt(n))
  )
})
norm_curves$sample_size <- factor(norm_curves$sample_size,
  levels = paste0("n = ", sample_sizes))

ggplot(clt_df, aes(x = xbar)) +
  geom_histogram(aes(y = after_stat(density)),
    bins = 60, fill = "steelblue",
    color = "white", alpha = 0.7) +
  geom_line(data = norm_curves, aes(x = x, y = y),
    color = "tomato", linewidth = 1.2) +
  facet_wrap(~sample_size, scales = "free", ncol = 2) +
  labs(title = "CLT: Simulated Sampling Distributions vs. Theoretical Normal",
```



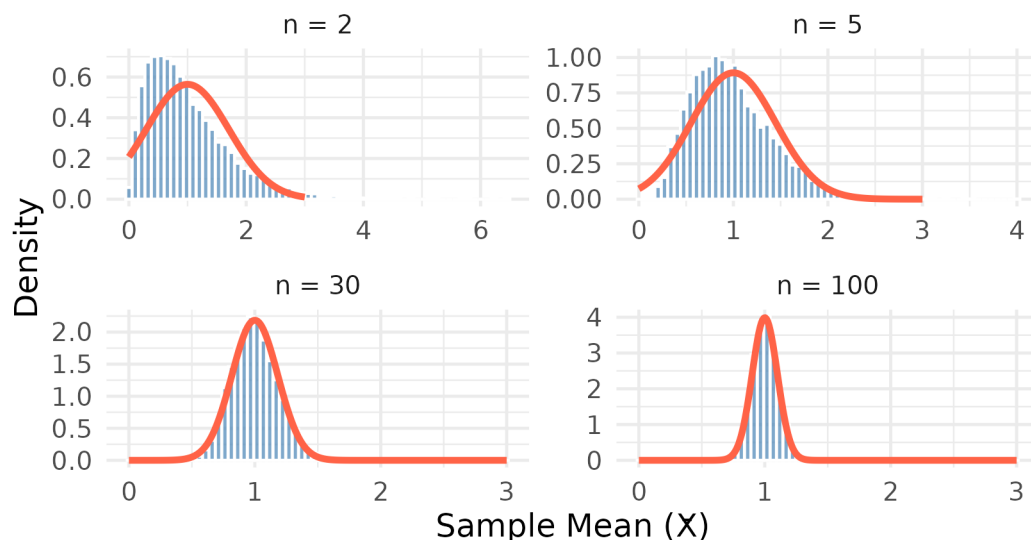
```

  subtitle = "Red curve = theoretical N( ,  $\sigma^2/n$ ) - fits better as n grows",
  x         = "Sample Mean ( $\bar{X}$ )", y         = "Density") +
theme_minimal(base_size = 12)

```

CLT: Simulated Sampling Distributions vs. Theoretical

Red curve = theoretical $N(\mu, \sigma^2/n)$ — fits better as n grows



Code explanation:

- `replicate(n_samp, mean(rexp(n, rate=1)))` is the core simulation: repeat `n_samp` times, drawing a sample of size `n` from an Exponential distribution and computing its mean.
- `map_dfr()` from `purrr` (loaded via `tidyverse`) applies a function across a vector and row-binds the results into a data frame — a clean alternative to a `for` loop.
- Separate `norm_curves` data frame computes the theoretical normal PDF for each panel, enabling correct overlay across facets with different scales.

Exercises

💡 Exercise 2.11

- Simulate the CLT using a **Uniform(0, 1)** parent distribution. Draw 10,000 samples of sizes $n = 1, 5, 20, 50$ and plot the sampling distribution of $\bar{X}$ for each.
- At what sample size does the sampling distribution appear approximately normal?
- Compare the simulated standard error to the theoretical value $\sigma/\sqrt{n}$ for each n .

 Exercise 2.12

A population of exam scores has $\mu = 72$ and $\sigma = 14$ (distribution unknown).

- (a) A random sample of $n = 49$ students is drawn. What is the probability that the sample mean exceeds 75?
 - (b) How large must n be so that $P(\bar{X} > 74) < 0.05$?
 - (c) Why can we use the normal distribution to answer (a) even though the population distribution is unknown?
-

Assessing Normality

Introduction

Many statistical tests — including t -tests, ANOVA, linear regression residual diagnostics, and Pearson correlation — assume that the data (or the residuals) are approximately normally distributed. **Assessing normality** is therefore a critical step in any data analysis workflow, not a formality. In this section, we cover both graphical methods (visual, flexible) and formal statistical tests (objective, sample-size dependent), and discuss how to interpret their results in practice.

Theory

Graphical Methods

Histogram with normal curve overlay: A histogram compared to the shape of a normal distribution provides an intuitive first check. Departures such as skewness, bimodality, or heavy tails are immediately visible.

QQ Plot (Quantile-Quantile Plot): A QQ plot compares the **quantiles of the observed data** to the **quantiles expected from a normal distribution**. If the data are normal, points fall approximately on a straight diagonal line. Specific patterns indicate specific departures:

Table 1: Interpreting QQ plot patterns

Pattern in QQ Plot	Interpretation
Points on the line	Approximately normal
S-shaped curve	Skewness (left or right)
Points curve up at both ends	Heavy tails (leptokurtic)
Points curve down at both ends	Light tails (platykurtic)
Step pattern	Discrete data

Formal Normality Tests

Formal tests assess H_0 : the data follow a normal distribution versus H_1 : the data do not.

Shapiro-Wilk Test (`shapiro.test` in R): - Best for small to moderate samples ($n \leq 2000$)

- Generally the most powerful normality test - W statistic close to 1 indicates normality; small p -value $\rightarrow$ reject normality

Kolmogorov-Smirnov Test (`ks.test` in R): - Compares empirical CDF to theoretical CDF - Less powerful than Shapiro-Wilk; requires estimating parameters separately - More appropriate for large samples

Anderson-Darling Test (`ad.test` in `nortest` package): - Places more weight on the tails than the KS test - Recommended when tail behavior is particularly important

A Critical Caveat About Normality Tests

With **large samples** ($n > 200$), normality tests become so powerful that they reject H_0 for trivial, practically unimportant deviations from normality. With **small samples**, tests have low power and may fail to detect real non-normality. **Always combine graphical inspection with formal tests**, and use subject-matter knowledge to judge whether any detected departure from normality matters for the analysis.

Transformations for Non-Normal Data

When normality is violated, transformations can make data approximately normal:

Table 2: Common variance-stabilizing transformations

Pattern	Recommended Transformation
Right skew	Log: $\log(x)$ or Square root: $\sqrt{x}$
Left skew	Square: x^2 or Exponential: e^x
Proportions (0 to 1)	Logit: $\log(p/(1-p))$
Count data	Square root: $\sqrt{x}$

Example: Assessing Normality of Exam Scores

Example 2.9. A researcher records exam scores for 50 students. Summary: $\bar{x} = 74.2$, $s = 11.8$, skewness = -0.42 . The Shapiro-Wilk test gives $W = 0.973$, $p = 0.32$.

Interpretation:

- Skewness of -0.42 suggests mild left skew (most students score above the mean, with a few very low scores).

- The QQ plot shows minor departures at the lower tail.
- Shapiro-Wilk $p = 0.32 > 0.05$: we **fail to reject** H_0 — no statistically significant evidence against normality.
- **Conclusion:** The data are approximately normal; parametric methods are appropriate.

R Example: Testing Normality

```
# --- Use the 'airquality' built-in dataset ---
# Focus on Ozone concentration (known to be right-skewed)
data(airquality)
ozone <- na.omit(airquality$Ozone)

cat("=== Ozone Concentration - Normality Assessment ===\n")
```

```
=== Ozone Concentration - Normality Assessment ===
```

```
cat("n:           ", length(ozone), "\n")
```

```
n:           116
```

```
cat("Mean:        ", round(mean(ozone), 2), "\n")
```

```
Mean:        42.13
```

```
cat("Median:      ", median(ozone), "\n")
```

```
Median:      31.5
```

```
cat("Skewness:    ", round(moments::skewness(ozone), 3), "\n\n")
```

```
Skewness:    1.226
```

```
# Shapiro-Wilk Test
sw_test <- shapiro.test(ozone)
cat("Shapiro-Wilk Test:\n")
```

```
Shapiro-Wilk Test:
```

```
cat("  W =", round(sw_test$statistic, 4),
    "  p-value =", round(sw_test$p.value, 4), "\n\n")
```

W = 0.8787 p-value = 0

```
# Anderson-Darling Test (from nortest package)
ad_result <- nortest::ad.test(ozone)
cat("Anderson-Darling Test:\n")
```

Anderson-Darling Test:

```
cat("  A =", round(ad_result$statistic, 4),
    "  p-value =", round(ad_result$p.value, 4), "\n")
```

A = 4.5211 p-value = 0

```
# --- Four-panel normality assessment ---
ozone_df <- data.frame(ozone = ozone, log_ozone = log(ozone))

p1 <- ggplot(ozone_df, aes(x = ozone)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20,
                 fill = "steelblue", color = "white", alpha = 0.8) +
  stat_function(fun = dnorm,
               args = list(mean = mean(ozone), sd = sd(ozone)),
               color = "tomato", linewidth = 1.2) +
  labs(title = "A. Histogram: Ozone (raw)",
       x = "Ozone (ppb)", y = "Density") +
  theme_minimal(base_size = 11)

p2 <- ggplot(ozone_df, aes(sample = ozone)) +
  stat_qq(color = "steelblue", size = 1.8) +
  stat_qq_line(color = "tomato", linewidth = 1) +
  labs(title = "B. QQ Plot: Ozone (raw)",
       x = "Theoretical Quantiles", y = "Sample Quantiles") +
  theme_minimal(base_size = 11)

p3 <- ggplot(ozone_df, aes(x = log_ozone)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20,
                 fill = "seagreen", color = "white", alpha = 0.8) +
  stat_function(fun = dnorm,
               args = list(mean = mean(log(ozone)), sd = sd(log(ozone))),
```

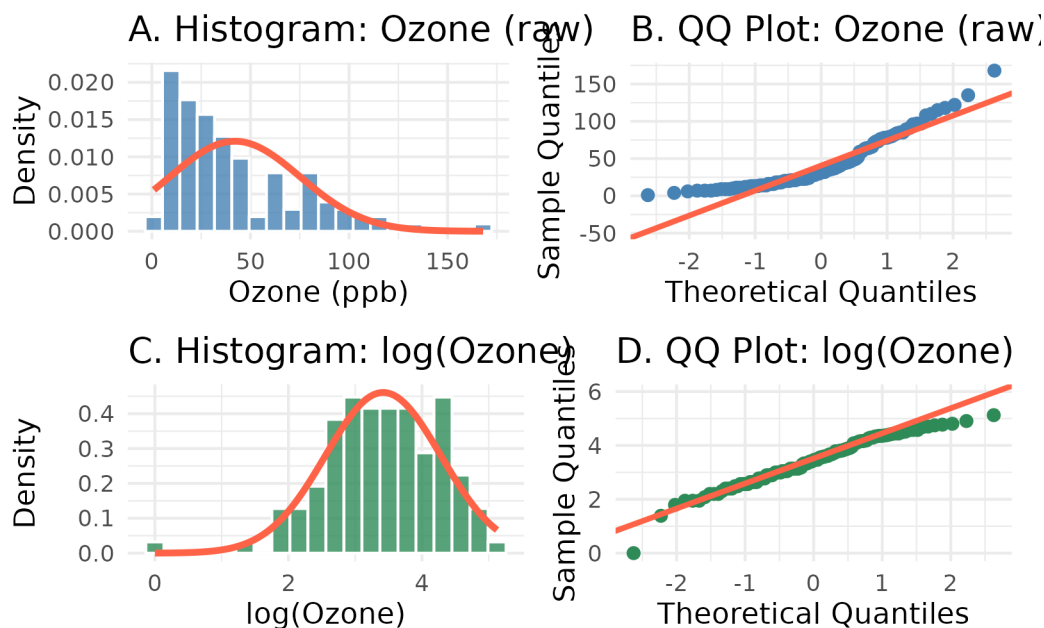
```

        color = "tomato", linewidth = 1.2) +
  labs(title = "C. Histogram: log(Ozone)",
        x = "log(Ozone)", y = "Density") +
  theme_minimal(base_size = 11)

p4 <- ggplot(ozone_df, aes(sample = log_ozone)) +
  stat_qq(color = "seagreen", size = 1.8) +
  stat_qq_line(color = "tomato", linewidth = 1) +
  labs(title = "D. QQ Plot: log(Ozone)",
        x = "Theoretical Quantiles", y = "Sample Quantiles") +
  theme_minimal(base_size = 11)

(p1 + p2) / (p3 + p4)

```



```

# --- Confirm transformation improves normality ---
sw_log <- shapiro.test(log(ozone))

cat("After log transformation:\n")

```

After log transformation:

```

cat("  Skewness:  ", round(moments::skewness(log(ozone)), 3), "\n")

```

Skewness: -0.555

```
cat("  Shapiro-Wilk W =", round(sw_log$statistic, 4),  
    "  p-value =", round(sw_log$p.value, 4), "\n")
```

Shapiro-Wilk W = 0.9717 p-value = 0.0147

```
cat("  Interpretation: p > 0.05 → fail to reject normality  \n")
```

Interpretation: p > 0.05 → fail to reject normality

Code explanation:

- `na.omit()` removes missing values before analysis — always handle NAs explicitly.
- `shapiro.test()` is built into base R; `nortest::ad.test()` requires the `nortest` package.
- The four-panel layout (raw vs. log-transformed, histogram vs. QQ) is the standard reporting format for normality assessment — it shows both the problem and the solution.
- `stat_function(fun = dnorm, args = list(...))` overlays the theoretical normal curve fitted to the data's mean and SD.

Try modifying: Apply the same normality assessment to the `wind` variable in `airquality`. Is it more or less normal than `ozone`?

Exercises

💡 Exercise 2.13

Using the `mtcars` dataset, assess the normality of the variables `mpg`, `hp`, and `wt`.

- For each variable, produce a histogram with normal overlay and a QQ plot.
- Conduct Shapiro-Wilk and Anderson-Darling tests for each variable.
- Based on your combined assessment, which variable(s) are approximately normal? Which are not?
- For any non-normal variable, apply an appropriate transformation and re-assess.

 Exercise 2.14 (Challenge)

Generate three samples of size $n = 20$, $n = 100$, and $n = 500$ from a t -distribution with 3 degrees of freedom.

- (a) Perform Shapiro-Wilk tests on all three samples. How do p -values change with n ?
 - (b) Produce QQ plots for all three. At which sample size does non-normality become visually obvious?
 - (c) What does this exercise illustrate about the relationship between sample size and the power of normality tests?
-

Chapter Lab Activity: Exploring Distributions with the airquality Dataset

Objectives

In this lab, you will apply the full spectrum of Chapter 2 concepts to the `airquality` dataset — daily air quality measurements in New York from May to September 1973. The goal is to characterize each variable's distribution, apply appropriate probability models, and assess normality — building the habit of distribution-first thinking before any analysis.

The Dataset

```
data(airquality)

kable(head(airquality),
      caption = "First 6 Rows of the airquality Dataset") |>
  kable_styling(bootstrap_options = c("striped", "hover"), font_size = 11)
```

Table 1: First 6 Rows of the airquality Dataset

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8.0	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
NA	NA	14.3	56	5	5
28	NA	14.9	66	5	6

```
cat("Dimensions:", nrow(airquality), "rows x", ncol(airquality), "columns\n")
```

Dimensions: 153 rows x 6 columns

```
cat("Missing values per variable:\n")
```

Missing values per variable:

```
print(colSums(is.na(airquality)))
```

Ozone	Solar.R	Wind	Temp	Month	Day
37	7	0	0	0	0

Variable descriptions:

Variable	Description	Scale	Type
Ozone	Ozone concentration (ppb)	Ratio	Continuous
Solar.R	Solar radiation (Langley)	Ratio	Continuous
Wind	Wind speed (mph)	Ratio	Continuous
Temp	Temperature (°F)	Interval	Continuous
Month	Month (5–9)	Ordinal	Discrete
Day	Day of month (1–31)	Ordinal	Discrete

Lab Task 1: Scale Identification and Missing Data

```
# Handle missing values and check variable types
airquality_clean <- airquality |>
  mutate(
    Month = factor(Month, levels = 5:9,
                   labels = c("May", "Jun", "Jul", "Aug", "Sep"),
                   ordered = TRUE),
    Day = as.integer(Day)
  )

# Summary of missing data
missing_summary <- data.frame(
  Variable = names(airquality),
  Type = c("Ratio", "Ratio", "Ratio", "Interval", "Ordinal", "Ordinal"),
  N_Missing = colSums(is.na(airquality)),
  Pct_Missing = round(colMeans(is.na(airquality)) * 100, 1)
)
rownames(missing_summary) <- NULL
```

```
kable(missing_summary,
      caption = "Variable Types and Missing Data Summary") |>
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE) |>
column_spec(4, bold = TRUE,
            color = ifelse(missing_summary$Pct_Missing > 0,
                          "tomato", "darkgreen"))
```

Table 2: Variable Types and Missing Data Summary

Variable	Type	N_Missing	Pct_Missing
Ozone	Ratio	37	24.2
Solar.R	Ratio	7	4.6
Wind	Ratio	0	0.0
Temp	Interval	0	0.0
Month	Ordinal	0	0.0
Day	Ordinal	0	0.0

Lab Task 2: Distribution Characterization

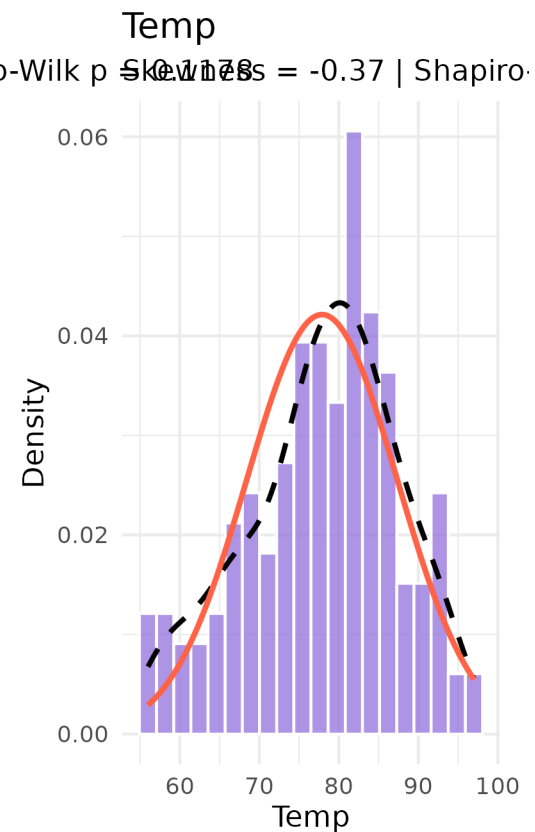
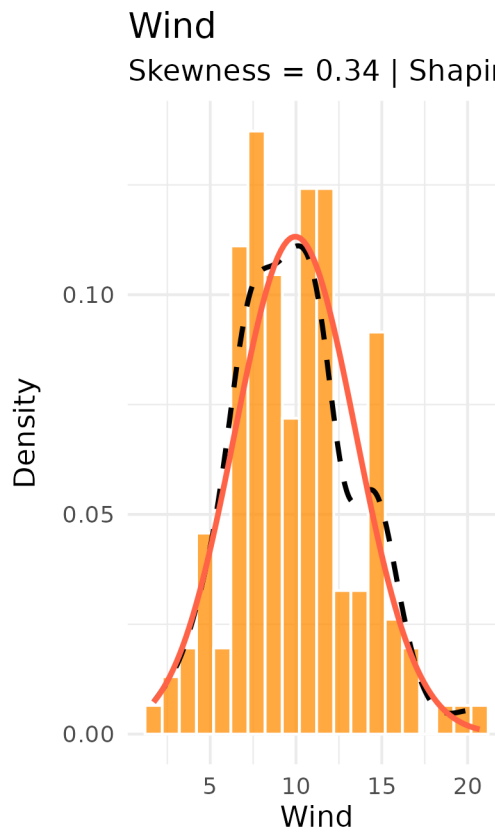
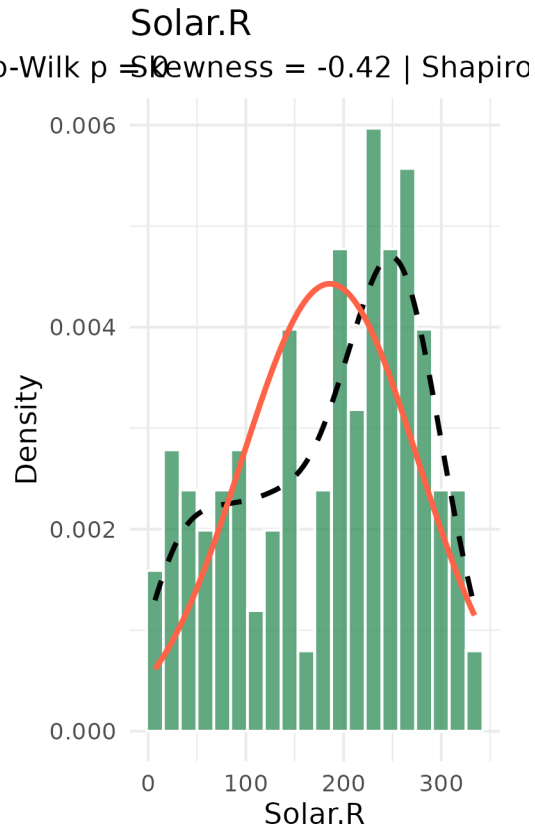
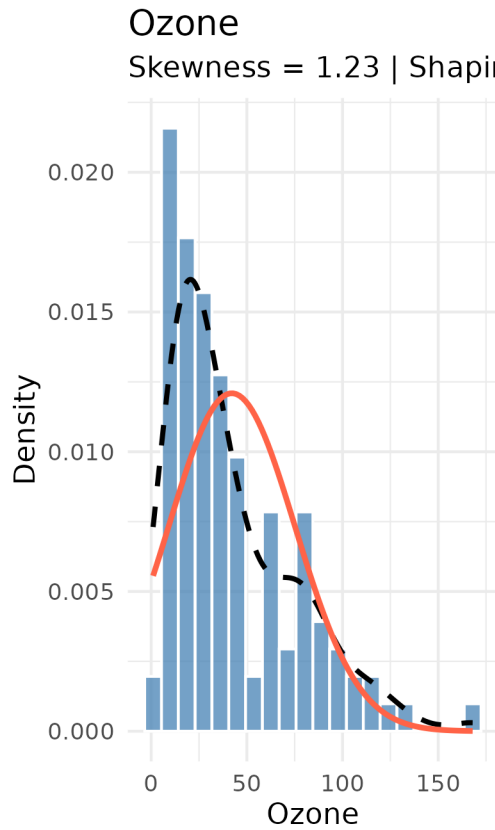
```
# Complete distributional profile for all four continuous variables
cont_vars <- c("Ozone", "Solar.R", "Wind", "Temp")
colors     <- c("steelblue", "seagreen", "darkorange", "mediumpurple")

plots <- map2(cont_vars, colors, function(var, col) {
  x <- na.omit(airquality[[var]])
  sw_p <- round(shapiro.test(x)$p.value, 4)
  sk    <- round(moments::skewness(x), 2)

  ggplot(data.frame(x = x), aes(x = x)) +
    geom_histogram(aes(y = after_stat(density)), bins = 20,
                  fill = col, color = "white", alpha = 0.75) +
    geom_density(color = "black", linewidth = 0.9, linetype = "dashed") +
    stat_function(fun = dnorm,
                  args = list(mean = mean(x), sd = sd(x)),
                  color = "tomato", linewidth = 1) +
    labs(title = var,
         subtitle = paste0("Skewness = ", sk,
                           " | Shapiro-Wilk p = ", sw_p),
         x = var, y = "Density") +
    theme_minimal(base_size = 11)
```

```
)
```

```
(plots[[1]] + plots[[2]]) / (plots[[3]] + plots[[4]])
```



Lab Task 3: Monthly Distributions and Probability Questions

```
# Temperature statistics by month
airquality_clean |>
  group_by(Month) |>
  summarise(
    n      = n(),
    Mean   = round(mean(Temp), 1),
    Median = round(median(Temp), 1),
    SD     = round(sd(Temp), 1),
    Min    = min(Temp),
    Max    = max(Temp)
  ) |>
  kable(caption = "Temperature (°F) Summary by Month") |>
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Table 3: Temperature (°F) Summary by Month

Month	n	Mean	Median	SD	Min	Max
May	31	65.5	66	6.9	56	81
Jun	30	79.1	78	6.6	65	93
Jul	31	83.9	84	4.3	73	92
Aug	31	84.0	82	6.6	72	97
Sep	30	76.9	76	8.4	63	93

```
# Using July temperature as approximately Normal: compute probabilities
july_temp <- na.omit(airquality$Temp[airquality$Month == 7])
mu_jul <- mean(july_temp)
sd_jul <- sd(july_temp)

cat("July Temperature: N( =", round(mu_jul,1),
    ",   =", round(sd_jul,1), ")\n")
```

July Temperature: N(= 83.9 , = 4.3)

```
cat("P(Temp > 90°F) in July: ",
    round(1 - pnorm(90, mu_jul, sd_jul), 4), "\n")
```

P(Temp > 90°F) in July: 0.0789

```
cat("P(Temp < 80°F) in July: ",
    round(pnorm(80, mu_jul, sd_jul), 4), "\n")
```

P(Temp < 80°F) in July: 0.1829

```
cat("90th percentile of July temp: ",
    round(qnorm(0.90, mu_jul, sd_jul), 1), "°F\n")
```

90th percentile of July temp: 89.4 °F

Lab Task 4: Normality Assessment and Transformations

```
# Systematic normality testing
ozone_clean <- na.omit(airquality$Ozone)

normality_results <- data.frame(
  Variable      = c("Ozone (raw)", "log(Ozone)", "Wind (raw)", "Temp (raw)"),
  Skewness      = round(c(
    moments::skewness(ozone_clean),
    moments::skewness(log(ozone_clean)),
    moments::skewness(na.omit(airquality$Wind)),
    moments::skewness(na.omit(airquality$Temp))
  ), 3),
  SW_statistic  = round(c(
    shapiro.test(ozone_clean)$statistic,
    shapiro.test(log(ozone_clean))$statistic,
    shapiro.test(na.omit(airquality$Wind))$statistic,
    shapiro.test(na.omit(airquality$Temp))$statistic
  ), 4),
  SW_pvalue     = round(c(
    shapiro.test(ozone_clean)$p.value,
    shapiro.test(log(ozone_clean))$p.value,
    shapiro.test(na.omit(airquality$Wind))$p.value,
    shapiro.test(na.omit(airquality$Temp))$p.value
  ), 4),
  Conclusion    = c("Non-normal ", "Approx. normal ",
                    "Check QQ plot", "Approx. normal ")
)

kable(normality_results,
```



```
caption = "Normality Assessment - airquality Variables",
col.names = c("Variable", "Skewness", "SW Statistic",
              "SW p-value", "Conclusion")) |>
kable_styling(bootstrap_options = c("striped", "hover")) |>
column_spec(5,
            color = ifelse(grepl(" ", normality_results$Conclusion),
                          "darkgreen", "tomato"),
            bold = TRUE)
```

Table 4: Normality Assessment — airquality Variables

Variable	Skewness	SW Statistic	SW p-value	Conclusion
Ozone (raw)	1.226	0.8787	0.0000	Non-normal
log(Ozone)	-0.555	0.9717	0.0147	Approx. normal
Wind (raw)	0.344	0.9857	0.1178	Check QQ plot
Temp (raw)	-0.374	0.9762	0.0093	Approx. normal

Lab Discussion Questions

Answer the following in writing:

1. **Measurement Scales:** The `Month` variable is stored as an integer in R. Explain why it should be converted to an ordered factor for analysis. What statistical operations become inappropriate if we treat it as a ratio-scale number?
2. **Missing Data:** `Ozone` has 37 missing values (24.2%). How might this affect your probability calculations? What assumptions are you implicitly making when you use `na.omit()`?
3. **Distribution Choice:** Based on Lab Task 2, which probability distribution (Normal, Poisson, Exponential, or other) would you use to model each of the four continuous variables? Justify each choice using skewness, shape, and domain knowledge.
4. **Bayes in Context:** Suppose that historically, 30% of summer days in New York have ozone levels considered “unhealthy” (above 100 ppb). A monitoring sensor flags a day as potentially unhealthy with 80% accuracy (sensitivity) and has a 10% false positive rate. Use Bayes’ Theorem to compute the probability that a flagged day is truly unhealthy.
5. **CLT Application:** The sample size for each month is small (roughly 30 days). Is the CLT applicable for computing probabilities about monthly mean temperatures? What assumptions are you relying on?

Chapter Summary

Summary

This chapter established the probabilistic foundation for statistical inference and data science:

- **Measurement scales** (nominal, ordinal, interval, ratio) determine what mathematical operations are permissible and what statistical methods are appropriate. Misidentifying scale leads to meaningless analyses.
- **Probability rules** (complement, addition, multiplication, total probability) provide the logical framework for computing uncertainty. Frequentist and Bayesian interpretations differ in meaning, not in rules.
- **Bayes' Theorem** updates prior beliefs with evidence to yield posterior probabilities — the foundation of diagnostic reasoning, spam filtering, and Bayesian machine learning.
- **Discrete distributions** (Binomial, Poisson) model count and binary outcome data, governed by probability mass functions.
- **Continuous distributions** (Normal, Uniform, Exponential) model measurable quantities via probability density functions and areas under curves.
- **The Central Limit Theorem** is the linchpin of inference: sample means from any distribution converge to normality as n grows, justifying the near-universal use of normal-based methods.
- **Normality assessment** combines graphical inspection (histogram, QQ plot) with formal tests (Shapiro-Wilk, Anderson-Darling). Transformations (log, square root) can correct non-normality.

! Key Formulas to Know

Conditional Probability:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Bayes' Theorem:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

Binomial PMF:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Poisson PMF:

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$$

Standard Error:

$$SE(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

Standardized Mean (CLT):

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0, 1)$$

References

- Anderson, E. (1935). The irises of the Gaspé Peninsula. *Bulletin of the American Iris Society*, 59, 2–5.
- Anderson, T. W., & Darling, D. A. (1954). A test of goodness of fit. *Journal of the American Statistical Association*, 49(268), 765–769.
- Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2), 211–243.
- Gross, J., & Ligges, U. (2015). *nortest: Tests for normality* (R package version 1.0-4). <https://CRAN.R-project.org/package=nortest>
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Kolmogorov, A. N. (1933). *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Springer.
- Komsta, L., & Novomestky, F. (2022). *moments: Moments, cumulants, skewness, kurtosis and related tests* (R package version 0.14.1). <https://CRAN.R-project.org/package=moments>
- New York State Department of Conservation. (1973). *Daily air quality measurements in New York, May–September 1973*.
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality. *Biometrika*, 52(3/4), 591–611.
- Stevens, S. S. (1946). On the theory of scales of measurement. *Science*, 103(2684), 677–680.

Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.

Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

Yeo, I. K., & Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), 954–959.

End of Chapter 2. Proceed to Chapter 3: Hypothesis Testing.

Chapter 3: Hypothesis Testing

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(moments)      # skewness
library(coin)         # exact non-parametric tests
library(pwr)          # power analysis
library(effectsize)   # Cohen's d and effect size measures
library(rstatix)      # tidy statistical tests
```

Chapter Overview

Introduction

Hypothesis testing is the formal procedure for using sample data to make decisions about population parameters. It is the engine behind scientific claims: *Does this drug reduce blood pressure? Is this algorithm more accurate than the baseline? Do students in different programs differ in exam performance?* Every such question can be framed as a hypothesis test.

This chapter covers:

- **The Logic of Hypothesis Testing** — the conceptual framework, errors, and p-values
- **One-Sample Tests** — testing a single mean against a known value
- **Two-Sample Tests** — comparing means between two independent groups
- **Paired Sample Test** — comparing means in matched or repeated-measures designs
- **One-Way ANOVA** — comparing means across three or more groups
- **Non-Parametric Alternatives** — robust tests when normality cannot be assumed
- **Effect Size and Statistical Power** — measuring practical significance and planning sample sizes

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain the logic of hypothesis testing including Type I and Type II errors.
2. Select and apply the correct test for one-sample, two-sample, and paired designs.
3. Conduct one-way ANOVA and interpret the F-statistic and post-hoc tests.
4. Choose appropriate non-parametric alternatives when parametric assumptions are violated.
5. Compute and interpret effect sizes and conduct power analyses for sample size planning.
6. Report hypothesis test results in a format appropriate for academic research.

The Logic of Hypothesis Testing

Introduction

Before computing a single test statistic, it is essential to understand *why* hypothesis testing works the way it does. The procedure is built on a deceptively simple idea: assume the null hypothesis is true, then ask how surprising the observed data would be under that assumption. If the data are sufficiently unlikely under the null, we reject it. This section establishes the conceptual framework that underlies every test in this chapter and beyond.

Theory

The Null and Alternative Hypotheses

Every hypothesis test begins with two competing claims:

- **Null Hypothesis (H_0):** The “status quo” — a statement of no effect, no difference, or equality. It is the hypothesis we seek evidence *against*.
- **Alternative Hypothesis (H_1 or H_a):** The claim we seek evidence *for*. It represents the presence of an effect, difference, or relationship.

Hypotheses are always stated about **population parameters**, never about sample statistics.

Directionality of H_1 :

Table 1: Types of hypothesis tests

Test Type	H_0	H_1	Rejection Region
Two-tailed	$\mu = \mu_0$	$\mu \neq \mu_0$	Both tails
Left-tailed	$\mu \geq \mu_0$	$\mu < \mu_0$	Left tail
Right-tailed	$\mu \leq \mu_0$	$\mu > \mu_0$	Right tail

The p-value

The **p-value** is the probability of observing a test statistic as extreme as (or more extreme than) the one computed from the sample, **assuming** H_0 is true.

$$p\text{-value} = P(\text{test statistic as extreme as observed} \mid H_0 \text{ is true})$$

A small p-value means the observed data are unlikely under H_0 — providing evidence against it. A large p-value means the data are consistent with H_0 .

⚠ What the p-value is NOT

- It is **not** $P(H_0 \text{ is true})$
- It is **not** the probability that the result occurred by chance
- It is **not** a measure of effect size or practical importance
- A statistically significant result is not necessarily practically important

The Significance Level α

The **significance level** α is the threshold below which we reject H_0 . It represents the maximum acceptable probability of making a Type I error. Common choices:

- $\alpha = 0.05$ — standard in most social and biological sciences
- $\alpha = 0.01$ — stricter; used when false positives are costly
- $\alpha = 0.001$ — very strict; used in genomics, particle physics

Decision rule: Reject H_0 if $p\text{-value} < \alpha$.

Type I and Type II Errors

No decision rule is perfect. Two types of errors are possible:

Table 2: Decision outcomes in hypothesis testing

	H_0 is True	H_0 is False
Reject H_0	Type I Error (α)	Correct (Power = $1 - \beta$)
Fail to Reject H_0	Correct	Type II Error (β)

- **Type I Error (False Positive):** Rejecting H_0 when it is actually true. Probability = α , controlled by the researcher.

- **Type II Error (False Negative):** Failing to reject H_0 when H_1 is true. Probability = β .
- **Power** = $1 - \beta$: the probability of correctly detecting a true effect.

There is an inherent tradeoff: decreasing α (stricter threshold) reduces Type I errors but increases β (more Type II errors). The only way to reduce both simultaneously is to **increase the sample size**.

The General Procedure

Every hypothesis test follows the same five-step procedure:

1. **State** H_0 and H_1 clearly, including directionality.
2. **Choose** the significance level α and the appropriate test.
3. **Compute** the test statistic from sample data.
4. **Find** the p-value (or compare test statistic to critical value).
5. **Decide** and **interpret** in context: reject or fail to reject H_0 , and state the practical meaning.

Quantiles and Critical Values

Step 4 above mentions comparing the test statistic to a **critical value** — we define that here, since we'll use it in every example going forward.

Every distribution has a CDF (`pnorm()`, `pt()`, ...) that goes from a value to a cumulative probability. It also has an inverse — the **quantile function** (`qnorm()`, `qt()`, ...) — that goes the other way: from a cumulative probability back to a value.

If $P(X \leq x_p) = p$, then x_p is the p -th quantile of X

Distribution	CDF (area to the left)	Quantile (inverse CDF)
Standard Normal	<code>pnorm(x)</code>	<code>qnorm(p)</code>
$t(df)$	<code>pt(x, df)</code>	<code>qt(p, df)</code>

Notation. We write the quantile using a subscript equal to the cumulative probability: z_p for the standard normal, $t_{p,df}$ for the t -distribution. **The subscript is always the area to the left** — matching the `p` argument you pass into `qnorm()` / `qt()` directly. A **critical value** is simply a quantile chosen so that the area beyond it equals α (or $\alpha/2$ per tail, for a two-tailed test).

 Convention check

Some textbooks write $t_{\alpha,df}$ for the area in the **right** tail — the opposite of the convention here. Always check the definition, not just the symbol. **In this book, the subscript is the left-tail cumulative probability.**

```
qnorm(0.975)      # z_0.975  = 1.959964
```

```
[1] 1.959964
```

```
qnorm(0.95)       # z_0.95   = 1.644854
```

```
[1] 1.644854
```

```
qt(0.95, df = 19) # t_0.95,19 = 1.729133
```

```
[1] 1.729133
```

$z_{0.975} \approx 1.96$ is the two-tailed 95% critical value (2.5% in each tail). $z_{0.95} \approx 1.645$ is the right-tailed 95% critical value (5% in the upper tail). Both reappear throughout this chapter.

 Calculated statistic vs. critical value

- **Calculated statistic** (t_{cal} , or z_{cal}): computed **from your sample** ($\bar{x}$, s , n). Changes with every new sample.
- **Critical value** (t_{crit} , e.g. $t_{0.95,19}$): determined **only by α and df** , via `qt()` / `qnorm()`. Does not depend on your data.

Every test in this chapter compares these two. If unsure which you're looking at, ask: “did this come from the data, or from `qt()`?”

A third notation you'll see, so all three don't blur together: T_{df} (capital T) denotes the **random variable** itself — “ T_{19} follows a t -distribution with 19 degrees of freedom” — used when *stating* a p-value, e.g. $P(T_{19} < -1.961)$. It is neither the quantile $t_{p,df}$ nor the calculated statistic t_{cal} ; it's the distribution the other two live on. When we *report* a result, we'll write $t_{\text{cal}} = \dots$ ($df = \dots$), matching the software convention `t(df) = value` you'll see in R output and in papers — never a bare t_{df} , since that subscript position is already reserved for quantiles.

Confidence Intervals and Hypothesis Tests

Confidence intervals and hypothesis tests are two sides of the same coin. A 95% confidence interval for μ contains all values of μ_0 for which the two-tailed test at $\alpha = 0.05$ would **fail to reject** $H_0 : \mu = \mu_0$. When a CI excludes the null value, the corresponding test rejects H_0 . CIs are often more informative than tests alone because they quantify the magnitude of the effect, not just its significance.

Example: The Logic of Hypothesis Testing

Example 3.1. A university claims that the average time for students to complete a master's degree is 2 years (24 months). A researcher suspects it takes longer and surveys 36 recent graduates, finding $\bar{x} = 26.5$ months with $s = 7.2$ months.

Step 1 — Hypotheses:

$$H_0 : \mu = 24 \quad H_1 : \mu > 24 \quad (\text{right-tailed})$$

Step 2 — Significance level: $\alpha = 0.05$

Step 3 — Test statistic (t-test, σ unknown):

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} = \frac{26.5 - 24}{7.2/\sqrt{36}} = \frac{2.5}{1.2} = 2.083$$

Step 4 — p-value (right-tailed, $df = 35$):

$$p\text{-value} = P(T_{35} > 2.083) \approx 0.022$$

Step 5 — Decision. Two equivalent ways to decide:

Method 1 — p-value. Compare the p-value to α :

$$p = 0.022 < \alpha = 0.05 \quad \Rightarrow \quad \text{Reject } H_0$$

Method 2 — critical value. This is a right-tailed test, so the rejection region is the upper 5% of $t(35)$:

$$t_{\text{crit}} = t_{0.95, 35} = \text{qt}(0.95, 35) = 1.690$$

Compare the sample statistic $t_{\text{cal}} = 2.083$ to this threshold:

$$\begin{aligned} &\text{Reject } H_0 \text{ if } t_{\text{cal}} > t_{\text{crit}} \\ &2.083 > 1.690 \quad \Rightarrow \quad \text{Reject } H_0 \end{aligned}$$

Both methods agree — they must, since $p = P(T_{35} > t_{\text{cal}})$ and t_{crit} is defined as the value with exactly α beyond it, so “ t_{cal} past t_{crit} ” and “ $p < \alpha$ ” are the same statement viewed two ways.

Interpretation: There is statistically significant evidence at the 5% level that the true average completion time exceeds 24 months ($t_{\text{cal}} = 2.08$, $df = 35$, $p = 0.022$ — reported in papers and R output as $t(35) = 2.08$). The sample mean of 26.5 months suggests students take approximately 2.5 months longer than the claimed average.

R Example: Visualizing the p-value

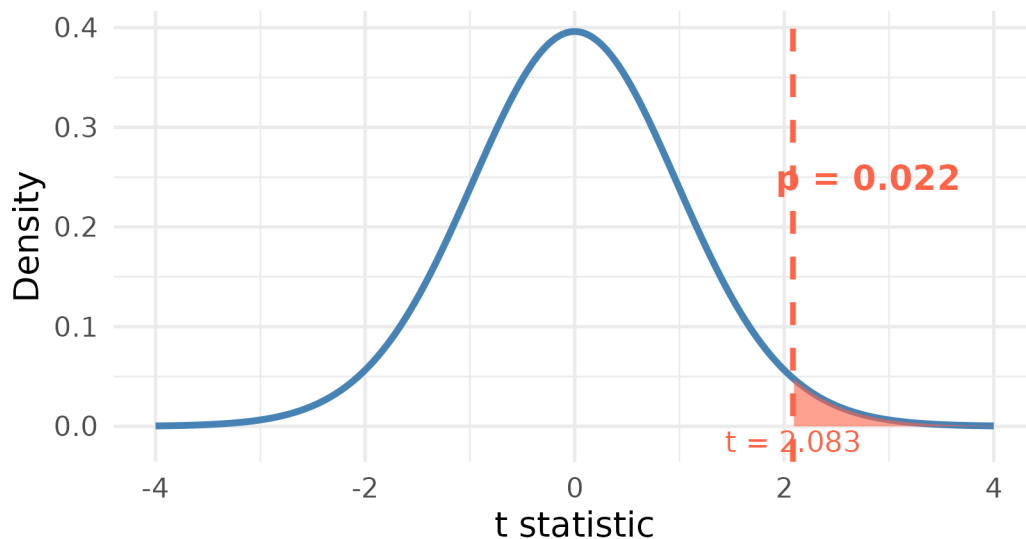
```
# --- Visualize the p-value for Example 3.1 ---
df_val  <- 35
t_obs   <- 2.083
p_val   <- pt(t_obs, df = df_val, lower.tail = FALSE)

x_seq <- seq(-4, 4, length.out = 500)
t_df  <- data.frame(x = x_seq, y = dt(x_seq, df = df_val))

ggplot(t_df, aes(x, y)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_area(data = subset(t_df, x >= t_obs),
            fill = "tomato", alpha = 0.6) +
  geom_vline(xintercept = t_obs, color = "tomato",
             linetype = "dashed", linewidth = 1) +
  annotate("text", x = 2.8, y = 0.25,
           label = paste0("p = ", round(p_val, 3)),
           color = "tomato", fontface = "bold", size = 4.5) +
  annotate("text", x = 2.083, y = -0.015,
           label = paste0("t = ", t_obs),
           color = "tomato", size = 3.8, hjust = 0.5) +
  labs(title = "t-Distribution (df = 35): Right-Tailed Test",
       subtitle = "Shaded area = p-value = P(T > 2.083 | H true)",
       x = "t statistic", y = "Density") +
  theme_minimal(base_size = 13)
```

t-Distribution (df = 35): Right-Tailed Test

Shaded area = p-value = $P(T > 2.083 \mid H_0 \text{ true})$



Code explanation:

- `dt()` computes the t-distribution density (PDF); `pt()` computes the CDF (cumulative probability).
- `pt(t_obs, df, lower.tail = FALSE)` gives the right-tail probability — the p-value for a right-tailed test.
- The shaded area visually represents the p-value: the probability of observing $t \geq 2.083$ if H_0 were true.

Exercises

💡 Exercise 3.1

For each scenario below, state H_0 and H_1 , identify the test direction (two-tailed, left-, or right-tailed), and justify your choice:

- A pharmaceutical company claims its new drug reduces systolic blood pressure by at least 10 mmHg. A regulator wants to check if the reduction is less than claimed.
- A data scientist wants to know whether a new recommendation algorithm changes average session duration compared to the old algorithm (could be longer or shorter).
- An educator believes that students who attend extra tutorials score higher than those who do not.

 Exercise 3.2

Explain in your own words why “failing to reject H_0 ” is not the same as “proving H_0 is true.” Use an analogy from everyday life to illustrate the difference.

One-Sample Tests

Introduction

The **one-sample test** addresses the simplest inferential question: does the population mean equal a specific hypothesized value μ_0 ? It is the natural starting point for learning hypothesis testing because it involves a single group and a known reference value. In data science, one-sample tests arise when benchmarking a system against a known standard, verifying a manufacturer's claim, or checking whether a model's accuracy exceeds chance.

Theory

One-Sample z-Test

Used when the **population standard deviation** σ is known (rare in practice) or the sample is very large ($n > 100$):

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} \sim N(0, 1) \text{ under } H_0$$

Critical values: $z^* = \pm 1.96$ (two-tailed, $\alpha = 0.05$), $z^* = 1.645$ (right-tailed, $\alpha = 0.05$).

One-Sample t-Test

Used when σ is **unknown** (estimated by s) — the typical case:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \sim t(n - 1) \text{ under } H_0$$

Assumptions:

1. The sample is a random sample from the population.
2. The variable is continuous (interval or ratio scale).
3. The population is approximately normally distributed **or** $n \geq 30$ (CLT applies).

Confidence Interval Approach

The $(1 - \alpha) \times 100\%$ confidence interval for μ is:

$$\bar{x} \pm t_{n-1}^* \cdot \frac{s}{\sqrt{n}}$$

where t_{n-1}^* is the critical value from the t-distribution with $n - 1$ degrees of freedom.

If μ_0 falls **outside** this interval, reject H_0 at significance level α .

Example: One-Sample t-Test

Example 3.2. A coffee machine is calibrated to dispense 250 ml per cup. A quality control inspector measures 20 cups and finds $\bar{x} = 246.8$ ml, $s = 7.3$ ml. Is there evidence the machine is under-dispensing?

Hypotheses:

$$H_0 : \mu = 250 \quad H_1 : \mu < 250 \quad (\text{left-tailed})$$

Test statistic:

$$t = \frac{246.8 - 250}{7.3/\sqrt{20}} = \frac{-3.2}{1.632} = -1.961$$

p-value (left-tailed, $df = 19$): $P(T_{19} < -1.961) = 0.032$

Decision — two equivalent methods.

Method 1 — p-value.

$$p = 0.032 < \alpha = 0.05 \quad \Rightarrow \quad \text{Reject } H_0$$

Method 2 — critical value. Left-tailed test, $\alpha = 0.05$, $df = 19$:

$$t_{\text{crit}} = -t_{0.95, 19} = -\text{qt}(0.95, 19) = -1.729$$

Compare to the sample statistic $t_{\text{cal}} = -1.961$:

$$\begin{aligned} &\text{Reject } H_0 \text{ if } t_{\text{cal}} < t_{\text{crit}} \\ -1.961 < -1.729 &\Rightarrow \text{Reject } H_0 \end{aligned}$$

Both methods agree: the p-value is the area beyond t_{cal} , and t_{crit} marks off area exactly α .

Interpretation: There is significant evidence that the machine dispenses less than 250 ml on average ($t_{\text{cal}} = -1.96$, $df = 19$, $p = 0.032$ — reported as $t(19) = -1.96$). The one-sided 95% upper confidence bound for the true mean is 249.6 ml — below the claimed 250 ml, consistent with rejecting H_0 in this left-tailed test.

R Example: One-Sample Tests

```
# --- Simulate the coffee machine data ---
set.seed(101)
cups <- c(245.2, 236.0, 254.2, 248.6, 239.8, 257.3, 243.2,
          247.9, 255.9, 237.1, 250.7, 243.5, 238.0, 259.8,
          248.8, 242.1, 253.5, 236.2, 246.0, 252.3)

# One-sample t-test (left-tailed: H1: mu < 250)
t_result <- t.test(cups, mu = 250, alternative = "less")
print(t_result)
```

One Sample t-test

```
data: cups
t = -1.9556, df = 19, p-value = 0.03269
alternative hypothesis: true mean is less than 250
95 percent confidence interval:
 -Inf 249.63
sample estimates:
mean of x
 246.805
```

```
# --- Clean summary table ---
data.frame(
  Statistic = c("Sample Mean", "Sample SD", "n",
               "t statistic", "df", "p-value (left-tailed)",
               "95% CI Lower", "95% CI Upper"),
  Value = c(round(mean(cups), 2), round(sd(cups), 2), length(cups),
            round(t_result$statistic, 3), t_result$parameter,
            round(t_result$p.value, 4),
            round(t_result$conf.int[1], 2),
            round(t_result$conf.int[2], 2))
) |>
kable(caption = "One-Sample t-Test Results: Coffee Machine",
      col.names = c("Statistic", "Value")) |>
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Table 1: One-Sample t-Test Results: Coffee Machine

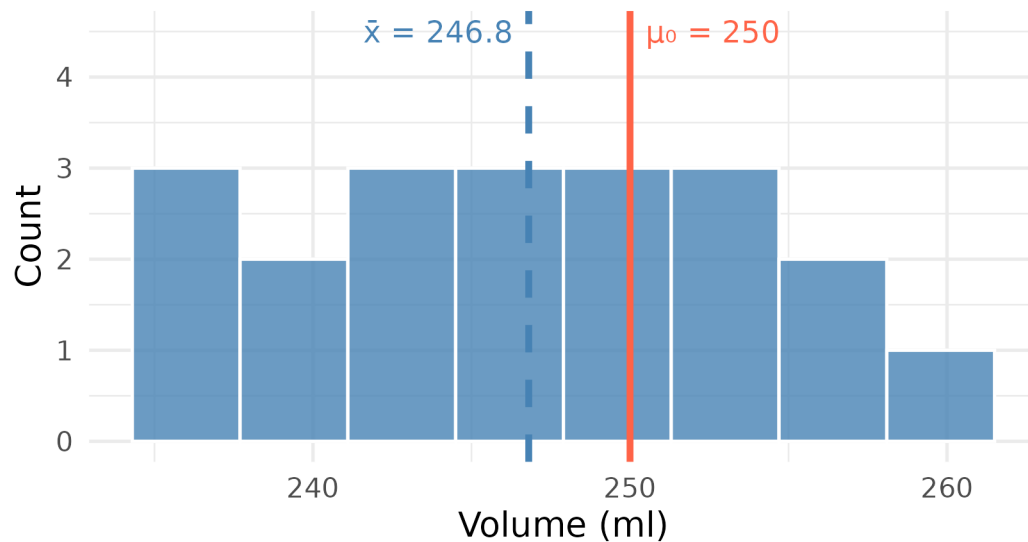
Statistic	Value
-----------	-------

Sample Mean	246.8000
Sample SD	7.3100
n	20.0000
t statistic	-1.9560
df	19.0000
p-value (left-tailed)	0.0327
95% CI Lower	-Inf
95% CI Upper	249.6300

```
# --- Visualize data with reference line ---
ggplot(data.frame(ml = cups), aes(x = ml)) +
  geom_histogram(bins = 8, fill = "steelblue",
                 color = "white", alpha = 0.8) +
  geom_vline(xintercept = mean(cups), color = "steelblue",
             linewidth = 1.2, linetype = "dashed") +
  geom_vline(xintercept = 250, color = "tomato",
             linewidth = 1.2, linetype = "solid") +
  annotate("text", x = mean(cups) - 0.5, y = 4.5,
          label = paste0("x̄ = ", round(mean(cups),1)),
          color = "steelblue", hjust = 1, size = 4) +
  annotate("text", x = 250.5, y = 4.5,
          label = " = 250", color = "tomato",
          hjust = 0, size = 4) +
  labs(title = "Coffee Machine Dispensing Volume",
       subtitle = "Sample mean (blue) vs. claimed mean (red)",
       x = "Volume (ml)", y = "Count") +
  theme_minimal(base_size = 13)
```

Coffee Machine Dispensing Volume

Sample mean (blue) vs. claimed mean (red)



Code explanation:

- `t.test(x, mu, alternative)` performs the one-sample t-test. `alternative` can be "less", "greater", or "two.sided".
- The result object contains `$statistic` (t value), `$parameter` (df), `$p.value`, and `$conf.int`.
- The confidence interval returned by `t.test()` with `alternative = "less"` gives a one-sided upper bound — for a two-sided CI, use `alternative = "two.sided"`.

Exercises

💡 Exercise 3.3

A manufacturer claims their batteries last 500 hours on average. You test 25 batteries and find $\bar{x} = 487$ hours, $s = 42$ hours.

- State H_0 and H_1 for a two-tailed test.
- Compute the t-statistic and p-value by hand. Verify in R.
- Construct a 95% confidence interval for the true mean battery life.
- Based on both the test and the CI, what do you conclude?

Exercise 3.4

Using the `sleep` dataset built into R (extra sleep hours gained with two drugs):

- (a) Test whether the mean extra sleep for group 1 (`group == 1`) differs from zero (two-tailed, $\alpha = 0.05$).
 - (b) Test whether the mean extra sleep for group 2 (`group == 2`) is greater than zero (right-tailed).
 - (c) Interpret both results and compare conclusions.
-

Two-Sample Tests

Introduction

In many research and data science contexts, the question is not whether a population mean equals a fixed value, but whether **two populations have the same mean**. Does the treatment group differ from the control? Do users of version A spend more time on the platform than users of version B? The **two-sample t-test** is the standard tool for comparing means from two independent groups, and it underlies A/B testing, clinical trials, and comparative studies across all fields.

Theory

Independent Samples t-Test (Equal Variances)

When we assume the two populations have equal variances ($\sigma_1^2 = \sigma_2^2$), we pool the variance estimates:

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$$

$$t = \frac{(\bar{x}_1 - \bar{x}_2) - \delta_0}{s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}} \sim t(n_1 + n_2 - 2) \text{ under } H_0$$

where δ_0 is the hypothesized difference (usually 0).

Welch's t-Test (Unequal Variances)

The equal-variance assumption is often untenable. **Welch's t-test** does not assume equal variances and uses a modified degrees of freedom (Satterthwaite approximation):

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

$$df \approx \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1-1} + \frac{(s_2^2/n_2)^2}{n_2-1}}$$

Recommendation: Welch's t-test is the default in R (`var.equal = FALSE`) and is preferred in practice — it performs well even when variances are equal, making it a safer choice.

Testing for Equal Variances: Levene's Test

Before choosing between pooled and Welch's tests, some analysts check whether variances are equal using **Levene's test** (H_0 : equal variances). However, many statisticians argue this pre-testing approach inflates overall Type I error and recommend simply always using Welch's test.

Assumptions for both tests:

1. Independent random samples from each population.
2. Continuous variable (interval or ratio scale).
3. Approximately normal populations **or** $n_1, n_2 \geq 30$.

Example: Two-Sample t-Test

Example 3.3. A researcher compares exam scores between two teaching methods. Group A (traditional): $n_1 = 30$, $\bar{x}_1 = 74.2$, $s_1 = 11.8$. Group B (flipped classroom): $n_2 = 28$, $\bar{x}_2 = 81.6$, $s_2 = 9.4$.

Hypotheses:

$$H_0 : \mu_A = \mu_B \quad H_1 : \mu_A \neq \mu_B \quad (\text{two-tailed})$$

Welch's t-statistic:

$$t = \frac{74.2 - 81.6}{\sqrt{\frac{11.8^2}{30} + \frac{9.4^2}{28}}} = \frac{-7.4}{\sqrt{4.643 + 3.153}} = \frac{-7.4}{2.792} = -2.651$$

df (Satterthwaite) ≈ 55.6 , **p-value** ≈ 0.010

Decision — two equivalent methods.

Method 1 — p-value.

$$p = 0.010 < \alpha = 0.05 \quad \Rightarrow \quad \text{Reject } H_0$$

Method 2 — critical value. This is a **two-tailed** test, so the rejection region is split into both tails: $\alpha/2 = 0.025$ in each. Unlike the one-tailed examples so far, this gives us **two** critical values, symmetric around zero:

$$t_{\text{crit}} = \pm t_{0.975, 55.6} = \pm \text{qt}(0.975, 55.6) = \pm 2.004$$

Compare the sample statistic $t_{\text{cal}} = -2.651$ — for a two-tailed test we compare its magnitude:

$$\begin{aligned} &\text{Reject } H_0 \text{ if } |t_{\text{cal}}| > t_{\text{crit}} \\ &|-2.651| = 2.651 > 2.004 \quad \Rightarrow \quad \mathbf{Reject } H_0 \end{aligned}$$

Both methods agree. Note the difference from Examples 3.1 and 3.2: a one-tailed test compares t_{cal} to a single critical value in one direction, while a two-tailed test compares $|t_{\text{cal}}|$ to a single critical value that represents *two* symmetric boundaries.

Interpretation: Students in the flipped classroom scored significantly higher than those in the traditional classroom ($t_{\text{cal}} = -2.65$, $df = 55.6$, $p = 0.010$ — reported as $t(55.6) = -2.65$). The mean difference of 7.4 points (95% CI: 1.7 to 13.1) is both statistically significant and potentially educationally meaningful.

R Example: Two-Sample Tests

```
# --- Simulate teaching method data ---
set.seed(202)
group_A <- rnorm(30, mean = 74.2, sd = 11.8)
group_B <- rnorm(28, mean = 81.6, sd = 9.4)

# Welch's t-test (default in R: var.equal = FALSE)
welch_result <- t.test(group_A, group_B,
                       alternative = "two.sided",
                       var.equal = FALSE)

# Pooled t-test for comparison
pooled_result <- t.test(group_A, group_B,
                       alternative = "two.sided",
                       var.equal = TRUE)

# Compare results
comparison <- data.frame(
  Test = c("Welch's t-test", "Pooled t-test"),
  t_statistic = round(c(welch_result$statistic,
                        pooled_result$statistic), 3),
```

```

df      = round(c(welch_result$parameter,
                  pooled_result$parameter), 1),
p_value = round(c(welch_result$p.value,
                  pooled_result$p.value), 4),
CI_lower = round(c(welch_result$conf.int[1],
                  pooled_result$conf.int[1]), 2),
CI_upper = round(c(welch_result$conf.int[2],
                  pooled_result$conf.int[2]), 2)
)

kable(comparison,
      caption = "Welch's vs. Pooled t-Test Results",
      col.names = c("Test", "t", "df", "p-value",
                    "95% CI Lower", "95% CI Upper")) |>
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Table 1: Welch's vs. Pooled t-Test Results

Test	t	df	p-value	95% CI Lower	95% CI Upper
Welch's t-test	-2.536	54.6	0.0141	-13.09	-1.53
Pooled t-test	-2.516	56.0	0.0148	-13.13	-1.49

```

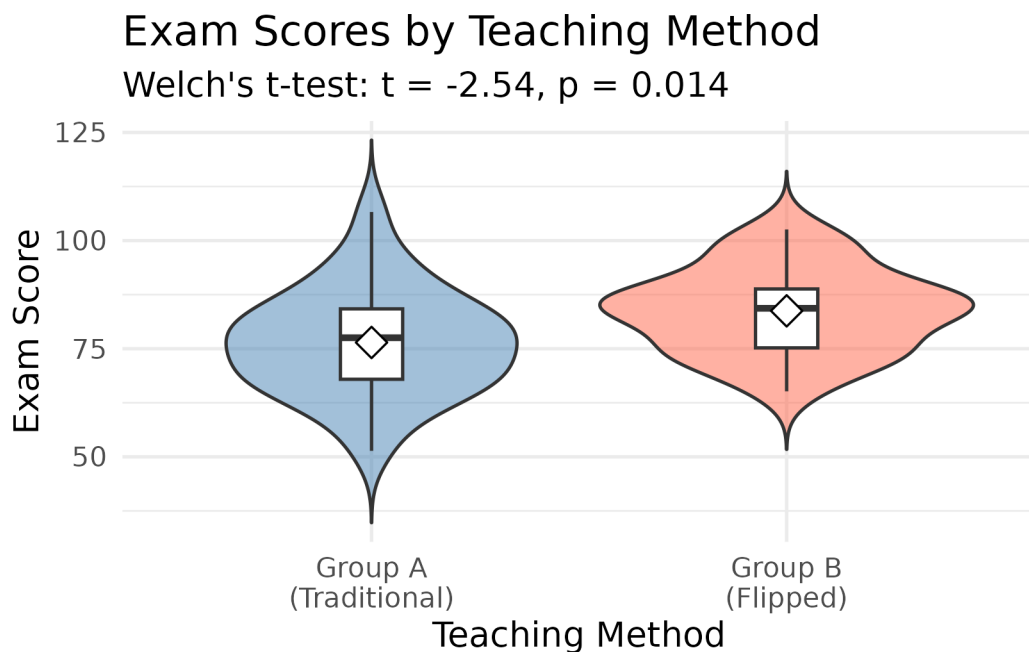
# --- Visualize group comparison ---
scores_df <- data.frame(
  score = c(group_A, group_B),
  group = rep(c("Group A\n(Traditional)", "Group B\n(Flipped)"),
             times = c(30, 28))
)

ggplot(scores_df, aes(x = group, y = score, fill = group)) +
  geom_violin(alpha = 0.5, trim = FALSE) +
  geom_boxplot(width = 0.15, fill = "white",
              outlier.shape = 21, outlier.size = 2) +
  stat_summary(fun = mean, geom = "point",
              shape = 23, size = 4, fill = "white") +
  scale_fill_manual(values = c("steelblue", "tomato")) +
  labs(title = "Exam Scores by Teaching Method",
       subtitle = paste0("Welch's t-test: t = ",
                        round(welch_result$statistic, 2),
                        ", p = ", round(welch_result$p.value, 3)),
       x = "Teaching Method",
       y = "Exam Score") +

```



```
theme_minimal(base_size = 13) +
theme(legend.position = "none")
```



Code explanation:

- `t.test(x, y, alternative, var.equal)` handles both Welch's (`var.equal = FALSE`) and pooled (`var.equal = TRUE`) variants.
- `stat_summary(fun = mean, geom = "point", shape = 23)` adds a diamond-shaped mean marker inside the boxplot — a clean way to show both median (boxplot line) and mean simultaneously.
- The violin plot reveals distributional shape alongside the five-number summary, providing richer information than a boxplot alone.

Exercises

💡 Exercise 3.5

Using the `mtcars` dataset, test whether automatic and manual transmission cars have different fuel efficiency (`mpg`).

- Check normality for each group using Shapiro-Wilk and QQ plots.
- Run Welch's t-test. State hypotheses, report t , df , p -value, and 95% CI.
- Run the pooled t-test. Compare results to Welch's.

- (d) Which test is more appropriate here? Justify using Levene's test (`var.test()` in R).

💡 Exercise 3.6

A/B testing scenario: Two versions of a webpage are shown to random users. Version A: $n = 200$, mean session = 4.2 min, $s = 1.8$ min. Version B: $n = 200$, mean session = 4.7 min, $s = 2.1$ min.

- (a) Conduct a two-tailed Welch's t-test at $\alpha = 0.05$. What do you conclude?
 - (b) Compute the 95% CI for the difference in means.
 - (c) Even if significant, should the company switch to Version B? What other factors matter?
-

Paired Sample Test

Introduction

The two-sample t-test assumes **independent** groups — the two samples are drawn from completely separate populations. But many research designs deliberately **match** or **pair** observations: measuring the same subject before and after a treatment, comparing twins, or matching patients by age and severity. In these designs, ignoring the pairing wastes information and inflates variability. The **paired t-test** exploits the within-pair relationship, making it more powerful than the independent samples test when pairing is effective.

Theory

The Paired t-Test

Given n pairs (x_{1i}, x_{2i}) , compute the difference for each pair:

$$d_i = x_{1i} - x_{2i}, \quad i = 1, 2, \dots, n$$

The paired t-test is simply a **one-sample t-test on the differences** d_i , testing whether the mean difference μ_d equals zero:

$$H_0 : \mu_d = 0 \quad H_1 : \mu_d \neq 0 \quad (\text{or one-sided})$$

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \sim t(n-1) \text{ under } H_0$$

where $\bar{d}$ is the sample mean of differences and s_d is the standard deviation of differences.

Key insight: By working with differences, we eliminate **between-subject variability** — a major source of noise in independent samples designs. The paired test is more powerful when subjects are heterogeneous but respond similarly to treatment.

When to use paired vs. independent:

Table 1: Choosing between paired and independent t-tests

Use Paired t-Test	Use Independent t-Test
Same subject measured twice	Two separate groups
Matched pairs (twins, siblings)	Random assignment to groups
Before-after design	Cross-sectional comparison

Assumptions:

1. Pairs are randomly and independently selected.
2. Differences d_i are approximately normally distributed (not the raw data).
3. The variable is continuous.

Example: Paired t-Test

Example 3.4. Ten patients' systolic blood pressure (mmHg) is measured before and after 8 weeks of a new medication:

Patient	Before	After	Difference (d_i)
1	145	132	13
2	158	148	10
3	162	151	11
4	149	140	9
5	155	143	12
6	170	159	11
7	152	147	5
8	141	136	5
9	163	150	13
10	157	148	9

$$\bar{d} = 9.8, s_d = 2.90, n = 10$$

Hypotheses: $H_0 : \mu_d = 0$ vs. $H_1 : \mu_d > 0$ (right-tailed — testing for reduction)

Test statistic:

$$t = \frac{9.8}{2.90/\sqrt{10}} = \frac{9.8}{0.917} = 10.69$$

p-value (right-tailed, $df = 9$): $P(T_9 > 10.69) < 0.0001$

Decision: Reject H_0 with overwhelming evidence.

Interpretation: The medication produced a highly significant reduction in systolic blood pressure ($t_{\text{cal}} = 10.69$, $df = 9$, $p < 0.0001$ — reported as $t(9) = 10.69$). The mean

reduction of 9.8 mmHg (95% CI: 7.73 to 11.87 mmHg) is clinically meaningful by conventional standards.

R Example: Paired t-Test

```
# --- Blood pressure data ---
before <- c(145, 158, 162, 149, 155, 170, 152, 141, 163, 157)
after  <- c(132, 148, 151, 140, 143, 159, 147, 136, 150, 148)
diff_d <- before - after

# Paired t-test
paired_result <- t.test(before, after,
                        paired      = TRUE,
                        alternative = "greater")

cat("Mean difference (before - after):", round(mean(diff_d), 2), "mmHg\n")
```

Mean difference (before - after): 9.8 mmHg

```
cat("SD of differences:                ", round(sd(diff_d), 2), "mmHg\n")
```

SD of differences: 2.9 mmHg

```
print(paired_result)
```

```
      Paired t-test

data:  before and after
t = 10.693, df = 9, p-value = 1.022e-06
alternative hypothesis: true mean difference is greater than 0
95 percent confidence interval:
 8.119924      Inf
sample estimates:
mean difference
      9.8
```

```

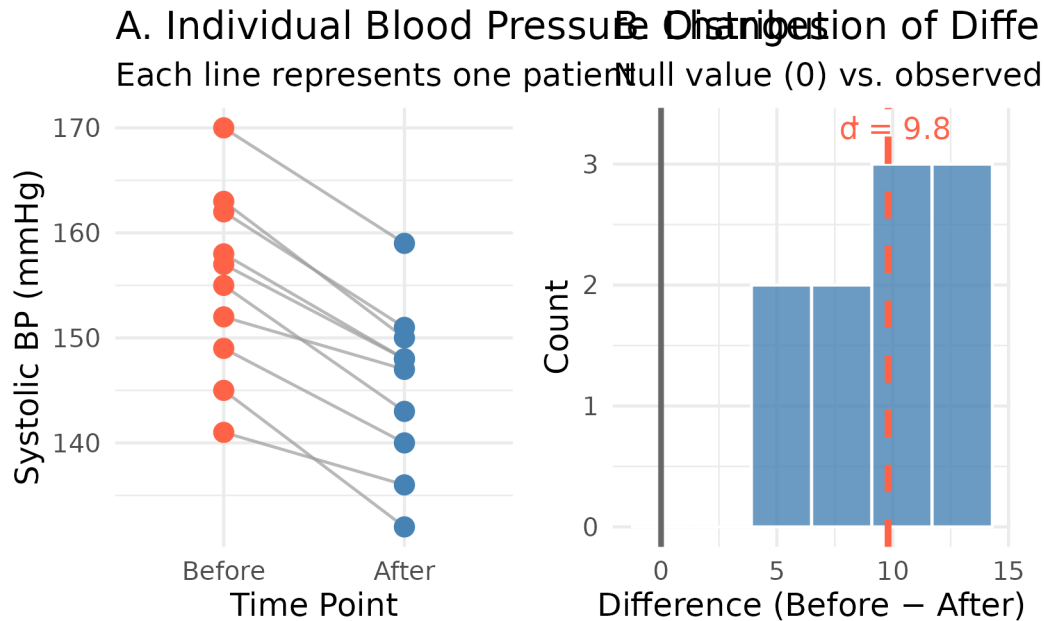
# --- Visualize paired data ---
bp_df <- data.frame(
  patient = rep(1:10, 2),
  time     = rep(c("Before", "After"), each = 10),
  bp       = c(before, after)
)
bp_df$time <- factor(bp_df$time, levels = c("Before", "After"))

p1 <- ggplot(bp_df, aes(x = time, y = bp, group = patient)) +
  geom_line(color = "gray60", alpha = 0.7) +
  geom_point(aes(color = time), size = 3) +
  scale_color_manual(values = c("Before" = "tomato", "After" = "steelblue")) +
  labs(title = "A. Individual Blood Pressure Changes",
       subtitle = "Each line represents one patient",
       x = "Time Point", y = "Systolic BP (mmHg)") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

p2 <- ggplot(data.frame(d = diff_d), aes(x = d)) +
  geom_histogram(bins = 6, fill = "steelblue",
                color = "white", alpha = 0.8) +
  geom_vline(xintercept = mean(diff_d), color = "tomato",
            linewidth = 1.2, linetype = "dashed") +
  geom_vline(xintercept = 0, color = "gray40",
            linewidth = 1, linetype = "solid") +
  annotate("text", x = mean(diff_d) + 0.3, y = 3.3,
         label = paste0("d̄ = ", round(mean(diff_d), 1)),
         color = "tomato", size = 4) +
  labs(title = "B. Distribution of Differences",
       subtitle = "Null value (0) vs. observed mean difference",
       x = "Difference (Before - After)", y = "Count") +
  theme_minimal(base_size = 12)

p1 + p2

```



Code explanation:

- `t.test(x, y, paired = TRUE)` runs the paired t-test in R. Internally, it computes differences and runs a one-sample test.
- The spaghetti plot (Panel A) is the canonical visualization for paired data — each line traces one subject's trajectory, making individual-level change visible.
- Panel B shows the distribution of differences. The red dashed line (mean difference) being far from zero confirms the visual story of Panel A.

Exercises

💡 Exercise 3.7

Use the built-in `sleep` dataset in R, which records extra hours of sleep with two drugs for 10 patients.

- Treat this as paired data (same 10 patients in both groups). Compute differences (group 2 – group 1).
- Test $H_0 : \mu_d = 0$ vs. $H_1 : \mu_d \neq 0$ using `t.test(..., paired = TRUE)`.
- Produce a spaghetti plot and a histogram of differences.
- Would you reach a different conclusion using an independent samples t-test? Why does pairing matter here?

One-Way ANOVA

Introduction

The t-test compares means between **two** groups. But what if we have three, four, or more groups? Conducting multiple pairwise t-tests is statistically invalid — with k groups, there are $\binom{k}{2}$ possible comparisons, and the probability of at least one false positive increases rapidly (**familywise error rate inflation**). For $k = 5$ groups and $\alpha = 0.05$, running all 10 pairwise tests gives a familywise error rate of $1 - 0.95^{10} \approx 40\%$. **One-way ANOVA** solves this by testing all groups simultaneously in a single test.

Theory

The ANOVA Model

One-way ANOVA tests whether the means of k independent groups are all equal:

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_k \quad H_1 : \text{at least one } \mu_i \neq \mu_j$$

The model decomposes each observation into components:

$$x_{ij} = \mu + \tau_i + \varepsilon_{ij}$$

where μ is the grand mean, τ_i is the effect of group i , and $\varepsilon_{ij} \sim N(0, \sigma^2)$ is random error.

Partitioning Variance

ANOVA works by partitioning the **total variability** in the data into two sources:

$$SS_{\text{Total}} = SS_{\text{Between}} + SS_{\text{Within}}$$

Table 1: ANOVA table structure

Source	Sum of Squares	df	Mean Square	F
Between groups	$SS_B = \sum_i n_i (\bar{x}_i - \bar{x})^2$	$k - 1$	$MS_B = SS_B / (k - 1)$	MS_B / MS_W
Within groups	$SS_W = \sum_i \sum_j (x_{ij} - \bar{x}_i)^2$	$N - k$	$MS_W = SS_W / (N - k)$	
Total	SS_T	$N - 1$		

The **F-statistic** is the ratio of between-group variability to within-group variability:

$$F = \frac{MS_{\text{Between}}}{MS_{\text{Within}}} \sim F(k - 1, N - k) \text{ under } H_0$$

If H_0 is true, $F \approx 1$ (between-group variation = within-group variation). A large F suggests genuine group differences.

Assumptions

1. **Independence:** Observations are independent within and across groups.
2. **Normality:** Observations within each group are approximately normally distributed.
3. **Homogeneity of variance (homoscedasticity):** $\sigma_1^2 = \sigma_2^2 = \dots = \sigma_k^2$. Test with Levene's test.

Post-Hoc Tests

A significant ANOVA only tells us that **at least one** mean differs — it does not tell us *which* pairs differ. **Post-hoc tests** conduct pairwise comparisons while controlling the familywise error rate:

- **Tukey's HSD (Honest Significant Difference):** Most common; compares all pairs; controls familywise error at α .
- **Bonferroni correction:** Divides α by the number of comparisons; conservative.
- **Scheffé's test:** Most flexible; controls error for all possible contrasts, not just pairwise.

Example: One-Way ANOVA

Example 3.5. A nutritionist compares weight loss (kg) over 12 weeks across three diet plans (A, B, C). Summary: $\bar{x}_A = 4.2$, $\bar{x}_B = 6.8$, $\bar{x}_C = 5.5$, $n_i = 15$ each. The ANOVA F-test gives $F(2, 42) = 8.43$, $p = 0.001$.

Decision: Reject H_0 — at least one diet plan produces different mean weight loss.

Post-hoc (Tukey HSD): Diet B vs. A: $p = 0.001$ (significant); Diet B vs. C: $p = 0.048$ (significant); Diet C vs. A: $p = 0.121$ (not significant).

Interpretation: Diet B produces significantly greater weight loss than both Diets A and C. Diets A and C do not differ significantly from each other.

R Example: One-Way ANOVA

```
# --- Use the built-in PlantGrowth dataset ---
data(PlantGrowth)

# Descriptive statistics by group
PlantGrowth |>
  group_by(group) |>
  summarise(
    n      = n(),
    Mean   = round(mean(weight), 3),
    SD     = round(sd(weight), 3),
    Median = round(median(weight), 3)
  ) |>
  kable(caption = "Plant Weight by Treatment Group") |>
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Table 2: Plant Weight by Treatment Group

group	n	Mean	SD	Median
ctrl	10	5.032	0.583	5.155
trt1	10	4.661	0.794	4.550
trt2	10	5.526	0.443	5.435

```
# --- One-way ANOVA ---
anova_model <- aov(weight ~ group, data = PlantGrowth)
anova_result <- summary(anova_model)
print(anova_result)
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
group      2   3.766   1.8832   4.846 0.0159 *
Residuals 27  10.492   0.3886
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

# --- Tukey HSD Post-Hoc Test ---
tukey_result <- TukeyHSD(anova_model)
print(tukey_result)

```

Tukey multiple comparisons of means
95% family-wise confidence level

```
Fit: aov(formula = weight ~ group, data = PlantGrowth)
```

```

$group
      diff      lwr      upr      p adj
trt1-ctrl -0.371 -1.0622161 0.3202161 0.3908711
trt2-ctrl  0.494 -0.1972161 1.1852161 0.1979960
trt2-trt1  0.865  0.1737839 1.5562161 0.0120064

```

```

# Tidy post-hoc table
tukey_df <- as.data.frame(tukey_result$group)
tukey_df$Comparison <- rownames(tukey_df)
tukey_df <- tukey_df[, c(5, 1:4)]
tukey_df[, 2:5] <- round(tukey_df[, 2:5], 4)

kable(tukey_df,
      caption = "Tukey HSD Post-Hoc Comparisons",
      row.names = FALSE,
      col.names = c("Comparison", "Difference",
                    "Lower CI", "Upper CI", "p adj")) |>
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE) |>
column_spec(5, bold = TRUE,
            color = ifelse(tukey_df[, 5] < 0.05, "tomato", "gray40"))

```

Table 3: Tukey HSD Post-Hoc Comparisons

Comparison	Difference	Lower CI	Upper CI	p adj
trt1-ctrl	-0.371	-1.0622	0.3202	0.3909
trt2-ctrl	0.494	-0.1972	1.1852	0.1980
trt2-trt1	0.865	0.1738	1.5562	0.0120

```

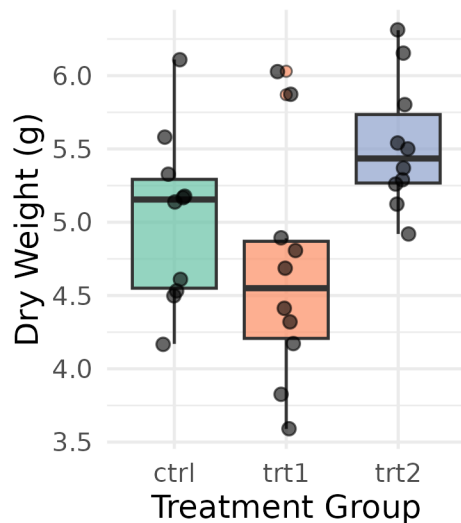
# --- Visualization ---
p1 <- ggplot(PlantGrowth, aes(x = group, y = weight, fill = group)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21) +
  geom_jitter(width = 0.1, size = 2, alpha = 0.6) +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "A. Plant Weight by Treatment",
       x = "Treatment Group",
       y = "Dry Weight (g)") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

# Tukey HSD confidence intervals plot
p2 <- as.data.frame(tukey_result$group) |>
  rownames_to_column("comparison") |>
  ggplot(aes(x = comparison, y = diff,
             ymin = lwr, ymax = upr, color = comparison)) +
  geom_pointrange(size = 0.9) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  scale_color_brewer(palette = "Set1") +
  coord_flip() +
  labs(title = "B. Tukey HSD 95% CIs",
       subtitle = "CI crossing zero = not significant",
       x = "Comparison",
       y = "Mean Difference") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

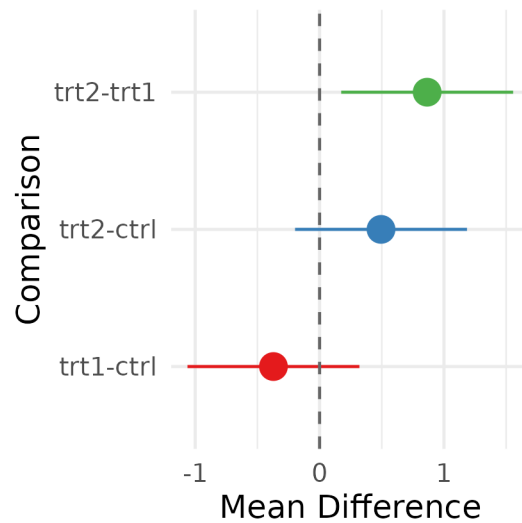
p1 + p2

```

A. Plant Weight by Treatment



B. Tukey HSD 95% CI



Code explanation:

- `aov(formula, data)` fits the ANOVA model; `summary()` produces the ANOVA table with F-statistic and p-value.
- `TukeyHSD(model)` performs all pairwise comparisons with familywise error control.
- The pointrange plot (Panel B) is the standard way to visualize Tukey HSD results — CIs crossing zero indicate non-significant differences.

Exercises

💡 Exercise 3.8

Using the `iris` dataset, test whether mean `Sepal.Length` differs across the three species.

- Check ANOVA assumptions: normality per group (Shapiro-Wilk) and equal variances (Levene's test via `car::leveneTest()`).
- Run one-way ANOVA. Report F, df, and p-value in APA format.
- Run Tukey HSD. Which species pairs differ significantly?
- Produce a boxplot and Tukey CI plot.

Exercise 3.9 (Challenge)

The `npk` dataset in R contains crop yield data from a factorial experiment.

- (a) Use one-way ANOVA to test whether nitrogen treatment (`N`) affects yield.
- (b) What happens to your conclusions if you violate the homoscedasticity assumption? Rerun the analysis using Welch's ANOVA (`oneway.test(var.equal = FALSE)`) and compare.

Non-Parametric Alternatives

Introduction

Parametric tests (t-tests, ANOVA) assume normally distributed populations and, in some cases, equal variances. When these assumptions are seriously violated — particularly with small samples, heavily skewed data, or ordinal outcomes — **non-parametric tests** provide valid alternatives. These tests make no assumption about the distributional form of the data, instead working with **ranks** rather than raw values. The cost is a small loss of statistical power when parametric assumptions are actually met.

Theory

Wilcoxon Signed-Rank Test (One-Sample / Paired)

The **non-parametric alternative to the one-sample and paired t-tests**. It tests whether the median of differences equals zero, without assuming normality.

Procedure:

1. Compute differences $d_i = x_{1i} - x_{2i}$ (or $d_i = x_i - \mu_0$ for one-sample).
2. Rank the **absolute values** $|d_i|$, ignoring zeros.
3. Assign signs to ranks based on the sign of d_i .
4. Compute W^+ (sum of positive ranks) and W^- (sum of negative ranks).
5. Test statistic: $W = \min(W^+, W^-)$.

For large samples, W is approximately normal and a z-approximation is used.

Mann-Whitney U Test (Two Independent Samples)

The **non-parametric alternative to the independent samples t-test**. Tests whether one population tends to have larger values than another (stochastic dominance), without assuming normality.

Procedure:

1. Pool and rank all observations from both groups.

2. Compute U_1 (sum of ranks from group 1 minus the minimum possible) and U_2 .
3. Test statistic: $U = \min(U_1, U_2)$.

In R, both the Wilcoxon signed-rank and Mann-Whitney U tests are implemented in `wilcox.test()`.

Kruskal-Wallis Test (Three or More Groups)

The **non-parametric alternative to one-way ANOVA**. Tests whether the distributions of k groups are identical (with the alternative that at least one group tends to have different values):

$$H = \frac{12}{N(N+1)} \sum_{i=1}^k \frac{R_i^2}{n_i} - 3(N+1) \sim \chi^2(k-1) \text{ under } H_0$$

where R_i is the sum of ranks in group i and N is the total sample size.

Post-hoc for Kruskal-Wallis: Dunn's test with Bonferroni or Benjamini-Hochberg correction.

When to Choose Non-Parametric Tests

Table 1: Non-parametric test selection guide

Situation	Non-Parametric Test	Parametric Equivalent
One sample or paired, non-normal	Wilcoxon signed-rank	Paired t-test
Two independent groups, non-normal	Mann-Whitney U	Independent t-test
Three or more groups, non-normal	Kruskal-Wallis	One-way ANOVA
Small n (< 15 per group) with skew	Any of the above	Requires normal assumption
Ordinal outcome variable	Any of the above	Inappropriate

Example: Mann-Whitney U Test

Example 3.6. A researcher compares reaction times (ms) between younger ($n_1 = 12$) and older ($n_2 = 10$) adults. The older group shows strong right skew (skewness = 2.1), violating normality. The Mann-Whitney U test gives $U = 18$, $p = 0.003$.

Interpretation: Older adults have significantly longer reaction times than younger adults ($W = 18$, $p = 0.003$). The non-parametric test was appropriate given the violation of normality in the older group.

R Example: Non-Parametric Tests

```
# --- Use airquality: compare Ozone in May vs. August ---
data(airquality)
ozone_may <- na.omit(airquality$Ozone[airquality$Month == 5])
ozone_aug <- na.omit(airquality$Ozone[airquality$Month == 8])

# Check normality
cat("Shapiro-Wilk - May Ozone:    p =",
    round(shapiro.test(ozone_may)$p.value, 4), "\n")
```

Shapiro-Wilk - May Ozone: p = 0

```
cat("Shapiro-Wilk - August Ozone: p =",
    round(shapiro.test(ozone_aug)$p.value, 4), "\n\n")
```

Shapiro-Wilk - August Ozone: p = 0.0903

```
# Mann-Whitney U test (non-parametric two-sample)
mw_result <- wilcox.test(ozone_may, ozone_aug,
                        alternative = "two.sided",
                        exact       = FALSE)
print(mw_result)
```

Wilcoxon rank sum test with continuity correction

```
data:  ozone_may and ozone_aug
W = 127.5, p-value = 0.0001208
alternative hypothesis: true location shift is not equal to 0
```

```
# Compare: Welch's t-test on the same data
t_result_ozone <- t.test(ozone_may, ozone_aug)
cat("\nWelch's t-test p-value:      ", round(t_result_ozone$p.value, 4), "\n")
```

Welch's t-test p-value: 2e-04

```
cat("Mann-Whitney U p-value:      ", round(mw_result$p.value, 4), "\n")
```

Mann-Whitney U p-value: 1e-04

```
# --- Kruskal-Wallis: Ozone across all months ---
airquality_kw <- na.omit(airquality[, c("Ozone", "Month")])

kw_result <- kruskal.test(Ozone ~ factor(Month), data = airquality_kw)
print(kw_result)
```

Kruskal-Wallis rank sum test

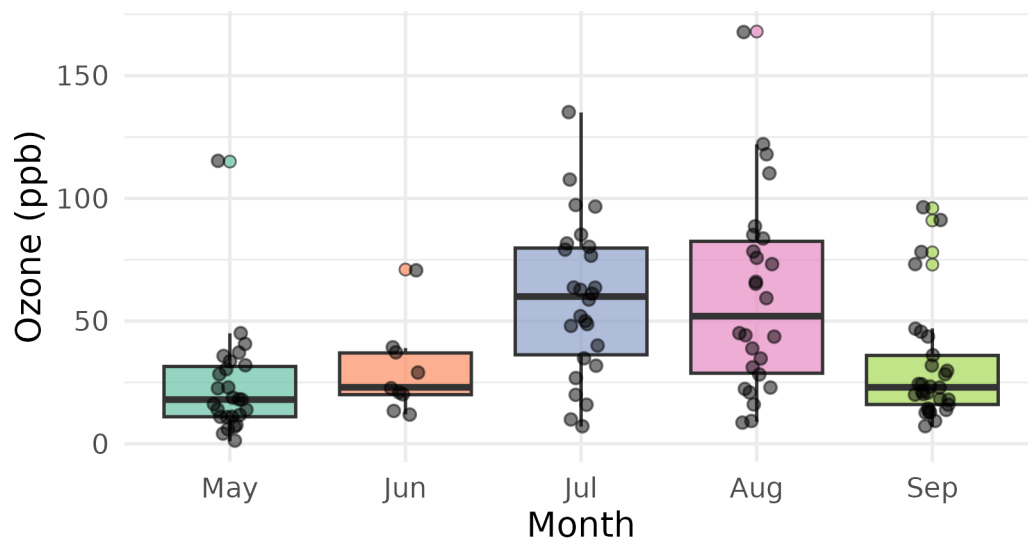
data: Ozone by factor(Month)
Kruskal-Wallis chi-squared = 29.267, df = 4, p-value = 6.901e-06

```
# --- Visualization ---
month_labels <- c("5" = "May", "6" = "Jun",
                 "7" = "Jul", "8" = "Aug", "9" = "Sep")

airquality_kw |>
  mutate(Month = factor(Month, labels = month_labels)) |>
  ggplot(aes(x = Month, y = Ozone, fill = Month)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21) +
  geom_jitter(width = 0.1, size = 1.8, alpha = 0.5) +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Ozone Concentration by Month",
       subtitle = paste0("Kruskal-Wallis: ² = ",
                          round(kw_result$statistic, 2),
                          ", df = ", kw_result$parameter,
                          ", p = ", round(kw_result$p.value, 4)),
       x = "Month", y = "Ozone (ppb)") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```

Ozone Concentration by Month

Kruskal-Wallis: $\chi^2 = 29.27$, $df = 4$, $p = 0$



Code explanation:

- `wilcox.test(x, y, alternative, exact)` runs both the Mann-Whitney U (two independent samples) and Wilcoxon signed-rank (one-sample or paired with `paired = TRUE`) test.
- `exact = FALSE` uses a normal approximation — appropriate when there are ties in the data.
- `kruskal.test(formula, data)` runs the Kruskal-Wallis test. For post-hoc, use `dunn_test()` from `rstatix`.

Exercises

💡 Exercise 3.10

Using the `sleep` dataset (extra sleep hours for two drugs, 10 patients each):

- (a) Test normality of differences. Is the paired t-test appropriate?
- (b) Run the Wilcoxon signed-rank test (`wilcox.test(..., paired = TRUE)`).
- (c) Compare results with the paired t-test from Exercise 3.7. Do conclusions agree?
- (d) Under what circumstances would the two tests disagree?

Exercise 3.11

Using the `iris` dataset, test whether `Petal.Length` differs across species using the Kruskal-Wallis test.

- (a) State why you might prefer Kruskal-Wallis over ANOVA for this variable.
 - (b) Run `kruskal.test()` and interpret the result.
 - (c) Perform Dunn's post-hoc test using `rstatix::dunn_test()` with Bonferroni correction.
 - (d) Do the conclusions match those from the parametric ANOVA in Exercise 3.8?
-

Effect Size and Statistical Power

Introduction

A statistically significant result answers only one question: *Is the effect real (not due to chance)?* It says nothing about whether the effect is **practically important**. With a large enough sample, even a trivially small difference becomes statistically significant. **Effect size** measures the magnitude of an effect independently of sample size, answering: *How big is this?* **Statistical power** determines whether our study is large enough to detect an effect if one truly exists. Together, effect size and power analysis are essential tools for designing rigorous research and interpreting published results.

Theory

Effect Size Measures

Cohen's d (for mean comparisons):

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{pooled}}}$$

Table 1: Cohen's d benchmarks

Cohen's d	Interpretation
0.2	Small effect
0.5	Medium effect
0.8	Large effect

η^2 (Eta-squared) for ANOVA:

$$\eta^2 = \frac{SS_{\text{Between}}}{SS_{\text{Total}}}$$

Represents the proportion of total variance explained by group membership. Values of 0.01, 0.06, and 0.14 correspond to small, medium, and large effects respectively.

r (Correlation coefficient) for non-parametric tests:

$$r = \frac{Z}{\sqrt{N}}$$

where Z is the standardized test statistic. Values of 0.1, 0.3, and 0.5 are small, medium, and large.

Statistical Power

$$\text{Power} = P(\text{Reject } H_0 \mid H_1 \text{ is true}) = 1 - \beta$$

Power depends on four interrelated factors:

Table 2: Factors affecting statistical power

Factor	Effect on Power
Sample size $n \uparrow$	Power $\uparrow$
Effect size $d \uparrow$	Power $\uparrow$
Significance level $\alpha \uparrow$	Power $\uparrow$
Variability $\sigma \uparrow$	Power $\downarrow$

Conventional target: Power ≥ 0.80 (80%) — meaning we have at least an 80% chance of detecting a true effect.

Power Analysis for Sample Size Planning

Power analysis is performed **before data collection** to determine the required sample size. Given:

- The desired power ($1 - \beta$, typically 0.80 or 0.90)
- The significance level (α , typically 0.05)
- The minimum effect size of practical interest (d or f)

We solve for n . This is a critical step in research design — under-powered studies waste resources and fail to detect real effects; over-powered studies waste money and may detect trivially small effects.

Example: Effect Size and Power

Example 3.7. Returning to the teaching method comparison (Example 3.3):

$$d = \frac{|81.6 - 74.2|}{\sqrt{(11.8^2 + 9.4^2)/2}} = \frac{7.4}{\sqrt{113.8}} = \frac{7.4}{10.67} \approx 0.69$$

A Cohen's d of 0.69 indicates a **medium-to-large** effect. The difference of 7.4 points is both statistically significant and educationally meaningful. A researcher planning a replication study who wants 80% power to detect $d = 0.69$ at $\alpha = 0.05$ (two-tailed) would need approximately $n = 32$ per group.

R Example: Effect Size and Power Analysis

```
# --- Cohen's d for teaching methods ---  
library(effectsize)  
  
set.seed(202)  
group_A <- rnorm(30, mean = 74.2, sd = 11.8)  
group_B <- rnorm(28, mean = 81.6, sd = 9.4)  
  
# Cohen's d  
d_result <- effectsize::cohens_d(group_B, group_A)  
print(d_result)
```

```
Cohen's d |          95% CI  
-----  
0.66      | [0.13, 1.19]
```

- Estimated using pooled SD.

```
interpret_cohens_d(d_result$Cohens_d)
```

```
[1] "medium"  
(Rules: cohen1988)
```

```
# --- Power analysis using pwr package ---
library(pwr)

# 1. What power do we have with n=30 per group and d=0.69?
power_current <- pwr.t.test(n      = 30,
                           d      = 0.69,
                           sig.level = 0.05,
                           type     = "two.sample",
                           alternative = "two.sided")
cat("Power with n=30, d=0.69:", round(power_current$power, 3), "\n\n")
```

Power with n=30, d=0.69: 0.748

```
# 2. What n do we need for 80% power with d=0.69?
n_needed <- pwr.t.test(power = 0.80,
                      d      = 0.69,
                      sig.level = 0.05,
                      type     = "two.sample",
                      alternative = "two.sided")
cat("n needed for 80% power, d=0.69:", ceiling(n_needed$n), "per group\n\n")
```

n needed for 80% power, d=0.69: 34 per group

```
# 3. What n do we need for 90% power?
n_needed_90 <- pwr.t.test(power = 0.90,
                          d      = 0.69,
                          sig.level = 0.05,
                          type     = "two.sample",
                          alternative = "two.sided")
cat("n needed for 90% power, d=0.69:", ceiling(n_needed_90$n), "per group\n")
```

n needed for 90% power, d=0.69: 46 per group

```
# --- Power curve: power vs. sample size for different effect sizes ---
n_seq      <- seq(10, 100, by = 2)
d_values   <- c(0.2, 0.5, 0.8)
d_labels   <- c("Small (d=0.2)", "Medium (d=0.5)", "Large (d=0.8)")

power_df <- map_dfr(seq_along(d_values), function(i) {
  d   <- d_values[i]
  lbl <- d_labels[i]
```



```

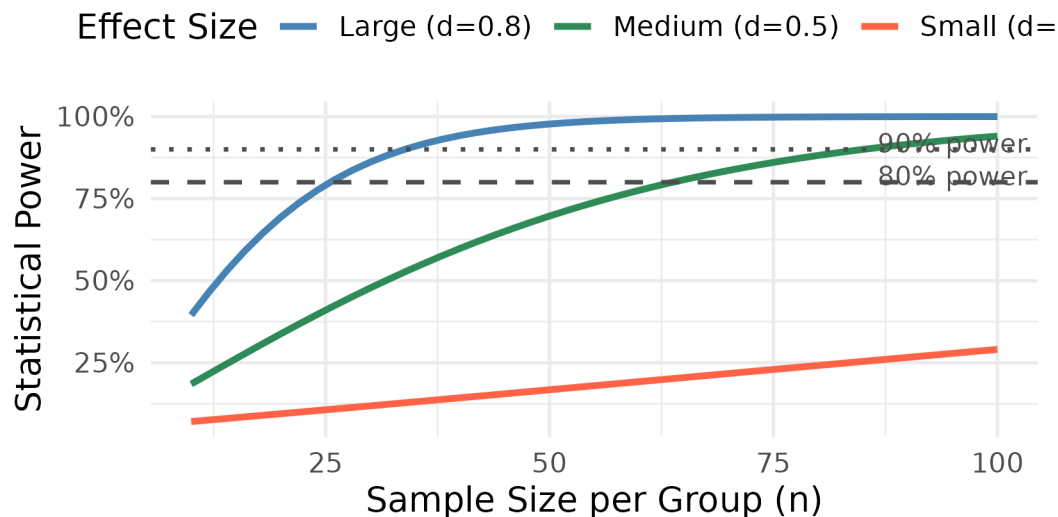
pwr_vals <- sapply(n_seq, function(n) {
  pwr.t.test(n = n, d = d, sig.level = 0.05,
    type = "two.sample",
    alternative = "two.sided")$power
})
data.frame(n = n_seq, power = pwr_vals, effect = lbl)
})

ggplot(power_df, aes(x = n, y = power, color = effect)) +
  geom_line(linewidth = 1.2) +
  geom_hline(yintercept = 0.80, linetype = "dashed",
    color = "gray30", linewidth = 0.8) +
  geom_hline(yintercept = 0.90, linetype = "dotted",
    color = "gray30", linewidth = 0.8) +
  annotate("text", x = 95, y = 0.82,
    label = "80% power", color = "gray30", size = 3.5) +
  annotate("text", x = 95, y = 0.92,
    label = "90% power", color = "gray30", size = 3.5) +
  scale_color_manual(values = c("steelblue", "seagreen", "tomato")) +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Power Curves: Two-Sample t-Test ( = 0.05)",
    subtitle = "Each curve shows a different effect size",
    x = "Sample Size per Group (n)",
    y = "Statistical Power",
    color = "Effect Size") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")

```

Power Curves: Two-Sample t-Test ($\alpha = 0.05$)

Each curve shows a different effect size



Code explanation:

- `cohens_d()` from `effectsize` computes Cohen's d with confidence intervals; `interpret_cohens_d()` gives the verbal label (small/medium/large).
- `pwr.t.test()` from `pwr` performs power analysis for t-tests. Set any three of (n, d, sig.level, power) and it solves for the fourth.
- The power curve plot is an essential tool for research design — it visually shows the sample size required to achieve desired power for effects of different magnitudes.

Exercises

💡 Exercise 3.12

For the `mtcars` analysis from Exercise 3.5 (mpg by transmission type):

- Compute Cohen's d for the mean difference between automatic and manual transmission cars.
- Interpret the effect size using Cohen's benchmarks.
- How many cars per group would be needed to achieve 80% power to detect this effect at $\alpha = 0.05$?
- Plot a power curve showing power vs. sample size for small, medium, and the observed effect size.

💡 Exercise 3.13 (Challenge)

A researcher is planning a three-arm clinical trial comparing a new drug, an existing drug, and placebo. They expect a medium effect size ($f = 0.25$ for ANOVA) and want 80% power at $\alpha = 0.05$.

- (a) Use `pwr.anova.test()` to determine the required sample size per group.
- (b) How does the required n change if power is increased to 90%?
- (c) Compute η^2 from the `PlantGrowth` ANOVA result and interpret its magnitude.

Chapter Lab Activity: Exploring Hypothesis Testing with the ToothGrowth Dataset

Objectives

In this lab you will apply the full hypothesis testing workflow to the `ToothGrowth` dataset — a classic R dataset recording the length of odontoblasts (cells responsible for tooth growth) in 60 guinea pigs, each receiving one of three doses of Vitamin C (0.5, 1, and 2 mg/day) via one of two delivery methods (orange juice or ascorbic acid). You will move from checking assumptions to selecting tests, reporting results, and computing effect sizes.

The Dataset

```
data(ToothGrowth)

kable(head(ToothGrowth, 12),
       caption = "First 12 Rows of the ToothGrowth Dataset") |>
  kable_styling(bootstrap_options = c("striped", "hover"), font_size = 11)
```

Table 1: First 12 Rows of the ToothGrowth Dataset

len	supp	dose
4.2	VC	0.5
11.5	VC	0.5
7.3	VC	0.5
5.8	VC	0.5
6.4	VC	0.5
10.0	VC	0.5
11.2	VC	0.5
11.2	VC	0.5
5.2	VC	0.5
7.0	VC	0.5

16.5	VC	1.0
16.5	VC	1.0

```
cat("Dimensions:", nrow(ToothGrowth), "rows x",
    ncol(ToothGrowth), "columns\n")
```

Dimensions: 60 rows x 3 columns

```
cat("Supplement types:", levels(factor(ToothGrowth$supp)), "\n")
```

Supplement types: OJ VC

```
cat("Dose levels:", unique(ToothGrowth$dose), "mg/day\n")
```

Dose levels: 0.5 1 2 mg/day

Variable descriptions:

Variable	Description	Scale
len	Odontoblast length (microns)	Ratio, continuous
supp	Supplement type: OJ (orange juice) or VC (ascorbic acid)	Nominal
dose	Vitamin C dose (0.5, 1.0, 2.0 mg/day)	Ordinal (treated as ratio here)

Lab Task 1: Descriptive Overview and Assumption Checks

```
# Descriptive statistics by supplement and dose
ToothGrowth |>
  mutate(dose = factor(dose)) |>
  group_by(supp, dose) |>
  summarise(
    n      = n(),
    Mean   = round(mean(len), 2),
    SD     = round(sd(len), 2),
    Median = round(median(len), 2),
    .groups = "drop"
  ) |>
  kable(caption = "Tooth Length by Supplement and Dose") |>
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Table 2: Tooth Length by Supplement and Dose

supp	dose	n	Mean	SD	Median
OJ	0.5	10	13.23	4.46	12.25
OJ	1	10	22.70	3.91	23.45
OJ	2	10	26.06	2.66	25.95
VC	0.5	10	7.98	2.75	7.15
VC	1	10	16.77	2.52	16.50
VC	2	10	26.14	4.80	25.95

```
# Normality check per group
ToothGrowth |>
  group_by(supp) |>
  summarise(
    SW_W      = round(shapiro.test(len)$statistic, 4),
    SW_pvalue = round(shapiro.test(len)$p.value, 4),
    Skewness  = round(moments::skewness(len), 3),
    .groups   = "drop"
  ) |>
  kable(caption = "Normality Assessment by Supplement Type") |>
  kable_styling(bootstrap_options = c("striped","hover"), full_width = FALSE)
```

Table 3: Normality Assessment by Supplement Type

supp	SW_W	SW_pvalue	Skewness
OJ	0.9178	0.0236	-0.55
VC	0.9657	0.4284	0.29

Lab Task 2: Two-Sample Test — OJ vs. VC

```
# Split by supplement
oj <- ToothGrowth$len[ToothGrowth$supp == "OJ"]
vc <- ToothGrowth$len[ToothGrowth$supp == "VC"]

# Welch's t-test
t_supp <- t.test(oj, vc, alternative = "two.sided")

# Mann-Whitney U (non-parametric comparison)
mw_supp <- wilcox.test(oj, vc, alternative = "two.sided", exact = FALSE)
```

```
# Cohen's d
d_supp <- effectsize::cohens_d(oj, vc)

results_supp <- data.frame(
  Test      = c("Welch's t-test", "Mann-Whitney U"),
  Statistic = round(c(t_supp$Statistic, mw_supp$Statistic), 3),
  p_value   = round(c(t_supp$p.value, mw_supp$p.value), 4),
  Decision  = ifelse(c(t_supp$p.value, mw_supp$p.value) < 0.05,
                    "Reject H", "Fail to Reject H")
)

kable(results_supp,
  caption = "OJ vs. VC: Test Results at  $\alpha = 0.05$ ",
  col.names = c("Test", "Statistic", "p-value", "Decision")) |>
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE) |>
column_spec(4, bold = TRUE,
  color = ifelse(results_supp$Decision == "Reject H",
                "tomato", "steelblue"))
```

Table 4: OJ vs. VC: Test Results at $\alpha = 0.05$

	Test	Statistic	p-value	Decision
t	Welch's t-test	1.915	0.0606	Fail to Reject H
W	Mann-Whitney U	575.500	0.0645	Fail to Reject H

```
cat("\nCohen's d:", round(d_supp$Cohens_d, 3),
    "(", interpret_cohens_d(d_supp$Cohens_d), ")\n")
```

Cohen's d: 0.495 (small)

Lab Task 3: One-Way ANOVA — Effect of Dose

```
# One-way ANOVA: does dose affect tooth length?
ToothGrowth$dose_f <- factor(ToothGrowth$dose)
anova_dose <- aov(len ~ dose_f, data = ToothGrowth)
tukey_dose <- TukeyHSD(anova_dose)

cat("=== ANOVA: Tooth Length by Dose ===\n")
```

```
=== ANOVA: Tooth Length by Dose ===
```

```
print(summary(anova_dose))
```

```
              Df Sum Sq Mean Sq F value    Pr(>F)
dose_f         2   2426    1213    67.42 9.53e-16 ***
Residuals     57   1026         18
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat("\n=== Tukey HSD Post-Hoc ===\n")
```

```
=== Tukey HSD Post-Hoc ===
```

```
print(tukey_dose)
```

```
Tukey multiple comparisons of means
 95% family-wise confidence level
```

```
Fit: aov(formula = len ~ dose_f, data = ToothGrowth)
```

```
$dose_f
      diff      lwr      upr    p adj
1-0.5  9.130  5.901805 12.358195 0.00e+00
2-0.5 15.495 12.266805 18.723195 0.00e+00
2-1     6.365  3.136805  9.593195 4.25e-05
```

```
# Eta-squared
ss_between <- summary(anova_dose)[[1]][1,"Sum Sq"]
ss_total   <- sum(summary(anova_dose)[[1]][,"Sum Sq"])
eta_sq     <- ss_between / ss_total
cat("\nEta-squared ( ^2):", round(eta_sq, 4),
    "- proportion of variance explained by dose\n")
```

```
Eta-squared ( ^2): 0.7029 - proportion of variance explained by dose
```

Lab Task 4: Comprehensive Visualization


```

# Panel A: Overall distribution by supplement
p1 <- ggplot(ToothGrowth, aes(x = supp, y = len, fill = supp)) +
  geom_violin(alpha = 0.5, trim = FALSE) +
  geom_boxplot(width = 0.12, fill = "white",
               outlier.shape = 21) +
  scale_fill_manual(values = c("OJ" = "tomato", "VC" = "steelblue")) +
  labs(title = "A. Length by Supplement",
       x = "Supplement", y = "Tooth Length (µm)") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "none")

# Panel B: Length by dose
p2 <- ggplot(ToothGrowth,
             aes(x = factor(dose), y = len, fill = factor(dose))) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21) +
  geom_jitter(width = 0.1, size = 1.8, alpha = 0.5) +
  scale_fill_brewer(palette = "Blues") +
  labs(title = "B. Length by Dose",
       x = "Dose (mg/day)", y = "Tooth Length (µm)") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "none")

# Panel C: Interaction - supplement × dose
p3 <- ToothGrowth |>
  group_by(supp, dose) |>
  summarise(mean_len = mean(len), se = sd(len)/sqrt(n()), .groups="drop") |>
  ggplot(aes(x = factor(dose), y = mean_len,
             group = supp, color = supp)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3.5) +
  geom_errorbar(aes(ymin = mean_len - se, ymax = mean_len + se),
               width = 0.1) +
  scale_color_manual(values = c("OJ" = "tomato", "VC" = "steelblue")) +
  labs(title = "C. Interaction: Supplement × Dose",
       x = "Dose (mg/day)", y = "Mean Tooth Length (µm)",
       color = "Supplement") +
  theme_minimal(base_size = 11)

# Panel D: Tukey HSD intervals for dose
p4 <- as.data.frame(tukey_dose$dose_f) |>
  rownames_to_column("comparison") |>
  ggplot(aes(x = comparison, y = diff,
             ymin = lwr, ymax = upr,
             color = diff > 0)) +

```

```

geom_pointrange(size = 0.9) +
geom_hline(yintercept = 0, linetype = "dashed") +
scale_color_manual(values = c("FALSE" = "steelblue",
                             "TRUE" = "tomato")) +

coord_flip() +
labs(title = "D. Tukey HSD: Dose Comparisons",
     x = "Comparison", y = "Mean Difference") +
theme_minimal(base_size = 11) +
theme(legend.position = "none")

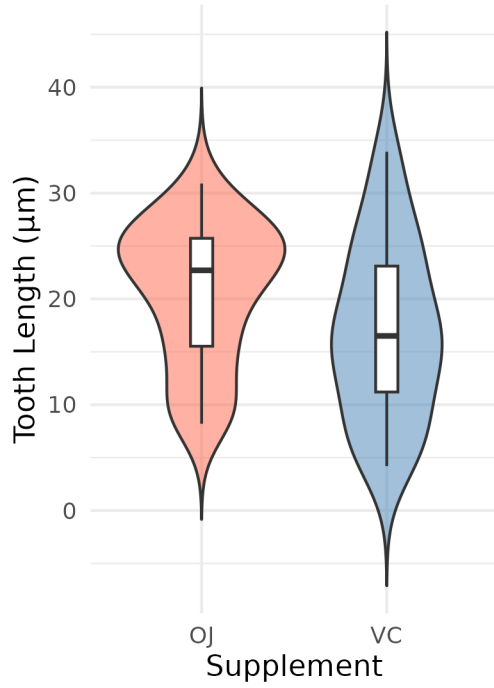
```

```

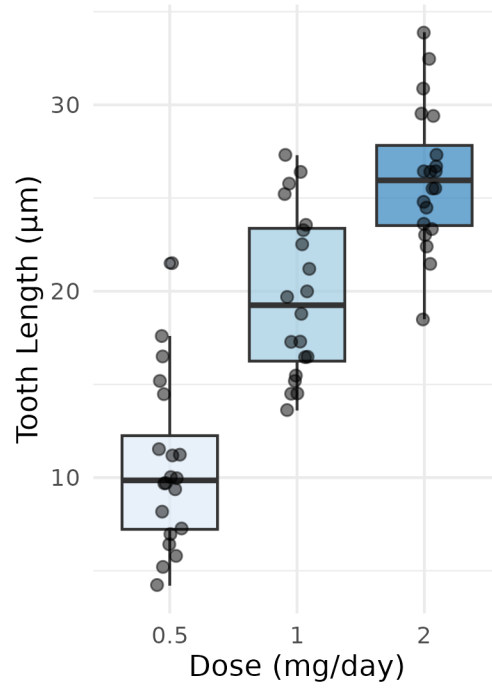
(p1 + p2) / (p3 + p4)

```

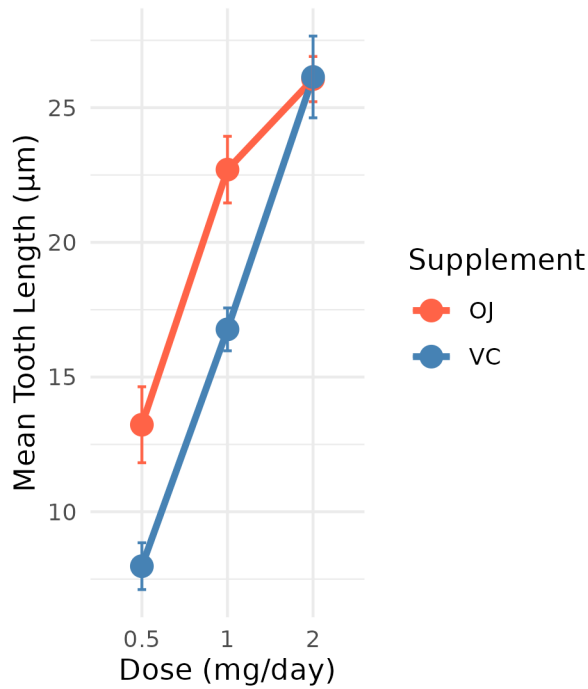
A. Length by Supplement



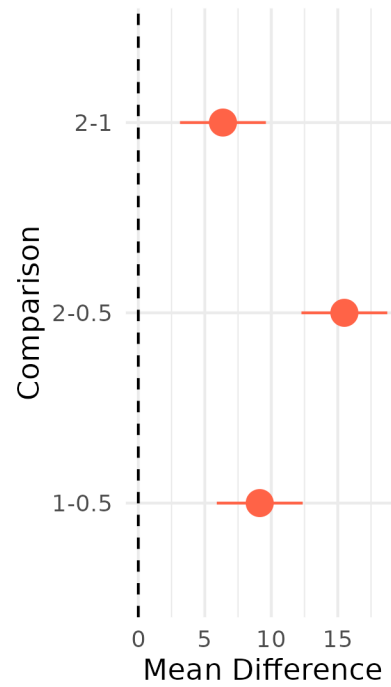
B. Length by Dose



C. Interaction: Supplement × Dose



D. Tukey HSD: Dc



Lab Discussion Questions

Answer the following in writing:

1. **Test Selection:** In Lab Task 2, both Welch's t-test and Mann-Whitney U give similar conclusions. Under what conditions might they disagree? Which would you report in a paper, and why?
 2. **Practical Significance:** The ANOVA in Task 3 is highly significant. Compute and interpret η^2 . Does the dose variable explain a large, medium, or small proportion of variance in tooth length?
 3. **Post-Hoc Interpretation:** Based on Tukey HSD, which dose transition produces the greatest gain in tooth length: $0.5 \rightarrow 1.0$ mg/day or $1.0 \rightarrow 2.0$ mg/day? What does this suggest about dose-response?
 4. **Interaction Pattern:** Panel C shows that at low doses, OJ outperforms VC, but at 2.0 mg/day the two converge. What does this interaction pattern imply for designing a Vitamin C supplementation study?
 5. **Power and Replication:** With only 10 animals per supplement-dose combination, compute the power of the two-sample t-test to detect the observed effect size between OJ and VC. Is this study adequately powered?
-

Chapter Summary

Summary

This chapter established hypothesis testing as a principled framework for data-driven decision making:

- **The logic of hypothesis testing** — stating H_0 and H_1 , computing p-values, controlling Type I and II errors — is the same across all tests; only the test statistic changes.
- **One-sample t-tests** compare a single population mean to a hypothesized value.
- **Two-sample tests** (Welch's t-test preferred) compare means between two independent groups.
- **Paired t-tests** exploit within-subject pairing to eliminate between-subject noise and increase power.
- **One-way ANOVA** extends t-tests to three or more groups, avoiding familywise error inflation; post-hoc tests (Tukey HSD) identify which pairs differ.
- **Non-parametric tests** (Wilcoxon, Mann-Whitney, Kruskal-Wallis) provide valid alternatives when normality or equal-variance assumptions are violated.
- **Effect size** (Cohen's d , η^2) measures practical importance independently of sample size; **power analysis** ensures studies are designed to detect meaningful effects.

! Key Formulas to Know

One-sample t-statistic:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \sim t(n-1)$$

Two-sample t-statistic (Welch's):

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$$

Paired t-statistic:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \sim t(n-1)$$

F-statistic (ANOVA):

$$F = \frac{MS_{\text{Between}}}{MS_{\text{Within}}} \sim F(k-1, N-k)$$

Cohen's d:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{pooled}}}$$

Eta-squared:

$$\eta^2 = \frac{SS_{\text{Between}}}{SS_{\text{Total}}}$$

References

- Anderson, E. (1935). The irises of the Gaspé Peninsula. *Bulletin of the American Iris Society*, 59, 2–5.
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.
- Fisher, R. A. (1925). *Statistical methods for research workers*. Oliver and Boyd.
- Fox, J., & Weisberg, S. (2019). *An R companion to applied regression* (3rd ed.). SAGE.
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Kassambara, A. (2023). *rstatix: Pipe-friendly framework for basic statistical tests* (R package version 0.7.2). <https://CRAN.R-project.org/package=rstatix>
- Lüdtke, D., Ben-Shachar, M. S., Patil, I., & Makowski, D. (2020). Extracting, computing and exploring the parameters of statistical models using R. *Journal of Open Source Software*, 5(53), 2445.
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1), 50–60.
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality. *Biometrika*, 52(3/4), 591–611.
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- Venables, W. N., & Ripley, B. D. (2002). *Modern applied statistics with S* (4th ed.). Springer.
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.

Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

End of Chapter 3. Proceed to Chapter 4: Test of Independence of Variables.

Chapter 4: Test of Independence of Variables

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(vcd)           # association measures, mosaic plots
library(corrplot)      # correlation matrix visualization
library(ggcorrplot)    # ggplot2-based correlation plots
library(rstatix)       # tidy correlation tests
library(ppcor)         # partial correlation
```

Chapter Overview

Introduction

One of the most fundamental questions in data analysis is whether two variables are **related** to each other, or whether they vary **independently**. Does a patient's smoking status relate to their disease outcome? Is a student's program of study associated with their level of satisfaction? Do income and education move together? Tests of independence answer these questions formally, across all combinations of variable types.

This chapter covers:

- **The Concept of Independence** — theoretical foundations linking back to probability
- **Chi-Square Test of Independence** — testing association between two categorical variables
- **Fisher's Exact Test** — a robust alternative for small or sparse contingency tables
- **Cramér's V and Effect Size** — quantifying the strength of categorical association
- **Correlation Tests** — Pearson and Spearman for continuous and ordinal variables
- **Point-Biserial and Phi Coefficients** — association involving binary variables
- **Partial Correlation** — measuring association while controlling for a third variable

Learning Objectives

By the end of this chapter, you will be able to:

1. Distinguish statistical independence from dependence for different variable types.
2. Conduct and interpret chi-square and Fisher's exact tests for categorical data.
3. Compute and interpret Cramér's V as a measure of association strength.
4. Test the significance of Pearson and Spearman correlations.
5. Select the appropriate association measure based on variable types.
6. Compute partial correlations to control for confounding variables.

The Concept of Independence

Introduction

In Chapter 2, we defined statistical independence in probability terms: two events A and B are independent if $P(A \cap B) = P(A) \cdot P(B)$. In data analysis, we extend this idea to variables: two variables are **statistically independent** if knowing the value of one provides no information about the value of the other. When they are not independent, we say they are **associated** or **dependent**.

Testing for independence is not the same as proving causation. Association tells us that two variables move together — it says nothing about which causes which, or whether both are driven by a third variable. This distinction is critical in data science, where spurious correlations are common in large datasets.

Theory

Independence for Categorical Variables

Two categorical variables X and Y are independent if, for all categories i and j :

$$P(X = i \cap Y = j) = P(X = i) \cdot P(Y = j)$$

In a contingency table, this means every **observed cell frequency** should equal the corresponding **expected cell frequency** under independence:

$$E_{ij} = \frac{(\text{Row } i \text{ total}) \times (\text{Column } j \text{ total})}{\text{Grand total}} = \frac{R_i \cdot C_j}{N}$$

Any systematic departure of observed frequencies from expected frequencies is evidence of association.

Independence for Continuous Variables

Two continuous variables X and Y are independent if their **joint distribution** equals the product of their marginal distributions:

$$f(x, y) = f_X(x) \cdot f_Y(y)$$

In practice, we test a weaker condition: **linear independence** (zero correlation). Note that zero correlation does not guarantee full independence — two variables can be nonlinearly related yet have zero Pearson correlation. This is why visualization always accompanies formal tests.

A Taxonomy of Association Measures

The correct test and measure of association depend on the **types of variables** being compared:

Table 1: Association measures by variable type

Variable 1	Variable 2	Test	Effect Size
Categorical	Categorical	Chi-square / Fisher's	Cramér's V
Continuous	Continuous	Pearson / Spearman correlation test	r / ρ
Binary	Continuous	Point-biserial correlation test	r_{pb}
Binary	Binary	Chi-square / Fisher's	Phi (ϕ)
Continuous (controlling for Z)	Continuous	Partial correlation	Partial r

Example: Identifying the Correct Test

Example 4.1. A researcher collects data on university students. Identify the appropriate test for each pair of variables:

Table 2: Example 4.1 — Test selection by variable type

Variable 1	Variable 2	Appropriate Test
Faculty (Science, Arts, Business)	Pass/Fail status	Chi-square test
Age (years)	GPA (0–4 scale)	Pearson or Spearman correlation
Gender (Male/Female)	Exam score (continuous)	Point-biserial correlation
Smoking (Yes/No)	Disease (Yes/No)	Chi-square or Fisher’s exact
Income	Happiness (controlling for age)	Partial correlation

R Example: Exploring Variable Relationships

```
# --- Use the built-in mtcars and titanic-style datasets ---
# Convert relevant mtcars variables to categorical
data(mtcars)
mtcars_cat <- mtcars |>
  mutate(
    transmission = factor(am, labels = c("Automatic", "Manual")),
    cylinders     = factor(cyl),
    efficiency    = factor(ifelse(mpg >= median(mpg), "High", "Low"),
                           levels = c("Low", "High"))
  )

# Quick overview of variable types
cat("=== Variable Type Overview ===\n")
```

```
=== Variable Type Overview ===
```

```
cat("Categorical × Categorical: transmission vs. efficiency\n")
```

```
Categorical × Categorical: transmission vs. efficiency
```

```
cat("Continuous × Continuous:   mpg vs. hp\n")
```

```
Continuous × Continuous:   mpg vs. hp
```

```
cat("Binary × Continuous:      transmission vs. mpg\n\n")
```

Binary × Continuous: transmission vs. mpg

```
# Contingency table: transmission × efficiency
tab <- table(mtcars_cat$transmission, mtcars_cat$efficiency)
kable(addmargins(tab),
      caption = "Contingency Table: Transmission × Fuel Efficiency") |>
kable_styling(bootstrap_options = c("striped","hover"), full_width = FALSE)
```

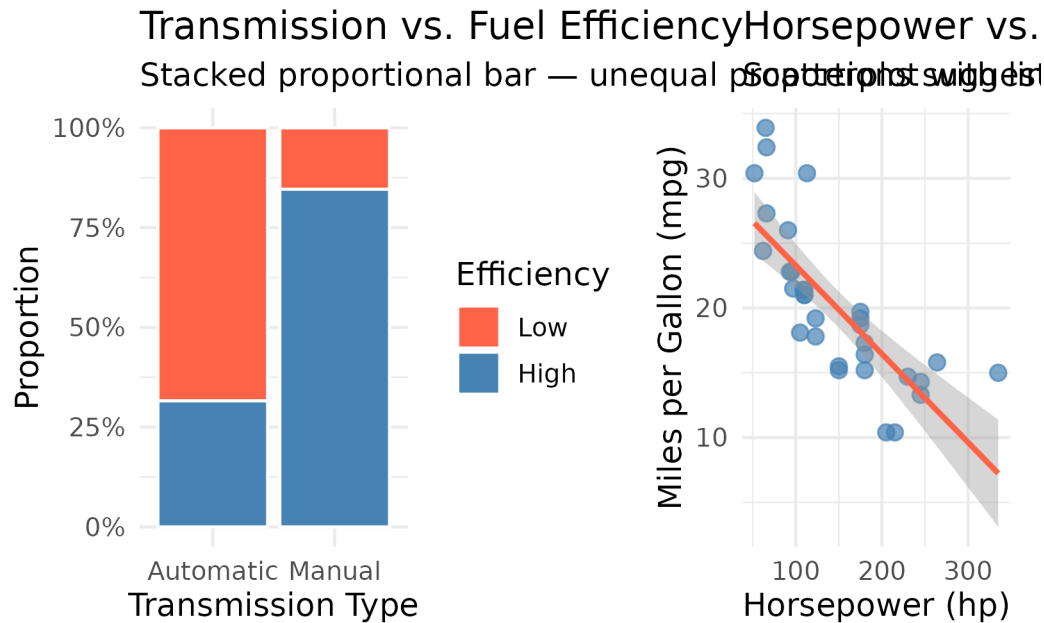
Table 3: Contingency Table: Transmission × Fuel Efficiency

	Low	High	Sum
Automatic	13	6	19
Manual	2	11	13
Sum	15	17	32

```
# --- Visualize potential associations ---
p1 <- ggplot(mtcars_cat,
            aes(x = transmission, fill = efficiency)) +
  geom_bar(position = "fill", color = "white") +
  scale_fill_manual(values = c("Low" = "tomato", "High" = "steelblue")) +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Transmission vs. Fuel Efficiency",
       subtitle = "Stacked proportional bar - unequal proportions suggest association",
       x = "Transmission Type",
       y = "Proportion",
       fill = "Efficiency") +
  theme_minimal(base_size = 12)

p2 <- ggplot(mtcars, aes(x = hp, y = mpg)) +
  geom_point(color = "steelblue", size = 2.5, alpha = 0.7) +
  geom_smooth(method = "lm", color = "tomato",
             se = TRUE, linewidth = 1) +
  labs(title = "Horsepower vs. Fuel Efficiency",
       subtitle = "Scatterplot with linear fit - downward trend suggests negative correlation",
       x = "Horsepower (hp)",
       y = "Miles per Gallon (mpg)") +
  theme_minimal(base_size = 12)

p1 + p2
```



Code explanation:

- `addmargins(table())` adds row and column totals to the contingency table — essential for computing expected frequencies by hand.
- `geom_bar(position = "fill")` creates proportional stacked bars, making differences in conditional distributions visually obvious.
- Visual inspection always precedes formal testing — if proportions look identical across groups, a significant test result warrants careful scrutiny.

Exercises

💡 Exercise 4.1

For each of the following research questions, identify: (a) the variable types, (b) the appropriate test of independence, and (c) what a significant result would mean substantively.

1. Do male and female students differ in their choice of academic major?
2. Is there a relationship between study hours per week and final exam score?
3. Does coffee consumption (cups/day) relate to reported anxiety level (1–10 scale)?
4. Is being a first-generation student associated with dropout status (yes/no)?

Chi-Square Test of Independence

Introduction

The **chi-square test of independence** is the standard procedure for testing whether two categorical variables are associated. It compares the **observed** frequencies in a contingency table to the **expected** frequencies that would arise if the variables were truly independent. It is among the most widely used tests in social science, medical research, and data science — applicable whenever we want to know if two categorical groupings are related.

Theory

The Test Statistic

Given a contingency table with r rows and c columns, the chi-square test statistic is:

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

where O_{ij} is the observed frequency and $E_{ij} = R_i C_j / N$ is the expected frequency in cell (i, j) .

Under H_0 (independence), $\chi^2 \sim \chi^2((r-1)(c-1))$ approximately.

Hypotheses:

H_0 : The two variables are independent

H_1 : The two variables are associated

Critical Value for the Chi-Square Test

Recall from Chapter 3: a critical value is a quantile chosen so that the area beyond it equals α . The same idea extends to any distribution — including the chi-square distribution, which is what we need here. The quantile function follows the same q-prefix convention:

Distribution	CDF	Quantile
$t(df)$	<code>pt(x, df)</code>	<code>qt(p, df)</code>
$\chi^2(df)$	<code>pchisq(x, df)</code>	<code>qchisq(p, df)</code>

The chi-square test is always **right-tailed** — larger χ^2 means more disagreement between observed and expected counts, so the rejection region is only in the upper tail. The critical value at significance α with df degrees of freedom is:

$$\chi_{\text{crit}}^2 = \chi_{1-\alpha, df}^2$$

```
qchisq(0.95, df = 2) # chi^2_0.95,2 = 5.991
```

```
[1] 5.991465
```

Just as before, keep χ_{cal}^2 (computed from your contingency table) separate from χ_{crit}^2 (determined only by α and df , via `qchisq()`).

Assumptions

1. Observations are **independent** (each subject appears in exactly one cell).
2. The data are **counts** (not percentages or proportions).
3. **Expected cell frequencies** are sufficiently large:
 - At least 80% of cells have $E_{ij} \geq 5$
 - No cell has $E_{ij} < 1$

When these conditions are violated — typically with small samples or rare categories — use **Fisher's Exact Test** (Section 3).

Yates' Continuity Correction

For 2×2 tables, a continuity correction is sometimes applied to account for the fact that we are approximating a discrete distribution with a continuous one:

$$\chi_{\text{Yates}}^2 = \sum \frac{(|O_{ij} - E_{ij}| - 0.5)^2}{E_{ij}}$$

Yates' correction is conservative (reduces test power) and is debated in the literature. Many statisticians prefer Fisher's Exact Test for 2×2 tables regardless of sample size.

Standardized Residuals

A significant chi-square test tells us *that* association exists but not *where* it comes from. **Standardized residuals** identify which cells contribute most:

$$r_{ij} = \frac{O_{ij} - E_{ij}}{\sqrt{E_{ij}}}$$

Cells with $|r_{ij}| > 2$ are typically considered notably discrepant from independence.

Example: Chi-Square Test

Example 4.2. A researcher investigates whether exercise frequency is associated with self-reported health status among 200 adults.

	Poor Health	Good Health	Total
Rarely exercises	40	25	65
Sometimes exercises	30	45	75
Regularly exercises	15	45	60
Total	85	115	200

Expected frequencies under H_0 :

$$E_{11} = \frac{65 \times 85}{200} = 27.6, \quad E_{12} = \frac{65 \times 115}{200} = 37.4$$

$$E_{21} = \frac{75 \times 85}{200} = 31.9, \quad E_{22} = \frac{75 \times 115}{200} = 43.1$$

$$E_{31} = \frac{60 \times 85}{200} = 25.5, \quad E_{32} = \frac{60 \times 115}{200} = 34.5$$

Test statistic:

$$\chi^2 = \frac{(40 - 27.6)^2}{27.6} + \frac{(25 - 37.4)^2}{37.4} + \dots = 5.54 + 4.10 + 0.11 + 0.08 + 4.32 + 3.20 = 17.35$$

$$\text{df} = (3 - 1)(2 - 1) = 2, \quad \text{p-value} = P(\chi_2^2 > 17.35) < 0.001$$

Decision — two equivalent methods.

Method 1 — p-value.

$$p < 0.001 < \alpha = 0.05 \quad \Rightarrow \quad \text{Reject } H_0$$

Method 2 — critical value. The chi-square test is right-tailed, so:

$$\chi_{\text{crit}}^2 = \chi_{0.95, 2}^2 = \text{qchisq}(0.95, 2) = 5.991$$

Compare to the calculated statistic:

$$\chi_{\text{cal}}^2 = 17.35 > \chi_{\text{crit}}^2 = 5.991 \Rightarrow \text{Reject } H_0$$

Both methods agree — exercise frequency and health status are significantly associated ($\chi_{\text{cal}}^2 = 17.35$, $df = 2$, $p < 0.001$ — reported as $\chi^2(2) = 17.35$).

Standardized residuals reveal the pattern: “Rarely exercises / Poor Health” is notably higher than expected ($r = +2.35$), while “Regularly exercises / Poor Health” is notably lower ($r = -2.08$) — people who exercise regularly are less likely to report poor health.

R Example: Chi-Square Test of Independence

```
# --- Build the exercise × health contingency table ---
exercise_health <- matrix(
  c(40, 25, 30, 45, 15, 45),
  nrow = 3, byrow = TRUE,
  dimnames = list(
    Exercise = c("Rarely", "Sometimes", "Regularly"),
    Health = c("Poor", "Good")
  )
)

# Display observed table with margins
kable(addmargins(exercise_health),
      caption = "Observed Frequencies: Exercise × Health Status") |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE)
```

Table 1: Observed Frequencies: Exercise × Health Status

	Poor	Good	Sum
Rarely	40	25	65
Sometimes	30	45	75
Regularly	15	45	60
Sum	85	115	200

```
# --- Chi-square test ---
chi_result <- chisq.test(exercise_health, correct = FALSE)
print(chi_result)
```

Pearson's Chi-squared test

```
data: exercise_health
X-squared = 17.352, df = 2, p-value = 0.0001706
```

```
# --- Expected frequencies ---
kable(round(chi_result$expected, 2),
      caption = "Expected Frequencies Under H (Independence)") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
               full_width = FALSE)
```

Table 2: Expected Frequencies Under H (Independence)

	Poor	Good
Rarely	27.62	37.38
Sometimes	31.88	43.12
Regularly	25.50	34.50

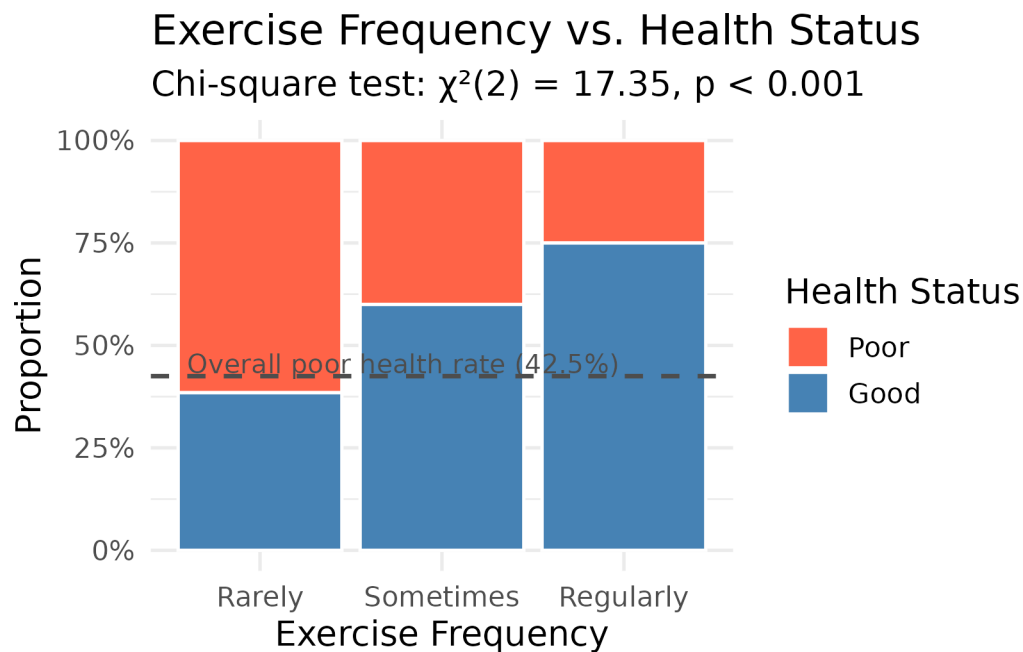
```
# --- Standardized residuals ---
kable(round(chi_result$residuals, 3),
      caption = "Standardized Residuals (|r| > 2 = notable departure)") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
               full_width = FALSE) |>
  column_spec(2, # Poor
             color = ifelse(abs(chi_result$residuals[, 1]) > 2, "tomato", "black")) |>
  column_spec(3, # Good
             color = ifelse(abs(chi_result$residuals[, 2]) > 2, "tomato", "black"))
```

Table 3: Standardized Residuals ($|r| > 2$ = notable departure)

	Poor	Good
Rarely	2.354	-2.024
Sometimes	-0.332	0.286
Regularly	-2.079	1.788

```
# --- Mosaic plot and association plot ---
# Convert to data frame for ggplot
eh_df <- as.table(exercise_health) |>
  as.data.frame(responseName = "Count")

# Proportional stacked bar
ggplot(eh_df, aes(x = Exercise, y = Count, fill = Health)) +
  geom_bar(stat = "identity", position = "fill", color = "white") +
  geom_hline(yintercept = 85/200, linetype = "dashed",
            color = "gray30", linewidth = 0.8) +
  annotate("text", x = 0.6, y = 85/200 + 0.03,
         label = "Overall poor health rate (42.5%)",
         color = "gray30", size = 3.5, hjust = 0) +
  scale_fill_manual(values = c("Poor" = "tomato", "Good" = "steelblue")) +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Exercise Frequency vs. Health Status",
       subtitle = paste0("Chi-square test:  $\chi^2(2) =$ ",
                        round(chi_result$statistic, 2),
                        ", p < 0.001"),
       x = "Exercise Frequency",
       y = "Proportion",
       fill = "Health Status") +
  theme_minimal(base_size = 13)
```



Code explanation:

- `chisq.test(table, correct = FALSE)` runs the chi-square test without Yates' correction. Set `correct = TRUE` for Yates' correction on 2×2 tables.
- The result object stores `$expected` (expected frequencies), `$stdres` (standardized residuals), and `$p.value`.
- The dashed horizontal line marks the overall “poor health” rate — bars above it indicate excess poor health relative to the population average, making deviations from independence visually clear.

Try modifying: Check `all(chi_result$expected >= 5)` to verify the minimum expected frequency assumption is met.

Exercises

Exercise 4.2

Use the `HairEyeColor` dataset built into R (hair color $\times$ eye color $\times$ sex for 592 statistics students).

- Collapse across sex: `margin.table(HairEyeColor, c(1,2))` to get a Hair $\times$ Eye contingency table.
- Run a chi-square test. Are hair and eye color independent?
- Compute and display standardized residuals. Which hair-eye combinations are most over- or under-represented?
- Produce a proportional stacked bar chart and interpret the pattern.

Exercise 4.3

A clinical study records treatment type (Drug A, Drug B, Placebo) and recovery outcome (Recovered, Not Recovered) for 150 patients.

- Create the contingency table in R with `matrix()`.
- Verify the minimum expected frequency assumption. If violated, what alternative test should you use?
- Run the chi-square test and report results in APA format: $\chi^2(df) = X.XX$, $p = .XXX$.

Fisher's Exact Test

Introduction

The chi-square test relies on a large-sample approximation — when expected cell frequencies are small (below 5), the χ^2 approximation becomes unreliable and p-values are inaccurate. **Fisher's Exact Test** solves this problem by computing the **exact** probability of observing the table, or one more extreme, under H_0 of independence. It was originally designed for 2×2 tables but extends to larger tables. In data science and biomedical research, small samples and rare events are common — Fisher's test is an essential tool.

Theory

The Hypergeometric Distribution

Fisher's test is based on the **hypergeometric distribution** — the exact probability of a specific 2×2 table with fixed marginal totals (R_1, R_2, C_1, C_2) :

$$P \left(\begin{bmatrix} a & b \\ c & d \end{bmatrix} \right) = \frac{\binom{R_1}{a} \binom{R_2}{c}}{\binom{N}{C_1}} = \frac{(R_1)!(R_2)!(C_1)!(C_2)!}{N! \cdot a! \cdot b! \cdot c! \cdot d!}$$

The p-value is the sum of probabilities for all tables as extreme as, or more extreme than, the observed table — while keeping marginal totals fixed.

Two-Sided Fisher's Test

For two-sided tests, “more extreme” is defined as tables with probability **less than or equal to** the probability of the observed table. R's `fisher.test()` implements this by default.

When to Use Fisher's Exact Test

Table 1: Fisher’s Exact Test usage guidelines

Situation	Recommendation
Any cell has $E_{ij} < 5$	Use Fisher’s
Total $N < 20$	Use Fisher’s
2×2 table, any sample size	Fisher’s is always valid (some prefer it over chi-square)
Large $r \times c$ table, large N	Chi-square is adequate

Example: Fisher’s Exact Test

Example 4.3. A rare disease study examines whether a genetic marker is associated with disease status. Only 14 subjects are available.

	Disease	No Disease	Total
Marker Present	5	2	7
Marker Absent	1	6	7
Total	6	8	14

Expected frequencies: $E_{11} = 6 \times 7/14 = 3.0$ — below 5, so chi-square is inappropriate.

Fisher’s exact p-value (two-sided): $p = 0.103$

Decision: Fail to reject H_0 at $\alpha = 0.05$. Despite the apparent pattern, there is insufficient evidence to conclude the marker is associated with disease — likely due to the very small sample size.

Interpretation note: This example illustrates an important limitation. The study is severely **underpowered** — a larger study might well find significance. Fisher’s test is honest about uncertainty; it does not manufacture significance from sparse data.

R Example: Fisher’s Exact Test

```
# --- Genetic marker × disease status ---
marker_disease <- matrix(
  c(5, 2, 1, 6),
  nrow      = 2, byrow  = TRUE,
  dimnames = list(
    Marker = c("Present", "Absent"),
    Disease = c("Disease", "No Disease")
  )
)
```

```
)
)

kable(addmargins(marker_disease),
      caption = "Observed: Genetic Marker × Disease Status") |>
  kable_styling(bootstrap_options = c("striped","hover"),
               full_width = FALSE)
```

Table 2: Observed: Genetic Marker × Disease Status

	Disease	No Disease	Sum
Present	5	2	7
Absent	1	6	7
Sum	6	8	14

```
# --- Check expected frequencies ---
chi_check <- chisq.test(marker_disease, correct = FALSE)
cat("Expected frequencies:\n")
```

Expected frequencies:

```
print(round(chi_check$expected, 2))
```

```
      Disease
Marker  Disease No Disease
Present      3         4
Absent       3         4
```

```
cat("\nMinimum expected frequency:",
    round(min(chi_check$expected), 2),
    "\n- chi-square assumption violated!\n\n")
```

Minimum expected frequency: 3 - chi-square assumption violated!

```
# --- Fisher's Exact Test ---
fisher_result <- fisher.test(marker_disease, alternative = "two.sided")
print(fisher_result)
```


Fisher's Exact Test for Count Data

```
data: marker_disease
p-value = 0.1026
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.7203391 817.9820107
sample estimates:
odds ratio
 11.71745
```

```
# --- Compare Fisher's vs Chi-square on same data ---
comparison <- data.frame(
  Test      = c("Chi-Square (no correction)",
                "Chi-Square (Yates' correction)",
                "Fisher's Exact Test"),
  Statistic = c(round(chisq.test(marker_disease,
                                correct=FALSE)$statistic, 3),
                round(chisq.test(marker_disease,
                                correct=TRUE)$statistic, 3),
                "-"),
  p_value   = c(round(chisq.test(marker_disease,
                                correct=FALSE)$p.value, 4),
                round(chisq.test(marker_disease,
                                correct=TRUE)$p.value, 4),
                round(fisher_result$p.value, 4)),
  Appropriate = c("No (E < 5)", "No (E < 5)", "Yes ")
)

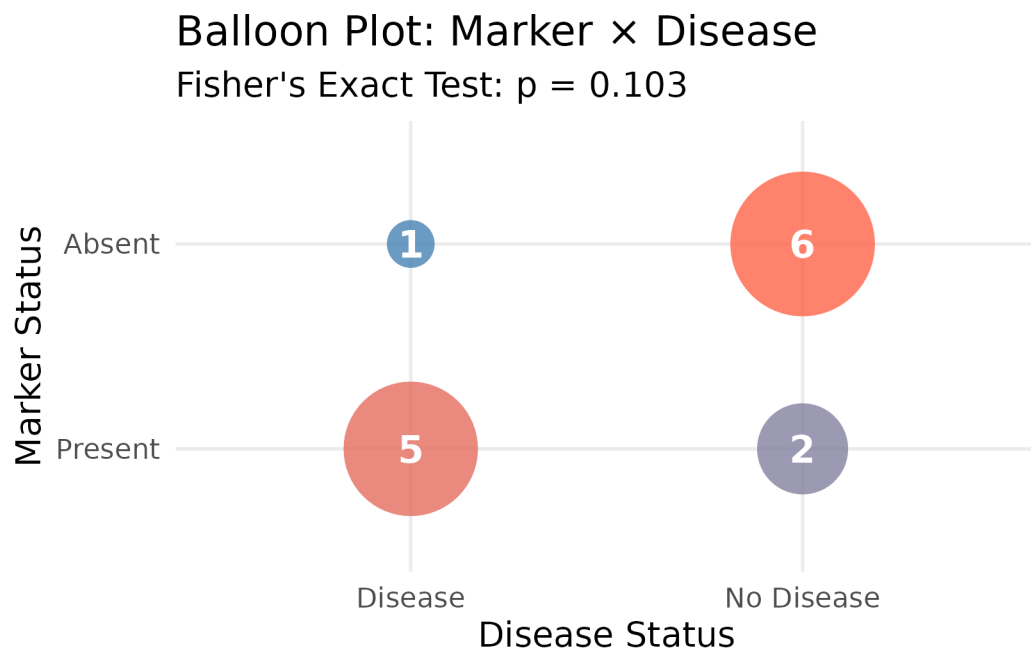
kable(comparison,
      caption = "Test Comparison: Small Sample Scenario",
      col.names = c("Test", "Statistic", "p-value", "Appropriate?")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(4, bold = TRUE,
            color = c("tomato", "tomato", "darkgreen"))
```

Table 3: Test Comparison: Small Sample Scenario

Test	Statistic	p-value	Appropriate?
Chi-Square (no correction)	4.667	0.0308	No (E < 5)
Chi-Square (Yates' correction)	2.625	0.1052	No (E < 5)
Fisher's Exact Test	—	0.1026	Yes

```
# --- Visualize with balloon plot ---
marker_df <- as.table(marker_disease) |>
  as.data.frame(responseName = "Count")

ggplot(marker_df, aes(x = Disease, y = Marker)) +
  geom_point(aes(size = Count, color = Count),
             shape = 16, alpha = 0.8) +
  geom_text(aes(label = Count), color = "white",
            fontface = "bold", size = 5) +
  scale_size_continuous(range = c(8, 25)) +
  scale_color_gradient(low = "steelblue", high = "tomato") +
  labs(title = "Balloon Plot: Marker × Disease",
       subtitle = paste0("Fisher's Exact Test: p = ",
                          round(fisher_result$p.value, 3)),
       x = "Disease Status",
       y = "Marker Status") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```



Code explanation:

- `fisher.test(table, alternative)` computes the exact p-value. For 2×2 tables it also returns an **odds ratio** with a confidence interval — a useful effect measure.
- The comparison table illustrates how chi-square p-values are unreliable with small expected frequencies — they differ substantially from Fisher's exact result.

- The balloon plot is an intuitive alternative to a contingency table for small tables: bubble area represents cell count, making the pattern immediately visible.

Exercises

Exercise 4.4

A new teaching intervention is tested in two small classrooms. Results:

	Passed	Failed
Intervention	8	2
Control	4	6

- Check whether chi-square assumptions are met.
- Run Fisher's Exact Test. What is the p-value and odds ratio?
- Interpret the odds ratio in plain language.
- What would a larger study need to show to confirm this result?

Cramér's V and Effect Size for Categorical Association

Introduction

A significant chi-square test only tells us that an association **exists** — it does not say how **strong** it is. With large samples, even trivially weak associations become statistically significant. **Effect size measures** for categorical variables answer the practical question: *How strongly are these two variables related?* The most versatile measure is **Cramér's V**, which standardizes chi-square to a 0–1 scale regardless of table dimensions.

Theory

Cramér's V

Cramér's V is derived from the chi-square statistic, normalized to fall between 0 and 1:

$$V = \sqrt{\frac{\chi^2/N}{\min(r-1, c-1)}}$$

where N is the total sample size, r is the number of rows, and c is the number of columns.

- $V = 0$: Perfect independence (no association)
- $V = 1$: Perfect association

Interpretation benchmarks (Cohen, 1988), adjusted for table size:

Table 1: Cramér's V interpretation by table dimensions

Table Size	Small	Medium	Large
2×2	0.10	0.30	0.50
3×3	0.07	0.21	0.35
4×4	0.06	0.17	0.29

The Phi Coefficient

For 2×2 tables specifically, the **phi coefficient** ϕ is a simpler equivalent:

$$\phi = \sqrt{\frac{\chi^2}{N}} = \frac{ad - bc}{\sqrt{(a+b)(c+d)(a+c)(b+d)}}$$

ϕ ranges from -1 to $+1$ for 2×2 tables (with sign indicating direction) and equals Cramér's V when $\min(r-1, c-1) = 1$.

Goodman and Kruskal's Lambda

An alternative measure of association is **Lambda** (λ), which measures the **proportional reduction in prediction error** — how much better we can predict one variable knowing the other:

$$\lambda = \frac{\sum_j \max_i f_{ij} - \max_i R_i}{N - \max_i R_i}$$

$\lambda = 0$ means knowing X does not help predict Y ; $\lambda = 1$ means perfect prediction.

Example: Cramér's V

Example 4.4. Returning to the exercise $\times$ health study (Example 4.2): $\chi^2 = 17.35$, $N = 200$, table is 3×2 .

$$V = \sqrt{\frac{17.35/200}{\min(3-1, 2-1)}} = \sqrt{\frac{0.0868}{1}} = \sqrt{0.0868} \approx 0.295$$

For a table with $\min(r-1, c-1) = 1$, benchmarks are 0.10/0.30/0.50 (small/medium/large). $V = 0.295$ is close to a **medium effect** — the association between exercise and health is statistically significant and of moderate practical importance.

R Example: Cramér's V and Association Measures

```
# --- Cramér's V for exercise × health ---
exercise_health <- matrix(
  c(40, 25, 30, 45, 15, 45),
  nrow = 3, byrow = TRUE,
  dimnames = list(
    Exercise = c("Rarely", "Sometimes", "Regularly"),
    Health   = c("Poor", "Good")
  )
)

chi_result <- chisq.test(exercise_health, correct = FALSE)
N          <- sum(exercise_health)
min_dim    <- min(nrow(exercise_health)-1, ncol(exercise_health)-1)
cramers_v  <- sqrt(chi_result$statistic / (N * min_dim))

cat("Chi-square statistic:", round(chi_result$statistic, 3), "\n")
```

Chi-square statistic: 17.352

```
cat("N:", N, "\n")
```

N: 200

```
cat("min(r-1, c-1):", min_dim, "\n")
```

min(r-1, c-1): 1

```
cat("Cramér's V:", round(cramers_v, 4), "\n")
```

Cramér's V: 0.2946

```
cat("Interpretation: Medium effect (benchmark ~0.30 for this table size)\n\n")
```

Interpretation: Medium effect (benchmark ~0.30 for this table size)

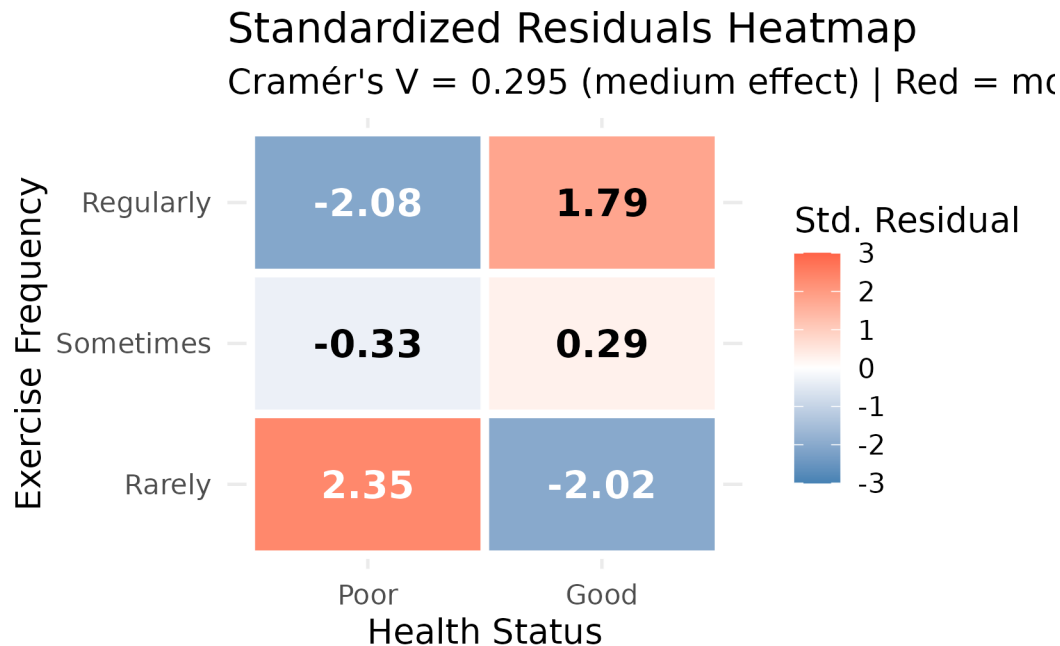
```
# --- Using vcd package for association measures ---
assoc_stats <- vcd::assocstats(exercise_health)
print(assoc_stats)
```

	X ²	df	P(> X ²)
Likelihood Ratio	17.694	2	0.00014384
Pearson	17.352	2	0.00017064

Phi-Coefficient : NA
Contingency Coeff.: 0.283
Cramer's V : 0.295

```
# --- Visualize standardized residuals as heatmap ---
stdres <- chi_result$residuals
stdres_df <- as.data.frame(as.table(stdres)) |>
  rename(Exercise = Exercise, Health = Health, Residual = Freq)

ggplot(stdres_df, aes(x = Health, y = Exercise, fill = Residual)) +
  geom_tile(color = "white", linewidth = 1.5) +
  geom_text(aes(label = round(Residual, 2)),
    size = 5, fontface = "bold",
    color = ifelse(abs(stdres_df$Residual) > 2,
      "white", "black")) +
  scale_fill_gradient2(low = "steelblue", mid = "white",
    high = "tomato", midpoint = 0,
    limits = c(-3, 3)) +
  labs(title = "Standardized Residuals Heatmap",
    subtitle = paste0("Cramér's V = ", round(cramers_v, 3),
      " (medium effect) | Red = more than expected, Blue = less"),
    x = "Health Status",
    y = "Exercise Frequency",
    fill = "Std. Residual") +
  theme_minimal(base_size = 13)
```



Code explanation:

- Cramér's V is computed manually here to show the formula clearly; `vcd::assocstats()` computes it (and phi, contingency coefficient) in one call.
- The heatmap of standardized residuals is more informative than a standard mosaic plot for larger tables — color immediately draws the eye to cells with the strongest departures from independence.
- `scale_fill_gradient2()` creates a diverging color scale centered at zero, perfect for signed residuals.

Exercises

💡 Exercise 4.5

Using the `HairEyeColor` chi-square result from Exercise 4.2:

- Compute Cramér's V manually from the χ^2 statistic.
- Use `vcd::assocstats()` to verify.
- Interpret the effect size using the appropriate benchmark for the table dimensions.
- Produce a standardized residuals heatmap and describe which hair-eye combinations drive the association.

Correlation Tests

Introduction

When both variables are continuous (or at least ordinal with many levels), **correlation** quantifies the strength and direction of their linear relationship. But a sample correlation r could arise by chance even when the true population correlation $\rho = 0$. A **correlation test** determines whether the observed correlation is statistically distinguishable from zero. In this section we cover the two most important correlation measures: **Pearson's** r for linear relationships between normally distributed variables, and **Spearman's** ρ for monotone relationships with no normality requirement.

Theory

Pearson Correlation Test

Hypotheses: $H_0 : \rho = 0$ vs. $H_1 : \rho \neq 0$ (or one-sided)

Given sample correlation r from n pairs, the test statistic is:

$$t = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}} \sim t(n-2) \text{ under } H_0$$

Assumptions:

1. Both variables are continuous.
2. The relationship is **linear**.
3. Both variables are approximately **normally distributed** (bivariate normality).
4. Observations are independent.
5. No extreme outliers (Pearson's r is sensitive to outliers).

Confidence interval for ρ : Use Fisher's z-transformation:

$$z = \frac{1}{2} \ln\left(\frac{1+r}{1-r}\right), \quad \text{SE}(z) = \frac{1}{\sqrt{n-3}}$$

Then transform back to the correlation scale.

Spearman Rank Correlation Test

Spearman's ρ replaces raw values with **ranks** and computes Pearson's r on the ranks. This makes it robust to:

- Non-normality
- Outliers
- Monotone but non-linear relationships
- Ordinal variables

$$\rho_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

where d_i is the difference in ranks for pair i . For large samples, a t-approximation is used for hypothesis testing.

Interpreting Correlation

Table 1: Correlation strength benchmarks

$ r $ or $ \rho $	Strength
0.00 – 0.19	Negligible
0.20 – 0.39	Weak
0.40 – 0.59	Moderate
0.60 – 0.79	Strong
0.80 – 1.00	Very strong

Critical Reminder: Correlation ≠ Causation

A significant correlation between X and Y does not imply that X causes Y , nor that Y causes X . Both could be driven by a third variable Z (confounding), or the association could be spurious. Always consider alternative explanations and visualize the scatterplot alongside the correlation coefficient.

Example: Pearson and Spearman Correlation Tests

Example 4.5. A researcher examines the relationship between hours of sleep per night (X) and cognitive performance score (Y) in 40 participants.

Results: $r = 0.61$, $t_{\text{cal}} = 4.73$, $df = 38$, $p < 0.001$ (reported as $t(38) = 4.73$), 95% CI: (0.37, 0.78).

Interpretation: There is a statistically significant, strong positive linear correlation between sleep duration and cognitive performance ($r = 0.61$, $p < 0.001$). The 95% CI (0.37, 0.78) excludes zero, confirming the association. However, this is an observational study — we cannot conclude that more sleep **causes** better performance.

R Example: Pearson and Spearman Correlation Tests

```
# --- Use mtcars: test correlation between hp and mpg ---
data(mtcars)

# Pearson correlation test
pearson_result <- cor.test(mtcars$hp, mtcars$mpg,
                           method      = "pearson",
                           alternative = "two.sided")

# Spearman correlation test
spearman_result <- cor.test(mtcars$hp, mtcars$mpg,
                            method      = "spearman",
                            alternative = "two.sided",
                            exact       = FALSE)

# Summary table
cor_summary <- data.frame(
  Method      = c("Pearson", "Spearman"),
  r_or_rho    = round(c(pearson_result$estimate,
                        spearman_result$estimate), 4),
  Statistic    = round(c(pearson_result$statistic,
                        spearman_result$statistic), 3),
  df           = c(pearson_result$parameter, "-"),
  p_value      = round(c(pearson_result$p.value,
                        spearman_result$p.value), 6),
  CI_lower     = c(round(pearson_result$conf.int[1], 3), "-"),
  CI_upper     = c(round(pearson_result$conf.int[2], 3), "-")
)

kable(cor_summary,
      caption    = "Correlation Test Results: Horsepower vs. MPG",
      col.names  = c("Method", "r / ", "t / S",
                    "df", "p-value", "95% CI Lower", "95% CI Upper")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE)
```

Table 2: Correlation Test Results: Horsepower vs. MPG

	Method	r /	t / S	df	p-value	95% CI Lower	95% CI Upper
cor	Pearson	-0.7762	-6.742	30	0	-0.885	-0.586
rho	Spearman	-0.8947	10337.290	—	0	—	—

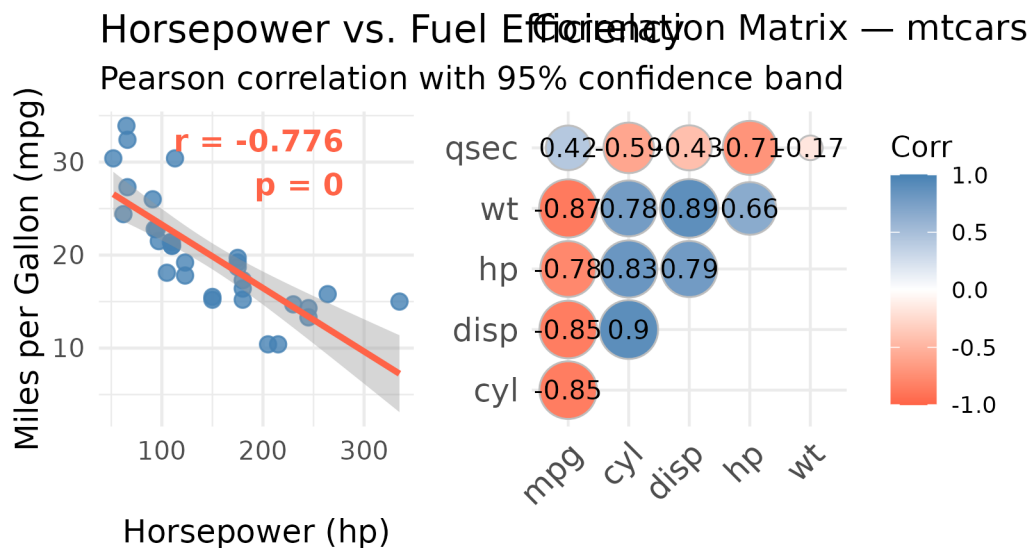
```
# --- Scatterplot with correlation annotation ---
r_val <- round(pearson_result$estimate, 3)
p_val <- round(pearson_result$p.value, 4)

p1 <- ggplot(mtcars, aes(x = hp, y = mpg)) +
  geom_point(color = "steelblue", size = 2.5, alpha = 0.8) +
  geom_smooth(method = "lm", color = "tomato",
              se = TRUE, linewidth = 1.1) +
  annotate("text", x = 280, y = 30,
           label = paste0("r = ", r_val,
                          "\np = ", p_val),
           color = "tomato", fontface = "bold",
           size = 4, hjust = 1) +
  labs(title = "Horsepower vs. Fuel Efficiency",
       subtitle = "Pearson correlation with 95% confidence band",
       x = "Horsepower (hp)",
       y = "Miles per Gallon (mpg)") +
  theme_minimal(base_size = 12)

# --- Full correlation matrix for mtcars ---
cont_vars <- mtcars[, c("mpg", "cyl", "disp", "hp", "wt", "qsec")]
cor_mat <- cor(cont_vars, method = "pearson")

p2 <- ggcorrplot(cor_mat,
                 method = "circle",
                 type = "upper",
                 lab = TRUE,
                 lab_size = 3.5,
                 colors = c("tomato", "white", "steelblue"),
                 title = "Correlation Matrix - mtcars",
                 ggtheme = theme_minimal(base_size = 11))

p1 + p2
```



Code explanation:

- `cor.test(x, y, method, alternative)` tests both Pearson ("pearson") and Spearman ("spearman") correlations. Pearson returns a CI; Spearman does not (due to the rank-based nature).
- `exact = FALSE` in Spearman uses a normal approximation — necessary when ties are present in the data.
- `ggcorrplot()` produces a publication-ready correlation matrix. `type = "upper"` shows only the upper triangle to avoid redundancy.

Exercises

💡 Exercise 4.6

Using the `iris` dataset, test the correlation between `Sepal.Length` and `Petal.Length`.

- Produce a scatterplot. Does the relationship appear linear?
- Check normality of both variables (Shapiro-Wilk). Which correlation method is more appropriate?
- Run both Pearson and Spearman tests. Compare and explain any differences.
- Compute the full 4×4 correlation matrix for all numeric variables and visualize with `ggcorrplot()`.

💡 Exercise 4.7

The **anscombe** dataset contains four pairs of variables with nearly identical correlations but very different scatterplot shapes.

- (a) Compute Pearson's r for all four pairs (x_1/y_1 , x_2/y_2 , x_3/y_3 , x_4/y_4).
- (b) Plot all four scatterplots in a 2×2 grid.
- (c) For which pairs is Pearson's r a misleading summary? What would be more appropriate?

Point-Biserial and Phi Coefficients

Introduction

Not all association problems involve two continuous or two categorical variables. In data science, binary variables are ubiquitous — pass/fail, churn/retain, click/no-click, disease/healthy. When one or both variables in a pair are binary, specialized correlation measures maintain interpretability while remaining mathematically consistent with the general correlation framework. The **point-biserial correlation** handles the binary-continuous case; the **phi coefficient** handles the binary-binary case.

Theory

Point-Biserial Correlation (r_{pb})

The **point-biserial correlation** measures the association between a **naturally binary variable** (coded 0/1) and a **continuous variable**. It is mathematically equivalent to Pearson's r computed between the 0/1 binary variable and the continuous variable:

$$r_{pb} = \frac{\bar{Y}_1 - \bar{Y}_0}{s_Y} \sqrt{\frac{n_1 n_0}{n^2}}$$

where $\bar{Y}_1$ and $\bar{Y}_0$ are the means of the continuous variable in the two groups, s_Y is the overall standard deviation, and n_1 , n_0 are the group sizes.

Significance testing: Because r_{pb} is algebraically equivalent to Pearson's r , the same t-test applies:

$$t = \frac{r_{pb} \sqrt{n-2}}{\sqrt{1-r_{pb}^2}} \sim t(n-2)$$

This is equivalent to running an **independent samples t-test** comparing the two groups — the same information, expressed differently.

Assumptions: The continuous variable should be approximately normally distributed within each group (same as the independent samples t-test).

Phi Coefficient (ϕ)

The **phi coefficient** measures association between **two binary variables**. It is Pearson's r applied to two 0/1 variables:

$$\phi = \frac{ad - bc}{\sqrt{(a+b)(c+d)(a+c)(b+d)}}$$

where a, b, c, d are the four cells of a 2×2 contingency table. ϕ ranges from -1 to $+1$ and equals Cramér's V for 2×2 tables.

Interpretation benchmarks (same as Pearson's r):

- $|\phi| < 0.10$: Negligible
- $0.10 \leq |\phi| < 0.30$: Weak
- $0.30 \leq |\phi| < 0.50$: Moderate
- $|\phi| \geq 0.50$: Strong

Biserial vs. Point-Biserial

A subtle distinction: the **point-biserial** is used when the binary variable is *truly* dichotomous (e.g., sex, treatment assignment). The **biserial correlation** is used when the binary variable is an artificially dichotomized continuous variable (e.g., pass/fail from a continuous score). The biserial correlation is larger in magnitude and corrects for the information lost by dichotomization.

Example: Point-Biserial Correlation

Example 4.6. A study records whether students passed a preparatory course (0 = No, 1 = Yes) and their final exam score. Results: $r_{pb} = 0.48$, $t_{\text{cal}} = 4.19$, $df = 58$, $p < 0.001$ (reported as $t(58) = 4.19$).

Interpretation: Students who passed the preparatory course scored significantly higher on the final exam ($r_{pb} = 0.48$, $p < 0.001$). This is a moderate-to-strong effect, indicating the preparatory course is associated with better exam performance. Note that this is equivalent to a two-sample t-test comparing exam scores between the two groups.

R Example: Point-Biserial and Phi Coefficients


```
# --- Point-biserial: transmission (binary) vs. mpg (continuous) ---
# am: 0 = automatic, 1 = manual
data(mtcars)

# Method 1: cor.test (Pearson on binary × continuous)
pb_result <- cor.test(mtcars$am, mtcars$mpg,
                      method = "pearson")

cat("=== Point-Biserial Correlation ===\n")
```

```
=== Point-Biserial Correlation ===
```

```
cat("r_pb:", round(pb_result$estimate, 4), "\n")
```

```
r_pb: 0.5998
```

```
cat("t:   ", round(pb_result$statistic, 3), "\n")
```

```
t:     4.106
```

```
cat("df:   ", pb_result$parameter, "\n")
```

```
df:    30
```

```
cat("p:     ", round(pb_result$p.value, 4), "\n")
```

```
p:     3e-04
```

```
cat("95% CI: [", round(pb_result$conf.int[1], 3),
    ",", round(pb_result$conf.int[2], 3), "]\n\n")
```

```
95% CI: [ 0.318 , 0.784 ]
```

```
# Equivalence to t-test
t_equiv <- t.test(mpg ~ am, data = mtcars, var.equal = FALSE)
cat("Equivalent Welch's t-test p-value:",
    round(t_equiv$p.value, 4), "(same conclusion)\n")
```

```
Equivalent Welch's t-test p-value: 0.0014 (same conclusion)
```

```
# --- Phi coefficient: am (auto/manual) × efficiency (high/low) ---
mtcars_bin <- mtcars |>
  mutate(
    transmission = factor(am, labels = c("Auto","Manual")),
    efficiency    = factor(ifelse(mpg >= median(mpg),
                                   "High","Low"),
                           levels = c("Low","High"))
  )

tab_2x2 <- table(mtcars_bin$transmission,
                 mtcars_bin$efficiency)

# Phi coefficient
chi_2x2 <- chisq.test(tab_2x2, correct = FALSE)
phi_val  <- sqrt(chi_2x2$statistic / sum(tab_2x2))

cat("=== Phi Coefficient ===\n")
```

```
=== Phi Coefficient ===
```

```
kable(addmargins(tab_2x2),
      caption = "Transmission × Efficiency (2×2 Table)" |>
      kable_styling(bootstrap_options = c("striped","hover"),
                    full_width = FALSE)
```

Table 1: Transmission × Efficiency (2×2 Table)

	Low	High	Sum
Auto	13	6	19
Manual	2	11	13
Sum	15	17	32

```
cat("Phi coefficient:", round(phi_val, 4), "\n")
```

```
Phi coefficient: 0.522
```

```
cat("Chi-square p-value:", round(chi_2x2$p.value, 4), "\n")
```

```
Chi-square p-value: 0.0031
```

```
cat("Interpretation: Moderate-to-strong association\n")
```

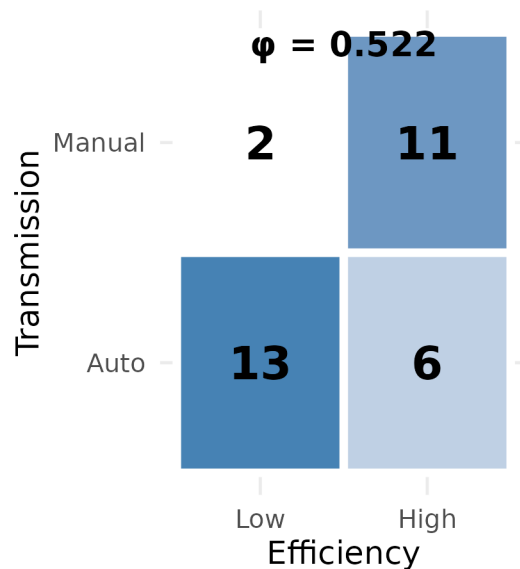
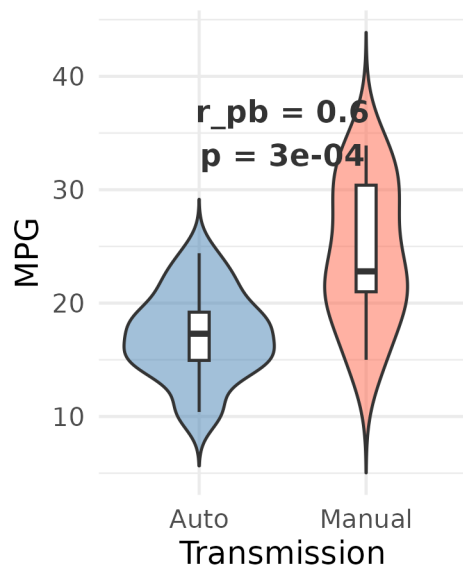
Interpretation: Moderate-to-strong association

```
# --- Visualize point-biserial ---
p1 <- ggplot(mtcars_bin, aes(x = transmission,
                             y = mpg, fill = transmission)) +
  geom_violin(alpha = 0.5, trim = FALSE) +
  geom_boxplot(width = 0.12, fill = "white") +
  scale_fill_manual(values = c("Auto" = "steelblue",
                                "Manual" = "tomato")) +
  annotate("text", x = 1.5, y = 35,
            label = paste0("r_pb = ",
                           round(pb_result$estimate, 3),
                           "\np = ", round(pb_result$p.value, 4)),
            size = 4, fontface = "bold", color = "gray20") +
  labs(title = "A. Point-Biserial: Transmission vs. MPG",
        x = "Transmission", y = "MPG") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

# --- Visualize phi ---
p2 <- as.data.frame(tab_2x2) |>
  rename(Transmission = Var1, Efficiency = Var2) |>
  ggplot(aes(x = Efficiency, y = Transmission,
              fill = Freq)) +
  geom_tile(color = "white", linewidth = 1.5) +
  geom_text(aes(label = Freq), size = 6, fontface = "bold") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  annotate("text", x = 1.5, y = 2.45,
            label = paste0("φ = ", round(phi_val, 3)),
            size = 4.5, fontface = "bold") +
  labs(title = "B. Phi Coefficient: Transmission × Efficiency",
        x = "Efficiency", y = "Transmission") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

p1 + p2
```

A. Point-Biserial: Transmission vs. MPG



Code explanation:

- `cor.test(binary, continuous, method = "pearson")` correctly computes the point-biserial correlation — no special function is needed since it is mathematically Pearson's r .
- Phi is computed as $\sqrt{\chi^2/N}$ — identical to Cramér's V for 2×2 tables.
- The equivalence between point-biserial and the independent samples t-test is shown explicitly — both tests use the same information and yield the same p-value (up to the choice of equal/unequal variances).

Exercises

💡 Exercise 4.8

Using the `mtcars` dataset:

- Compute the point-biserial correlation between `vs` (engine shape: 0 = V-shaped, 1 = straight) and `qsec` (quarter-mile time).
- Verify this equals the Pearson correlation between the two variables.
- Is there a significant association? Report in full.
- Compute the phi coefficient between `vs` and `am` (both binary). What is the strength of this association?

Partial Correlation

Introduction

Simple bivariate correlation measures the relationship between two variables, but in real datasets variables are rarely isolated — many are influenced by common underlying factors. **Partial correlation** measures the association between two variables X and Y **after removing the influence of one or more control variables** Z . It answers: *Is the relationship between X and Y real, or is it merely a reflection of both being related to Z ?* This is a crucial step in distinguishing genuine associations from spurious ones, and a natural bridge toward multiple regression (Chapter 10).

Theory

First-Order Partial Correlation

The **first-order partial correlation** between X and Y controlling for Z is:

$$r_{XY \cdot Z} = \frac{r_{XY} - r_{XZ} \cdot r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}$$

where r_{XY} , r_{XZ} , r_{YZ} are the zero-order (simple) Pearson correlations.

Interpretation: $r_{XY \cdot Z}$ is the correlation between the **residuals** of regressing X on Z and the **residuals** of regressing Y on Z — it is the “pure” relationship between X and Y once the linear effect of Z is removed from both.

Significance Testing

The partial correlation is tested using a t-distribution with $n - 2 - k$ degrees of freedom, where k is the number of control variables:

$$t = \frac{r_{XY \cdot Z} \sqrt{n - 2 - k}}{\sqrt{1 - r_{XY \cdot Z}^2}} \sim t(n - 2 - k)$$

Partial vs. Semi-Partial Correlation

A related concept is the **semi-partial (part) correlation** $r_{X(Y \cdot Z)}$: the correlation between X and the part of Y that is **unique** (not shared with Z). It measures the unique contribution of Y to X , which is central to multiple regression (it is the square root of the ΔR^2 when Y is added to a regression already containing Z).

Spurious Correlation

A **spurious correlation** occurs when X and Y are correlated only because both are caused by a third variable Z . After partialling out Z , the partial correlation drops to (near) zero. Classic example: ice cream sales and drowning rates are positively correlated — both are caused by hot weather (the confounder). The partial correlation controlling for temperature would be near zero.

Example: Partial Correlation

Example 4.7. In the `mtcars` dataset, `mpg` and `hp` are strongly negatively correlated ($r = -0.78$). But both are also related to car weight (`wt`): heavier cars tend to have more horsepower and worse fuel economy. Is the `mpg-hp` correlation “real” or partly a reflection of their shared relationship with `wt`?

- $r_{\text{mpg, hp}} = -0.78$
- $r_{\text{mpg, wt}} = -0.87$
- $r_{\text{hp, wt}} = +0.66$

Partial correlation controlling for `wt`:

$$r_{\text{mpg, hp} \cdot \text{wt}} = \frac{-0.78 - (-0.87)(0.66)}{\sqrt{(1 - 0.87^2)(1 - 0.66^2)}} = \frac{-0.78 + 0.574}{\sqrt{0.243 \times 0.564}} = \frac{-0.206}{0.370} \approx -0.557$$

The partial correlation (-0.56) is notably weaker than the zero-order correlation (-0.78), suggesting that part of the `mpg-hp` relationship is mediated through vehicle weight. However, a meaningful negative relationship remains even after controlling for weight.

R Example: Partial Correlation

```
# --- Partial correlations in mtcars ---
data(mtcars)
```

```
# Zero-order correlations (for context)
cat("=== Zero-Order Correlations ===\n")
```

```
=== Zero-Order Correlations ===
```

```
cor_mat <- cor(mtcars[, c("mpg","hp","wt","disp")])
round(cor_mat, 3)
```

	mpg	hp	wt	disp
mpg	1.000	-0.776	-0.868	-0.848
hp	-0.776	1.000	0.659	0.791
wt	-0.868	0.659	1.000	0.888
disp	-0.848	0.791	0.888	1.000

```
# --- Partial correlation: mpg ~ hp controlling for wt ---
library(ppcor)
```

```
# First-order partial correlation (controlling for one variable)
partial_result <- pcor.test(mtcars$mpg, mtcars$hp, mtcars$wt)

cat("=== Partial Correlation: mpg & hp | wt ===\n")
```

```
=== Partial Correlation: mpg & hp | wt ===
```

```
cat("Zero-order r(mpg, hp):      ",
    round(cor(mtcars$mpg, mtcars$hp), 4), "\n")
```

```
Zero-order r(mpg, hp):      -0.7762
```

```
cat("Partial r(mpg, hp | wt):    ",
    round(partial_result$estimate, 4), "\n")
```

```
Partial r(mpg, hp | wt):    -0.547
```

```
cat("t-statistic:                ",
    round(partial_result$statistic, 3), "\n")
```

```
t-statistic:                -3.519
```

```
cat("df:                                ", partial_result$n - 2 - 1, "\n")
```

```
df:                                29
```

```
cat("p-value:                                ",  
    round(partial_result$p.value, 4), "\n\n")
```

```
p-value:                                0.0015
```

```
# Full partial correlation matrix (controlling for wt)  
pcor_matrix <- pcor(mtcars[, c("mpg","hp","disp","qsec")],  
                    method = "pearson")  
  
cat("=== Partial Correlation Matrix (all pairs, no controls) ===\n")
```

```
=== Partial Correlation Matrix (all pairs, no controls) ===
```

```
round(pcor_matrix$estimate, 3)
```

	mpg	hp	disp	qsec
mpg	1.000	-0.342	-0.568	-0.154
hp	-0.342	1.000	0.393	-0.668
disp	-0.568	0.393	1.000	0.151
qsec	-0.154	-0.668	0.151	1.000

```
# --- Compare zero-order vs partial correlation visually ---  
# Compute residuals of mpg and hp after removing wt's influence  
resid_mpg <- residuals(lm(mpg ~ wt, data = mtcars))  
resid_hp  <- residuals(lm(hp ~ wt, data = mtcars))  
  
p1 <- ggplot(mtcars, aes(x = hp, y = mpg)) +  
  geom_point(color = "steelblue", size = 2.5, alpha = 0.8) +  
  geom_smooth(method = "lm", color = "tomato",  
              se = FALSE, linewidth = 1) +  
  annotate("text", x = 280, y = 31,  
           label = paste0("r = ",  
                           round(cor(mtcars$hp, mtcars$mpg), 3)),  
           color = "tomato", fontface = "bold", size = 4) +  
  labs(title = "A. Zero-Order Correlation",  
        subtitle = "mpg vs. hp (no control)",
```



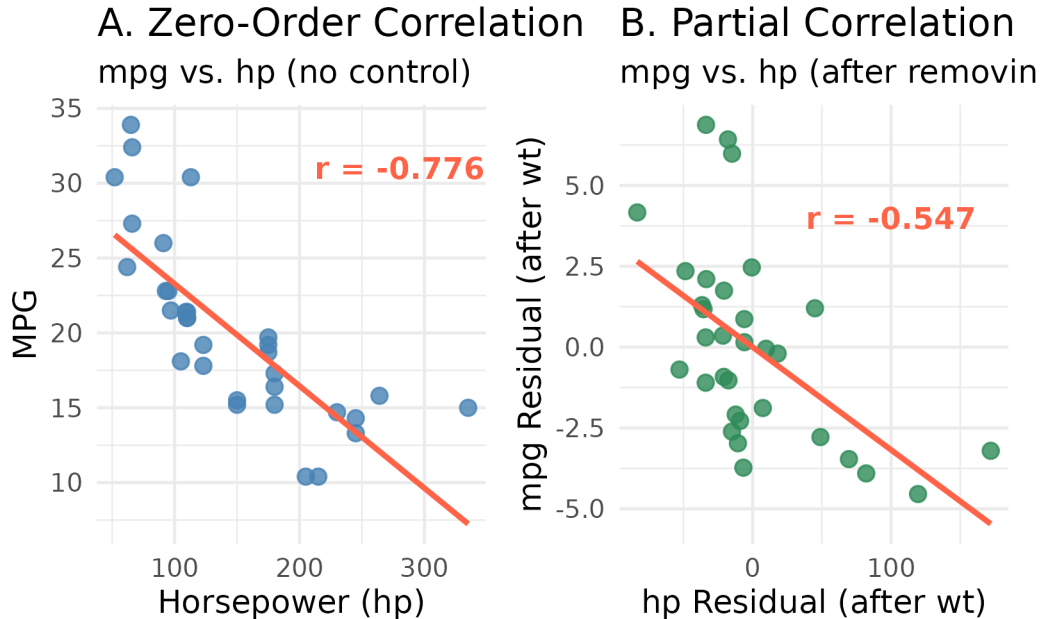
```

    x = "Horsepower (hp)", y = "MPG") +
  theme_minimal(base_size = 12)

p2 <- ggplot(data.frame(res_hp = resid_hp,
                        res_mpg = resid_mpg),
            aes(x = res_hp, y = res_mpg)) +
  geom_point(color = "seagreen", size = 2.5, alpha = 0.8) +
  geom_smooth(method = "lm", color = "tomato",
             se = FALSE, linewidth = 1) +
  annotate("text", x = 100, y = 4,
         label = paste0("r = ",
                        round(cor(resid_hp, resid_mpg), 3)),
         color = "tomato", fontface = "bold", size = 4) +
  labs(title = "B. Partial Correlation",
       subtitle = "mpg vs. hp (after removing wt influence)",
       x = "hp Residual (after wt)",
       y = "mpg Residual (after wt)") +
  theme_minimal(base_size = 12)

p1 + p2

```



Code explanation:

- `pcor.test(x, y, z)` from `ppcor` computes the first-order partial correlation between `x` and `y` controlling for `z`, with a significance test.

- `pcor(data_frame)` computes the full partial correlation matrix (each pair controlled for all others) — analogous to `cor()` for zero-order correlations.
- Panels A and B illustrate the concept geometrically: Panel B shows the scatterplot of **residuals** — the pure association between `mpg` and `hp` after both have been “de-weighted” by `wt`.

Try modifying: Control for both `wt` and `disp` simultaneously using `pcor.test(mpg, hp, cbind(wt, disp))`. Does the partial correlation weaken further?

Exercises

💡 Exercise 4.9

Using the `iris` dataset, `Sepal.Length` and `Petal.Length` are strongly correlated ($r \approx 0.87$). `Species` is a potential confounder.

- Compute the zero-order Pearson correlation between `Sepal.Length` and `Petal.Length`.
- Encode `Species` as a numeric variable (1, 2, 3) and compute the partial correlation controlling for it using `pcor.test()`.
- Does the partial correlation differ substantially from the zero-order correlation? What does this suggest?
- Visualize both zero-order and partial relationships as scatterplots (color points by species in panel A).

💡 Exercise 4.10 (Challenge)

Using `mtcars`, construct a full analysis comparing zero-order and partial correlations for the variable pair (`wt`, `mpg`), controlling for `cyl` (number of cylinders).

- Compute zero-order $r(\text{wt}, \text{mpg})$ and partial $r(\text{wt}, \text{mpg} \mid \text{cyl})$.
- Do the correlations differ? What does this tell us about the role of engine size?
- Produce a 2-panel residual scatterplot similar to the chapter example.
- Write a 150-word research summary reporting both correlations in APA format.

Chapter Lab Activity: Exploring Independence with the titanic and mtcars Datasets

Objectives

In this lab you will apply the full spectrum of independence tests to real data — choosing the correct test based on variable types, checking assumptions, reporting effect sizes, and visualizing associations. You will work with the built-in `Titanic` dataset (categorical variables) and `mtcars` (mixed variable types).

The Datasets

```
# --- Titanic dataset (built-in) ---
titanic_df <- as.data.frame(Titanic)
titanic_expanded <- titanic_df[rep(seq_len(nrow(titanic_df)),
                                titanic_df$Freq), -5]
rownames(titanic_expanded) <- NULL

kable(head(titanic_expanded, 10),
      caption = "Titanic Dataset (first 10 rows)" |>
      kable_styling(bootstrap_options = c("striped", "hover"),
                    font_size = 11)
```

Table 1: Titanic Dataset (first 10 rows)

Class	Sex	Age	Survived
3rd	Male	Child	No
3rd	Male	Child	No
3rd	Male	Child	No
3rd	Male	Child	No
3rd	Male	Child	No

3rd	Male	Child	No
3rd	Male	Child	No
3rd	Male	Child	No
3rd	Male	Child	No
3rd	Male	Child	No

```
cat("Total passengers:", nrow(titanic_expanded), "\n")
```

Total passengers: 2201

```
cat("Variables:", names(titanic_expanded), "\n")
```

Variables: Class Sex Age Survived

Lab Task 1: Survival × Class — Chi-Square Test

```
# Contingency table: Class × Survived
class_surv <- table(titanic_expanded$Class,
                    titanic_expanded$Survived)

kable(addmargins(class_surv),
      caption = "Passenger Class × Survival Status") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
                full_width = FALSE)
```

Table 2: Passenger Class × Survival Status

	No	Yes	Sum
1st	122	203	325
2nd	167	118	285
3rd	528	178	706
Crew	673	212	885
Sum	1490	711	2201

```
# Chi-square test
chi_titanic <- chisq.test(class_surv, correct = FALSE)
cat("\nChi-Square Test Results:\n")
```

Chi-Square Test Results:

```
cat("  $\chi^2$ (", chi_titanic$parameter, ") =",
    round(chi_titanic$statistic, 2), "\n")
```

$\chi^2(3) = 190.4$

```
cat("p-value:", round(chi_titanic$p.value, 6), "\n")
```

p-value: 0

```
cat("All expected  $\geq 5$ :", all(chi_titanic$expected >= 5), "\n")
```

All expected ≥ 5 : TRUE

```
# Cramér's V
N_t      <- sum(class_surv)
min_dim  <- min(nrow(class_surv)-1, ncol(class_surv)-1)
v_t      <- sqrt(chi_titanic$statistic / (N_t * min_dim))
cat("Cramér's V:", round(v_t, 4), "\n")
```

Cramér's V: 0.2941

Lab Task 2: Survival $\times$ Sex — Fisher's Exact Test

```
# Sex  $\times$  Survived (2 $\times$ 2 table)
sex_surv <- table(titanic_expanded$Sex,
                  titanic_expanded$Survived)

kable(addmargins(sex_surv),
      caption = "Sex  $\times$  Survival Status") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
               full_width = FALSE)
```

Table 3: Sex $\times$ Survival Status

	No	Yes	Sum
Male	1364	367	1731
Female	126	344	470

Sum	1490	711	2201
-----	------	-----	------

```
# Both chi-square and Fisher's for comparison
chi_sex    <- chisq.test(sex_surv, correct = FALSE)
fisher_sex <- fisher.test(sex_surv)

phi_sex <- sqrt(chi_sex$statistic / sum(sex_surv))

cat("Chi-Square:   $\chi^2(1)$  =", round(chi_sex$statistic, 2),
    ", p =", round(chi_sex$p.value, 6), "\n")
```

Chi-Square: $\chi^2(1)$ = 456.87 , p = 0

```
cat("Fisher's Exact: p =", round(fisher_sex$p.value, 6), "\n")
```

Fisher's Exact: p = 0

```
cat("Odds Ratio:", round(fisher_sex$estimate, 3),
    " 95% CI: [",
    round(fisher_sex$conf.int[1], 3), ",",
    round(fisher_sex$conf.int[2], 3), "]\n")
```

Odds Ratio: 10.132 95% CI: [7.977 , 12.929]

```
cat("Phi coefficient:", round(phi_sex, 4), "\n")
```

Phi coefficient: 0.4556

Lab Task 3: Correlation Analysis — mtcars

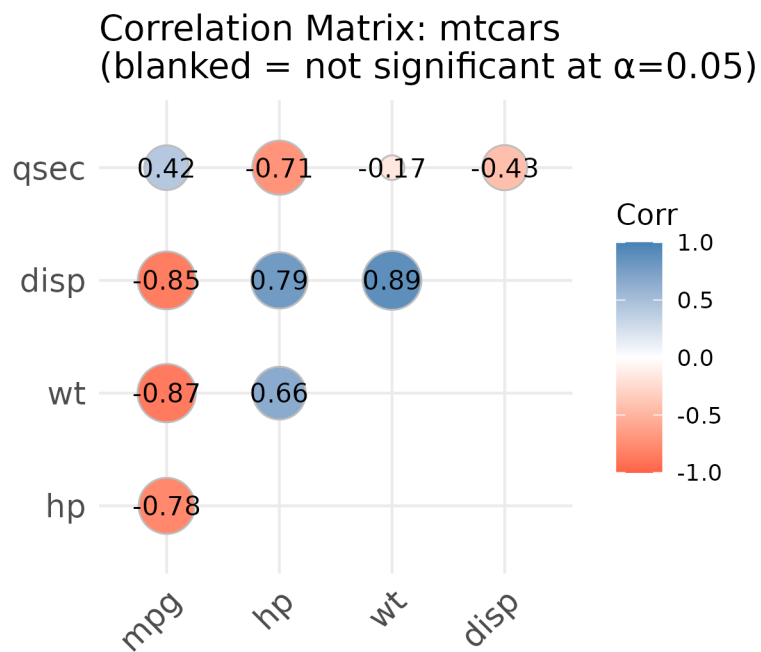
```
# Full correlation matrix with significance tests
data(mtcars)
cont_vars <- mtcars[, c("mpg", "hp", "wt", "disp", "qsec")]

# Correlation matrix
cor_mat <- cor(cont_vars)
#p_mat    <- cor_pmat(cont_vars)  # p-value matrix from ggcorrplot
ggcorrplot(cor_mat,
```

```

method    = "circle",
type      = "upper",
lab       = TRUE,
sig.level = 0.05,
lab_size  = 3.5,
insig     = "blank",
colors    = c("tomato","white","steelblue"),
title     = "Correlation Matrix: mtcars\n(blank = not significant at  $\alpha=0.05$ )",
ggtheme   = theme_minimal(base_size = 11))

```



Lab Task 4: Partial Correlation — Controlling for Weight

```

# Compare zero-order vs partial correlations
pairs_to_test <- list(
  c("mpg", "hp"),
  c("mpg", "disp"),
  c("hp", "disp")
)

partial_summary <- map_dfr(pairs_to_test, function(pair) {
  x <- mtcars[[pair[1]]]
  y <- mtcars[[pair[2]]]
  z <- mtcars$wt

```

```

r0 <- cor(x, y)
res <- pcor.test(x, y, z)

data.frame(
  Pair          = paste(pair[1], "~", pair[2]),
  Zero_order_r  = round(r0, 4),
  Partial_r_wt  = round(res$estimate, 4),
  p_value       = round(res$p.value, 4),
  Change        = round(res$estimate - r0, 4)
)
})

kable(partial_summary,
      caption = "Zero-Order vs. Partial Correlations (controlling for wt)",
      col.names = c("Variable Pair", "Zero-Order r",
                     "Partial r | wt", "p-value",
                     "Change in r")) |>
kable_styling(bootstrap_options = c("striped", "hover")) |>
column_spec(5, bold = TRUE,
            color = ifelse(abs(partial_summary$Change) > 0.1,
                           "tomato", "gray40"))

```

Table 4: Zero-Order vs. Partial Correlations (controlling for wt)

Variable Pair	Zero-Order r	Partial r | wt	p-value	Change in r
mpg ~ hp	-0.7762	-0.5470	0.0015	0.2292
mpg ~ disp	-0.8476	-0.3372	0.0636	0.5104
hp ~ disp	0.7909	0.5954	0.0004	-0.1956

Lab Task 5: Comprehensive Visualization

```

# Panel A: Survival rates by class (Titanic)
p1 <- as.data.frame(class_surv) |>
  rename(Class = Var1, Survived = Var2) |>
  group_by(Class) |>
  mutate(pct = Freq / sum(Freq)) |>
  ggplot(aes(x = Class, y = pct, fill = Survived)) +
  geom_col(color = "white") +
  scale_fill_manual(values = c("No" = "tomato",
                               "Yes" = "steelblue")) +

```



```

scale_y_continuous(labels = scales::percent) +
labs(title = "A. Survival Rate by Passenger Class",
      subtitle = paste0("Cramér's V = ", round(v_t, 3)),
      x = "Class", y = "Proportion") +
theme_minimal(base_size = 11)

# Panel B: Survival by sex
p2 <- as.data.frame(sex_surv) |>
  rename(Sex = Var1, Survived = Var2) |>
  group_by(Sex) |>
  mutate(pct = Freq / sum(Freq)) |>
  ggplot(aes(x = Sex, y = pct, fill = Survived)) +
  geom_col(color = "white") +
  scale_fill_manual(values = c("No" = "tomato",
                                "Yes" = "steelblue")) +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "B. Survival Rate by Sex",
        subtitle = paste0(" = ", round(phi_sex, 3),
                           " | OR = ",
                           round(fisher_sex$estimate, 2)),
        x = "Sex", y = "Proportion") +
  theme_minimal(base_size = 11)

# Panel C: Standardized residuals heatmap (Class × Survived)
stdres_df <- as.data.frame(chi_titanic$stdres) |>
  rename(Class = Var1, Survived = Var2, Residual = Freq)

p3 <- ggplot(stdres_df,
             aes(x = Survived, y = Class, fill = Residual)) +
  geom_tile(color = "white", linewidth = 1.2) +
  geom_text(aes(label = round(Residual, 2)),
            fontface = "bold", size = 4.5,
            color = ifelse(abs(stdres_df$Residual) > 2,
                           "white", "black")) +
  scale_fill_gradient2(low = "steelblue", mid = "white",
                      high = "tomato", midpoint = 0) +
  labs(title = "C. Standardized Residuals",
        subtitle = "Class × Survived",
        x = "Survived", y = "Class") +
  theme_minimal(base_size = 11)

# Panel D: Partial vs zero-order (hp vs mpg)
resid_mpg <- residuals(lm(mpg ~ wt, data = mtcars))
resid_hp <- residuals(lm(hp ~ wt, data = mtcars))

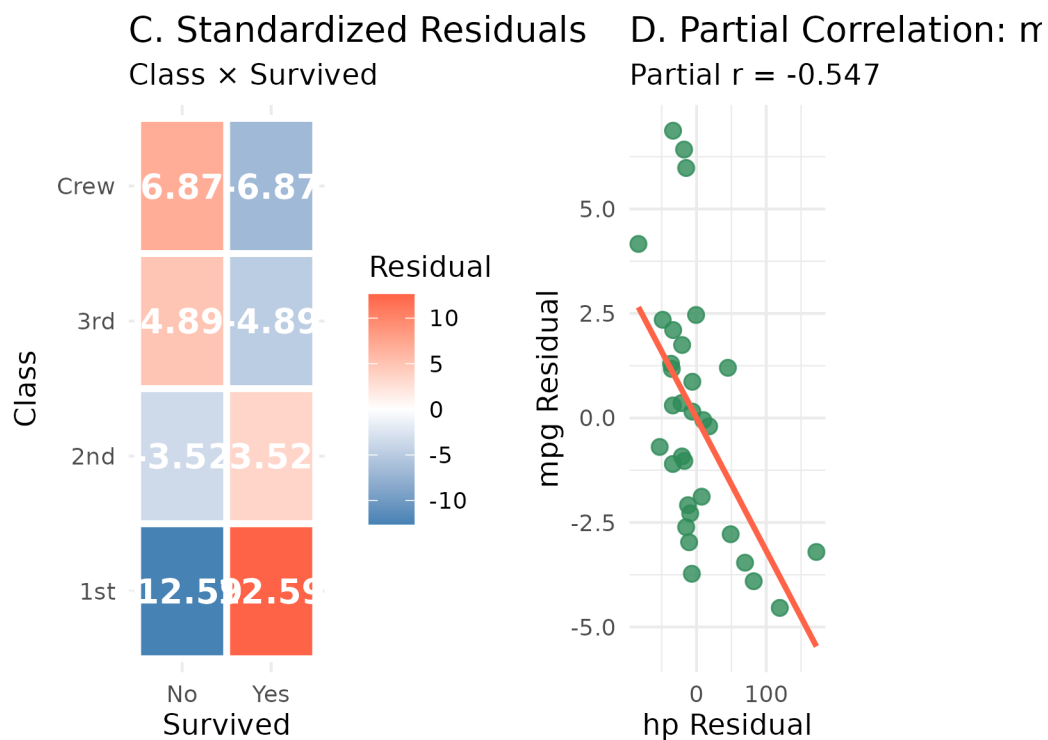
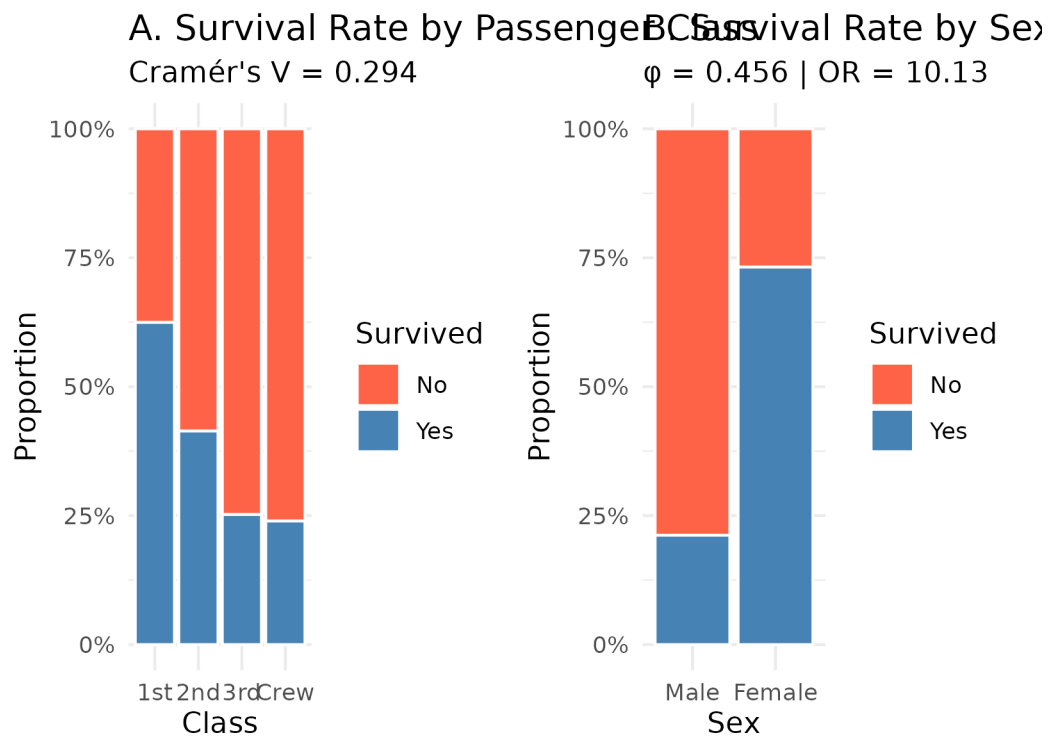
```

```

p4 <- ggplot(data.frame(res_hp = resid_hp,
                        res_mpg = resid_mpg),
             aes(x = res_hp, y = res_mpg)) +
  geom_point(color = "seagreen", size = 2.5, alpha = 0.8) +
  geom_smooth(method = "lm", color = "tomato",
             se = FALSE, linewidth = 1) +
  labs(title = "D. Partial Correlation: mpg & hp | wt",
       subtitle = paste0("Partial r = ",
                        round(pcor.test(mtcars$mpg,
                                       mtcars$hp,
                                       mtcars$wt)$estimate,3)),
       x = "hp Residual", y = "mpg Residual") +
  theme_minimal(base_size = 11)

(p1 + p2) / (p3 + p4)

```



Lab Discussion Questions

Answer the following in writing:

1. **Test Selection:** In Lab Task 2, both chi-square and Fisher's exact test give nearly identical p-values. Why? Does the very large sample size ($N = 2201$) affect which test is preferable?
 2. **Effect Size vs. Significance:** The chi-square test for $\text{Class} \times \text{Survival}$ is highly significant ($p < 0.001$). Interpret Cramér's $V = 0.294$. Is this a practically important association? How would you communicate this to a non-statistician?
 3. **Odds Ratio Interpretation:** Fisher's test for $\text{Sex} \times \text{Survival}$ gives an odds ratio. Interpret this in plain language: what does the odds ratio tell us about the relationship between being female and surviving?
 4. **Partial Correlation:** After controlling for weight (`wt`), the correlation between `mpg` and `hp` weakens. Does this mean weight fully "explains" the mpg-hp relationship? What other variables might also act as confounders?
 5. **Spurious Correlation:** Can you identify a potential spurious correlation in the `mtcars` dataset? Propose which third variable might be the confounder and describe how you would test this using partial correlation.
-

Chapter Summary

Summary

This chapter built a comprehensive toolkit for testing independence and measuring association across all combinations of variable types:

- **The concept of independence** — two variables are independent if knowing one provides no information about the other; this must be tested formally, not assumed.
- **Chi-square test** — the standard test for two categorical variables; based on comparing observed to expected frequencies in a contingency table.
- **Fisher's Exact Test** — the correct alternative when expected cell frequencies are small; computes exact probabilities rather than relying on asymptotic approximations.
- **Cramér's V** — standardizes chi-square to a 0–1 scale, enabling interpretation of association strength independently of sample size and table dimensions.
- **Pearson and Spearman correlation tests** — test whether linear (Pearson) or monotone (Spearman) relationships between continuous or ordinal variables are significantly different from zero.
- **Point-biserial and phi coefficients** — extend the correlation framework to binary variables, maintaining mathematical consistency with Pearson's r .
- **Partial correlation** — isolates the “pure” relationship between two variables after removing the influence of one or more control variables, helping distinguish real associations from spurious ones.

! Key Formulas to Know

Chi-Square Statistic:

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}, \quad E_{ij} = \frac{R_i C_j}{N}$$

Cramér's V:

$$V = \sqrt{\frac{\chi^2 / N}{\min(r - 1, c - 1)}}$$

Pearson Correlation t-test:

$$t = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}} \sim t(n-2)$$

First-Order Partial Correlation:

$$r_{XY \cdot Z} = \frac{r_{XY} - r_{XZ} \cdot r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}$$

Phi Coefficient (2×2):

$$\phi = \frac{ad - bc}{\sqrt{(a + b)(c + d)(a + c)(b + d)}}$$

References

- Anderson, E. (1935). The irises of the Gaspé Peninsula. *Bulletin of the American Iris Society*, 59, 2–5.
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.
- Fox, J., & Weisberg, S. (2019). *An R companion to applied regression* (3rd ed.). SAGE.
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Kassambara, A. (2023). *rstatix: Pipe-friendly framework for basic statistical tests* (R package version 0.7.2). <https://CRAN.R-project.org/package=rstatix>
- Kim Larsen. (2015). *ppcor: An R package for a fast calculation to semi-partial correlation coefficients* (R package version 1.1). <https://CRAN.R-project.org/package=ppcor>
- Meyer, D., Zeileis, A., & Hornik, K. (2023). *vcd: Visualizing categorical data* (R package version 1.4-11). <https://CRAN.R-project.org/package=vcd>
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality. *Biometrika*, 52(3/4), 591–611.
- Sievert, C., & others. (2023). *ggcorrplot: Visualization of a correlation matrix using ggplot2* (R package version 0.1.4). <https://CRAN.R-project.org/package=ggcorrplot>
- Spearman, C. (1904). “General intelligence,” objectively determined and measured. *American Journal of Psychology*, 15(2), 201–292.
- Wei, T., & Simko, V. (2021). *corrplot: Visualization of a correlation matrix* (R package version 0.92). <https://CRAN.R-project.org/package=corrplot>

Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.

Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

End of Chapter 4. Proceed to Chapter 5: Data Sampling Techniques.

Chapter 5: Data Sampling Techniques

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(sampling)      # probability sampling methods
library(boot)          # bootstrap resampling
library(survey)        # complex survey analysis
```

Chapter Overview

Introduction

Every data science project begins with data — but where does that data come from? The way data is collected determines what conclusions can be drawn from it. A poorly designed sample produces biased estimates no matter how sophisticated the analysis. Conversely, a well-designed sample allows powerful inferences from surprisingly small amounts of data. **Sampling** is the bridge between the population we want to understand and the data we actually have.

This chapter covers:

- **Why Sampling Matters** — population vs. sample, sources of error, and the stakes of sampling design
- **Probability Sampling Methods** — simple random, systematic, stratified, and cluster sampling
- **Non-Probability Sampling Methods** — convenience, purposive, snowball, and quota sampling
- **Sample Size Determination** — computing required n for means and proportions
- **Sampling Bias and Common Pitfalls** — selection bias, non-response, undercoverage
- **Bootstrap Resampling** — a computational approach to uncertainty estimation
- **Evaluating Sample Quality** — checking representativeness after data collection

Learning Objectives

By the end of this chapter, you will be able to:

1. Distinguish between probability and non-probability sampling and justify the choice of method.
2. Implement simple random, systematic, stratified, and cluster sampling in R.
3. Compute the required sample size for estimating means and proportions.
4. Identify and describe common sources of sampling bias.
5. Apply bootstrap resampling to estimate standard errors and confidence intervals.
6. Evaluate whether a collected sample is representative of the target population.

Why Sampling Matters

Introduction

In an ideal world, we would study every member of a population — a **census**. In practice, populations are often too large, too expensive, or too inaccessible to study in full. Sampling solves this problem: by studying a carefully chosen subset, we can draw valid inferences about the whole. But “carefully chosen” is the key phrase. The history of statistics is littered with catastrophic sampling failures — the 1936 Literary Digest poll that confidently predicted the wrong US presidential election winner, based on 2.4 million responses, remains one of the most famous examples of how a large but biased sample can be worse than a small representative one.

Theory

Key Terminology

Table 1: Core sampling terminology

Term	Definition
Population	The complete set of all units of interest
Sample	A subset of the population selected for study
Sampling frame	The list or mechanism from which the sample is drawn
Parameter	A numerical characteristic of the population (e.g., μ , σ^2 , p)
Statistic	A numerical characteristic of the sample (e.g., $\bar{x}$, s^2 , $\hat{p}$)
Estimator	A function of sample data used to estimate a parameter

Two Sources of Error

Every sample-based estimate differs from the true population parameter. This difference arises from two distinct sources:

Sampling error (random error): The natural variation between samples due to random selection. It is **unavoidable** but **quantifiable** — it decreases as sample size increases and forms the basis of confidence intervals and margin of error.

$$\text{Sampling Error} = \hat{\theta} - \theta$$

Non-sampling error (systematic error / bias): Error arising from flaws in the study design, data collection process, or measurement instrument. Unlike sampling error, it does **not** decrease with larger samples — a biased sampling method applied to a million observations is still biased.

$$\text{Bias} = E[\hat{\theta}] - \theta$$

This distinction is critical: **a larger sample reduces sampling error but cannot fix bias.**

The Mean Squared Error Framework

The quality of an estimator is captured by its **Mean Squared Error (MSE)**:

$$\text{MSE}(\hat{\theta}) = \text{Variance}(\hat{\theta}) + [\text{Bias}(\hat{\theta})]^2$$

A good estimator minimizes both variance (through adequate sample size) and bias (through sound sampling design). This decomposition mirrors the bias-variance tradeoff encountered in machine learning model evaluation.

When Is a Census Preferable?

Sampling is not always the right choice. A census is preferable when:

- The population is small (e.g., all 50 branch managers of a company).
- Every unit must be measured (e.g., 100% quality inspection for safety-critical parts).
- The cost of sampling error is unacceptably high.

For most data science applications involving large populations, sampling is necessary and, when done well, sufficient.

Example: Sampling Error vs. Bias

Example 5.1. A university wants to estimate the average GPA of its 10,000 students.

Scenario A — Simple random sample of 200: The sample mean $\bar{x} = 3.21$ differs from the true mean $\mu = 3.18$ by 0.03. This difference is **sampling error** — it would disappear on average across repeated samples, and a larger sample would reduce it.

Scenario B — Convenience sample of 200 from the honors college: The sample mean $\bar{x} = 3.74$ differs from $\mu = 3.18$ by 0.56. This difference is **bias** — it persists regardless of sample size because honors students systematically have higher GPAs. No amount of statistical analysis can recover the true mean from this biased sample.

Key lesson: Scenario B with $n = 200$ is far worse than Scenario A with $n = 50$. Sampling design matters more than sample size.

R Example: Sampling Error vs. Bias

```
# --- Simulate sampling error vs. bias ---
set.seed(42)

# True population: 10,000 students, GPA ~ N(3.18, 0.4^2)
population <- data.frame(
  id      = 1:10000,
  gpa     = rnorm(10000, mean = 3.18, sd = 0.4),
  honors  = c(rep(TRUE, 1000), rep(FALSE, 9000)) # 10% honors students
)
# Honors students have higher GPA
population$gpa[population$honors] <-
  population$gpa[population$honors] + 0.55

true_mean <- mean(population$gpa)
cat("True population mean GPA:", round(true_mean, 4), "\n\n")
```

True population mean GPA: 3.2305

```
# Simulate 1000 simple random samples of n=200
srs_means <- replicate(1000, {
  s <- population[sample(nrow(population), 200), ]
  mean(s$gpa)
})
```

```
# Simulate 1000 biased (honors-only) samples of n=200
biased_means <- replicate(1000, {
  honors_pool <- population[population$honors, ]
  s <- honors_pool[sample(nrow(honors_pool), 200), ]
  mean(s$gpa)
})

cat("Simple Random Sampling (n=200):\n")
```

Simple Random Sampling (n=200):

```
cat("  Mean of sample means:", round(mean(srs_means), 4), "\n")
```

Mean of sample means: 3.2328

```
cat("  Bias:                ", round(mean(srs_means) - true_mean, 4), "\n")
```

Bias: 0.0023

```
cat("  SE (sampling error): ", round(sd(srs_means), 4), "\n\n")
```

SE (sampling error): 0.0304

```
cat("Biased Sampling (honors only, n=200):\n")
```

Biased Sampling (honors only, n=200):

```
cat("  Mean of sample means:", round(mean(biased_means), 4), "\n")
```

Mean of sample means: 3.7196

```
cat("  Bias:                ", round(mean(biased_means) - true_mean, 4), "\n")
```

Bias: 0.4892

```
cat("  SE:                ", round(sd(biased_means), 4), "\n")
```

SE: 0.0249

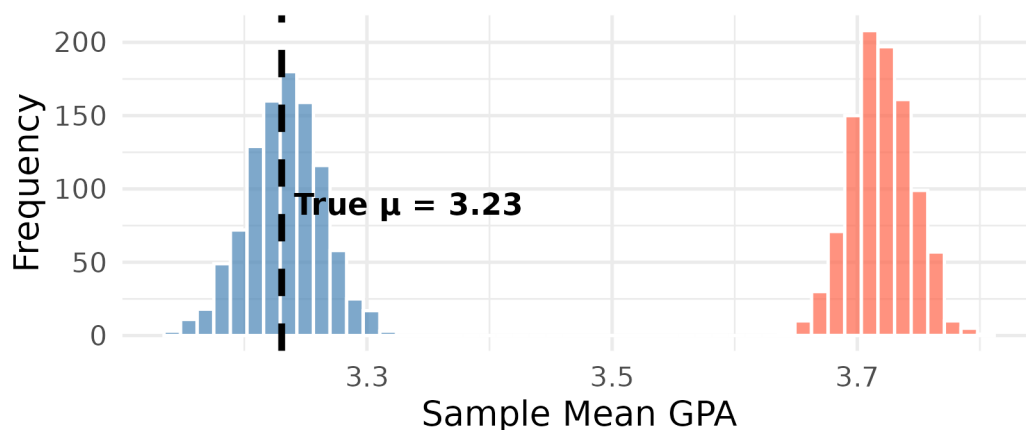
```
# --- Visualize sampling distributions ---
sim_df <- data.frame(
  mean = c(srs_means, biased_means),
  method = rep(c("Simple Random Sample",
                 "Biased Sample (Honors Only)"), each = 1000)
)

ggplot(sim_df, aes(x = mean, fill = method)) +
  geom_histogram(bins = 50, alpha = 0.7,
                color = "white", position = "identity") +
  geom_vline(xintercept = true_mean, color = "black",
             linewidth = 1.2, linetype = "dashed") +
  annotate("text", x = true_mean + 0.01, y = 90,
          label = paste0("True  = ", round(true_mean, 2)),
          hjust = 0, size = 4, fontface = "bold") +
  scale_fill_manual(values = c("Simple Random Sample" = "steelblue",
                              "Biased Sample (Honors Only)" = "tomato")) +
  labs(title = "Sampling Error vs. Bias: 1000 Simulated Samples",
       subtitle = "SRS centers on true mean; biased sample is systematically off",
       x = "Sample Mean GPA",
       y = "Frequency",
       fill = "Sampling Method") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")
```

Sampling Error vs. Bias: 1000 Simulated Samples

SRS centers on true mean; biased sample is systematic

Sampling Method ■ Biased Sample (Honors Only) ■ Simple Random



Code explanation:

- `replicate(n, expr)` repeats an expression `n` times and collects results — the cleanest way to simulate repeated sampling in R.
- The simulation demonstrates a fundamental truth: the SRS distribution centers on the true mean (unbiased), while the biased distribution is shifted entirely away from it regardless of n .
- Setting `position = "identity"` in `geom_histogram()` overlaps the two distributions for direct visual comparison.

Exercises

Exercise 5.1

Using the simulated population from the R example:

- (a) Repeat the simulation with $n = 50$, $n = 200$, and $n = 1000$ for the SRS. How does the standard error change? Verify against the theoretical formula $SE = \sigma/\sqrt{n}$.
- (b) Does increasing the biased sample to $n = 1000$ reduce the bias? Show with simulation.
- (c) Write a 100-word explanation of why a biased large sample is worse than an unbiased small sample.

Probability Sampling Methods

Introduction

Probability sampling methods give every unit in the population a **known, non-zero probability** of being selected. This property is what allows valid statistical inference — without it, we cannot compute unbiased estimates or valid confidence intervals. There are four fundamental probability sampling designs, each with different trade-offs between cost, precision, and practical feasibility.

Theory

Simple Random Sampling (SRS)

In **Simple Random Sampling**, every possible sample of size n from a population of size N has an equal probability of selection. Each unit has probability n/N of being included.

With replacement (SRSWR): Each draw is independent; a unit can appear more than once.

Without replacement (SRSWOR): Each unit can appear at most once. More common in practice.

Estimator for mean: $\hat{\mu} = \bar{x}$, with $SE(\bar{x}) = \sqrt{\frac{s^2}{n} (1 - \frac{n}{N})}$

The term $(1 - n/N)$ is the **finite population correction (FPC)** — it reduces the SE when the sample is a substantial fraction of the population. When $n/N < 0.05$, the FPC is negligible.

Advantages: Simple, unbiased, easy to analyze. **Disadvantages:** Requires a complete sampling frame; may oversample or undersample important subgroups by chance.

Systematic Sampling

Select every k -th unit from a list, where $k = N/n$ is the **sampling interval**, after a random start between 1 and k .

Procedure: 1. Compute $k = \lfloor N/n \rfloor$. 2. Randomly select a starting point $r \in \{1, 2, \dots, k\}$. 3. Select units $r, r + k, r + 2k, \dots$

Advantages: Simple to implement; spreads the sample evenly across the list. **Disadvantages:** If the list has a **periodic pattern** with period k , systematic sampling can be badly biased (e.g., always selecting the same day of the week).

Stratified Sampling

Divide the population into H non-overlapping, exhaustive **strata** (subgroups) based on a known characteristic (e.g., gender, region, age group), then draw independent SRS samples from each stratum.

Proportional allocation: Sample from each stratum proportional to its size: $n_h = n \cdot N_h/N$.

Optimal (Neyman) allocation: Allocate more to strata with greater variability: $n_h \propto N_h \sigma_h$.

Estimator for mean:

$$\hat{\mu}_{st} = \sum_{h=1}^H W_h \bar{x}_h, \quad W_h = N_h/N$$

Advantages: Guarantees representation of all strata; more precise than SRS when strata are internally homogeneous. **Disadvantages:** Requires prior knowledge of strata; more complex analysis.

Cluster Sampling

Divide the population into **clusters** (naturally occurring groups, e.g., schools, villages, hospitals), randomly select a sample of clusters, then survey **all** (or a sample of) units within selected clusters.

One-stage: Select clusters, survey all units within. **Two-stage:** Select clusters, then randomly sample units within selected clusters.

Advantages: No complete sampling frame needed (only a list of clusters); cost-effective when population is geographically dispersed. **Disadvantages:** Units within clusters tend to be similar (**intraclass correlation**), reducing effective sample size. Less precise than SRS of the same n .

Design effect (DEFF): The ratio of the variance under cluster sampling to the variance under SRS:

$$\text{DEFF} = 1 + (m - 1)\rho$$

where m is the average cluster size and ρ is the intraclass correlation coefficient.

Summary Comparison

Table 1: Probability sampling methods comparison

Method	Frame Required	Precision vs. SRS	Best Used When
SRS	Complete list	Baseline	Population is homogeneous
Systematic	Ordered list	SRS (if no periodicity)	List is available and random
Stratified	Complete list + strata info	Better	Known subgroups differ
Cluster	List of clusters only	Worse	Geographically dispersed

Example: Stratified vs. Simple Random Sampling

Example 5.2. A researcher wants to estimate average student satisfaction (0–100) at a university with 3 faculties: Science (1,200 students), Arts (800), Business (500). Total $N = 2,500$, target $n = 100$.

SRS: Select 100 students at random. Possible but might severely under-represent Business (only 20% of population).

Proportional stratified sampling: - Science: $n_1 = 100 \times 1200/2500 = 48$ - Arts: $n_2 = 100 \times 800/2500 = 32$ - Business: $n_3 = 100 \times 500/2500 = 20$

This guarantees each faculty is represented proportionally, and if satisfaction differs between faculties, stratified sampling will be more precise than SRS.

R Example: Probability Sampling Methods

```
# --- Build a simulated university population ---
set.seed(123)
N <- 2500
population <- data.frame(
  id      = 1:N,
  faculty = c(rep("Science", 1200),
               rep("Arts",    800),
               rep("Business", 500)),
  year    = sample(1:4, N, replace = TRUE),
  satisfaction = c(
    rnorm(1200, mean = 72, sd = 12), # Science
    rnorm(800,  mean = 78, sd = 10), # Arts
    rnorm(500,  mean = 68, sd = 15)  # Business
  )
)
population$satisfaction <- pmin(pmax(
  round(population$satisfaction), 0), 100)

true_mean <- mean(population$satisfaction)
cat("True population mean satisfaction:",
    round(true_mean, 2), "\n\n")
```

True population mean satisfaction: 73.09

```
# === 1. SIMPLE RANDOM SAMPLING ===
n <- 100
srs_sample <- population[sample(N, n, replace = FALSE), ]
cat("SRS estimate:", round(mean(srs_sample$satisfaction), 2), "\n")
```

SRS estimate: 73.3

```
# === 2. SYSTEMATIC SAMPLING ===
k <- floor(N / n)      # sampling interval = 25
start <- sample(1:k, 1) # random start
systematic_idx <- seq(start, N, by = k)[1:n]
sys_sample <- population[systematic_idx, ]
cat("Systematic estimate:",
    round(mean(sys_sample$satisfaction), 2), "\n")
```

Systematic estimate: 75.25

```
# === 3. STRATIFIED SAMPLING (proportional) ===
strata_sizes <- c(Science = 48, Arts = 32, Business = 20)

stratified_sample <- population |>
  group_by(faculty) |>
  group_modify(~ {
    nh <- strata_sizes[.y$faculty]
    .x[sample(nrow(.x), nh), ]
  }) |>
  ungroup()

# Weighted estimate
strat_estimate <- stratified_sample |>
  group_by(faculty) |>
  summarise(mean_sat = mean(satisfaction),
            nh = n(), .groups = "drop") |>
  mutate(Wh = c(800, 500, 1200) / N) |>
  summarise(est = sum(Wh * mean_sat)) |>
  pull(est)

cat("Stratified estimate:", round(strat_estimate, 2), "\n")
```

Stratified estimate: 73.17

```
# === 4. CLUSTER SAMPLING ===
# Treat year groups as clusters (4 clusters)
# Randomly select 2 clusters, survey all within
selected_years <- sample(1:4, 2, replace = FALSE)
cluster_sample <- population |>
  filter(year %in% selected_years)

cat("Cluster estimate (",
    paste(selected_years, collapse=" & "),
    "year):",
    round(mean(cluster_sample$satisfaction), 2), "\n\n")
```

Cluster estimate (4 & 2 year): 72.93

```
# --- Compare all methods ---
comparison <- data.frame(
  Method = c("True Mean", "SRS", "Systematic",
             "Stratified", "Cluster"),
```

```

Estimate = round(c(true_mean,
                    mean(srs_sample$satisfaction),
                    mean(sys_sample$satisfaction),
                    strat_estimate,
                    mean(cluster_sample$satisfaction)), 2),
n         = c(N, n, n, n, nrow(cluster_sample))
)

kable(comparison,
      caption = "Sampling Method Comparison",
      col.names = c("Method", "Mean Estimate", "Sample Size")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
row_spec(1, bold = TRUE, background = "#EEF2FF")

```

Table 2: Sampling Method Comparison

Method	Mean Estimate	Sample Size
True Mean	73.09	2500
SRS	73.30	100
Systematic	75.25	100
Stratified	73.17	100
Cluster	72.93	1227

```

# --- Visualize stratified vs SRS sample composition ---
srs_comp <- srs_sample |>
  count(faculty) |>
  mutate(method = "SRS", pct = n / sum(n))

strat_comp <- stratified_sample |>
  count(faculty) |>
  mutate(method = "Stratified", pct = n / sum(n))

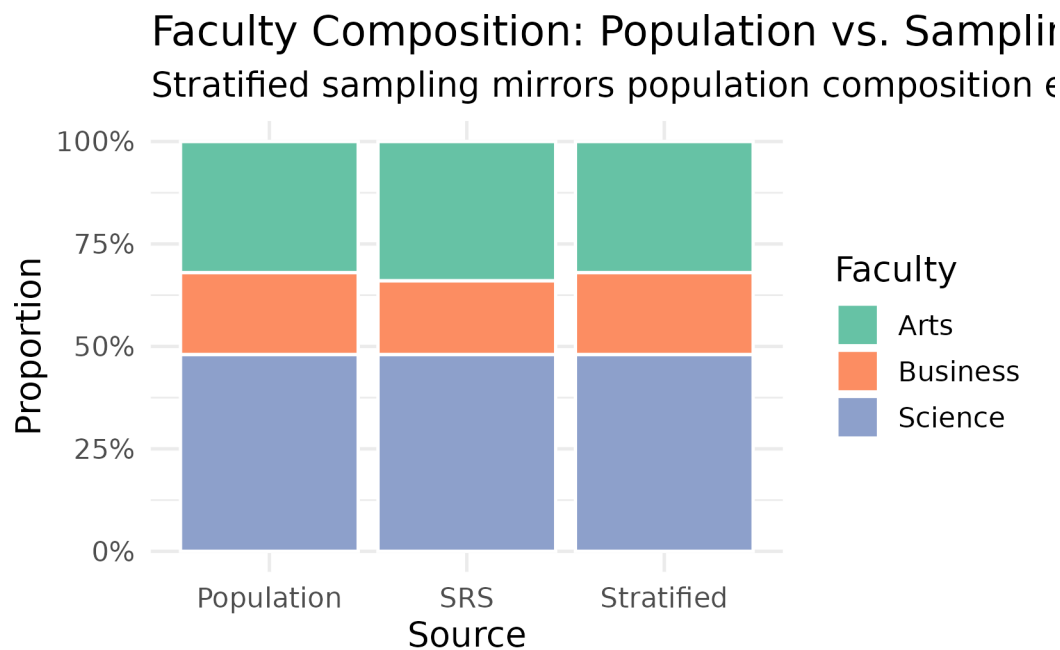
pop_comp <- population |>
  count(faculty) |>
  mutate(method = "Population", pct = n / sum(n))

comp_df <- bind_rows(srs_comp, strat_comp, pop_comp)
comp_df$method <- factor(comp_df$method,
                        levels = c("Population", "SRS", "Stratified"))

ggplot(comp_df, aes(x = method, y = pct, fill = faculty)) +
  geom_col(color = "white", position = "fill") +

```

```
scale_fill_brewer(palette = "Set2") +
scale_y_continuous(labels = scales::percent) +
labs(title = "Faculty Composition: Population vs. Sampling Methods",
      subtitle = "Stratified sampling mirrors population composition exactly",
      x = "Source",
      y = "Proportion",
      fill = "Faculty") +
theme_minimal(base_size = 13)
```



Code explanation:

- `group_modify()` applies a function within each group and row-binds the results — the cleanest way to implement stratified sampling in `tidyverse`.
- `seq(start, N, by = k)[1:n]` generates systematic indices starting from a random point.
- The composition plot visually demonstrates why stratified sampling is more representative — it guarantees the sample reflects population proportions, while SRS can deviate by chance.

Exercises

Exercise 5.2

Using the `population` data frame created in the R example:

- (a) Implement proportional stratified sampling by **year** (instead of faculty) with total $n = 120$.
- (b) Compute the stratified mean estimate and compare to the true mean.
- (c) Simulate 500 SRS samples and 500 stratified samples, each of size 100. Plot the two sampling distributions side by side. Which is more precise (lower variance)?

Exercise 5.3

A retail chain has 80 stores across 5 regions. You want to estimate the average weekly sales using cluster sampling.

- (a) Describe how you would implement one-stage and two-stage cluster sampling.
 - (b) What is the main statistical disadvantage of cluster sampling? Define the design effect (DEFF).
 - (c) If $\rho = 0.15$ and average cluster size $m = 20$, compute the DEFF and the effective sample size for a sample of $n = 200$.
-

Non-Probability Sampling Methods

Introduction

Probability sampling requires a complete sampling frame and often significant resources. In many real-world situations — exploratory research, pilot studies, social media data collection, qualitative work — probability sampling is impractical. **Non-probability sampling** methods select units based on convenience, judgment, or referral rather than random selection. While these methods cannot support formal statistical inference about populations, they are widely used and their limitations must be understood.

Theory

Convenience Sampling

Units are selected because they are **easy to reach** — students in a classroom, website visitors, volunteers. It is the most common sampling method in published research, and the most criticized.

Limitations: High potential for selection bias; results are not generalizable. The Literary Digest 1936 disaster used a form of convenience sampling (telephone and car ownership lists in the Depression era).

Purposive (Judgmental) Sampling

The researcher deliberately selects units believed to be **representative or informative** based on expert judgment. Common in qualitative research and case studies.

Subtypes:

- *Typical case sampling:* Select units that are “average” or “normal.”
- *Extreme case sampling:* Select outliers to understand the range.
- *Critical case sampling:* Select cases that are most informative for the research question.

Limitations: Results depend heavily on researcher judgment; no mechanism for assessing representativeness.

Snowball Sampling

Initial participants **recruit further participants** from their social networks. Used when the target population is hard to reach (e.g., undocumented migrants, drug users, rare disease patients).

Limitations: Sample is biased toward well-connected individuals; risk of clustering within social networks.

Quota Sampling

Divide the population into subgroups and fill **predetermined quotas** for each — similar in structure to stratified sampling, but without random selection within quotas.

Example: Survey 50 males and 50 females, selecting whoever is available until quotas are filled.

Limitations: Selection within quotas is non-random (convenience-based); harder to assess bias than stratified sampling.

When Non-Probability Sampling Is Acceptable

Table 1: Appropriateness of non-probability sampling

Purpose	Acceptable?	Caution
Exploratory/pilot research	Yes	Don't generalize findings
Hypothesis generation	Yes	Confirm with probability sample
Qualitative understanding	Yes	Not intended for inference
Population-level estimation	No	Use probability sampling
Machine learning (IID data)	Partially	Check for covariate shift

Example: Comparing Sampling Methods in Practice

Example 5.3. A researcher wants to understand attitudes toward remote work among Thai university employees.

- **Convenience sample:** Survey colleagues in the same department → fast but severely biased toward one unit.
- **Snowball sample:** Ask initial respondents to forward a survey link → reaches dispersed staff but over-represents social clusters.
- **Quota sample:** Recruit until 100 academic and 50 administrative staff have responded → better balance but non-random within groups.

- **Stratified random sample:** Obtain staff list from HR, stratify by role and faculty, randomly select from each stratum → most valid for inference, requires HR cooperation.

The right method depends on resources, research purpose, and the required level of generalizability.

R Example: Simulating Non-Probability Sampling Bias

```
# --- Simulate convenience sampling bias ---
set.seed(77)

# Population: employees with income and satisfaction
N <- 5000
employee_pop <- data.frame(
  id          = 1:N,
  department  = sample(c("Research","Teaching",
                        "Admin","Support"),
                      N, replace = TRUE,
                      prob = c(0.3, 0.4, 0.2, 0.1)),
  income      = c(rnorm(1500, 65000, 12000), # Research
                  rnorm(2000, 55000, 10000), # Teaching
                  rnorm(1000, 45000, 8000),  # Admin
                  rnorm(500, 38000, 7000)),  # Support
  satisfaction = NA
)

# Satisfaction correlates with income but varies by dept
employee_pop$satisfaction <-
  40 + 0.0003 * employee_pop$income +
  rnorm(N, 0, 8)
employee_pop$satisfaction <-
  pmin(pmax(round(employee_pop$satisfaction), 0), 100)

true_mean_sat <- mean(employee_pop$satisfaction)
true_mean_inc <- mean(employee_pop$income)

cat("True mean satisfaction:", round(true_mean_sat, 2), "\n")
```

True mean satisfaction: 56.21

```
cat("True mean income:      ", round(true_mean_inc, 0), "\n\n")
```

```
True mean income:      54205
```

```
# Convenience sample: only Research dept (easiest to reach)
convenience <- employee_pop |>
  filter(department == "Research") |>
  slice_sample(n = 200)
```

```
# Quota sample: 50 per dept
quota <- employee_pop |>
  group_by(department) |>
  slice_sample(n = 50) |>
  ungroup()
```

```
# SRS
srs <- employee_pop |> slice_sample(n = 200)
```

```
# Compare
results <- data.frame(
  Method          = c("True Population",
                      "SRS (n=200)",
                      "Convenience (Research only)",
                      "Quota (50 per dept)"),
  Mean_Satisfaction = round(c(
    true_mean_sat,
    mean(srs$satisfaction),
    mean(convenience$satisfaction),
    mean(quota$satisfaction)
  ), 2),
  Mean_Income      = round(c(
    true_mean_inc,
    mean(srs$income),
    mean(convenience$income),
    mean(quota$income)
  ), 0),
  Bias_Satisfaction = round(c(
    0,
    mean(srs$satisfaction) - true_mean_sat,
    mean(convenience$satisfaction) - true_mean_sat,
    mean(quota$satisfaction) - true_mean_sat
  ), 2)
)
```

```
kable(results,
  caption = "Sampling Method Bias Comparison",
  col.names = c("Method", "Mean Satisfaction",
    "Mean Income", "Bias")) |>
kable_styling(bootstrap_options = c("striped", "hover")) |>
column_spec(4, bold = TRUE,
  color = ifelse(abs(results$Bias_Satisfaction) > 1,
    "tomato", "darkgreen"))
```

Table 2: Sampling Method Bias Comparison

Method	Mean Satisfaction	Mean Income	Bias
True Population	56.21	54205	0.00
SRS ($n=200$)	55.20	53772	-1.00
Convenience (Research only)	55.03	54774	-1.17
Quota (50 per dept)	56.06	55292	-0.15

Code explanation:

- `slice_sample(n)` randomly samples n rows from a data frame — the tidyverse equivalent of `sample()`.
- `group_by()` |> `slice_sample(n)` implements quota or stratified sampling within groups easily.
- The bias column quantifies how far each method's estimate is from the truth — making the cost of convenience sampling concrete and visible.

Exercises

💡 Exercise 5.4

- For the employee population above, simulate 500 convenience samples (Research dept only, $n = 100$) and 500 SRS samples ($n = 100$). Plot both sampling distributions with a vertical line at the true mean.
- Compute the bias and variance of each sampling distribution.
- Explain why the convenience sample's variance is low but its MSE is high.

Sample Size Determination

Introduction

One of the most common questions in research design is: *How many observations do I need?* Too few and the study lacks power to detect real effects; too many and resources are wasted. **Sample size determination** is a formal calculation based on the desired precision or power, the expected variability, and the acceptable error rates. This section covers the two most common scenarios: estimating a population mean and estimating a population proportion.

Theory

Sample Size for Estimating a Mean

We want the **margin of error** E (half-width of the CI) to be no larger than a specified value:

$$E = z_{1-\alpha/2} \cdot \frac{\sigma}{\sqrt{n}} \quad \Rightarrow \quad n = \left(\frac{z_{1-\alpha/2} \cdot \sigma}{E} \right)^2$$

 Watch the subscript here

This is exactly the convention trap flagged in Chapter 3: many sources write $z_{\alpha/2}$ for this quantity, using the *right-tail* convention (area $\alpha/2$ remaining above the value). Under **this book's** left-tail-cumulative convention, that same value — e.g. 1.96 for a 95% CI — is $z_{1-\alpha/2}$, not $z_{\alpha/2}$. Written the other way, $z_{\alpha/2}$ would evaluate to -1.96 under our convention, the wrong sign for a margin of error. Always check which convention a formula was written in before plugging in `qnorm()`.

Since σ is usually unknown, we substitute a prior estimate, a pilot study result, or the rule of thumb $\sigma \approx \text{range}/4$.

With finite population correction:

$$n^* = \frac{n}{1 + (n - 1)/N}$$

Sample Size for Estimating a Proportion

For a binary outcome with proportion p :

$$n = \frac{z_{1-\alpha/2}^2 \cdot p(1-p)}{E^2}$$

When p is unknown, use $p = 0.5$ (maximizes $p(1-p) = 0.25$, giving the most conservative — largest — required n).

Sample Size for Hypothesis Testing

For a two-sample t-test with equal group sizes, the required n per group to detect effect size d with power $1 - \beta$ at significance α :

$$n = \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2}{d^2}$$

where $d = |\mu_1 - \mu_2|/\sigma$ is Cohen's d . Note $z_{1-\beta}$ follows the same convention: for 80% power ($\beta = 0.20$), $z_{1-\beta} = z_{0.80} = \text{qnorm}(0.80) = 0.842$. In practice, use `pwr.t.test()` as in Chapter 3.

Common z-Values

Table 1: Critical z-values for common confidence levels

Confidence Level	α	$z_{1-\alpha/2}$
90%	0.10	1.645
95%	0.05	1.960
99%	0.01	2.576

Example: Sample Size Calculation

Example 5.4 — Estimating a mean. A hospital administrator wants to estimate average patient waiting time within ± 3 minutes, with 95% confidence. From a pilot study, $\sigma \approx 18$ minutes. Total patient population $N = 8,000$.

$$n = \left(\frac{1.96 \times 18}{3} \right)^2 = (11.76)^2 = 138.3 \approx 139$$

Applying FPC (since $n/N = 139/8000 = 1.7\%$ — small, so FPC barely matters):

$$n^* = \frac{139}{1 + 138/8000} = \frac{139}{1.01725} \approx 137$$

Example 5.5 — Estimating a proportion. A data scientist wants to estimate the proportion of app users who click on a recommendation, within $\pm 2\%$ with 95% confidence. No prior estimate of p is available.

$$n = \frac{(1.96)^2 \times 0.5 \times 0.5}{(0.02)^2} = \frac{3.8416 \times 0.25}{0.0004} = 2401$$

Using $p = 0.5$ guarantees the sample will be large enough regardless of the true click rate.

R Example: Sample Size Calculations

```
# === SAMPLE SIZE FOR ESTIMATING A MEAN ===
sample_size_mean <- function(sigma, E, conf = 0.95, N = Inf) {
  z <- qnorm(1 - (1 - conf) / 2)
  n <- ceiling((z * sigma / E)^2)
  # Finite population correction
  if (is.finite(N)) {
    n_fpc <- ceiling(n / (1 + (n - 1) / N))
  } else {
    n_fpc <- n
  }
  data.frame(
    Confidence = paste0(conf * 100, "%"),
    Sigma      = sigma,
    Margin_E   = E,
    n_infinite = n,
    n_FPC      = n_fpc
  )
}

# Waiting time example
cat("=== Sample Size for Mean (Waiting Time) ===\n")
```

```
=== Sample Size for Mean (Waiting Time) ===
```

```
map_dfr(c(0.90, 0.95, 0.99), ~
  sample_size_mean(sigma = 18, E = 3,
    conf = .x, N = 8000)) |>
  kable(caption = "Required n for Estimating Mean Waiting Time",
    col.names = c("Confidence", "$\\sigma$", "Margin E",
      "n (infinite pop)", "n (N=8000)")) |>
  kable_styling(bootstrap_options = c("striped", "hover"),
    full_width = FALSE)
```

Table 2: Required n for Estimating Mean Waiting Time

Confidence	σ	Margin E	n (infinite pop)	n (N=8000)
90%	18	3	98	97
95%	18	3	139	137
99%	18	3	239	233

```
# === SAMPLE SIZE FOR ESTIMATING A PROPORTION ===
sample_size_prop <- function(p, E, conf = 0.95, N = Inf) {
  z <- qnorm(1 - (1 - conf) / 2)
  n <- ceiling(z^2 * p * (1 - p) / E^2)
  if (is.finite(N)) {
    n_fpc <- ceiling(n / (1 + (n - 1) / N))
  } else {
    n_fpc <- n
  }
  data.frame(p = p, Margin_E = E,
    n_infinite = n, n_FPC = n_fpc)
}

cat("\n=== Sample Size for Proportion (Click Rate) ===\n")
```

=== Sample Size for Proportion (Click Rate) ===

```
# Compare different assumed p values
map_dfr(c(0.1, 0.3, 0.5, 0.7, 0.9), ~
  sample_size_prop(p = .x, E = 0.02, conf = 0.95)) |>
  kable(caption = "Required n for Estimating Proportion (E=2$\\%$, 95$\\%$ CI)",
    col.names = c("Assumed p", "Margin E",
      "n (infinite)", "n (FPC N=50000)")) |>
  kable_styling(bootstrap_options = c("striped", "hover"),
    full_width = FALSE)
```


Table 3: Required n for Estimating Proportion (E=2\$\\%\$, 95\$\\%\$ CI)

Assumed p	Margin E	n (infinite)	n (FPC N=50000)
0.1	0.02	865	865
0.3	0.02	2017	2017
0.5	0.02	2401	2401
0.7	0.02	2017	2017
0.9	0.02	865	865

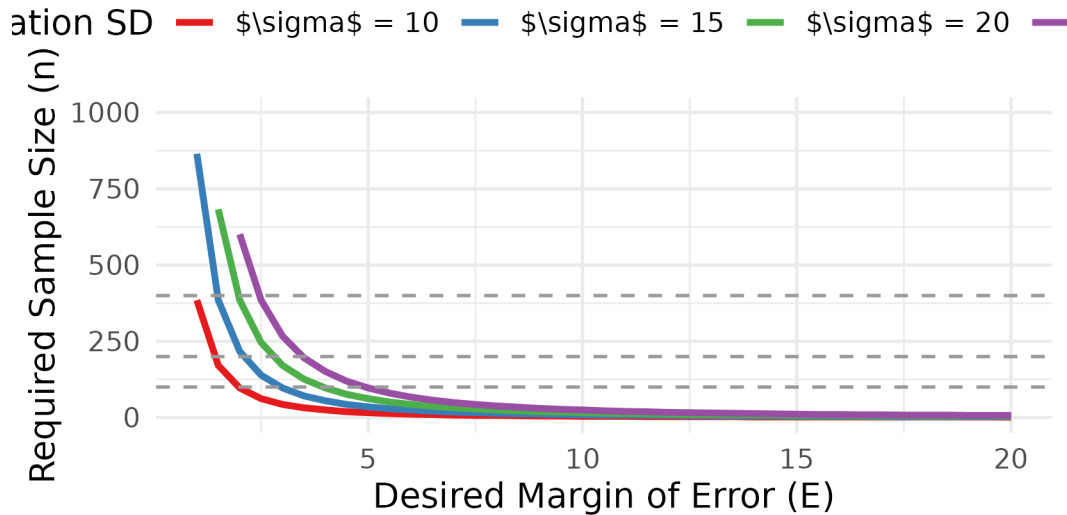
```
# --- Visualize: n vs. margin of error for different sigma ---
E_seq    <- seq(1, 20, by = 0.5)
sigma_vals <- c(10, 15, 20, 25)

n_df <- map_dfr(sigma_vals, function(s) {
  data.frame(
    E      = E_seq,
    n      = ceiling((1.96 * s / E_seq)^2),
    sigma  = paste0("$\\sigma$ = ", s)
  )
})

ggplot(n_df, aes(x = E, y = n, color = sigma)) +
  geom_line(linewidth = 1.2) +
  geom_hline(yintercept = c(100, 200, 400),
    linetype = "dashed", color = "gray60",
    linewidth = 0.6) +
  scale_color_brewer(palette = "Set1") +
  scale_y_continuous(limits = c(0, 1000)) +
  labs(title      = "Required Sample Size vs. Margin of Error",
    subtitle     = "95% confidence level; dashed lines at n = 100, 200, 400",
    x            = "Desired Margin of Error (E)",
    y            = "Required Sample Size (n)",
    color        = "Population SD") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")
```

Required Sample Size vs. Margin of Error

95% confidence level; dashed lines at $n = 100, 200,$



Code explanation:

- `qnorm(1 - alpha/2)` gives the critical z-value for any confidence level — no need to look up tables.
- `ceiling()` rounds up to ensure the sample is at least large enough.
- `map_dfr()` applies a function across a vector and stacks results — clean for building comparison tables across parameter values.
- The plot shows the crucial insight: required n drops sharply as E increases, but there are diminishing returns — halving precision does not halve cost.

Exercises

💡 Exercise 5.5

A researcher wants to estimate the average monthly expenditure of university students.

- With $\sigma = 2,500$ THB and desired margin of error $E = 200$ THB at 95% confidence, compute the required n .
- How does n change if the confidence level is increased to 99%?
- If the university has $N = 15,000$ students, apply the FPC. Does it make a meaningful difference?
- Plot required n vs. E for E ranging from 100 to 1,000 THB.

 Exercise 5.6

An election poll wants to estimate the proportion of voters who support a candidate within $\pm 3\%$ at 95% confidence.

- (a) Compute the required n assuming $p = 0.5$ (worst case).
- (b) If a prior poll suggests $p \approx 0.35$, how does the required n change?
- (c) If the polling budget only allows $n = 600$, what is the resulting margin of error at 95% confidence?

Sampling Bias and Common Pitfalls

Introduction

Even with careful planning, sampling can go wrong in ways that are difficult to detect after the fact. **Sampling bias** systematically distorts estimates in one direction, producing results that are internally consistent but fundamentally misleading. Understanding the mechanisms of common biases is essential for both designing better studies and critically evaluating published research.

Theory

Selection Bias

Selection bias occurs when the probability of inclusion in the sample is related to the outcome of interest — certain types of units are systematically more or less likely to be selected.

Examples:

- *Volunteer bias*: People who volunteer for studies are typically more health-conscious, educated, or motivated than the general population.
- *Survivorship bias*: Studying only units that “survived” a selection process (successful companies, published studies, military veterans who returned home) ignores the failures.
- *Ascertainment bias*: In medical research, patients who seek care are sicker than those who don’t, biasing disease prevalence estimates upward.

Non-Response Bias

Non-response bias occurs when units selected for the sample do not respond, and the non-responders differ systematically from responders.

Example: A survey on working conditions sent to all employees. Dissatisfied employees may be more motivated to respond, while satisfied employees ignore it — producing an overly negative picture.

Rule of thumb: Response rates below 70% should trigger careful investigation of non-response bias. Compare known characteristics (age, gender, department) of responders and non-responders if possible.

Undercoverage

Undercoverage occurs when the sampling frame does not include all members of the target population.

Classic example: Telephone surveys using landline directories miss mobile-only households, which are disproportionately young and lower-income. Internet surveys miss elderly and rural populations without internet access.

Measurement Bias

Even a perfectly representative sample produces biased estimates if the **measurement instrument** is flawed:

- *Social desirability bias:* Respondents answer in ways they think are socially acceptable rather than truthfully (e.g., underreporting alcohol consumption, overreporting charitable giving).
- *Leading questions:* Survey wording that suggests a preferred answer.
- *Recall bias:* Asking about past events; memory is imperfect and systematically distorted.

Publication Bias

In research, studies with significant positive results are more likely to be published than null results. This creates a biased literature where effect sizes appear larger than they truly are — a form of **survivorship bias** at the level of the scientific record.

Example: Survivorship Bias

Example 5.6. During World War II, the statistician Abraham Wald was asked to analyze bullet holes on returning aircraft to recommend where to add armor. The military wanted to reinforce the areas with the most damage. Wald correctly pointed out that they should armor the areas with *least* damage — because aircraft hit in those areas did not return. The sample (returning aircraft) was biased: it excluded the most informative cases (aircraft shot down).

This is a perfect illustration of survivorship bias: the sample of “survivors” systematically misrepresents the population of “all aircraft.”

R Example: Detecting Non-Response Bias

```
# --- Simulate non-response bias ---
set.seed(55)
N <- 3000

# True population: satisfaction correlated with income
full_pop <- data.frame(
  id          = 1:N,
  income_group = sample(c("Low", "Middle", "High"),
                        N, replace = TRUE,
                        prob = c(0.35, 0.45, 0.20)),
  satisfaction = NA
)

full_pop$satisfaction <- ifelse(
  full_pop$income_group == "Low",    rnorm(N, 55, 12),
  ifelse(full_pop$income_group == "Middle", rnorm(N, 68, 10),
        rnorm(N, 79, 9))
)

full_pop$satisfaction <- pmin(pmax(
  round(full_pop$satisfaction), 0), 100)

# Non-response: high-income people less likely to respond (busy)
full_pop$response_prob <- ifelse(
  full_pop$income_group == "Low",    0.75,
  ifelse(full_pop$income_group == "Middle", 0.60,
        0.30)
)

# Select a stratified sample and simulate non-response
selected <- full_pop[sample(N, 600), ]
responded <- selected[runif(nrow(selected)) <
  selected$response_prob, ]

true_mean <- mean(full_pop$satisfaction)
sample_mean_all <- mean(selected$satisfaction)
sample_mean_resp <- mean(responded$satisfaction)

cat("True population mean satisfaction:  ",
    round(true_mean, 2), "\n")
```

True population mean satisfaction: 65.49

```
cat("Sample mean (all selected, n=600):  ",
    round(sample_mean_all, 2), "\n")
```

Sample mean (all selected, n=600): 65.79

```
cat("Sample mean (respondents only, n=",
    nrow(responded), "):", round(sample_mean_resp, 2), "\n\n")
```

Sample mean (respondents only, n= 326): 63.6

```
# Income group composition comparison
comp <- bind_rows(
  full_pop  |> count(income_group) |>
    mutate(pct = n/sum(n), source = "Population"),
  responded |> count(income_group) |>
    mutate(pct = n/sum(n), source = "Respondents")
)

kable(comp |> select(source, income_group, pct) |>
  mutate(pct = round(pct * 100, 1)) |>
  pivot_wider(names_from = income_group,
              values_from = pct),
  caption = "Income Group Composition: Population vs. Respondents ($\\%$)",
  col.names = c("Source", "High", "Low", "Middle")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
  full_width = FALSE)
```

Table 1: Income Group Composition: Population vs. Respondents (\$\\%\$)

Source	High	Low	Middle
Population	20.3	34.0	45.7
Respondents	10.7	44.5	44.8

Code explanation:

- `response_prob` simulates differential non-response by income group — a realistic representation of how non-response actually works.
- The composition table reveals the mechanism of bias: high-income (higher satisfaction) respondents are under-represented in the responding sample, biasing the satisfaction estimate downward.
- This pattern is detectable if demographic data is available for both responders and non-responders — the first diagnostic step in non-response analysis.

Exercises

Exercise 5.7

- (a) In the non-response simulation above, compute the non-response rate overall and by income group.
 - (b) If you could only follow up with 100 non-respondents, which income group should you prioritize and why?
 - (c) Apply a simple **non-response weight** (inverse of response probability) to the respondent data and recompute the mean. Does it recover the true mean more accurately?
-

Bootstrap Resampling

Introduction

Classical inference relies on distributional assumptions (normality, known variance) and closed-form formulas for standard errors. But what about statistics with no simple formula — the median, a trimmed mean, a correlation coefficient, or a machine learning model’s accuracy? **Bootstrap resampling** is a computational technique that estimates uncertainty by repeatedly resampling **with replacement** from the observed data, treating the sample as a proxy for the population. It requires minimal assumptions and works for virtually any statistic.

Theory

The Bootstrap Principle

The **bootstrap principle** states: the relationship between the population and the sample mirrors the relationship between the sample and bootstrap samples drawn from it.

Algorithm:

1. From the original sample of size n , draw B bootstrap samples, each of size n , **with replacement**.
2. Compute the statistic of interest $\hat{\theta}_b^*$ for each bootstrap sample $b = 1, \dots, B$.
3. The distribution of $\hat{\theta}_b^* - \hat{\theta}$ approximates the sampling distribution of $\hat{\theta} - \theta$.

Bootstrap standard error:

$$\widehat{\text{SE}}_{\text{boot}} = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}_b^* - \bar{\theta}^*)^2}$$

Bootstrap Confidence Intervals

Percentile CI: Use the $\alpha/2$ and $1 - \alpha/2$ quantiles of the bootstrap distribution:

$$\text{CI} = [\hat{\theta}_{(\alpha/2)}^*, \hat{\theta}_{(1-\alpha/2)}^*]$$

BCa (Bias-Corrected and Accelerated) CI: Corrects for bias and skewness in the bootstrap distribution — preferred in practice and implemented in R's `boot.ci()`.

When to Use the Bootstrap

Table 1: Bootstrap usage guide

Situation	Bootstrap Appropriate?
No closed-form SE formula	Yes
Small sample, unknown distribution	Yes
Complex statistic (e.g., ratio, quantile)	Yes
Simple mean, large sample, normal population	Unnecessary (t-interval works)
Time series data (dependent observations)	Use block bootstrap instead

Example: Bootstrapping the Median

Example 5.7. A sample of 30 house prices (in million THB) has a median of 4.2M. The classical SE formula for the median is complex and assumes normality. The bootstrap provides a distribution-free CI.

Bootstrap result ($B = 5,000$): 95% percentile CI = (3.6M, 5.1M).

Interpretation: We are 95% confident the true population median house price lies between 3.6M and 5.1M THB, with no normality assumption required.

R Example: Bootstrap Resampling

```
# --- Bootstrap CI for the median ---
set.seed(314)

# Simulate right-skewed house prices (log-normal)
house_prices <- exp(rnorm(30, mean = log(4.2), sd = 0.5))
observed_median <- median(house_prices)
cat("Observed sample median:", round(observed_median, 3),
    "million THB\n\n")
```

Observed sample median: 3.756 million THB

```
# Manual bootstrap (educational)
B <- 5000
boot_medians <- replicate(B, {
  boot_sample <- sample(house_prices, length(house_prices),
                        replace = TRUE)
  median(boot_sample)
})

# Bootstrap SE and CI
boot_se <- sd(boot_medians)
boot_ci <- quantile(boot_medians, c(0.025, 0.975))

cat("Bootstrap SE of median:      ", round(boot_se, 4), "\n")
```

Bootstrap SE of median: 0.3839

```
cat("95% Percentile CI:  [",
    round(boot_ci[1], 3), ", ",
    round(boot_ci[2], 3), "]\n\n")
```

95% Percentile CI: [3.021 , 4.534]

```
# --- Using the boot package (more rigorous) ---
library(boot)

# Define statistic function
median_fn <- function(data, indices) {
  median(data[indices])
}
```

```
boot_result <- boot(data      = house_prices,
                    statistic = median_fn,
                    R        = 5000)

# BCa confidence interval
bca_ci <- boot.ci(boot_result, type = "bca")
print(bca_ci)
```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS

Based on 5000 bootstrap replicates

CALL :

```
boot.ci(boot.out = boot_result, type = "bca")
```

Intervals :

Level BCa

95% (3.015, 4.534)

Calculations and Intervals on Original Scale

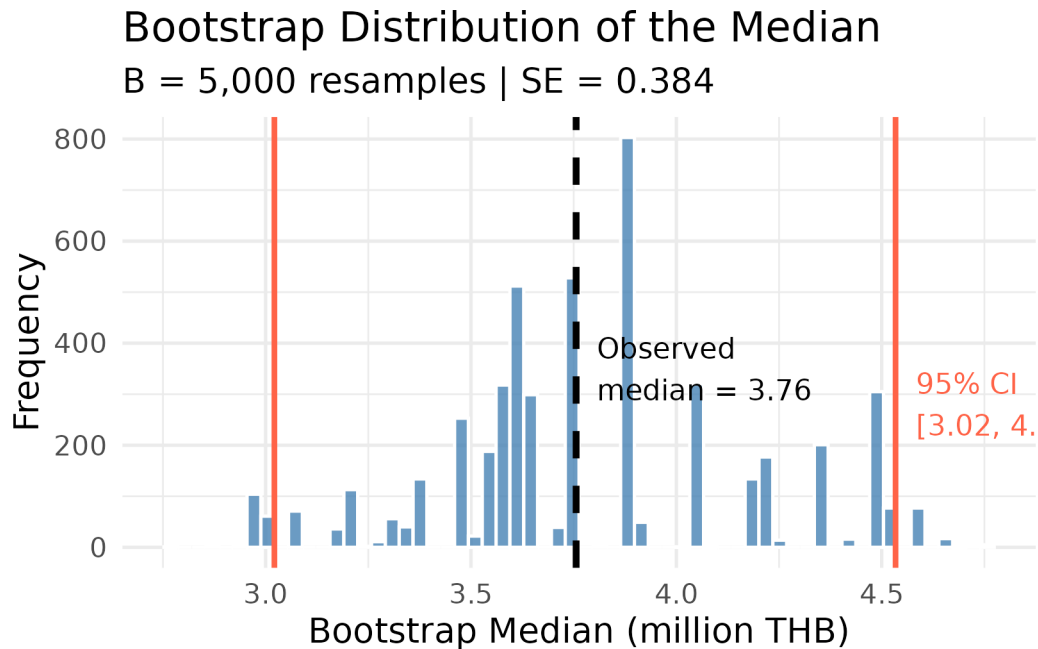
```
# --- Visualize bootstrap distribution ---
boot_df <- data.frame(median = boot_medians)

ggplot(boot_df, aes(x = median)) +
  geom_histogram(bins = 60, fill = "steelblue",
                color = "white", alpha = 0.8) +
  geom_vline(xintercept = observed_median,
             color = "black", linewidth = 1.2,
             linetype = "dashed") +
  geom_vline(xintercept = boot_ci,
             color = "tomato", linewidth = 1,
             linetype = "solid") +
  annotate("text", x = observed_median + 0.05,
           y = 350,
           label = paste0("Observed\nmedian = ",
                          round(observed_median, 2)),
           hjust = 0, size = 3.8) +
  annotate("text", x = boot_ci[2] + 0.05,
           y = 280,
           label = paste0("95% CI\n[",
                          round(boot_ci[1], 2), ", ",
                          round(boot_ci[2], 2), "]",
                          color = "tomato", hjust = 0, size = 3.8) +
  labs(title = "Bootstrap Distribution of the Median",
```

```

subtitle = paste0("B = 5,000 resamples | SE = ",
                  round(boot_se, 3)),
x        = "Bootstrap Median (million THB)",
y        = "Frequency") +
theme_minimal(base_size = 13)

```



Code explanation:

- `sample(x, n, replace = TRUE)` is the core of bootstrap resampling — drawing n observations with replacement from the data.
- `replicate(B, expr)` runs the resampling loop efficiently without explicit `for` loops.
- The `boot` package's `boot.ci(type = "bca")` provides the BCa interval, which is more accurate than the simple percentile interval for skewed distributions.
- The histogram visualizes the bootstrap distribution — its spread represents uncertainty in the median estimate.

Exercises

💡 Exercise 5.8

Using the `airquality` dataset (Ozone column, removing NAs):

- Compute the observed mean and median of Ozone.
- Bootstrap both statistics ($B = 5,000$). Compute SE and 95% percentile CIs for each.

- (c) Compare the bootstrap CI for the mean to the classical t-interval (`t.test()`). Do they agree?
- (d) Why is the bootstrap particularly valuable for the median in this dataset?

 Exercise 5.9 (Challenge)

Bootstrap the **correlation coefficient** between `mpg` and `hp` in `mtcars`.

- (a) Compute the observed Pearson r .
 - (b) Bootstrap r with $B = 5,000$ and plot the distribution.
 - (c) Compute the 95% BCa CI using `boot.ci()`.
 - (d) Compare to the analytical CI from `cor.test()`. Which is wider, and why?
-

Evaluating Sample Quality

Introduction

After data has been collected, a critical question remains: **Is this sample representative of the target population?** A good sampling design gives a high probability of representativeness, but it does not guarantee it. Evaluating sample quality is essential before drawing any conclusions — it protects against over-confident inference and reveals where caution is needed. This section covers practical tools for comparing sample and population distributions and detecting imbalance.

Theory

Comparing Sample and Population Distributions

When population-level data is available (from a census, administrative records, or prior studies), we can directly compare:

Categorical variables: Compare proportions using chi-square goodness-of-fit test:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

where O_i is the observed count in category i and $E_i = n \cdot p_i^{\text{pop}}$ is the expected count based on population proportions.

Continuous variables: Compare distributions using the Kolmogorov-Smirnov test or by visual comparison of histograms/density plots.

Weighting for Representativeness

When the sample over- or under-represents certain groups, **post-stratification weights** can partially correct for this:

$$w_i = \frac{p_i^{\text{pop}}}{p_i^{\text{sample}}}$$

Applying these weights to estimates adjusts for the imbalance. However, weights cannot fix severe under-representation (e.g., if a group is entirely absent from the sample).

Key Diagnostics

Table 1: Sample quality diagnostics

Diagnostic	Tool	What to Look For
Demographic balance	Chi-square goodness-of-fit	Sample proportions population proportions
Distribution shape	KS test, QQ plot	Sample distribution known distribution
Non-response pattern	Compare respondents vs. frame	No systematic differences
Outliers from selection	Mahalanobis distance	No extreme imbalance in multivariate space

Example: Goodness-of-Fit for Sample Representativeness

Example 5.8. A survey of 400 employees is collected. The company’s HR records show the true gender and department breakdown. We test whether the sample matches the population.

If the chi-square goodness-of-fit test gives $p < 0.05$, the sample is significantly different from the population in its composition — estimates should be weighted before reporting.

R Example: Evaluating Sample Quality


```
# --- Evaluate sample representativeness ---
set.seed(88)

# Known population proportions (from HR records)
pop_props <- c(Science = 0.30, Arts = 0.40,
               Business = 0.20, Admin = 0.10)

# Simulate a biased sample (Science over-represented)
n_sample <- 400
sample_depts <- sample(
  names(pop_props), n_sample,
  replace = TRUE,
  prob = c(0.45, 0.35, 0.15, 0.05) # biased draw
)

# Observed counts
obs_counts <- table(sample_depts)
exp_counts <- n_sample * pop_props[names(obs_counts)]

# Chi-square goodness-of-fit test
gof_test <- chisq.test(obs_counts,
                      p = pop_props[names(obs_counts)])
print(gof_test)
```

Chi-squared test for given probabilities

```
data:  obs_counts
X-squared = 47.038, df = 3, p-value = 3.412e-10
```

```
# --- Post-stratification weights ---
obs_props <- obs_counts / n_sample
weights <- pop_props[names(obs_props)] / obs_props

cat("\nPost-Stratification Weights:\n")
```

Post-Stratification Weights:

```
print(round(weights, 3))
```

```
sample_depts
  Admin    Arts Business  Science
  1.905    1.143    1.356    0.667
```

```
# Apply weights to a satisfaction estimate
sample_data <- data.frame(
  department = sample_depts,
  satisfaction = rnorm(n_sample, mean = 70, sd = 12)
)

# Weighted mean
w_vector <- weights[sample_data$department]
weighted_mean <- weighted.mean(sample_data$satisfaction,
                               w = w_vector)
unweighted_mean <- mean(sample_data$satisfaction)

cat("\nUnweighted mean satisfaction:", round(unweighted_mean, 2))
```

Unweighted mean satisfaction: 69.97

```
cat("\nWeighted mean satisfaction: ", round(weighted_mean, 2), "\n")
```

Weighted mean satisfaction: 70.21

```
# --- Visualize sample vs. population composition ---
comp_df <- data.frame(
  Department = names(pop_props),
  Population = as.numeric(pop_props),
  Sample      = as.numeric(obs_counts / n_sample)
) |>
  pivot_longer(cols = c(Population, Sample),
               names_to = "Source",
               values_to = "Proportion")

ggplot(comp_df, aes(x = Department, y = Proportion,
                   fill = Source)) +
  geom_col(position = "dodge", color = "white",
           width = 0.65) +
  geom_text(aes(label = scales::percent(Proportion, 1)),
            position = position_dodge(width = 0.65),
            vjust = -0.4, size = 3.5) +
  scale_fill_manual(values = c("Population" = "steelblue",
                              "Sample" = "tomato")) +
  scale_y_continuous(labels = scales::percent,
                    limits = c(0, 0.55)) +
  labs(title = "Sample vs. Population Composition",
```

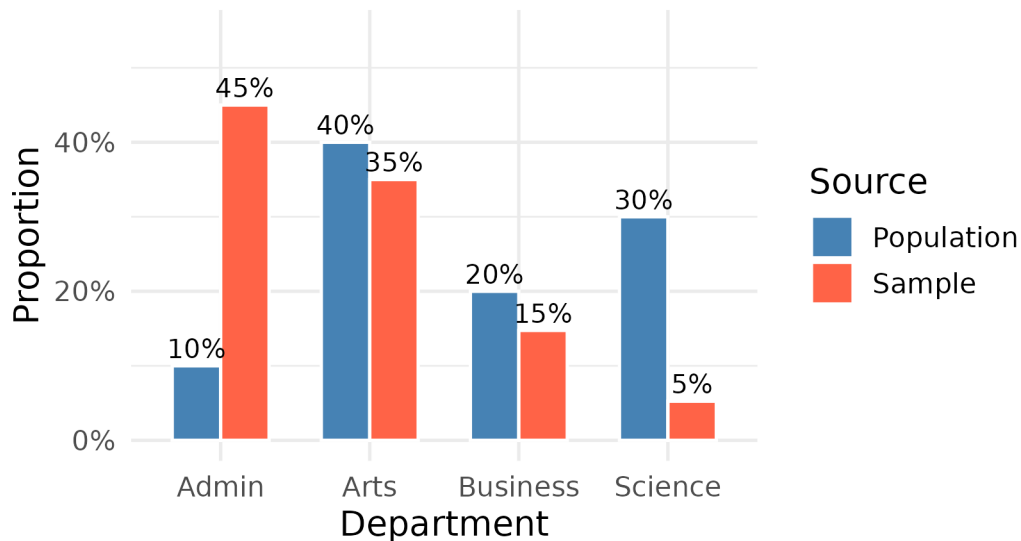
```

subtitle = paste0("Chi-square GOF test:  $\chi^2$ (",
  gof_test$parameter, ") = ",
  round(gof_test$statistic, 2),
  ", p = ",
  round(gof_test$p.value, 4)),
x        = "Department",
y        = "Proportion",
fill     = "Source") +
theme_minimal(base_size = 13)

```

Sample vs. Population Composition

Chi-square GOF test: $\chi^2(3) = 47.04$, $p = 0$



Code explanation:

- `chisq.test(observed_counts, p = population_proportions)` performs the goodness-of-fit test — note `p` takes the expected proportions (must sum to 1).
- Post-stratification weights are computed as the ratio of population to sample proportions. `weighted.mean(x, w)` applies them.
- The side-by-side bar chart immediately reveals which departments are over- or under-represented — Science is clearly over-sampled (45% vs. 30% in the population).

Exercises

Exercise 5.10

Using the `population` data frame from Section 2 (the university satisfaction example):

- (a) Draw a sample of $n = 150$ using convenience sampling (Arts faculty only).
 - (b) Test representativeness using chi-square goodness-of-fit against the known population proportions.
 - (c) Compute post-stratification weights and apply them to estimate mean satisfaction.
 - (d) Compare unweighted, weighted, and true mean estimates.
-

Chapter Lab Activity: Exploring Sampling with nhanes-Style Data

Objectives

In this lab you will apply the full sampling workflow — from designing a sampling strategy to evaluating sample quality and applying bootstrap inference — using a simulated population representative of a national health survey. You will compare different sampling methods, diagnose bias, and use bootstrap resampling to estimate uncertainty for a non-standard statistic.

Simulated Population

```
# --- Create a realistic simulated health survey population ---
set.seed(2024)
N_pop <- 20000

health_pop <- data.frame(
  id      = 1:N_pop,
  region  = sample(c("North", "Central", "South", "East", "West"),
                  N_pop, replace = TRUE,
                  prob = c(0.20, 0.30, 0.20, 0.15, 0.15)),
  age_group = sample(c("18-30", "31-45", "46-60", "61+"),
                    N_pop, replace = TRUE,
                    prob = c(0.25, 0.30, 0.25, 0.20)),
  income   = exp(rnorm(N_pop, log(35000), 0.6)),
  bmi      = rnorm(N_pop, 24.5, 4.2),
  smoker   = rbinom(N_pop, 1, 0.22)
)

# Introduce realistic correlations
health_pop$bmi <- health_pop$bmi +
  ifelse(health_pop$age_group == "61+", 1.5,
  ifelse(health_pop$age_group == "46-60", 0.8, 0))
```

```
health_pop$bmi <- pmax(health_pop$bmi, 15)

cat("Population size:", N_pop, "\n")
```

Population size: 20000

```
cat("True mean BMI: ", round(mean(health_pop$bmi), 3), "\n")
```

True mean BMI: 25.065

```
cat("True mean income:", round(mean(health_pop$income), 0), "\n")
```

True mean income: 41961

```
cat("True smoking rate:", round(mean(health_pop$smoker), 4), "\n\n")
```

True smoking rate: 0.2177

```
# Population composition
health_pop |>
  count(region, age_group) |>
  pivot_wider(names_from = age_group, values_from = n) |>
  kable(caption = "Population: Region x Age Group") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
                font_size = 11)
```

Table 1: Population: Region x Age Group

region	18-30	31-45	46-60	61+
Central	1522	1794	1450	1236
East	740	894	788	587
North	1009	1186	940	793
South	1004	1199	1063	854
West	729	888	748	576

Lab Task 1: Implement Four Sampling Methods

```

set.seed(42)
n_target <- 400

# 1. SRS
srs_lab <- health_pop |> slice_sample(n = n_target)

# 2. Systematic
k_sys <- floor(N_pop / n_target)
start <- sample(1:k_sys, 1)
sys_lab <- health_pop[seq(start, N_pop, by = k_sys)[1:n_target], ]

# 3. Stratified by region (proportional)
region_counts <- table(health_pop$region)
strat_lab <- health_pop |>
  group_by(region) |>
  group_modify(~ {
    nh <- round(n_target * nrow(.x) / N_pop)
    slice_sample(.x, n = max(nh, 1))
  }) |>
  ungroup()

# 4. Cluster by region (select 3 of 5 regions, survey all)
selected_regions <- sample(unique(health_pop$region), 3)
cluster_lab <- health_pop |>
  filter(region %in% selected_regions)

# Summary
true_bmi <- mean(health_pop$bmi)
sampling_comparison <- data.frame(
  Method      = c("True Population", "SRS",
                  "Systematic", "Stratified", "Cluster"),
  n           = c(N_pop, nrow(srs_lab), nrow(sys_lab),
                  nrow(strat_lab), nrow(cluster_lab)),
  Mean_BMI    = round(c(true_bmi,
                        mean(srs_lab$bmi),
                        mean(sys_lab$bmi),
                        mean(strat_lab$bmi),
                        mean(cluster_lab$bmi)), 4),
  Bias        = round(c(0,
                        mean(srs_lab$bmi) - true_bmi,
                        mean(sys_lab$bmi) - true_bmi,
                        mean(strat_lab$bmi) - true_bmi,
                        mean(cluster_lab$bmi) - true_bmi), 4)

```

```
)

kable(sampling_comparison,
      caption = "Sampling Method Comparison: Mean BMI",
      col.names = c("Method", "n", "Mean BMI", "Bias")) |>
kable_styling(bootstrap_options = c("striped", "hover")) |>
row_spec(1, bold = TRUE, background = "#EEF2FF") |>
column_spec(4, color = ifelse(
  abs(sampling_comparison$Bias) > 0.1, "tomato", "darkgreen"),
  bold = TRUE)
```

Table 2: Sampling Method Comparison: Mean BMI

Method	n	Mean BMI	Bias
True Population	20000	25.0650	0.0000
SRS	400	24.6688	-0.3963
Systematic	400	25.2202	0.1551
Stratified	400	25.2113	0.1462
Cluster	13131	25.0620	-0.0031

Lab Task 2: Sample Size Planning

```
# For the smoking rate (proportion)
true_p <- mean(health_pop$smoker)
cat("True smoking rate:", round(true_p, 4), "\n\n")
```

True smoking rate: 0.2177

```
# Required sample sizes for different margins of error
margins <- c(0.01, 0.02, 0.03, 0.05)
n_required <- data.frame(
  Margin_E = margins,
  n_p_unknown = ceiling((1.96^2 * 0.5 * 0.5) / margins^2),
  n_p_known = ceiling((1.96^2 * true_p * (1-true_p)) / margins^2)
)

kable(n_required,
      caption = "Required n for Estimating Smoking Rate (95% CI)",
      col.names = c("Margin of Error", "n (p=0.5)",
```



```

      paste0("n (p=", round(true_p,2), ")")))) |>
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE)

```

Table 3: Required n for Estimating Smoking Rate (95% CI)

Margin of Error	n (p=0.5)	n (p=0.22)
0.01	9604	6543
0.02	2401	1636
0.03	1068	727
0.05	385	262

Lab Task 3: Bootstrap Inference

```

# Bootstrap the 75th percentile of BMI (no closed-form CI)
bmi_sample <- srs_lab$bmi
obs_p75    <- quantile(bmi_sample, 0.75)

p75_fn <- function(data, indices) {
  quantile(data[indices], 0.75)
}

boot_p75 <- boot(data = bmi_sample, statistic = p75_fn,
                 R = 5000)
ci_p75   <- boot.ci(boot_p75, type = "bca")

cat("Observed 75th percentile BMI: ",
    round(obs_p75, 3), "\n")

```

Observed 75th percentile BMI: 27.267

```

cat("True 75th percentile (pop): ",
    round(quantile(health_pop$bmi, 0.75), 3), "\n")

```

True 75th percentile (pop): 27.915

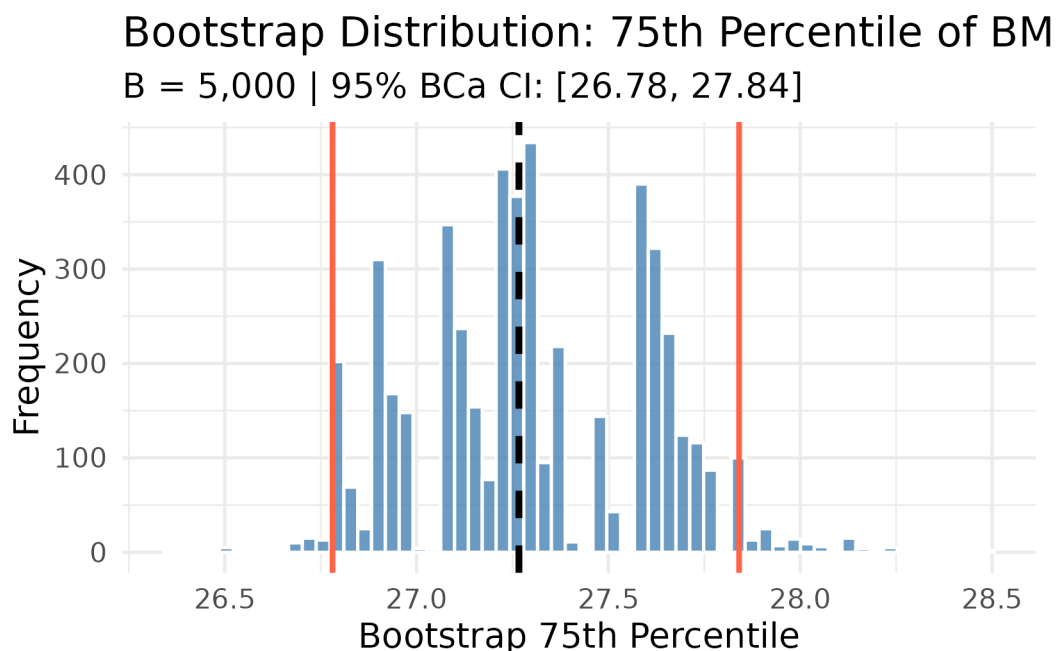
```

cat("95% BCa CI: [",
    round(ci_p75$bca[4], 3), ", ",
    round(ci_p75$bca[5], 3), "]\n")

```

95% BCa CI: [26.781 , 27.841]

```
# Plot bootstrap distribution
ggplot(data.frame(p75 = boot_p75$t), aes(x = p75)) +
  geom_histogram(bins = 60, fill = "steelblue",
                 color = "white", alpha = 0.8) +
  geom_vline(xintercept = obs_p75, color = "black",
              linewidth = 1.2, linetype = "dashed") +
  geom_vline(xintercept = c(ci_p75$bca[4], ci_p75$bca[5]),
              color = "tomato", linewidth = 1) +
  labs(title = "Bootstrap Distribution: 75th Percentile of BMI",
        subtitle = paste0("B = 5,000 | 95% BCa CI: [",
                           round(ci_p75$bca[4],2), ", ",
                           round(ci_p75$bca[5],2), "]" ),
        x = "Bootstrap 75th Percentile",
        y = "Frequency") +
  theme_minimal(base_size = 13)
```



Lab Task 4: Representativeness Check

```
# Check if SRS sample is representative by region
pop_region_props <- prop.table(table(health_pop$region))
srs_region_counts <- table(srs_lab$region)
```

```

gof_region <- chisq.test(
  srs_region_counts,
  p = pop_region_props[names(srs_region_counts)]
)

cat("Goodness-of-Fit Test (Region):\n")

```

Goodness-of-Fit Test (Region):

```

cat("  $\chi^2$ (", gof_region$parameter, ") =",
    round(gof_region$statistic, 3),
    " p =", round(gof_region$p.value, 4), "\n\n")

```

$\chi^2(4) = 0.686$ $p = 0.953$

```

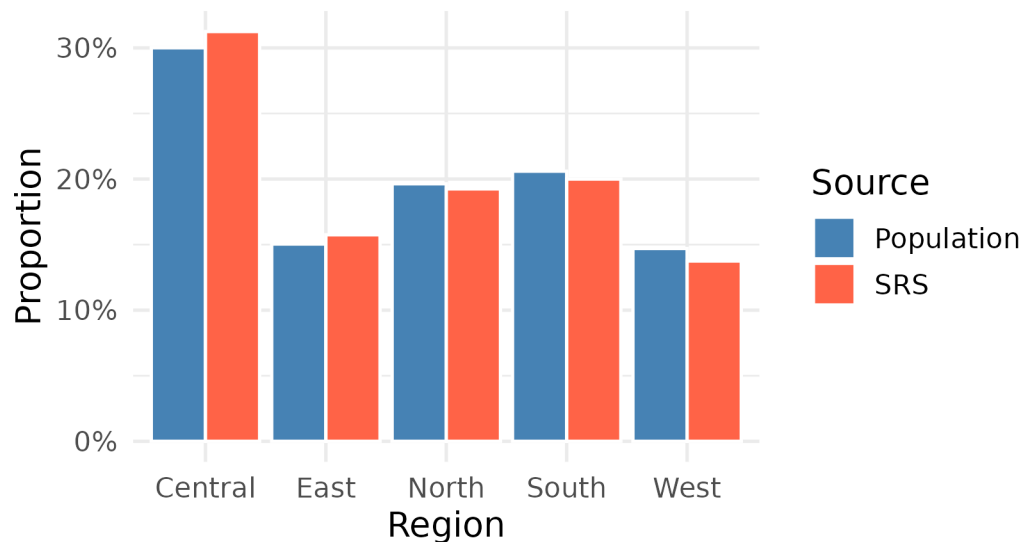
# Visual comparison
comp_region <- data.frame(
  Region      = names(pop_region_props),
  Population  = as.numeric(pop_region_props),
  SRS         = as.numeric(table(srs_lab$region) /
                                nrow(srs_lab))[
                                order(names(table(srs_lab$region)))]
) |>
  pivot_longer(c(Population, SRS),
    names_to = "Source", values_to = "Proportion")

ggplot(comp_region,
  aes(x = Region, y = Proportion, fill = Source)) +
  geom_col(position = "dodge", color = "white") +
  scale_fill_manual(values = c("Population" = "steelblue",
                              "SRS" = "tomato")) +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "SRS Representativeness Check: Region",
    subtitle = paste0("GOF test p = ",
                      round(gof_region$p.value, 3),
                      " - sample composition matches population"),
    x = "Region", y = "Proportion") +
  theme_minimal(base_size = 13)

```

SRS Representativeness Check: Region

GOF test $p = 0.953$ — sample composition matches μ



Lab Discussion Questions

Answer the following in writing:

1. **Sampling Design Choice:** In Lab Task 1, which sampling method produced the estimate closest to the true mean BMI? Is the “best” method always the most accurate for a single sample? What matters more — accuracy on average (bias) or consistency across samples (variance)?
2. **Sample Size Trade-offs:** In Lab Task 2, the required n drops substantially when prior knowledge of p is used. In practice, researchers often set $p = 0.5$ to be “safe.” Under what circumstances is this overly conservative, and when is it genuinely necessary?
3. **Bootstrap vs. Classical:** In Lab Task 3, the bootstrap was used for the 75th percentile. Could you use a classical formula instead? Look up or derive the asymptotic SE of a sample quantile. When does the bootstrap offer a real advantage?
4. **Representativeness:** Lab Task 4 tests whether the SRS sample matches population region proportions. Even if the test passes, does this guarantee the sample is representative on all variables? What else would you check?
5. **Real-World Application:** You are hired to estimate the prevalence of diabetes in Thailand using a sample of 2,000 adults. Describe your complete sampling strategy: sampling method, strata (if any), sample size justification, and how you would evaluate the final sample’s quality.

Chapter Summary

Summary

This chapter established sampling as the foundation of all empirical data science:

- **Why sampling matters** — sampling error is unavoidable but quantifiable; bias is avoidable but insidious. The MSE framework combines both, and sound sampling design is more important than large sample size.
- **Probability sampling** (SRS, systematic, stratified, cluster) provides known inclusion probabilities and supports valid inference. Stratified sampling improves precision when subgroups differ; cluster sampling reduces cost at the expense of precision.
- **Non-probability sampling** (convenience, purposive, snowball, quota) is widely used but does not support formal population-level inference; its limitations must be clearly acknowledged.
- **Sample size determination** balances desired precision (margin of error), confidence level, and population variability. The finite population correction reduces required n when sampling a substantial fraction of the population.
- **Sampling bias** (selection bias, non-response, undercoverage, survivorship bias) systematically distorts estimates and cannot be corrected by larger samples.
- **Bootstrap resampling** provides distribution-free uncertainty estimates for any statistic, requiring only that the sample represent the population.
- **Sample quality evaluation** using goodness-of-fit tests and compositional comparisons guards against over-confident inference from unrepresentative samples.

! Key Formulas to Know

Standard Error of Sample Mean:

$$SE(\bar{x}) = \sqrt{\frac{s^2}{n} \left(1 - \frac{n}{N}\right)}$$

Sample Size for Mean:

$$n = \left(\frac{z_{1-\alpha/2} \cdot \sigma}{E} \right)^2$$

Sample Size for Proportion:

$$n = \frac{z_{1-\alpha/2}^2 \cdot p(1-p)}{E^2}$$

Bootstrap Standard Error:

$$\widehat{\text{SE}}_{\text{boot}} = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}_b^* - \bar{\theta}^*)^2}$$

Design Effect:

$$\text{DEFF} = 1 + (m-1)\rho$$

Post-Stratification Weight:

$$w_i = \frac{p_i^{\text{pop}}}{p_i^{\text{sample}}}$$

References

- Cochran, W. G. (1977). *Sampling techniques* (3rd ed.). Wiley.
- Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *Annals of Statistics*, 7(1), 1–26.
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Lumley, T. (2023). *survey: Analysis of complex survey samples* (R package version 4.2). <https://CRAN.R-project.org/package=survey>
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Tillé, Y., & Matei, A. (2021). *sampling: Survey sampling* (R package version 2.9). <https://CRAN.R-project.org/package=sampling>
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.
- Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

End of Chapter 5. Proceed to Chapter 6: Data Preprocessing.

Chapter 6: Data Preprocessing

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(mice)           # multiple imputation
library(VIM)           # missing data visualization
library(car)           # Box-Cox transformation
library(MASS)          # boxcox()
library(recipes)        # preprocessing pipelines
library(naniar)         # missing data tools
library(corrplot)       # correlation visualization
```

Chapter Overview

Introduction

Raw data is almost never ready for analysis. In practice, data scientists spend the majority of their time — estimates range from 60% to 80% — cleaning, transforming, and preparing data before any modeling begins. This is not merely a technical chore: the decisions made during preprocessing directly shape the conclusions drawn from the analysis. A model trained on poorly preprocessed data is unreliable regardless of its algorithmic sophistication.

This chapter covers:

- **Why Preprocessing Matters** — the preprocessing pipeline and its role in data science
- **Handling Missing Data** — types of missingness, deletion strategies, and imputation
- **Outlier Detection and Treatment** — IQR, z-score, and multivariate methods
- **Data Transformation** — log, Box-Cox, normalization, and standardization
- **Encoding Categorical Variables** — dummy, one-hot, ordinal, and target encoding
- **Feature Scaling** — min-max, z-score, and robust scaling
- **Data Integration and Reshaping** — merging, pivoting, and tidy data principles

Learning Objectives

By the end of this chapter, you will be able to:

1. Describe the role of preprocessing in the data science pipeline.
2. Identify the type of missingness and apply appropriate handling strategies.
3. Detect and treat outliers using univariate and multivariate methods.
4. Apply appropriate transformations to correct distributional issues.
5. Encode categorical variables correctly for statistical and machine learning models.
6. Scale features appropriately for distance-based and gradient-based algorithms.
7. Reshape and integrate data using tidyverse tools following tidy data principles.

Why Preprocessing Matters

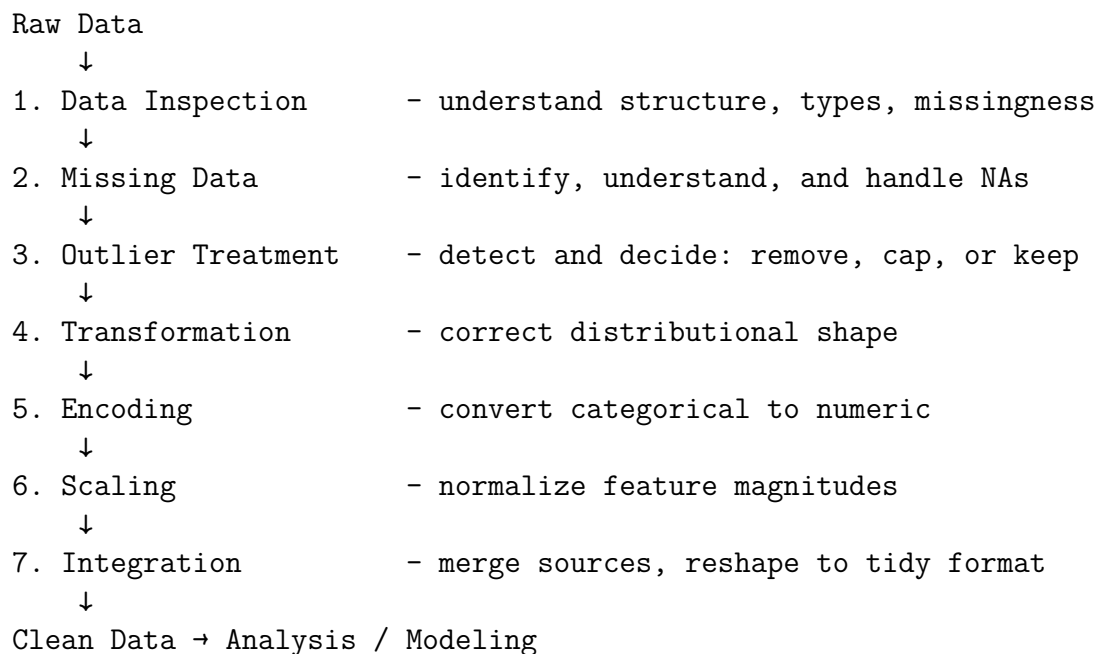
Introduction

The phrase “**garbage in, garbage out**” (GIGO) captures a fundamental truth in data science: the quality of any analysis is bounded by the quality of its input data. Missing values, outliers, inconsistent formatting, wrong data types, and poorly scaled features can silently distort statistical estimates, inflate model error, and lead to conclusions that seem analytically sound but are empirically baseless. Preprocessing is the systematic process of transforming raw data into a form suitable for analysis.

Theory

The Data Preprocessing Pipeline

A typical preprocessing pipeline consists of sequential steps applied before modeling:



The order matters: outliers should be addressed before scaling; missing data should be handled before transformation; encoding precedes scaling for newly created numeric features.

Why Each Step Matters

Table 1: Consequences of skipping preprocessing steps

Issue	Consequence if Ignored	Preprocessing Fix
Missing values	Biased estimates, model errors	Imputation or deletion
Outliers	Inflated variance, distorted means	Detection + treatment
Skewed distributions	Violated normality assumptions	Log/Box-Cox transformation
Mixed scales	Distance-based models dominated by large-scale features	Standardization
Raw categories	Most algorithms require numeric input	Encoding
Inconsistent formats	Merge failures, incorrect grouping	Cleaning + reshaping

The Leakage Problem

A critical principle in preprocessing: **all transformations must be fitted on training data only and then applied to test data**. Fitting a scaler or imputer on the full dataset before splitting allows test set information to “leak” into the training process, producing overly optimistic performance estimates. This is why preprocessing pipelines (like `recipes` in R or `sklearn.pipeline` in Python) are best practice — they enforce the correct order of operations and prevent leakage.

Example: The Cost of Skipping Preprocessing

Example 6.1. A data scientist trains a regression model to predict hospital readmission risk. The dataset has:

- 18% missing values in the key predictor “number of prior admissions” (not handled → model drops these rows, losing 18% of data and biasing toward patients with complete records)
- One outlier patient with 47 prior admissions (not handled → inflates the regression slope)

- Age in years and income in dollars (not scaled → income dominates distance calculations)
- Insurance type as a string variable (not encoded → model crashes or silently drops it)

Each issue independently degrades model performance. Together, they produce a model that is unreliable despite sophisticated methodology.

R Example: Initial Data Inspection

```
# --- Use airquality as a running example ---
data(airquality)
```

```
# Step 1: Structural overview
cat("=== Dataset Structure ===\n")
```

```
=== Dataset Structure ===
```

```
glimpse(airquality)
```

```
Rows: 153
Columns: 6
$ Ozone    <int> 41, 36, 12, 18, NA, 28, 23, 19, 8, NA, 7, 16, 11, 14, 18, 14, ~
$ Solar.R  <int> 190, 118, 149, 313, NA, NA, 299, 99, 19, 194, NA, 256, 290, 27~
$ Wind     <dbl> 7.4, 8.0, 12.6, 11.5, 14.3, 14.9, 8.6, 13.8, 20.1, 8.6, 6.9, 9~
$ Temp     <int> 67, 72, 74, 62, 56, 66, 65, 59, 61, 69, 74, 69, 66, 68, 58, 64~
$ Month    <int> 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, ~
$ Day      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, ~
```

```
# Step 2: Missing value summary
cat("\n=== Missing Values ===\n")
```

```
=== Missing Values ===
```

```
miss_summary <- data.frame(
  Variable    = names(airquality),
  Type        = sapply(airquality, class),
  N_Missing    = colSums(is.na(airquality)),
  Pct_Missing  = round(colMeans(is.na(airquality)) * 100, 1)
)
rownames(miss_summary) <- NULL
```

```
kable(miss_summary,
      caption = "Variable Overview and Missing Data",
      col.names = c("Variable", "Type", "N Missing", "% Missing")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(4, bold = TRUE,
            color = ifelse(miss_summary$Pct_Missing > 0,
                          "tomato", "darkgreen"))
```

Table 2: Variable Overview and Missing Data

Variable	Type	N Missing	% Missing
Ozone	integer	37	24.2
Solar.R	integer	7	4.6
Wind	numeric	0	0.0
Temp	integer	0	0.0
Month	integer	0	0.0
Day	integer	0	0.0

```
# Step 3: Basic descriptive statistics
summary(airquality)
```

Ozone	Solar.R	Wind	Temp
Min. : 1.00	Min. : 7.0	Min. : 1.700	Min. :56.00
1st Qu.: 18.00	1st Qu.:115.8	1st Qu.: 7.400	1st Qu.:72.00
Median : 31.50	Median :205.0	Median : 9.700	Median :79.00
Mean : 42.13	Mean :185.9	Mean : 9.958	Mean :77.88
3rd Qu.: 63.25	3rd Qu.:258.8	3rd Qu.:11.500	3rd Qu.:85.00
Max. :168.00	Max. :334.0	Max. :20.700	Max. :97.00
NA's :37	NA's :7		
Month	Day		
Min. :5.000	Min. : 1.0		
1st Qu.:6.000	1st Qu.: 8.0		
Median :7.000	Median :16.0		
Mean :6.993	Mean :15.8		
3rd Qu.:8.000	3rd Qu.:23.0		
Max. :9.000	Max. :31.0		

Code explanation:

- `glimpse()` provides a compact structural overview: variable names, types, and first few values — always the first step.
- Building a custom `miss_summary` data frame is more informative than `is.na()` alone — it quantifies the scale of the missing data problem per variable.
- `summary()` gives a quick distributional picture including min, max, quartiles, and NA counts — useful for spotting impossible values and gross outliers.

Exercises

Exercise 6.1

Load the `mtcars` dataset and perform a full initial inspection:

- (a) Check variable types with `str()` and `glimpse()`. Which variables are stored incorrectly?
 - (b) Create a missing value summary table. Are there any missing values?
 - (c) Use `summary()` to identify any suspicious values (impossible ranges, extreme outliers).
 - (d) Write a 100-word “data quality report” describing the state of the data before preprocessing.
-

Handling Missing Data

Introduction

Missing data is one of the most pervasive problems in real-world datasets. How missing values are handled has a profound effect on the validity of subsequent analysis — and the wrong strategy can introduce bias far worse than the original missingness. The first step is always to understand **why** data is missing, because the appropriate handling strategy depends on the mechanism of missingness.

Theory

Types of Missingness

Little and Rubin (1987) established the canonical taxonomy of missing data mechanisms:

Missing Completely At Random (MCAR): The probability of missingness is unrelated to any variable — observed or missing. The missing data is a random subsample of the complete data.

$$P(\text{missing} \mid Y_{\text{obs}}, Y_{\text{mis}}) = P(\text{missing})$$

Example: A sensor randomly malfunctions regardless of the measurement value. MCAR is the most benign mechanism — complete case analysis (listwise deletion) is unbiased, though it loses power.

Missing At Random (MAR): The probability of missingness depends on **observed** variables but not on the missing values themselves — after conditioning on observed data.

$$P(\text{missing} \mid Y_{\text{obs}}, Y_{\text{mis}}) = P(\text{missing} \mid Y_{\text{obs}})$$

Example: Older patients are less likely to complete a digital survey (age is observed; the missing survey responses are not related to their content). MAR is the most common mechanism and supports valid imputation.

Missing Not At Random (MNAR): The probability of missingness depends on the missing values themselves — even after conditioning on observed data.

$$P(\text{missing} \mid Y_{\text{obs}}, Y_{\text{mis}}) \neq P(\text{missing} \mid Y_{\text{obs}})$$

Example: People with very high incomes decline to report income. MNAR is the most problematic — no standard statistical method can fully correct for it without additional assumptions or external data.

Strategies for Handling Missing Data

1. Complete Case Analysis (Listwise Deletion)

Remove all rows containing any missing value. Simple and default in most software.

- Valid and unbiased under MCAR
- Loses power; biased under MAR or MNAR
- Can lose substantial data if missingness is spread across many variables

2. Mean / Median / Mode Imputation

Replace missing values with the variable's mean (continuous), median (skewed/ordinal), or mode (categorical).

- Simple and fast
- Underestimates variance (artificially concentrates values at center)
- Distorts correlations between variables
- Valid only approximately under MCAR

3. Multiple Imputation (MI)

Generate m complete datasets by imputing missing values m times using a statistical model (typically a multivariate normal or predictive mean matching), analyze each, and combine results using **Rubin's rules**.

$$\hat{\theta} = \frac{1}{m} \sum_{i=1}^m \hat{\theta}_i, \quad \text{Var}(\hat{\theta}) = \bar{W} + \left(1 + \frac{1}{m}\right) B$$

where $\bar{W}$ is the within-imputation variance and B is the between-imputation variance.

- Valid under MAR; gold standard for missing data handling
- Correctly propagates uncertainty from missingness
- More complex to implement and interpret

4. Model-Based Imputation

Use a regression model to predict missing values from other variables (K-nearest neighbors imputation, random forest imputation).

- Preserves relationships between variables better than mean imputation
- Works well for MAR data
- Risk of overfitting if not cross-validated

Diagnosing the Missingness Mechanism

Table 1: Missingness diagnostics

Test	Purpose
Compare observed vs. missing on other variables	Check for MAR patterns
Little's MCAR test	Formal test of MCAR assumption
Missing data pattern visualization	Identify which variables co-occur in missingness

Example: Missing Data in a Clinical Study

Example 6.2. A clinical dataset records blood pressure, cholesterol, age, and BMI for 500 patients. Cholesterol has 22% missing values. Investigation reveals: older patients (age > 65) are less likely to have cholesterol measured (they were referred to a different lab). Since age is observed and the missingness depends on age (not on the cholesterol value itself), this is **MAR** — multiple imputation using age and other observed variables as predictors is the appropriate strategy.

If instead, patients with very high cholesterol avoided the test (fearing bad news), this would be **MNAR** — no standard imputation method can fully correct for this without external information.

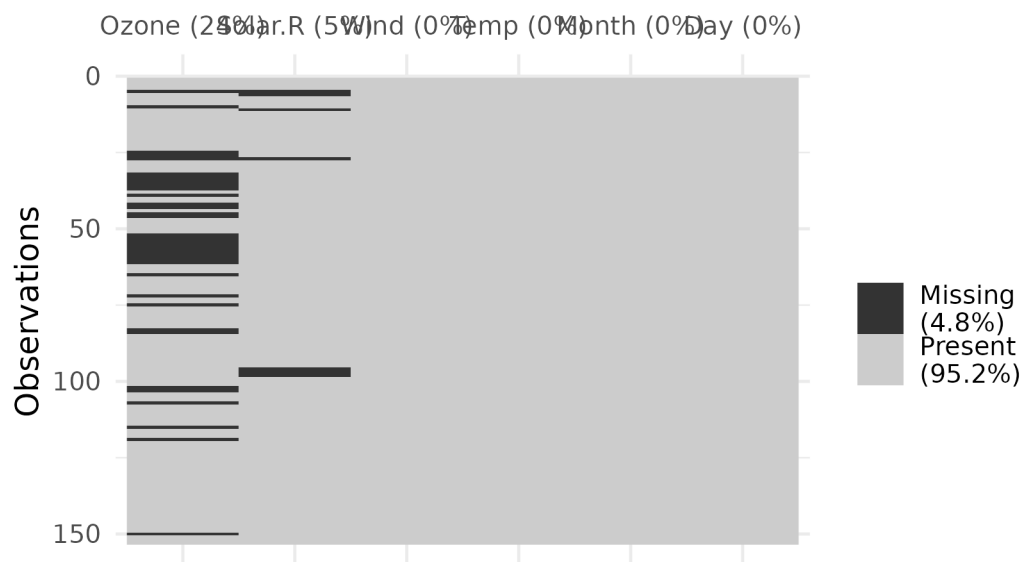
R Example: Missing Data Analysis and Imputation

```
# --- Missing data visualization ---  
# airquality: Ozone (37 missing) and Solar.R (7 missing)  
  
# Pattern of missingness  
naniar::vis_miss(airquality) +
```



```
labs(title = "Missing Data Pattern: airquality Dataset") +
theme_minimal(base_size = 12)
```

Missing Data Pattern: airquality Dataset



```
# --- Explore MAR: does missingness in Ozone relate to other vars? ---
airquality <- airquality |>
mutate(ozone_missing = is.na(Ozone))

# Compare Temp and Wind between missing and non-missing Ozone groups
missing_comparison <- airquality |>
group_by(ozone_missing) |>
summarise(
  n = n(),
  Mean_Temp = round(mean(Temp, na.rm = TRUE), 2),
  Mean_Wind = round(mean(Wind, na.rm = TRUE), 2),
  Mean_Solar = round(mean(Solar.R, na.rm = TRUE), 2),
  .groups = "drop"
)

kable(missing_comparison,
  caption = "Variable Means by Ozone Missingness Status",
  col.names = c("Ozone Missing?", "n", "Mean Temp",
    "Mean Wind", "Mean Solar.R")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
  full_width = FALSE)
```

Table 2: Variable Means by Ozone Missingness Status

Ozone Missing?	n	Mean Temp	Mean Wind	Mean Solar.R
FALSE	116	77.87	9.86	184.80
TRUE	37	77.92	10.26	189.51

```
# --- Compare imputation methods ---
airquality_clean <- airquality |> dplyr::select(-ozone_missing)

# Method 1: Complete case analysis
cc_data <- na.omit(airquality_clean)
cat("Complete cases:", nrow(cc_data),
    "of", nrow(airquality_clean), "\n")
```

Complete cases: 111 of 153

```
# Method 2: Mean imputation
mean_imputed <- airquality_clean |>
  mutate(
    Ozone = ifelse(is.na(Ozone),
                   mean(Ozone, na.rm = TRUE), Ozone),
    Solar.R = ifelse(is.na(Solar.R),
                     mean(Solar.R, na.rm = TRUE), Solar.R)
  )

# Method 3: Multiple imputation via mice
set.seed(42)
mice_out <- mice(airquality_clean, m = 5,
                 method = "pmm", printFlag = FALSE)
mi_data <- complete(mice_out, 1) # first imputed dataset

# Compare distributions of imputed Ozone
imp_compare <- data.frame(
  Method = c("Complete Cases", "Mean Imputation",
             "Multiple Imputation (MI)"),
  n = c(nrow(cc_data), nrow(mean_imputed), nrow(mi_data)),
  Mean_Ozone = round(c(mean(cc_data$Ozone),
                       mean(mean_imputed$Ozone),
                       mean(mi_data$Ozone)), 2),
  SD_Ozone = round(c(sd(cc_data$Ozone),
                     sd(mean_imputed$Ozone),
                     sd(mi_data$Ozone)), 2)
)
```

```
kable(imp_compare,
      caption = "Imputation Method Comparison: Ozone Variable",
      col.names = c("Method", "n", "Mean Ozone", "SD Ozone")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE)
```

Table 3: Imputation Method Comparison: Ozone Variable

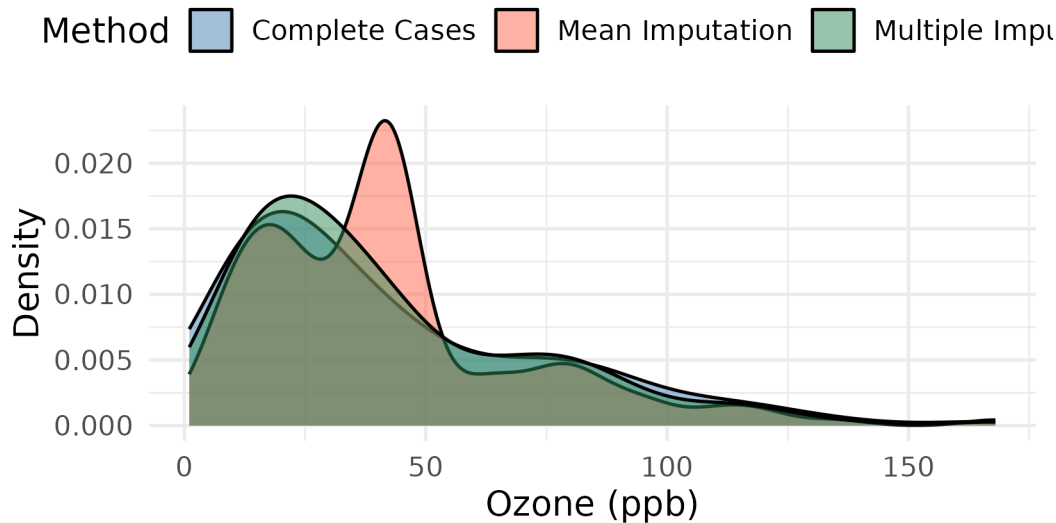
Method	n	Mean Ozone	SD Ozone
Complete Cases	111	42.10	33.28
Mean Imputation	153	42.13	28.69
Multiple Imputation (MI)	153	41.50	30.91

```
# --- Visualize effect of imputation on distribution ---
dist_df <- bind_rows(
  data.frame(Ozone = cc_data$Ozone,
             Method = "Complete Cases"),
  data.frame(Ozone = mean_imputed$Ozone,
             Method = "Mean Imputation"),
  data.frame(Ozone = mi_data$Ozone,
             Method = "Multiple Imputation")
)

ggplot(dist_df, aes(x = Ozone, fill = Method)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("steelblue", "tomato", "seagreen")) +
  labs(title = "Ozone Distribution Under Different Imputation Methods",
       subtitle = "Mean imputation artificially spikes at the mean; MI preserves shape",
       x = "Ozone (ppb)", y = "Density",
       fill = "Method") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")
```

Ozone Distribution Under Different Imputation

Mean imputation artificially spikes at the mean; MI μ



Code explanation:

- `naniar::vis_miss()` produces a tile plot of missingness — each column is a variable, each row an observation, and missing values appear in a contrasting color.
- `mice(data, m=5, method="pmm")` runs multiple imputation with predictive mean matching — the most robust method for continuous variables. `m=5` creates 5 imputed datasets.
- The density plot reveals why mean imputation is problematic: it creates an artificial spike at the mean, underestimating variance and distorting the distribution.

Exercises

💡 Exercise 6.2

Using the `airquality` dataset:

- (a) Test whether `Ozone` missingness is related to `Month` using a chi-square test or proportion comparison. What type of missingness does this suggest?
- (b) Implement KNN imputation using `VIM::kNN()`. Compare the distribution of imputed `Ozone` to mean imputation and MI.
- (c) After multiple imputation (`m=5`), pool results using `mice::pool()` for a linear regression of `Ozone` on `Temp` and `Wind`. Report the pooled coefficients.

Exercise 6.3

Create a dataset with MNAR missingness: simulate 200 observations of income where values above 100,000 are set to NA (high earners hide income).

- (a) Apply mean imputation and MI. Compare estimates of the mean income to the true value.
 - (b) Show that neither method recovers the true mean. What does this demonstrate about MNAR?
-

Outlier Detection and Treatment

Introduction

An **outlier** is an observation that deviates markedly from the general pattern of the data. Outliers arise from data entry errors, measurement failures, genuine extreme values, or population heterogeneity. The key question is always: *Is this outlier an error to be corrected, or a genuine extreme value to be retained?* Blindly removing outliers is as dangerous as blindly keeping them — context and domain knowledge are essential.

Theory

Univariate Methods

IQR Method (Tukey’s Fences):

An observation is a potential outlier if it falls below the lower fence or above the upper fence:

$$\text{Lower fence} = Q_1 - 1.5 \times \text{IQR}$$

$$\text{Upper fence} = Q_3 + 1.5 \times \text{IQR}$$

Using a multiplier of 3.0 instead of 1.5 defines **extreme outliers**. This method is robust — it uses quartiles rather than the mean, so it is not affected by the outliers it is trying to detect.

Z-Score Method:

An observation is an outlier if its standardized value exceeds a threshold (typically $|z| > 3$):

$$z_i = \frac{x_i - \bar{x}}{s}$$

This method is sensitive to the outliers themselves (since $\bar{x}$ and s are both affected), making it less reliable than the IQR method for heavily contaminated data. The **modified z-score** using the median and MAD (median absolute deviation) is more robust:

$$\tilde{z}_i = \frac{0.6745(x_i - \tilde{x})}{\text{MAD}}, \quad \text{MAD} = \text{median}(|x_i - \tilde{x}|)$$

Observations with $|\tilde{z}_i| > 3.5$ are flagged as outliers.

Multivariate Methods

Mahalanobis Distance:

For multivariate data, univariate methods miss outliers that are unusual in the joint distribution but not in any individual variable. The Mahalanobis distance measures how far an observation is from the multivariate centroid, accounting for correlations:

$$D_i^2 = (\mathbf{x}_i - \bar{\mathbf{x}})^T \mathbf{S}^{-1} (\mathbf{x}_i - \bar{\mathbf{x}})$$

Under multivariate normality, $D_i^2 \sim \chi^2(p)$ where p is the number of variables. Observations with $D_i^2 > \chi_{p,0.975}^2$ are flagged.

Treatment Options

Table 1: Outlier treatment options

Treatment	When to Use
Remove	Confirmed data entry errors or measurement failures
Cap (Winsorize)	Genuine extreme values; replace with fence values
Transform	Log or sqrt reduces the influence of large values
Keep	Genuine extreme values that are part of the population
Robust methods	Use median-based estimators instead of mean-based

Never Remove Outliers Automatically

Always investigate before removing. An outlier might be the most scientifically interesting observation. Document every removal decision with a justification.

Example: Outlier Detection

Example 6.3. In a dataset of 100 monthly salaries (THB), the IQR method flags a salary of 850,000 THB as an outlier (all others are below 120,000). Investigation reveals this is the CEO — a genuine extreme value, not an error. The appropriate treatment is to **retain** it for descriptive analysis but **report separately** when computing “typical” employee salary, or to use the **median** as the summary statistic.

R Example: Outlier Detection and Treatment

```
# --- Univariate outlier detection ---
data(airquality)
ozone <- na.omit(airquality$Ozone)

# IQR method
Q1 <- quantile(ozone, 0.25)
Q3 <- quantile(ozone, 0.75)
IQR_val <- Q3 - Q1
lower <- Q1 - 1.5 * IQR_val
upper <- Q3 + 1.5 * IQR_val

# Z-score method
z_scores <- abs(scale(ozone))

# Modified z-score (robust)
MAD_val <- mad(ozone)
mod_z <- abs(0.6745 * (ozone - median(ozone)) / MAD_val)

outlier_summary <- data.frame(
  Method = c("IQR (1.5×IQR)",
             "IQR (3.0×IQR - extreme)",
             "Z-score (|z|>3)",
             "Modified Z-score (|z̄|>3.5)"),
  N_Flagged = c(
    sum(ozone < lower | ozone > upper),
    sum(ozone < Q1 - 3*IQR_val | ozone > Q3 + 3*IQR_val),
    sum(z_scores > 3),
    sum(mod_z > 3.5)
  ),
  Values_Flagged = c(
    paste(sort(ozone[ozone < lower | ozone > upper]),
          collapse = ", ")
  )
)
```



```

paste(sort(ozone[ozone < Q1-3*IQR_val |
              ozone > Q3+3*IQR_val])),
      collapse = ", "),
paste(sort(ozone[z_scores > 3]), collapse = ", "),
paste(sort(ozone[mod_z > 3.5]), collapse = ", ")
)
)

kable(outlier_summary,
      caption = "Outlier Detection: Ozone Variable",
      col.names = c("Method", "N Flagged", "Values")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE)

```

Table 2: Outlier Detection: Ozone Variable

Method	N Flagged	Values
IQR (1.5×IQR)	2	135, 168
IQR (3.0×IQR — extreme)	0	
Z-score ($ z > 3$)	1	168
Modified Z-score ($ z_{mod} > 3.5$)	1	168

```

# --- Multivariate outlier: Mahalanobis distance ---
mtcars_cont <- mtcars[, c("mpg", "hp", "wt", "disp")]

maha_dist <- mahalanobis(mtcars_cont,
                        colMeans(mtcars_cont),
                        cov(mtcars_cont))
chi_crit <- qchisq(0.975, df = ncol(mtcars_cont))

outlier_flag <- maha_dist > chi_crit

cat("Mahalanobis Distance Outliers ( ^2 , p=0.975 threshold =",
    round(chi_crit, 2), "):\n")

```

Mahalanobis Distance Outliers (^2 , p=0.975 threshold = 11.14):

```

print(mtcars[outlier_flag,
              c("mpg", "hp", "wt", "disp")])

```

```

      mpg  hp   wt disp
Maserati Bora  15 335 3.57 301

```

```
# --- Winsorization (capping) ---
winsorize <- function(x, lower_p = 0.05, upper_p = 0.95) {
  lo <- quantile(x, lower_p, na.rm = TRUE)
  hi <- quantile(x, upper_p, na.rm = TRUE)
  pmin(pmax(x, lo), hi)
}

ozone_wins <- winsorize(ozone, 0.05, 0.95)

cat("\nOzone before Winsorization: mean =",
    round(mean(ozone), 2), " sd =", round(sd(ozone), 2), "\n")
```

Ozone before Winsorization: mean = 42.13 sd = 32.99

```
cat("Ozone after Winsorization: mean =",
    round(mean(ozone_wins), 2), " sd =",
    round(sd(ozone_wins), 2), "\n")
```

Ozone after Winsorization: mean = 41.25 sd = 30.06

```
# --- Visualize outliers ---
ozone_df <- data.frame(
  ozone = ozone,
  outlier = ozone > upper | ozone < lower
)

p1 <- ggplot(ozone_df, aes(x = "", y = ozone)) +
  geom_boxplot(fill = "steelblue", alpha = 0.6,
               outlier.shape = NA) +
  geom_jitter(aes(color = outlier), width = 0.15,
              size = 2.5, alpha = 0.8) +
  geom_hline(yintercept = c(lower, upper),
             color = "tomato", linetype = "dashed",
             linewidth = 0.9) +
  scale_color_manual(values = c("FALSE" = "steelblue",
                                "TRUE" = "tomato")) +
  labs(title = "A. IQR Outlier Detection",
       subtitle = paste0("Red dashed = fences [",
                          round(lower,1), ", ",
                          round(upper,1), "]" ),
       x = "", y = "Ozone (ppb)",
       color = "Outlier") +
```

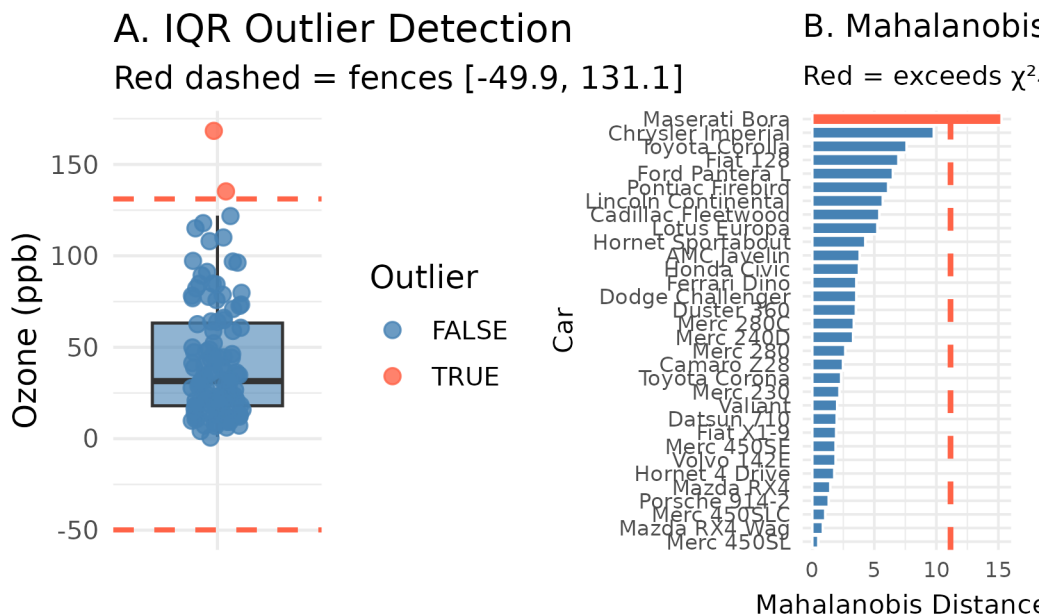
```

theme_minimal(base_size = 12)

p2 <- ggplot(data.frame(car = rownames(mtcars),
                        maha = maha_dist,
                        flag = outlier_flag),
            aes(x = reorder(car, maha), y = maha,
                fill = flag)) +
  geom_col(color = "white") +
  geom_hline(yintercept = chi_crit, color = "tomato",
            linetype = "dashed", linewidth = 1) +
  scale_fill_manual(values = c("FALSE" = "steelblue",
                              "TRUE" = "tomato")) +
  coord_flip() +
  labs(title = "B. Mahalanobis Distance (mtcars)",
       subtitle = paste0("Red = exceeds  $\chi^2$  threshold (",
                          round(chi_crit,1), ")"),
       x = "Car", y = "Mahalanobis Distance2",
       fill = "Outlier") +
  theme_minimal(base_size = 10) +
  theme(legend.position = "none")

p1 + p2

```



Code explanation:

- `mahalanobis(x, center, cov)` computes squared Mahalanobis distances; `qchisq(0.975,`

`df=p)` gives the critical value.

- The `winsorize()` function clamps extreme values at specified percentiles — a common and principled alternative to deletion.
- The bar chart of Mahalanobis distances (Panel B) makes it easy to rank observations by multivariate extremity and identify which cars are unusual across all dimensions simultaneously.

Exercises

Exercise 6.4

Using the `mtcars` dataset:

- Apply IQR and modified z-score outlier detection to `hp`. Do both methods agree?
- Compute Mahalanobis distances using `mpg`, `hp`, `wt`, and `qsec`. Which cars are multivariate outliers?
- Compare the regression of `mpg ~ hp + wt` with and without the identified outliers. How much do the coefficients change?

Exercise 6.5

For the `airquality` Ozone variable:

- Winsorize at the 5th and 95th percentiles. Plot original vs. winsorized distributions.
- Compare the mean and SD before and after winsorization.
- Would you recommend winsorization or log transformation for this variable? Justify using skewness and normality tests.

Data Transformation

Introduction

Many statistical methods assume that variables are approximately normally distributed, have constant variance, or are measured on a linear scale. When these assumptions are violated — as they often are with real data — **transformations** can correct the distributional shape, stabilize variance, and linearize relationships. Transformations are applied to the variable itself, not to the model; after transformation, the standard methods can proceed validly.

Theory

Log Transformation

The most common transformation for right-skewed, positive data:

$$x' = \log(x) \quad (\text{natural log}) \quad \text{or} \quad x' = \log_{10}(x)$$

When to use: Right-skewed distributions (income, reaction times, biological measurements, counts). Requires $x > 0$; use $\log(x + c)$ for data containing zeros, where c is a small constant.

Effect: Compresses large values more than small ones, pulling the right tail toward the center.

Square Root Transformation

$$x' = \sqrt{x}$$

When to use: Count data (Poisson-distributed), moderate right skew. Less aggressive than log; applicable when $x \geq 0$.

Box-Cox Transformation

A flexible family of power transformations that includes log and square root as special cases:

$$x'(\lambda) = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \log(x) & \lambda = 0 \end{cases}$$

The optimal λ is estimated by maximum likelihood to maximize normality. Common values: $\lambda = 1$ (no transformation), $\lambda = 0.5$ (square root), $\lambda = 0$ (log), $\lambda = -1$ (reciprocal).

Yeo-Johnson Transformation

An extension of Box-Cox that handles **zero and negative values**:

$$\psi(x, \lambda) = \begin{cases} [(x+1)^\lambda - 1]/\lambda & x \geq 0, \lambda \neq 0 \\ \log(x+1) & x \geq 0, \lambda = 0 \\ -[(-x+1)^{2-\lambda} - 1]/(2-\lambda) & x < 0, \lambda \neq 2 \\ -\log(-x+1) & x < 0, \lambda = 2 \end{cases}$$

Normalization vs. Standardization

These are **scaling** operations, covered in detail in Section 6, but distinguished here from transformation:

- **Transformation** changes the distributional **shape** (skewness, tail behavior).
- **Scaling/Normalization** changes the **location and spread** without changing the shape.

Example: Choosing a Transformation

Example 6.4. Household income data has skewness = 3.8, indicating severe right skew. Three transformations are compared:

Transformation	Skewness After	Shapiro-Wilk p
None	3.8	< 0.001
Square root	1.9	0.008
Log	0.4	0.21
Box-Cox ($\lambda = 0.12$)	0.1	0.38

Box-Cox achieves the best normalization, followed closely by log transformation. For most practical purposes, log transformation is preferred for its interpretability (log-income differences correspond to percentage differences).

R Example: Data Transformations

```
# --- Compare transformations on Ozone ---
data(airquality)
ozone <- na.omit(airquality$Ozone)

# Compute skewness before and after
transforms <- data.frame(
  Transformation = c("None", "Square Root",
                     "Log", "Box-Cox"),
  Data           = list(
    ozone,
    sqrt(ozone),
    log(ozone),
    NA # filled below
  )
)

# Box-Cox optimal lambda
bc <- MASS::boxcox(ozone ~ 1, lambda = seq(-2, 2, 0.1),
                  plotit = FALSE)
lambda_opt <- bc$x[which.max(bc$y)]
cat("Optimal Box-Cox lambda:", round(lambda_opt, 3), "\n")
```

Optimal Box-Cox lambda: 0.2

```
ozone_bc <- (ozone^lambda_opt - 1) / lambda_opt

# Summary table
transform_summary <- data.frame(
  Transformation = c("None", "Square Root", "Log",
                     paste0("Box-Cox (=",
                           round(lambda_opt, 2), ")")),
  Mean           = round(c(mean(ozone), mean(sqrt(ozone)),
                          mean(log(ozone)), mean(ozone_bc)), 3),
  SD             = round(c(sd(ozone), sd(sqrt(ozone)),
                          sd(log(ozone)), sd(ozone_bc)), 3),
```

```

Skewness      = round(c(moments::skewness(ozone),
                        moments::skewness(sqrt(ozone)),
                        moments::skewness(log(ozone)),
                        moments::skewness(ozone_bc)), 3),
SW_p          = round(c(shapiro.test(ozone)$p.value,
                        shapiro.test(sqrt(ozone))$p.value,
                        shapiro.test(log(ozone))$p.value,
                        shapiro.test(ozone_bc)$p.value), 4)
)

kable(transform_summary,
      caption = "Transformation Comparison: Ozone Variable",
      col.names = c("Transformation", "Mean", "SD",
                    "Skewness", "Shapiro-Wilk p")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(4, bold = TRUE,
            color = ifelse(abs(transform_summary$Skewness) < 0.5,
                          "darkgreen", "tomato"))

```

Table 1: Transformation Comparison: Ozone Variable

Transformation	Mean	SD	Skewness	Shapiro-Wilk p
None	42.129	32.988	1.226	0.0000
Square Root	6.023	2.430	0.514	0.0044
Log	3.419	0.865	-0.555	0.0147
Box-Cox (=0.2)	5.050	1.676	-0.024	0.3399

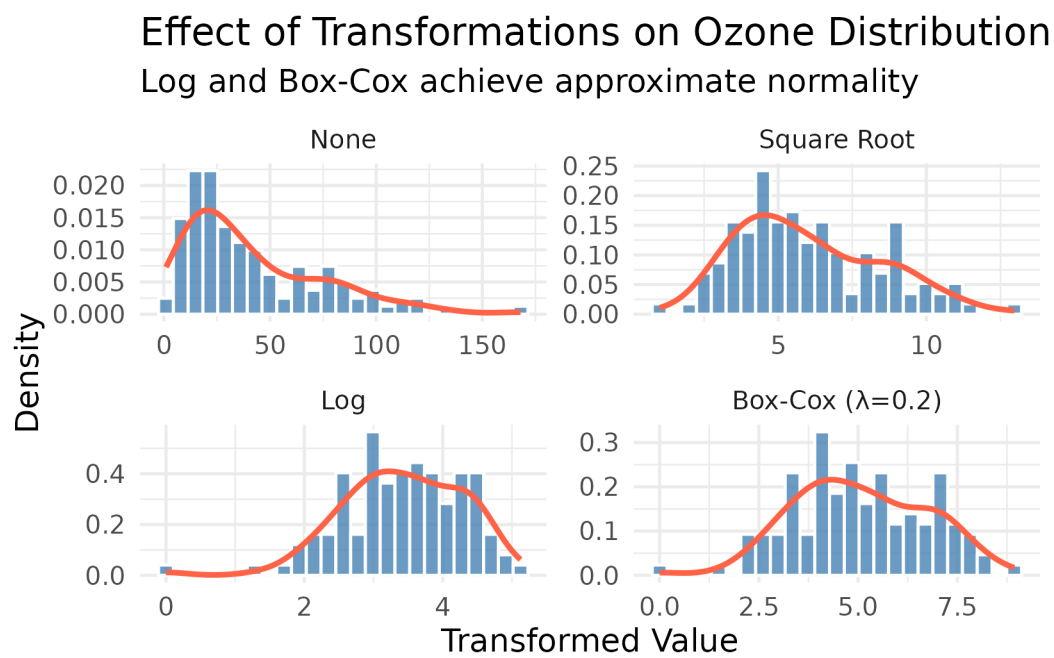
```

# --- Visual comparison of transformations ---
trans_df <- data.frame(
  None      = ozone,
  Square_Root = sqrt(ozone),
  Log       = log(ozone),
  Box_Cox   = ozone_bc
) |>
pivot_longer(everything(),
             names_to = "Transformation",
             values_to = "Value") |>
mutate(Transformation = factor(
  Transformation,
  levels = c("None", "Square_Root", "Log", "Box_Cox"),
  labels = c("None", "Square Root", "Log",
             paste0("Box-Cox ( =", round(lambda_opt, 2), ")"))))

```



```
ggplot(trans_df, aes(x = Value)) +
  geom_histogram(aes(y = after_stat(density)),
    bins = 25, fill = "steelblue",
    color = "white", alpha = 0.8) +
  geom_density(color = "tomato", linewidth = 1) +
  facet_wrap(~Transformation, scales = "free", ncol = 2) +
  labs(title = "Effect of Transformations on Ozone Distribution",
    subtitle = "Log and Box-Cox achieve approximate normality",
    x = "Transformed Value", y = "Density") +
  theme_minimal(base_size = 12)
```



Code explanation:

- `MASS::boxcox(y ~ 1, lambda)` estimates the optimal λ by maximum likelihood. The `plotit = FALSE` option suppresses the automatic plot so we can create our own.
- `pivot_longer()` reshapes the wide data frame of transformed values into a long format suitable for `facet_wrap()`.
- `facet_wrap(scales = "free")` allows each panel to have its own axis range — essential when the transformed scales differ dramatically.

Exercises

Exercise 6.6

Using the `mtcars` dataset:

- (a) Apply log, square root, and Box-Cox transformations to `hp`. Compare skewness and Shapiro-Wilk p-values.
 - (b) Apply the Yeo-Johnson transformation using `bestNormalize::yeojohnson()`. Does it outperform Box-Cox?
 - (c) For `mpg`, determine the optimal Box-Cox λ . Is a transformation needed?
-

Encoding Categorical Variables

Introduction

Most statistical models and machine learning algorithms operate on numeric data. **Encoding** converts categorical variables into numeric representations that preserve the relevant information while meeting the mathematical requirements of the algorithm. Choosing the wrong encoding can introduce artificial ordering, create near-singular matrices, or lose important information about category relationships.

Theory

Dummy Coding (Treatment Coding)

For a categorical variable with k levels, create $k - 1$ binary (0/1) dummy variables, leaving one category as the **reference** (baseline):

Category	D1 (Arts)	D2 (Business)
Science (reference)	0	0
Arts	1	0
Business	0	1

The coefficient for each dummy variable represents the difference from the reference category. This is the standard encoding for **linear regression and ANOVA** — using k dummies causes perfect multicollinearity (dummy variable trap).

One-Hot Encoding

Create k binary variables, one per category — each is 1 if the observation belongs to that category, 0 otherwise. Unlike dummy coding, **no reference category** is dropped.

Used in **machine learning** algorithms (tree-based models, neural networks) that do not suffer from multicollinearity. Problematic for linear regression without regularization.

Ordinal Encoding

Assign consecutive integers to ordered categories:

$$\text{Low} \rightarrow 1, \quad \text{Medium} \rightarrow 2, \quad \text{High} \rightarrow 3$$

Valid for **ordinal variables** where the ordering is meaningful and the assumption of equal spacing is reasonable. **Never** apply ordinal encoding to nominal variables — it imposes a spurious ordering.

Target Encoding (Mean Encoding)

Replace each category with the **mean of the target variable** for that category:

$$\text{Encoded value for category } c = \frac{1}{n_c} \sum_{i: x_i=c} y_i$$

Powerful for high-cardinality categorical variables (many categories), but prone to **overfitting** — requires smoothing or cross-validation. Should never be applied using the full dataset; use out-of-fold means in a cross-validation framework.

Binary Encoding

Convert the integer-encoded category to **binary representation** using $\log(k)$ bits. More compact than one-hot for high-cardinality variables.

Encoding Selection Guide

Table 1: Categorical encoding selection guide

Variable Type	Recommended Encoding	Notes
Nominal, few levels ($k \leq 10$)	Dummy (regression) or One-hot (ML)	Drop reference for regression
Nominal, many levels ($k > 10$)	Target or Binary encoding	Avoid one-hot explosion
Ordinal, equal spacing	Ordinal integer	Verify spacing assumption
Ordinal, unequal spacing	Dummy or contrast coding	Preserves non-linearity

Variable Type	Recommended Encoding	Notes
Binary	Keep as 0/1	No encoding needed

Example: Encoding Faculty Variable

Example 6.5. A dataset contains `faculty` (Science, Arts, Business) and `outcome_satisfaction`. For a linear regression:

- **Dummy coding** (reference = Science): Creates `Arts` (1/0) and `Business` (1/0). Coefficients represent mean satisfaction difference from Science.
- **One-hot** (for a random forest): Creates `Science`, `Arts`, `Business` — all three binary columns.
- **Ordinal** (inappropriate here): Would assign 1, 2, 3 — implying Arts is “twice” Science, which is meaningless.

R Example: Categorical Encoding

```
# --- Build example dataset ---
set.seed(42)
n <- 200
student_data <- data.frame(
  faculty      = sample(c("Science", "Arts", "Business"),
                        n, replace = TRUE,
                        prob = c(0.4, 0.35, 0.25)),
  year         = sample(c("First", "Second", "Third", "Fourth"),
                        n, replace = TRUE),
  satisfaction = NA
)
student_data$satisfaction <-
  60 +
  ifelse(student_data$faculty == "Science", 10,
  ifelse(student_data$faculty == "Arts", 7, 0)) +
  ifelse(student_data$year == "Fourth", 8,
  ifelse(student_data$year == "Third", 5,
  ifelse(student_data$year == "Second", 3, 0))) +
  rnorm(n, 0, 8)

cat("Original data (first 6 rows):\n")
```

Original data (first 6 rows):

```
head(student_data)
```

	faculty	year	satisfaction
1	Business	Fourth	51.99257
2	Business	Second	65.67022
3	Science	First	79.37060
4	Business	Fourth	84.47631
5	Arts	Third	60.98511
6	Arts	First	57.79316

```
# === 1. DUMMY CODING (reference = Business) ===
student_data$faculty <- factor(student_data$faculty,
                                levels = c("Business",
                                              "Science", "Arts"))
dummy_encoded <- model.matrix(~ faculty, data = student_data)
cat("\nDummy coding (first 6 rows):\n")
```

Dummy coding (first 6 rows):

```
head(dummy_encoded)
```

	(Intercept)	facultyScience	facultyArts
1	1	0	0
2	1	0	0
3	1	1	0
4	1	0	0
5	1	0	1
6	1	0	1

```
# === 2. ONE-HOT ENCODING ===
onehot_encoded <- model.matrix(~ faculty - 1,
                                data = student_data)
cat("One-hot encoding (first 6 rows):\n")
```

One-hot encoding (first 6 rows):

```
head(onehot_encoded)
```

	facultyBusiness	facultyScience	facultyArts
1	1	0	0
2	1	0	0
3	0	1	0
4	1	0	0
5	0	0	1
6	0	0	1

```
# === 3. ORDINAL ENCODING ===
student_data$year_ordinal <- as.integer(factor(
  student_data$year,
  levels = c("First","Second","Third","Fourth"),
  ordered = TRUE
))

cat("Ordinal encoding (year):\n")
```

Ordinal encoding (year):

```
head(student_data[, c("year","year_ordinal")])
```

	year	year_ordinal
1	Fourth	4
2	Second	2
3	First	1
4	Fourth	4
5	Third	3
6	First	1

```
# === 4. TARGET ENCODING ===
faculty_means <- student_data |>
  group_by(faculty) |>
  summarise(target_enc = mean(satisfaction), .groups = "drop")

student_data <- student_data |>
  left_join(faculty_means, by = "faculty")

cat("\nTarget encoding (faculty → mean satisfaction):\n")
```

Target encoding (faculty → mean satisfaction):

```
print(faculty_means)
```

```
# A tibble: 3 x 2
  faculty target_enc
  <fct>      <dbl>
1 Business    65.1
2 Science    73.9
3 Arts       69.5
```

```
# Compare: dummy regression vs. target-encoded model
model_dummy <- lm(satisfaction ~ faculty,
                  data = student_data)
model_target <- lm(satisfaction ~ target_enc,
                  data = student_data)

cat("\nDummy model R2: ",
    round(summary(model_dummy)$r.squared, 4), "\n")
```

```
Dummy model R2: 0.1669
```

```
cat("Target enc. R2: ",
    round(summary(model_target)$r.squared, 4), "\n")
```

```
Target enc. R2: 0.1669
```

Code explanation:

- `model.matrix(~ factor)` creates dummy-coded columns automatically, dropping the reference level. `model.matrix(~ factor - 1)` creates one-hot encoding (no reference dropped).
- `factor(..., ordered = TRUE)` followed by `as.integer()` implements ordinal encoding, preserving the category order.
- Target encoding is implemented via `group_by() |> summarise() + left_join()`. In production, this should be done within cross-validation folds to prevent data leakage.

Exercises

Exercise 6.7

Using the `iris` dataset:

- (a) Apply dummy coding to `Species` (reference = `setosa`). Verify with `model.matrix()`.
 - (b) Apply one-hot encoding. Confirm you have 3 columns instead of 2.
 - (c) Fit a linear regression of `Petal.Length ~ Species` using dummy encoding. Interpret each coefficient.
 - (d) When would you prefer target encoding over one-hot for the `Species` variable?
-

Feature Scaling

Introduction

Many statistical and machine learning algorithms are sensitive to the **scale** of input features. K-nearest neighbors computes Euclidean distances — a variable measured in thousands will dominate one measured in single digits. Gradient descent converges faster when features are on similar scales. Principal Component Analysis (Chapter 7) is entirely scale-dependent. **Feature scaling** brings all features to a comparable range without changing their distributional shape.

Theory

Min-Max Normalization

Scales each feature to the range $[0, 1]$:

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Properties: - Preserves the shape of the original distribution - Sensitive to outliers (extreme values compress all others toward 0 or 1) - Appropriate for algorithms requiring bounded input (neural networks with sigmoid activation)

Z-Score Standardization

Centers the variable at 0 and scales to unit variance:

$$x'_i = \frac{x_i - \bar{x}}{s}$$

Properties: - Transformed variable has mean = 0, SD = 1 - Does not bound the range (values can exceed ± 3) - Robust to different scales; appropriate for most algorithms - Assumes approximately normal distribution for optimal behavior

Robust Scaling

Uses the **median** and **IQR** instead of mean and SD:

$$x'_i = \frac{x_i - \tilde{x}}{\text{IQR}}$$

Properties: - Robust to outliers (median and IQR are not affected by extremes) - Appropriate when data contains outliers that cannot be removed

MaxAbs Scaling

Divides by the maximum absolute value, scaling to $[-1, 1]$:

$$x'_i = \frac{x_i}{\max(|x|)}$$

Useful for sparse data (preserves zero entries).

When to Scale

Table 1: Feature scaling requirements by algorithm

Algorithm	Scaling Needed?	Recommended Method
Linear/logistic regression	Yes (for comparability)	Z-score
K-nearest neighbors	Yes (essential)	Z-score or min-max
PCA	Yes (essential)	Z-score
Decision trees / Random forests	No	None
Neural networks	Yes	Min-max or Z-score
SVM	Yes (essential)	Z-score or min-max
K-means clustering	Yes (essential)	Z-score

Example: Effect of Scaling on KNN

Example 6.6. A dataset contains age (range: 20–70 years) and income (range: 20,000–500,000 THB). Without scaling, the Euclidean distance between two observations is dominated entirely by income differences — a 1-year age difference ($\Delta = 1$) is dwarfed by a 1,000 THB income difference. After Z-score standardization, both variables contribute equally to the distance metric.

R Example: Feature Scaling

```
# --- Compare scaling methods ---
data(mtcars)
vars_to_scale <- mtcars[, c("mpg","hp","wt","disp")]

# Z-score standardization
z_scaled <- scale(vars_to_scale)

# Min-max normalization
minmax_scale <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}
minmax_scaled <- apply(vars_to_scale, 2, minmax_scale)

# Robust scaling
robust_scale <- function(x) {
  (x - median(x)) / IQR(x)
}
robust_scaled <- apply(vars_to_scale, 2, robust_scale)

# Summary comparison
scaling_summary <- function(mat, name) {
  data.frame(
    Method      = name,
    Variable    = colnames(mat),
    Mean        = round(colMeans(mat), 4),
    SD          = round(apply(mat, 2, sd), 4),
    Min         = round(apply(mat, 2, min), 4),
    Max         = round(apply(mat, 2, max), 4)
  )
}

bind_rows(
  scaling_summary(as.data.frame(vars_to_scale), "Original"),
  scaling_summary(as.data.frame(z_scaled),      "Z-Score"),
  scaling_summary(as.data.frame(minmax_scaled),  "Min-Max"),
  scaling_summary(as.data.frame(robust_scaled),  "Robust")
) |>
  filter(Variable == "hp") |>
  kable(caption    = "Scaling Methods Comparison: hp variable",
        col.names = c("Method","Variable","Mean",
                      "SD","Min","Max")) |>
```

```
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE)
```

Table 2: Scaling Methods Comparison: hp variable

	Method	Variable	Mean	SD	Min	Max
hp...1	Original	hp	146.6875	68.5629	52.0000	335.0000
hp...2	Z-Score	hp	0.0000	1.0000	-1.3810	2.7466
hp...3	Min-Max	hp	0.3346	0.2423	0.0000	1.0000
hp...4	Robust	hp	0.2837	0.8211	-0.8503	2.5389

```
# --- Visualize effect of scaling on all variables ---
orig_long <- as.data.frame(vars_to_scale) |>
  mutate(Method = "Original") |>
  pivot_longer(~Method, names_to = "Variable",
               values_to = "Value")

z_long <- as.data.frame(z_scaled) |>
  mutate(Method = "Z-Score") |>
  pivot_longer(~Method, names_to = "Variable",
               values_to = "Value")

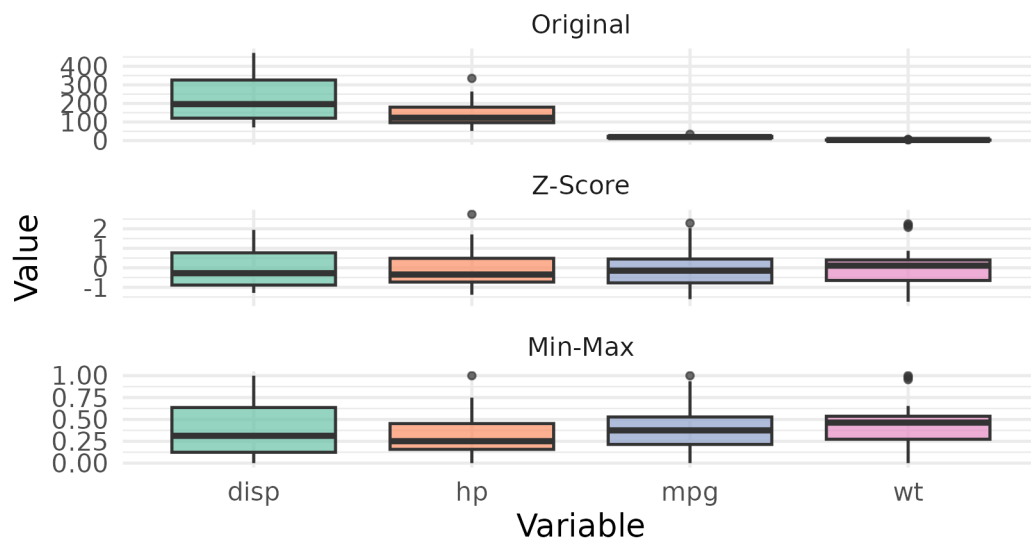
mm_long <- as.data.frame(minmax_scaled) |>
  mutate(Method = "Min-Max") |>
  pivot_longer(~Method, names_to = "Variable",
               values_to = "Value")

all_scaled <- bind_rows(orig_long, z_long, mm_long)
all_scaled$Method <- factor(all_scaled$Method,
                           levels = c("Original",
                                       "Z-Score", "Min-Max"))

ggplot(all_scaled, aes(x = Variable, y = Value,
                      fill = Variable)) +
  geom_boxplot(alpha = 0.7, outlier.size = 1) +
  scale_fill_brewer(palette = "Set2") +
  facet_wrap(~Method, scales = "free_y", ncol = 1) +
  labs(title = "Feature Scaling: Original vs. Z-Score vs. Min-Max",
       subtitle = "Z-score centers at 0; min-max bounds to [0,1]",
       x = "Variable", y = "Value") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")
```

Feature Scaling: Original vs. Z-Score vs. Min-Max

Z-score centers at 0; min-max bounds to [0,1]



Code explanation:

- `scale(data)` performs Z-score standardization for all columns simultaneously — the most concise method in R.
- `apply(mat, 2, fun)` applies a function column-wise (2 = columns, 1 = rows) — used to apply custom min-max and robust scaling functions.
- The three-panel boxplot immediately shows why scaling is necessary: in the “Original” panel, `disp` and `hp` dwarf `mpg` and `wt` — after scaling, all variables are comparable.

Exercises

💡 Exercise 6.8

Using the `iris` dataset (numeric variables only):

- Apply Z-score, min-max, and robust scaling to all four variables.
- For each scaling method, verify: (i) mean after Z-score = 0, (ii) range after min-max [0,1].
- Perform K-means clustering ($k = 3$) on unscaled vs. Z-scored data. Do the cluster assignments differ? Which matches the true species grouping better?

Data Integration and Reshaping

Introduction

Real-world data rarely arrives in a single, perfectly structured table. Data scientists routinely work with multiple source tables that must be merged, and with data that is organized in ways that do not match the format required for analysis. **Tidy data** principles define the target structure, and **reshaping** operations transform data between wide and long formats. Mastering these skills is essential for the practical data pipeline.

Theory

Tidy Data Principles

Tidy data (Wickham, 2014) follows three rules:

1. Each **variable** forms a column.
2. Each **observation** forms a row.
3. Each **observational unit** forms a table.

Most raw data violates at least one of these rules — column headers are values (not variable names), multiple variables are stored in one column, or variables are stored in both rows and columns.

Pivoting: Wide Long

Wide format: Each time point or category is a separate column. Common in spreadsheets and data entry.

Long format: Each observation has its own row, with a column for the category identifier. Required by most R visualization and modeling functions.

Wide:

id	week1	week2
1	80	85
2	72	78

Long:

id	week	score
1	1	80
1	2	85
2	1	72
2	2	78

Joining Tables

Table 1: dplyr join types

Join Type	Result
<code>inner_join</code>	Only rows with matches in both tables
<code>left_join</code>	All rows from left table; NAs for non-matches from right
<code>right_join</code>	All rows from right table; NAs for non-matches from left
<code>full_join</code>	All rows from both; NAs where no match
<code>anti_join</code>	Rows in left with NO match in right

String and Date Handling

Common integration tasks include:

- Parsing dates with `lubridate::ymd()`, `dmy()`, `mdy()`
- Cleaning strings with `stringr::str_trim()`, `str_to_lower()`, `str_replace()`
- Splitting compound columns with `tidyr::separate()`
- Uniting columns with `tidyr::unite()`

Example: Reshaping for Analysis

Example 6.7. A student performance dataset is stored in wide format with columns `score_Q1`, `score_Q2`, `score_Q3`, `score_Q4`. For a repeated-measures analysis (Chapter 3) or a time series plot, we need long format with columns `quarter` and `score`. `pivot_longer()` achieves this in one step.

R Example: Data Integration and Reshaping

```
# --- Create a wide-format dataset ---
set.seed(42)
student_wide <- data.frame(
  student_id = 1:20,
  faculty    = sample(c("Science", "Arts", "Business"),
                      20, replace = TRUE),
  score_Q1   = round(rnorm(20, 70, 10)),
  score_Q2   = round(rnorm(20, 72, 9)),
  score_Q3   = round(rnorm(20, 75, 8)),
  score_Q4   = round(rnorm(20, 78, 7))
)

cat("Wide format (first 4 rows):\n")
```

Wide format (first 4 rows):

```
head(student_wide, 4)
```

	student_id	faculty	score_Q1	score_Q2	score_Q3	score_Q4
1	1	Science	70	64	89	84
2	2	Science	83	81	80	89
3	3	Science	80	66	52	71
4	4	Science	79	62	83	75

```
# --- Pivot to long format ---
student_long <- student_wide |>
  pivot_longer(
    cols      = starts_with("score_"),
    names_to  = "quarter",
    values_to = "score"
  ) |>
  mutate(quarter = str_remove(quarter, "score_"))

cat("\nLong format (first 8 rows):\n")
```

Long format (first 8 rows):

```
head(student_long, 8)
```

```
# A tibble: 8 x 4
  student_id faculty quarter score
    <int>   <chr>   <chr>   <dbl>
1         1 Science Q1      70
2         1 Science Q2      64
3         1 Science Q3      89
4         1 Science Q4      84
5         2 Science Q1      83
6         2 Science Q2      81
7         2 Science Q3      80
8         2 Science Q4      89
```

```
# --- Pivot back to wide ---
student_back_wide <- student_long |>
  pivot_wider(
    names_from = quarter,
    values_from = score,
    names_prefix = "score_"
  )
cat("\nReturned to wide (first 4 rows):\n")
```

Returned to wide (first 4 rows):

```
head(student_back_wide, 4)
```

```
# A tibble: 4 x 6
  student_id faculty score_Q1 score_Q2 score_Q3 score_Q4
    <int>   <chr>   <dbl>   <dbl>   <dbl>   <dbl>
1         1 Science      70      64      89      84
2         2 Science      83      81      80      89
3         3 Science      80      66      52      71
4         4 Science      79      62      83      75
```

```
# --- Joining tables ---
# Faculty information table
faculty_info <- data.frame(
  faculty    = c("Science", "Arts", "Business"),
  dean       = c("Dr. Smith", "Dr. Lee", "Dr. Park"),
  building    = c("Building A", "Building B", "Building C")
)
```

```
# Left join: keep all students, add faculty info
student_full <- student_wide |>
  left_join(faculty_info, by = "faculty")

cat("\nAfter left join (first 4 rows):\n")
```

After left join (first 4 rows):

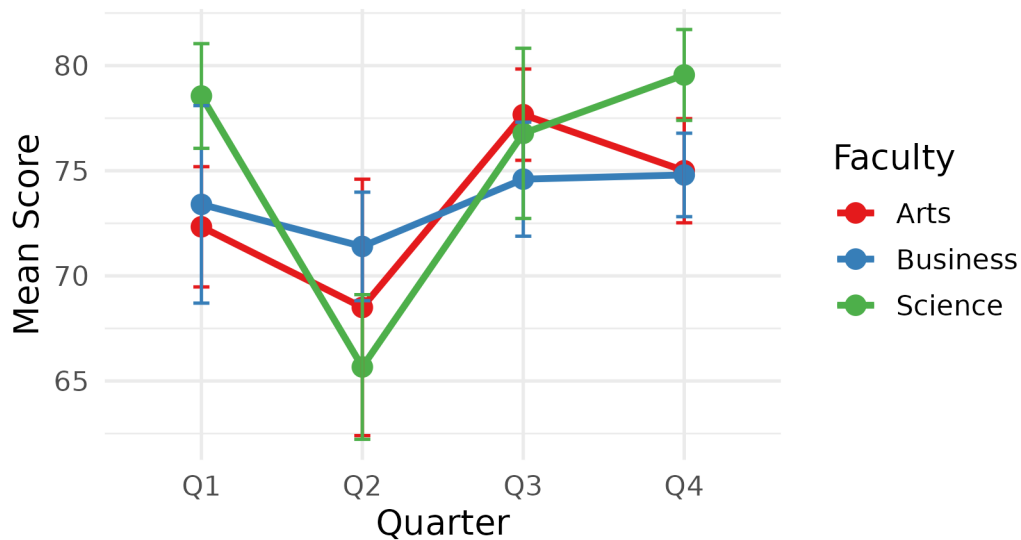
```
head(student_full[, c("student_id", "faculty",
                     "dean", "score_Q1")], 4)
```

	student_id	faculty	dean	score_Q1
1	1	Science	Dr. Smith	70
2	2	Science	Dr. Smith	83
3	3	Science	Dr. Smith	80
4	4	Science	Dr. Smith	79

```
# --- Visualize trends in long format ---
student_long |>
  group_by(faculty, quarter) |>
  summarise(mean_score = mean(score),
            se          = sd(score)/sqrt(n()),
            .groups     = "drop") |>
  ggplot(aes(x = quarter, y = mean_score,
            group = faculty, color = faculty)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_score - se,
                  ymax = mean_score + se),
              width = 0.1) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "Mean Score by Quarter and Faculty",
       subtitle = "Long format enables grouped time-series visualization",
       x = "Quarter", y = "Mean Score",
       color = "Faculty") +
  theme_minimal(base_size = 13)
```

Mean Score by Quarter and Faculty

Long format enables grouped time-series visualization



Code explanation:

- `pivot_longer(cols, names_to, values_to)` converts wide to long format. `starts_with("score_")` selects all score columns by prefix.
- `str_remove(quarter, "score_")` cleans the quarter labels from “score_Q1” to “Q1” — a common post-pivot cleanup step.
- `pivot_wider(names_from, values_from)` reverses the operation — useful for creating summary tables after analysis.
- `left_join(y, by)` merges two tables on a key variable, preserving all rows from the left table.

Exercises

💡 Exercise 6.9

The built-in `tidyr::who` dataset contains tuberculosis (TB) counts in wide format with many messy column names encoding country, year, age group, sex, and diagnosis method.

- Use `pivot_longer()` to convert to long format.
- Use `separate()` to split the messy column names into meaningful variables.
- Filter to Thailand (`country == "Thailand"`) and plot TB counts over time by sex and age group.

Exercise 6.10

Create two data frames: one with student demographics (id, name, program) and one with exam scores (id, exam, score). Simulate 5 students and 3 exams each.

- (a) Inner join the two tables and explain what happens if a student has no exam record.
 - (b) Left join and full join — how do the results differ?
 - (c) Pivot the joined data so each exam is a column. Compute the average score per student.
-

Chapter Lab Activity: Full Preprocessing Pipeline with msleep

Objectives

In this lab you will apply the complete preprocessing pipeline — from initial inspection through missing data handling, outlier treatment, transformation, encoding, scaling, and reshaping — to the `msleep` dataset (mammal sleep data from `ggplot2`). This mirrors a real data science workflow where raw data must be made analysis-ready before modeling.

The Dataset

```
# msleep: sleep patterns of 83 mammals
data(msleep, package = "ggplot2")

cat("Dimensions:", nrow(msleep), "rows x",
    ncol(msleep), "columns\n\n")
```

Dimensions: 83 rows × 11 columns

```
# Overview
glimpse(msleep)
```

```
Rows: 83
Columns: 11
$ name      <chr> "Cheetah", "Owl monkey", "Mountain beaver", "Greater shor~
$ genus     <chr> "Acinonyx", "Aotus", "Aplodontia", "Blarina", "Bos", "Bra~
$ vore      <chr> "carni", "omni", "herbi", "omni", "herbi", "herbi", "carn~
$ order     <chr> "Carnivora", "Primates", "Rodentia", "Soricomorpha", "Art~
$ conservation <chr> "lc", NA, "nt", "lc", "domesticated", NA, "vu", NA, "dome~
$ sleep_total <dbl> 12.1, 17.0, 14.4, 14.9, 4.0, 14.4, 8.7, 7.0, 10.1, 3.0, 5~
$ sleep_rem  <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2, 1.4, NA, 2.9, NA, 0.6, 0.8, ~
$ sleep_cycle <dbl> NA, NA, NA, 0.1333333, 0.6666667, 0.7666667, 0.3833333, N~
```

```
$ awake      <dbl> 11.9, 7.0, 9.6, 9.1, 20.0, 9.6, 15.3, 17.0, 13.9, 21.0, 1~
$ brainwt    <dbl> NA, 0.01550, NA, 0.00029, 0.42300, NA, NA, NA, 0.07000, 0~
$ bodywt     <dbl> 50.000, 0.480, 1.350, 0.019, 600.000, 3.850, 20.490, 0.04~
```

```
# Missing value map
miss_summary_lab <- data.frame(
  Variable      = names(msleep),
  Type          = sapply(msleep, class),
  N_Missing      = colSums(is.na(msleep)),
  Pct_Missing    = round(colMeans(is.na(msleep)) * 100, 1)
) |> filter(N_Missing > 0)
rownames(miss_summary_lab) <- NULL

kable(miss_summary_lab,
      caption = "Variables with Missing Data - msleep",
      col.names = c("Variable", "Type",
                    "N Missing", "% Missing")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(4, bold = TRUE, color = "tomato")
```

Table 1: Variables with Missing Data — msleep

Variable	Type	N Missing	% Missing
vore	character	7	8.4
conservation	character	29	34.9
sleep_rem	numeric	22	26.5
sleep_cycle	numeric	51	61.4
brainwt	numeric	27	32.5

Lab Task 1: Missing Data Handling

```
# Impute numeric missing values with median (simple approach)
# Impute sleep_rem and brainwt using mice
msleep_num <- msleep |>
  dplyr::select(sleep_total, sleep_rem, sleep_cycle, awake, brainwt, bodywt)

set.seed(42)
mice_lab <- mice(msleep_num, m = 3,
                 method = "pmm", printFlag = FALSE)
```

```
msleep_imputed_num <- complete(mice_lab, 1)

# Attach back with non-numeric columns
msleep_proc <- msleep |>
  dplyr::select(name, genus, vore, order, conservation) |>
  bind_cols(msleep_imputed_num)

cat("Missing values after imputation:\n")
```

Missing values after imputation:

```
print(colSums(is.na(msleep_proc)))
```

name	genus	vore	order	conservation	sleep_total
0	0	7	0	29	0
sleep_rem	sleep_cycle	awake	brainwt	bodywt	
0	0	0	0	0	

Lab Task 2: Outlier Detection

```
# IQR outlier detection for bodywt and brainwt
detect_outliers_iqr <- function(x, var_name) {
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_v <- Q3 - Q1
  flags <- x < Q1 - 1.5*IQR_v | x > Q3 + 1.5*IQR_v
  data.frame(
    Variable = var_name,
    N_Outliers = sum(flags, na.rm = TRUE),
    Outlier_Values = paste(
      round(sort(x[flags]), 1), collapse = ", "
    )
  )
}

bind_rows(
  detect_outliers_iqr(msleep_proc$bodywt, "bodywt"),
  detect_outliers_iqr(msleep_proc$brainwt, "brainwt"),
  detect_outliers_iqr(msleep_proc$sleep_total, "sleep_total")
) |>
```



```
kable(caption = "IQR Outlier Detection - msleep") |>
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE)
```

Table 2: IQR Outlier Detection — msleep

Variable	N_Outliers	Outlier_Values
bodywt	11	161.5, 162.6, 173.3, 187, 207.5, 521, 600, 800, 900, 2547, 6654
brainwt	11	0.3, 0.3, 0.4, 0.4, 0.4, 0.4, 0.4, 0.7, 1.3, 4.6, 5.7
sleep_total	0	

Lab Task 3: Transformation

```
# bodywt and brainwt are extremely right-skewed - apply log
msleep_proc <- msleep_proc |>
  mutate(
    log_bodywt = log(bodywt + 1),
    log_brainwt = log(brainwt + 1)
  )

transform_check <- data.frame(
  Variable      = c("bodywt", "log(bodywt+1)",
                    "brainwt", "log(brainwt+1)"),
  Skewness      = round(c(
    moments::skewness(msleep_proc$bodywt),
    moments::skewness(msleep_proc$log_bodywt),
    moments::skewness(msleep_proc$brainwt),
    moments::skewness(msleep_proc$log_brainwt)
  ), 3)
)

kable(transform_check,
      caption = "Skewness Before and After Log Transformation") |>
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE) |>
column_spec(2, bold = TRUE,
            color = ifelse(
              abs(transform_check$Skewness) < 1,
              "darkgreen", "tomato"))
```

Table 3: Skewness Before and After Log Transformation

Variable	Skewness
bodywt	7.230
log(bodywt+1)	1.144
brainwt	5.836
log(brainwt+1)	4.575

Lab Task 4: Encoding and Scaling

```
# Encode vore (dietary classification) as dummy variables
msleep_proc$vore <- factor(
  msleep_proc$vore,
  levels = c("herbi", "carni", "omni", "insecti")
)
msleep_proc$vore[is.na(msleep_proc$vore)] <- "herbi"

# Dummy encode (reference = herbi)
vore_dummies <- model.matrix(~ vore, data = msleep_proc)

# Z-score scale numeric features
features_to_scale <- msleep_proc |>
  dplyr::select(sleep_total, sleep_rem, awake,
    log_bodywt, log_brainwt)

scaled_features <- as.data.frame(scale(features_to_scale))
names(scaled_features) <- paste0(names(scaled_features),
  "_scaled")

# Verify scaling
kable(data.frame(
  Variable = names(features_to_scale),
  Mean_Before = round(colMeans(features_to_scale), 3),
  SD_Before   = round(apply(features_to_scale, 2, sd), 3),
  Mean_After  = round(colMeans(scaled_features), 3),
  SD_After    = round(apply(scaled_features, 2, sd), 3)
),
caption = "Z-Score Scaling Verification",
col.names = c("Variable", "Mean Before", "SD Before",
  "Mean After", "SD After")) |>
```

```
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE)
```

Table 4: Z-Score Scaling Verification

	Variable	Mean Before	SD Before	Mean After	SD After
sleep_total	sleep_total	10.434	4.450	0	1
sleep_rem	sleep_rem	1.913	1.184	0	1
awake	awake	13.567	4.452	0	1
log_bodywt	log_bodywt	1.951	2.208	0	1
log_brainwt	log_brainwt	0.119	0.302	0	1

Lab Task 5: Final Clean Dataset and Visualization

```
# Assemble final preprocessed dataset
msleep_final <- bind_cols(
  msleep_proc |> dplyr::select(name, vore, order),
  scaled_features
)

cat("Final preprocessed dataset dimensions:",
    nrow(msleep_final), "x", ncol(msleep_final), "\n\n")
```

Final preprocessed dataset dimensions: 83 × 8

```
cat("No missing values:", sum(is.na(msleep_final)) == 0, "\n")
```

No missing values: TRUE

```
# Visualize the full pipeline outcome
p1 <- ggplot(msleep_proc, aes(x = bodywt)) +
  geom_histogram(bins = 20, fill = "tomato",
                 color = "white", alpha = 0.8) +
  labs(title = "A. bodywt: Before (raw)", x = "bodywt") +
  theme_minimal(base_size = 11)

p2 <- ggplot(msleep_proc, aes(x = log_bodywt)) +
  geom_histogram(bins = 20, fill = "seagreen",
```

```

        color = "white", alpha = 0.8) +
labs(title = "B. bodywt: After log transform",
      x = "log(bodywt+1)") +
theme_minimal(base_size = 11)

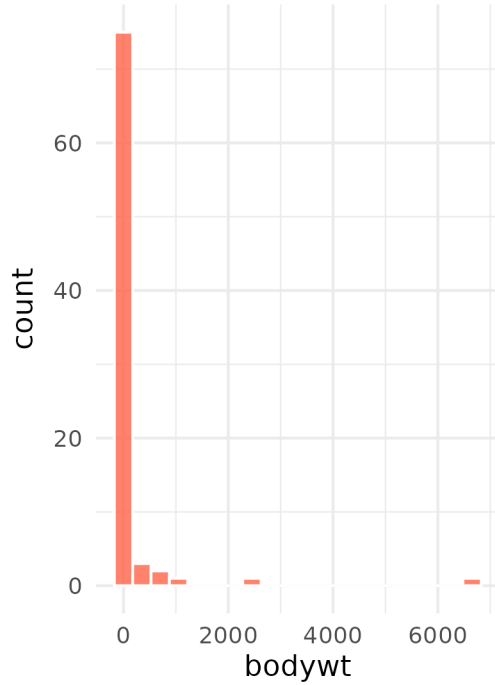
p3 <- ggplot(scaled_features,
            aes(x = sleep_total_scaled)) +
  geom_histogram(bins = 20, fill = "steelblue",
                color = "white", alpha = 0.8) +
labs(title = "C. sleep_total: After Z-scaling",
      x = "sleep_total (scaled)") +
theme_minimal(base_size = 11)

p4 <- msleep_proc |>
  filter(!is.na(vore)) |>
  ggplot(aes(x = vore, y = log_bodywt, fill = vore)) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_brewer(palette = "Set2") +
labs(title = "D. log(bodywt) by Diet Type",
      x = "Diet (vore)", y = "log(bodywt+1)") +
theme_minimal(base_size = 11) +
theme(legend.position = "none")

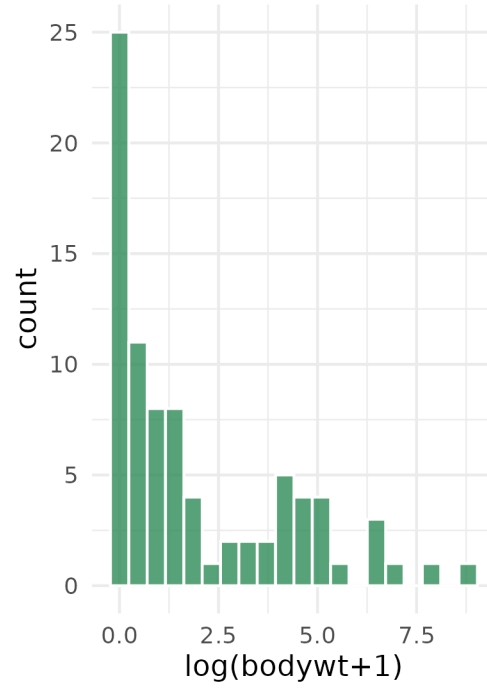
(p1 + p2) / (p3 + p4)

```

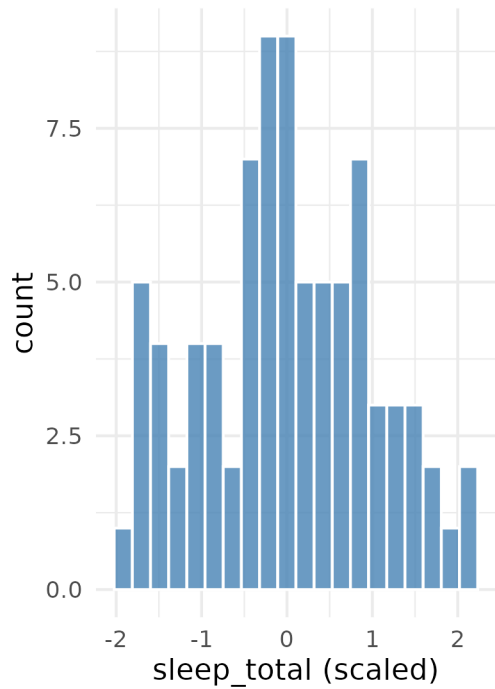
A. bodywt: Before (raw)



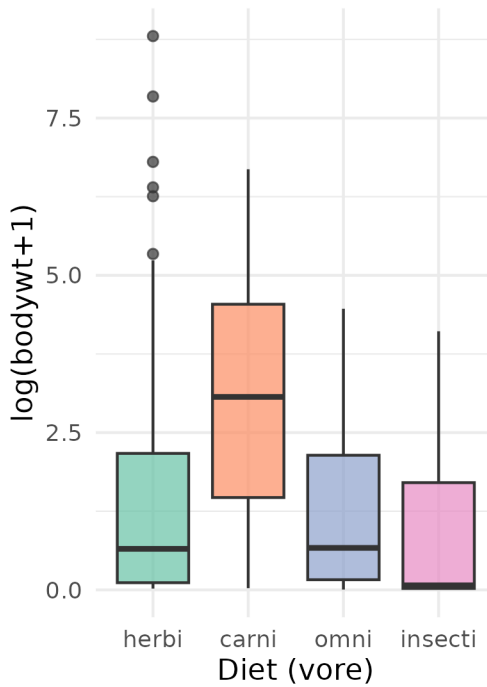
B. bodywt: After log tran



C. sleep_total: After Z-scaling



D. log(bodywt) by Diet T



Lab Discussion Questions

Answer the following in writing:

1. **Missing Data Strategy:** `brainwt` has 27% missing values. Is multiple imputation appropriate here, or would you recommend a different strategy? What type of missingness (MCAR/MAR/MNAR) do you suspect for brain weight in mammals?
 2. **Outlier Decision:** The elephant has extreme `bodywt` and `brainwt` values flagged as outliers. Should it be removed? What are the consequences of removal for a study of mammal sleep patterns?
 3. **Transformation Justification:** Why is log transformation more appropriate than square root for `bodywt`? Use the skewness values from Lab Task 3 to support your answer.
 4. **Encoding Choice:** `vore` has 4 levels including some NAs. Justify your reference category choice for dummy encoding. How would target encoding differ, and what leakage risk would it introduce?
 5. **Scaling and Algorithms:** If you were to cluster mammals by sleep patterns using K-means (Chapter 8), which preprocessing steps from this lab are essential, and which are optional? Justify each decision.
-

Chapter Summary

Summary

This chapter built a complete data preprocessing toolkit essential for any data science project:

- **Why preprocessing matters** — the GIGO principle; each step in the pipeline has a distinct purpose and must be applied in the correct order; the leakage problem requires all fitting to occur on training data only.
- **Missing data** — MCAR, MAR, and MNAR require different strategies; multiple imputation is the gold standard under MAR; no method fully corrects MNAR.
- **Outlier detection** — IQR and modified z-score for univariate detection; Mahalanobis distance for multivariate; treatment depends on whether outliers are errors or genuine extreme values.
- **Transformation** — log and Box-Cox correct right skew and stabilize variance; Yeo-Johnson handles negative values; transformation changes shape, not location or spread.
- **Encoding** — dummy coding for regression (drop reference); one-hot for ML (no reference dropped); ordinal for ordered categories; target encoding for high cardinality with cross-validation.
- **Feature scaling** — Z-score for most algorithms; min-max for bounded ranges; robust scaling when outliers are present; tree-based models do not require scaling.
- **Data integration and reshaping** — tidy data principles; `pivot_longer/wider` for format conversion; `left/inner/full_join` for table merging.

! Key Formulas to Know

IQR Outlier Fences:

$$\text{Lower} = Q_1 - 1.5 \times \text{IQR}, \quad \text{Upper} = Q_3 + 1.5 \times \text{IQR}$$

Mahalanobis Distance:

$$D_i^2 = (\mathbf{x}_i - \bar{\mathbf{x}})^T \mathbf{S}^{-1} (\mathbf{x}_i - \bar{\mathbf{x}}) \sim \chi^2(p)$$

Box-Cox Transformation:

$$x'(\lambda) = \begin{cases} (x^\lambda - 1)/\lambda & \lambda \neq 0 \\ \log(x) & \lambda = 0 \end{cases}$$

Z-Score Standardization:

$$x'_i = \frac{x_i - \bar{x}}{s}$$

Min-Max Normalization:

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Post-Stratification Weight:

$$w_i = \frac{p_i^{\text{pop}}}{p_i^{\text{sample}}}$$

References

- Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2), 211–243.
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Little, R. J. A., & Rubin, D. B. (2002). *Statistical analysis with missing data* (2nd ed.). Wiley.
- New York State Department of Conservation. (1973). *Daily air quality measurements in New York, May–September 1973*.
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality. *Biometrika*, 52(3/4), 591–611.
- Templ, M., Alfons, A., Kowarik, A., & Prantner, B. (2023). *VIM: Visualization and imputation of missing values* (R package version 6.2.2). <https://CRAN.R-project.org/package=VIM>
- Tierney, N., & Cook, D. (2023). *nanian: Data structures, summaries, and visualisations for missing data* (R package version 1.0.0). <https://CRAN.R-project.org/package=nanian>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, 45(3), 1–67.
- Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23.
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.

Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

Yeo, I. K., & Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), 954–959.

End of Chapter 6. Proceed to Chapter 7: Data Dimension Reduction.

Chapter 7: Data Dimension Reduction

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(factoextra)      # PCA visualization
library(FactoMineR)      # PCA and FA
library(psych)            # factor analysis
library(MASS)             # LDA
library(Rtsne)            # t-SNE
library(caret)            # feature selection wrappers
library(corrplot)         # correlation visualization
library(moments)          # skewness
```

Chapter Overview

Introduction

Modern datasets routinely contain hundreds or thousands of variables. Working directly in such high-dimensional spaces presents serious statistical, computational, and interpretive challenges. **Dimension reduction** addresses these challenges by finding a lower-dimensional representation of the data that retains as much useful information as possible — either by constructing new composite variables (feature extraction) or by selecting a subset of the original variables (feature selection).

This chapter covers:

- **The Curse of Dimensionality** — why high dimensions are problematic
- **Principal Component Analysis (PCA)** — the foundational linear extraction method
- **Factor Analysis (FA)** — discovering latent constructs underlying observed variables
- **Linear Discriminant Analysis (LDA)** — supervised reduction maximizing class separation
- **t-SNE** — non-linear reduction for high-dimensional visualization
- **Feature Selection Methods** — filter, wrapper, and embedded approaches
- **Evaluating Dimension Reduction** — criteria for choosing the right number of dimensions

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain the curse of dimensionality and its practical consequences.
2. Perform and interpret PCA including scree plots, biplots, and loadings.
3. Distinguish PCA from factor analysis and apply FA with rotation.
4. Apply LDA for supervised dimension reduction and visualize class separation.
5. Use t-SNE for non-linear visualization of high-dimensional data.
6. Apply filter, wrapper, and embedded feature selection methods.
7. Evaluate and compare dimension reduction solutions using principled criteria.

The Curse of Dimensionality

Introduction

Adding more variables to a dataset seems intuitively beneficial — more information should lead to better analysis. In reality, as the number of dimensions grows, the data becomes increasingly sparse, distances lose their meaning, and models require exponentially more observations to maintain statistical reliability. This phenomenon, coined by Richard Bellman in 1957, is the **curse of dimensionality** — and understanding it is the essential motivation for all dimension reduction methods.

Theory

Sparsity in High Dimensions

Consider a unit hypercube in p dimensions. To capture 10% of the data volume, the required edge length of a sub-hypercube is:

$$\ell = (0.10)^{1/p}$$

In 2 dimensions: $\ell = 0.316$ (32% of the range). In 10 dimensions: $\ell = 0.794$ (79% of the range!). In 100 dimensions: $\ell = 0.977$ (98% of the range!).

As p grows, any local neighborhood must span almost the entire range of each variable to contain even a small fraction of the data. **Local methods** (KNN, kernel density estimation, local regression) break down entirely in high dimensions.

Distance Concentration

In high dimensions, all pairwise distances between random points converge to the same value:

$$\frac{\max_{\mathbf{x}, \mathbf{y}} d(\mathbf{x}, \mathbf{y}) - \min_{\mathbf{x}, \mathbf{y}} d(\mathbf{x}, \mathbf{y})}{\min_{\mathbf{x}, \mathbf{y}} d(\mathbf{x}, \mathbf{y})} \rightarrow 0 \quad \text{as } p \rightarrow \infty$$

When all distances are nearly equal, the concept of “nearest neighbor” becomes meaningless. KNN classification, K-means clustering, and any distance-based method degrades catastrophically.

! Caveat: this is not a universal law

Distance concentration is sharpest when the p features are close to independent and identically distributed (the “random noise” setting simulated below). Real datasets are rarely like this — features are usually correlated, and the data often lie near a much lower-dimensional manifold embedded in the p -dimensional space (its **intrinsic dimensionality** is lower than p). In that case distances stay far more informative than this formula suggests. This is precisely *why* dimension reduction (PCA, t-SNE, etc.) works in practice: it recovers the low intrinsic dimensionality hiding inside a high nominal dimensionality.

The Sample Size Requirement

To maintain a fixed density of observations in p -dimensional space, the required sample size grows **exponentially** with p :

$$n \propto c^p$$

To have 10 observations per unit cell in 1D requires $n = 10$; in 10D requires $n = 10^{10}$; in 20D requires $n = 10^{20}$. Most real datasets are **severely undersampled** relative to their dimensionality.

! Caveat: this applies to *local, non-parametric* methods

The exponential-sample-size argument above applies to methods that estimate quantities locally (KNN, kernel density/regression, histogram-based estimators), where every “cell” of the space needs its own observations. It is **not** a universal law for every statistical method. Regularized/structured models — Ridge and, especially, **Lasso regression** (Chapter 10) — are explicitly designed to work well even when $p > n$, by exploiting an assumption of *sparsity* (only a few predictors truly matter) rather than trying to fill every cell of the feature space with data. Keep this in mind when we reach Chapter 10: seeing Lasso perform well with many predictors and few observations is not a contradiction of the curse of dimensionality — it is a demonstration of how structural assumptions can partially escape it.

Practical Consequences

Table 1: Practical consequences of high dimensionality

Problem	Consequence in High Dimensions
Sparsity	Local methods unreliable
Distance concentration	Clustering and KNN degrade
Sample scarcity	Overfitting; model instability
Multicollinearity	Matrix inversion fails; regression unstable
Visualization	Impossible beyond 3 dimensions
Computation	Memory and time scale poorly

Example: Distance Concentration

Example 7.1. We simulate 100 random points uniformly distributed in a p -dimensional unit hypercube, and compute the ratio $(\max - \min) / \min$ distance as p increases from 1 to 100. This ratio should approach 0 as distances concentrate — demonstrating the breakdown of distance-based reasoning.

R Example: Visualizing the Curse of Dimensionality

```
# --- Distance concentration demonstration ---
set.seed(42)
n_pts <- 100
dims <- c(1, 2, 5, 10, 20, 50, 100)

dist_ratio <- sapply(dims, function(p) {
  X <- matrix(runif(n_pts * p), nrow = n_pts)
  D <- as.matrix(dist(X))
  diag(D) <- NA
  d_max <- max(D, na.rm = TRUE)
  d_min <- min(D, na.rm = TRUE)
  (d_max - d_min) / d_min
})

dist_df <- data.frame(
  Dimensions = dims,
  Distance_Ratio = round(dist_ratio, 4)
)

kable(dist_df,
  caption = "Distance Concentration as Dimensions Increase",
```

```
col.names = c("Dimensions (p)",
              "(max - min) / min Distance")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(2, bold = TRUE,
            color = ifelse(dist_df$Distance_Ratio < 0.5,
                          "tomato", "steelblue"))
```

Table 2: Distance Concentration as Dimensions Increase

Dimensions (p)	(max - min) / min Distance
1	189165.9970
2	290.1437
5	9.2778
10	4.4779
20	1.6065
50	0.9678
100	0.5288

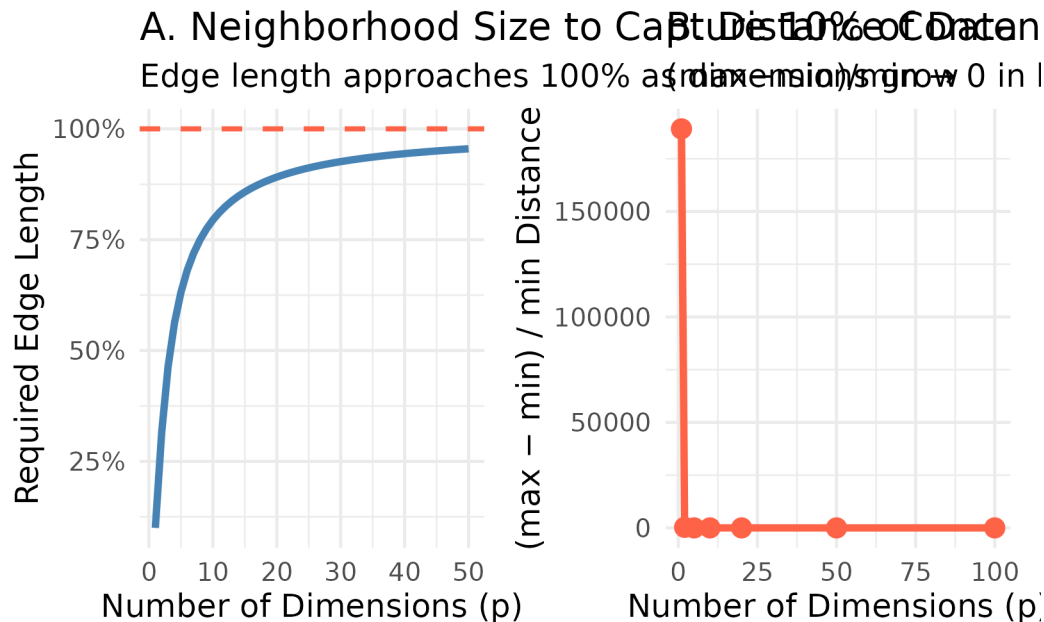
```
# --- Sparsity: neighborhood size needed to capture 10% of data ---
p_seq    <- 1:50
edge_len <- (0.10)^(1 / p_seq)

p1 <- ggplot(data.frame(p = p_seq, edge = edge_len),
             aes(x = p, y = edge)) +
  geom_line(color = "steelblue", linewidth = 1.3) +
  geom_hline(yintercept = 1, linetype = "dashed",
             color = "tomato", linewidth = 0.9) +
  scale_y_continuous(labels = scales::percent) +
  labs(title    = "A. Neighborhood Size to Capture 10% of Data",
       subtitle = "Edge length approaches 100% as dimensions grow",
       x        = "Number of Dimensions (p)",
       y        = "Required Edge Length") +
  theme_minimal(base_size = 12)

# Distance ratio plot
p2 <- ggplot(dist_df, aes(x = Dimensions,
                         y = Distance_Ratio)) +
  geom_line(color = "tomato", linewidth = 1.3) +
  geom_point(size = 3, color = "tomato") +
  labs(title    = "B. Distance Concentration",
       subtitle = "(max-min)/min → 0 in high dimensions",
```

```
x      = "Number of Dimensions (p)",
y      = "(max - min) / min Distance" +
theme_minimal(base_size = 12)
```

p1 + p2



Code explanation:

- `dist(X)` computes the Euclidean distance matrix for all pairs of rows. `diag(D) <- NA` removes self-distances (which are 0).
- `(0.10)^(1/p_seq)` directly implements the hypercube edge-length formula — a vectorized calculation across all dimensions simultaneously.
- The two plots together show both faces of the curse: Panel A shows that local neighborhoods must become global, Panel B shows that all pairwise distances converge.

Exercises

💡 Exercise 7.1

- Extend the distance concentration simulation to $p = 200$ and $p = 500$. Does the ratio continue to decrease?
- Repeat the simulation with $n = 1000$ points. Does a larger sample size solve the concentration problem?
- In your own words, explain why K-nearest neighbors classification is expected to

perform poorly on a dataset with 500 features and 200 observations.

Principal Component Analysis (PCA)

Introduction

Principal Component Analysis is the most widely used method for linear dimension reduction. It finds a new set of orthogonal axes — the **principal components** — that are linear combinations of the original variables, ordered so that the first component captures the maximum variance, the second captures the maximum remaining variance orthogonal to the first, and so on. PCA is unsupervised: it uses only the structure of X , not any outcome variable Y .

Theory

Mathematical Foundation

Given n observations of p variables, arranged in a centered data matrix $\mathbf{X}$ (each column has mean zero), PCA finds the directions $\mathbf{w}_k$ that maximize variance:

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|=1} \text{Var}(\mathbf{X}\mathbf{w})$$

The solution is the **eigendecomposition** of the sample covariance matrix $\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$:

$$\mathbf{S}\mathbf{w}_k = \lambda_k \mathbf{w}_k$$

where λ_k are eigenvalues (variance explained by each component) and $\mathbf{w}_k$ are eigenvectors (**loadings** — the weights of each original variable in the component).

The **principal component scores** (projections of observations onto new axes) are:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}$$

where $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_k]$.

Proportion of Variance Explained

The proportion of total variance explained by component k is:

$$\text{PVE}_k = \frac{\lambda_k}{\sum_{j=1}^p \lambda_j}$$

The **cumulative PVE** is used to decide how many components to retain. Common thresholds: retain enough components to explain 70–90% of total variance.

Key Assumptions and Properties

- Components are **orthogonal** (uncorrelated with each other).
- PCA finds **linear** combinations only — it cannot capture non-linear structure.
- PCA is **scale-sensitive**: always standardize variables (Z-score) before PCA unless all variables are measured in the same units.
- The components are ordered by variance explained, not by interpretability.

i Common point of confusion: sign ambiguity

The **sign** of a loading vector $\mathbf{w}_k$ (and hence of its scores) is not identified: if $\mathbf{w}_k$ maximizes variance, so does $-\mathbf{w}_k$, and both are equally valid solutions. Different software, different R package versions, or even the same code re-run on slightly perturbed data can flip the sign of a component. This does **not** change the interpretation (a PC that separates “high scorers” from “low scorers” is the same finding whether high scorers get positive or negative scores) — but it does mean you should never be alarmed if your PC1 looks like the mirror image of a classmate’s or a textbook’s PC1.

Loadings vs. Scores

Table 1: PCA terminology

Term	Definition	Interpretation
Loadings $\mathbf{w}_k$	Coefficients of original variables in the k -th component	How much each variable contributes
Scores z_{ik}	Projection of observation i onto component k	Coordinates of observations in new space
Biplot	Overlays scores and loadings	Shows observations and variables together

Example: PCA on Student Exam Data

Example 7.2. A dataset records scores on 5 subjects (Math, Physics, Chemistry, History, Literature) for 100 students. PCA reveals:

- PC1 (explains 45% variance): high positive loadings on all subjects — a **general academic ability** factor.
- PC2 (explains 22% variance): high positive loadings on Math/Physics/Chemistry, negative on History/Literature — a **STEM vs. Humanities** contrast.

Two components explain 67% of the total variance, reducing 5 dimensions to 2 while retaining most information.

R Example: PCA

```
# --- PCA on iris dataset (4 numeric variables) ---
data(iris)
iris_scaled <- scale(iris[, 1:4])

# Perform PCA
pca_result <- prcomp(iris_scaled, center = FALSE,
                     scale. = FALSE) # already scaled

# Summary
summary(pca_result)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	1.7084	0.9560	0.38309	0.14393
Proportion of Variance	0.7296	0.2285	0.03669	0.00518
Cumulative Proportion	0.7296	0.9581	0.99482	1.00000

```
# --- Loadings (rotation matrix) ---
loadings_df <- as.data.frame(pca_result$rotation)
loadings_df$Variable <- rownames(loadings_df)
loadings_df <- loadings_df |>
  dplyr::select(Variable, PC1, PC2, PC3, PC4) |>
  mutate(across(PC1:PC4, ~round(.x, 4)))

kable(loadings_df,
      caption = "PCA Loadings: iris Dataset",
```

```

    row.names = FALSE) |>
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE) |>
column_spec(2:3, bold = TRUE)

```

Table 2: PCA Loadings: iris Dataset

Variable	PC1	PC2	PC3	PC4
Sepal.Length	0.5211	-0.3774	0.7196	0.2613
Sepal.Width	-0.2693	-0.9233	-0.2444	-0.1235
Petal.Length	0.5804	-0.0245	-0.1421	-0.8014
Petal.Width	0.5649	-0.0669	-0.6343	0.5236

```

# --- Variance explained ---
pve <- pca_result$sdev^2 / sum(pca_result$sdev^2)
cum_pve <- cumsum(pve)

var_df <- data.frame(
  Component      = paste0("PC", 1:4),
  Eigenvalue     = round(pca_result$sdev^2, 4),
  PVE            = round(pve * 100, 2),
  Cumulative_PVE = round(cum_pve * 100, 2)
)

kable(var_df,
      caption = "Variance Explained by Each Principal Component",
      col.names = c("Component", "Eigenvalue",
                    "% Variance", "Cumulative %")) |>
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE) |>
column_spec(4, bold = TRUE,
           color = ifelse(var_df$Cumulative_PVE >= 95,
                         "darkgreen","steelblue"))

```

Table 3: Variance Explained by Each Principal Component

Component	Eigenvalue	% Variance	Cumulative %
PC1	2.9185	72.96	72.96
PC2	0.9140	22.85	95.81
PC3	0.1468	3.67	99.48
PC4	0.0207	0.52	100.00

```

# --- Scree plot ---
p1 <- fviz_eig(pca_result,
              addlabels = TRUE,
              barfill   = "steelblue",
              barcolor   = "white",
              linecolor  = "tomato") +
  labs(title = "A. Scree Plot: Variance Explained") +
  theme_minimal(base_size = 12)

# --- Biplot: scores colored by species ---
scores_df <- as.data.frame(pca_result$x)
scores_df$Species <- iris$Species

p2 <- ggplot(scores_df, aes(x = PC1, y = PC2,
                          color = Species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_color_brewer(palette = "Set1") +
  labs(title   = "B. PCA Score Plot (PC1 vs PC2)",
       subtitle = paste0("PC1: ", round(pve[1]*100, 1),
                          "% | PC2: ",
                          round(pve[2]*100, 1), "% variance"),
       x = paste0("PC1 (", round(pve[1]*100,1), "%)"),
       y = paste0("PC2 (", round(pve[2]*100,1), "%)")) +
  theme_minimal(base_size = 12)

# --- Variable contributions ---
p3 <- fviz_pca_var(pca_result,
                  col.var = "contrib",
                  gradient.cols = c("steelblue",
                                    "gold", "tomato"),
                  repel = TRUE) +
  labs(title = "C. Variable Contributions to PC1 & PC2") +
  theme_minimal(base_size = 12)

# --- Loading heatmap ---
load_long <- as.data.frame(pca_result$rotation[, 1:2]) |>
  rownames_to_column("Variable") |>
  pivot_longer(-Variable,
               names_to = "Component",
               values_to = "Loading")

p4 <- ggplot(load_long,
             aes(x = Component, y = Variable,
                 fill = Loading)) +

```

```

geom_tile(color = "white", linewidth = 1.2) +
geom_text(aes(label = round>Loading, 3)),
          fontface = "bold", size = 4) +
scale_fill_gradient2(low = "steelblue",
                     mid = "white",
                     high = "tomato",
                     midpoint = 0) +
labs(title = "D. Loading Heatmap: PC1 & PC2") +
theme_minimal(base_size = 12)

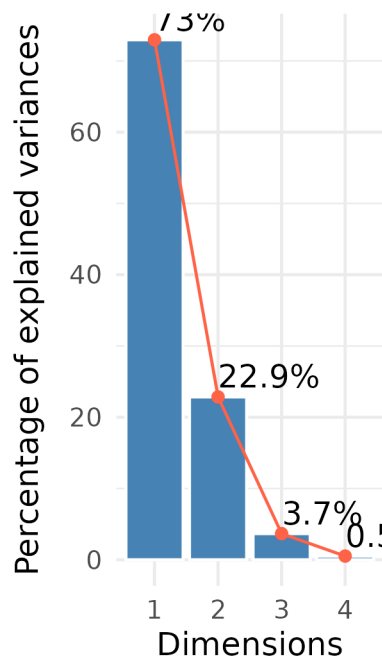
```

```

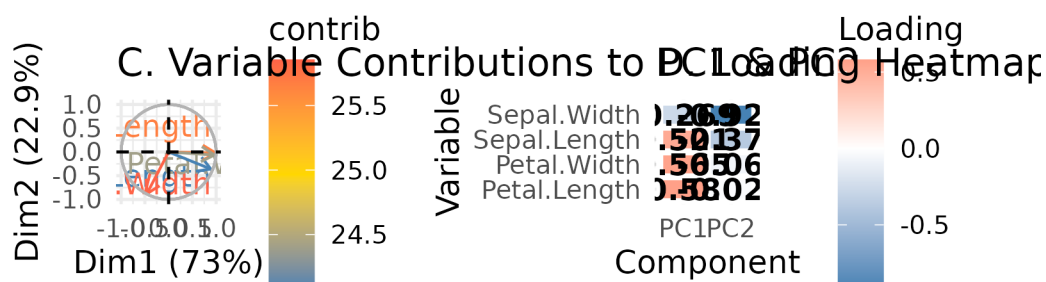
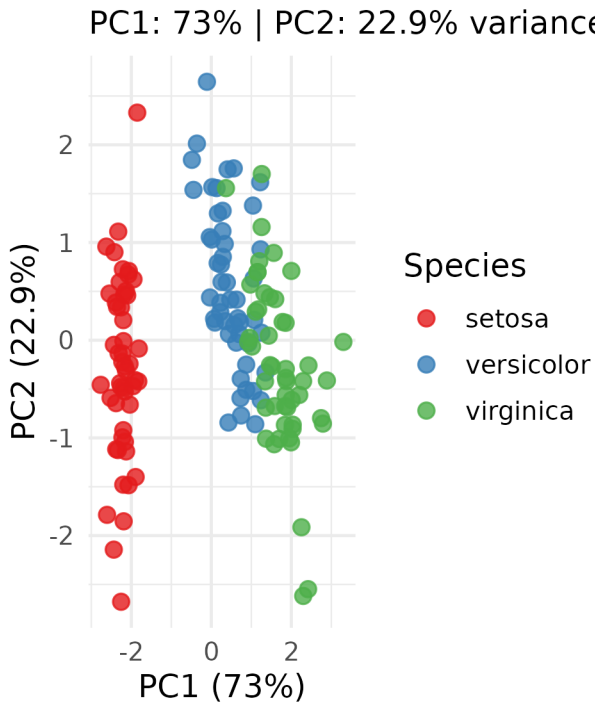
(p1 + p2) / (p3 + p4)

```

A. Scree Plot: Variance Explained



B. PCA Score Plot (PC1 vs PC2)



Code explanation:

- `prcomp(X, center, scale.)` performs PCA. When data is pre-scaled, set `center = FALSE` and `scale. = FALSE` to avoid double-standardization.

- `pca_result$rotation` contains the loadings matrix; `pca_result$x` contains the scores.
- `fviz_eig()` and `fviz_pca_var()` from `factoextra` produce publication-ready PCA diagnostics with minimal code.
- The loading heatmap (Panel D) makes the contribution of each original variable to each component immediately readable.

Exercises

Exercise 7.2

Using the `mtcars` dataset (all numeric variables):

- Standardize the data and perform PCA. How many components are needed to explain 80% of variance?
- Interpret the loadings of PC1 and PC2. What underlying dimension does each represent?
- Create a score plot colored by number of cylinders (`cyl`). Do cars cluster by cylinder count in PC space?
- Which original variable contributes most to PC1? Use `fviz_contrib()`.

Exercise 7.3 (Challenge)

Apply PCA to the `USArrests` dataset.

- Should you standardize before PCA? Compare results with and without standardization. Why does it matter here?
- Produce a full biplot using `fviz_pca_biplot()`. Which states are most similar? Which variables cluster together?
- Compute the reconstruction error for $k = 1, 2, 3$ components. At what k does reconstruction become adequate?

Factor Analysis

Introduction

Factor Analysis (FA) shares mathematical roots with PCA but has a fundamentally different purpose. PCA is a purely descriptive technique for summarizing variance. FA is a **latent variable model** — it assumes the observed variables are caused by a smaller number of unobserved **latent factors**, and seeks to identify these underlying constructs. FA is the method of choice when the research question is about the structure of a psychological scale, the dimensions of a concept, or the latent traits driving observed responses.

Theory

The Factor Model

The common factor model expresses each observed variable x_j as a linear combination of m common factors $F_1, F_2, \dots, F_m$ plus a unique error:

$$x_j = \lambda_{j1}F_1 + \lambda_{j2}F_2 + \dots + \lambda_{jm}F_m + \varepsilon_j$$

where: - λ_{jk} are **factor loadings** — how strongly factor k influences variable j - F_k are **common factors** — latent variables shared across observed variables - ε_j is the **unique factor** — variance in x_j not explained by common factors

The **communality** of variable j is the proportion of its variance explained by the common factors:

$$h_j^2 = \sum_{k=1}^m \lambda_{jk}^2$$

The **uniqueness** (specific variance) is $1 - h_j^2$.

PCA vs. Factor Analysis

Table 1: PCA vs. Factor Analysis

Feature	PCA	Factor Analysis
Goal	Maximize variance explained	Identify latent factors
Model Factors	No error term As many as variables	Includes unique variance Fewer than variables
Interpretation	Components are combinations of variables	Variables are indicators of factors
Use case	Data compression, preprocessing	Scale development, construct validation

Factor Rotation

Unrotated factor solutions are often difficult to interpret — many variables have moderate loadings on all factors. **Rotation** transforms the factor axes to achieve **simple structure**: each variable loads highly on one factor and near-zero on others.

Orthogonal rotation (Varimax): Keeps factors uncorrelated. Maximizes the variance of squared loadings within each factor — produces clean, interpretable factors.

Oblique rotation (Promax, Oblimin): Allows factors to be correlated. More realistic when underlying constructs are expected to relate to each other (e.g., verbal and spatial ability in intelligence research).

Determining the Number of Factors

Table 2: Criteria for number of factors

Criterion	Rule
Kaiser criterion	Retain factors with eigenvalue > 1
Scree plot	Retain factors before the “elbow”
Parallel analysis	Compare eigenvalues to random data eigenvalues

Criterion	Rule
Theory	Retain the number justified by the research context

Parallel analysis is the most defensible criterion and is recommended for research purposes. Note that Kaiser's criterion was originally derived for **PCA** on a correlation matrix (where each standardized variable has variance 1); applying it to common **factor analysis** — which analyzes a *reduced* correlation matrix with communalities (not 1s) on the diagonal — is more contested in the literature and is well documented to often over-extract factors. Treat it as a rough starting point, not a rule to rely on alone.

Example: Factor Analysis of a Psychological Scale

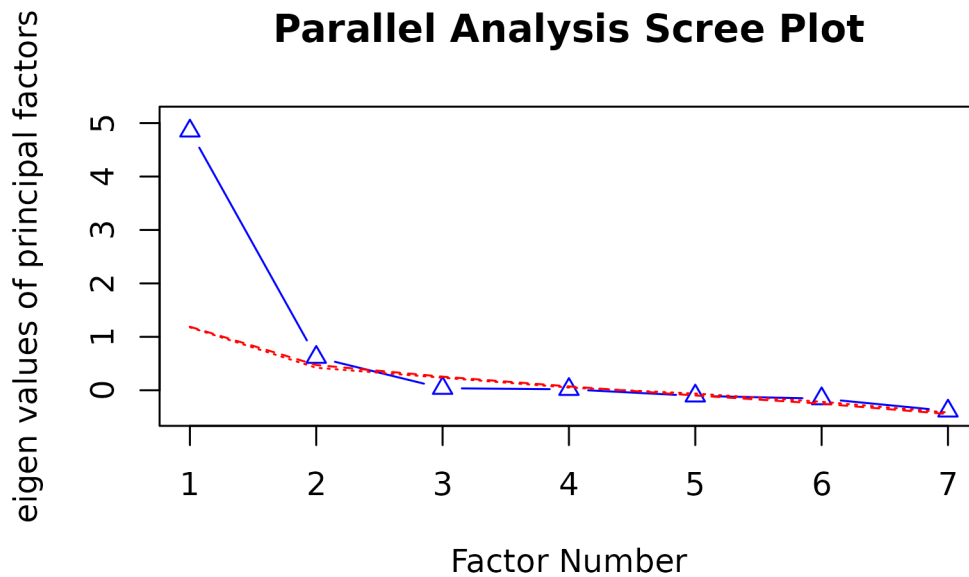
Example 7.3. A questionnaire measures student engagement with 10 items across two hypothesized dimensions: academic engagement (items 1–5) and social engagement (items 6–10). FA with 2 factors and Varimax rotation produces:

- **Factor 1:** High loadings (0.70–0.85) on items 1–5, near-zero on items 6–10 → Academic Engagement.
- **Factor 2:** Near-zero on items 1–5, high loadings (0.65–0.80) on items 6–10 → Social Engagement.
- Communalities range from 0.52 to 0.78 — most item variance is captured.
- Cronbach's alpha = 0.84 (Factor 1), 0.79 (Factor 2) — good internal consistency.

R Example: Factor Analysis

```
# --- Factor analysis on mtcars ---
# Use continuous variables
data(mtcars)
mtcars_fa <- mtcars[, c("mpg", "cyl", "disp",
                       "hp", "drat", "wt", "qsec")]

# Determine number of factors: parallel analysis
fa_parallel <- psych::fa.parallel(
  mtcars_fa, fm = "ml", fa = "fa",
  show.legend = FALSE, main = "Parallel Analysis Scree Plot"
)
```



Parallel analysis suggests that the number of factors = 2 and the number of components

```
# Fit 2-factor model with Varimax rotation
fa_result <- psych::fa(mtcars_fa,
                      nfactors = 2,
                      rotate    = "varimax",
                      fm        = "ml")

# Factor loadings
print(fa_result$loadings, cutoff = 0.3)
```

Loadings:

	ML2	ML1
mpg	-0.842	-0.365
cyl	0.778	0.543
disp	0.881	0.378
hp	0.608	0.672
drat	-0.783	
wt	0.951	
qsec		-0.995

	ML2	ML1
SS loadings	3.983	2.029
Proportion Var	0.569	0.290
Cumulative Var	0.569	0.859

```
# --- Clean loadings table ---
loadings_mat <- as.data.frame(
  unclass(fa_result$loadings)
)
loadings_mat$Communality <- fa_result$communality
loadings_mat$Uniqueness <- fa_result$uniquenesses
loadings_mat <- round(loadings_mat, 3)

kable(loadings_mat,
  caption = "Factor Loadings (Varimax Rotation) - mtcars") |>
kable_styling(bootstrap_options = c("striped", "hover"),
  full_width = FALSE) |>
column_spec(2:3, bold = TRUE)
```

Table 3: Factor Loadings (Varimax Rotation) — mtcars

	ML2	ML1	Communality	Uniqueness
mpg	-0.842	-0.365	0.842	0.158
cyl	0.778	0.543	0.901	0.099
disp	0.881	0.378	0.920	0.080
hp	0.608	0.672	0.821	0.179
drat	-0.783	-0.041	0.615	0.385
wt	0.951	0.114	0.917	0.083
qsec	-0.065	-0.995	0.995	0.005

```
# --- Factor loading plot ---
load_df <- as.data.frame(unclass(fa_result$loadings)) |>
  rownames_to_column("Variable")

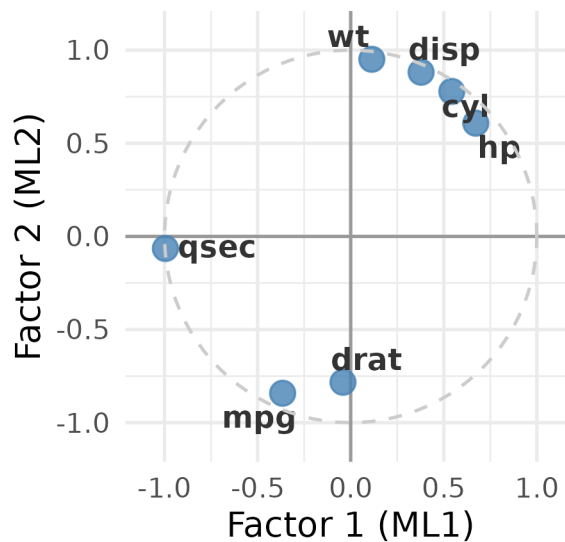
ggplot(load_df, aes(x = ML1, y = ML2,
  label = Variable)) +
  geom_hline(yintercept = 0, color = "gray60") +
  geom_vline(xintercept = 0, color = "gray60") +
  geom_point(color = "steelblue", size = 4, alpha = 0.8) +
  ggrepel::geom_text_repel(size = 4, fontface = "bold",
    color = "gray20") +
  #geom_circle_factor <- function() NULL # placeholder
  annotate("path",
    x = cos(seq(0, 2*pi, length.out = 100)),
    y = sin(seq(0, 2*pi, length.out = 100)),
    color = "gray80", linetype = "dashed") +
  labs(title = "Factor Loading Plot (Varimax Rotation)",
```

```

    subtitle = "Variables closer to an axis load primarily on that factor",
    x         = "Factor 1 (ML1)",
    y         = "Factor 2 (ML2)" +
coord_equal(xlim = c(-1.1, 1.1),
            ylim = c(-1.1, 1.1)) +
theme_minimal(base_size = 13)

```

Factor Loading Plot (Varimax Rota
Variables closer to an axis load primarily



Code explanation:

- `psych::fa.parallel()` conducts parallel analysis — the recommended method for choosing the number of factors.
- `psych::fa(data, nfactors, rotate, fm)` fits the factor model. `fm = "ml"` uses maximum likelihood estimation; `rotate = "varimax"` applies orthogonal rotation.
- `print(loadings, cutoff = 0.3)` suppresses loadings below 0.3 to highlight the simple structure — a standard reporting convention.
- The loading plot (also called a **factor map**) places each variable as a point; variables near the same axis are dominated by the same factor.

Exercises

Exercise 7.4

Using the `Harman23.cor` dataset in the `psych` package (correlations among 23 psychological tests):

- (a) Perform parallel analysis to determine the number of factors.
 - (b) Fit the factor model with Varimax rotation.
 - (c) Interpret each factor by examining which variables load on it.
 - (d) Compare Varimax (orthogonal) to Oblimin (oblique) rotation. Which produces a cleaner simple structure?
-

Linear Discriminant Analysis (LDA)

Introduction

PCA and FA are **unsupervised** — they ignore the class labels of observations. When class membership is known, we can do better: **Linear Discriminant Analysis** finds the linear combination of variables that **maximally separates** predefined classes while minimizing within-class variation. LDA is simultaneously a dimension reduction technique and a classification method. It is the supervised counterpart to PCA — and when class structure is strong, it provides far more informative low-dimensional representations.

Theory

Fisher's Discriminant Criterion

LDA finds the projection vector $\mathbf{w}$ that maximizes the ratio of **between-class variance** to **within-class variance**:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

where: - $\mathbf{S}_B = \sum_{k=1}^K n_k (\bar{\mathbf{x}}_k - \bar{\mathbf{x}})(\bar{\mathbf{x}}_k - \bar{\mathbf{x}})^T$ is the **between-class scatter matrix** - $\mathbf{S}_W = \sum_{k=1}^K \sum_{i \in C_k} (\mathbf{x}_i - \bar{\mathbf{x}}_k)(\mathbf{x}_i - \bar{\mathbf{x}}_k)^T$ is the **within-class scatter matrix**

The solution is the generalized eigendecomposition $\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w}$.

Number of Discriminants

For K classes and p variables, at most $\min(K-1, p)$ discriminant functions exist. For $K = 3$ classes, there are at most 2 discriminants — allowing a complete 2D visualization of class separation.

LDA Assumptions

1. Observations within each class follow a multivariate normal distribution.
2. All classes share the same covariance matrix (homoscedasticity).
3. Predictors are not severely multicollinear, and observations are independent — needed for $\mathbf{S}_W$ to be invertible and for the scatter-matrix estimates to be stable.

Quadratic Discriminant Analysis (QDA) relaxes assumption 2, allowing class-specific covariance matrices, at the cost of more parameters.

LDA vs. PCA

Table 1: LDA vs. PCA comparison

Feature	PCA	LDA
Supervision	Unsupervised	Supervised (uses class labels)
Objective	Maximize total variance	Maximize class separation
Max dimensions	p	$\min(K - 1, p)$
Best when	No class labels available	Class labels known

Example: LDA for Species Separation

Example 7.4. For the `iris` dataset (3 species, 4 variables), LDA produces 2 discriminant functions. LD1 explains 99.1% of between-group variance, primarily contrasting *setosa* against the other two species. LD2 (0.9%) separates *versicolor* from *virginica*. Together, the 2 LDs provide near-perfect visual separation of all three species in 2D — far cleaner than the PCA projection.

R Example: Linear Discriminant Analysis

```
# --- LDA on iris ---
data(iris)
lda_result <- MASS::lda(Species ~ .,
                        data = iris)

# Print LDA output
print(lda_result)
```

Call:

```
lda(Species ~ ., data = iris)
```

Prior probabilities of groups:

```
      setosa versicolor virginica
```

```
0.3333333 0.3333333 0.3333333
```

Group means:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
setosa	5.006	3.428	1.462	0.246
versicolor	5.936	2.770	4.260	1.326
virginica	6.588	2.974	5.552	2.026

Coefficients of linear discriminants:

	LD1	LD2
Sepal.Length	0.8293776	-0.02410215
Sepal.Width	1.5344731	-2.16452123
Petal.Length	-2.2012117	0.93192121
Petal.Width	-2.8104603	-2.83918785

Proportion of trace:

LD1	LD2
0.9912	0.0088

```
# --- Compute discriminant scores ---
lda_pred  <- predict(lda_result, iris)
lda_scores <- as.data.frame(lda_pred$x)
lda_scores$Species <- iris$Species

# Proportion of trace (between-group variance explained)
prop_trace <- lda_result$svd^2 / sum(lda_result$svd^2)
cat("Proportion of between-group variance:\n")
```

Proportion of between-group variance:

```
cat("LD1:", round(prop_trace[1]*100, 2), "%\n")
```

LD1: 99.12 %

```
cat("LD2:", round(prop_trace[2]*100, 2), "%\n")
```

LD2: 0.88 %

```

# --- Compare PCA vs LDA separation ---
# PCA scores
pca_lda    <- prcomp(scale(iris[,1:4]))
pca_scores <- as.data.frame(pca_lda$x[,1:2])
pca_scores$Species <- iris$Species

p1 <- ggplot(pca_scores,
             aes(x = PC1, y = PC2, color = Species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  stat_ellipse(level = 0.95, linewidth = 0.8) +
  scale_color_brewer(palette = "Set1") +
  labs(title    = "A. PCA: First Two Components",
       subtitle = "Unsupervised - ignores species labels",
       x = "PC1", y = "PC2") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top")

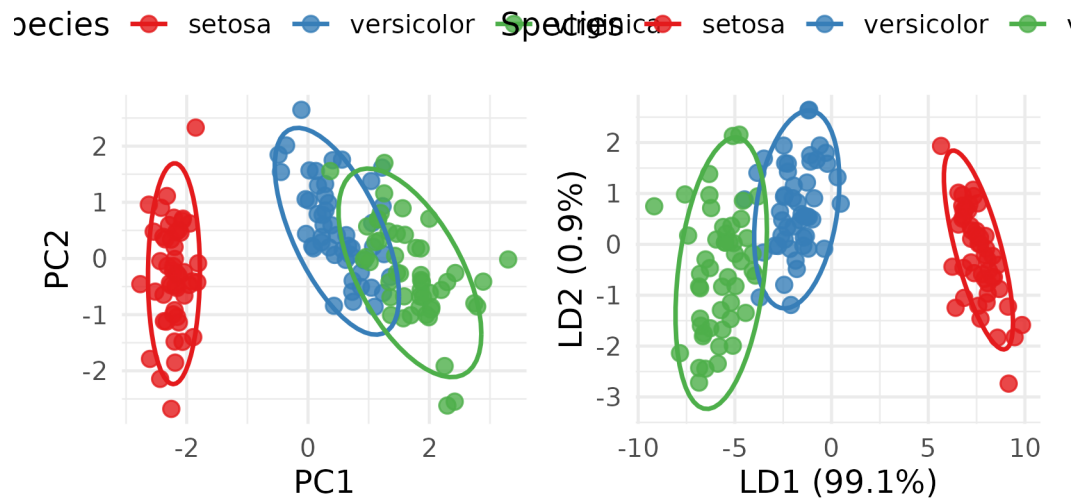
p2 <- ggplot(lda_scores,
             aes(x = LD1, y = LD2, color = Species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  stat_ellipse(level = 0.95, linewidth = 0.8) +
  scale_color_brewer(palette = "Set1") +
  labs(title    = "B. LDA: First Two Discriminants",
       subtitle = "Supervised - maximizes class separation",
       x = paste0("LD1 (",round(prop_trace[1]*100,1),"%)" ),
       y = paste0("LD2 (",round(prop_trace[2]*100,1),"%)" )) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top")

p1 + p2

```

A. PCA: First Two Components vs LDA: First Two Discriminants

Unsupervised — ignores species labels Supervised — maximizes class separation



```
# --- Classification accuracy ---
lda_class <- lda_pred$class
conf_mat <- table(Predicted = lda_class,
                  Actual     = iris$Species)

kable(conf_mat,
      caption = "LDA Classification: Confusion Matrix") |>
  kable_styling(bootstrap_options = c("striped","hover"),
                full_width = FALSE)
```

Table 2: LDA Classification: Confusion Matrix

	setosa	versicolor	virginica
setosa	50	0	0
versicolor	0	48	1
virginica	0	2	49

```
cat("Overall accuracy:",
    round(mean(lda_class == iris$Species)*100, 1), "%\n")
```

Overall accuracy: 98 %

Code explanation:

- `MASS::lda(formula, data)` fits the LDA model. The formula specifies the outcome (class variable) on the left, predictors on the right.
- `predict(lda_result, data)` returns a list with `$class` (predicted labels) and `$x` (discriminant scores).
- `stat_ellipse(level = 0.95)` draws 95% confidence ellipses for each group — a clean way to show class separation visually.
- Comparing PCA (Panel A) and LDA (Panel B) side by side demonstrates LDA's advantage when class structure is the primary interest.

Exercises

Exercise 7.5

Using the `Wine` dataset from the `rattle` package (or simulate a 3-class dataset):

- Fit LDA and compute discriminant scores for all observations.
 - Plot the first two discriminant functions. How well do the classes separate?
 - Compare classification accuracy of LDA to a simple decision tree on the same data.
 - Test the homoscedasticity assumption using `biotools::boxM()`. If violated, fit QDA (`MASS::qda()`) and compare accuracy.
-

t-SNE

Introduction

PCA and LDA produce **linear** projections — they can only capture structure that is linearly embedded in the data. Many real high-dimensional datasets have **non-linear** structure: clusters that are curved, nested, or interleaved in ways that no linear projection can reveal. **t-distributed Stochastic Neighbor Embedding (t-SNE)**, developed by van der Maaten and Hinton (2008), is the dominant method for non-linear visualization of high-dimensional data. It is widely used in genomics, single-cell biology, NLP (visualizing word embeddings), and image analysis.

Theory

The t-SNE Algorithm

t-SNE works in two steps:

Step 1 — High-dimensional similarities: For each pair of points i, j in the original space, compute a conditional probability $p_{j|i}$ proportional to the Gaussian density centered at x_i :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

Symmetrize: $p_{ij} = (p_{j|i} + p_{i|j}) / 2n$.

Step 2 — Low-dimensional similarities: In the 2D map, use a Student's t-distribution (heavier tails than Gaussian) for similarities:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

Optimization: Minimize the KL divergence between P and Q using gradient descent:

$$\text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

The heavy-tailed t-distribution in the low-dimensional space allows moderate distances to be represented as large distances — preventing the “crowding problem” that plagues Gaussian-based methods.

The Perplexity Parameter

The **perplexity** controls the effective number of neighbors each point considers. It is the most important hyperparameter:

$$\text{Perplexity} = 2^{H(P_i)}, \quad H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

Typical range: 5–50. **Low perplexity** focuses on local structure (tight clusters); **high perplexity** captures more global structure. The optimal value depends on the dataset — always try multiple values.

Critical Limitations of t-SNE

t-SNE Interpretation Cautions

1. **Distances between clusters are not meaningful** — cluster separation in a t-SNE plot does not represent true distances in the original space.
2. **Cluster sizes are not meaningful** — a large cluster in t-SNE may not be larger than a small one in reality.
3. **Results are not reproducible** without fixing the random seed — always set `set.seed()`.
4. **t-SNE is for visualization only** — do not use t-SNE coordinates as features for downstream modeling.
5. **Perplexity must be tuned** — a single plot can be misleading; always explore multiple perplexity values.

Example: t-SNE on MNIST-style Data

Example 7.5. t-SNE applied to the 784-dimensional MNIST handwritten digit dataset (60,000 images) produces a 2D map where each digit class forms a distinct cluster. Digits 4 and 9 partially overlap (visually similar), as do 3 and 5. This structure is completely invisible

in a PCA projection. The t-SNE visualization directly reveals the intrinsic difficulty of the classification problem.

R Example: t-SNE

```
# --- t-SNE on iris (as a simple demonstration) ---
data(iris)
set.seed(42)

# Run t-SNE (perplexity = 30 is default)
tsne_result <- Rtsne::Rtsne(
  iris[, 1:4],
  dims      = 2,
  perplexity = 15,
  verbose   = FALSE,
  max_iter  = 1000,
  check_duplicates = FALSE
)

tsne_df <- data.frame(
  tSNE1 = tsne_result$Y[, 1],
  tSNE2 = tsne_result$Y[, 2],
  Species = iris$Species
)
```

```
# --- Compare three perplexity values ---
perplexities <- c(5, 15, 40)

tsne_list <- lapply(perplexities, function(perp) {
  set.seed(42)
  res <- Rtsne::Rtsne(iris[, 1:4], dims = 2,
    perplexity = perp, verbose = FALSE,
    max_iter = 1000,
    check_duplicates = FALSE)

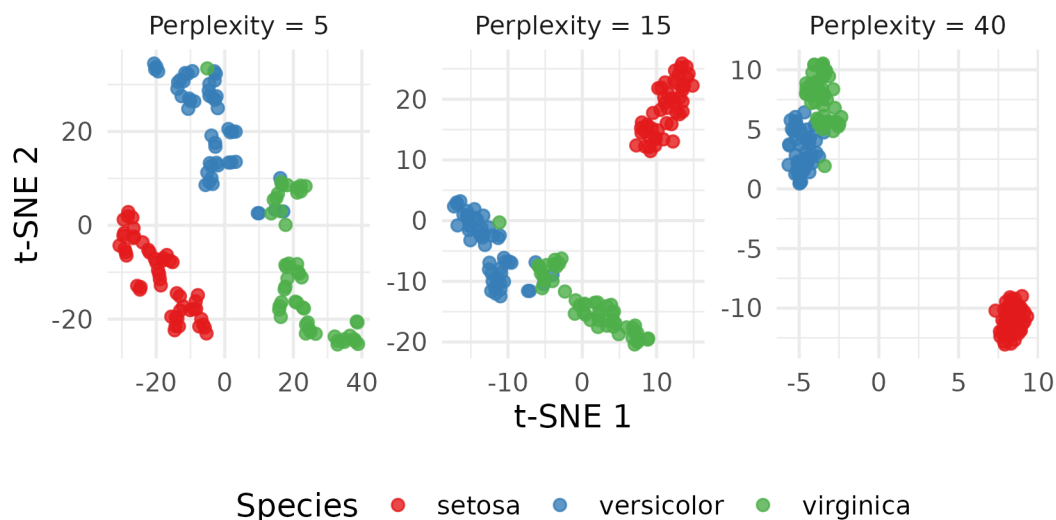
  data.frame(
    tSNE1 = res$Y[, 1],
    tSNE2 = res$Y[, 2],
    Species = iris$Species,
    Perplexity = paste0("Perplexity = ", perp)
  )
})
```

```
tsne_all <- bind_rows(tsne_list)
tsne_all$Perplexity <- factor(tsne_all$Perplexity,
                             levels = paste0("Perplexity = ",
                                              perplexities))

ggplot(tsne_all, aes(x = tSNE1, y = tSNE2,
                    color = Species)) +
  geom_point(size = 1.8, alpha = 0.8) +
  scale_color_brewer(palette = "Set1") +
  facet_wrap(~Perplexity, scales = "free", ncol = 3) +
  labs(title = "t-SNE with Different Perplexity Values",
       subtitle = "iris dataset - 3 classes, 4 dimensions → 2D",
       x = "t-SNE 1", y = "t-SNE 2") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```

t-SNE with Different Perplexity Values

iris dataset — 3 classes, 4 dimensions → 2D



```
# --- Side-by-side: PCA vs t-SNE ---
pca_iris <- prcomp(scale(iris[, 1:4]))
pca_df <- data.frame(pca_iris$x[, 1:2],
                    Species = iris$Species)

p1 <- ggplot(pca_df, aes(x = PC1, y = PC2,
                        color = Species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_color_brewer(palette = "Set1") +
```

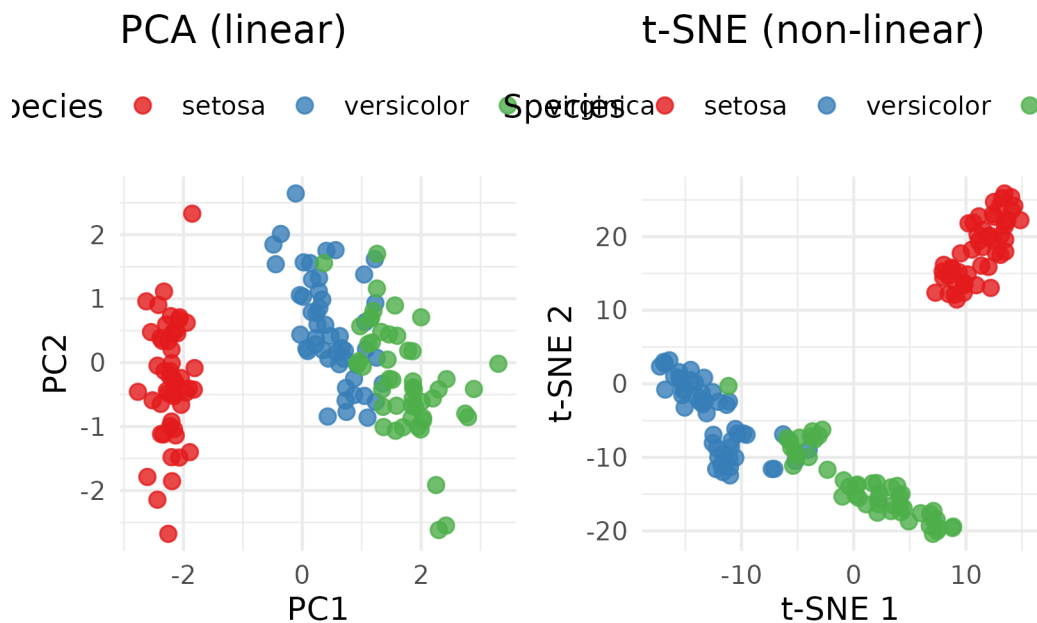
```

labs(title = "PCA (linear)",
      x = "PC1", y = "PC2") +
theme_minimal(base_size = 12) +
theme(legend.position = "top")

p2 <- ggplot(tsne_df, aes(x = tSNE1, y = tSNE2,
                        color = Species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "t-SNE (non-linear)",
        x = "t-SNE 1", y = "t-SNE 2") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top")

p1 + p2

```



Code explanation:

- `Rtsne::Rtsne(X, dims, perplexity, max_iter)` runs t-SNE. Always set `set.seed()` before calling — results are stochastic.
- `check_duplicates = FALSE` prevents errors when the dataset has identical rows (iris has some).
- `lapply()` over perplexity values followed by `bind_rows()` cleanly generates multiple t-SNE runs for comparison.
- `facet_wrap(scales = "free")` is essential for t-SNE comparisons — axes have no consistent meaning across runs, so free scales prevent misleading size comparisons.

Exercises

Exercise 7.6

Using the `digits` dataset (available via `keras` or as a subset of MNIST):

- (a) Run t-SNE with perplexities 5, 30, and 100. How does the cluster structure change?
- (b) Run PCA on the same data and compare the first two PCs to the t-SNE plot. Which reveals clearer cluster structure?
- (c) Does t-SNE preserve global structure (e.g., similarity between digit classes)? How would you verify this?

Exercise 7.7

Apply both PCA and t-SNE to the `mtcars` dataset.

- (a) In the PCA plot, color points by `cyl` (number of cylinders). In the t-SNE plot, do the same.
 - (b) Do the two methods agree on which cars are most similar?
 - (c) Given that `mtcars` has only 32 observations and 11 variables, is t-SNE appropriate here? What are the risks?
-

Feature Selection Methods

Introduction

Feature extraction (PCA, LDA, t-SNE) creates new variables as combinations of the originals. **Feature selection** instead chooses a subset of the original variables to keep, discarding the rest. Feature selection preserves interpretability — the selected features remain the original, meaningful variables. It is essential for high-dimensional datasets where many features are redundant, irrelevant, or noisy, and where model interpretability is required.

Theory

Filter Methods

Filter methods assess the relevance of each feature **independently of any model**, using statistical measures. They are fast and scalable but ignore feature interactions.

Variance threshold: Remove features with near-zero variance — they carry little information.

Correlation filter: Remove one variable from each highly correlated pair ($|r| > 0.85$; the exact threshold is a judgment call — common choices range from 0.80 to 0.90).

Statistical tests: - ANOVA F-test or mutual information for continuous features vs. categorical outcome. - Chi-square test for categorical feature vs. categorical outcome. - Spearman correlation for continuous feature vs. continuous outcome.

Wrapper Methods

Wrapper methods evaluate feature subsets by training and testing a model on each subset. They account for feature interactions but are computationally expensive.

Forward selection: Start with no features; add the feature that most improves model performance at each step.

Backward elimination: Start with all features; remove the feature whose removal least degrades performance.

Recursive Feature Elimination (RFE): Fit the model on all features, rank by importance, remove the least important, and repeat until the desired number remains.

Embedded Methods

Embedded methods perform feature selection **as part of model training** — the most computationally efficient approach.

LASSO (L1 regularization): Shrinks some coefficients exactly to zero, effectively excluding those features.

Ridge (L2 regularization): Shrinks coefficients toward zero but rarely to exactly zero — does not perform selection.

Random Forest importance: Measures each feature’s contribution to reducing impurity across all trees. The `varImp()` function in `caret` extracts these.

Comparison

Table 1: Feature selection method comparison

Method	Speed	Interactions	Interpretability	Best For
Filter	Fast	No	High	Preprocessing, first pass
Wrapper	Slow	Yes	Medium	Final model tuning
Embedded	Medium	Partial	Medium	Regularized models

Example: Feature Selection for Predicting MPG

Example 7.6. In `mtcars`, 10 potential predictors of `mpg` are available. Filter: correlation analysis removes `disp` (highly correlated with `cyl`, $r = 0.90$). Wrapper RFE: selects `wt`, `cyl`, `hp` as the top 3. Embedded (LASSO): retains `wt`, `hp`, `am` with non-zero coefficients. All methods agree that `wt` and `hp` are the most important predictors.

R Example: Feature Selection

```
# === FILTER METHOD: Correlation-based ===
data(mtcars)
predictors <- mtcars[, !names(mtcars) %in% "mpg"]

# Correlation matrix
cor_mat <- cor(predictors)

# Find highly correlated pairs (|r| > 0.85)
high_cor <- which(abs(cor_mat) > 0.85 &
                  abs(cor_mat) < 1, arr.ind = TRUE)
high_cor_df <- data.frame(
  Var1 = rownames(cor_mat)[high_cor[,1]],
  Var2 = colnames(cor_mat)[high_cor[,2]],
  r     = round(cor_mat[high_cor], 3)
) |>
  filter(Var1 < Var2) |> # remove duplicates
  arrange(desc(abs(r)))

kable(high_cor_df,
      caption = "Highly Correlated Feature Pairs (|r| > 0.85)",
      col.names = c("Variable 1", "Variable 2", "Pearson r")) |>
  kable_styling(bootstrap_options = c("striped", "hover"),
                full_width = FALSE)
```

Table 2: Highly Correlated Feature Pairs ($|r| > 0.85$)

Variable 1	Variable 2	Pearson r
cyl	disp	0.902
disp	wt	0.888

```
# === WRAPPER METHOD: Recursive Feature Elimination ===
set.seed(42)
ctrl <- rfeControl(functions = lmFuncs,
                  method     = "cv",
                  number     = 10,
                  verbose    = FALSE)

rfe_result <- rfe(x      = predictors,
                  y      = mtcars$mpg,
                  sizes   = c(2, 3, 4, 5, 6),
                  rfeControl = ctrl)

cat("RFE Selected Features:\n")
```

RFE Selected Features:

```
print(predictors(rfe_result))
```

```
[1] "wt"    "am"    "gear"
```

```
cat("\nOptimal number of features:", rfe_result$optsize, "\n")
```

Optimal number of features: 3

```
# === EMBEDDED METHOD: LASSO ===
library(glmnet)

X      <- as.matrix(predictors)
y      <- mtcars$mpg
X_std  <- scale(X)

# Cross-validated LASSO
set.seed(42)
lasso_cv <- cv.glmnet(X_std, y, alpha = 1, nfolds = 10)

# Coefficients at optimal lambda
lasso_coef <- coef(lasso_cv, s = "lambda.min")
lasso_df   <- data.frame(
  Variable    = rownames(lasso_coef)[-1],
  Coefficient = round(as.numeric(lasso_coef)[-1], 4)
) |>
  filter(Coefficient != 0) |>
  arrange(desc(abs(Coefficient)))

kable(lasso_df,
  caption = "LASSO Selected Features ( = .min)" |>
  kable_styling(bootstrap_options = c("striped", "hover"),
    full_width = FALSE)
```

Table 3: LASSO Selected Features (= .min)

Variable	Coefficient
wt	-2.6498
cyl	-1.5825
hp	-0.8011

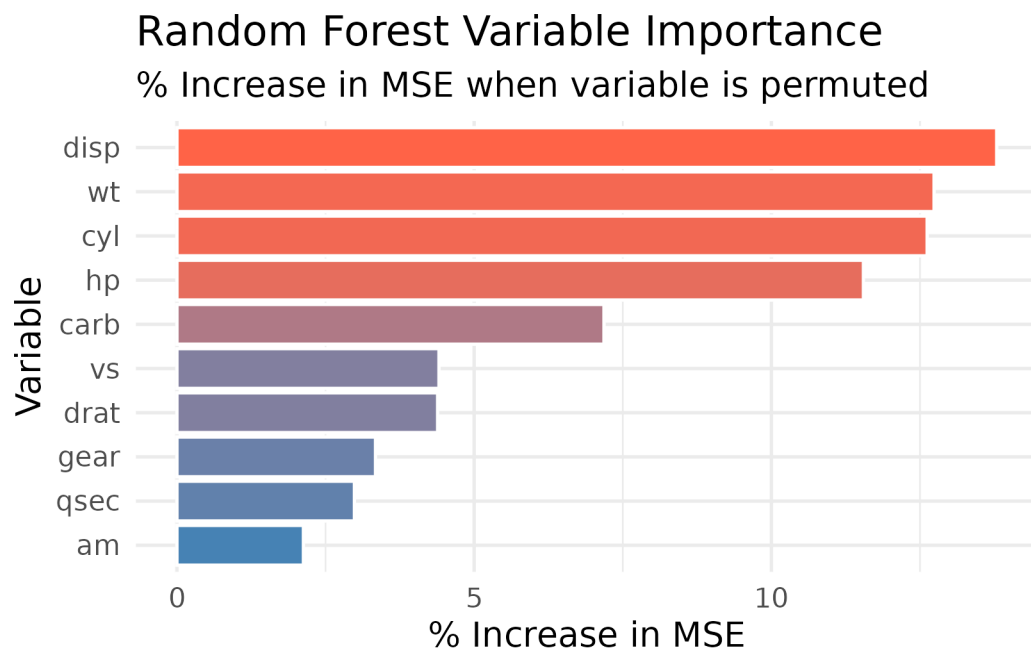

```

# --- Variable importance comparison ---
# Random Forest importance
library(randomForest)
set.seed(42)
rf_model <- randomForest(mpg ~ ., data = mtcars,
                        importance = TRUE)

imp_df <- as.data.frame(importance(rf_model)) |>
  rownames_to_column("Variable") |>
  arrange(desc(`%IncMSE`)) |>
  rename(Importance = `%IncMSE`)

ggplot(imp_df, aes(x = reorder(Variable, Importance),
                  y = Importance, fill = Importance)) +
  geom_col(color = "white") +
  scale_fill_gradient(low = "steelblue", high = "tomato") +
  coord_flip() +
  labs(title = "Random Forest Variable Importance",
       subtitle = "% Increase in MSE when variable is permuted",
       x = "Variable",
       y = "% Increase in MSE") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")

```



Code explanation:

- `rfe(x, y, sizes, rfeControl)` runs Recursive Feature Elimination using cross-validation. `lmFuncs` specifies linear regression as the model.
- `cv.glmnet(X, y, alpha=1)` fits LASSO with cross-validated lambda selection. `alpha=1` is LASSO; `alpha=0` is Ridge; values between are Elastic Net.
- `randomForest(formula, importance=TRUE)` computes permutation-based variable importance — a widely used approach for ranking features that may interact non-linearly (though not the only option; SHAP values and other permutation-importance variants are also common alternatives worth knowing about).

Exercises

Exercise 7.8

Using the **Boston** dataset from **MASS** (housing prices):

- Apply correlation filter: remove features with $|r| > 0.85$ with any other feature.
 - Apply RFE with cross-validation to identify the top 5 features for predicting `medv`.
 - Fit a LASSO model. Which features are retained at `lambda.min` vs. `lambda.1se`?
 - Do all three methods agree on the most important features? Summarize in a comparison table.
-

Evaluating Dimension Reduction

Introduction

Dimension reduction involves an inherent trade-off: more dimensions retain more information but defeat the purpose of reduction. Fewer dimensions are more interpretable and computationally efficient but lose information. **Evaluating** a dimension reduction solution means answering: *How many dimensions do we need, and how much do we lose by using fewer?* This section provides a principled toolkit of criteria for making this decision.

Theory

Scree Plot and the Elbow Rule

A **scree plot** displays eigenvalues (or variance explained) against the component number. The **elbow** — where the curve bends and flattens — indicates the number of components beyond which additional components contribute little. The elbow is identified visually and is subjective; it works best when there is a clear break.

Kaiser's Criterion

For PCA on standardized data, retain components with **eigenvalue** > 1 . The rationale: a component with eigenvalue < 1 explains less variance than a single original standardized variable — it is not “earning its keep.” This criterion tends to retain too many components in large datasets.

Cumulative Variance Threshold

Retain the minimum number of components needed to explain a specified proportion of total variance (commonly 70%, 80%, or 90%). The threshold should be chosen based on the purpose:

- For **visualization**: 2–3 components (interpretability is paramount)
- For **preprocessing before modeling**: 80–90% variance
- For **lossy compression**: problem-specific

Reconstruction Error

The **reconstruction error** measures how well the reduced representation can recreate the original data. Using the covariance definition from this chapter, $\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$, the *sum-of-squares* version of the reconstruction error is:

$$\text{SSE}(k) = \|\mathbf{X} - \hat{\mathbf{X}}_k\|_F^2 = (n-1) \sum_{j=k+1}^p \lambda_j$$

where $\hat{\mathbf{X}}_k = \mathbf{Z}_k \mathbf{W}_k^T$ is the rank- k approximation. Dividing by $(n-1)$ gives the **mean reconstruction error per unit of total variance**:

$$\text{RE}(k) = \frac{\text{SSE}(k)}{n-1} = \sum_{j=k+1}^p \lambda_j$$

so $\text{RE}(k)$ (not $\text{SSE}(k)$) is what equals the sum of the discarded eigenvalues — confirming that PCA is the optimal linear reconstruction (Eckart–Young theorem).

 Watch the normalization when you compute this in R

It's easy to introduce an off-by-a-constant error here. `mean((X - X_hat)^2)` in R averages over **all** $n \times p$ **matrix entries**, i.e. it divides $\text{SSE}(k)$ by np , not by $(n-1)$. That is a *different, smaller* number than $\text{RE}(k)$ above by a factor of roughly p . If you want your computed reconstruction error to numerically match $\sum_{j=k+1}^p \lambda_j$, divide by $(n-1)$ explicitly — see the corrected R code below.

Parallel Analysis

Compare the eigenvalues from the actual data to eigenvalues from **random data** of the same dimensions. Retain components whose eigenvalues exceed the corresponding random eigenvalues. This is more rigorous than the elbow rule or Kaiser's criterion and is the current methodological recommendation.

Cross-Validation for Dimension Selection

In a supervised context, use cross-validation to evaluate predictive performance as a function of the number of retained dimensions — the optimal k balances dimensionality reduction against predictive accuracy.

Example: Choosing k for PCA

Example 7.7. For a dataset with 20 variables:

Criterion	Suggested k
Elbow rule	3
Kaiser (eigenvalue > 1)	5
80% variance	4
Parallel analysis	3
Cross-validation (classification)	4

Most criteria cluster around 3–4 components, but they do not all agree: parallel analysis and the elbow rule point to the more conservative $k = 3$, while the 80% variance threshold and cross-validation point to $k = 4$. There is no single “correct” answer here — it is a trade-off. Because the purpose in this scenario is a downstream classification model (where cross-validated predictive performance is the most directly relevant criterion), we choose $k = 4$, accepting one extra dimension in exchange for the measured gain in predictive accuracy. Had the goal instead been a parsimonious 2D visualization, $k = 3$ from parallel analysis would be the more defensible choice. The lesson: report *which* criteria you used and *why* you weighted one over another, rather than implying they all agreed.

R Example: Evaluating Dimension Reduction

```
# --- Comprehensive evaluation for mtcars PCA ---
data(mtcars)
mtcars_scaled <- scale(mtcars)
pca_eval      <- prcomp(mtcars_scaled)

# Eigenvalues and variance explained
eval_df <- data.frame(
  Component = 1:ncol(mtcars),
  Eigenvalue = round(pca_eval$sdev^2, 4),
  PVE        = round(pca_eval$sdev^2 /
                     sum(pca_eval$sdev^2) * 100, 2),
  Cum_PVE    = round(cumsum(pca_eval$sdev^2 /
                           sum(pca_eval$sdev^2)) * 100, 2)
)

kable(eval_df,
      caption = "PCA Evaluation: mtcars",
```

```

col.names = c("Component", "Eigenvalue",
              "% Variance", "Cumulative %")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(2, bold = TRUE,
            color = ifelse(eval_df$Eigenvalue > 1,
                          "darkgreen", "tomato")) |>
column_spec(4, bold = TRUE,
            color = ifelse(eval_df$Cum_PVE >= 80,
                          "darkgreen", "steelblue"))

```

Table 1: PCA Evaluation: mtcars

Component	Eigenvalue	% Variance	Cumulative %
1	6.6084	60.08	60.08
2	2.6505	24.10	84.17
3	0.6272	5.70	89.87
4	0.2696	2.45	92.32
5	0.2235	2.03	94.36
6	0.2116	1.92	96.28
7	0.1353	1.23	97.51
8	0.1229	1.12	98.63
9	0.0770	0.70	99.33
10	0.0520	0.47	99.80
11	0.0220	0.20	100.00

```

# --- Reconstruction error as function of k ---
X   <- mtcars_scaled
W   <- pca_eval$rotation
Z   <- pca_eval$x

n_obs <- nrow(mtcars)

recon_error <- sapply(1:ncol(mtcars), function(k) {
  X_hat <- Z[, 1:k, drop=FALSE] %*% t(W[, 1:k, drop=FALSE])
  # Divide by (n - 1), matching the S = X'X/(n-1) convention used
  # throughout this chapter -- NOT mean(), which divides by n*p and
  # would silently rescale the result by a factor of p.
  sum((X - X_hat)^2) / (n_obs - 1)
})

recon_df <- data.frame(

```

```

k      = 1:ncol(mtcars),
error = round(recon_error, 4),
pve    = eval_df$Cum_PVE
)

# Sanity check: RE(k) should equal the sum of the discarded eigenvalues,
# sum_{j=k+1}^p lambda_j (0 once k = p, i.e. nothing left to discard)
eigenvalues <- pca_eval$sdev^2
discarded_sum <- sapply(1:ncol(mtcars), function(k) {
  if (k == ncol(mtcars)) 0 else sum(eigenvalues[(k+1):ncol(mtcars)])
})
stopifnot(all.equal(recon_error, discarded_sum, tolerance = 1e-8))

```

```

# --- Four evaluation plots ---
p1 <- ggplot(eval_df, aes(x = Component,
                          y = Eigenvalue)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_point(color = "steelblue", size = 3) +
  geom_hline(yintercept = 1, color = "tomato",
             linetype = "dashed", linewidth = 1) +
  annotate("text", x = 8, y = 1.15,
          label = "Kaiser criterion (=1)",
          color = "tomato", size = 3.5) +
  labs(title = "A. Scree Plot",
       x = "Component", y = "Eigenvalue") +
  theme_minimal(base_size = 11)

p2 <- ggplot(eval_df, aes(x = Component,
                          y = Cum_PVE)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_point(color = "steelblue", size = 3) +
  geom_hline(yintercept = c(70, 80, 90),
             color = c("tomato", "seagreen", "purple"),
             linetype = "dashed", linewidth = 0.8) +
  annotate("text", x = 9.5, y = c(71.5, 81.5, 91.5),
          label = c("70%", "80%", "90%"),
          color = c("tomato", "seagreen", "purple"),
          size = 3.2) +
  labs(title = "B. Cumulative Variance Explained",
       x = "Component", y = "Cumulative %") +
  theme_minimal(base_size = 11)

p3 <- ggplot(recon_df, aes(x = k, y = error)) +
  geom_line(color = "tomato", linewidth = 1.2) +

```

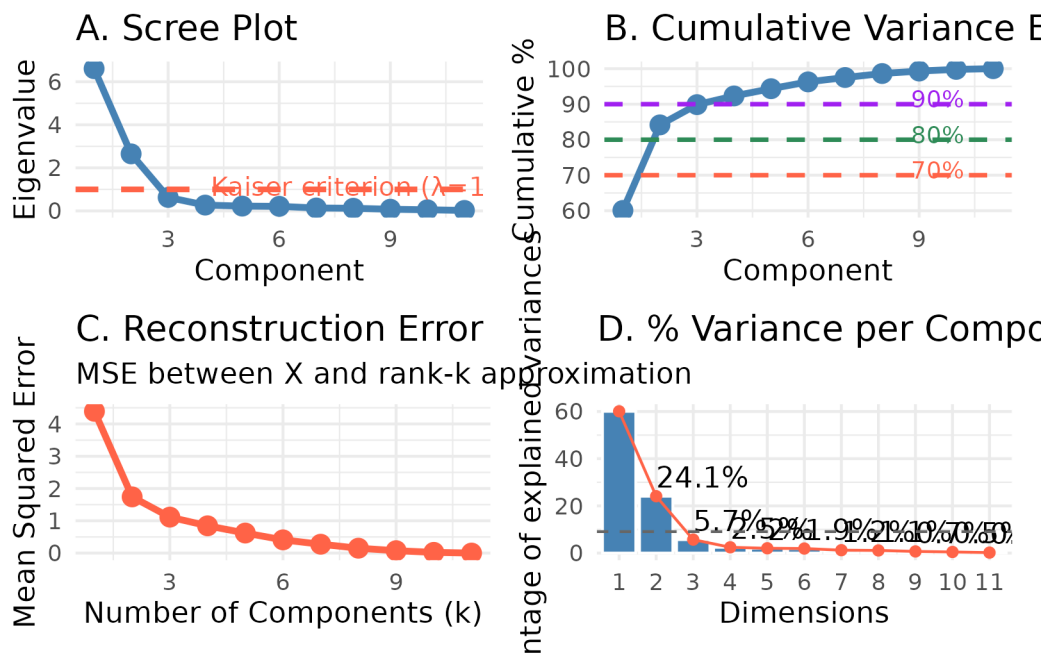
```

geom_point(color = "tomato", size = 3) +
labs(title = "C. Reconstruction Error",
      subtitle = "MSE between X and rank-k approximation",
      x = "Number of Components (k)",
      y = "Mean Squared Error") +
theme_minimal(base_size = 11)

p4 <- fviz_eig(pca_eval, addlabels = TRUE,
              barfill = "steelblue",
              barcolor = "white",
              linecolor = "tomato",
              ncp = 11) +
geom_hline(yintercept = 100/ncol(mtcars),
           linetype = "dashed", color = "gray40") +
labs(title = "D. % Variance per Component") +
theme_minimal(base_size = 11)

(p1 + p2) / (p3 + p4)

```



Code explanation:

- `Z[, 1:k] %*% t(W[, 1:k])` computes the rank- k reconstruction of the standardized data. The reconstruction error decreases monotonically as k increases.
- The four panels together provide a complete evaluation framework: scree plot for elbow, cumulative variance for threshold, reconstruction error for information loss, and bar chart for per-component contribution.

- The dashed line at $100/p$ in Panel D represents the “equal contribution” baseline — components above this line explain more than their fair share.

Exercises

💡 Exercise 7.9

Using the `decathlon2` dataset from `factoextra`:

- Perform PCA and apply all four criteria (elbow, Kaiser, 80% variance, parallel analysis) to choose k .
- Do the criteria agree? Which k would you choose and why?
- Compute reconstruction errors for $k = 1, \dots, p$. At what k does the error drop below 20% of the original variance?

💡 Exercise 7.10 (Challenge)

Compare PCA and t-SNE as preprocessing steps for K-means clustering on the `iris` dataset.

- Cluster using raw data (no reduction), first 2 PCA components, and 2 t-SNE dimensions.
- Compare cluster quality using the Adjusted Rand Index (`mclust::adjustedRandIndex()`).
- Which preprocessing leads to better recovery of the true species groups? Why?

Chapter Lab Activity: Dimension Reduction Pipeline with decathlon2

Objectives

In this lab you will apply the full dimension reduction pipeline — PCA, FA, LDA, t-SNE, feature selection, and evaluation — to the `decathlon2` dataset from `factoextra`. This dataset records performance of athletes across 10 events, providing a rich multivariate structure with a clear theoretical interpretation (general athletic ability vs. event-specific skills).

The Dataset

```
# decathlon2: 27 athletes × 13 variables
# (10 events + rank + points + competition)
data(decathlon2, package = "factoextra")

# Use event scores only (columns 1-10)
dec_events <- decathlon2[, 1:10]

cat("Dimensions:", nrow(dec_events), "athletes ×",
    ncol(dec_events), "events\n\n")
```

Dimensions: 27 athletes × 10 events

```
kable(head(round(dec_events, 2), 6),
      caption = "decathlon2: First 6 Athletes") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
               font_size = 11)
```

Table 1: decathlon2: First 6 Athletes

	X100m	Long.jump	Shot.put	High.jump	X400m	X110m.hurdle	Discus	Pole.vault	Javelin
--	-------	-----------	----------	-----------	-------	--------------	--------	------------	---------

SEBRLE	11.04	7.58	14.83	2.07	49.81	14.69	43.75	5.02
CLAY	10.76	7.40	14.26	1.86	49.37	14.05	50.72	4.92
BERNARD	11.02	7.23	14.25	1.92	48.93	14.99	40.87	5.32
YURKOV	11.34	7.09	15.19	2.10	50.42	15.31	46.26	4.72
ZSIVOCZKY	11.13	7.30	13.48	2.01	48.62	14.17	45.67	4.42
McMULLEN	10.83	7.31	13.76	2.13	49.91	14.38	44.41	4.42

Lab Task 1: PCA — Structure of Athletic Performance

```
# Standardize (events have different units/scales)
dec_scaled <- scale(dec_events)
pca_dec    <- prcomp(dec_scaled)

# Variance explained
pve_dec    <- pca_dec$sdev^2 / sum(pca_dec$sdev^2)
cum_pve_dec <- cumsum(pve_dec)

var_table <- data.frame(
  PC          = paste0("PC", 1:10),
  Eigenvalue  = round(pca_dec$sdev^2, 3),
  PVE         = round(pve_dec * 100, 2),
  Cum_PVE     = round(cum_pve_dec * 100, 2)
)

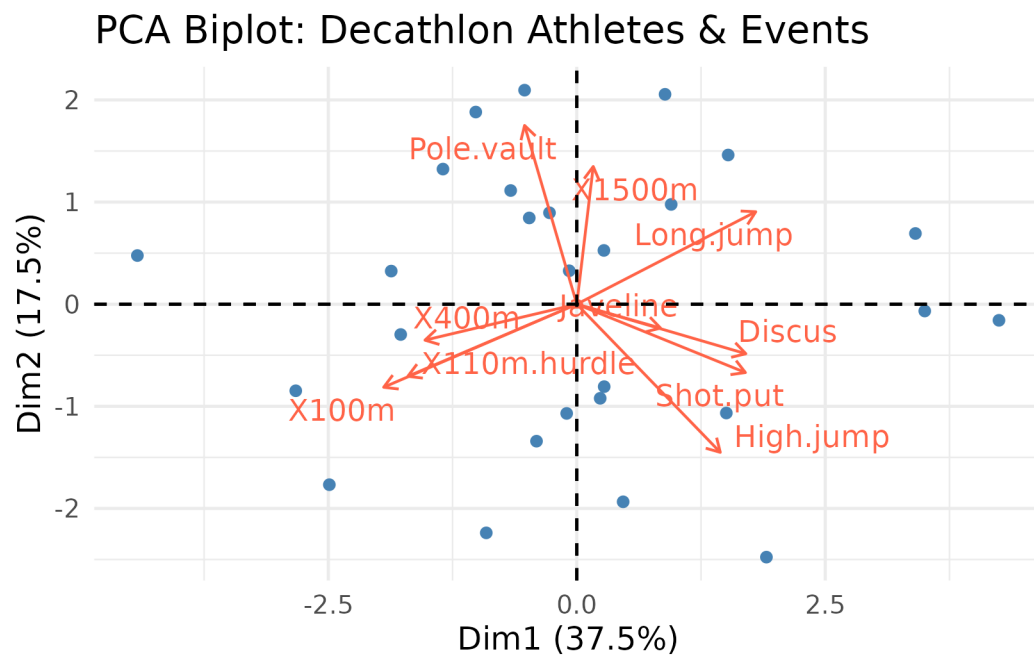
kable(var_table,
      caption  = "PCA Variance Explained - Decathlon Events",
      col.names = c("Component", "Eigenvalue",
                    "% Variance", "Cumulative %")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(2, color = ifelse(var_table$Eigenvalue > 1,
                              "darkgreen", "tomato"),
            bold = TRUE)
```

Table 2: PCA Variance Explained — Decathlon Events

Component	Eigenvalue	% Variance	Cumulative %
PC1	3.750	37.50	37.50
PC2	1.745	17.45	54.95
PC3	1.518	15.18	70.13
PC4	1.032	10.32	80.45

PC5	0.618	6.18	86.63
PC6	0.428	4.28	90.91
PC7	0.326	3.26	94.17
PC8	0.279	2.79	96.97
PC9	0.191	1.91	98.88
PC10	0.112	1.12	100.00

```
# Biplot
fviz_pca_biplot(pca_dec,
  label      = "var",
  col.var    = "tomato",
  col.ind    = "steelblue",
  repel      = TRUE,
  title      = "PCA Biplot: Decathlon Athletes & Events") +
  theme_minimal(base_size = 12)
```

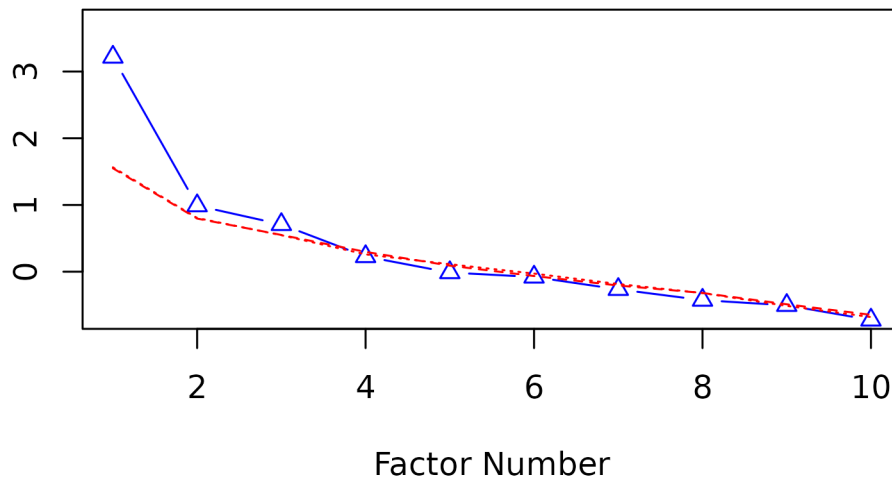


Lab Task 2: Factor Analysis — Latent Athletic Dimensions

```
# Parallel analysis to determine number of factors
psych::fa.parallel(dec_scaled, fm = "ml", fa = "fa",
  show.legend = FALSE,
  main = "Parallel Analysis: Decathlon Events")
```

eigen values of principal factors

Parallel Analysis: Decathlon Events



Parallel analysis suggests that the number of factors = 1 and the number of components

```
# 2-factor model with Varimax rotation
fa_dec <- psych::fa(dec_scaled, nfactors = 2,
                    rotate = "varimax", fm = "ml")

# Loadings table
load_dec <- as.data.frame(unclass(fa_dec$loadings)) |>
  rownames_to_column("Event") |>
  mutate(
    ML1 = round(ML1, 3),
    ML2 = round(ML2, 3),
    Communality = round(fa_dec$communality, 3),
    Uniqueness = round(fa_dec$uniquenesses, 3)
  )

kable(load_dec,
      caption = "Factor Loadings: Decathlon (Varimax Rotation)") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
                full_width = FALSE) |>
  column_spec(2:3, bold = TRUE)
```

Table 3: Factor Loadings: Decathlon (Varimax Rotation)

Event	ML2	ML1	Communality	Uniqueness
X100m	0.854	-0.290	0.814	0.186
Long.jump	-0.732	0.305	0.628	0.372
Shot.put	-0.097	0.993	0.995	0.005
High.jump	-0.166	0.563	0.344	0.656
X400m	0.645	-0.146	0.437	0.563
X110m.hurdle	0.731	-0.201	0.575	0.425
Discus	-0.239	0.704	0.553	0.447
Pole.vault	0.025	-0.069	0.005	0.995
Javeline	-0.052	0.473	0.227	0.773
X1500m	-0.157	-0.005	0.025	0.975

Lab Task 3: t-SNE Visualization

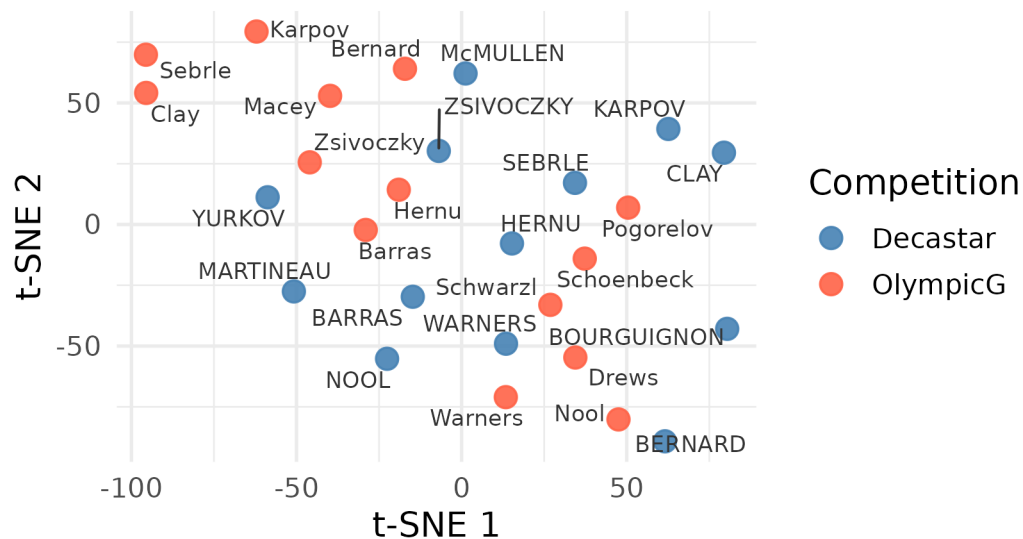
```
set.seed(42)
tsne_dec <- Rtsne::Rtsne(dec_scaled, dims = 2,
                        perplexity = 8,
                        max_iter   = 1000,
                        verbose     = FALSE,
                        check_duplicates = FALSE)

tsne_dec_df <- data.frame(
  tSNE1      = tsne_dec$Y[, 1],
  tSNE2      = tsne_dec$Y[, 2],
  Athlete    = rownames(dec_events),
  Competition = decathlon2$Competition
)

ggplot(tsne_dec_df, aes(x = tSNE1, y = tSNE2,
                        color = Competition,
                        label = Athlete)) +
  geom_point(size = 3.5, alpha = 0.9) +
  ggrepel::geom_text_repel(size = 3, color = "gray20") +
  scale_color_manual(values = c("Decastar" = "steelblue",
                                "OlympicG" = "tomato")) +
  labs(title = "t-SNE: Decathlon Athletes",
       subtitle = "Colored by competition (Decastar vs. Olympic Games)",
       x = "t-SNE 1", y = "t-SNE 2") +
  theme_minimal(base_size = 13)
```

t-SNE: Decathlon Athletes

Colored by competition (Decastar vs. Olympic Games)



Lab Task 4: Feature Selection for Predicting Total Score

```
# Use total points as outcome
dec_full <- decathlon2[, 1:11] # 10 events + Points
names(dec_full)[11] <- "Points"

# Correlation filter
cor_points <- cor(dec_full)[,"Points"]
cor_filter_df <- data.frame(
  Event      = names(cor_points[-11]),
  Correlation = round(cor_points[-11], 3)
) |>
  arrange(desc(abs(Correlation)))

kable(cor_filter_df,
      caption = "Filter Method: Correlation of Events with Total Points") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
                full_width = FALSE) |>
  column_spec(2, bold = TRUE,
              color = ifelse(abs(cor_filter_df$Correlation) > 0.5,
                             "tomato", "steelblue"))
```

Table 4: Filter Method: Correlation of Events with Total Points

	Event	Correlation
Discus	Discus	-0.633
Shot.put	Shot.put	-0.604
Long.jump	Long.jump	-0.499
High.jump	High.jump	-0.498
Javeline	Javeline	-0.430
X100m	X100m	0.374
X400m	X400m	0.368
X1500m	X1500m	-0.286
X110m.hurdle	X110m.hurdle	0.197
Pole.vault	Pole.vault	-0.014

Lab Task 5: Comprehensive Evaluation

```

p1 <- fviz_eig(pca_dec, addlabels = TRUE,
               barfill = "steelblue",
               barcolor = "white",
               linecolor = "tomato",
               ncp = 10) +
  labs(title = "A. Scree Plot") +
  theme_minimal(base_size = 11)

p2 <- ggplot(data.frame(k = 1:10,
                        cum = cum_pve_dec * 100),
             aes(x = k, y = cum)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_point(size = 3, color = "steelblue") +
  geom_hline(yintercept = 80, color = "tomato",
             linetype = "dashed") +
  labs(title = "B. Cumulative Variance",
       x = "k", y = "Cumulative %") +
  theme_minimal(base_size = 11)

p3 <- ggplot(load_dec,
             aes(x = ML1, y = ML2, label = Event)) +
  geom_hline(yintercept = 0, color = "gray70") +
  geom_vline(xintercept = 0, color = "gray70") +
  geom_point(color = "steelblue", size = 4) +
  ggrepel::geom_text_repel(size = 3.5, fontface = "bold") +

```



```

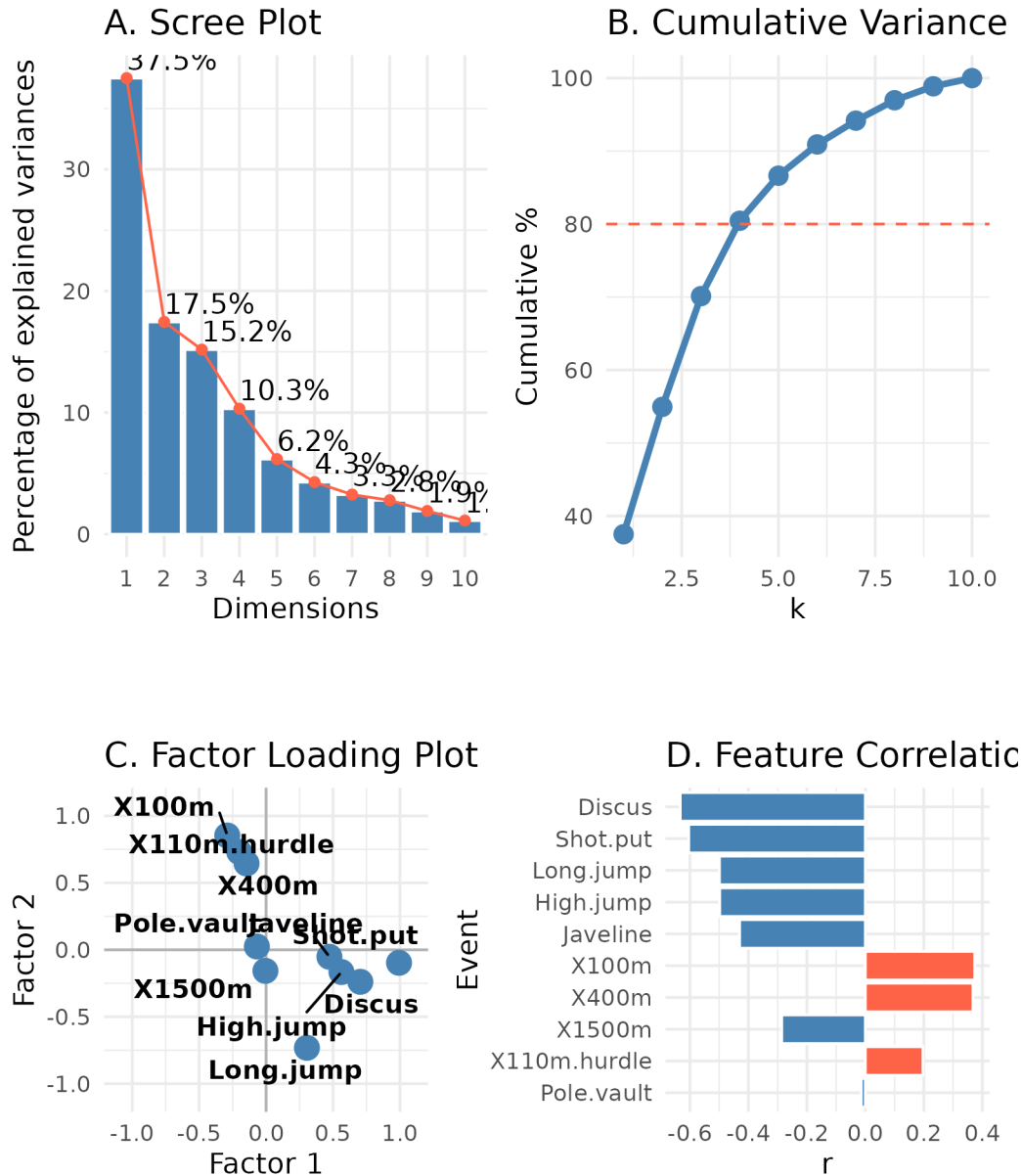
labs(title = "C. Factor Loading Plot",
      x = "Factor 1", y = "Factor 2") +
coord_equal(xlim = c(-1.1,1.1), ylim = c(-1.1,1.1)) +
theme_minimal(base_size = 11)

p4 <- ggplot(cor_filter_df,
             aes(x = reorder(Event, abs(Correlation)),
                 y = Correlation, fill = Correlation > 0)) +
geom_col(color = "white") +
scale_fill_manual(values = c("FALSE" = "steelblue",
                             "TRUE" = "tomato")) +

coord_flip() +
labs(title = "D. Feature Correlations with Points",
      x = "Event", y = "r") +
theme_minimal(base_size = 11) +
theme(legend.position = "none")

(p1 + p2) / (p3 + p4)

```



Lab Discussion Questions

Answer the following in writing:

1. **PCA Interpretation:** Based on the biplot in Lab Task 1, which events cluster together? What does PC1 represent physically (hint: consider which events load positively and which negatively on PC1)?

2. **PCA vs. FA:** Both PCA and FA reduce the decathlon data to 2 dimensions. Compare the loadings — do they tell a similar story? When would you prefer FA over PCA for this dataset?
 3. **t-SNE Findings:** In Lab Task 3, do athletes from the same competition (Decastar vs. Olympic Games) cluster together in the t-SNE plot? What would it mean if they did not?
 4. **Feature Selection:** Based on Lab Task 4, which events are most predictive of total points? Is this consistent with the PCA loadings? What events might be candidates for removal from a predictive model?
 5. **Curse of Dimensionality:** The decathlon dataset has 27 observations and 10 variables. Is this ratio (observations to variables) concerning? How does dimension reduction help, and what are the risks of reducing to 2 dimensions?
-

Chapter Summary

Summary

This chapter established dimension reduction as an essential toolkit for working with high-dimensional data:

- **The curse of dimensionality** — sparsity, distance concentration, and exponential sample requirements make direct analysis of high-dimensional data unreliable; dimension reduction is a necessary response.
- **PCA** — the foundational linear method; finds orthogonal components maximizing variance; requires standardization; interpreted via loadings and score plots.
- **Factor Analysis** — a latent variable model identifying unobserved constructs; rotation (Varimax, Oblimin) improves interpretability; distinct from PCA in purpose and assumptions.
- **LDA** — supervised reduction maximizing between-class separation; more informative than PCA when class labels are available; limited to $K - 1$ dimensions.
- **t-SNE** — non-linear visualization of complex structure; powerful but distances and cluster sizes are not interpretable; perplexity must be tuned; for visualization only.
- **Feature selection** — filter (fast, model-free), wrapper (slower, interaction-aware), and embedded (LASSO, RF importance) methods select original variables rather than creating new ones.
- **Evaluation** — scree plot, Kaiser criterion, cumulative variance, reconstruction error, and parallel analysis provide complementary perspectives for choosing the number of dimensions.

! Key Formulas to Know

PCA Eigendecomposition:

$$\mathbf{S}\mathbf{w}_k = \lambda_k \mathbf{w}_k, \quad \text{PVE}_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

Reconstruction Error (with $\mathbf{S} = \mathbf{X}^T \mathbf{X} / (n - 1)$):

$$\text{RE}(k) = \frac{\|\mathbf{X} - \hat{\mathbf{X}}_k\|_F^2}{n - 1} = \sum_{j=k+1}^p \lambda_j \quad (\text{in R: } \text{sum}(\dots)/(n-1), \text{ not mean}(\dots))$$

Fisher's LDA Criterion:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

Factor Model:

$$x_j = \sum_{k=1}^m \lambda_{jk} F_k + \varepsilon_j, \quad h_j^2 = \sum_{k=1}^m \lambda_{jk}^2$$

t-SNE KL Divergence:

$$\text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

References

- Bellman, R. (1957). *Dynamic programming*. Princeton University Press.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
- Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.
- Harrison, D., & Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5(1), 81–102.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer. <https://hastie.su.domains/ElemStatLearn/>
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning with applications in R* (2nd ed.). Springer. <https://www.statlearning.com>
- Kassambara, A. (2023). *factoextra: Extract and visualize the results of multivariate data analyses* (R package version 1.0.7). <https://CRAN.R-project.org/package=factoextra>
- Krijthe, J. H. (2015). *Rtsne: T-distributed stochastic neighbor embedding* (R package version 0.15). <https://github.com/jkrijthe/Rtsne>
- Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of Statistical Software*, 28(5), 1–26.
- Lê, S., Josse, J., & Husson, F. (2008). FactoMineR: An R package for multivariate analysis. *Journal of Statistical Software*, 25(1), 1–18.

- Maaten, L. van der, & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Revelle, W. (2023). *psych: Procedures for psychological, psychometric, and personality research* (R package version 2.3.6). Northwestern University.
- Slowikowski, K. (2023). *ggrepel: Automatically position non-overlapping text labels with ggplot2* (R package version 0.9.3). <https://CRAN.R-project.org/package=ggrepel>
- Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- Venables, W. N., & Ripley, B. D. (2002). *Modern applied statistics with S* (4th ed.). Springer.
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.
- Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

End of Chapter 7. Proceed to Chapter 8: Data Clustering.

Chapter 8: Data Clustering

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(cluster)      # silhouette, pam
library(factoextra)   # cluster visualization
library(dbSCAN)       # DBSCAN clustering
library(mclust)       # Gaussian mixture models
library(dendextend)   # enhanced dendrograms
library(NbClust)      # multiple cluster validity indices
library(fpc)          # cluster statistics

# Davies-Bouldin index (centroid-based; matches the DBI formula in this chapter)
davies_bouldin <- function(x, cl) {
  x    <- as.matrix(x)
  ks   <- sort(unique(cl))
  K    <- length(ks)
  cent <- t(sapply(ks, function(c) colMeans(x[cl == c, , drop = FALSE])))
  S    <- sapply(seq_len(K), function(i) {
    pts <- x[cl == ks[i], , drop = FALSE]
    mean(sqrt(rowSums((pts - matrix(cent[i, ], nrow(pts), ncol(x), byrow = TRUE))^2)))
  })
  D <- as.matrix(dist(cent))
  mean(sapply(seq_len(K), function(i) max((S[i] + S[-i]) / D[i, -i])))
}
```

Chapter Overview

Introduction

Clustering is one of the most fundamental tasks in unsupervised machine learning: given a collection of observations with no predefined labels, find natural groupings — **clusters** — such that observations within a group are more similar to each other than to observations in other groups. Clustering drives customer segmentation, anomaly detection, document organization, gene expression analysis, image compression, and countless other applications.

Unlike classification (Chapter 9), clustering has no “correct” answer to compare against. Evaluating and interpreting clusters requires both statistical criteria and domain knowledge — making it as much an art as a science.

This chapter covers:

- **Introduction to Clustering** — concepts, types, and applications
- **K-Means Clustering** — the workhorse algorithm with formal optimization
- **Hierarchical Clustering** — agglomerative methods and dendrogram interpretation
- **DBSCAN** — density-based clustering for irregular shapes and noise
- **Cluster Validation** — internal and external validity measures
- **Gaussian Mixture Models** — probabilistic soft clustering with model selection
- **Practical Clustering Workflow** — end-to-end decision framework

Learning Objectives

By the end of this chapter, you will be able to:

1. Distinguish between types of clustering algorithms and select appropriately.
2. Implement and interpret K-means clustering including optimal k selection.
3. Construct and interpret hierarchical cluster dendrograms with different linkage methods.
4. Apply DBSCAN and tune its parameters for density-based clustering.
5. Evaluate clustering solutions using silhouette scores, Davies-Bouldin index, and other criteria.
6. Fit Gaussian mixture models and use BIC for model selection.
7. Apply a principled end-to-end clustering workflow to real data.

Introduction to Clustering

Introduction

Clustering is the task of partitioning a dataset into groups (clusters) such that observations within a cluster are more similar to each other than to observations in other clusters. It is an **unsupervised** method — no labels or outcome variables are used. The algorithm discovers structure entirely from the patterns in the feature space.

Clustering appears across virtually every domain of data science:

- **Marketing:** Customer segmentation for targeted campaigns
- **Genomics:** Grouping genes by expression profiles
- **Finance:** Portfolio construction based on asset correlation patterns
- **NLP:** Document clustering and topic discovery
- **Medicine:** Patient subtyping for precision medicine
- **Anomaly detection:** Normal behavior clusters; anomalies fall outside

Theory

Types of Clustering Algorithms

Table 1: Types of clustering algorithms

Type	Description	Examples
Partitional	Divides data into k non-overlapping clusters	K-Means, K-Medoids
Hierarchical	Builds a tree of nested clusters	Agglomerative, Divisive
Density-based	Clusters are dense regions separated by sparse regions	DBSCAN, OPTICS
Model-based	Assumes data generated from mixture of distributions	GMM
Fuzzy	Each point has partial membership in multiple clusters	Fuzzy C-Means

The Similarity/Distance Problem

All clustering algorithms depend on a measure of **similarity** or **distance** between observations. The most common choices:

Euclidean distance (most common for continuous data):

$$d(x, y) = \sqrt{\sum_{j=1}^p (x_j - y_j)^2}$$

Manhattan distance (robust to outliers):

$$d(x, y) = \sum_{j=1}^p |x_j - y_j|$$

Cosine similarity (for text and high-dimensional sparse data):

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

Gower distance (for mixed-type data):

$$d_G(x, y) = \frac{\sum_{j=1}^p w_j d_j(x_j, y_j)}{\sum_{j=1}^p w_j}$$

where d_j is adapted to variable type (range-normalized for continuous, 0/1 for binary, etc.).

Key Challenges

- **No ground truth:** Without labels, “correct” clustering cannot be verified directly.
- **Scale sensitivity:** Most algorithms require standardization — see Chapter 6.
- **Choosing k :** The number of clusters is rarely known in advance.
- **Cluster shape:** K-Means assumes spherical clusters; DBSCAN handles arbitrary shapes.
- **High dimensionality:** The curse of dimensionality (Chapter 7) affects distance-based clustering.
- **Interpretation:** Statistical clusters may not align with meaningful domain categories.

Example: When Do Clusters Exist?

Example 8.1. Three datasets each have 150 observations in 2D:

- **Dataset A:** Three well-separated spherical groups → K-Means works perfectly.
- **Dataset B:** Two concentric rings → K-Means fails (not spherical); DBSCAN succeeds.
- **Dataset C:** Uniformly distributed points → No meaningful clusters exist; any algorithm will produce arbitrary groupings.

This illustrates a critical principle: **clustering algorithms always find clusters** — even in random data. The challenge is determining whether the discovered clusters are real or artifacts.

R Example: Exploring Cluster Structure

```
# --- Visualize different cluster structures ---
set.seed(42)

# Dataset A: well-separated spherical clusters
n <- 150
dA <- data.frame(
  x = c(rnorm(50, 0, 0.5), rnorm(50, 3, 0.5),
        rnorm(50, 1.5, 0.5)),
  y = c(rnorm(50, 0, 0.5), rnorm(50, 0, 0.5),
        rnorm(50, 2.5, 0.5)),
  type = "A: Spherical"
)

# Dataset B: two rings (non-spherical)
theta1 <- runif(75, 0, 2*pi)
theta2 <- runif(75, 0, 2*pi)
dB <- data.frame(
  x = c(cos(theta1) * 1 + rnorm(75, 0, 0.08),
        cos(theta2) * 2.5 + rnorm(75, 0, 0.08)),
  y = c(sin(theta1) * 1 + rnorm(75, 0, 0.08),
        sin(theta2) * 2.5 + rnorm(75, 0, 0.08)),
  type = "B: Non-spherical"
)

# Dataset C: random noise
dC <- data.frame(
  x = runif(150, -3, 3),
```

```

y = runif(150, -3, 3),
type = "C: No structure"
)

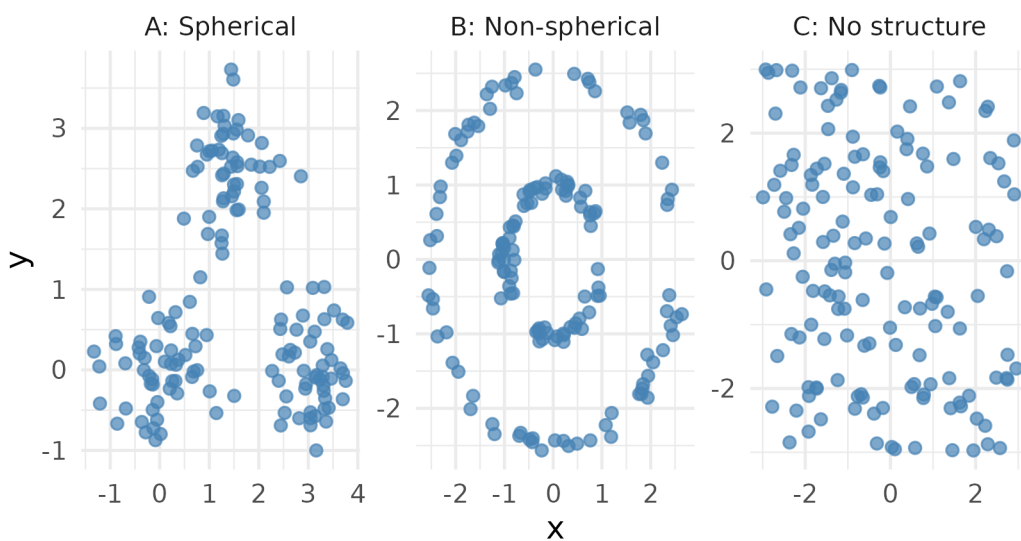
all_data <- bind_rows(dA, dB, dC)

ggplot(all_data, aes(x = x, y = y)) +
  geom_point(color = "steelblue", size = 1.8,
            alpha = 0.7) +
  facet_wrap(~type, scales = "free", ncol = 3) +
  labs(title = "Three Datasets with Different Cluster Structures",
       subtitle = "Algorithm choice must match the data geometry") +
  theme_minimal(base_size = 12)

```

Three Datasets with Different Cluster Structures

Algorithm choice must match the data geometry



Code explanation:

- Trigonometric functions `cos()` and `sin()` generate ring-shaped data — a classic test case for non-linear clustering methods.
- `facet_wrap(scales = "free")` allows each panel its own axis range, preventing one dataset's scale from distorting the others.
- Visual inspection is always the first step in clustering — before choosing an algorithm, look at the data (in 2D projections or PCA space).

Exercises

Exercise 8.1

- (a) Generate a dataset with 4 clusters of 50 points each, arranged in a 2×2 grid pattern (well-separated). Plot it.
 - (b) Generate a dataset where two clusters overlap substantially. What challenges does this pose for clustering?
 - (c) Apply `scale()` to the `iris` numeric variables and compute the Euclidean distance matrix using `dist()`. Inspect the first 6×6 block. Are setosa observations closer to each other than to versicolor?
-

K-Means Clustering

Introduction

K-Means is the most widely used clustering algorithm, valued for its simplicity, scalability, and intuitive interpretation. It partitions n observations into exactly k clusters by minimizing the total within-cluster variance — finding compact, spherical clusters in feature space.

Theory

The K-Means Objective

K-Means minimizes the **total within-cluster sum of squares (WCSS)**:

$$\text{WCSS} = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \mu_k\|^2$$

where μ_k is the centroid (mean) of cluster k and C_k is the set of observations assigned to cluster k .

This is an NP-hard optimization problem in general. The **Lloyd algorithm** finds a local optimum:

Algorithm:

1. Initialize K centroids (randomly or via K-Means++)
2. **Assignment step:** Assign each observation to the nearest centroid:

$$C_k = \{i : \|\mathbf{x}_i - \mu_k\|^2 \leq \|\mathbf{x}_i - \mu_j\|^2 \forall j\}$$

3. **Update step:** Recompute centroids as cluster means:

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$$

4. Repeat steps 2–3 until assignments do not change.

Convergence: The WCSS decreases at every iteration and is bounded below by 0, guaranteeing convergence. However, convergence to the **global** optimum is not guaranteed — run with multiple random starts (`nstart` in R).

K-Means++ Initialization

Random initialization can lead to poor solutions. **K-Means++** selects initial centroids with probability proportional to squared distance from existing centroids:

$$P(\mathbf{x}_i \text{ chosen}) \propto \min_k \|\mathbf{x}_i - \mu_k\|^2$$

This spreads initial centroids apart, dramatically improving solution quality and speed of convergence. K-Means++ is the default initializer in Python’s scikit-learn, but base R’s `kmeans()` does **not** use it — R initializes centroids by random selection. (The `algorithm = "Hartigan-Wong"` default controls the *iterative refinement* step, which is separate from initialization.) R’s safeguard against poor random starts is the `nstart` argument, which runs many random initializations and keeps the best.

Choosing K : The Elbow Method

Plot WCSS against K . The “elbow” — where WCSS begins to decrease slowly — suggests the optimal K . Beyond the elbow, adding clusters provides diminishing returns.

Choosing K : The Silhouette Method

The **silhouette score** for observation i measures how well it fits its assigned cluster relative to the next best cluster:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the mean distance to other points in the same cluster (cohesion) and $b(i)$ is the mean distance to points in the nearest other cluster (separation).

- $s(i) \approx 1$: well-clustered
- $s(i) \approx 0$: on the boundary between clusters
- $s(i) < 0$: possibly misassigned

The optimal K maximizes the **average silhouette width**.

Limitations of K-Means

- Assumes **spherical, equally-sized** clusters
- Sensitive to **outliers** (centroids are means)
- Requires specifying K in advance
- Non-deterministic — different runs may yield different solutions
- Fails on **non-convex** cluster shapes

Example: K-Means on Customer Data

Example 8.2. A retailer segments 200 customers by annual spending (THB) and visit frequency. K-Means with $K = 3$ identifies:

- **Cluster 1:** High spending, high frequency — “loyal premium customers”
- **Cluster 2:** Low spending, high frequency — “frequent budget shoppers”
- **Cluster 3:** High spending, low frequency — “occasional big spenders”

Each segment suggests a different marketing strategy — demonstrating how clustering translates to business action.

R Example: K-Means Clustering

```
# --- K-Means on iris (4 features, known ground truth) ---
data(iris)
iris_scaled <- scale(iris[, 1:4])

# Run K-Means with K=3, multiple starts
set.seed(42)
km_result <- kmeans(iris_scaled, centers = 3,
                    nstart = 25, iter.max = 100)

cat("K-Means Results (K=3):\n")
```

K-Means Results (K=3):

```
cat("Cluster sizes:", km_result$size, "\n")
```

Cluster sizes: 50 53 47


```
cat("Total WCSS:   ", round(km_result$tot.withinss, 3), "\n")
```

Total WCSS: 138.888

```
cat("Between-SS / Total SS:",  
    round(km_result$betweenss / km_result$totss * 100, 1),  
    "%\n\n")
```

Between-SS / Total SS: 76.7 %

```
# Compare clusters to true species  
table(Cluster = km_result$cluster,  
       Species = iris$Species)
```

	Species		
Cluster	setosa	versicolor	virginica
1	50	0	0
2	0	39	14
3	0	11	36

```
# --- Elbow and Silhouette plots for optimal K ---  
set.seed(42)  
k_range <- 2:8  
  
wcscs_vals <- sapply(k_range, function(k) {  
  kmeans(iris_scaled, centers = k,  
         nstart = 25)$tot.withinss  
})  
  
sil_vals <- sapply(k_range, function(k) {  
  km <- kmeans(iris_scaled, centers = k, nstart = 25)  
  sil <- silhouette(km$cluster, dist(iris_scaled))  
  mean(sil[, 3])  
})  
  
opt_df <- data.frame(k = k_range,  
                     wcscs = wcscs_vals,  
                     sil = sil_vals)  
  
p1 <- ggplot(opt_df, aes(x = k, y = wcscs)) +  
  geom_line(color = "steelblue", linewidth = 1.2) +  
  geom_point(size = 3, color = "steelblue") +
```

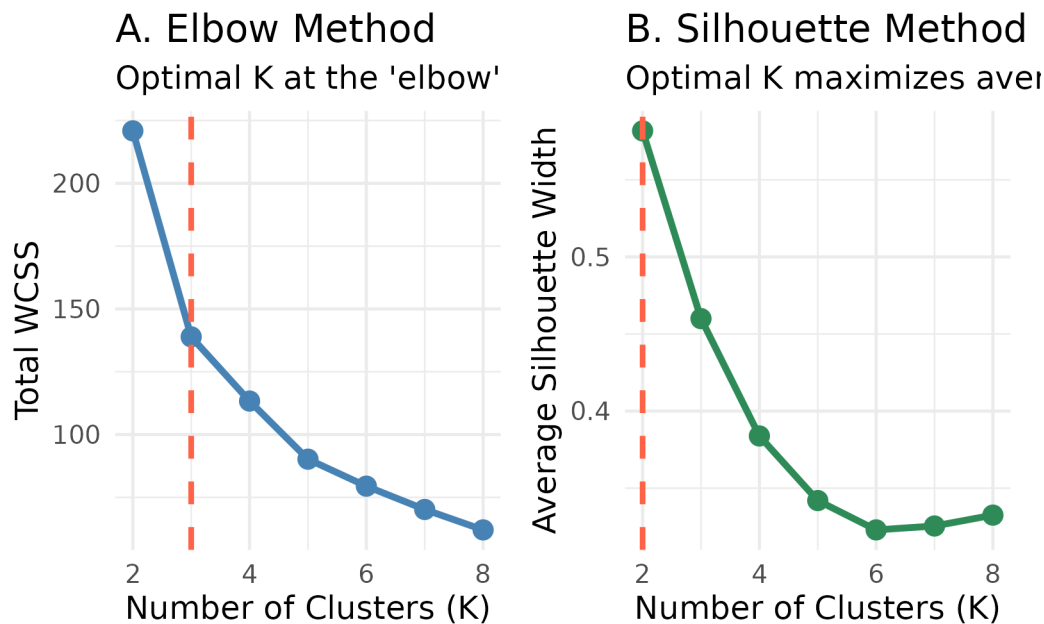
```

geom_vline(xintercept = 3, color = "tomato",
           linetype = "dashed", linewidth = 1) +
labs(title = "A. Elbow Method",
     subtitle = "Optimal K at the 'elbow'",
     x = "Number of Clusters (K)",
     y = "Total WCSS") +
theme_minimal(base_size = 12)

p2 <- ggplot(opt_df, aes(x = k, y = sil)) +
  geom_line(color = "seagreen", linewidth = 1.2) +
  geom_point(size = 3, color = "seagreen") +
  geom_vline(xintercept = k_range[which.max(sil_vals)],
            color = "tomato", linetype = "dashed",
            linewidth = 1) +
  labs(title = "B. Silhouette Method",
       subtitle = "Optimal K maximizes average silhouette",
       x = "Number of Clusters (K)",
       y = "Average Silhouette Width") +
  theme_minimal(base_size = 12)

p1 + p2

```



```

# --- Visualize clusters in PCA space ---
pca_km <- prcomp(iris_scaled)
km_plot <- data.frame(pca_km$x[, 1:2],

```

```

      Cluster = factor(km_result$cluster),
      Species = iris$Species)

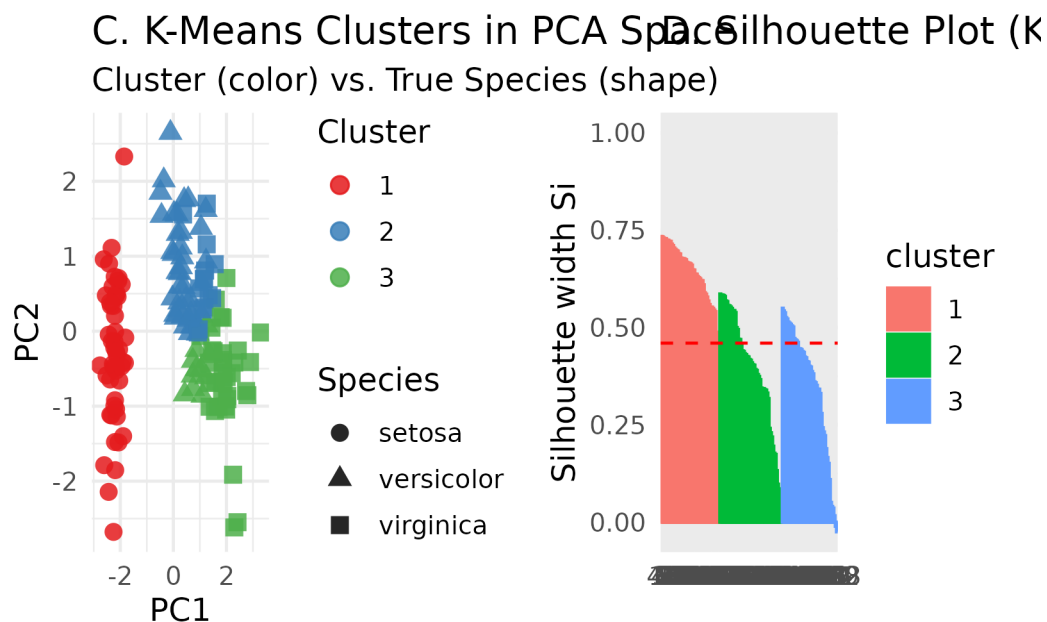
p3 <- ggplot(km_plot, aes(x = PC1, y = PC2,
                        color = Cluster,
                        shape = Species)) +
  geom_point(size = 2.8, alpha = 0.85) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "C. K-Means Clusters in PCA Space",
       subtitle = "Cluster (color) vs. True Species (shape)") +
  theme_minimal(base_size = 12)

# Silhouette plot
sil_km <- silhouette(km_result$cluster,
                    dist(iris_scaled))
p4 <- fviz_silhouette(sil_km) +
  labs(title = "D. Silhouette Plot (K=3)") +
  theme_minimal(base_size = 12)

```

	cluster	size	ave.sil.width
1	1	50	0.64
2	2	53	0.39
3	3	47	0.35

p3 + p4



Code explanation:

- `kmeans(x, centers, nstart, iter.max)`: `nstart = 25` runs 25 random initializations and keeps the best — essential for avoiding poor local minima.
- `silhouette(cluster, dist_matrix)` computes per-observation silhouette scores. `mean(sil[, 3])` extracts the average width.
- `fviz_silhouette()` produces a publication-ready silhouette plot with per-cluster averages highlighted.
- Plotting clusters in PCA space (Panel C) is the standard way to visualize high-dimensional clustering results in 2D.

Exercises

Exercise 8.2

Using the `USArrests` dataset:

- Standardize the data and apply K-Means for $K = 2, 3, 4, 5$.
- Use the elbow method and silhouette method to choose the optimal K .
- For the optimal K , identify which states belong to each cluster. Interpret the clusters using cluster centroids.
- Visualize clusters on a US map (use `ggplot2::map_data("state")`).

Exercise 8.3

K-Means is sensitive to initialization. For the `iris` data with $K = 3$:

- Run K-Means 10 times with `nstart = 1` and record the WCSS each time. How much does it vary?
- Run once with `nstart = 50`. Does this reliably find the best solution?
- Compare K-Means (Euclidean) to K-Medoids (`cluster::pam()`) on the same data. Which is more robust to the outlier you add: `iris[1, 1:4] <- c(10, 10, 10, 10)`?

Hierarchical Clustering

Introduction

Hierarchical clustering builds a tree-like structure of nested clusters — a **dendrogram** — that captures cluster relationships at every level of granularity simultaneously. Unlike K-Means, you do not need to specify K in advance; instead, you choose the number of clusters by “cutting” the dendrogram at a chosen height. This makes hierarchical clustering particularly valuable for **exploratory analysis** and for data where the cluster hierarchy is itself scientifically meaningful (e.g., evolutionary trees, organizational hierarchies).

Theory

Agglomerative Clustering (Bottom-Up)

The most common approach: start with each observation in its own cluster, then repeatedly merge the two most similar clusters until one cluster remains.

Algorithm:

1. Compute the $n \times n$ distance matrix $\mathbf{D}$.
2. Find the pair of clusters (C_i, C_j) with minimum inter-cluster distance.
3. Merge C_i and C_j into a new cluster C_{ij} .
4. Update $\mathbf{D}$ with distances from C_{ij} to all other clusters.
5. Repeat until one cluster remains.

Linkage Methods

The choice of **linkage** defines how the distance between two clusters is computed from pairwise distances:

Table 1: Linkage methods for hierarchical clustering

Linkage	Definition	Properties
Single	Min distance between any pair of points	Chaining effect; finds elongated clusters
Complete	Max distance between any pair of points	Compact, spherical clusters; sensitive to outliers
Average (UPGMA)	Mean of all pairwise distances	Compromise; most widely recommended
Ward's	Minimizes within-cluster variance increase	Most similar to K-Means; finds compact clusters
Centroid	Distance between cluster centroids	Can produce inversions in dendrogram

Ward's linkage is the most commonly recommended for general use as it produces compact, well-separated clusters and is consistent with the K-Means objective.

The Dendrogram

A **dendrogram** displays the hierarchical cluster structure:

- The **height** of each merge represents the distance at which two clusters joined.
- **Cutting** the dendrogram at height h produces clusters that were formed by merges at distances $\leq h$.
- The **cophenetic correlation** measures how faithfully the dendrogram represents the original pairwise distances — a diagnostic of how well the hierarchical structure fits the data.

Divisive Clustering (Top-Down)

DIANA (DIvisive ANAlysis): Starts with all observations in one cluster, then recursively splits the most heterogeneous cluster. An *exhaustive* search over all possible 2-way splits of a cluster is $O(2^n)$; in practice, implementations (e.g. `cluster::diana()`) use heuristics to avoid full enumeration, but divisive methods are still markedly more expensive than agglomerative ones and are rarely used on large datasets.

Example: Hierarchical Clustering of Countries

Example 8.3. The `USArrests` dataset is clustered hierarchically with Ward's linkage. Cutting at $K = 4$ reveals:

- **Cluster 1:** High crime states (e.g., Florida, California)
- **Cluster 2:** Mid-high crime (e.g., Arizona, Colorado)
- **Cluster 3:** Mid-low crime (e.g., Pennsylvania, Ohio)
- **Cluster 4:** Low crime states (e.g., Vermont, Maine)

The dendrogram shows that Clusters 3 and 4 merge before merging with Clusters 1 and 2 — low and mid-low crime states are more similar to each other than to high-crime states.

R Example: Hierarchical Clustering

```
# --- Hierarchical clustering on USArrests ---
data(USArrests)
arrests_scaled <- scale(USArrests)

# Compute distance matrix
dist_mat <- dist(arrests_scaled, method = "euclidean")

# Four linkage methods
linkages <- c("single", "complete", "average", "ward.D2")

hc_list <- lapply(linkages, function(lnk) {
  hclust(dist_mat, method = lnk)
})
names(hc_list) <- linkages

# Cophenetic correlation for each
coph_cor <- sapply(hc_list, function(hc) {
  round(cor(dist_mat, cophenetic(hc)), 4)
})

data.frame(Linkage = linkages,
           Cophenetic_r = coph_cor) |>
  kable(caption = "Cophenetic Correlation by Linkage Method",
        col.names = c("Linkage", "Cophenetic r")) |>
  kable_styling(bootstrap_options = c("striped", "hover"),
               full_width = FALSE) |>
  column_spec(2, bold = TRUE,
              color = ifelse(coph_cor == max(coph_cor),
                            "darkgreen", "steelblue"))
```

Table 2: Cophenetic Correlation by Linkage Method

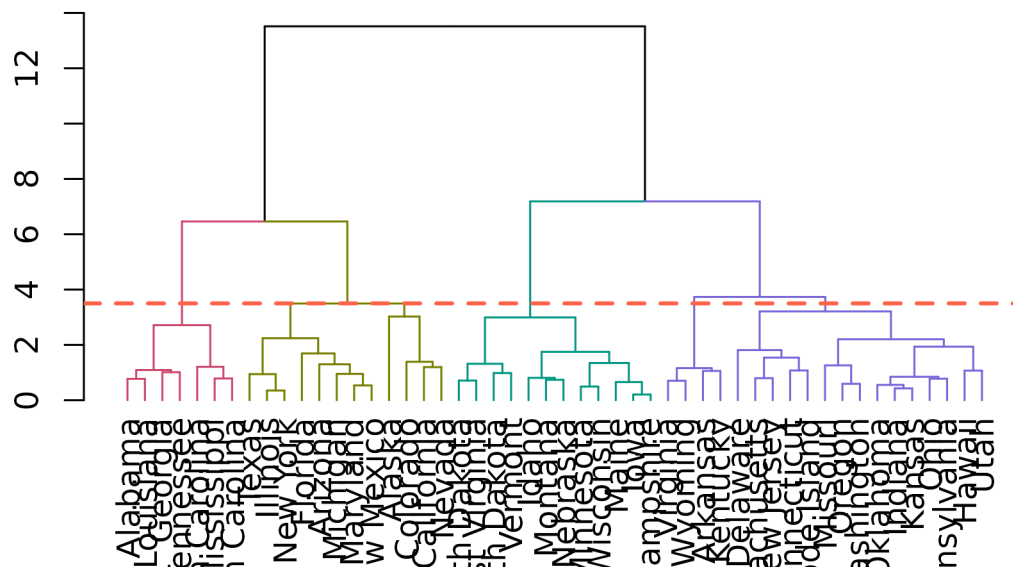
	Linkage	Cophenetic r
single	single	0.5413
complete	complete	0.6979
average	average	0.7180
ward.D2	ward.D2	0.6975

```
# --- Enhanced dendrogram (Ward's linkage) ---
hc_ward <- hc_list[["ward.D2"]]
dend    <- as.dendrogram(hc_ward)

# Color 4 clusters
dend_colored <- color_branches(dend, k = 4)

par(mar = c(4, 2, 3, 1))
plot(dend_colored,
     main = "Hierarchical Clustering: USArrests (Ward's Linkage)",
     ylab = "Height",
     cex  = 0.75)
abline(h = 3.5, col = "tomato",
       lty = 2, lwd = 2)
```

Hierarchical Clustering: USArrests (Ward's Linkage)




```

# --- Cut tree at K=4 and compare linkage methods ---
k_cut <- 4

clusters_comparison <- sapply(hc_list, function(hc) {
  cutree(hc, k = k_cut)
})

# Show cluster assignments for Ward's
ward_clusters <- cutree(hc_ward, k = k_cut)

cluster_summary <- USArrests |>
  mutate(Cluster = ward_clusters) |>
  group_by(Cluster) |>
  summarise(
    n = n(),
    Murder = round(mean(Murder), 2),
    Assault = round(mean(Assault), 2),
    UrbanPop = round(mean(UrbanPop), 2),
    Rape = round(mean(Rape), 2),
    .groups = "drop"
  )

kable(cluster_summary,
  caption = "Cluster Profiles: USArrests (Ward's, K=4)" |>
  kable_styling(bootstrap_options = c("striped", "hover"),
    full_width = FALSE)

```

Table 3: Cluster Profiles: USArrests (Ward's, K=4)

Cluster	n	Murder	Assault	UrbanPop	Rape
1	7	14.67	251.29	54.29	21.69
2	12	10.97	264.00	76.50	33.61
3	19	6.21	142.05	71.26	19.18
4	12	3.09	76.00	52.08	11.83

```

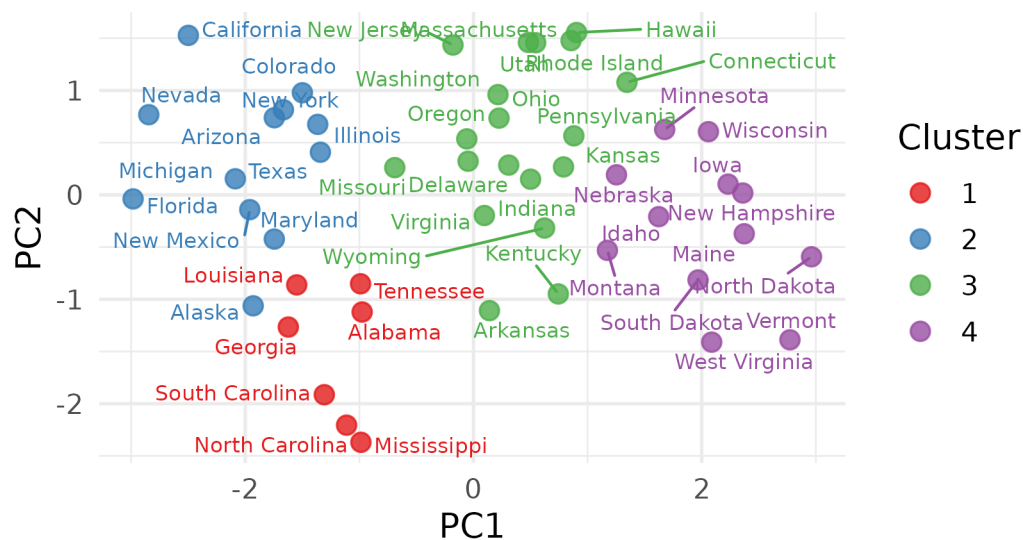
# --- Visualize clusters in PCA space ---
pca_arrests <- prcomp(arrests_scaled)
hc_plot_df <- data.frame(
  pca_arrests$x[, 1:2],
  Cluster = factor(ward_clusters),
  State = rownames(USArrests)
)

```

```
ggplot(hc_plot_df, aes(x = PC1, y = PC2,
                      color = Cluster,
                      label = State)) +
  geom_point(size = 3, alpha = 0.8) +
  ggrepel::geom_text_repel(size = 2.8,
                          show.legend = FALSE) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "Hierarchical Clusters in PCA Space",
       subtitle = "Ward's linkage, K=4 - USArrests dataset",
       x = "PC1", y = "PC2") +
  theme_minimal(base_size = 13)
```

Hierarchical Clusters in PCA Space

Ward's linkage, K=4 — USArrests dataset



Code explanation:

- `hclust(dist, method)` fits hierarchical clustering. `method = "ward.D2"` implements Ward's criterion using squared distances (the standard Ward implementation in R).
- `cophenetic(hc)` computes the cophenetic distance matrix; `cor(dist_mat, cophenetic(hc))` gives the cophenetic correlation — a model fit diagnostic.
- `color_branches(dend, k)` from `dendextend` colors the dendrogram branches by cluster assignment, making the cut point visually clear.
- `cutree(hc, k)` cuts the dendrogram to produce exactly k clusters — no need to specify height directly.

Exercises

Exercise 8.4

Using the `mtcars` dataset:

- (a) Compute the Euclidean distance matrix on standardized data.
 - (b) Apply hierarchical clustering with all four linkage methods. Compare dendrograms visually.
 - (c) Choose the linkage with the highest cophenetic correlation. Cut at $K = 3$ and interpret the clusters using group means.
 - (d) How do the hierarchical clusters compare to K-Means clusters ($K = 3$) from Exercise 8.2? Use a cross-tabulation.
-

DBSCAN

Introduction

K-Means and hierarchical clustering both struggle with clusters of **arbitrary shape** — elongated, curved, or interlocked — and with **noise** (outliers that do not belong to any cluster). **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**, introduced by Ester et al. (1996), solves these problems by defining clusters as dense regions of observations separated by sparse regions. It is particularly powerful for geospatial data, anomaly detection, and any domain where clusters are not compact spheres.

Theory

Core Concepts

DBSCAN has two parameters:

- ε (**eps**): The neighborhood radius. Two points are neighbors if their distance is $\leq \varepsilon$.
- **MinPts**: The minimum number of points required to form a dense region.

Three types of points:

1. **Core point**: Has at least MinPts neighbors within distance ε (including itself).
2. **Border point**: Within ε of a core point but has fewer than MinPts neighbors.
3. **Noise point (outlier)**: Not a core point and not within ε of any core point.

Density-reachability: Point q is density-reachable from core point p if there is a chain of core points $p = p_1, p_2, \dots, p_n = q$ where each p_{i+1} is within ε of p_i .

A **cluster** is a maximal set of density-connected points.

The DBSCAN Algorithm

1. For each unvisited point p :
 - If p is a core point: create a new cluster and expand it by adding all density-reachable points.
 - If p is a border or noise point: skip (may be assigned later when a core point reaches it).
2. Points never assigned to any cluster are labeled **noise** (typically coded as 0 or -1).

Complexity: $O(n \log n)$ with spatial indexing (k-d tree); $O(n^2)$ naively.

Advantages and Limitations

Table 1: DBSCAN vs. K-Means comparison

Feature	DBSCAN	K-Means
Handles arbitrary shapes		
Detects outliers/noise		
Number of clusters K	Automatic	Must specify
Sensitive to scale	Yes	Yes
Parameter tuning	ε , MinPts	K
Scales to large data	(with indexing)	

Choosing ε : The k -Distance Plot

Sort observations by their distance to the k -th nearest neighbor (typically $k = \text{MinPts} - 1$). Plot these sorted distances. The “elbow” in this plot suggests the optimal ε — at that distance, the density drops sharply from dense core regions to sparse noise.

Example: DBSCAN on Spatial Data

Example 8.4. GPS coordinates of 500 restaurant locations in a city are clustered with DBSCAN ($\varepsilon = 0.3$ km, MinPts = 5). The algorithm discovers:

- 4 natural **restaurant districts** of varying shapes (downtown, university area, shopping mall, tourist zone)
- 12 isolated restaurants labeled as **noise** (no nearby neighbors)

K-Means with $K = 4$ on the same data produces circular, equally-sized regions that cut across natural district boundaries — demonstrating DBSCAN’s advantage for geographic data.

R Example: DBSCAN

```
# --- DBSCAN on ring-shaped data ---
set.seed(42)
n <- 200
theta1 <- runif(n/2, 0, 2*pi)
theta2 <- runif(n/2, 0, 2*pi)

rings <- data.frame(
  x = c(cos(theta1)*1 + rnorm(n/2, 0, 0.08),
        cos(theta2)*2.5 + rnorm(n/2, 0, 0.08)),
  y = c(sin(theta1)*1 + rnorm(n/2, 0, 0.08),
        sin(theta2)*2.5 + rnorm(n/2, 0, 0.08)),
  true_cluster = rep(1:2, each = n/2)
)

# K-Means (will fail on rings)
set.seed(42)
km_rings <- kmeans(rings[,1:2], centers = 2, nstart = 25)

# DBSCAN
db_rings <- dbscan::dbscan(rings[,1:2],
                           eps = 0.3, minPts = 5)

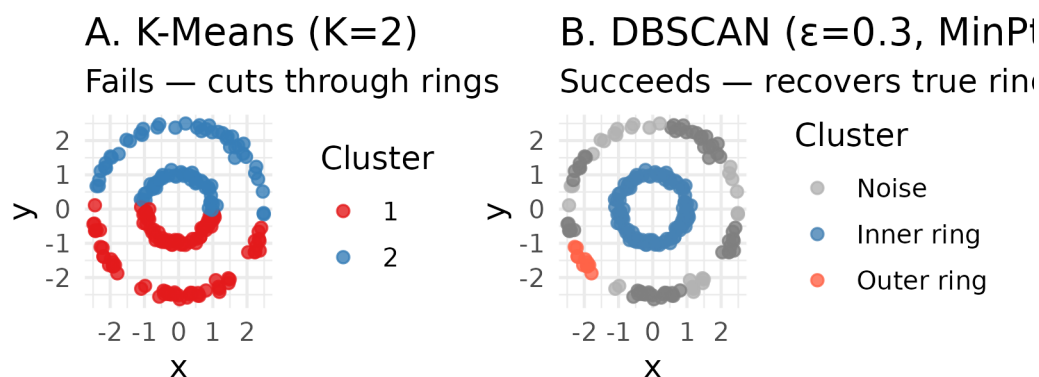
rings$km_cluster <- factor(km_rings$cluster)
rings$db_cluster <- factor(db_rings$cluster)

p1 <- ggplot(rings, aes(x, y, color = km_cluster)) +
  geom_point(size = 1.8, alpha = 0.8) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "A. K-Means (K=2)",
       subtitle = "Fails - cuts through rings",
       color = "Cluster") +
  theme_minimal(base_size = 12) +
  coord_equal()

p2 <- ggplot(rings, aes(x, y, color = db_cluster)) +
  geom_point(size = 1.8, alpha = 0.8) +
  scale_color_manual(
    values = c("0" = "gray70", "1" = "steelblue",
              "2" = "tomato"),
    labels = c("0" = "Noise", "1" = "Inner ring",
              "2" = "Outer ring")) +
```

```
labs(title = "B. DBSCAN ( $\epsilon=0.3$ , MinPts=5)",
     subtitle = "Succeeds - recovers true rings",
     color = "Cluster") +
theme_minimal(base_size = 12) +
coord_equal()
```

p1 + p2



```
# --- k-distance plot for epsilon selection ---
# Using iris data
iris_sc <- scale(iris[, 1:4])

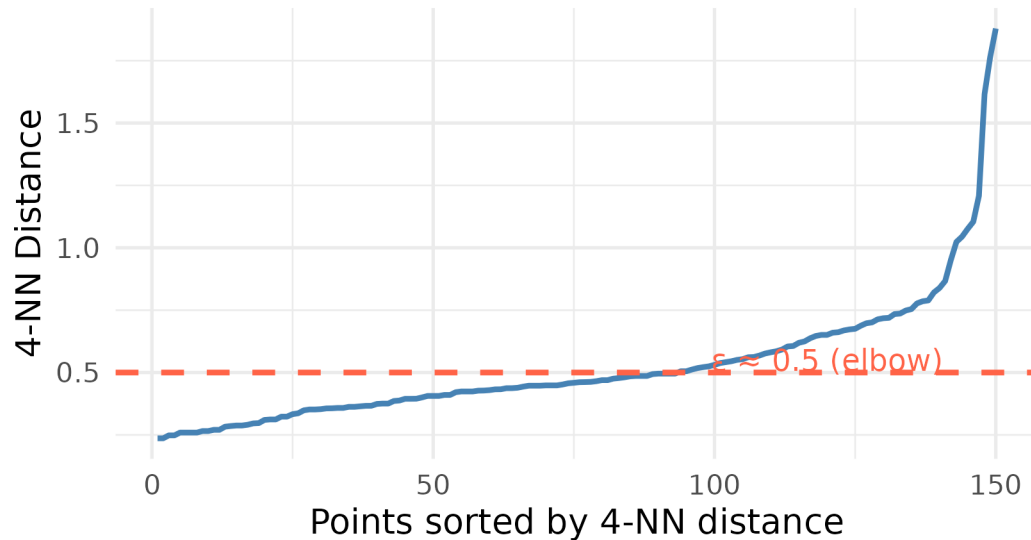
kNNdist_vals <- sort(dbSCAN::kNNdist(iris_sc, k = 4))
knn_df <- data.frame(
  idx = seq_along(kNNdist_vals),
  dist = kNNdist_vals
)

ggplot(knn_df, aes(x = idx, y = dist)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_hline(yintercept = 0.5, color = "tomato",
            linetype = "dashed", linewidth = 1) +
  annotate("text", x = 120, y = 0.55,
         label = "0.5 (elbow)",
         color = "tomato", size = 4) +
```

```
labs(title = "k-Distance Plot for Selection",
     subtitle = "iris dataset, k=4 | elbow suggests optimal ",
     x = "Points sorted by 4-NN distance",
     y = "4-NN Distance") +
theme_minimal(base_size = 13)
```

k-Distance Plot for ϵ Selection

iris dataset, k=4 | elbow suggests optimal ϵ



```
# --- Apply DBSCAN to iris ---
set.seed(42)
db_iris <- dbscan::dbscan(iris_sc, eps = 0.5, minPts = 5)

cat("DBSCAN Results (iris):\n")
```

DBSCAN Results (iris):

```
cat("Cluster sizes:\n")
```

Cluster sizes:

```
print(table(db_iris$cluster))
```

```
0  1  2
34 45 71
```



```
cat("Noise points (cluster 0):",
    sum(db_iris$cluster == 0), "\n\n")
```

Noise points (cluster 0): 34

```
# Compare to true species
table(DBSCAN = db_iris$cluster,
      Species = iris$Species)
```

	Species		
DBSCAN	setosa	versicolor	virginica
0	5	11	18
1	45	0	0
2	0	39	32

Code explanation:

- `dbscan::dbscan(x, eps, minPts)` runs DBSCAN. Cluster 0 represents noise points — always check how many observations are labeled as noise.
- `dbscan::kNNdist(x, k)` computes the k -nearest neighbor distance for each point; sorting and plotting these produces the k -distance plot.
- The ring example (Panels A and B) is the canonical demonstration of DBSCAN's advantage — K-Means cannot recover the correct structure while DBSCAN does so effortlessly.

Exercises

Exercise 8.5

Using the **faithful** dataset (Old Faithful geyser eruptions):

- Plot **eruptions** vs. **waiting**. How many clusters do you expect visually?
- Apply DBSCAN. Use the k -distance plot to choose ε .
- Compare DBSCAN to K-Means for this dataset. Which performs better?
- How does changing MinPts from 3 to 10 affect the clustering? How many noise points are created?

Exercise 8.6

Generate a dataset with 3 clusters of different densities and one cluster of outliers.

- Show that K-Means struggles (merges the sparse cluster with noise).

- (b) Show that DBSCAN handles it correctly with appropriate parameters.
 - (c) What are the trade-offs when choosing a very small vs. very large ε ?
-

Cluster Validation

Introduction

Any clustering algorithm will produce clusters — even on purely random data. **Cluster validation** provides formal, quantitative criteria for evaluating whether a clustering solution is meaningful, stable, and optimal. Without validation, we have no principled basis for choosing between algorithms, comparing solutions with different K , or deciding whether the clusters are real or artifacts.

Theory

Internal Validation Indices

Internal indices evaluate cluster quality using only the data and cluster assignments — no external labels required.

Silhouette Index (SI):

$$\bar{s} = \frac{1}{n} \sum_{i=1}^n s(i), \quad s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Range: $[-1, 1]$. Higher is better. Values above 0.5 indicate reasonable clustering; above 0.7 indicates strong clustering.

Davies-Bouldin Index (DBI):

$$\text{DBI} = \frac{1}{K} \sum_{k=1}^K \max_{j \neq k} \left(\frac{s_k + s_j}{d_{kj}} \right)$$

where s_k is the average distance within cluster k and d_{kj} is the distance between cluster centroids k and j . **Lower is better** (tighter clusters, farther apart).

Calinski-Harabasz Index (CHI) (also called Variance Ratio Criterion):

$$\text{CHI} = \frac{SS_B / (K - 1)}{SS_W / (n - K)}$$

where SS_B is between-cluster sum of squares and SS_W is within-cluster sum of squares. **Higher is better.**

Dunn Index:

$$DI = \frac{\min_{k \neq j} d(C_k, C_j)}{\max_k \text{diam}(C_k)}$$

Ratio of minimum inter-cluster distance to maximum cluster diameter. **Higher is better.**

External Validation Indices

When ground truth labels are available, external indices measure agreement between cluster assignments and true labels.

Adjusted Rand Index (ARI):

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$$

Range: $[-1, 1]$. $ARI = 1$ means perfect agreement; $ARI = 0$ means random (no better than chance); $ARI < 0$ means worse than random. Corrects for agreement due to chance.

Normalized Mutual Information (NMI):

$$NMI = \frac{2 \cdot I(C; T)}{H(C) + H(T)}$$

where $I(C; T)$ is mutual information between clusters and true labels, H is entropy. Range: $[0, 1]$. Higher is better. (Note: this “sum” normalization is one of several conventions in the literature — other common variants normalize by $\sqrt{H(C)H(T)}$ or $\max(H(C), H(T))$, which can give slightly different numeric values for the same clustering.)

Stability-Based Validation

Bootstrap the data multiple times, cluster each bootstrap sample, and measure how consistent the cluster assignments are. High variability across bootstrap samples suggests the solution is unstable and possibly spurious.

Example: Comparing Validation Indices

Example 8.5. For the `iris` dataset clustered with K-Means at $K = 2, 3, 4, 5$:

K	Silhouette	Davies-Bouldin	CHI	ARI (vs. species)
2	0.58	0.59	251	0.57
3	0.46	0.83	242	0.62
4	0.38	0.88	207	0.47
5	0.35	0.95	203	0.42

Here the internal and external indices disagree — which is itself an important lesson. **Internal** indices (Silhouette, Davies-Bouldin, CHI) all favour $K = 2$: Silhouette is highest, Davies-Bouldin is lowest, and CHI is largest at $K = 2$. This is because scaled iris splits most cleanly into two well-separated groups — *versicolor* and *virginica* overlap heavily and are hard to distinguish geometrically. The **external** index tells a different story: ARI peaks at $K = 3$, because there really are three species. Internal validity rewards geometric separation; external validity rewards alignment with ground truth. The two need not coincide, and iris is a canonical example of why.

R Example: Cluster Validation

```
# --- Comprehensive validation for iris K-Means ---
data(iris)
iris_sc <- scale(iris[, 1:4])
true_labels <- as.integer(iris$Species)

k_vals <- 2:6

validation_df <- map_dfr(k_vals, function(k) {
  set.seed(42)
  km <- kmeans(iris_sc, centers = k, nstart = 25)
  sil <- silhouette(km$cluster, dist(iris_sc))

  # CHI via fpc; Davies-Bouldin via davies_bouldin() helper (q=1, matches formula)
  cs <- fpc::cluster.stats(dist(iris_sc), km$cluster)

  data.frame(
    K = k,
    Silhouette = round(mean(sil[,3]), 4),
    Davies_Bouldin = round(davies_bouldin(iris_sc, km$cluster), 4),
    CHI = round(cs$ch, 2),
    ARI = round(mclust::adjustedRandIndex(
      km$cluster, true_labels), 4)
  )
})
```

```
kable(validation_df,
      caption = "Cluster Validation Indices: iris K-Means",
      col.names = c("K", "Silhouette ↑",
                    "Davies-Bouldin ↓", "CHI ↑", "ARI ↑")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
row_spec(which(validation_df$ARI ==
              max(validation_df$ARI)),
         bold = TRUE, background = "#EEF2FF")
```

Table 1: Cluster Validation Indices: iris K-Means

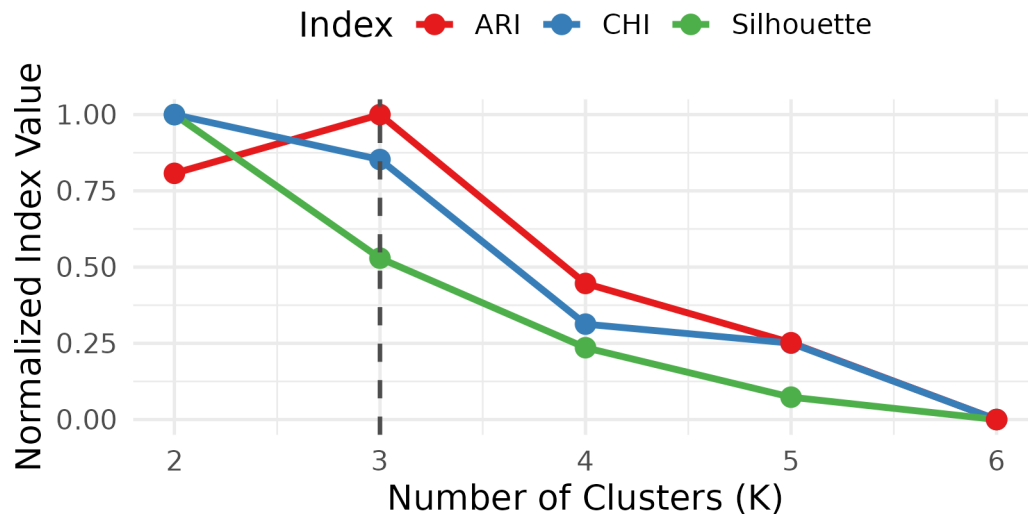
K	Silhouette ↑	Davies-Bouldin ↓	CHI ↑	ARI ↑
2	0.5818	0.5933	251.35	0.5681
3	0.4599	0.8336	241.90	0.6201
4	0.3839	0.8797	207.27	0.4706
5	0.3419	0.9530	203.27	0.4181
6	0.3229	1.0574	187.20	0.3501

```
# --- Validation index profiles ---
val_long <- validation_df |>
select(K, Silhouette, CHI, ARI) |>
pivot_longer(-K, names_to = "Index",
             values_to = "Value") |>
group_by(Index) |>
mutate(Value_norm = (Value - min(Value)) /
      (max(Value) - min(Value))) |>
ungroup()

ggplot(val_long, aes(x = K, y = Value_norm,
                    color = Index, group = Index)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +
  geom_vline(xintercept = 3, color = "gray30",
            linetype = "dashed", linewidth = 0.8) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "Normalized Validation Indices vs. K",
       subtitle = "Dashed line: K=3 (true number of species)",
       x = "Number of Clusters (K)",
       y = "Normalized Index Value",
       color = "Index") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")
```

Normalized Validation Indices vs. K

Dashed line: $K=3$ (true number of species)



Code explanation:

- `fpc::cluster.stats()` computes many cluster validity indices simultaneously — efficient for comparing multiple solutions.
- `mclust::adjustedRandIndex()` computes ARI between two cluster assignment vectors.
- Normalizing indices to $[0, 1]$ before plotting allows direct visual comparison of indices on different scales — the peak at $K = 3$ is immediately visible across all indices.

Exercises

💡 Exercise 8.7

Using the `USArrests` dataset (standardized):

- Compute Silhouette, Davies-Bouldin, and CHI for K-Means with $K = 2, \dots, 6$.
- Also compute these for hierarchical clustering (Ward's) with the same K values.
- Produce a side-by-side comparison plot of validation indices for both methods.
- Which method and K combination has the best overall validation profile?

Gaussian Mixture Models

Introduction

K-Means assigns each observation to exactly one cluster — **hard assignment**. But cluster membership is rarely this clear-cut: an observation near a cluster boundary plausibly belongs to multiple clusters with different degrees of certainty. **Gaussian Mixture Models (GMM)** provide a probabilistic generalization: each observation has a **probability of belonging to each cluster** — a **soft assignment**. GMMs also provide a principled statistical framework with formal model selection criteria.

Theory

The Mixture Model

A GMM assumes the data is generated from a mixture of K Gaussian distributions:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$

where: - π_k are **mixing proportions** ($\sum_k \pi_k = 1$, $\pi_k \geq 0$) - μ_k is the **mean** of component k - Σ_k is the **covariance matrix** of component k

The EM Algorithm

GMM parameters are estimated by the **Expectation-Maximization (EM)** algorithm:

E-step: Compute the **posterior probability** (responsibility) of each component for each observation:

$$r_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i \mid \mu_j, \Sigma_j)}$$

M-step: Update parameters using the responsibilities:

$$\pi_k^{\text{new}} = \frac{\sum_i r_{ik}}{n}, \quad \mu_k^{\text{new}} = \frac{\sum_i r_{ik} \mathbf{x}_i}{\sum_i r_{ik}}, \quad \Sigma_k^{\text{new}} = \frac{\sum_i r_{ik} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^T}{\sum_i r_{ik}}$$

EM converges to a local maximum of the log-likelihood. Run multiple times with different initializations.

Covariance Structure

The covariance matrix Σ_k can be constrained to reduce the number of parameters:

Table 1: Common GMM covariance models (mclust notation)

Model	Σ_k	Shape
EII (spherical, equal)	$\lambda \mathbf{I}$	Equal spheres
VII (spherical, variable)	$\lambda_k \mathbf{I}$	Different-size spheres
EEE (ellipsoidal, equal)	Σ	Equal ellipsoids
VVV (ellipsoidal, variable)	Σ_k	Different ellipsoids

Model Selection: BIC

The **Bayesian Information Criterion** selects the number of components K and covariance model:

$$\text{BIC} = -2\ell(\hat{\theta}) + p \log n$$

where $\ell(\hat{\theta})$ is the maximized log-likelihood and p is the number of parameters. **Higher BIC is better** in the `mclust` package convention (they maximize $-\text{BIC}$). BIC penalizes complexity, balancing fit against parsimony.

Example: GMM vs. K-Means

Example 8.6. Two overlapping Gaussian clusters are modeled:

- **K-Means** assigns hard boundaries — observations near the boundary are assigned to one cluster with certainty.
- **GMM** assigns probabilities — an observation on the boundary might have $P(\text{Cluster 1}) = 0.55$ and $P(\text{Cluster 2}) = 0.45$, honestly reflecting uncertainty.
- For well-separated clusters, GMM and K-Means give nearly identical results. For overlapping clusters, GMM is more honest and often more accurate.

R Example: Gaussian Mixture Models

```
# --- GMM on iris using mclust ---
data(iris)
iris_sc <- scale(iris[, 1:4])

# Fit GMM with automatic model and K selection via BIC
gmm_result <- mclust::Mclust(iris_sc, G = 1:6,
                             verbose = FALSE)

cat("=== Best GMM Model ===\n")
```

```
=== Best GMM Model ===
```

```
cat("Model type:          ", gmm_result$modelName, "\n")
```

```
Model type:          VV
```

```
cat("Number of components:", gmm_result$G, "\n")
```

```
Number of components: 2
```

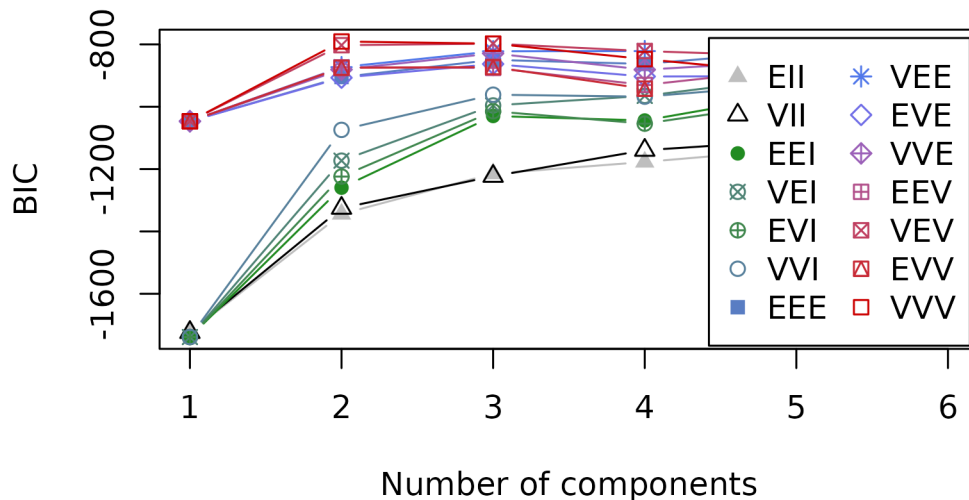
```
cat("BIC:                  ", round(gmm_result$bic, 2), "\n\n")
```

```
BIC:                  -790.7
```

```
# Classification table
table(GMM      = gmm_result$classification,
      Species = iris$Species)
```

```
Species
GMM setosa versicolor virginica
1    50          0          0
2     0         50         50
```

```
# --- BIC plot for model selection ---
plot(gmm_result, what = "BIC",
     main = "GMM BIC: Model and Component Selection")
```



```
# --- Soft assignments: observation uncertainty ---
uncertainty_df <- data.frame(
  pca_iris    <- prcomp(iris_sc)$x[, 1:2],
  Uncertainty = gmm_result$uncertainty,
  Cluster     = factor(gmm_result$classification),
  Species     = iris$Species
)

p1 <- ggplot(uncertainty_df,
  aes(x = PC1, y = PC2,
      color = Cluster,
```

```

        size = Uncertainty)) +
geom_point(alpha = 0.75) +
scale_color_brewer(palette = "Set1") +
scale_size_continuous(range = c(1, 5)) +
labs(title = "A. GMM Clusters with Uncertainty",
      subtitle = "Larger points = higher assignment uncertainty",
      x = "PC1", y = "PC2") +
theme_minimal(base_size = 12)

# Posterior probabilities for uncertain observations
top_uncertain <- order(gmm_result$uncertainty,
                      decreasing = TRUE)[1:10]
prob_df <- as.data.frame(gmm_result$z[top_uncertain,])
prob_df$Obs <- top_uncertain
prob_df$Species <- iris$Species[top_uncertain]
prob_df <- prob_df |>
  pivot_longer(cols = starts_with("V"),
               names_to = "Component",
               values_to = "Probability") |>
  mutate(Component = paste0("Cluster ",
                           str_remove(Component, "V")))

p2 <- ggplot(prob_df, aes(x = factor(Obs),
                          y = Probability,
                          fill = Component)) +

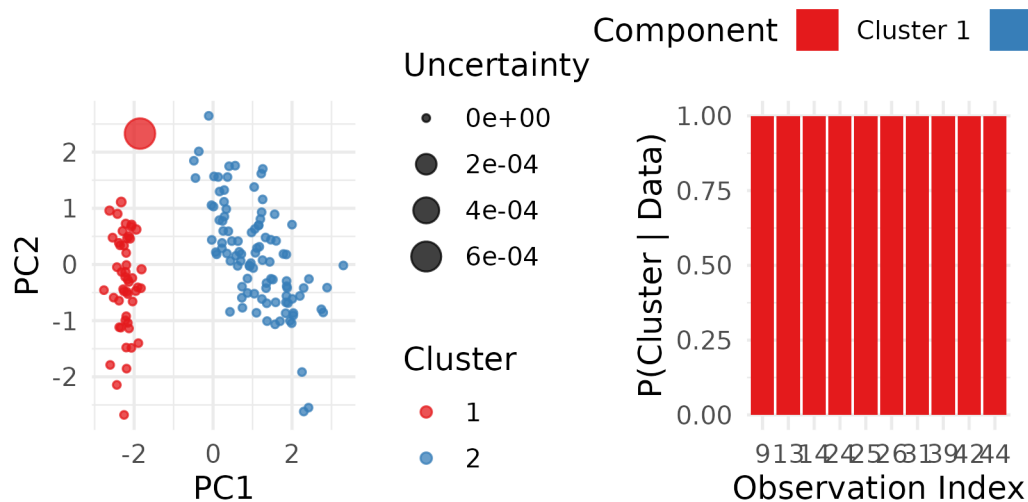
  geom_col() +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "B. Posterior Probabilities",
        subtitle = "10 most uncertain observations",
        x = "Observation Index",
        y = "P(Cluster | Data)") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top")

p1 + p2

```

A. GMM Clusters with Uncertainty B. Posterior Pro

Larger points = higher assignment uncertainty



Code explanation:

- `mclust::Mclust(x, G)` automatically searches over a range of K values and covariance models, selecting the best by BIC.
- `gmm_result$uncertainty` gives the classification uncertainty for each observation, defined as $1 - \max_k z_{ik}$ — one minus the probability of the *most likely* (winning) cluster. **Common misconception:** this is *not* the same as “the probability of the second-best cluster” except in the special case of exactly $K = 2$ components — with $K > 2$, the remaining probability mass $1 - \max_k z_{ik}$ is spread across all $K - 1$ other clusters, not concentrated in the runner-up.
- `gmm_result$z` is the $n \times K$ matrix of posterior probabilities — the soft assignments that distinguish GMM from K-Means.
- Sizing points by uncertainty (Panel A) immediately identifies which observations are most ambiguously assigned.

Exercises

💡 Exercise 8.8

Using the `faithful` dataset:

- Fit a GMM using `mclust::Mclust()`. Which model and K does BIC select?
- Plot the fitted density contours over a scatterplot of the data.
- Identify the 5 most uncertain observations. Where do they fall in the scatterplot?
- Compare GMM classification to K-Means ($K = 2$) on this dataset. Which per-

forms better?

Practical Clustering Workflow

Introduction

The preceding sections introduced individual clustering algorithms. In practice, clustering is an iterative, decision-rich process — there is no single correct approach. This section synthesizes the chapter into a **practical workflow** that guides the end-to-end clustering process from raw data to interpretable cluster profiles.

Theory

The End-to-End Clustering Workflow

1. Define the clustering objective
↓
2. Preprocess the data
 - Handle missing values (Chapter 6)
 - Remove or treat outliers
 - Scale features (Z-score recommended)
 - Reduce dimensions if $p \gg n$ (Chapter 7)↓
3. Choose an algorithm
 - Spherical clusters, known $K \rightarrow$ K-Means
 - Hierarchical structure important $\rightarrow$ Hierarchical
 - Arbitrary shapes, noise present $\rightarrow$ DBSCAN
 - Probabilistic membership needed $\rightarrow$ GMM
 - Very large data ($n > 100K$) $\rightarrow$ Mini-batch K-Means↓
4. Determine the number of clusters
 - Elbow method + Silhouette + BIC
 - Domain knowledge↓
5. Validate the solution
 - Internal: Silhouette, Davies-Bouldin, CHI
 - External (if labels available): ARI, NMI
 - Stability: bootstrap resampling

- ↓
6. Interpret and profile clusters
 - Cluster centroids / medoids
 - Statistical differences between clusters
 - Visualize in PCA / t-SNE space

↓

 7. Communicate and act
 - Name clusters meaningfully
 - Report actionable insights

Algorithm Selection Guide

Table 1: Clustering algorithm selection guide

Data Characteristics	Recommended Algorithm
Large n , spherical clusters	K-Means
Small n , need hierarchy	Hierarchical (Ward's)
Outliers, arbitrary shapes	DBSCAN
Overlapping clusters, uncertainty	GMM
Mixed data types	K-Medoids (PAM)
Very high dimensions	PCA first, then K-Means

Cluster Profiling

After assigning cluster labels, profile each cluster by computing:

- **Means/medians** of continuous variables per cluster
- **Proportions** of categorical variables per cluster
- **Statistical tests** for differences: ANOVA or Kruskal-Wallis for continuous; Chi-square for categorical
- **Effect sizes:** η^2 (ANOVA) or Cramér's V (chi-square)

R Example: Complete Clustering Workflow

```
# === COMPLETE WORKFLOW: mtcars ===
data(mtcars)

# Step 1: Preprocess
mt_scaled <- scale(mtcars[, c("mpg","hp","wt","disp","qsec")])
```



```
# Step 2: Determine K
set.seed(42)
k_range <- 2:7
sil_wf <- sapply(k_range, function(k) {
  km <- kmeans(mt_scaled, centers = k, nstart = 25)
  sil <- silhouette(km$cluster, dist(mt_scaled))
  mean(sil[, 3])
})
optimal_k <- k_range[which.max(sil_wf)]
cat("Optimal K (silhouette):", optimal_k, "\n")
```

Optimal K (silhouette): 2

```
# Step 3: Fit final model
set.seed(42)
km_final <- kmeans(mt_scaled, centers = optimal_k,
                  nstart = 50)

# Step 4: Validate
sil_final <- silhouette(km_final$cluster,
                      dist(mt_scaled))
cs_final <- fpc::cluster.stats(dist(mt_scaled),
                              km_final$cluster)

cat("Final solution validation:\n")
```

Final solution validation:

```
cat("  Average silhouette: ",
    round(mean(sil_final[,3]), 4), "\n")
```

Average silhouette: 0.4497

```
cat("  Davies-Bouldin:      ",
    round(davies_bouldin(mt_scaled, km_final$cluster), 4), "\n")
```

Davies-Bouldin: 0.8199

```
cat("  CHI:                  ",
    round(cs_final$ch, 2), "\n\n")
```

CHI: 38.34

```
# Step 5: Profile clusters
mtcars$Cluster <- factor(km_final$cluster)

profile <- mtcars |>
  group_by(Cluster) |>
  summarise(
    n      = n(),
    mpg    = round(mean(mpg), 1),
    hp     = round(mean(hp), 1),
    wt     = round(mean(wt), 2),
    disp   = round(mean(displacement), 1),
    qsec   = round(mean(qsec), 2),
    .groups = "drop"
  )

kable(profile,
  caption = paste0("Cluster Profiles: mtcars (K=",
    optimal_k, ")")) |>
  kable_styling(bootstrap_options = c("striped", "hover"),
    full_width = FALSE)
```

Table 2: Cluster Profiles: mtcars (K=2)

Cluster	n	mpg	hp	wt	disp	qsec
1	14	15.1	209.2	4.00	353.1	16.77
2	18	24.0	98.1	2.61	135.5	18.69

```
# Step 6: Visualize
pca_wf <- prcomp(mt_scaled)
wf_plot <- data.frame(
  pca_wf$x[,1:2],
  Cluster = factor(km_final$cluster),
  Car     = rownames(mtcars)
)

p1 <- ggplot(wf_plot, aes(x = PC1, y = PC2,
  color = Cluster,
  label = Car)) +
  geom_point(size = 3, alpha = 0.9) +
  ggrepel::geom_text_repel(size = 2.8,
    show.legend = FALSE) +
```

```

scale_color_brewer(palette = "Set1") +
labs(title = "A. Clusters in PCA Space",
      x = "PC1", y = "PC2") +
theme_minimal(base_size = 12)

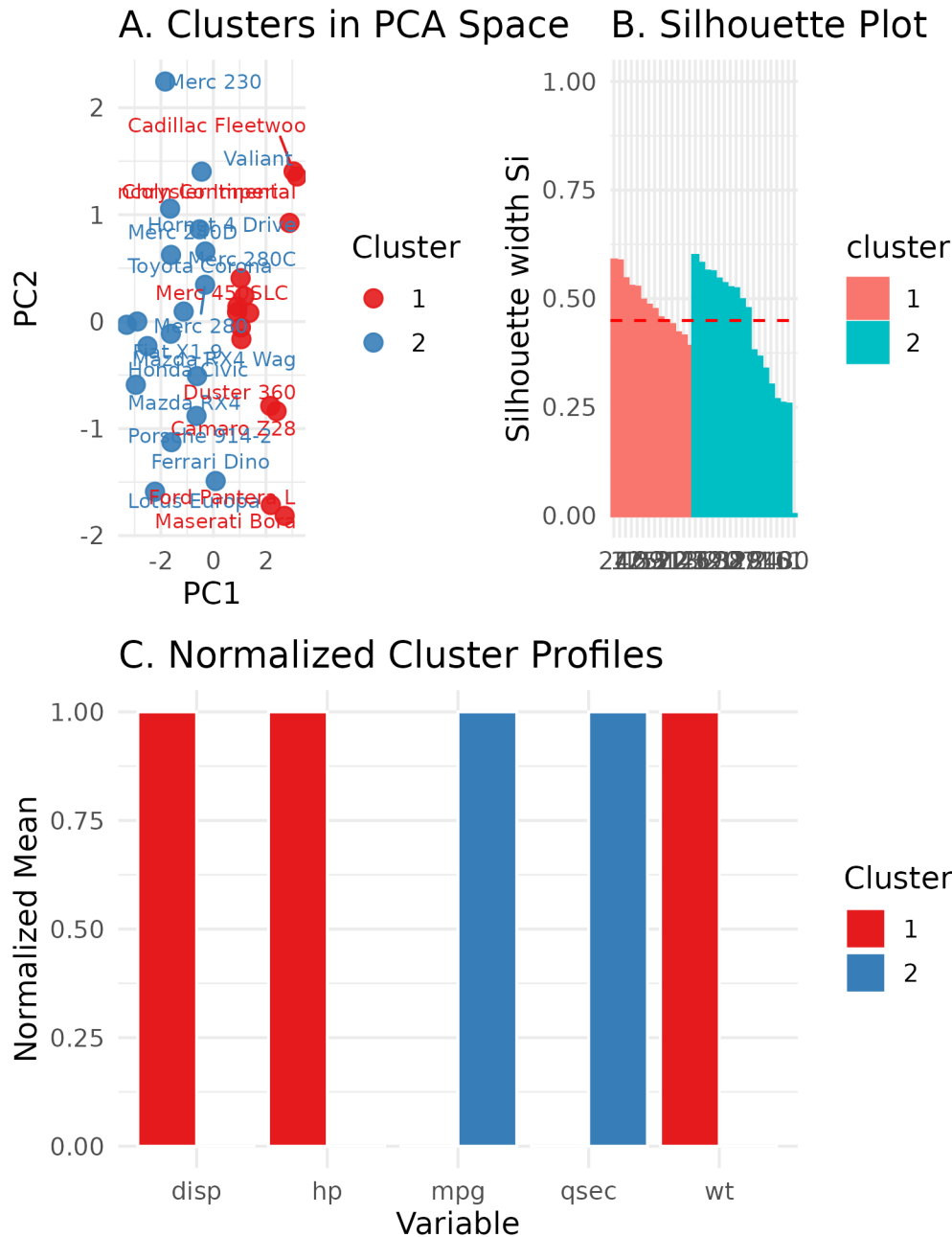
p2 <- fviz_silhouette(sil_final) +
labs(title = "B. Silhouette Plot") +
theme_minimal(base_size = 12)

  cluster size ave.sil.width
1         1   14         0.49
2         2   18         0.42

p3 <- profile |>
  pivot_longer(-c(Cluster, n),
               names_to = "Variable",
               values_to = "Mean") |>
  group_by(Variable) |>
  mutate(Mean_norm = (Mean - min(Mean)) /
         (max(Mean) - min(Mean))) |>
  ggplot(aes(x = Variable, y = Mean_norm,
             fill = Cluster)) +
  geom_col(position = "dodge", color = "white") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "C. Normalized Cluster Profiles",
       x = "Variable", y = "Normalized Mean") +
  theme_minimal(base_size = 12)

(p1 + p2) / p3

```



Code explanation:

- `nstart = 50` in the final K-Means fit ensures the best solution is found reliably.
- `group_by(Cluster) |> summarise(...)` profiles each cluster — this is the most important step for interpretation and business communication.
- The radar-chart-style normalized profile (Panel C) compares clusters across all variables simultaneously, making their distinctive characteristics immediately apparent.

Exercises

Exercise 8.9

Apply the full clustering workflow to the `wine` dataset (available via `rattle` or `pgmm`):

- (a) Preprocess: standardize all numeric variables.
 - (b) Apply K-Means, hierarchical (Ward's), and GMM. Use validation indices to choose K for each.
 - (c) Compare the three solutions using ARI against the true wine variety labels.
 - (d) Profile the best cluster solution. Write a 200-word interpretation describing each cluster.
-

Chapter Lab Activity: Customer Segmentation with wholesales Data

Objectives

In this lab you will apply the complete clustering workflow to a real-world customer segmentation problem using the `Wholesale customers` dataset. You will segment customers based on their annual spending across product categories, compare multiple algorithms, validate the solutions, and translate statistical clusters into actionable business insights.

The Dataset

```
# Simulate wholesale customer data
# (based on UCI Wholesale customers dataset structure)
set.seed(2024)
n_cust <- 440

wholesale <- data.frame(
  Channel = sample(c("Horeca", "Retail"), n_cust,
                  replace = TRUE, prob = c(0.7, 0.3)),
  Region  = sample(c("Lisbon", "Oporto", "Other"),
                  n_cust, replace = TRUE,
                  prob = c(0.3, 0.2, 0.5)),
  Fresh   = round(exp(rnorm(n_cust, 8.5, 1.0))),
  Milk    = round(exp(rnorm(n_cust, 8.0, 0.9))),
  Grocery = round(exp(rnorm(n_cust, 8.2, 0.9))),
  Frozen  = round(exp(rnorm(n_cust, 7.5, 1.1))),
  Detergents = round(exp(rnorm(n_cust, 6.8, 1.3))),
  Delicassen = round(exp(rnorm(n_cust, 7.2, 1.0)))
)

cat("Dataset dimensions:",
    nrow(wholesale), "x", ncol(wholesale), "\n\n")
```

Dataset dimensions: 440 × 8

```
summary(wholesale[, 3:8])
```

Fresh		Milk		Grocery		Frozen	
Min.	: 266	Min.	: 308	Min.	: 322	Min.	: 71.0
1st Qu.:	2437	1st Qu.:	1664	1st Qu.:	2010	1st Qu.:	941.5
Median :	4269	Median :	2930	Median :	3592	Median :	1866.5
Mean :	7784	Mean :	4407	Mean :	5528	Mean :	3280.7
3rd Qu.:	9236	3rd Qu.:	5325	3rd Qu.:	6677	3rd Qu.:	4117.8
Max.	:107044	Max.	:29563	Max.	:43554	Max.	:36176.0

Detergents		Delicassen	
Min.	: 18.0	Min.	: 63.0
1st Qu.:	395.8	1st Qu.:	648.8
Median :	991.0	Median :	1344.5
Mean :	2009.5	Mean :	2189.8
3rd Qu.:	2230.5	3rd Qu.:	2786.8
Max.	:44391.0	Max.	:24303.0

Lab Task 1: Preprocessing

```
# Log-transform (right-skewed expenditures)
ws_log <- wholesale |>
  select(Fresh:Delicassen) |>
  mutate(across(everything(), log))

# Standardize
ws_scaled <- scale(ws_log)

# Check skewness before/after
skew_before <- apply(wholesale[,3:8], 2,
  moments::skewness)
skew_after  <- apply(ws_log, 2, moments::skewness)

data.frame(
  Variable      = names(skew_before),
  Skew_Raw      = round(skew_before, 3),
  Skew_Log      = round(skew_after, 3)
) |>
  kable(caption = "Skewness: Raw vs. Log-Transformed",
    col.names = c("Variable", "Skewness (Raw)",
```

```

      "Skewness (Log)")) |>
kable_styling(bootstrap_options = c("striped","hover"),
              full_width = FALSE)

```

Table 1: Skewness: Raw vs. Log-Transformed

	Variable	Skewness (Raw)	Skewness (Log)
Fresh	Fresh	4.507	0.223
Milk	Milk	2.678	0.023
Grocery	Grocery	2.959	0.187
Frozen	Frozen	3.607	-0.080
Detergents	Detergents	5.925	0.016
Delicassen	Delicassen	3.237	-0.016

Lab Task 2: K-Means Segmentation

```

set.seed(42)
k_range_lab <- 2:7

lab_sil <- sapply(k_range_lab, function(k) {
  km <- kmeans(ws_scaled, centers = k, nstart = 25)
  sil <- silhouette(km$cluster, dist(ws_scaled))
  mean(sil[, 3])
})

lab_wcss <- sapply(k_range_lab, function(k) {
  kmeans(ws_scaled, centers = k, nstart = 25)$tot.withinss
})

opt_k_lab <- k_range_lab[which.max(lab_sil)]
cat("Optimal K:", opt_k_lab, "\n")

```

Optimal K: 7

```

# Fit final model
set.seed(42)
km_lab <- kmeans(ws_scaled, centers = opt_k_lab,
                 nstart = 50)

```



```

# Profile clusters
wholesale$Cluster <- factor(km_lab$cluster)

profile_lab <- wholesale |>
  group_by(Cluster) |>
  summarise(
    n          = n(),
    Fresh      = round(mean(Fresh), 0),
    Milk       = round(mean(Milk), 0),
    Grocery    = round(mean(Grocery), 0),
    Frozen     = round(mean(Frozen), 0),
    Detergents = round(mean(Detergents), 0),
    Delicassen = round(mean(Delicassen), 0),
    .groups    = "drop"
  )

kable(profile_lab,
  caption = paste0("Customer Segment Profiles (K=",
    opt_k_lab, ")"),
  format.args = list(big.mark = ",")) |>
  kable_styling(bootstrap_options = c("striped", "hover"),
    full_width = FALSE)

```

Table 2: Customer Segment Profiles (K=7)

Cluster	n	Fresh	Milk	Grocery	Frozen	Detergents	Delicassen
1	58	5,578	3,592	3,667	10,002	2,089	855
2	69	5,006	4,841	15,356	1,941	962	1,088
3	51	3,331	10,031	3,811	3,497	1,685	4,559
4	56	6,939	5,587	2,661	1,798	297	841
5	71	5,100	2,550	2,693	1,456	5,675	1,287
6	69	4,553	1,394	4,092	2,587	906	4,022
7	66	23,052	4,466	5,200	2,552	1,949	2,883

Lab Task 3: Hierarchical and GMM Comparison

```

# Hierarchical
dist_lab <- dist(ws_scaled)
hc_lab   <- hclust(dist_lab, method = "ward.D2")
hc_cuts  <- cutree(hc_lab, k = opt_k_lab)

```

```
# GMM
gmm_lab <- mclust::Mclust(ws_scaled,
                        G = 2:6, verbose = FALSE)

cat("GMM Best Model:", gmm_lab$modelName,
    "K =", gmm_lab$G, "\n\n")
```

GMM Best Model: EII K = 2

```
# ARI comparison (using hierarchical as reference)
ari_hc_km <- mclust::adjustedRandIndex(
  hc_cuts, km_lab$cluster)
ari_gmm_km <- mclust::adjustedRandIndex(
  gmm_lab$classification, km_lab$cluster)

data.frame(
  Comparison = c("K-Means vs. Hierarchical",
                 "K-Means vs. GMM"),
  ARI        = round(c(ari_hc_km, ari_gmm_km), 4)
) |>
kable(caption = "Agreement Between Clustering Methods (ARI)") |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE)
```

Table 3: Agreement Between Clustering Methods (ARI)

Comparison	ARI
K-Means vs. Hierarchical	0.274
K-Means vs. GMM	0.022

Lab Task 4: Visualization

```
pca_lab <- prcomp(ws_scaled)
lab_pca_df <- data.frame(
  pca_lab$x[, 1:2],
  KMeans = factor(km_lab$cluster),
  HC      = factor(hc_cuts),
  GMM     = factor(gmm_lab$classification)
)
```

```

p1 <- ggplot(lab_pca_df, aes(PC1, PC2,
                             color = KMeans)) +
  geom_point(size = 1.8, alpha = 0.7) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "A. K-Means Clusters") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "top")

p2 <- ggplot(lab_pca_df, aes(PC1, PC2,
                             color = HC)) +
  geom_point(size = 1.8, alpha = 0.7) +
  scale_color_brewer(palette = "Set2") +
  labs(title = "B. Hierarchical (Ward's)") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "top")

p3 <- ggplot(lab_pca_df, aes(PC1, PC2,
                             color = GMM)) +
  geom_point(size = 1.8, alpha = 0.7) +
  scale_color_brewer(palette = "Set3") +
  labs(title = "C. GMM Clusters") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "top")

p4 <- profile_lab |>
  pivot_longer(-c(Cluster, n),
               names_to = "Category",
               values_to = "Spend") |>
  group_by(Category) |>
  mutate(Spend_norm = Spend / max(Spend)) |>
  ggplot(aes(x = Category, y = Spend_norm,
             fill = Cluster)) +
  geom_col(position = "dodge", color = "white") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "D. Normalized Spending Profiles",
       x = "Product Category",
       y = "Normalized Mean Spend") +
  theme_minimal(base_size = 11) +
  theme(axis.text.x = element_text(angle = 30,
                                    hjust = 1))

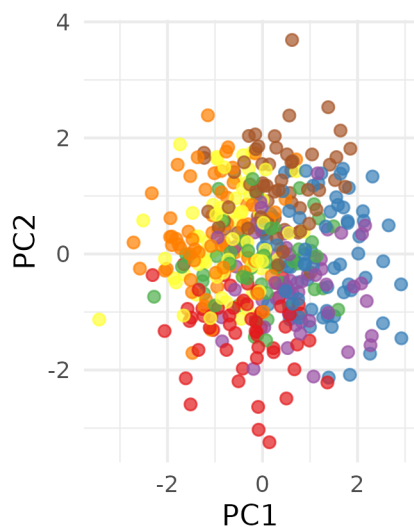
(p1 + p2) / (p3 + p4)

```

A. K-Means Clusters

KMeans

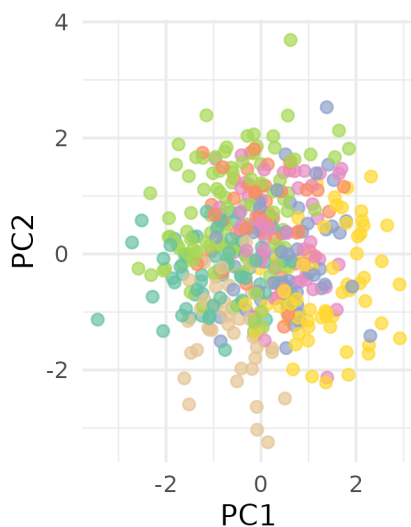
1	3	5	7
2	4	6	



B. Hierarchical (Ward's)

HC

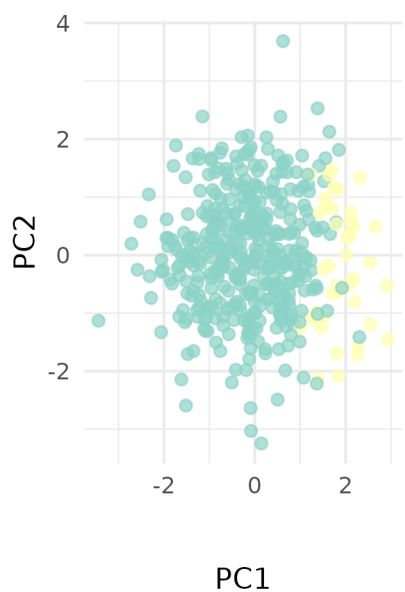
1	3	5	7
2	4	6	



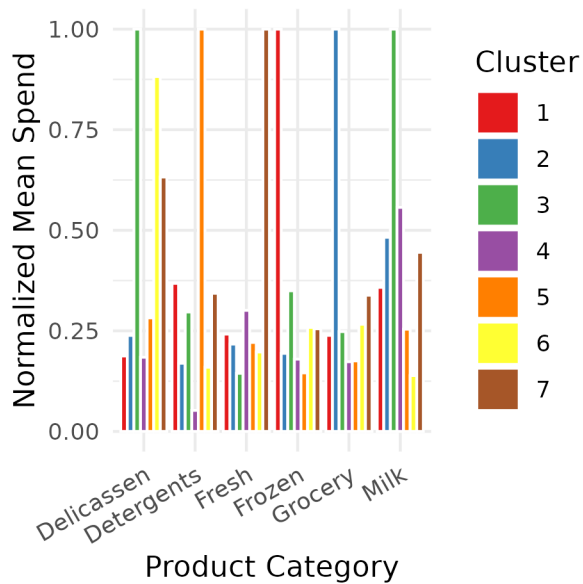
C. GMM Clusters

GMM

1	2
---	---



D. Normalized Spending Profi



Lab Discussion Questions

Answer the following in writing:

1. **Algorithm Choice:** For customer segmentation, why might K-Means be preferred over DBSCAN? Under what business scenario would DBSCAN be more appropriate?
 2. **Preprocessing Impact:** In Lab Task 1, you log-transformed the spending variables before clustering. What would happen to the cluster solution if you clustered on the raw (untransformed) data? Would the same customers be grouped together?
 3. **Agreement Between Methods:** The ARI between K-Means and hierarchical clustering measures their agreement. If ARI is high, does this mean both methods found the “true” clusters? What else could explain high agreement?
 4. **Business Interpretation:** Based on the cluster profiles in Lab Task 2, assign a descriptive business name to each customer segment (e.g., “High-volume fresh food buyers”). What marketing strategy would you recommend for each segment?
 5. **Validation Limitations:** Internal validation indices like Silhouette and CHI were used to choose K . What are the limitations of relying solely on these indices? What additional information would make you more confident in the chosen K ?
-

Chapter Summary

Summary

This chapter built a comprehensive clustering toolkit spanning the full spectrum of approaches:

- **Introduction** — clustering is unsupervised grouping; algorithm choice must match data geometry; distance measures and scaling are foundational decisions.
- **K-Means** — minimizes WCSS; uses an iterative refinement algorithm (R's default is Hartigan-Wong); good initialization such as K-Means++ (scikit-learn's default) improves results; elbow and silhouette methods guide K selection; assumes spherical, equal-size clusters.
- **Hierarchical clustering** — builds a dendrogram of nested clusters; Ward's linkage recommended for most applications; cophenetic correlation diagnoses fit; `cutree()` extracts any K .
- **DBSCAN** — density-based; detects arbitrary shapes and noise; requires tuning ε and MinPts via k -distance plot; more robust than K-Means for spatial and irregular data.
- **Cluster validation** — Silhouette, Davies-Bouldin, and CHI for internal validation; ARI and NMI for external; stability analysis for robustness.
- **Gaussian Mixture Models** — soft probabilistic assignments; EM algorithm; BIC selects model and K ; uncertainty quantification distinguishes GMM from hard methods.
- **Practical workflow** — preprocessing $\rightarrow$ algorithm selection $\rightarrow$ K determination $\rightarrow$ validation $\rightarrow$ profiling $\rightarrow$ communication; no single algorithm is universally best.

! Key Formulas to Know

K-Means Objective:

$$\text{WCSS} = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \mu_k\|^2$$

Silhouette Score:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Davies-Bouldin Index:

$$\text{DBI} = \frac{1}{K} \sum_{k=1}^K \max_{j \neq k} \frac{s_k + s_j}{d_{kj}}$$

GMM Density:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$

Adjusted Rand Index:

$$\text{ARI} = \frac{\text{RI} - E[\text{RI}]}{\max(\text{RI}) - E[\text{RI}]}$$

BIC:

$$\text{BIC} = -2\ell(\hat{\theta}) + p \log n$$

References

- Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), 224–227.
- Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1), 1–38.
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining* (pp. 226–231). AAAI Press.
- Galili, T. (2015). dendextend: An R package for visualizing, adjusting, and comparing trees of hierarchical clustering. *Bioinformatics*, 31(22), 3718–3720.
- Hahsler, M., Piekenbrock, M., & Doran, D. (2019). dbscan: Fast density-based clustering with R. *Journal of Statistical Software*, 91(1), 1–30.
- Hartigan, J. A., & Wong, M. A. (1979). Algorithm AS 136: A K-means clustering algorithm. *Journal of the Royal Statistical Society: Series C*, 28(1), 100–108.
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Hennig, C. (2023). *fpc: Flexible procedures for clustering* (R package version 2.2-10). <https://CRAN.R-project.org/package=fpc>
- Kassambara, A. (2023). *factoextra: Extract and visualize the results of multivariate data analyses* (R package version 1.0.7). <https://CRAN.R-project.org/package=factoextra>

- Maechler, M., Rousseeuw, P., Struyf, A., Hubert, M., & Hornik, K. (2023). *cluster: Cluster analysis basics and extensions* (R package version 2.1.4). <https://CRAN.R-project.org/package=cluster>
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- Schwarz, G. (1978). Estimating the dimension of a model. *Annals of Statistics*, 6(2), 461–464.
- Scrucca, L., Fop, M., Murphy, T. B., & Raftery, A. E. (2016). mclust 5: Clustering, classification and density estimation using Gaussian finite mixture models. *The R Journal*, 8(1), 289–317.
- Slowikowski, K. (2023). *ggrepel: Automatically position non-overlapping text labels with ggplot2* (R package version 0.9.3). <https://CRAN.R-project.org/package=ggrepel>
- Ward, J. H., Jr. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301), 236–244.
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.
- Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

End of Chapter 8. Proceed to Chapter 9: Data Classification.

Chapter 9: Data Classification

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(caret)           # unified ML framework
library(rpart)           # decision trees
library(rpart.plot)      # tree visualization
library(randomForest)    # random forest
library(pROC)            # ROC curves
library(class)           # KNN
library(MASS)            # datasets
```

Chapter Overview

Introduction

Classification is the task of predicting which category — class, label, group — a new observation belongs to, based on its features. It is the most widely applied form of supervised machine learning: spam vs. legitimate email, disease vs. healthy, fraud vs. genuine transaction, high-risk vs. low-risk loan applicant. Every classification problem shares a common structure: a labeled training set, a set of features, a learning algorithm, and a performance evaluation framework.

This chapter covers:

- **Introduction to Classification** — supervised learning, decision boundaries, problem types
- **Logistic Regression** — the statistical foundation of binary classification
- **K-Nearest Neighbors** — the simplest non-parametric classifier
- **Decision Trees** — interpretable rule-based classification
- **Random Forest and Ensemble Methods** — combining classifiers for superior performance
- **Model Evaluation** — confusion matrix, precision, recall, F1, AUC-ROC
- **Model Comparison and Selection** — cross-validation, bias-variance, choosing the best model

Learning Objectives

By the end of this chapter, you will be able to:

1. Distinguish classification from clustering and regression; frame a problem correctly.
2. Fit and interpret logistic regression including odds ratios and ROC curves.
3. Implement KNN classification and choose the optimal K.
4. Build, prune, and interpret decision trees.
5. Train random forests and extract variable importance measures.
6. Evaluate classifiers using appropriate metrics beyond accuracy.
7. Compare multiple classifiers using cross-validation and select the best model.

Introduction to Classification

Introduction

In **supervised learning**, we have a labeled dataset: n observations (x_i, y_i) where $x_i \in \mathbb{R}^p$ is a feature vector and $y_i \in \{1, 2, \dots, K\}$ is a class label. The goal is to learn a function $f: \mathbb{R}^p \rightarrow \{1, \dots, K\}$ that predicts the class of new, unseen observations.

Classification is fundamentally different from clustering (Chapter 8) — we have labels and we use them. It is also different from regression (Chapter 10) — the outcome is categorical, not continuous.

Theory

Types of Classification Problems

Table 1: Types of classification problems

Type	Output	Examples
Binary	2 classes	Spam/not spam, disease/healthy
Multiclass	$K > 2$ classes	Digit recognition, species ID
Multilabel	Multiple labels per observation	Image tagging, document categorization
Imbalanced	Very unequal class frequencies	Fraud detection, rare disease

The Decision Boundary

A classifier partitions the feature space into regions, one per class. The **decision boundary** is the surface separating these regions. Its shape depends on the algorithm:

- **Linear classifiers** (logistic regression, LDA): flat hyperplane boundary
- **KNN**: locally adaptive, potentially complex boundary
- **Decision trees**: axis-aligned rectangular regions
- **Random forests**: complex non-linear boundaries
- **SVM**: maximum-margin hyperplane (possibly non-linear via kernel)

The Training-Test Split

A fundamental principle: **never evaluate a model on the data used to train it**. The training set is used to fit the model; the **test set** (held out before training) provides an unbiased estimate of generalization performance.

$$\text{Dataset} \xrightarrow{\text{split}} \begin{cases} \text{Training set (70-80\%)} & \text{fit model} \\ \text{Test set (20-30\%)} & \text{evaluate model} \end{cases}$$

In practice, a **validation set** or **cross-validation** is used for hyperparameter tuning, with the test set reserved for final evaluation only.

i A Note on This Chapter's Example Dataset

The `Pima.tr` / `Pima.te` split shipped with the `MASS` package is **200 training / 332 test observations — 37.6% train, 62.4% test**, the reverse of the 70-80%/20-30% guideline above. This is a historical artifact of how the data was originally partitioned (Ripley, *Pattern Recognition and Neural Networks*, 1996), not a split you would choose deliberately. We use it throughout this chapter because it is a convenient, well-known built-in dataset, but **do not treat this split as a template** — for your own projects, create the partition yourself (e.g., with `caret::createDataPartition()`) using the 70-80%/20-30% guideline.

The Bayes Error Rate

The **Bayes error rate** is the lowest achievable error rate for any classifier:

$$\text{Bayes error} = 1 - E \left[\max_k P(Y = k \mid X) \right]$$

It represents irreducible error — the overlap between class distributions in feature space. No model can achieve error below the Bayes rate. Real classifiers aim to approach, but not exceed, this bound.

Example: Classification Problem Setup

Example 9.1. A bank wants to predict whether a loan applicant will default (1) or not (0) based on income, credit score, loan amount, and employment length.

- **Problem type:** Binary classification
- **Features:** $p = 4$ (income, credit score, loan amount, employment)
- **Training set:** Historical loans with known outcomes

- **Evaluation metric:** Not accuracy alone — the cost of a false negative (predicting no default when the applicant will default) is much higher than a false positive.

This asymmetric cost structure requires careful metric choice — a recurring theme throughout this chapter.

R Example: Setting Up a Classification Problem

```
# --- Use the Pima Indians diabetes dataset ---
# Available in MASS package
data(Pima.tr, package = "MASS")
data(Pima.te, package = "MASS")

# Combine and inspect
pima <- bind_rows(
  Pima.tr |> mutate(set = "train"),
  Pima.te |> mutate(set = "test")
)

cat("Dataset dimensions:", nrow(pima), "x",
    ncol(pima) - 1, "\n")
```

Dataset dimensions: 532 × 8

```
cat("Class distribution:\n")
```

Class distribution:

```
print(table(pima$type))
```

```
No Yes
355 177
```

```
cat("Class balance:",
    round(mean(pima$type == "Yes") * 100, 1),
    "% positive (diabetic)\n\n")
```

Class balance: 33.3 % positive (diabetic)

```
# Inspect features
glimpse(pima)
```

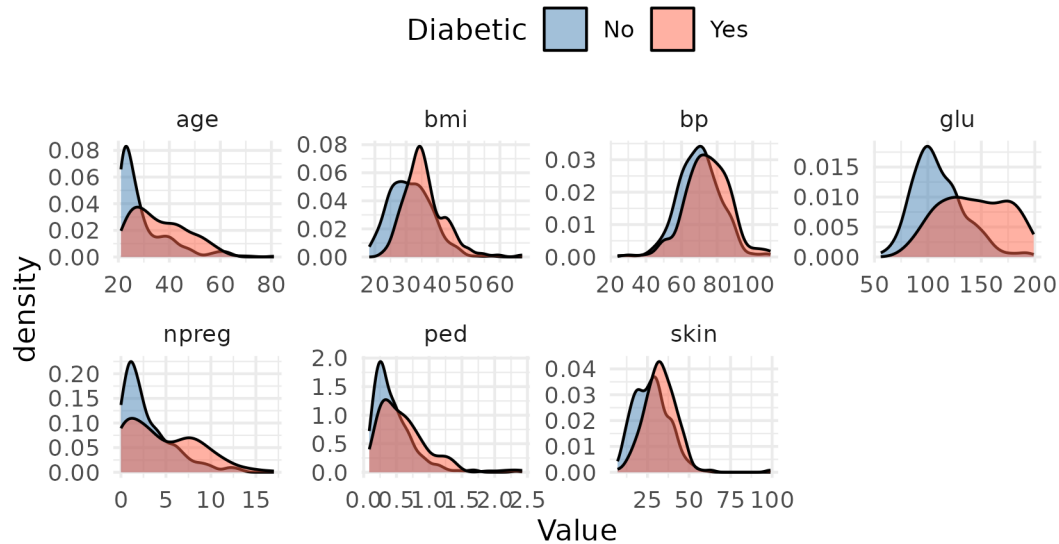
```
Rows: 532
Columns: 9
$ npreg <int> 5, 7, 5, 0, 0, 5, 3, 1, 3, 2, 0, 9, 1, 12, 1, 4, 1, 11, 1, 0, 2, ~
$ glu   <int> 86, 195, 77, 165, 107, 97, 83, 193, 142, 128, 137, 154, 189, 92, ~
$ bp    <int> 68, 70, 82, 76, 60, 76, 58, 50, 80, 78, 40, 78, 60, 62, 66, 76, ~
$ skin  <int> 28, 33, 41, 43, 25, 27, 31, 16, 15, 37, 35, 30, 23, 7, 52, 15, 8~
$ bmi   <dbl> 30.2, 25.1, 35.8, 47.9, 26.4, 35.6, 34.3, 25.9, 32.4, 43.3, 43.1~
$ ped   <dbl> 0.364, 0.163, 0.156, 0.259, 0.133, 0.378, 0.336, 0.655, 0.200, 1~
$ age   <int> 24, 55, 35, 26, 23, 52, 25, 24, 63, 31, 33, 45, 59, 44, 29, 21, ~
$ type  <fct> No, Yes, No, No, No, Yes, No, No, No, Yes, Yes, No, Yes, Yes, No~
$ set   <chr> "train", "train", "train", "train", "train", "train", "train", "~
```

```
# --- Visualize class separation ---
pima_long <- pima |>
  dplyr::select(-set) |>
  pivot_longer(-type,
               names_to = "Feature",
               values_to = "Value")

ggplot(pima_long, aes(x = Value, fill = type)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("No" = "steelblue",
                              "Yes" = "tomato")) +
  facet_wrap(~Feature, scales = "free", ncol = 4) +
  labs(title = "Feature Distributions by Class",
       subtitle = "Pima Indians Diabetes Dataset",
       fill = "Diabetic") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "top")
```

Feature Distributions by Class

Pima Indians Diabetes Dataset



Code explanation:

- `bind_rows()` with a `set` identifier preserves train/test membership while enabling combined visualization — always maintain this split discipline.
- `pivot_longer()` reshapes to long format for `facet_wrap()` — the cleanest way to compare feature distributions across all variables simultaneously.
- Visual separation (density overlap) gives a first impression of which features will be most useful for classification.

Exercises

💡 Exercise 9.1

Using the `iris` dataset reformulated as a binary classification problem (setosa vs. non-setosa):

- Create a binary outcome variable `is_setosa`.
- Split into 75% training and 25% test sets using `caret::createDataPartition()`.
- Visualize feature distributions by class. Which two features appear most discriminating?
- What is the class balance? If you built a “majority class” classifier, what accuracy would it achieve?

Logistic Regression

Introduction

Logistic regression is the natural extension of linear regression to binary classification. Rather than predicting a continuous outcome, it models the **probability** that an observation belongs to the positive class. Despite its name, logistic regression is a classification algorithm — it outputs probabilities that are converted to class predictions by applying a threshold (typically 0.5). It remains one of the most widely used and interpretable classifiers in statistics and data science.

Theory

The Logistic Model

For a binary outcome $Y \in \{0, 1\}$, logistic regression models:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p) = \frac{1}{1 + e^{-(\beta_0 + \beta^T \mathbf{x})}}$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the **sigmoid (logistic) function**, which maps any real number to $(0, 1)$.

The Log-Odds (Logit) Interpretation

Taking the log of the odds ratio:

$$\log \frac{P(Y = 1)}{P(Y = 0)} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

The **log-odds** (logit) is linear in the features. This gives a clear interpretation:

- e^{β_j} is the **odds ratio** for a one-unit increase in x_j , holding all other variables constant.
- $e^{\beta_j} > 1$: increasing x_j increases the odds of $Y = 1$.
- $e^{\beta_j} < 1$: increasing x_j decreases the odds of $Y = 1$.

- $e^{\beta_j} = 1$ (i.e., $\beta_j = 0$): x_j has no effect.

Parameter Estimation: Maximum Likelihood

Coefficients are estimated by **maximum likelihood** — finding β that maximizes the probability of observing the training labels:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

This is the **binary cross-entropy loss** (negative log-likelihood). It is convex — gradient descent finds the global optimum.

Hypothesis Testing for Coefficients

Each coefficient β_j is tested with:

$$z = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)} \sim N(0, 1) \text{ under } H_0 : \beta_j = 0$$

The **likelihood ratio test** compares the full model to a reduced model and follows a χ^2 distribution.

Multinomial Logistic Regression

For $K > 2$ classes, the model extends to **multinomial (softmax) regression**:

$$P(Y = k \mid \mathbf{x}) = \frac{e^{\beta_k^T \mathbf{x}}}{\sum_{j=1}^K e^{\beta_j^T \mathbf{x}}}$$

One class is set as reference; $K - 1$ sets of coefficients are estimated.

Regularized Logistic Regression

With many features (or features that are correlated), unregularized MLE coefficients can become unstable or overfit. **Regularization** adds a penalty on coefficient size to the loss being minimized:

$$\text{Ridge (L2): } \ell(\beta) - \lambda \sum_{j=1}^p \beta_j^2 \quad \text{Lasso (L1): } \ell(\beta) - \lambda \sum_{j=1}^p |\beta_j|$$

Ridge shrinks all coefficients toward zero without eliminating any; lasso can shrink coefficients exactly to zero, performing automatic feature selection. The penalty strength λ is chosen by cross-validation. In R, `glmnet::glmnet(x, y, family = "binomial", alpha = 0)` fits ridge (`alpha = 1` for lasso, values in between for elastic net). For a data science project with dozens or hundreds of candidate predictors, regularized logistic regression is usually preferable to the plain `glm()` fit shown below.

Example: Logistic Regression for Diabetes Prediction

Example 9.2. Using the Pima dataset, a logistic regression predicts diabetes from glucose level, BMI, age, and diabetes pedigree function (matching the model fit in the R example below). Coefficient for glucose: $\hat{\beta} = 0.031$, $e^{\hat{\beta}} = 1.032$.

Interpretation: For each 1 mg/dL increase in glucose concentration, the odds of diabetes increase by 3.2%, holding BMI, age, and pedigree function constant. This is statistically significant ($z = 4.72$, $p < 0.001$).

R Example: Logistic Regression

```
# --- Logistic regression on Pima dataset ---
data(Pima.tr, package = "MASS")
data(Pima.te, package = "MASS")

# Convert outcome to binary
Pima.tr$diabetes <- ifelse(Pima.tr$type == "Yes", 1, 0)
Pima.te$diabetes <- ifelse(Pima.te$type == "Yes", 1, 0)

# Fit logistic regression
lr_model <- glm(diabetes ~ glu + bmi + age + ped,
               data = Pima.tr,
               family = binomial(link = "logit"))
```

```
summary(lr_model)
```

Call:

```
glm(formula = diabetes ~ glu + bmi + age + ped, family = binomial(link = "logit"),  
     data = Pima.tr)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-9.971388	1.527587	-6.528	6.69e-11	***
glu	0.031255	0.006627	4.716	2.40e-06	***
bmi	0.077030	0.032251	2.388	0.016921	*
age	0.058603	0.017574	3.335	0.000854	***
ped	1.719794	0.656088	2.621	0.008760	**

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 256.41 on 199 degrees of freedom
Residual deviance: 181.08 on 195 degrees of freedom
AIC: 191.08

Number of Fisher Scoring iterations: 5

```
# --- Odds ratios with confidence intervals ---  
or_df <- data.frame(  
  Variable = names(coef(lr_model))[-1],  
  OR        = round(exp(coef(lr_model))[-1], 4),  
  CI_lower  = round(exp(confint(lr_model))[-1, 1], 4),  
  CI_upper  = round(exp(confint(lr_model))[-1, 2], 4),  
  p_value   = round(summary(lr_model)$coefficients[-1, 4], 4)  
)  
  
kable(or_df,  
      caption = "Odds Ratios: Logistic Regression",  
      col.names = c("Variable", "OR", "95% CI Lower",  
                    "95% CI Upper", "p-value")) |>  
  kable_styling(bootstrap_options = c("striped", "hover"),  
                full_width = FALSE) |>  
  column_spec(2, bold = TRUE,  
              color = ifelse(or_df$OR > 1,  
                             "tomato", "steelblue"))
```

Table 1: Odds Ratios: Logistic Regression

	Variable	OR	95% CI Lower	95% CI Upper	p-value
glu	glu	1.0317	1.0190	1.0460	0.0000
bmi	bmi	1.0801	1.0154	1.1531	0.0169
age	age	1.0604	1.0252	1.0988	0.0009
ped	ped	5.5834	1.5873	20.9615	0.0088

```
# --- Predictions and classification ---
lr_probs <- predict(lr_model, Pima.te,
                    type = "response")
lr_class <- ifelse(lr_probs >= 0.5, 1, 0)

# Confusion matrix
conf_lr <- table(Predicted = lr_class,
                 Actual     = Pima.te$diabetes)
acc_lr  <- mean(lr_class == Pima.te$diabetes)

cat("Accuracy:", round(acc_lr * 100, 1), "%\n\n")
```

Accuracy: 79.2 %

```
kable(conf_lr,
      caption = "Confusion Matrix: Logistic Regression") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
               full_width = FALSE)
```

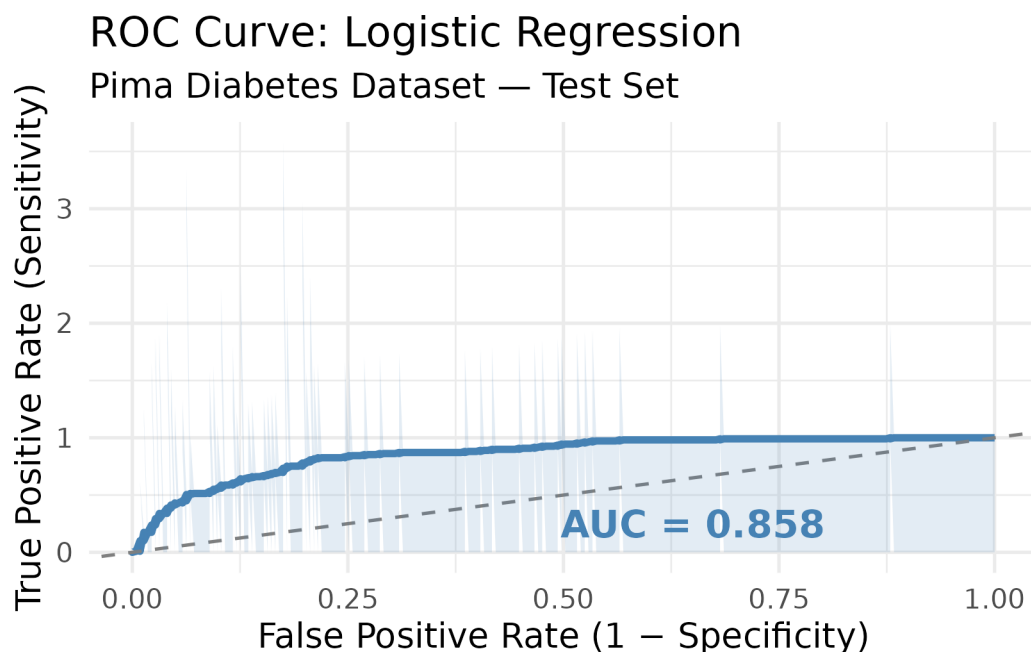
Table 2: Confusion Matrix: Logistic Regression

	0	1
0	196	42
1	27	67

```
# --- ROC curve ---
roc_lr <- pROC::roc(Pima.te$diabetes, lr_probs,
                   quiet = TRUE)

roc_df <- data.frame(
  FPR = 1 - roc_lr$specificities,
  TPR = roc_lr$sensitivities
)
```

```
ggplot(roc_df, aes(x = FPR, y = TPR)) +
  geom_line(color = "steelblue", linewidth = 1.3) +
  geom_abline(slope = 1, intercept = 0,
              linetype = "dashed", color = "gray50") +
  geom_area(fill = "steelblue", alpha = 0.15) +
  annotate("text", x = 0.65, y = 0.25,
          label = paste0("AUC = ",
                          round(pROC::auc(roc_lr), 3)),
          size = 5, fontface = "bold",
          color = "steelblue") +
  labs(title = "ROC Curve: Logistic Regression",
       subtitle = "Pima Diabetes Dataset - Test Set",
       x = "False Positive Rate (1 - Specificity)",
       y = "True Positive Rate (Sensitivity)") +
  theme_minimal(base_size = 13)
```



Code explanation:

- `glm(formula, family = binomial)` fits logistic regression. `family = binomial(link = "logit")` specifies the logit link function.
- `exp(coef(lr_model))` converts log-odds coefficients to odds ratios; `exp(confint())` gives the CIs on the OR scale.
- `pROC::roc()` computes the ROC curve from predicted probabilities and true labels. `pROC::auc()` extracts the area under the curve.
- The shaded area under the ROC curve visually represents the AUC — the probability that the model ranks a random positive higher than a random negative.

Exercises

Exercise 9.2

Using the full Pima dataset:

- (a) Fit logistic regression with all 7 predictors. Which are significant at $\alpha = 0.05$?
 - (b) Interpret the odds ratio for `glu`. How does it change when you control for all other variables?
 - (c) Try different classification thresholds (0.3, 0.4, 0.5, 0.6). How do precision and recall change?
 - (d) Fit a multinomial logistic regression on the `iris` dataset using `nnet::multinom()`. Report the AIC.
-

K-Nearest Neighbors

Introduction

K-Nearest Neighbors (KNN) is the simplest non-parametric classifier. It makes no assumptions about the form of the decision boundary — instead, it classifies each new observation based on the majority class among its K nearest neighbors in the training set. KNN is intuitive, easy to implement, and can capture complex non-linear boundaries. Its main limitations are computational cost at prediction time (must compute distances to all training points) and sensitivity to irrelevant features and scale.

Theory

The KNN Algorithm

Training: Simply store all training observations. There is no explicit model fitting.

Prediction for a new point x_0 : 1. Compute the distance from x_0 to every training point. 2. Identify the K nearest neighbors $\mathcal{N}(x_0)$. 3. Predict the most common class among the K neighbors:

$$\hat{y} = \arg \max_k \sum_{i \in \mathcal{N}(x_0)} \mathbf{1}(y_i = k)$$

For probability estimates, use the proportion of class k among the K neighbors.

Choosing K

- **Small K (e.g., $K = 1$):** Very flexible decision boundary; low bias, high variance; overfits to noise.
- **Large K (e.g., $K = n$):** Smooth boundary; high bias, low variance; predicts majority class always.
- **Optimal K :** Found by cross-validation — minimizes validation error.

The Bias-Variance Tradeoff in KNN

As K increases from 1 to n :

$$\text{Bias} \uparrow, \quad \text{Variance} \downarrow$$

The optimal K minimizes the total error = Bias² + Variance.

KNN Requirements

1. **Scale:** All features must be standardized — a feature with large range dominates Euclidean distances.
2. **Dimensionality:** KNN degrades in high dimensions (curse of dimensionality, Chapter 7) — consider PCA preprocessing.
3. **Computational cost:** Prediction is $O(np)$ per query — slow for large training sets. Use k-d trees or ball trees for speed.

Weighted KNN

Give closer neighbors more influence by weighting votes by inverse distance:

$$w_i = \frac{1}{d(x_0, x_i)^2}$$

This reduces the influence of distant neighbors and often improves performance near decision boundaries.

Example: KNN Decision Boundary

Example 9.3. For the Pima dataset with $K = 1$: perfect training accuracy (100%, each point is its own nearest neighbor), but weaker test accuracy (69.9%) — a classic overfitting signature. With $K = 21$: a smoother boundary generalizes better (79.5% test accuracy). Five-fold cross-validation on the training set (using the *same* folds for every candidate K , as in the corrected R code below) identifies $K = 23$ as optimal (CV accuracy 75%), achieving 77.7% test accuracy. Note that the CV-accuracy curve is fairly flat for K between roughly 15 and 29 (all within about 1 percentage point) — with only $n = 200$ training observations, the “optimal” K is not sharply defined, and a different random fold split could easily favor a neighboring value of K . In practice, any K in that flat region is a reasonable, defensible choice.

R Example: KNN Classification

```
# --- KNN on Pima dataset ---
# Standardize features
train_x <- scale(Pima.tr[, c("glu","bmi","age","ped")])
test_x  <- scale(Pima.te[, c("glu","bmi","age","ped")],
                 center = attr(train_x,"scaled:center"),
                 scale  = attr(train_x,"scaled:scale"))
train_y <- Pima.tr$type
test_y  <- Pima.te$type

# Tune K via cross-validation
set.seed(42)
# IMPORTANT: generate the folds ONCE and reuse them for every K.
# Redrawing folds inside the loop (a common mistake) means each K is
# evaluated on a different random split, adding noise that has nothing
# to do with K itself and can shift which K looks "best".
folds <- createFolds(train_y, k = 5)
k_vals <- seq(1, 31, by = 2)
cv_acc <- sapply(k_vals, function(k) {
  fold_acc <- sapply(folds, function(idx) {
    knn_pred <- class::knn(train = train_x[-idx, ],
                          test  = train_x[idx, ],
                          cl    = train_y[-idx],
                          k     = k)

    mean(knn_pred == train_y[idx])
  })
  mean(fold_acc)
})

optimal_k_knn <- k_vals[which.max(cv_acc)]
cat("Optimal K:", optimal_k_knn, "\n")
```

Optimal K: 23

```
# --- Final KNN model ---
knn_pred <- class::knn(train = train_x,
                      test  = test_x,
                      cl    = train_y,
                      k     = optimal_k_knn,
                      prob  = TRUE)
```

```
knn_acc <- mean(knn_pred == test_y)
cat("KNN Test Accuracy (K=",
    optimal_k_knn, "):",
    round(knn_acc * 100, 1), "%\n")
```

KNN Test Accuracy (K= 23): 77.7 %

```
table(Predicted = knn_pred, Actual = test_y)
```

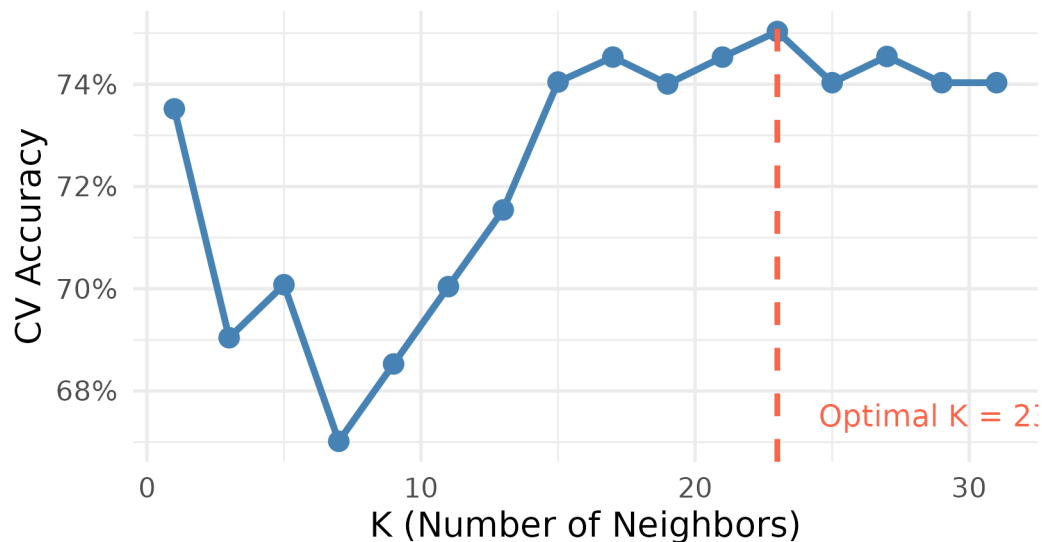
	Actual	
Predicted	No	Yes
No	200	51
Yes	23	58

```
# --- K vs. accuracy plot ---
knn_df <- data.frame(K = k_vals, Accuracy = cv_acc)

ggplot(knn_df, aes(x = K, y = Accuracy)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_point(size = 3, color = "steelblue") +
  geom_vline(xintercept = optimal_k_knn,
             color = "tomato", linetype = "dashed",
             linewidth = 1) +
  annotate("text",
          x = optimal_k_knn + 1.5,
          y = min(cv_acc) + 0.005,
          label = paste0("Optimal K = ",
                        optimal_k_knn),
          color = "tomato", hjust = 0, size = 4) +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "KNN: Cross-Validated Accuracy vs. K",
       subtitle = "5-fold CV on training set",
       x = "K (Number of Neighbors)",
       y = "CV Accuracy") +
  theme_minimal(base_size = 13)
```

KNN: Cross-Validated Accuracy vs. K

5-fold CV on training set



Code explanation:

- `scale(test_x, center = ..., scale = ...)` applies training set scaling to test data — critical to prevent data leakage; always scale test using training set parameters.
- `createFolds(y, k = 5)` creates stratified cross-validation folds, preserving class proportions in each fold. Generate the folds **once**, outside the loop over candidate hyperparameter values, and reuse the same folds for every value — this ensures each K is compared on identical data, which is essential for a fair comparison.
- `class::knn(..., prob = TRUE)` stores, via `attr(pred, "prob")`, the vote proportion for the *predicted (winning)* class — not the positive class. For ROC, convert it first: `ifelse(pred == "Yes", p, 1 - p)`.

Exercises

💡 Exercise 9.3

Using the `iris` dataset:

- Split into 70% train / 30% test. Standardize features.
- Tune K from 1 to 20 using 10-fold cross-validation. Plot CV accuracy vs. K .
- Fit the final KNN model. What is the test accuracy?
- Compare KNN to logistic regression on the same split. Which performs better for multiclass iris?

Decision Trees

Introduction

Decision trees classify observations by recursively partitioning the feature space using a series of simple binary rules — “if $x_j \leq t$ then go left, else go right.” Each leaf node represents a class prediction. Decision trees are uniquely valuable for their **interpretability** — the learned rules can be visualized and explained to non-technical stakeholders. They are also the building blocks of the most powerful ensemble methods.

Theory

Tree Construction: Recursive Binary Splitting

Starting from the root node containing all training data, at each node:

1. For each feature x_j and each possible split point t , compute the **impurity reduction** from splitting.
2. Choose the split (x_j^*, t^*) that maximally reduces impurity.
3. Recursively apply to each child node.
4. Stop when a stopping criterion is met (minimum node size, maximum depth, or minimum impurity reduction).

Impurity Measures

Gini impurity (used in CART):

$$G(t) = \sum_{k=1}^K p_k(t)(1 - p_k(t)) = 1 - \sum_{k=1}^K p_k(t)^2$$

Entropy (used in C4.5/C5.0):

$$H(t) = - \sum_{k=1}^K p_k(t) \log_2 p_k(t)$$

Misclassification rate:

$$E(t) = 1 - \max_k p_k(t)$$

where $p_k(t)$ is the proportion of class k observations at node t . All three are minimized when a node is pure (one class only). Gini and entropy are preferred for growing trees because they are more sensitive to impurity changes than misclassification rate.

The Information Gain (and Gini Reduction)

The quality of a split is measured by how much it reduces impurity. With **entropy**, this is called the **information gain**:

$$\text{IG}(t, x_j, s) = H(t) - \frac{n_L}{n} H(t_L) - \frac{n_R}{n} H(t_R)$$

The identical idea applied to **Gini impurity** is usually just called the **Gini reduction** (or Gini gain) — same notation, H replaced by G :

$$\Delta G(t, x_j, s) = G(t) - \frac{n_L}{n} G(t_L) - \frac{n_R}{n} G(t_R)$$

where t_L and t_R are the left and right child nodes produced by splitting node t on feature x_j at threshold s , and n_L, n_R are the number of observations sent to each child ($n_L + n_R = n$). **rpart** (used in the R examples below) grows trees by choosing, at every node, the split (x_j, s) that **maximizes** ΔG .

Worked Example: Computing a Split's Gini Reduction

Suppose a node t has 8 observations with 2 features (**income**, **age**) and label **default** (4 Yes, 4 No):

- **Before the split:** $p_{\text{Yes}} = p_{\text{No}} = 0.5 \Rightarrow G(t) = 1 - (0.5^2 + 0.5^2) = 0.5$.
- **Splitting on income $\leq s$:** left child gets 5 obs (4 No, 1 Yes) $\Rightarrow G(t_L) = 1 - (0.8^2 + 0.2^2) = 0.32$; right child gets 3 obs, all Yes $\Rightarrow G(t_R) = 0$ (pure).
- **Gini reduction:** $\Delta G = 0.5 - \frac{5}{8}(0.32) - \frac{3}{8}(0) = 0.5 - 0.2 = 0.3$.

If splitting on **age** at its best threshold gives a smaller ΔG (say 0.18), **income** is chosen for this node. This is exactly the calculation **rpart** performs — at every node, for every candidate feature and threshold — to pick each split, and it is the building block for the variable importance measure below.

Variable Importance in a Single Tree

A feature's **importance** in a fitted tree is the total impurity reduction it is responsible for, summed over every node where it was used to split, weighted by how much of the data reached that node:

$$\text{Importance}(x_j) = \sum_{t: \text{split variable at } t = x_j} \frac{n_t}{n} \Delta G(t, x_j, s_t^*)$$

where the sum runs over all nodes t that were split on x_j , n_t is the number of training observations at node t , and s_t^* is the threshold chosen at that node. A feature used in many splits, or in splits near the root (where n_t is large), accumulates high importance; a feature never chosen for any split gets zero. This is precisely what **rpart** reports as `tree_pruned$variable.importance` in the R example below — now you can see where those numbers come from. (Two details worth knowing: **rpart** credits some importance to **surrogate splits** — backup variables highly correlated with the chosen splitter, used when the primary variable's value is missing — so its output can differ slightly from summing ΔG by hand; and this whole idea reappears, unchanged, as **Mean Decrease in Gini (MDG)** when we average it across all trees in a Random Forest later in this chapter — MDG is not a separate concept, just this same single-tree importance averaged over many trees.)

Pruning

A fully grown tree overfits. **Post-pruning** (cost-complexity pruning, also called weakest-link pruning) removes branches that do not improve generalization:

$$\text{Minimize: } R_\alpha(T) = R(T) + \alpha|T|$$

where $R(T)$ is the training error, $|T|$ is the number of leaves, and $\alpha \geq 0$ is the complexity parameter. Cross-validation selects the optimal α .

Advantages and Limitations

Advantages: - Highly interpretable (“white box” model) - Handles mixed data types natively - No scaling required - Captures non-linear relationships and interactions

Limitations: - High variance — small changes in data produce different trees - Biased toward features with many levels - Poor extrapolation beyond training data range - Suboptimal compared to ensemble methods

Example: Decision Tree for Diabetes Classification

Example 9.4. A decision tree for the Pima dataset with maximum depth 3 (`rpart.control(maxdepth = 3, cp = 0.001, minsplit = 10)`, `set.seed(42)`) produces:

```
If glu < 123.5:                → No (86% leaf purity)
If glu >= 123.5:
  If ped < 0.31:
    If glu < 166:                → No (78% leaf purity)
    If glu >= 166:               → Yes (75% leaf purity)
  If ped >= 0.31:
    If bmi < 28.65:              → No (73% leaf purity)
    If bmi >= 28.65:            → Yes (84% leaf purity)
```

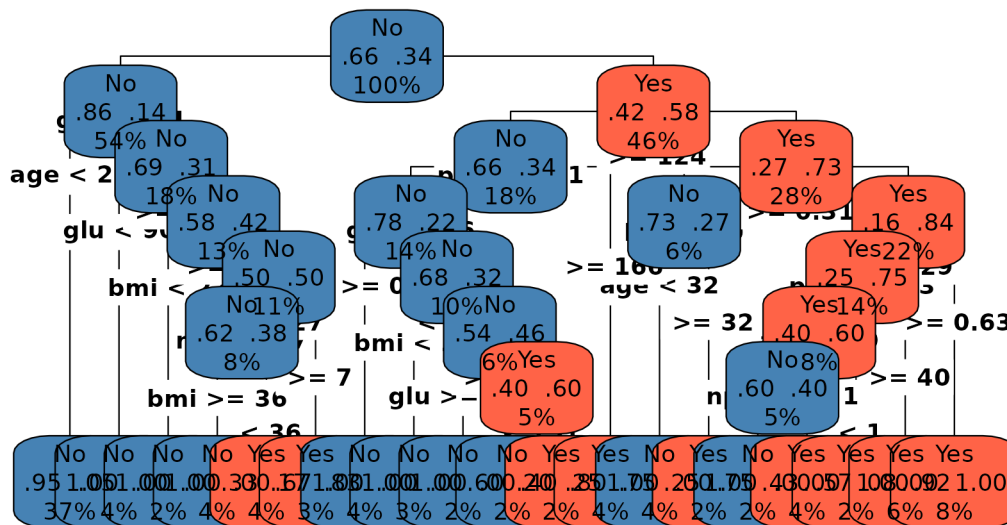
The tree reveals that glucose level is the primary predictor — consistent with medical knowledge. Once glucose is high (≥ 123.5), the diabetes pedigree function (`ped`) becomes the next most informative split, followed by BMI. Note that a decision tree's exact split points and structure are specific to the training sample and algorithm settings (`cp`, `minsplit`, `maxdepth`); rerunning this code is the best way to see the actual tree for your data — always prefer `rpart.plot()` output over a hand-copied rule list when reporting results.

R Example: Decision Trees

```
# --- Decision tree on Pima dataset ---
set.seed(42)
tree_model <- rpart(type ~ glu + bmi + age + ped + npreg,
  data      = Pima.tr,
  method    = "class",
  control   = rpart.control(
    minsplit = 10,
    cp       = 0.001,
    maxdepth = 10
  ))

# Visualize the tree
rpart.plot(tree_model,
  type      = 4,
  extra     = 104,
  box.palette = c("steelblue", "tomato"),
  main      = "Decision Tree: Pima Diabetes",
  cex       = 0.75)
```

Decision Tree: Pima Diabetes



```
# --- Cost-complexity pruning ---
# Print CP table
printcp(tree_model)
```

Classification tree:

```
rpart(formula = type ~ glu + bmi + age + ped + npreg, data = Pima.tr,
      method = "class", control = rpart.control(minsplit = 10,
        cp = 0.001, maxdepth = 10))
```

Variables actually used in tree construction:

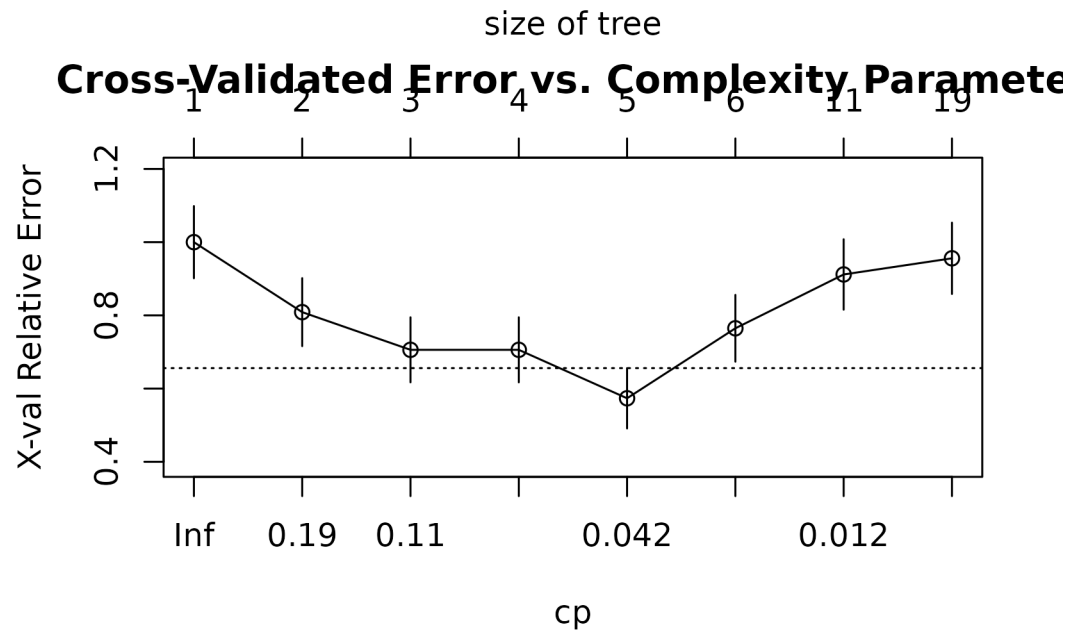
```
[1] age  bmi  glu  npreg ped
```

Root node error: 68/200 = 0.34

n= 200

	CP	nsplit	rel error	xerror	xstd
1	0.2205882	0	1.00000	1.00000	0.098518
2	0.1617647	1	0.77941	0.80882	0.092863
3	0.0735294	2	0.61765	0.70588	0.088822
4	0.0588235	3	0.54412	0.70588	0.088822
5	0.0294118	4	0.48529	0.57353	0.082399
6	0.0147059	5	0.45588	0.76471	0.091224
7	0.0098039	10	0.35294	0.91176	0.096186
8	0.0010000	18	0.26471	0.95588	0.097409


```
# Plot CV error vs. complexity
plotcp(tree_model,
       main = "Cross-Validated Error vs. Complexity Parameter")
```



```
# Prune to optimal cp
opt_cp <- tree_model$cptable[
  which.min(tree_model$cptable[, "xerror"]),
  "CP"]
tree_pruned <- prune(tree_model, cp = opt_cp)

cat("\nOptimal CP:", round(opt_cp, 5), "\n")
```

Optimal CP: 0.02941

```
cat("Tree size before pruning:",
    nrow(tree_model$frame), "nodes\n")
```

Tree size before pruning: 37 nodes

```
cat("Tree size after pruning: ",
    nrow(tree_pruned$frame), "nodes\n")
```

Tree size after pruning: 9 nodes

```
# --- Predictions ---
tree_pred <- predict(tree_pruned, Pima.te,
                     type = "class")
tree_probs <- predict(tree_pruned, Pima.te,
                     type = "prob")[, "Yes"]
tree_acc   <- mean(tree_pred == Pima.te$type)

cat("\nPruned Tree Test Accuracy:",
    round(tree_acc * 100, 1), "%\n")
```

Pruned Tree Test Accuracy: 75.6 %

```
# Variable importance
imp_df <- data.frame(
  Variable = names(tree_pruned$variable.importance),
  Importance = round(tree_pruned$variable.importance, 2)
) |> arrange(desc(Importance))

kable(imp_df,
      caption = "Variable Importance: Decision Tree") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
               full_width = FALSE)
```

Table 1: Variable Importance: Decision Tree

	Variable	Importance
glu	glu	24.00
bmi	bmi	7.82
ped	ped	7.17
age	age	6.04
npreg	npreg	4.66

Code explanation:

- `rpart(formula, method = "class")` fits a classification tree. `method = "anova"` is used for regression.
- `printcp()` displays the cross-validated error for each tree size — the row with minimum `xerror` gives the optimal pruning level.
- `prune(tree, cp)` prunes the tree to the specified complexity. `which.min(cptable[, "xerror"])` automates optimal α selection.
- `rpart.plot(..., extra = 104)` displays class probabilities and percentages in each leaf — making the tree a complete interpretable model.

Exercises

Exercise 9.4

Using the `iris` dataset:

- (a) Grow a full decision tree (no depth limit). What is the training accuracy?
 - (b) Apply cost-complexity pruning. Plot the CV error curve and identify the optimal tree size.
 - (c) Visualize the pruned tree using `rpart.plot()`. How many levels deep is it?
 - (d) Compare Gini impurity (`parms = list(split = "gini")`) vs. entropy (`"information"`). Do they produce different trees?
-

Random Forest and Ensemble Methods

Introduction

A single decision tree has high variance — small perturbations in the training data produce very different trees. **Ensemble methods** address this by combining many models, each trained on a perturbed version of the data, and aggregating their predictions. The result is dramatically lower variance with minimal increase in bias, producing classifiers that are among the most accurate available for tabular data. **Random Forest** and **Gradient Boosting** are the dominant ensemble methods in modern data science.

Theory

Bagging (Bootstrap Aggregating)

Bagging trains B decision trees on B bootstrap samples of the training data, then aggregates predictions by majority vote (classification) or mean (regression):

$$\hat{f}_{\text{bag}}(x) = \text{majority vote}\{\hat{f}_b(x)\}_{b=1}^B$$

The key insight: averaging reduces variance by a factor of B without increasing bias **if the predictors were independent**. In practice they are not — every bootstrap sample is drawn from the *same* training set, so the resulting trees are positively correlated, and the achievable variance reduction is smaller than a full factor of B . The more correlated the trees, the less bagging helps. This is precisely the motivation for Random Forest’s extra feature-randomization step below: by decorrelating the trees further, it recovers variance reduction that plain bagging leaves on the table.

Out-of-Bag (OOB) Error: Each bootstrap sample omits approximately 37% of training observations. These out-of-bag observations can be used to estimate generalization error without a separate validation set.

Random Forest

Random Forest extends bagging with **feature randomization**: at each split, only a random subset of m features is considered (not all p):

$$m \approx \sqrt{p} \text{ (classification)}, \quad m \approx p/3 \text{ (regression)}$$

This **decorrelates** the trees — even if one feature is very strong, it doesn't dominate every split in every tree. Decorrelation further reduces variance beyond what bagging alone achieves.

Variable Importance:

- **Mean Decrease in Accuracy (MDA)**: Permute each feature and measure the drop in OOB accuracy.
- **Mean Decrease in Gini (MDG)**: The single-tree importance defined earlier ($\sum \frac{n_t}{n} \Delta G$ over the nodes split on x_j), averaged across all B trees in the forest.

MDA is more principled and robust; MDG is faster to compute.

Boosting

Boosting builds trees sequentially, with each new tree focused on correcting the mistakes of the ensemble built so far. The exact mechanism differs by algorithm — AdaBoost reweights misclassified observations, while Gradient Boosting fits new trees to the ensemble's **residuals** (errors):

AdaBoost: Increases weights of misclassified observations so subsequent trees focus on hard cases.

Gradient Boosting (GBM): Fits each tree to the negative gradient of the loss function — a generalization of residual fitting.

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

where η is the learning rate and h_m is the m -th tree fit to residuals. **XGBoost** and **LightGBM** are highly optimized gradient boosting implementations.

Bagging vs. Boosting

Table 1: Bagging vs. Boosting comparison

Feature	Bagging / Random Forest	Boosting
Tree building	Parallel (independent)	Sequential (dependent)
Main effect	Reduces variance	Reduces bias
Overfitting risk	Low	Higher (needs regularization)
Interpretability	Moderate (variable importance)	Low
Typical performance	Very good	Often best-in-class

Example: Random Forest Feature Importance

Example 9.5. Random forest on the Pima dataset ($B = 500$ trees, $m = 2$) identifies glucose as the most important feature by both MDA and MDG — consistent with medical knowledge and the logistic regression results. Beyond glucose, though, **MDA and MDG disagree** on the ranking: by MDA, age and diabetes pedigree function (`ped`) outrank BMI, with BMI actually last; by MDG, `ped` and age again outrank BMI, but BMI outranks number of pregnancies (`npreg`). This is a known property of the two measures — MDG (based on impurity reduction) tends to favor continuous predictors with many possible split points, while MDA (based on permutation) is generally considered the more reliable measure of true predictive importance. The lesson: **check both importance measures rather than relying on either alone**, and don't expect them to agree exactly.

R Example: Random Forest and Ensemble Methods

```
# --- Random Forest on Pima dataset ---
set.seed(42)
rf_model <- randomForest(
  type ~ glu + bmi + age + ped + npreg,
  data      = Pima.tr,
  ntree     = 500,
  mtry      = 2,
  importance = TRUE
)

print(rf_model)
```

Call:

```
randomForest(formula = type ~ glu + bmi + age + ped + npreg,      data = Pima.tr, ntree
              Type of random forest: classification
```

Number of trees: 500
No. of variables tried at each split: 2

OOB estimate of error rate: 25.5%

Confusion matrix:

	No	Yes	class.error
No	111	21	0.1590909
Yes	30	38	0.4411765

```
# --- Variable importance ---
imp_rf <- as.data.frame(importance(rf_model))
imp_rf$Variable <- rownames(imp_rf)
imp_rf <- imp_rf |>
  arrange(desc(MeanDecreaseAccuracy)) |>
  rename(MDA = MeanDecreaseAccuracy,
         MDG = MeanDecreaseGini)

kable(round(imp_rf[, c("MDA","MDG")], 3),
      caption = "Random Forest Variable Importance",
      col.names = c("MDA","MDG"),
      row.names = TRUE) |>
  kable_styling(bootstrap_options = c("striped","hover"),
               full_width = FALSE)
```

Table 2: Random Forest Variable Importance

	MDA	MDG
glu	19.054	26.638
age	12.441	17.263
ped	10.775	17.480
npreg	10.498	11.309
bmi	9.849	16.633

```
# --- Predictions ---
rf_pred <- predict(rf_model, Pima.te)
rf_probs <- predict(rf_model, Pima.te, type = "prob")[, "Yes"]
rf_acc <- mean(rf_pred == Pima.te$type)

cat("Random Forest Test Accuracy:",
    round(rf_acc * 100, 1), "%\n")
```

Random Forest Test Accuracy: 76.8 %

```

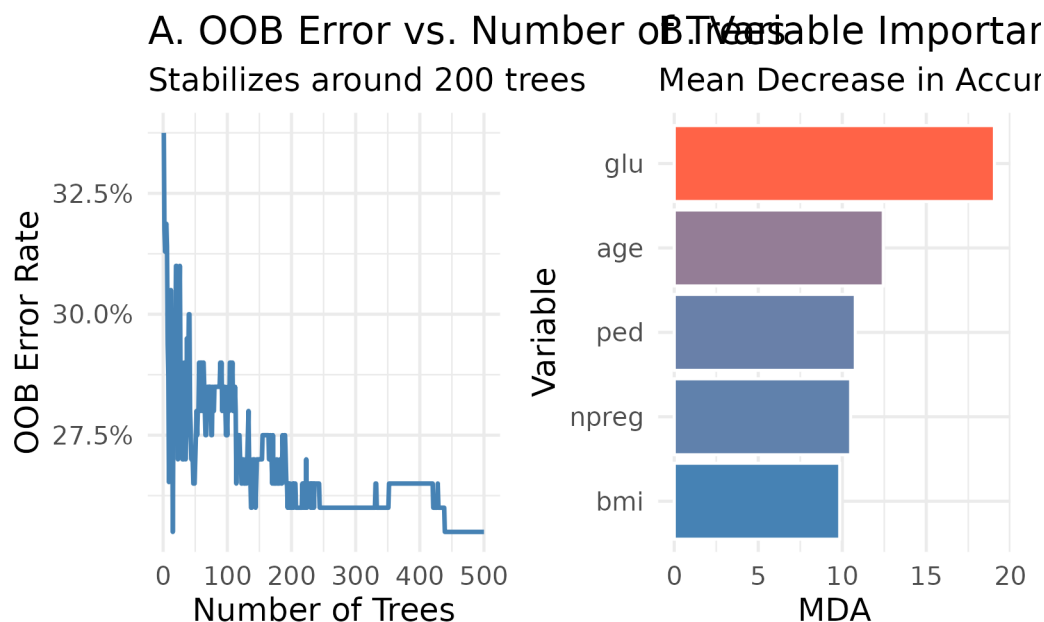
# --- OOB error vs. number of trees ---
oob_df <- data.frame(
  Trees      = 1:rf_model$ntree,
  OOB_Error  = rf_model$err.rate[, "OOB"]
)

p1 <- ggplot(oob_df, aes(x = Trees, y = OOB_Error)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  scale_y_continuous(labels = scales::percent) +
  labs(title      = "A. OOB Error vs. Number of Trees",
       subtitle   = "Stabilizes around 200 trees",
       x          = "Number of Trees",
       y          = "OOB Error Rate") +
  theme_minimal(base_size = 12)

# Variable importance plot
p2 <- imp_rf |>
  ggplot(aes(x = reorder(Variable, MDA),
             y = MDA, fill = MDA)) +
  geom_col(color = "white") +
  scale_fill_gradient(low = "steelblue",
                    high = "tomato") +
  coord_flip() +
  labs(title      = "B. Variable Importance (MDA)",
       subtitle   = "Mean Decrease in Accuracy",
       x          = "Variable", y = "MDA") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")

p1 + p2

```

Code explanation:

- `randomForest(formula, ntree, mtry, importance = TRUE)` fits the model. `importance = TRUE` enables both MDA and MDG calculation.
- `rf_model$err.rate[, "OOB"]` gives the OOB error after each tree is added — plotting this reveals the number of trees needed for convergence.
- `predict(rf_model, type = "prob")` returns class probabilities — needed for ROC analysis.
- The OOB error curve (Panel A) shows that around 200 trees, additional trees provide diminishing returns — useful for balancing accuracy and computation time.

Exercises

💡 Exercise 9.5

Using the `iris` dataset:

- Train a random forest with $B = 500$ trees. Tune `mtry` using OOB error for $m = 1, 2, 3, 4$.
- Compare OOB accuracy to test accuracy. Are they similar?
- Plot variable importance (MDA and MDG). Do both methods agree on the most important features?
- Fit a gradient boosting model using `gbm::gbm()` or `xgboost`. Compare test accuracy to random forest.

💡 Exercise 9.6 (Challenge)

Explore the bias-variance tradeoff in random forests:

- (a) Train forests with $B = 1, 5, 20, 100, 500$ trees on the Pima dataset. Record test accuracy for each.
- (b) Is there a point of diminishing returns? At what B does accuracy stabilize?
- (c) Compare single decision tree vs. bagging vs. random forest test accuracy. Which variance reduction is greater: bagging or random forests?

Model Evaluation

Introduction

Accuracy — the proportion of correctly classified observations — is the most intuitive performance metric. But it is often deeply misleading: a classifier that always predicts “No disease” achieves 90% accuracy on a dataset where 90% of patients are healthy, yet is completely useless. **Comprehensive model evaluation** requires a suite of metrics that capture different aspects of performance, particularly for imbalanced classes and asymmetric misclassification costs.

Theory

The Confusion Matrix

For binary classification (positive = 1, negative = 0):

Table 1: Confusion matrix structure

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Watch the Orientation

The layout above (rows = Predicted, columns = Actual) matches R’s `table(Predicted = ..., Actual = ...)` used throughout this chapter. Python’s `sklearn.metrics.confusion_matrix()` uses the **opposite** orientation by default (rows = Actual/true label, columns = Predicted). The formulas for precision, recall, etc. are the same either way, but if you read a confusion matrix off a plot or printout without checking which axis is which, you can flip precision and recall by mistake. Always check the axis labels.

Derived Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{of predicted positives, how many are truly positive?})$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad (\text{of actual positives, how many did we catch?})$$

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (\text{of actual negatives, how many did we correctly identify?})$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{harmonic mean of precision and recall})$$

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (\beta > 1: \text{recall weighted more; } \beta < 1: \text{precision weighted more})$$

The Precision-Recall Tradeoff

Lowering the classification threshold increases recall (catch more positives) but decreases precision (more false alarms). The **precision-recall curve** visualizes this tradeoff across all thresholds. The **average precision (AP)** summarizes the curve.

ROC Curve and AUC

The **Receiver Operating Characteristic (ROC)** curve plots Sensitivity (TPR) vs. $1 - \text{Specificity}$ (FPR) across all thresholds.

AUC (Area Under the Curve): - AUC = 1.0: Perfect classifier - AUC = 0.5: Random classifier (diagonal line) - AUC < 0.5: Worse than random

AUC has a probabilistic interpretation: the probability that the classifier ranks a random positive observation higher than a random negative one.

Metrics for Imbalanced Classes

When one class is rare, accuracy is misleading. Preferred metrics:

Table 2: Metrics for imbalanced classification

Metric	Use When
Precision, Recall, F1	Minority class matters most
AUC-ROC	Overall ranking ability
Matthews Correlation Coefficient (MCC)	Balanced summary for imbalanced data
Cohen's Kappa	Agreement above chance

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC ranges from -1 to $+1$; values near $+1$ indicate excellent classification.

Handling Imbalanced Classes

Choosing the right metric only measures the problem — it doesn't fix it. When the minority class is rare, consider:

- **Resampling the training data:** oversample the minority class (randomly, or synthetically with **SMOTE**, which generates new minority points by interpolating between existing ones), or undersample the majority class. `DMwR::SMOTE()` and `themis::step_smote()` (tidymodels) are common R implementations.
- **Class weighting / cost-sensitive learning:** give a higher misclassification cost to the minority class during training (e.g., `class.weights` in `e1071::svm()`, `classwt` in `randomForest()`, or `weight` in `glm()`).
- **Threshold moving:** keep the model as-is but choose a classification threshold below 0.5 to trade precision for recall (see the threshold-analysis example in the chapter lab).

Resampling changes what the model *learns*; threshold moving changes only how its output probabilities are *converted to labels* — the two are complementary, and it's a common mistake to use only one when both are appropriate.

Example: Why Accuracy Alone is Insufficient

Example 9.6. A fraud detection model flags 1% of transactions as fraudulent (true fraud rate = 0.5%). A model that flags nothing achieves 99.5% accuracy. The fraud-detection model that flags 1% correctly catches 80% of all fraud (recall = 0.80) but has many false

alarms (precision = 0.40). Accuracy would rank the “flag nothing” model higher, but recall and F1 correctly identify the fraud detector as more valuable.

R Example: Comprehensive Model Evaluation

```
# --- Compute all metrics for all models ---
# Collect predictions from all models trained earlier
models_df <- data.frame(
  true      = Pima.te$type,
  lr_class  = factor(ifelse(lr_probs >= 0.5,
                             "Yes", "No"),
                    levels = c("No", "Yes")),
  lr_prob   = lr_probs,
  knn_class = knn_pred,
  knn_prob  = ifelse(knn_pred == "Yes", attr(knn_pred, "prob"), 1 - attr(knn_pred, "prob")),
  tree_class = tree_pred,
  tree_prob = tree_probs,
  rf_class  = rf_pred,
  rf_prob   = rf_probs
)

# Function to compute metrics from confusion matrix
compute_metrics <- function(pred, true, prob,
                             model_name) {
  cm <- table(Predicted = pred, Actual = true)
  TP <- cm["Yes", "Yes"]; TN <- cm["No", "No"]
  FP <- cm["Yes", "No"]; FN <- cm["No", "Yes"]
  n  <- sum(cm)

  acc <- (TP + TN) / n
  prec <- TP / (TP + FP)
  rec <- TP / (TP + FN)
  spec <- TN / (TN + FP)
  f1 <- 2 * prec * rec / (prec + rec)
  mcc <- (TP*TN - FP*FN) /
    sqrt((TP+FP)*(TP+FN)*(TN+FP)*(TN+FN))
  auc <- pROC::auc(
    pROC::roc(as.integer(true == "Yes"),
              prob, quiet = TRUE))

  data.frame(
    Model      = model_name,
```

```

    Accuracy    = round(acc, 4),
    Precision   = round(prec, 4),
    Recall      = round(rec, 4),
    Specificity = round(spec, 4),
    F1          = round(f1, 4),
    MCC         = round(mcc, 4),
    AUC         = round(as.numeric(auc), 4)
  )
}

metrics_table <- bind_rows(
  compute_metrics(models_df$lr_class,
    models_df$true,
    models_df$lr_prob, "Logistic Reg."),
  compute_metrics(models_df$knn_class,
    models_df$true,
    models_df$knn_prob, "KNN"),
  compute_metrics(models_df$tree_class,
    models_df$true,
    models_df$tree_prob, "Decision Tree"),
  compute_metrics(models_df$rf_class,
    models_df$true,
    models_df$rf_prob, "Random Forest")
)

kable(metrics_table,
  caption = "Comprehensive Model Evaluation: Pima Dataset") |>
kable_styling(bootstrap_options = c("striped", "hover"),
  full_width = FALSE) |>
column_spec(8, bold = TRUE,
  color = ifelse(
    metrics_table$AUC == max(metrics_table$AUC),
    "darkgreen", "steelblue"))

```

Table 3: Comprehensive Model Evaluation: Pima Dataset

Model	Accuracy	Precision	Recall	Specificity	F1	MCC	AUC
Logistic Reg.	0.7922	0.7128	0.6147	0.8789	0.6601	0.5145	0.8585
KNN	0.7771	0.7160	0.5321	0.8969	0.6105	0.4690	0.8495
Decision Tree	0.7560	0.6591	0.5321	0.8655	0.5888	0.4230	0.7278
Random Forest	0.7681	0.6509	0.6330	0.8341	0.6419	0.4705	0.8199

```

# --- Overlay ROC curves ---
roc_list <- list(
  "Logistic Reg." = pROC::roc(
    as.integer(models_df$true == "Yes"),
    models_df$lr_prob, quiet = TRUE),
  "KNN" = pROC::roc(
    as.integer(models_df$true == "Yes"),
    models_df$knn_prob, quiet = TRUE),
  "Decision Tree" = pROC::roc(
    as.integer(models_df$true == "Yes"),
    models_df$tree_prob, quiet = TRUE),
  "Random Forest" = pROC::roc(
    as.integer(models_df$true == "Yes"),
    models_df$rf_prob, quiet = TRUE)
)

colors <- c("steelblue", "seagreen", "darkorange", "tomato")

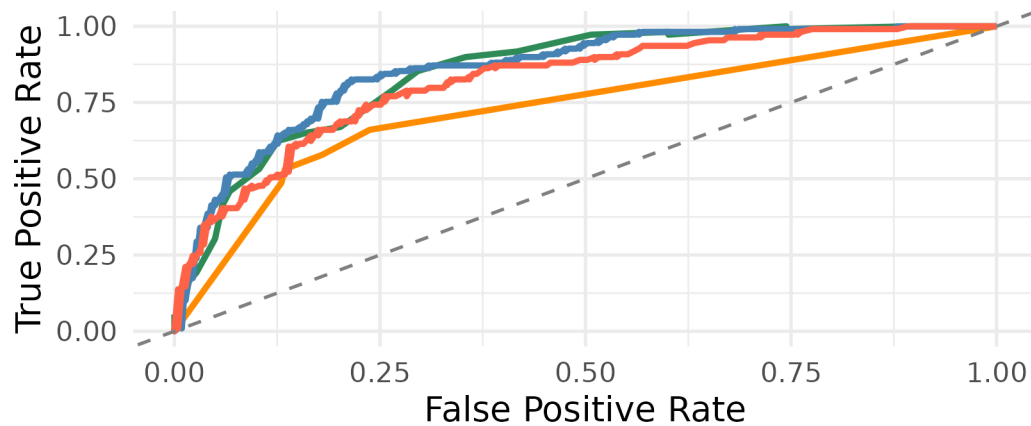
roc_df_all <- map2_dfr(roc_list, names(roc_list),
  function(r, nm) {
    data.frame(
      FPR = 1 - r$specificities,
      TPR = r$sensitivities,
      Model = nm,
      AUC = round(as.numeric(pROC::auc(r)), 3)
    )
  }) |>
  mutate(Label = paste0(Model, " (AUC=", AUC, ")"))

ggplot(roc_df_all, aes(x = FPR, y = TPR,
  color = Label)) +
  geom_line(linewidth = 1.1) +
  geom_abline(slope = 1, intercept = 0,
    linetype = "dashed", color = "gray50") +
  scale_color_manual(values = setNames(
    colors, unique(roc_df_all$Label))) +
  labs(title = "ROC Curves: All Models",
    subtitle = "Pima Diabetes Dataset - Test Set",
    x = "False Positive Rate",
    y = "True Positive Rate",
    color = "Model") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")

```


ROC Curves: All Models

Pima Diabetes Dataset — Test Set



Tree (AUC=0.728) — KNN (AUC=0.85) — Logistic Reg. (AUC=0.858) —

Code explanation:

- The `compute_metrics()` function computes all metrics from a confusion matrix — a reusable utility applicable to any binary classifier.
- `pROC::roc()` requires numeric labels (0/1) not factor levels — `as.integer(true == "Yes")` handles this conversion cleanly.
- Overlaying ROC curves on the same plot enables direct visual comparison of all models' discrimination ability across all thresholds simultaneously.

Exercises

💡 Exercise 9.7

A cancer screening test has the following confusion matrix on 1,000 patients (50 with cancer):

	Pred. Cancer	Pred. Healthy
Actual Cancer	42	8
Actual Healthy	95	855

- Compute accuracy, precision, recall, specificity, F1, and MCC.
- Is accuracy a good metric here? Why or why not?
- At what threshold would recall = 0.95? What is the precision at that threshold?
- Compute and plot the precision-recall curve using `PRROC::pr.curve()`.

Model Comparison and Selection

Introduction

With multiple classifiers trained and evaluated, the final question is: **which model should be deployed?** This requires more than comparing test set metrics — we need to account for the **variance** in performance estimates, guard against overfitting the test set through repeated evaluation, and consider practical constraints (interpretability, prediction speed, maintenance cost). Cross-validation and statistical testing provide a rigorous framework for model comparison.

Theory

K-Fold Cross-Validation

K -fold cross-validation partitions the training data into K equal folds. For each fold k :

1. Train the model on all folds except k .
2. Evaluate on fold k .

The cross-validated performance estimate is:

$$\hat{\text{Perf}}_{\text{CV}} = \frac{1}{K} \sum_{k=1}^K \text{Perf}_k$$

Stratified K -fold preserves class proportions in each fold — essential for imbalanced data.

Common choices: $K = 5$ (fast, slightly higher bias) or $K = 10$ (standard, near-unbiased).

Repeated K -fold: Repeat the entire K -fold process r times with different random splits, averaging over $K \times r$ evaluations. More stable estimates.

The Bias-Variance of CV Estimates

- **Leave-one-out CV (LOOCV):** $K = n$. Unbiased but high variance; computationally expensive.
- $K = 10$: Slight bias (models trained on 90% of data), low variance — the best practical choice.
- $K = 5$: Higher bias, even lower variance — used when computation is limited.

Statistical Comparison of Models

After cross-validation, each model has K performance scores. To test whether Model A outperforms Model B:

McNemar's test: Tests whether two classifiers make significantly different errors on the same test set.

Paired t-test on CV scores: Tests whether the mean CV performance differs between two models:

$$t = \frac{\bar{d}}{s_d/\sqrt{K}}$$

where d_k is the performance difference in fold k .

The Test Set Discipline

The Test Set Must Remain Untouched

The test set is for **final, one-time evaluation only**. Every time you peek at test results and tune the model accordingly, the test set becomes part of your training process — and your performance estimate is optimistically biased. Use cross-validation for all tuning decisions; reserve the test set for the final reported result.

Model Selection Criteria

Beyond performance, consider:

Table 1: Model selection criteria beyond performance

Factor	Question
Interpretability	Does the stakeholder need to understand the model?
Prediction speed	How fast must predictions be made?

Factor	Question
Training cost	How often will the model be retrained?
Robustness	How stable is performance across data shifts?
Maintenance	How easy is the model to update and monitor?

Example: Cross-Validated Model Comparison

Example 9.7. Four classifiers are evaluated with 10-fold cross-validation on the combined Pima dataset, **using the same 10 fold assignments for every model** (essential for a valid paired comparison — see the caution below). Results (mean AUC $\pm$ SD):

Model	Mean AUC	SD AUC
Logistic Regression	0.843	0.080
KNN	0.837	0.062
Random Forest	0.836	0.068
Decision Tree	0.790	0.072

Logistic Regression, KNN, and Random Forest achieve statistically indistinguishable mean AUC (all pairwise comparisons $p > 0.05$ after Bonferroni correction). The single Decision Tree, however, performs significantly worse than both Logistic Regression ($p = 0.031$) and Random Forest ($p = 0.022$) — consistent with the theory that a single tree has higher variance than an ensemble or a low-variance linear model. Because Logistic Regression is *at least as accurate* as the more complex KNN and Random Forest models here, and is far more interpretable, it is a strong candidate for clinical deployment — a case where the simplest adequate model wins.

A Common and Serious Mistake: Mismatched CV Folds

If each model in a comparison is cross-validated with its *own*, independently drawn set of folds, the resulting scores are **not paired** — fold 3 for Model A and fold 3 for Model B are different subsets of data. Running a paired t-test (or `caret::diff(resamples(...))`) on such scores silently violates the test's assumptions and can produce misleading conclusions about which model is better. Always fix the fold assignments once and pass them to every model (e.g., via `trainControl(index = ...)` in `caret`) before comparing.

R Example: Cross-Validation and Model Comparison

```
# --- 10-fold CV comparison using caret ---
set.seed(42)
pima_all <- bind_rows(Pima.tr, Pima.te)

# Define CV control
# IMPORTANT: pre-generate the 10 fold assignments ONCE and pass them via
# `index =`. Without this, each call to train() below draws its own random
# folds, so the four models are evaluated on DIFFERENT data splits. That
# breaks the "paired" comparison that resamples()/diff() assumes, and can
# flip which model looks best (see the callout box above).
cv_folds <- createFolds(pima_all$type, k = 10, returnTrain = TRUE)

ctrl_cv <- trainControl(
  method      = "cv",
  number      = 10,
  index       = cv_folds,
  classProbs  = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# Train all models with caret
models_cv <- list()

# Logistic Regression
models_cv$LR <- train(
  type ~ glu + bmi + age + ped + npreg,
  data      = pima_all,
  method    = "glm",
  family    = "binomial",
  trControl = ctrl_cv,
  metric    = "ROC"
)

# KNN
models_cv$KNN <- train(
  type ~ glu + bmi + age + ped + npreg,
  data      = pima_all,
  method    = "knn",
  trControl = ctrl_cv,
  tuneGrid  = data.frame(k = c(5, 9, 13, 17, 21)),
```

```

  metric      = "ROC",
  preProcess = c("center","scale")
)

# Decision Tree
models_cv$Tree <- train(
  type ~ glu + bmi + age + ped + npreg,
  data      = pima_all,
  method    = "rpart",
  trControl = ctrl_cv,
  tuneLength = 10,
  metric    = "ROC"
)

# Random Forest
models_cv$RF <- train(
  type ~ glu + bmi + age + ped + npreg,
  data      = pima_all,
  method    = "rf",
  trControl = ctrl_cv,
  tuneGrid  = data.frame(mtry = c(1,2,3,4)),
  metric    = "ROC"
)

# Compare results
cv_results <- resamples(models_cv)
summary(cv_results)

```

Call:

```
summary.resamples(object = cv_results)
```

Models: LR, KNN, Tree, RF

Number of resamples: 10

ROC

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
LR	0.6466049	0.8211835	0.8638228	0.8433204	0.8864749	0.9367284	0
KNN	0.6944444	0.8318394	0.8511016	0.8374926	0.8672619	0.9228395	0
Tree	0.6504630	0.7675881	0.7922386	0.7897734	0.8120370	0.9074074	0
RF	0.6682099	0.8247899	0.8518298	0.8361464	0.8730159	0.9205247	0

Sens

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
LR	0.7500000	0.8297619	0.9011905	0.8788095	0.9428571	0.9722222	0

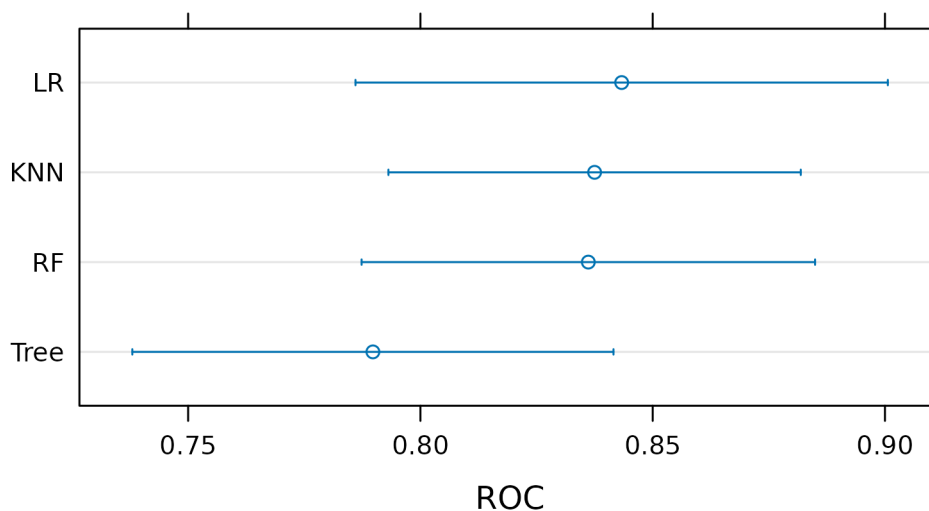
KNN	0.8000000	0.8650794	0.9015873	0.8929365	0.9160714	0.9722222	0
Tree	0.7142857	0.7777778	0.8170635	0.8280952	0.8809524	0.9428571	0
RF	0.6666667	0.8581349	0.8857143	0.8705556	0.9097222	0.9722222	0

Spec

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
LR	0.3888889	0.4918301	0.5882353	0.5702614	0.6111111	0.7777778	0
KNN	0.2941176	0.4583333	0.5294118	0.5408497	0.6111111	0.7777778	0
Tree	0.4444444	0.5555556	0.5718954	0.6042484	0.6960784	0.7777778	0
RF	0.4705882	0.5555556	0.5718954	0.6140523	0.7222222	0.7777778	0

```
# --- Visualize CV comparison ---
dotplot(cv_results, metric = "ROC",
        main = "10-Fold CV: AUC Comparison Across Models")
```

10-Fold CV: AUC Comparison Across Models



Confidence Level: 0.95

```
# --- Statistical comparison ---
cv_diff <- diff(cv_results)
summary(cv_diff)
```

Call:

```
summary.diff.resamples(object = cv_diff)
```

p-value adjustment: bonferroni

Upper diagonal: estimates of the difference

Lower diagonal: p-value for H0: difference = 0

ROC

	LR	KNN	Tree	RF
LR		0.005828	0.053547	0.007174
KNN	1.00000		0.047719	0.001346
Tree	0.03084	0.05979		-0.046373
RF	1.00000	1.00000	0.02195	

Sens

	LR	KNN	Tree	RF
LR		-0.014127	0.050714	0.008254
KNN	1.00000		0.064841	0.022381
Tree	0.41435	0.02937		-0.042460
RF	1.00000	1.00000	0.59448	

Spec

	LR	KNN	Tree	RF
LR		0.029412	-0.033987	-0.043791
KNN	1.00000		-0.063399	-0.073203
Tree	1.00000	0.23777		-0.009804
RF	1.00000	0.04223	1.00000	

```
# --- Final performance table ---
# 1. Get the basic stats
cv_summary <- summary(cv_results)$statistics$ROC

# 2. Calculate the SD separately (since summary doesn't include it)
# We pull this from the raw values stored in the resamples object
cv_sd <- apply(cv_results$values[, grep("~ROC", names(cv_results$values))], 2, sd)

# 3. Build and arrange the table
cv_table <- data.frame(
  Model      = rownames(cv_summary),
  Mean_AUC   = round(cv_summary[, "Mean"], 4),
  SD_AUC     = round(as.numeric(cv_sd), 4), # Ensure it's treated as a numeric vector
  Min_AUC    = round(cv_summary[, "Min."], 4),
  Max_AUC    = round(cv_summary[, "Max."], 4)
) |>
  arrange(desc(Mean_AUC))

# 4. View the result

kable(cv_table,
  caption   = "10-Fold Cross-Validated AUC Comparison",
  col.names = c("Model", "Mean AUC", "SD",
```

```

      "Min", "Max")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
row_spec(1, bold = TRUE, background = "#EEF2FF")

```

Table 2: 10-Fold Cross-Validated AUC Comparison

	Model	Mean AUC	SD	Min	Max
LR	LR	0.8433	0.0801	0.6466	0.9367
	KNN	0.8375	0.0620	0.6944	0.9228
	RF	0.8361	0.0682	0.6682	0.9205
	Tree	0.7898	0.0724	0.6505	0.9074

Code explanation:

- `caret::train()` provides a unified interface to dozens of algorithms with consistent cross-validation. `method = "glm", "knn", "rpart", "rf"` access different algorithms.
- `trainControl(index = cv_folds, ...)` fixes the same 10 fold assignments across every model in the comparison — without this, `train()` draws a fresh random split for each model, invalidating the paired comparison used below.
- `trainControl(summaryFunction = twoClassSummary, metric = "ROC")` evaluates models by AUC rather than accuracy — critical for imbalanced data.
- `resamples(models_cv)` collects CV results from multiple models for comparison. `diff(cv_results)` computes pairwise differences with statistical tests.
- `dotplot(cv_results)` from `lattice` produces a confidence-interval-based comparison plot — the standard visualization for comparing CV results.

Exercises

Exercise 9.8

Using the **Sonar** dataset from the **mlbench** package (binary classification: rock vs. mine):

- Apply 10-fold stratified CV to logistic regression, KNN, decision tree, and random forest.
- Report mean and SD of accuracy and AUC for each model.
- Use `diff(resamples())` to test pairwise differences. Which pairs are significantly different?
- Given that the dataset is highly imbalanced, rerun with F1 as the metric. Do rankings change?

 Exercise 9.9 (Challenge)

Implement a complete classification pipeline with `caret`:

- (a) Load the `GermanCredit` dataset from `caret`. Predict `Class` (Good/Bad credit).
- (b) Apply SMOTE oversampling for class imbalance using `DMwR::SMOTE()`.
- (c) Train and compare random forest vs. gradient boosting (`method = "gbm"`).
- (d) Report all metrics: accuracy, precision, recall, F1, AUC, MCC. Which model would you deploy?

Chapter Lab Activity: Medical Diagnosis Classification with Pima Data

Objectives

In this lab you will apply the complete classification pipeline — from exploratory analysis through model training, evaluation, and comparison — to the Pima Indians diabetes dataset. The medical context makes metric choice particularly important: false negatives (missing a diabetic patient) are more costly than false positives.

The Dataset

```
data(Pima.tr, package = "MASS")
data(Pima.te, package = "MASS")

pima_full <- bind_rows(
  Pima.tr |> mutate(partition = "Train"),
  Pima.te |> mutate(partition = "Test")
)

cat("Training set:", nrow(Pima.tr), "observations\n")
```

Training set: 200 observations

```
cat("Test set:      ", nrow(Pima.te), "observations\n\n")
```

Test set: 332 observations

```
# Class distribution
table(pima_full$type, pima_full$partition) |>
  addmargins() |>
  kable(caption = "Class Distribution by Partition") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
    full_width = FALSE)
```

Table 1: Class Distribution by Partition

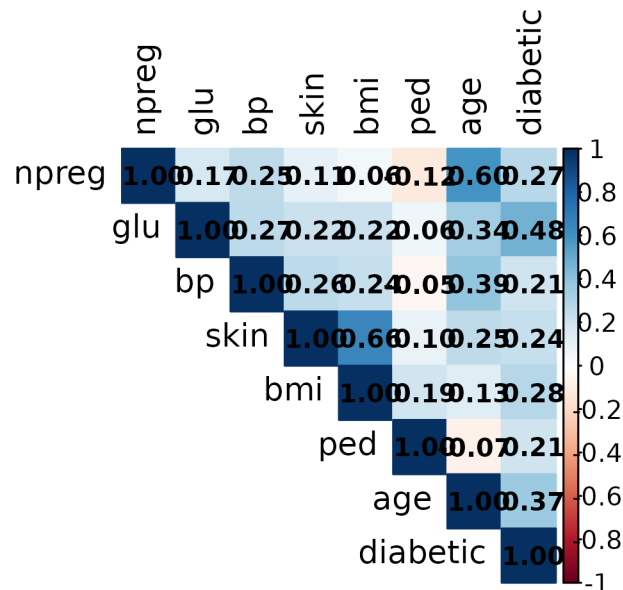
	Test	Train	Sum
No	223	132	355
Yes	109	68	177
Sum	332	200	532

Lab Task 1: Exploratory Analysis

```
# Feature correlation matrix
pima_num <- Pima.tr |>
  mutate(diabetic = as.integer(type == "Yes")) |>
  dplyr::select(-type)

cor_pima <- cor(pima_num)
corrplot::corrplot(cor_pima,
                    method = "color",
                    type = "upper",
                    addCoef.col = "black",
                    tl.col = "black",
                    number.cex = 0.8,
                    title = "Feature Correlation Matrix",
                    mar = c(0,0,2,0))
```

Feature Correlation Matrix



Lab Task 2: Train All Classifiers

```
# Standardize
train_features <- c("glu","bmi","age","ped","npreg","bp","skin")
X_train <- scale(Pima.tr[, train_features])
X_test  <- scale(Pima.te[, train_features],
                 center = attr(X_train,"scaled:center"),
                 scale  = attr(X_train,"scaled:scale"))
y_train <- Pima.tr$type
y_test  <- Pima.te$type

# 1. Logistic Regression
lr_full <- glm(type ~ .,
               data  = Pima.tr,
               family = binomial)
lr_p    <- predict(lr_full, Pima.te, type = "response")
lr_c    <- factor(ifelse(lr_p >= 0.5,"Yes","No"),
                 levels = c("No","Yes"))

# 2. KNN
set.seed(42)
knn_full <- class::knn(X_train, X_test,
                      y_train, k = 15, prob = TRUE)
knn_p    <- attr(knn_full,"prob")
knn_p    <- ifelse(knn_full == "Yes", knn_p, 1 - knn_p)

# 3. Decision Tree
tree_full <- rpart(type ~ ., data = Pima.tr,
                  method = "class",
                  control = rpart.control(cp = 0.01))
tree_c    <- predict(tree_full, Pima.te, type = "class")
tree_p    <- predict(tree_full, Pima.te, type = "prob")[, "Yes"]

# 4. Random Forest
set.seed(42)
rf_full <- randomForest(type ~ ., data = Pima.tr,
                       ntree = 500, importance = TRUE)
rf_c    <- predict(rf_full, Pima.te)
rf_p    <- predict(rf_full, Pima.te, type = "prob")[, "Yes"]

cat("All models trained successfully.\n")
```

All models trained successfully.

Lab Task 3: Comprehensive Evaluation

```
# Compute metrics for all models
eval_all <- bind_rows(
  compute_metrics(lr_c, y_test, lr_p, "Logistic Reg."),
  compute_metrics(knn_full, y_test, knn_p, "KNN (K=15)"),
  compute_metrics(tree_c, y_test, tree_p, "Decision Tree"),
  compute_metrics(rf_c, y_test, rf_p, "Random Forest")
)

kable(eval_all,
  caption = "Full Evaluation: All Models on Test Set") |>
kable_styling(bootstrap_options = c("striped","hover")) |>
column_spec(7, bold = TRUE,
  color = ifelse(eval_all$F1 ==
    max(eval_all$F1),
    "darkgreen","steelblue")) |>
column_spec(8, bold = TRUE,
  color = ifelse(eval_all$AUC ==
    max(eval_all$AUC),
    "darkgreen","steelblue"))
```

Table 2: Full Evaluation: All Models on Test Set

Model	Accuracy	Precision	Recall	Specificity	F1	MCC	AUC
Logistic Reg.	0.8012	0.7416	0.6055	0.8969	0.6667	0.5326	0.8659
KNN (K=15)	0.7711	0.7089	0.5138	0.8969	0.5957	0.4528	0.8213
Decision Tree	0.7319	0.5980	0.5596	0.8161	0.5782	0.3825	0.7764
Random Forest	0.7681	0.6667	0.5872	0.8565	0.6244	0.4595	0.8182

Lab Task 4: ROC and Threshold Analysis

```
# ROC curves
make_roc_df <- function(prob, true, model_name) {
  roc_obj <- pROC::roc(as.integer(true == "Yes"),
    prob, quiet = TRUE)
  data.frame(
    FPR = 1 - roc_obj$specificities,
    TPR = roc_obj$sensitivities,
```

```

    Model = model_name,
    AUC    = round(as.numeric(pROC::auc(roc_obj)), 3)
  )
}

roc_all <- bind_rows(
  make_roc_df(lr_p,    y_test, "Logistic Reg."),
  make_roc_df(knn_p,   y_test, "KNN"),
  make_roc_df(tree_p,  y_test, "Decision Tree"),
  make_roc_df(rf_p,    y_test, "Random Forest")
) |> mutate(Label = paste0(Model, " (",AUC,")"))

p1 <- ggplot(roc_all,
  aes(x = FPR, y = TPR, color = Label)) +
  geom_line(linewidth = 1.1) +
  geom_abline(slope = 1, intercept = 0,
    linetype = "dashed", color = "gray60") +
  scale_color_manual(
    values = c("steelblue","seagreen",
      "darkorange","tomato") |>
    setNames(unique(roc_all$Label))) +
  labs(title = "A. ROC Curves: All Models",
    x = "FPR", y = "TPR", color = "Model") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "bottom",
    legend.text = element_text(size = 8))

# Threshold analysis for RF (clinical context)
thresholds <- seq(0.2, 0.8, by = 0.05)
thresh_df <- map_dfr(thresholds, function(t) {
  pred_t <- factor(ifelse(rf_p >= t,"Yes","No"),
    levels = c("No","Yes"))
  tp <- sum(pred_t == "Yes" & y_test == "Yes")
  fp <- sum(pred_t == "Yes" & y_test == "No")
  fn <- sum(pred_t == "No"  & y_test == "Yes")
  tn <- sum(pred_t == "No"  & y_test == "No")
  data.frame(
    Threshold = t,
    Recall    = tp / (tp + fn),
    Precision = tp / (tp + fp),
    F1        = 2*tp / (2*tp + fp + fn)
  )
})

```



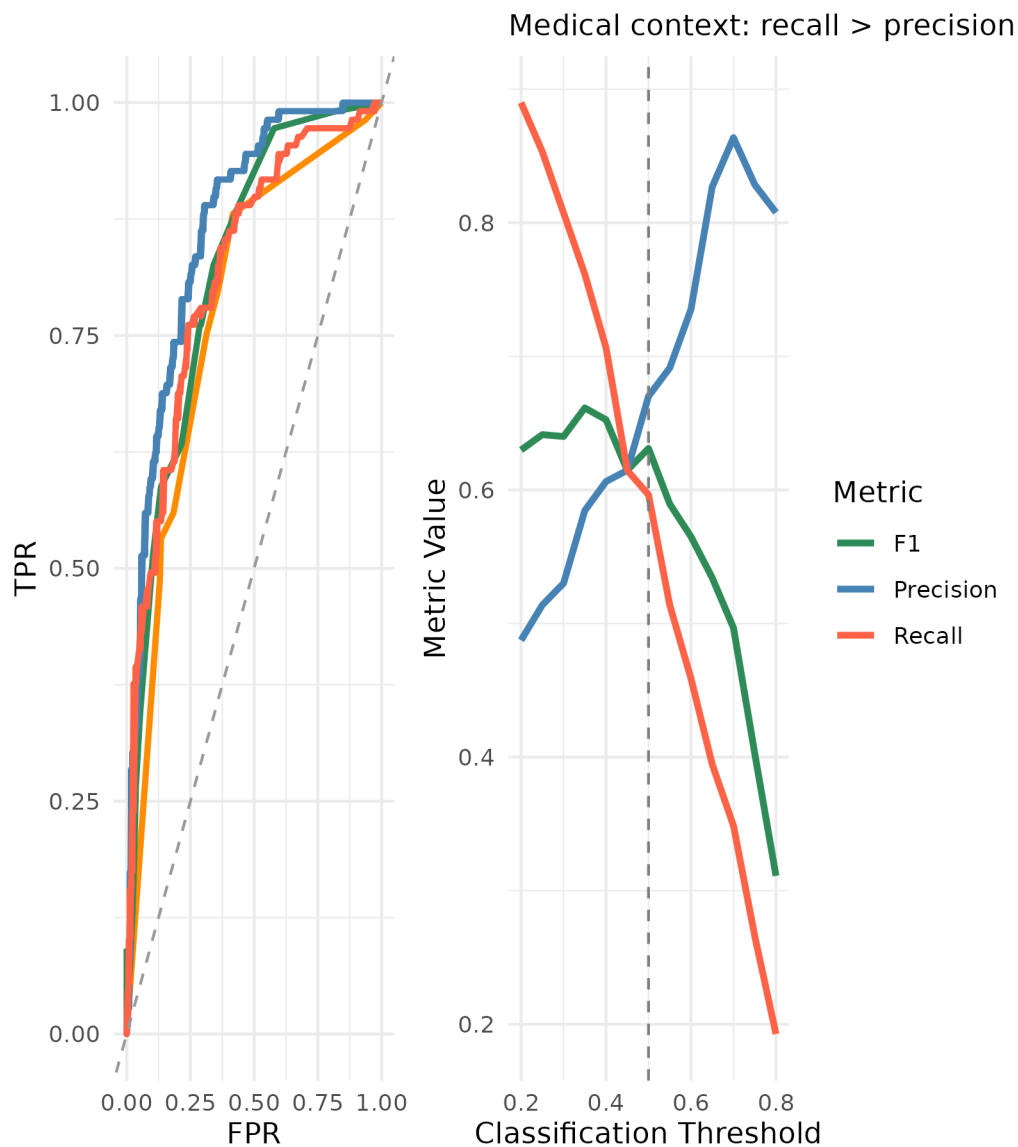
```

p2 <- thresh_df |>
  pivot_longer(-Threshold,
               names_to = "Metric",
               values_to = "Value") |>
  ggplot(aes(x = Threshold, y = Value,
             color = Metric)) +
  geom_line(linewidth = 1.2) +
  geom_vline(xintercept = 0.5,
            linetype = "dashed", color = "gray50") +
  scale_color_manual(
    values = c("Recall" = "tomato",
               "Precision" = "steelblue",
               "F1" = "seagreen")) +
  labs(title = "B. RF: Threshold vs. Metrics",
       subtitle = "Medical context: recall > precision",
       x = "Classification Threshold",
       y = "Metric Value") +
  theme_minimal(base_size = 11)

p1 + p2

```

A. ROC Curves: All Models B. RF: Threshold vs. Metrics



.776) KNN (0.821) Logistic Reg. (0.866) Random Forest (0.818)

Lab Discussion Questions

Answer the following in writing:

1. **Metric Choice:** In a diabetes screening context, which metric matters most — precision or recall? At what classification threshold (from Panel B) would you deploy the random forest model, and why?

2. **Model Interpretability:** The decision tree and logistic regression are more interpretable than the random forest. For a clinical setting where doctors must understand predictions, which model would you recommend? How does interpretability trade off against accuracy?
 3. **Class Imbalance:** The Pima dataset has approximately 33% positive cases (diabetic). Is this a serious imbalance problem? How would you handle a dataset where only 2% of cases are positive?
 4. **Overfitting Check:** Compare training accuracy to test accuracy for each model. Which model shows the most overfitting? What regularization or pruning approach would you apply?
 5. **Deployment Decision:** You must choose one model to deploy in a rural health clinic with limited computational resources. The predictions must be explainable to doctors. Which model do you choose, and what is your justification beyond just AUC?
-

Chapter Summary

Summary

This chapter built a comprehensive classification toolkit from foundations to practical deployment:

- **Introduction** — classification predicts categorical outcomes using labeled training data; decision boundary shape varies by algorithm; the training-test split discipline is inviolable.
- **Logistic regression** — models probability via the sigmoid function; coefficients are log-odds, exponentiated to odds ratios; estimated by maximum likelihood; the most interpretable classifier.
- **KNN** — classifies by majority vote among K nearest neighbors; non-parametric; requires scaling; K tuned by cross-validation; suffers in high dimensions.
- **Decision trees** — recursive binary splitting minimizes impurity (Gini/entropy); pruned by cost-complexity to prevent overfitting; highly interpretable; building block of ensemble methods.
- **Random forest** — bagging + feature randomization; reduces variance; OOB error estimates generalization; variable importance via MDA and MDG; among the most reliable classifiers for tabular data.
- **Model evaluation** — accuracy alone is insufficient; precision, recall, F1, MCC, and AUC capture different aspects; ROC and precision-recall curves visualize threshold tradeoffs.
- **Model comparison** — cross-validation provides unbiased performance estimates; statistical tests compare models; test set reserved for final evaluation only; practical factors (interpretability, speed) inform final selection.

! Key Formulas to Know

Logistic Model:

$$P(Y = 1|\mathbf{x}) = \frac{1}{1 + e^{-\beta^T \mathbf{x}}}$$

Gini Impurity:

$$G(t) = 1 - \sum_{k=1}^K p_k(t)^2$$

Precision, Recall, F1:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Matthews Correlation Coefficient:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Cross-Validated Performance:

$$\hat{\text{Perf}}_{\text{CV}} = \frac{1}{K} \sum_{k=1}^K \text{Perf}_k$$

References

- Anderson, E. (1935). The irises of the Gaspé Peninsula. *Bulletin of the American Iris Society*, 59, 2–5.
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer. <https://hastie.su.domains/ElemStatLearn/>
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning with applications in R* (2nd ed.). Springer. <https://www.statlearning.com>
- Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of Statistical Software*, 28(5), 1–26.

- Liaw, A., & Wiener, M. (2002). Classification and regression by randomForest. *R News*, 2(3), 18–22.
- Matthews, B. W. (1975). Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta*, 405(2), 442–451.
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Robin, X., Turck, N., Hainard, A., Tiberti, N., Lisacek, F., Sanchez, J.-C., & Müller, M. (2011). pROC: An open-source package for R and S+ to analyze and compare ROC curves. *BMC Bioinformatics*, 12, 77.
- Scrucca, L., Fop, M., Murphy, T. B., & Raftery, A. E. (2016). mclust 5: Clustering, classification and density estimation using Gaussian finite mixture models. *The R Journal*, 8(1), 289–317.
- Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. *Proceedings of the Symposium on Computer Applications and Medical Care* (pp. 261–265). IEEE.
- Therneau, T., & Atkinson, B. (2023). *rpart: Recursive partitioning and regression trees* (R package version 4.1-19). <https://CRAN.R-project.org/package=rpart>
- Venables, W. N., & Ripley, B. D. (2002). *Modern applied statistics with S* (4th ed.). Springer.
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.
- Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.

End of Chapter 9. Proceed to Chapter 10: Regression Analysis.

Chapter 10: Regression Analysis

```
# Load all libraries needed for this chapter
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
library(car)           # regression diagnostics
library(lmtest)        # Breusch-Pagan, Durbin-Watson
library(glmnet)        # Ridge and Lasso
library(mgcv)          # GAM
library(MASS)          # stepwise selection, datasets
library(broom)         # tidy model output
```

Chapter Overview

Introduction

Regression analysis is the statistical method for modeling the relationship between a continuous outcome variable and one or more predictor variables. It is arguably the most widely used technique in quantitative research — estimating the effect of education on wages, predicting house prices from property characteristics, quantifying how temperature affects crop yields. Regression bridges descriptive and inferential statistics: it summarizes relationships, tests hypotheses about effects, and makes predictions about new observations.

This chapter covers:

- **Simple Linear Regression** — the foundational one-predictor model
- **Multiple Linear Regression** — extending to many predictors, interpretation, and inference
- **Regression Diagnostics** — verifying model assumptions and identifying problems
- **Variable Selection** — choosing the right predictors systematically
- **Regularized Regression** — Ridge and Lasso for high-dimensional data
- **Non-Linear Regression** — polynomial, spline, and generalized additive models
- **Regression for Count and Categorical Outcomes** — Poisson and logistic regression extensions

Learning Objectives

By the end of this chapter, you will be able to:

1. Fit and interpret simple and multiple linear regression models.
2. Conduct and report hypothesis tests for regression coefficients.
3. Perform systematic regression diagnostics and address violations.
4. Apply variable selection methods including stepwise and information criteria.
5. Fit regularized regression models (Ridge, Lasso) and tune λ .
6. Recognize non-linearity and fit polynomial and spline regression models.
7. Select the appropriate regression model for count and categorical outcomes.

Simple Linear Regression

Introduction

Simple linear regression models the linear relationship between a single predictor X and a continuous outcome Y . It is simultaneously the simplest and most foundational regression model — and despite its simplicity, it remains the first tool a data scientist should reach for when exploring a bivariate relationship. Understanding it deeply prepares the ground for all extensions covered in this chapter.

Theory

The Simple Linear Regression Model

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2)$$

where: - β_0 is the **intercept** — the expected value of Y when $X = 0$ - β_1 is the **slope** — the expected change in Y for a one-unit increase in X - ε_i are independent, normally distributed errors with constant variance σ^2

Ordinary Least Squares (OLS) Estimation

The OLS estimators minimize the **residual sum of squares (RSS)**:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

The closed-form solutions are:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{S_{XY}}{S_{XX}}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

The **Gauss-Markov theorem** guarantees that OLS is the Best Linear Unbiased Estimator (BLUE) when linearity, independent/uncorrelated errors with mean zero, and constant

variance hold. Note that **normality is not required** for this result — it is needed only for exact t/F inference in small samples, not for OLS to be BLUE.

Inference for Regression Coefficients

The standard error of $\hat{\beta}_1$ is:

$$\text{SE}(\hat{\beta}_1) = \frac{s}{\sqrt{S_{XX}}}, \quad s = \sqrt{\frac{\text{RSS}}{n-2}}$$

where s is the **residual standard error** (estimated σ). The t-test for $H_0 : \beta_1 = 0$:

$$t = \frac{\hat{\beta}_1}{\text{SE}(\hat{\beta}_1)} \sim t(n-2) \text{ under } H_0$$

The Coefficient of Determination R^2

R^2 measures the proportion of total variance in Y explained by X :

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$R^2 \in [0, 1]$; higher values indicate better fit. For simple linear regression, $R^2 = r_{XY}^2$ — the square of the Pearson correlation coefficient.

Prediction vs. Confidence Intervals

- **Confidence interval for the mean response** at x_0 : Interval for $E[Y | X = x_0]$ — uncertainty about the population mean.
- **Prediction interval** for a new observation at x_0 : Wider interval accounting for both mean uncertainty and individual-level variability (σ^2).

$$\text{PI} = \hat{y}_0 \pm t_{n-2}^* \cdot s \sqrt{1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{XX}}}$$

Example: Simple Linear Regression

Example 10.1. The relationship between car weight (`wt`, in 1000 lbs) and fuel efficiency (`mpg`) in the `mtcars` dataset.

Fitted model: $\hat{mpg} = 37.29 - 5.34 \times wt$

Interpretation: - Intercept: A car weighing 0 lbs would have an expected mpg of 37.29 — extrapolation beyond the data range; not meaningful. - Slope: Each additional 1,000 lbs of weight is associated with a decrease of 5.34 mpg, on average. - $R^2 = 0.753$: Weight explains 75.3% of the variance in fuel efficiency. - Testing $H_0 : \beta_1 = 0$ vs. $H_1 : \beta_1 \neq 0$ (two-tailed), $n = 32$, $df = n - 2 = 30$:

Method 1 — p-value. $t_{cal} = -9.56$, $p < 0.001 < \alpha = 0.05 \Rightarrow$ **Reject H_0 .**

Method 2 — critical value. Two-tailed test, so:

$$t_{crit} = \pm t_{0.975, 30} = \pm qt(0.975, 30) = \pm 2.042$$

$$|t_{cal}| = 9.56 > 2.042 \Rightarrow \text{Reject } H_0$$

Both methods agree: the slope is highly significantly different from zero ($t_{cal} = -9.56$, $df = 30$, $p < 0.001$ — reported as $t(30) = -9.56$).

R Example: Simple Linear Regression

```
# --- Simple linear regression: mpg ~ wt ---
data(mtcars)
slr_model <- lm(mpg ~ wt, data = mtcars)
summary(slr_model)
```

Call:

```
lm(formula = mpg ~ wt, data = mtcars)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.5432	-2.3647	-0.1252	1.4096	6.8727

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.2851	1.8776	19.858	< 2e-16 ***
wt	-5.3445	0.5591	-9.559	1.29e-10 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared: 0.7528, Adjusted R-squared: 0.7446
F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10

```
# --- Tidy output ---
tidy_slr <- broom::tidy(slr_model, conf.int = TRUE)

tidy_slr$term <- sub("^\\(Intercept\\)$", "Intercept", tidy_slr$term)

tidy_slr[, -c(1)] |>
  round(4) |>
  # Add the column while it's still a data frame/tibble
  add_column(Term = tidy_slr$term, .before = 1) |>
  # Now turn it into a kable
  kable(caption = "Simple Linear Regression: mpg ~ wt",
        col.names = c("Term", "Estimate", "SE", "t", "p-value",
                      "CI Lower", "CI Upper"),
        row.names = FALSE) |>
  kable_styling(bootstrap_options = c("striped", "hover"),
                full_width = FALSE)
```

Table 1: Simple Linear Regression: mpg ~ wt

Term	Estimate	SE	t	p-value	CI Lower	CI Upper
Intercept	37.2851	1.8776	19.8576	0	33.4505	41.1198
wt	-5.3445	0.5591	-9.5590	0	-6.4863	-4.2026

```
# --- Regression plot with CI and PI ---
pred_df <- data.frame(wt = seq(min(mtcars$wt),
                              max(mtcars$wt),
                              length.out = 100))

ci_band <- predict(slr_model, pred_df,
                  interval = "confidence")
pi_band <- predict(slr_model, pred_df,
                  interval = "prediction")

plot_df <- bind_cols(pred_df,
                    as.data.frame(ci_band),
                    as.data.frame(pi_band) |>
                      rename(pi_lwr = lwr,
                             pi_upr = upr,
                             pi_fit = fit))
```

```

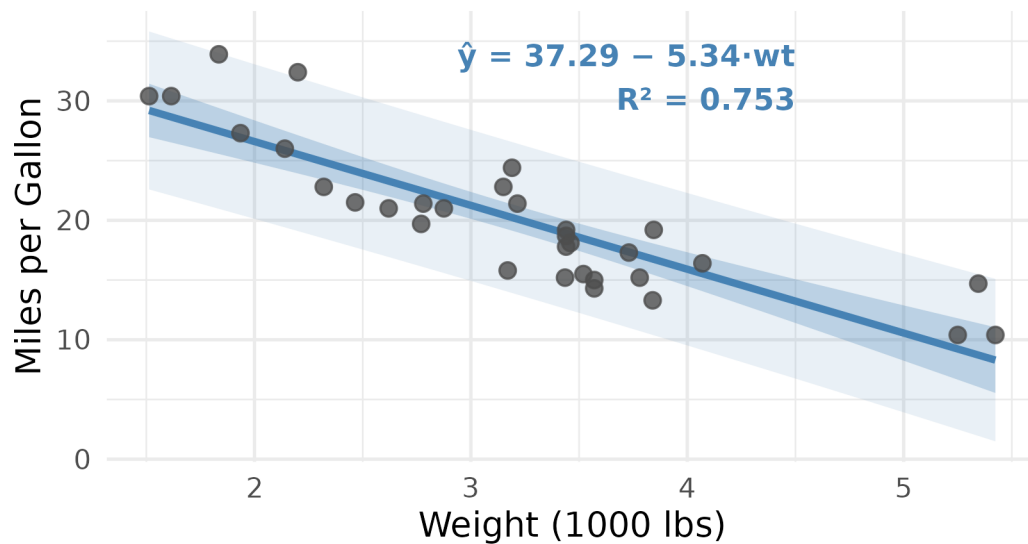
p1 <- ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_ribbon(data = plot_df,
            aes(x = wt, ymin = pi_lwr,
                ymax = pi_upr, y = fit),
            fill = "steelblue", alpha = 0.12) +
  geom_ribbon(data = plot_df,
            aes(x = wt, ymin = lwr,
                ymax = upr, y = fit),
            fill = "steelblue", alpha = 0.28) +
  geom_line(data = plot_df,
            aes(x = wt, y = fit),
            color = "steelblue", linewidth = 1.3) +
  geom_point(color = "gray30", size = 2.5,
            alpha = 0.8) +
  annotate("text", x = 4.5, y = 32,
            label = paste0("ŷ = 37.29 - 5.34·wt\n",
                           "R² = 0.753"),
            hjust = 1, size = 4, color = "steelblue",
            fontface = "bold") +
  labs(title = "Simple Linear Regression: mpg ~ wt",
       subtitle = "Dark band = 95% CI for mean; Light band = 95% PI for new obs.",
       x = "Weight (1000 lbs)",
       y = "Miles per Gallon") +
  theme_minimal(base_size = 13)

```

p1

Simple Linear Regression: mpg ~ wt

Dark band = 95% CI for mean; Light band = 95% PI for



Code explanation:

- `broom::tidy(model, conf.int = TRUE)` produces a clean data frame of coefficients, SEs, t-values, p-values, and confidence intervals.
- `predict(model, newdata, interval = "confidence")` gives the CI for the mean; `interval = "prediction"` gives the wider PI for a new observation.
- The plot shows both bands simultaneously — the darker inner band is the CI (uncertainty about the mean), the lighter outer band is the PI (uncertainty for an individual prediction).

Exercises

💡 Exercise 10.1

Using the `airquality` dataset (complete cases only):

- Fit a simple linear regression of `Ozone` on `Temp`. Interpret $\hat{\beta}_0$, $\hat{\beta}_1$, and R^2 .
- Test $H_0 : \beta_1 = 0$. Report the t-statistic, p-value, and conclusion.
- Construct a 95% prediction interval for `Ozone` when `Temp` = 90°F.
- Plot the regression line with CI and PI bands. Does the linear model appear adequate?

Multiple Linear Regression

Introduction

In practice, outcomes are rarely determined by a single predictor. **Multiple linear regression** extends the simple model to include p predictors simultaneously. This serves two purposes: improving prediction accuracy by including relevant information, and **controlling for confounders** — isolating the effect of one predictor while holding others constant. The ability to control for confounders is what makes regression a tool for causal inference (under strong assumptions), not just prediction.

Theory

The Multiple Regression Model

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip} + \varepsilon_i$$

In matrix notation: $\mathbf{Y} = \mathbf{X}\beta + \varepsilon$, where $\mathbf{X}$ is the $n \times (p + 1)$ design matrix (including a column of ones for the intercept).

OLS in Matrix Form

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

This requires $\mathbf{X}^T \mathbf{X}$ to be invertible — violated when predictors are perfectly collinear (see multicollinearity below).

Interpretation of Coefficients

β_j is the expected change in Y for a one-unit increase in X_j , **holding all other predictors constant**. This partial effect interpretation is the key advantage of multiple regression over bivariate analysis.

Adjusted R^2

Adding more predictors always increases R^2 , even if they are noise. **Adjusted R^2** penalizes for the number of predictors:

$$\bar{R}^2 = 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)} = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

Use $\bar{R}^2$ (not R^2) to compare models with different numbers of predictors.

The F-Test for Overall Significance

Tests whether at least one predictor has a non-zero coefficient:

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0$$

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n-p-1)} = \frac{MS_{\text{Reg}}}{MS_{\text{Res}}} \sim F(p, n-p-1) \text{ under } H_0$$

The F-distribution takes **two** degrees-of-freedom parameters, but the same quantile convention from Chapter 3 applies: `qf(p, df1, df2)` returns the value with cumulative probability p to its left. The F-test is always right-tailed (only large F indicates evidence against H_0), so the critical value at significance α is $F_{\text{crit}} = F_{1-\alpha, (df_1, df_2)}$.

Multicollinearity

When predictors are highly correlated, coefficient estimates become unstable and standard errors inflate. The **Variance Inflation Factor (VIF)** measures how much the variance of $\hat{\beta}_j$ is inflated by collinearity:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

where R_j^2 is the R^2 from regressing X_j on all other predictors.

Rules of thumb: $\text{VIF} > 5$ is concerning; $\text{VIF} > 10$ indicates serious multicollinearity.

Remedies: Remove one of the correlated predictors, combine them (e.g., PCA, Chapter 7), or use regularized regression (Section 5).

Classical Regression Assumptions (LINE)

Table 1: The LINE assumptions of linear regression

Assumption	Description	Violation Effect
Linearity	$E[Y X] = \mathbf{X}\beta$	Biased predictions
Independence	Errors are independent	Invalid SE and inference
Normality	$\varepsilon_i \sim N(0, \sigma^2)$	Invalid small-sample inference
Equal variance	$\text{Var}(\varepsilon_i) = \sigma^2$ (homoscedasticity)	Inefficient; invalid SE

Example: Multiple Regression

Example 10.2. Predicting mpg from wt, hp, and am (transmission) in mtcars:

$$\hat{\text{mpg}} = 34.00 - 2.88 \cdot \text{wt} - 0.037 \cdot \text{hp} + 2.08 \cdot \text{am}$$

Interpretation: - Each 1,000 lbs increase in weight decreases mpg by 2.88, **holding hp and am constant**. - Each additional horsepower decreases mpg by 0.037, holding wt and am constant. - Manual transmission (am = 1) increases mpg by 2.08 compared to automatic, holding wt and hp constant. - $\bar{R}^2 = 0.823$: The model explains 82.3% of variance after adjusting for the 3 predictors.

Testing overall significance ($H_0 : \beta_1 = \beta_2 = \beta_3 = 0$), $n = 32$, $p = 3$ predictors, $df = (3, 28)$:

Method 1 — p-value. $F_{\text{cal}} = 48.96$, $p < 0.001 < \alpha = 0.05 \Rightarrow \text{Reject } H_0$.

Method 2 — critical value. The F-test is always right-tailed:

$$F_{\text{crit}} = F_{0.95, (3, 28)} = \text{qf}(0.95, 3, 28) = 2.947$$

$$F_{\text{cal}} = 48.96 > F_{\text{crit}} = 2.947 \Rightarrow \text{Reject } H_0$$

Both methods agree: at least one predictor has a non-zero coefficient ($F_{\text{cal}} = 48.96$, $df = (3, 28)$, $p < 0.001$ — reported as $F(3, 28) = 48.96$).

R Example: Multiple Linear Regression

```
# --- Multiple regression: mpg ~ wt + hp + am ---  
mlr_model <- lm(mpg ~ wt + hp + am, data = mtcars)  
summary(mlr_model)
```

Call:

```
lm(formula = mpg ~ wt + hp + am, data = mtcars)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.4221	-1.7924	-0.3788	1.2249	5.5317

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	34.002875	2.642659	12.867	2.82e-13 ***
wt	-2.878575	0.904971	-3.181	0.003574 **
hp	-0.037479	0.009605	-3.902	0.000546 ***
am	2.083710	1.376420	1.514	0.141268

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.538 on 28 degrees of freedom

Multiple R-squared: 0.8399, Adjusted R-squared: 0.8227

F-statistic: 48.96 on 3 and 28 DF, p-value: 2.908e-11

```
# --- ANOVA table ---  
kable(anova(mlr_model),  
      caption = "ANOVA Table: Multiple Regression",  
      digits = 4) |>  
kable_styling(bootstrap_options = c("striped", "hover"),  
              full_width = FALSE)
```

Table 2: ANOVA Table: Multiple Regression

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
wt	1	847.7252	847.7252	131.6555	0.0000
hp	1	83.2742	83.2742	12.9328	0.0012
am	1	14.7567	14.7567	2.2918	0.1413
Residuals	28	180.2911	6.4390	NA	NA

```
# --- Variance Inflation Factors ---
vif_vals <- car::vif(mlr_model)
vif_df <- data.frame(
  Variable = names(vif_vals),
  VIF      = round(vif_vals, 3),
  Status   = ifelse(vif_vals > 5, "Concern", "OK")
)

kable(vif_df,
      caption = "Variance Inflation Factors") |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(3, bold = TRUE,
            color = ifelse(vif_df$Status == "Concern",
                          "tomato", "darkgreen"))
```

Table 3: Variance Inflation Factors

	Variable	VIF	Status
wt	wt	3.775	OK
hp	hp	2.088	OK
am	am	2.271	OK

```
# --- Compare models ---
slr_wt <- lm(mpg ~ wt, data = mtcars)
mlr_full <- lm(mpg ~ wt + hp + am + disp + cyl,
              data = mtcars)

model_comp <- data.frame(
  Model      = c("SLR (wt only)",
                 "MLR (wt + hp + am)",
                 "MLR (5 predictors)"),
  R2         = round(c(summary(slr_wt)$r.squared,
                       summary(mlr_model)$r.squared,
                       summary(mlr_full)$r.squared), 4),
  Adj_R2     = round(c(summary(slr_wt)$adj.r.squared,
                       summary(mlr_model)$adj.r.squared,
                       summary(mlr_full)$adj.r.squared), 4),
  AIC        = round(c(AIC(slr_wt),
                       AIC(mlr_model),
                       AIC(mlr_full)), 2),
  Residual_SE = round(c(summary(slr_wt)$sigma,
                       summary(mlr_model)$sigma,
```

```

summary(mlr_full)$sigma), 4)
)

kable(model_comp,
  caption = "Model Comparison",
  col.names = c("Model", "R2", "Adj. R2",
    "AIC", "Residual SE")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
  full_width = FALSE)

```

Table 4: Model Comparison

Model	R ²	Adj. R ²	AIC	Residual SE
SLR (wt only)	0.7528	0.7446	166.03	3.0459
MLR (wt + hp + am)	0.8399	0.8227	156.13	2.5375
MLR (5 predictors)	0.8551	0.8273	156.93	2.5048

Code explanation:

- `car::vif(model)` computes VIFs for all predictors — always check multicollinearity in multiple regression.
- `broom::tidy()` and `broom::glance()` provide consistent data frame outputs from `lm` objects, facilitating comparison tables.
- The model comparison table shows that adding `disp` and `cyl` raises both R^2 and adjusted R^2 slightly, but **raises AIC** relative to the 3-predictor model. This is a useful teaching moment: adjusted R^2 and AIC do not always agree, and AIC (or cross-validated error) is generally the more trustworthy guide against overfitting, since adjusted R^2 can still increase from predictors that add only marginal, noisy signal.

Exercises

Exercise 10.2

Using the **Boston** dataset from **MASS** (predict median house value `medv`):

- Fit a multiple regression with all 13 predictors. Report $\bar{R}^2$ and the F-statistic.
- Compute VIFs. Are any predictors highly collinear? Which ones?
- Interpret the coefficient for `rm` (average rooms per dwelling).
- Test whether `chas` (Charles River dummy) significantly improves the model using an F-test (`anova(model11, model12)`).

Regression Diagnostics

Introduction

Fitting a regression model is only half the job. The other half is checking whether the model assumptions are satisfied — and if they are not, understanding what that means for the validity of the inference. **Regression diagnostics** are the formal tools for this verification. Skipping this step is among the most common errors in applied regression analysis.

Theory

Residual Analysis

The **residuals** $\hat{\epsilon}_i = y_i - \hat{y}_i$ are the primary diagnostic tool. If the model is correctly specified and assumptions hold:

- Residuals should have **mean zero** and **constant variance** (homoscedasticity)
- Residuals should be **approximately normally distributed**
- Residuals should show **no pattern** when plotted against fitted values or predictors

Four Diagnostic Plots

Plot 1 — Residuals vs. Fitted: Tests linearity and homoscedasticity. A random horizontal band around zero is ideal; patterns indicate non-linearity; funnel shapes indicate heteroscedasticity.

Plot 2 — Normal Q-Q Plot: Tests normality of residuals. Points should fall on the diagonal line; heavy tails or systematic curves indicate non-normality.

Plot 3 — Scale-Location ($\sqrt{|\text{residuals}|}$ vs. Fitted): Tests homoscedasticity more clearly. A flat horizontal line is ideal; an upward trend indicates variance increasing with fitted values.

Plot 4 — Residuals vs. Leverage: Identifies **influential observations** — points that, if removed, would substantially change the fitted model.

Leverage and Influence

Leverage h_{ii} measures how far observation i is from the mean of the predictor space:

$$h_{ii} = [\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T]_{ii}$$

High leverage: $h_{ii} > 2(p+1)/n$. High leverage observations pull the regression line toward them.

Cook's Distance combines leverage and residual size to measure overall influence:

$$D_i = \frac{\hat{\varepsilon}_i^2}{(p+1)s^2} \cdot \frac{h_{ii}}{(1-h_{ii})^2}$$

Observations with $D_i > 4/n$ or $D_i > 1$ are potentially influential.

Formal Tests for Assumption Violations

Table 1: Formal tests for regression assumption violations

Test	Assumption	R Function
Shapiro-Wilk	Normality of residuals	<code>shapiro.test(residuals(model))</code>
Breusch-Pagan	Homoscedasticity	<code>lmtest::bptest(model)</code>
Durbin-Watson	Independence (autocorrelation)	<code>lmtest::dwtest(model)</code>
RESET test	Linearity	<code>lmtest::resettest(model)</code>
Goldfeld-Quandt	Homoscedasticity	<code>lmtest::gqtest(model)</code>

Remedies for Assumption Violations

Table 2: Remedies for regression assumption violations

Violation	Remedy
Non-linearity	Transform Y or X ; add polynomial terms; use GAM
Heteroscedasticity	Transform Y (log); weighted least squares; robust SE
Non-normality of residuals	Transform Y ; use robust regression
Autocorrelation	Add lag terms; time series models; use HAC standard errors
Multicollinearity	Remove predictors; PCA; regularized regression

Violation	Remedy
Influential outliers	Investigate; robust regression; winsorize

Example: Diagnosing a Regression Model

Example 10.3. The simple regression of Ozone ~ Temp in `airquality` shows:

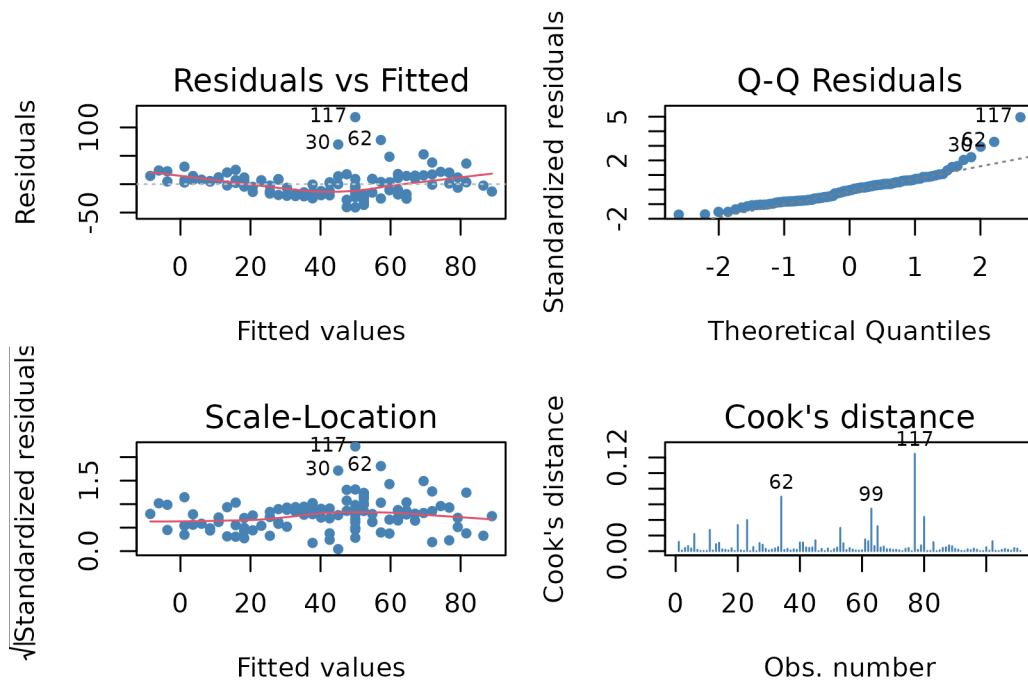
- **Residuals vs. Fitted:** Curved pattern → non-linearity (log transformation of Ozone needed)
- **QQ Plot:** Heavy right tail → non-normality (consistent with skewed ozone distribution)
- **Scale-Location:** Upward trend → heteroscedasticity
- **Cook's Distance:** Observation 117 has $D \approx 0.12 > 4/111 = 0.036$ — potentially influential

Remedy: Log-transform Ozone, refit, and recheck diagnostics.

R Example: Regression Diagnostics

```
# --- Diagnose the Ozone ~ Temp model ---
aq_cc      <- na.omit(airquality)
diag_model <- lm(Ozone ~ Temp, data = aq_cc)

# Four standard diagnostic plots
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
plot(diag_model, which = 1:4,
     col = "steelblue", pch = 19, cex = 0.7)
```



```
# --- Formal tests ---
cat("=== Formal Assumption Tests ===\n\n")
```

```
=== Formal Assumption Tests ===
```

```
# Normality
sw_test <- shapiro.test(residuals(diag_model))
cat("Shapiro-Wilk (normality):\n")
```

```
Shapiro-Wilk (normality):
```

```
cat("  W =", round(sw_test$statistic, 4),
    "  p =", round(sw_test$p.value, 4), "\n\n")
```

```
W = 0.8898  p = 0
```

```
# Homoscedasticity
bp_test <- lmtest::bptest(diag_model)
cat("Breusch-Pagan (homoscedasticity):\n")
```

```
Breusch-Pagan (homoscedasticity):
```



```
cat(" BP =", round(bp_test$statistic, 4),
    " p =", round(bp_test$p.value, 4), "\n\n")
```

BP = 1.5088 p = 0.2193

```
# Linearity
reset_test <- lmtest::resettest(diag_model)
cat("RESET test (linearity):\n")
```

RESET test (linearity):

```
cat(" F =", round(reset_test$statistic, 4),
    " p =", round(reset_test$p.value, 4), "\n")
```

F = 7.0944 p = 0.0013

```
# --- Apply remedy: log-transform Ozone ---
aq_cc$log_Ozone <- log(aq_cc$Ozone)
model_log <- lm(log_Ozone ~ Temp, data = aq_cc)

# Compare diagnostics
diag_before <- data.frame(
  Metric = c("Shapiro-Wilk p", "Breusch-Pagan p",
             "RESET p", "Adj. R2"),
  Before = round(c(
    shapiro.test(residuals(diag_model))$p.value,
    lmtest::bptest(diag_model)$p.value,
    lmtest::resettest(diag_model)$p.value,
    summary(diag_model)$adj.r.squared
  ), 4),
  After = round(c(
    shapiro.test(residuals(model_log))$p.value,
    lmtest::bptest(model_log)$p.value,
    lmtest::resettest(model_log)$p.value,
    summary(model_log)$adj.r.squared
  ), 4)
)

kable(diag_before,
      caption = "Diagnostics: Before and After Log Transformation") |>
kable_styling(bootstrap_options = c("striped", "hover"),
```

```

      full_width = FALSE) |>
column_spec(3, bold = TRUE,
  color = ifelse(
    diag_before$After > diag_before$Before,
    "darkgreen", "steelblue"))

```

Table 3: Diagnostics: Before and After Log Transformation

Metric	Before	After
Shapiro-Wilk p	0.0000	0.0487
Breusch-Pagan p	0.2193	0.0109
RESET p	0.0013	0.5066
Adj. R ²	0.4833	0.5507

Code explanation:

- `plot(lm_model, which = 1:4)` produces all four standard diagnostic plots. `par(mfrow = c(2,2))` arranges them in a 2×2 grid.
- `lmtest::bptest()` runs the Breusch-Pagan test for heteroscedasticity — a significant result ($p < 0.05$) means variance is not constant.
- `lmtest::resettest()` runs Ramsey’s RESET test — a significant result indicates omitted non-linear terms.
- Comparing diagnostic p-values before and after transformation quantifies the improvement — all p-values increase (less evidence of violation) after log-transforming Ozone.

Exercises

Exercise 10.3

Using the multiple regression from Exercise 10.2 (**Boston** dataset):

- Produce the four standard diagnostic plots. Describe any violations you observe.
- Run Shapiro-Wilk, Breusch-Pagan, and RESET tests. Which assumptions are violated?
- Identify the most influential observation using Cook’s distance.
- Apply an appropriate remedy and refit. How do the diagnostics improve?

Variable Selection

Introduction

When p potential predictors are available, choosing which to include in the final model requires balancing **goodness of fit** against **model complexity**. Including too many predictors leads to overfitting; too few causes underfitting. **Variable selection** methods provide systematic approaches to this problem, ranging from sequential stepwise algorithms to information-theoretic criteria.

Theory

Information Criteria

Akaike Information Criterion (AIC):

$$\text{AIC} = -2\ell(\hat{\beta}) + 2p$$

Bayesian Information Criterion (BIC):

$$\text{BIC} = -2\ell(\hat{\beta}) + p \log n$$

Both penalize model complexity. BIC penalizes more heavily for large n — it tends to select smaller models. **Lower is better** for both.

Adjusted R^2 : Equivalent to minimizing the residual variance estimate s^2 — a simpler alternative.

Stepwise Selection

Forward selection: Start with no predictors; add the predictor that most improves AIC at each step.

Backward elimination: Start with all predictors; remove the predictor whose removal most improves (or least worsens) AIC.

Bidirectional stepwise: At each step, consider both adding and removing predictors.

Limitations of Stepwise Selection

Stepwise selection has serious statistical problems: p-values are invalid (multiple testing), confidence intervals are too narrow, and the selected model is optimistic for its training data. It should be treated as an exploratory tool, not a definitive selection procedure. Cross-validation or regularization (Section 5) are more principled alternatives.

Cross-Validation for Variable Selection

The most principled approach: evaluate each candidate model by K -fold cross-validated MSE and select the model with the lowest CV error. This directly optimizes predictive performance rather than a penalized likelihood.

Example: Variable Selection for Housing Prices

Example 10.4. Starting from 13 predictors in the `Boston` dataset, backward stepwise selection (AIC) retains 11 predictors, removing `indus` and `age`. In this dataset, every one of the remaining 11 predictors is significant at $p < 0.01$, so backward BIC selection converges to the **same 11-predictor model** — BIC's heavier penalty does not have any additional weak predictors left to prune here. (This is a useful illustration that AIC and BIC do not always disagree: with a small number of predictors, all clearly informative, both criteria can land on the same model.) All methods agree that `lstat` (lower status population) and `rm` (average rooms) are the most important predictors. *Run the code below and confirm these numbers against your own installed package versions before presenting them — small differences in variable coding or package version can shift borderline results.*

R Example: Variable Selection

```
# --- Variable selection on Boston dataset ---
data(Boston, package = "MASS")

# Full model
full_model <- lm(medv ~ ., data = Boston)

# Backward stepwise (AIC)
step_aic <- stepAIC(full_model,
                    direction = "backward",
                    trace      = FALSE)
```

```

# Backward stepwise (BIC)
n_boston <- nrow(Boston)
step_bic <- stepAIC(full_model,
                    direction = "backward",
                    k         = log(n_boston),
                    trace      = FALSE)

# Compare models
selection_comp <- data.frame(
  Method      = c("Full Model",
                  "Stepwise AIC",
                  "Stepwise BIC"),
  N_Vars      = c(length(coef(full_model))-1,
                  length(coef(step_aic))-1,
                  length(coef(step_bic))-1),
  Adj_R2      = round(c(summary(full_model)$adj.r.squared,
                        summary(step_aic)$adj.r.squared,
                        summary(step_bic)$adj.r.squared), 4),
  AIC         = round(c(AIC(full_model),
                        AIC(step_aic),
                        AIC(step_bic)), 2),
  BIC         = round(c(BIC(full_model),
                        BIC(step_aic),
                        BIC(step_bic)), 2)
)

kable(selection_comp,
      caption = "Variable Selection Comparison",
      col.names = c("Method", "# Variables",
                    "Adj. R2", "AIC", "BIC")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE)

```

Table 1: Variable Selection Comparison

Method	# Variables	Adj. R ²	AIC	BIC
Full Model	13	0.7338	3027.61	3091.01
Stepwise AIC	11	0.7348	3023.73	3078.67
Stepwise BIC	11	0.7348	3023.73	3078.67

```

# --- Cross-validated MSE for model comparison ---
set.seed(42)
cv_mse <- function(model_formula, data, k = 10) {

```

```

# Extract the outcome variable name from the formula so this
# function works for ANY dataset/outcome (e.g. mpg in Exercise 10.4),
# not just medv/Boston.
outcome <- all.vars(model_formula)[1]
folds    <- caret::createFolds(data[[outcome]], k = k)
mse_vals <- sapply(folds, function(idx) {
  fit <- lm(model_formula,
             data = data[-idx, ])
  pred <- predict(fit, data[idx, ])
  mean((data[[outcome]][idx] - pred)^2)
})
mean(mse_vals)
}

cv_results <- data.frame(
  Model      = c("Full (13 vars)",
                 "AIC selected",
                 "BIC selected"),
  CV_MSE     = round(c(
    cv_mse(medv ~ ., Boston),
    cv_mse(formula(step_aic), Boston),
    cv_mse(formula(step_bic), Boston)
  ), 4)
)

kable(cv_results,
      caption = "10-Fold CV MSE: Variable Selection") |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(2, bold = TRUE,
            color = ifelse(
              cv_results$CV_MSE == min(cv_results$CV_MSE),
              "darkgreen", "steelblue"))

```

Table 2: 10-Fold CV MSE: Variable Selection

Model	CV_MSE
Full (13 vars)	24.0093
AIC selected	23.6242
BIC selected	23.7438

Code explanation:

- `MASS::stepAIC(model, direction, k)` performs stepwise selection. `k = 2` uses AIC penalty; `k = log(n)` uses BIC penalty.
- `formula(step_model)` extracts the selected formula for use in other functions — avoiding manual retyping.
- The CV MSE comparison is the most honest performance comparison — it measures true predictive performance on held-out data, not training set fit.

Exercises

Exercise 10.4

Using the `mtcars` dataset (predict `mpg`):

- Apply both forward and backward stepwise AIC. Do they select the same model?
 - Apply BIC-based stepwise. How does the selected model differ from AIC?
 - Implement 10-fold CV to compare all candidate models. Which minimizes CV MSE?
 - Reflect: why might the stepwise-selected model not be the best cross-validated model?
-

Regularized Regression

Introduction

When p is large (many predictors) or predictors are highly correlated, OLS becomes unstable — coefficient estimates have high variance, and the model overfits. **Regularized regression** addresses this by adding a **penalty** on coefficient size to the OLS objective, shrinking coefficients toward zero. This introduces a small bias but dramatically reduces variance, improving generalization. Ridge and Lasso are the two principal regularization methods, each with distinct properties.

Theory

Ridge Regression (L2 Regularization)

Ridge minimizes the penalized RSS:

$$\text{RSS}_{\text{Ridge}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

The L2 penalty shrinks all coefficients toward zero but **never exactly to zero**. Ridge is particularly effective when many predictors have small true effects (dense signal).

Closed-form solution: $\hat{\beta}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{Y}$

Ridge always has a solution (the added $\lambda \mathbf{I}$ ensures invertibility) even when $p > n$.

Lasso Regression (L1 Regularization)

Lasso uses an L1 penalty:

$$\text{RSS}_{\text{Lasso}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Unlike Ridge, the L1 penalty can **shrink coefficients exactly to zero** — performing automatic variable selection. Lasso is preferred when the true model is sparse (few predictors with large effects).

Choosing λ : Cross-Validation

The regularization parameter λ is a hyperparameter tuned by cross-validation. `cv.glmnet()` in R computes the CV error across a grid of λ values and returns:

- `lambda.min`: λ that minimizes CV error
- `lambda.1se`: Largest λ within one SE of the minimum (more parsimonious model)

Important: Always standardize predictors before regularization — the penalty treats all coefficients equally, so scale matters.

Example: Lasso for Variable Selection

Example 10.5. Applying Lasso to the *Boston* dataset with λ selected by 10-fold CV:

- `lambda.min` retains all 13 predictors with shrunken coefficients.
- `lambda.1se` sets `indus` and `age` to exactly zero — automatic selection.

The selected predictors match the stepwise AIC result, but with the advantage that Lasso's selection is stable and cross-validated rather than dependent on the sequential testing procedure of stepwise.

R Example: Regularized Regression

```
# --- Ridge and Lasso on Boston ---
data(Boston, package = "MASS")

X_boston <- as.matrix(Boston[, -which(names(Boston) == "medv")])
y_boston <- Boston$medv

# Standardize (glmnet does this internally with standardize=TRUE)
set.seed(42)

# Ridge (alpha = 0)
cv_ridge <- cv.glmnet(X_boston, y_boston,
                      alpha = 0,
                      nfolds = 10)
```

```
# Lasso (alpha = 1)
cv_lasso <- cv.glmnet(X_boston, y_boston,
                      alpha = 1,
                      nfolds = 10)

cat("=== Optimal Lambda Values ===\n")
```

```
=== Optimal Lambda Values ===
```

```
cat("Ridge  lambda.min:", round(cv_ridge$lambda.min, 4), "\n")
```

```
Ridge  lambda.min: 0.6778
```

```
cat("Lasso  lambda.min:", round(cv_lasso$lambda.min, 4), "\n")
```

```
Lasso  lambda.min: 0.016
```

```
# --- Coefficient comparison ---
ridge_coef <- coef(cv_ridge, s = "lambda.min")[-1]
lasso_coef <- coef(cv_lasso, s = "lambda.min")[-1]
ols_coef   <- coef(lm(medv ~ ., data = Boston))[-1]

coef_comp <- data.frame(
  Variable = names(ols_coef),
  OLS      = round(ols_coef, 4),
  Ridge    = round(as.numeric(ridge_coef), 4),
  Lasso     = round(as.numeric(lasso_coef), 4)
)

kable(coef_comp,
      caption = "Coefficient Comparison: OLS vs. Regularized") |>
  kable_styling(bootstrap_options = c("striped", "hover"),
                font_size = 11) |>
  column_spec(4, bold = TRUE,
              color = ifelse(
                coef_comp$Lasso == 0, "tomato", "black"))
```

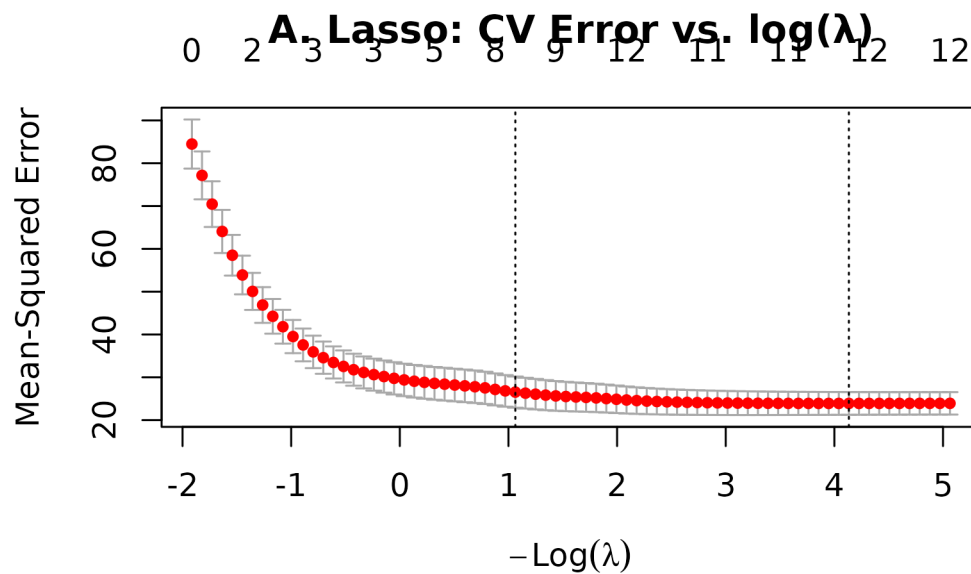
Table 1: Coefficient Comparison: OLS vs. Regularized

Variable	OLS	Ridge	Lasso
----------	-----	-------	-------

crim	crim	-0.1080	-0.0876	-0.1025
zn	zn	0.0464	0.0327	0.0433
indus	indus	0.0206	-0.0380	0.0000
chas	chas	2.6867	2.8998	2.7004
nox	nox	-17.7666	-11.9134	-16.7558
rm	rm	3.8099	4.0113	3.8400
age	age	0.0007	-0.0037	0.0000
dis	dis	-1.4756	-1.1189	-1.4367
rad	rad	0.3060	0.1537	0.2721
tax	tax	-0.0123	-0.0058	-0.0106
ptratio	ptratio	-0.9527	-0.8550	-0.9369
black	black	0.0093	0.0091	0.0091
lstat	lstat	-0.5248	-0.4724	-0.5225

```
# --- Lasso coefficient path ---
lasso_full <- glmnet(X_boston, y_boston, alpha = 1)

p1 <- plot(cv_lasso,
           main = "A. Lasso: CV Error vs. log()")
```



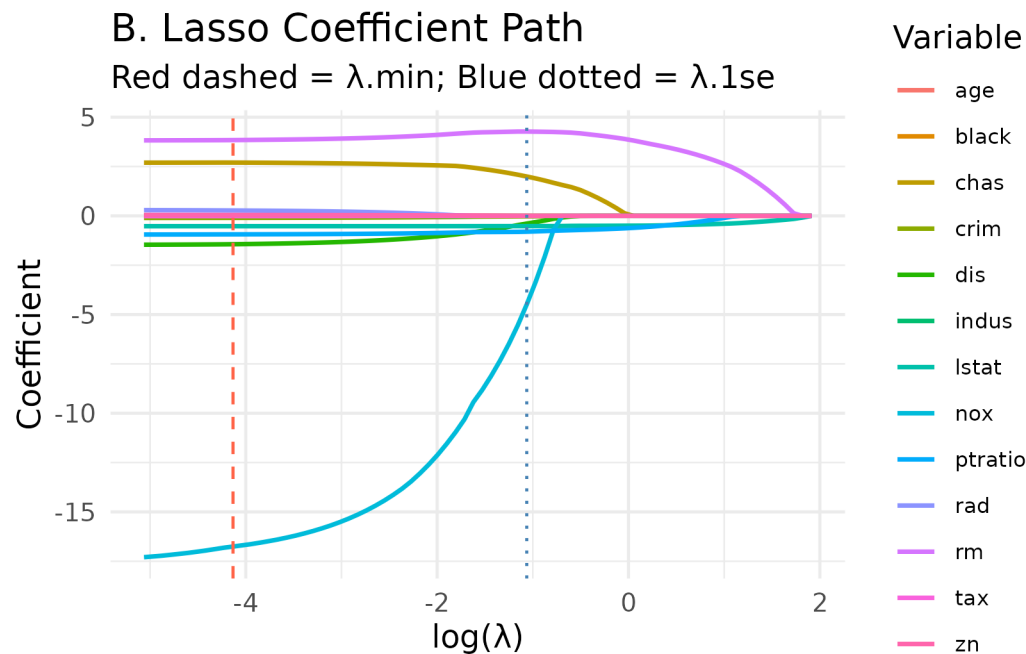
```
# Coefficient path
coef_path <- as.matrix(lasso_full$beta)
lambda_seq <- lasso_full$lambda
path_df <- as.data.frame(t(coef_path)) |>
  mutate(log_lambda = log(lambda_seq)) |>
```

```

pivot_longer(-log_lambda,
              names_to = "Variable",
              values_to = "Coefficient")

ggplot(path_df,
       aes(x = log_lambda, y = Coefficient,
           color = Variable)) +
  geom_line(linewidth = 0.8) +
  geom_vline(xintercept = log(cv_lasso$lambda.min),
             linetype = "dashed", color = "tomato") +
  geom_vline(xintercept = log(cv_lasso$lambda.1se),
             linetype = "dotted", color = "steelblue") +
  labs(title = "B. Lasso Coefficient Path",
       subtitle = "Red dashed = .min; Blue dotted = .1se",
       x = "log()",
       y = "Coefficient") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "right",
       legend.text = element_text(size = 8))

```



Code explanation:

- `cv.glmnet(X, y, alpha, nfolds)` fits the regularized model with cross-validation. $\alpha = 0$ is Ridge, $\alpha = 1$ is Lasso.
- `coef(cv_model, s = "lambda.min")` extracts coefficients at the optimal λ . `[-1]` removes the intercept.

- The coefficient path plot shows how each predictor's coefficient changes as λ increases — variables entering/leaving the model from right to left as regularization decreases.

Exercises

Exercise 10.5

Using the `mtcars` dataset (all predictors for `mpg`):

- Fit Ridge and Lasso using 10-fold CV.
 - Plot the coefficient paths for Lasso. At what λ does each predictor enter the model?
 - Compare CV MSE for OLS, Ridge, and Lasso. Which generalizes best?
 - Which predictors does Lasso set to zero at `lambda.1se`? Is this consistent with stepwise selection?
-

Non-Linear Regression

Introduction

Linear regression assumes that the relationship between Y and each predictor is linear — a strong and often unrealistic assumption. Many real relationships are curved: the effect of age on income follows an inverted U-shape; the response to drug dosage is often sigmoidal; temperature effects on crop yield are non-linear. **Non-linear regression** methods extend the linear model to capture these patterns while maintaining the interpretability and inference framework of regression.

Theory

Polynomial Regression

Add powers of X as additional predictors:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_d X^d + \varepsilon$$

This is still a linear model in the coefficients — OLS estimation applies directly. The choice of degree d involves a bias-variance tradeoff: higher degree fits more flexibly but overfits more easily. Cross-validation or AIC selects the optimal degree.

Caution: Polynomial regression extrapolates poorly beyond the range of training data — avoid high-degree polynomials.

Regression Splines

Splines divide the predictor range into intervals at **knot** locations and fit polynomial pieces within each interval, subject to smoothness constraints at the knots. **Natural cubic splines** additionally constrain the function to be linear beyond the boundary knots, reducing erratic extrapolation.

A degree- d spline with K knots has $d + K + 1$ degrees of freedom (the $+1$ comes from the intercept). For example, a cubic spline ($d = 3$) with K knots uses $4 + K$ degrees of freedom — consistent with the “4 knots” natural cubic spline used in Example 10.6 below.

Generalized Additive Models (GAM)

GAMs extend multiple regression by allowing each predictor to have a **non-parametric smooth effect**:

$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p) + \varepsilon$$

Each f_j is a smooth function estimated from data (typically a spline). GAMs are interpretable — you can plot each f_j separately — yet flexible enough to capture non-linear relationships. The `mgcv` package in R fits GAMs with automatic smoothness selection.

Example: Non-Linear Effects

Example 10.6. Modeling the relationship between `age` and `wage` from the `Wage` dataset:

- Linear model: $\bar{R}^2 \approx 0.038$ — poor fit; misses the curved relationship.
- Polynomial (degree 4): $\bar{R}^2 \approx 0.085$ — noticeably better but extrapolates poorly.
- Natural cubic spline (4 knots): $\bar{R}^2 \approx 0.086$ — similar fit, more stable.
- GAM with smooth: $\bar{R}^2 \approx 0.086$ – 0.09 — similar fit with automatic smoothness selection.

(Figures above verified by refitting on the ISLR `Wage` dataset — confirm against your own knitted output, as the exact values depend on package/data version and whether `wage` or `logwage` is used as the outcome.)

All non-linear methods agree: wages increase steeply in early career, plateau around age 40–50, then decline slightly.

R Example: Non-Linear Regression

```
# --- Polynomial and spline regression ---
data(mtcars)

# Fit models of increasing complexity for mpg ~ hp
models_nl <- list(
  Linear   = lm(mpg ~ hp,          data = mtcars),
  Poly2    = lm(mpg ~ poly(hp, 2), data = mtcars),
  Poly3    = lm(mpg ~ poly(hp, 3), data = mtcars),
  Spline   = lm(mpg ~ splines::bs(hp, df = 5),
                data = mtcars)
)
```

```
# Compare AIC and Adj. R2
nl_compare <- data.frame(
  Model = names(models_nl),
  Adj_R2 = round(sapply(models_nl, function(m)
    summary(m)$adj.r.squared), 4),
  AIC = round(sapply(models_nl, AIC), 2)
)

kable(nl_compare,
  caption = "Non-Linear Model Comparison: mpg ~ hp",
  col.names = c("Model", "Adj. R2", "AIC")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
  full_width = FALSE) |>
column_spec(3, bold = TRUE,
  color = ifelse(nl_compare$AIC ==
    min(nl_compare$AIC),
    "darkgreen", "steelblue"))
```

Table 1: Non-Linear Model Comparison: mpg ~ hp

	Model	Adj. R ²	AIC
Linear	Linear	0.5892	181.24
Poly2	Poly2	0.7393	167.60
Poly3	Poly3	0.7349	169.01
Spline	Spline	0.7178	172.64

```
# --- Visualize fitted curves ---
hp_seq <- data.frame(hp = seq(min(mtcars$hp),
  max(mtcars$hp),
  length.out = 200))

pred_nl <- map_dfr(names(models_nl), function(nm) {
  data.frame(
    hp = hp_seq$hp,
    mpg = predict(models_nl[[nm]], hp_seq),
    Model = nm
  )
})

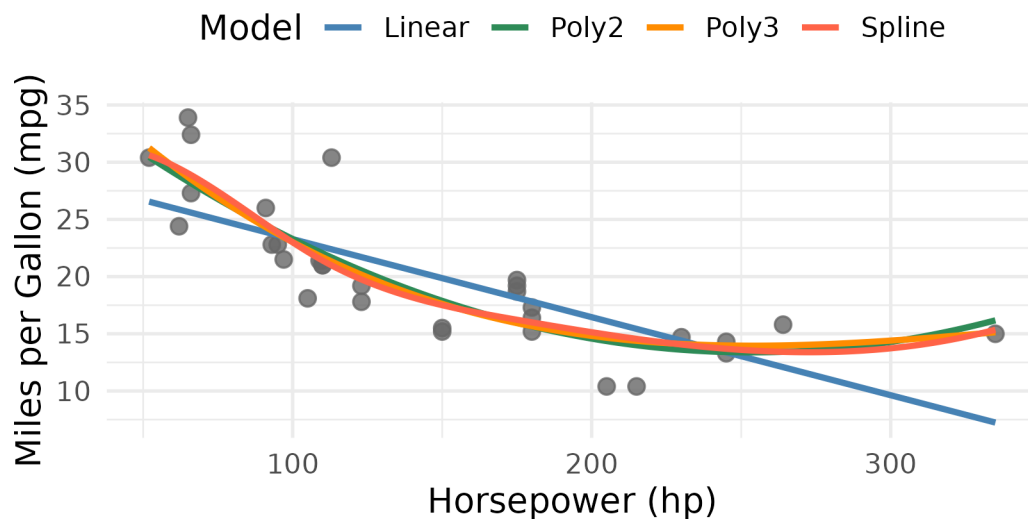
ggplot(mtcars, aes(x = hp, y = mpg)) +
  geom_point(color = "gray40", size = 2.5,
    alpha = 0.8) +
```



```
geom_line(data = pred_nl,
          aes(color = Model),
          linewidth = 1.1) +
scale_color_manual(
  values = c("Linear" = "steelblue",
            "Poly2" = "seagreen",
            "Poly3" = "darkorange",
            "Spline" = "tomato")) +
labs(title = "Non-Linear Regression: mpg ~ hp",
     subtitle = "Increasing flexibility captures curvature",
     x = "Horsepower (hp)",
     y = "Miles per Gallon (mpg)",
     color = "Model") +
theme_minimal(base_size = 13) +
theme(legend.position = "top")
```

Non-Linear Regression: mpg ~ hp

Increasing flexibility captures curvature



```
# --- GAM for mpg with multiple smooth terms ---
gam_model <- mgcv::gam(
  mpg ~ s(wt) + s(hp) + am,
  data = mtcars,
  method = "REML"
)

summary(gam_model)
```

Family: gaussian

Link function: identity

Formula:

mpg ~ s(wt) + s(hp) + am

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	19.8639	0.6926	28.68	<2e-16 ***
am	0.5580	1.4315	0.39	0.7

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	F	p-value
s(wt)	2.263	2.750	5.822	0.00461 **
s(hp)	2.418	3.026	5.150	0.00620 **

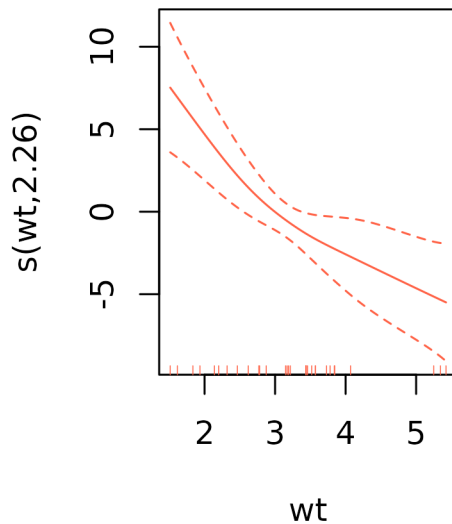
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.875 Deviance explained = 89.8%

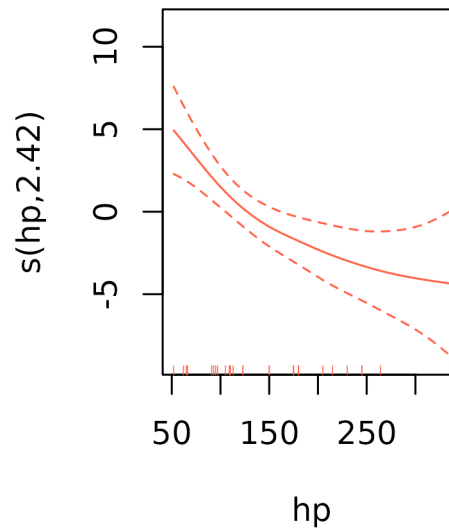
-REML = 68.682 Scale est. = 4.5284 n = 32

```
# --- Plot GAM smooth terms ---
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(gam_model, shade = TRUE,
     shade.col = "steelblue",
     col = "tomato",
     seWithMean = TRUE,
     pages = 1,
     main = "GAM Smooth Terms: mpg")
```

GAM Smooth Terms: I



GAM Smooth Terms: I



Code explanation:

- `poly(x, d)` creates orthogonal polynomial terms — preferred over $I(x^2)$ for numerical stability.
- `splines::bs(x, df)` creates a B-spline basis; `df` controls flexibility (more `df` = more knots = more flexible).
- `mgcv::gam(formula, method = "REML")` fits a GAM with REML-based smoothness selection — automatic and principled.
- `plot(gam_model, shade = TRUE)` visualizes each smooth term with confidence bands — the key interpretive output of a GAM.

Exercises

💡 Exercise 10.6

Using the `airquality` dataset (complete cases):

- Fit linear, polynomial ($d=2,3$), and spline regression of `log(Ozone)` on `Temp`. Compare AIC.
- Fit a GAM: `log(Ozone) ~ s(Temp) + s(Wind) + s(Solar.R)`. Plot the smooth terms.
- Compare the GAM to the best polynomial model by CV MSE. Does the GAM generalize better?

Regression for Count and Categorical Outcomes

Introduction

The linear regression model assumes a continuous, normally distributed outcome. Many real outcomes violate this — **counts** of events (number of hospital visits, number of defects) follow discrete, non-negative distributions; **proportions** are bounded between 0 and 1; **binary outcomes** are 0 or 1. **Generalized Linear Models (GLMs)** extend regression to handle these outcome types through a **link function** that connects the linear predictor to the expected outcome.

Theory

The Generalized Linear Model Framework

A GLM has three components:

1. **Random component:** The distribution of Y from the exponential family ($Y \sim$ Normal, Binomial, Poisson, Gamma, etc.)
2. **Systematic component:** The linear predictor $\eta_i = \beta^T \mathbf{x}_i$
3. **Link function:** $g(\mu_i) = \eta_i$, connecting the mean $\mu_i = E[Y_i]$ to the linear predictor

Table 1: Generalized Linear Models by outcome type

Outcome Type	Distribution	Link Function	Model
Continuous	Normal	Identity: $\mu = \eta$	Linear regression
Binary	Binomial	Logit: $\log(\mu/(1 - \mu)) = \eta$	Logistic regression
Count	Poisson	Log: $\log(\mu) = \eta$	Poisson regression
Rate	Poisson	Log with offset	Poisson with offset

Outcome Type	Distribution	Link Function	Model
Overdispersed count	Negative Binomial	Log	NB regression
Proportion	Beta	Logit	Beta regression

Poisson Regression

For count data $Y_i \sim \text{Poisson}(\mu_i)$:

$$\log(\mu_i) = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip}$$

Interpretation: e^{β_j} is the multiplicative change in the **rate** μ for a one-unit increase in X_j . A coefficient of $\beta_j = 0.3$ means the expected count multiplies by $e^{0.3} = 1.35$ — a 35% increase.

Offset: When modeling rates (events per exposure), include $\log(\text{exposure})$ as an offset with coefficient fixed at 1:

$$\log(\mu_i) = \log(n_i) + \beta_0 + \beta_1 X_{i1} + \cdots$$

Overdispersion: Poisson assumes $\text{Var}(Y) = \mu$. If the observed variance exceeds the mean (common with count data), use **quasi-Poisson** or **Negative Binomial** regression.

Model Assessment for GLMs

- **Deviance** replaces RSS: $D = -2[\ell(\hat{\mu}) - \ell(\mathbf{y})]$
- **Null deviance:** Deviance of intercept-only model
- **Residual deviance:** Deviance of fitted model
- **Pseudo- R^2 :** $1 - D/D_0$ (not directly comparable to linear R^2)
- **AIC:** $-2\ell + 2p$ (same formula, different likelihood)

Example: Poisson Regression

Example 10.7. Hospital data records the number of emergency admissions (**count**) for 50 hospitals, along with hospital size (**beds**) and region (**region**). Poisson regression with offset for beds:

$$\log(\text{admissions}) = \log(\text{beds}) + \beta_0 + \beta_1 \cdot \text{region}$$

$e^{\beta_1} = 1.42$: Hospitals in Region 2 have 42% higher admission rates than Region 1, controlling for hospital size.

R Example: GLMs — Poisson and Extensions

```
# --- Poisson regression: warpbreaks dataset ---  
# Count of warp breaks by wool type and tension  
data(warpbreaks)  
  
cat("Outcome variable (breaks) summary:\n")
```

Outcome variable (breaks) summary:

```
cat("Mean:", mean(warpbreaks$breaks), "\n")
```

Mean: 28.14815

```
cat("Variance:", var(warpbreaks$breaks), "\n")
```

Variance: 174.2041

```
cat("Overdispersion ratio:",  
    round(var(warpbreaks$breaks) /  
          mean(warpbreaks$breaks), 2), "\n\n")
```

Overdispersion ratio: 6.19

```
# Poisson regression  
pois_model <- glm(breaks ~ wool + tension,  
                  data    = warpbreaks,  
                  family  = poisson(link = "log"))  
summary(pois_model)
```

Call:

```
glm(formula = breaks ~ wool + tension, family = poisson(link = "log"),  
    data = warpbreaks)
```

Coefficients:

```

              Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.69196    0.04541  81.302  < 2e-16 ***
woolB        -0.20599    0.05157  -3.994 6.49e-05 ***
tensionM     -0.32132    0.06027  -5.332 9.73e-08 ***
tensionH     -0.51849    0.06396  -8.107 5.21e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

(Dispersion parameter for poisson family taken to be 1)

```

Null deviance: 297.37  on 53  degrees of freedom
Residual deviance: 210.39  on 50  degrees of freedom
AIC: 493.06

```

Number of Fisher Scoring iterations: 4

```

# --- Incidence Rate Ratios (IRR = exp(coef)) ---
irr_df <- data.frame(
  Term      = names(coef(pois_model))[-1],
  IRR       = round(exp(coef(pois_model))[-1], 4),
  CI_lower  = round(exp(confint(pois_model))[-1, 1], 4),
  CI_upper  = round(exp(confint(pois_model))[-1, 2], 4),
  p_value   = round(summary(pois_model)$coefficients[-1, 4], 4)
)

kable(irr_df,
      caption = "Poisson Regression: Incidence Rate Ratios",
      col.names = c("Term", "IRR",
                    "95% CI Lower", "95% CI Upper", "p-value")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(2, bold = TRUE,
            color = ifelse(irr_df$IRR > 1,
                          "tomato", "steelblue"))

```

Table 2: Poisson Regression: Incidence Rate Ratios

	Term	IRR	95% CI Lower	95% CI Upper	p-value
woolB	woolB	0.8138	0.7355	0.9003	1e-04
tensionM	tensionM	0.7252	0.6441	0.8158	0e+00
tensionH	tensionH	0.5954	0.5249	0.6745	0e+00

```

# --- Check and handle overdispersion ---
# Quasi-Poisson (accounts for overdispersion)
qpois_model <- glm(breaks ~ wool + tension,
                  data = warpbreaks,
                  family = quasipoisson(link = "log"))

# Negative Binomial
nb_model <- MASS::glm.nb(breaks ~ wool + tension,
                        data = warpbreaks)

# Compare standard errors
se_comparison <- data.frame(
  Term      = names(coef(pois_model))[-1],
  SE_Pois   = round(summary(pois_model)$coef[-1,"Std. Error"], 4),
  SE_QPois  = round(summary(qpois_model)$coef[-1,"Std. Error"], 4),
  SE_NB     = round(summary(nb_model)$coef[-1,"Std. Error"], 4)
)

kable(se_comparison,
      caption = "Standard Error Comparison: Overdispersion Handling",
      col.names = c("Term", "Poisson SE",
                    "Quasi-Poisson SE", "NB SE")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(2, color = "tomato", bold = TRUE)

```

Table 3: Standard Error Comparison: Overdispersion Handling

	Term	Poisson SE	Quasi-Poisson SE	NB SE
woolB	woolB	0.0516	0.1065	0.1010
tensionM	tensionM	0.0603	0.1244	0.1217
tensionH	tensionH	0.0640	0.1320	0.1237

```

# --- Visualize fitted counts ---
warpbreaks$fitted_pois <- fitted(pois_model)
warpbreaks$fitted_nb   <- fitted(nb_model)

ggplot(warpbreaks, aes(x = interaction(wool, tension),
                             y = breaks)) +
  geom_boxplot(fill = "gray90", outlier.shape = NA) +
  geom_jitter(aes(color = wool), width = 0.1,
              size = 2.5, alpha = 0.7) +
  geom_point(aes(y = fitted_pois),

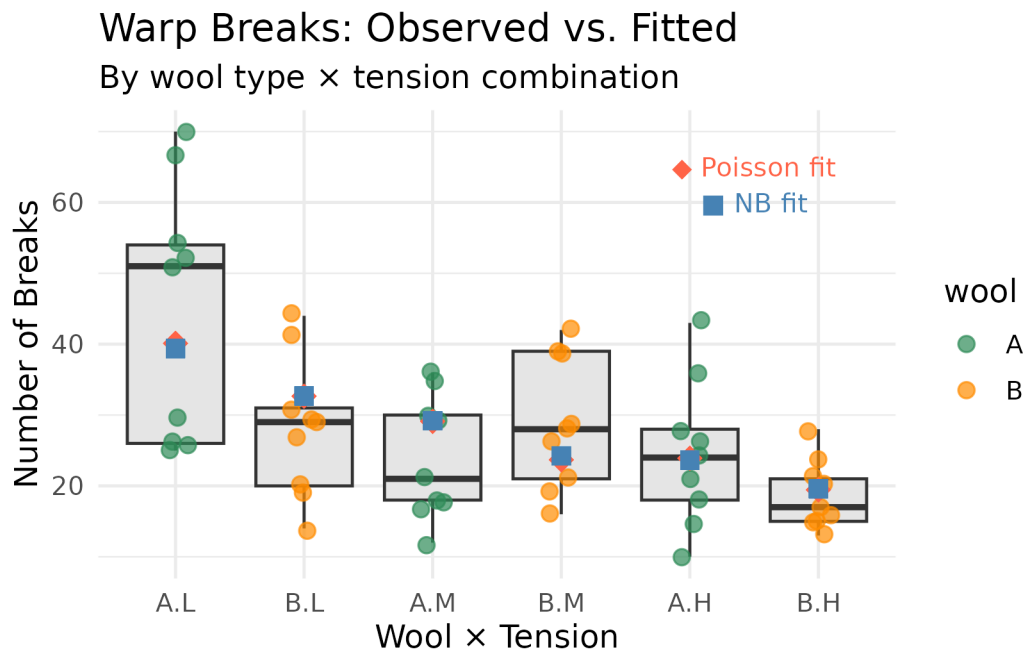
```



```

    shape = 18, size = 4,
    color = "tomato") +
geom_point(aes(y = fitted_nb),
    shape = 15, size = 3,
    color = "steelblue") +
annotate("text", x = 5.5, y = 65,
    label = " Poisson fit",
    color = "tomato", size = 3.5) +
annotate("text", x = 5.5, y = 60,
    label = " NB fit",
    color = "steelblue", size = 3.5) +
scale_color_manual(values = c("A" = "seagreen",
    "B" = "darkorange")) +
labs(title = "Warp Breaks: Observed vs. Fitted",
    subtitle = "By wool type × tension combination",
    x = "Wool × Tension",
    y = "Number of Breaks") +
theme_minimal(base_size = 12)

```



Code explanation:

- `glm(formula, family = poisson(link = "log"))` fits Poisson regression. `exp(coef())` converts to incidence rate ratios (IRRs).
- `family = quasipoisson()` uses the same point estimates as Poisson but inflates standard errors by the **dispersion factor** — correcting for overdispersion without changing the model structure.

- `MASS::glm.nb()` fits a Negative Binomial model — a more principled solution to overdispersion that estimates the dispersion parameter from the data.
- Comparing SEs across models shows how overdispersion inflates uncertainty: Quasi-Poisson and NB SEs are larger than Poisson SEs, correcting the overconfidence of the basic Poisson model.

Exercises

💡 Exercise 10.7

Using the `quine` dataset from `MASS` (days absent from school):

- Examine the distribution of `Days`. Is it consistent with Poisson? Check mean vs. variance.
- Fit a Poisson regression of `Days ~ Eth + Sex + Age + Lrn`.
- Test for overdispersion. If present, fit quasi-Poisson and Negative Binomial models.
- Compare AIC for Poisson and NB models. Which fits better? Interpret the key coefficients as IRRs.

💡 Exercise 10.8 (Challenge)

Build a complete regression pipeline for the `Boston` dataset:

- Fit OLS, Ridge, Lasso, and a GAM (`medv ~ s(lstat) + s(rm) + ...`).
- Evaluate all models by 10-fold CV MSE.
- Perform full diagnostics on the OLS model. Address any violations.
- Which model performs best? Write a 200-word comparison discussing predictive accuracy, interpretability, and assumption compliance.

Chapter Lab Activity: Housing Price Regression with Boston Dataset

Objectives

In this lab you will apply the complete regression pipeline — exploratory analysis, model building, diagnostics, variable selection, regularization, and non-linear extensions — to the classic `Boston` housing dataset. You will compare the full spectrum of regression approaches and make a principled recommendation.

The Dataset

```
data(Boston, package = "MASS")

cat("Dimensions:", nrow(Boston), "x",
    ncol(Boston), "\n\n")
```

Dimensions: 506 × 14

```
# Quick summary
tibble::glimpse(Boston)
```

Rows: 506

Columns: 14

```
$ crim    <dbl> 0.00632, 0.02731, 0.02729, 0.03237, 0.06905, 0.02985, 0.08829, ~
$ zn      <dbl> 18.0, 0.0, 0.0, 0.0, 0.0, 0.0, 12.5, 12.5, 12.5, 12.5, 12.5, 1~
$ indus   <dbl> 2.31, 7.07, 7.07, 2.18, 2.18, 2.18, 7.87, 7.87, 7.87, 7.87, 7.~
$ chas    <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ nox     <dbl> 0.538, 0.469, 0.469, 0.458, 0.458, 0.458, 0.524, 0.524, 0.524, ~
$ rm      <dbl> 6.575, 6.421, 7.185, 6.998, 7.147, 6.430, 6.012, 6.172, 5.631, ~
$ age     <dbl> 65.2, 78.9, 61.1, 45.8, 54.2, 58.7, 66.6, 96.1, 100.0, 85.9, 9~
$ dis     <dbl> 4.0900, 4.9671, 4.9671, 6.0622, 6.0622, 6.0622, 5.5605, 5.9505~
$ rad     <int> 1, 2, 2, 3, 3, 3, 5, 5, 5, 5, 5, 5, 5, 4, 4, 4, 4, 4, 4, 4, ~
```

```
$ tax      <dbl> 296, 242, 242, 222, 222, 222, 311, 311, 311, 311, 311, 311, 31~
$ ptratio <dbl> 15.3, 17.8, 17.8, 18.7, 18.7, 18.7, 15.2, 15.2, 15.2, 15.2, 15~
$ black   <dbl> 396.90, 396.90, 392.83, 394.63, 396.90, 394.12, 395.60, 396.90~
$ lstat   <dbl> 4.98, 9.14, 4.03, 2.94, 5.33, 5.21, 12.43, 19.15, 29.93, 17.10~
$ medv    <dbl> 24.0, 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15~
```

```
# Missing values
cat("Missing values:", sum(is.na(Boston)), "\n\n")
```

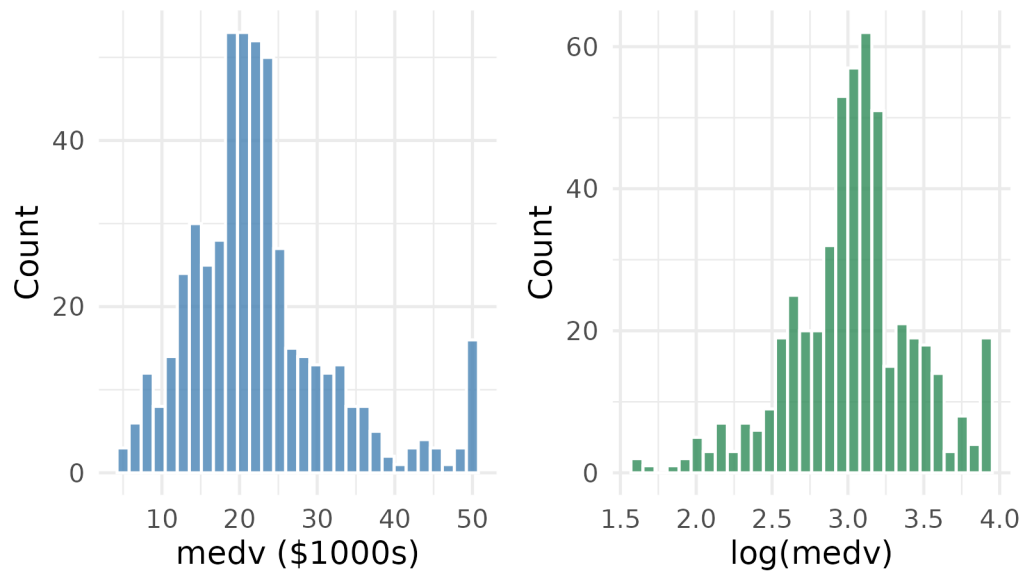
Missing values: 0

```
# Distribution of outcome
p1 <- ggplot(Boston, aes(x = medv)) +
  geom_histogram(bins = 30, fill = "steelblue",
                 color = "white", alpha = 0.8) +
  labs(title = "Outcome: Median Home Value (medv)",
       x = "medv ($1000s)", y = "Count") +
  theme_minimal(base_size = 12)

p2 <- ggplot(Boston, aes(x = log(medv))) +
  geom_histogram(bins = 30, fill = "seagreen",
                 color = "white", alpha = 0.8) +
  labs(title = "log(medv)",
       x = "log(medv)", y = "Count") +
  theme_minimal(base_size = 12)

p1 + p2
```

Outcome: Median Home Value (medv)



Lab Task 1: Exploratory Regression Analysis

```
# Correlation of each predictor with medv
cor_medv <- sort(cor(Boston)[,"medv"], decreasing = TRUE)
cor_df <- data.frame(
  Variable = names(cor_medv),
  Correlation = round(cor_medv, 4)
) |> filter(Variable != "medv")

kable(cor_df,
  caption = "Predictor Correlations with medv") |>
kable_styling(bootstrap_options = c("striped","hover"),
  full_width = FALSE) |>
column_spec(2, bold = TRUE,
  color = ifelse(abs(cor_df$Correlation) > 0.5,
    "tomato","steelblue"))
```

Table 1: Predictor Correlations with medv

	Variable	Correlation
rm	rm	0.6954
zn	zn	0.3604

black	black	0.3335
dis	dis	0.2499
chas	chas	0.1753
age	age	-0.3770
rad	rad	-0.3816
crim	crim	-0.3883
nox	nox	-0.4273
tax	tax	-0.4685
indus	indus	-0.4837
ptratio	ptratio	-0.5078
lstat	lstat	-0.7377

Lab Task 2: OLS and Diagnostics

```
# Fit OLS
ols_boston <- lm(medv ~ ., data = Boston)
cat("OLS Model Summary:\n")
```

OLS Model Summary:

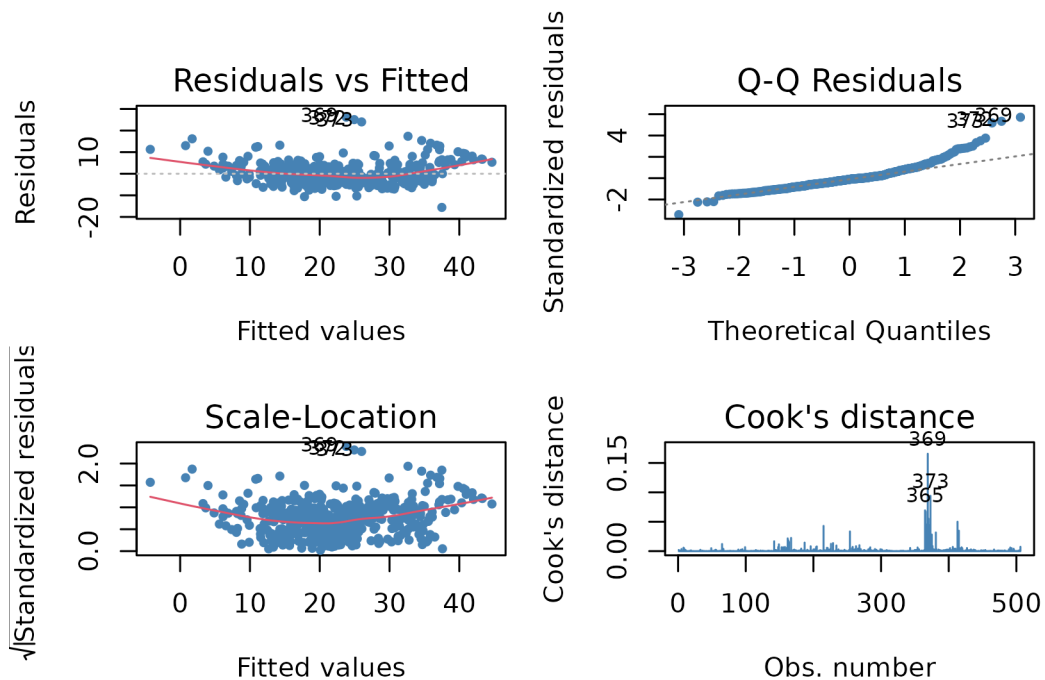
```
cat("Adj. R2:", round(summary(ols_boston)$adj.r.squared, 4), "\n")
```

Adj. R²: 0.7338

```
cat("Residual SE:", round(summary(ols_boston)$sigma, 4), "\n\n")
```

Residual SE: 4.7453

```
# Diagnostics
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
plot(ols_boston, which = 1:4,
     col = "steelblue", pch = 19, cex = 0.6)
```



```
# Formal tests
diag_results <- data.frame(
  Test      = c("Shapiro-Wilk (normality)",
                "Breusch-Pagan (homoscedasticity)",
                "RESET (linearity)"),
  p_value   = round(c(
    shapiro.test(residuals(ols_boston))$p.value,
    lmtest::bptest(ols_boston)$p.value,
    lmtest::resettest(ols_boston)$p.value
  ), 4),
  Decision  = NA
)
diag_results$Decision <- ifelse(
  diag_results$p_value < 0.05,
  "Violated ", "Satisfied ")

kable(diag_results,
      caption = "Regression Diagnostic Tests") |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
column_spec(3, bold = TRUE,
            color = ifelse(
              diag_results$Decision == "Satisfied ",
              "darkgreen", "tomato"))
```

Table 2: Regression Diagnostic Tests

Test	p_value	Decision
Shapiro-Wilk (normality)	0	Violated
Breusch-Pagan (homoscedasticity)	0	Violated
RESET (linearity)	0	Violated

Lab Task 3: Model Comparison

```
# Variable selection
step_boston <- stepAIC(ols_boston, direction = "both",
                      trace = FALSE)

# Regularization
X_b <- as.matrix(Boston[, -which(names(Boston) == "medv")])
y_b <- Boston$medv
set.seed(42)
cv_l <- cv.glmnet(X_b, y_b, alpha = 1, nfolds = 10)
cv_r <- cv.glmnet(X_b, y_b, alpha = 0, nfolds = 10)

# GAM
gam_boston <- mgcv::gam(
  medv ~ s(lstat) + s(rm) + s(dis) + s(nox) +
    chas + s(age) + s(rad, k = 5) + s(tax) + # Reduced k for rad
    s(ptratio) + s(black) + s(crim) + s(zn) + s(indus),
  data = Boston,
  method = "REML"
)

# 10-fold CV MSE for all models
set.seed(42)
folds_b <- caret::createFolds(Boston$medv, k = 10)

cv_mse_model <- function(model_obj, data_df) {
  mse_vals <- sapply(folds_b, function(idx) {
    pred <- predict(model_obj,
                    newdata = data_df[idx,])
    mean((data_df$medv[idx] - pred)^2)
  })
  mean(mse_vals)
}
```



```

# Lasso predictions
lasso_preds <- predict(cv_l, X_b,
                      s = "lambda.min")[,1]
lasso_cv_mse <- mean(sapply(folds_b, function(idx) {
  fit <- glmnet(X_b[-idx,], y_b[-idx],
               alpha = 1,
               lambda = cv_l$lambda.min)
  pred <- predict(fit, X_b[idx,])[,1]
  mean((y_b[idx] - pred)^2)
})))

final_comparison <- data.frame(
  Model      = c("OLS (full)", "Stepwise AIC",
                "Lasso", "Ridge", "GAM"),
  # NOTE: for the GAM, we use effective degrees of freedom (edf), not
  # raw coef() count. coef() on a mgcv::gam object returns every spline
  # basis coefficient, which vastly overstates model complexity since
  # penalization shrinks most of that basis flexibility away. edf is
  # the honest measure of how many "parameters worth" of flexibility
  # the fitted GAM is actually using.
  N_Params = c(length(coef(ols_boston))-1,
               length(coef(step_boston))-1,
               sum(coef(cv_l,
                       s="lambda.min") != 0)-1,
               length(coef(cv_r,
                       s="lambda.min"))-1,
               round(sum(gam_boston$edf), 1)),
  CV_MSE    = round(c(
    cv_mse_model(ols_boston, Boston),
    cv_mse_model(step_boston, Boston),
    lasso_cv_mse,
    mean(sapply(folds_b, function(idx) {
      fit <- glmnet(X_b[-idx,], y_b[-idx],
                   alpha = 0,
                   lambda = cv_r$lambda.min)
      pred <- predict(fit, X_b[idx,])[,1]
      mean((y_b[idx] - pred)^2)
    })),
    cv_mse_model(gam_boston, Boston)
  ), 3)
)

kable(final_comparison |> arrange(CV_MSE),
      caption = "Final Model Comparison: 10-Fold CV MSE",

```

```
col.names = c("Model", "# Parameters", "CV MSE")) |>
kable_styling(bootstrap_options = c("striped", "hover"),
              full_width = FALSE) |>
row_spec(1, bold = TRUE, background = "#EEF2FF")
```

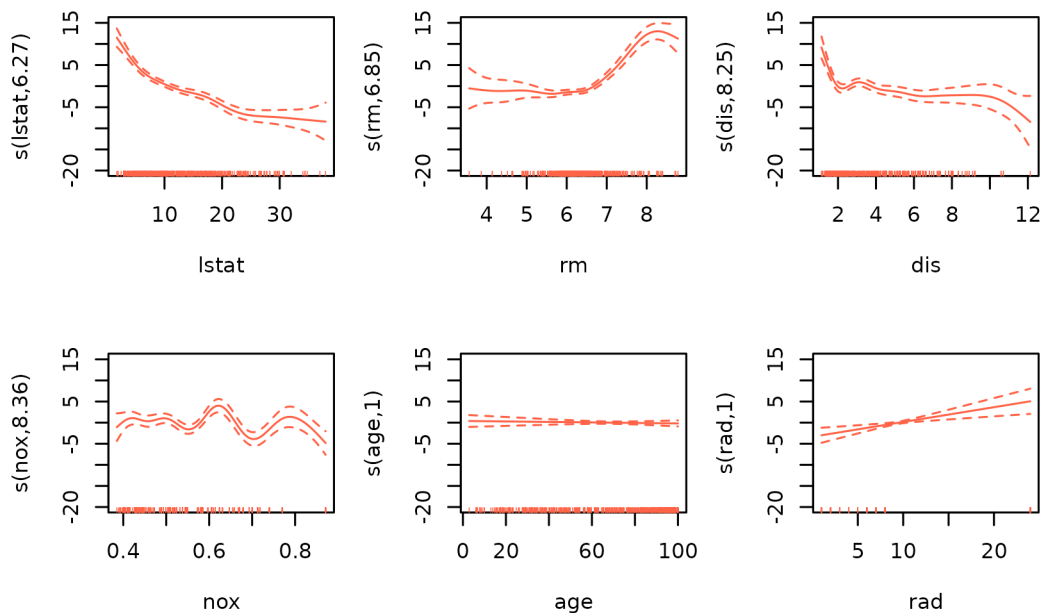
Table 3: Final Model Comparison: 10-Fold CV MSE

Model	# Parameters	CV MSE
GAM	51	8.962
OLS (full)	13	21.898
Stepwise AIC	11	21.903
Lasso	11	23.979
Ridge	13	24.287

Lab Task 4: GAM Visualization

```
# Plot GAM smooth terms for key predictors
par(mfrow = c(2, 3), mar = c(4, 4, 3, 1))

# Loop through the first 6 smooth terms
for(i in 1:6) {
  plot(gam_boston, select = i,
       shade = TRUE, shade.col = "lightblue",
       col = "tomato", seWithMean = TRUE)
}
```



Lab Discussion Questions

Answer the following in writing:

1. **Assumption Violations:** The diagnostic tests in Lab Task 2 reveal violations of homoscedasticity and linearity. What are the practical consequences for inference (confidence intervals and p-values) from the OLS model? How does the GAM address the linearity violation?
2. **Regularization vs. Selection:** Lasso performs variable selection automatically, while stepwise AIC uses sequential testing. Compare their CV MSEs. Which approach is more principled from a statistical perspective, and why?
3. **GAM Interpretation:** From the smooth term plots in Lab Task 4, describe the relationship between `lstat` (lower status population) and `medv`. Is it linear? At what values of `lstat` does the effect of additional lower-status population appear largest?
4. **Model Recommendation:** Based on the full comparison table, which model would you recommend for (a) a real estate valuation company that needs accurate predictions, and (b) a city planner who needs to understand which neighborhood characteristics drive housing prices?
5. **The Bias-Variance Perspective:** The GAM is the most flexible model but not necessarily the best by CV MSE. Explain this in terms of the bias-variance tradeoff. Under what conditions would the OLS model outperform the GAM in out-of-sample prediction?

Chapter Summary

Summary

This final chapter completed the regression toolkit from foundations to advanced methods:

- **Simple linear regression** — OLS estimation, slope interpretation, R^2 , confidence vs. prediction intervals; the Gauss-Markov theorem justifies OLS as BLUE under Linearity, Independence, and Equal variance (**not** Normality — Normality is needed only for exact small-sample t/F inference).
- **Multiple regression** — partial effects, adjusted R^2 , F-test, multicollinearity (VIF), and the importance of controlling for confounders in observational data.
- **Regression diagnostics** — the four diagnostic plots, formal tests (Shapiro-Wilk, Breusch-Pagan, RESET), Cook's distance; transformations and robust SE as remedies.
- **Variable selection** — AIC, BIC, stepwise methods, and cross-validation; stepwise has known statistical problems; CV is more principled.
- **Regularized regression** — Ridge (L2) shrinks all coefficients; Lasso (L1) performs selection; λ tuned by CV.
- **Non-linear regression** — polynomial, spline, and GAM flexibly capture curved relationships while maintaining interpretability; GAM smooth plots are the key diagnostic output.
- **GLMs for count and categorical outcomes** — Poisson regression for counts, with quasi-Poisson or Negative Binomial for overdispersion; link functions connect the linear predictor to the outcome scale.

! Key Formulas to Know

OLS Estimator:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

Adjusted R^2 :

$$\bar{R}^2 = 1 - \frac{\text{RSS}/(n - p - 1)}{\text{TSS}/(n - 1)}$$

Ridge Objective:

$$\text{RSS} + \lambda \sum_j \beta_j^2$$

Lasso Objective:

$$\text{RSS} + \lambda \sum_j |\beta_j|$$

VIF:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

Poisson Log-Link:

$$\log(\mu_i) = \beta_0 + \beta^T \mathbf{x}_i, \quad \text{IRR} = e^{\beta_j}$$

GAM:

$$Y = \beta_0 + \sum_{j=1}^p f_j(X_j) + \varepsilon$$

Textbook Epilogue

Congratulations on completing *Statistics for Data Science*. Over ten chapters, you have built a comprehensive statistical toolkit — from describing and visualizing data (Chapters 1–2), through the formal machinery of inference (Chapters 3–4), to the practical realities of data collection and preparation (Chapters 5–6), and finally to the analytical methods that drive modern data science (Chapters 7–10).

The journey from descriptive statistics to dimension reduction, clustering, classification, and regression reflects the actual workflow of a data scientist: understand your data, prepare it carefully, find structure within it, and build models that are both accurate and interpretable.

Three principles have threaded through every chapter:

1. **Assumptions matter.** Every method rests on assumptions. Violating them silently is not an option — it is a research integrity issue. Always check and report.
2. **Effect size and practical significance are as important as p-values.** Statistical significance does not equal importance. Always accompany test results with measures of effect magnitude and uncertainty.
3. **Generalization is the goal.** Training accuracy, R^2 , and within-sample fit are not the end goal. Cross-validation, out-of-sample testing, and honest uncertainty quantification are.

The field continues to evolve rapidly. The methods in this textbook are the stable, well-validated foundations — from here, you are equipped to engage with the research literature, implement new methods, and evaluate claims critically.

Good luck with your research and data science practice.

References

- Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2), 211–243.
- Breusch, T. S., & Pagan, A. R. (1979). A simple test for heteroscedasticity and random coefficient variation. *Econometrica*, 47(5), 1287–1294.

- Cook, R. D. (1977). Detection of influential observation in linear regression. *Technometrics*, 19(1), 15–18.
- Durbin, J., & Watson, G. S. (1950). Testing for serial correlation in least squares regression, I. *Biometrika*, 37(3/4), 409–428.
- Fisher, R. A. (1925). *Statistical methods for research workers*. Oliver and Boyd.
- Fox, J., & Weisberg, S. (2019). *An R companion to applied regression* (3rd ed.). SAGE.
- Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.
- Harrison, D., & Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5(1), 81–102.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer. <https://hastie.su.domains/ElemStatLearn/>
- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411.
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning with applications in R* (2nd ed.). Springer. <https://www.statlearning.com>
- McCullagh, P., & Nelder, J. A. (1989). *Generalized linear models* (2nd ed.). Chapman and Hall.
- New York State Department of Conservation. (1973). *Daily air quality measurements in New York, May–September 1973*.
- Pedersen, T. L. (2022). *patchwork: The composer of plots* (R package version 1.1.2). <https://CRAN.R-project.org/package=patchwork>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Ramsey, J. B. (1969). Tests for specification errors in classical linear least squares regression analysis. *Journal of the Royal Statistical Society: Series B*, 31(2), 350–371.
- Robinson, D., Hayes, A., & Couch, S. (2023). *broom: Convert statistical objects into tidy tibbles* (R package version 1.0.5). <https://CRAN.R-project.org/package=broom>
- Schwarz, G. (1978). Estimating the dimension of a model. *Annals of Statistics*, 6(2), 461–464.
- Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- Venables, W. N., & Ripley, B. D. (2002). *Modern applied statistics with S* (4th ed.). Springer.

- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.
- Wood, S. N. (2011). Fast stable restricted maximum likelihood and marginal likelihood estimation of semiparametric generalized linear models. *Journal of the Royal Statistical Society: Series B*, 73(1), 3–36.
- Wood, S. N. (2023). *mgcv: Mixed GAM computation vehicle* (R package version 1.9-0). <https://CRAN.R-project.org/package=mgcv>
- Xie, Y. (2015). *Dynamic documents with R and knitr* (2nd ed.). CRC Press.
- Zeileis, A., & Hothorn, T. (2002). Diagnostic checking in regression relationships. *R News*, 2(3), 7–10.
- Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2), 301–320.

End of Chapter 10 and the Textbook.